Nicolae URSU-FISCHER
Mihai URSU

Programare cu C în inginerie

Casa Cărții de Știință
Cluj-Napoca 2001

Nicolae URSU-FISCHER Mihai URSU

PROGRAMARE cu C

în

INGINERIE

Casa Cartii de Stiinta
Cluj-Napoca - 2001

Nicolae URSU-FISCHER, Mihai URSU
Programare cu C în inginerie

ISBN 973-686-227-5

© Casa Cărţii de Ştiinţă, 2001, Cluj-Napoca

Copyright © 2001 by Casa Cărții de Știință. All right reserved. Printed in Romania. No part of this publication may be reproduced or distributed in any form or by any means or stored in a database or retrieval system, without the prior permission of the publisher and authors.

Copyright © 2001 by Casa Cărții de Știință. Toate drepturile rezervate. Tipărit în România. Nici o parte din această lucrare nu poate fi reprodusă sub nici o formă, prin nici un mijloc mecanic sau electronic sau stocată într-o bază de date, fără acordul prealabil, în scris, al editurii şi autorilor.

Director: ing. Mircea TRIFU

Coperta: Mihai URSU

Tehnoredactare computerizată: autorii

Desene: autorii şi Maria Teberean

Corectura: autorii

Tiparul executat la
Casa Cărții de Știință
3400 Cluj-Napoca
B-dul Eroilor 6-8
Tel/fax: 064-431920
www.casacartii.com
email: editura@casacartii.com

On two occasions I have been asked, "Pray, Mr Babbage, if you put into machine wrong figures, will the right answers come out ?" I am not able rightly to apprehend the kind of confusion of ideas that could provoke such a question …

Charles Babbage (1791-1871)

Prefata

Scrierea unui carti privind programarea în limbajul C este fara îndoiala o sarcina dificila cu toata multitudinea documentatiei existente în acest moment. Sunt o serie de întrebari la care trebuie sa raspunda un prezumtiv autor : cui se adreseaza, cat de mare sa fie lucrarea, cum sa se ordoneze materialul, sa se prezinte integral toate instructiunile si functiile limbajului sau sa se faca o selectie, care sa fie ponderea aplicatiilor in raport cu prezentarea limbajului, etc.

Autorii si-au propus elaborarea unei lucrari care se adreseaza in special studentilor de la specializarile cu profil ingineresc dar care poate fi de asemenea utilizata de oricine doreste sa invete sau sa aprofundeze acest limbaj de programare.

Nici un manual nu poate fi conceput ca o enciclopedie, în care sa se gaseasca toate informatiile necesare, chiar si mai succint prezentate, pentru aceasta existând documentatia firmelor producatoare.

Nu s-a pus problema unei prezentari a tuturor functiilor existente ale limbajului, autorii au încercat sa le selecteze pe cele mai importante, cu o utilizare mai frecventa.

Ordinea de prezentare a materialului a urmarit intr-o mare masura modul cum se poate preda un astfel de curs.

S-au selectat si prezentat atat probleme "clasice", existente si în alte carti de programare, indiferent de limbajul prezentat, probleme proprii limbajului C cât si un numar mare de probleme care intervin în munca unui inginer.

Din experienta noastra a rezultat ca exista dificultati destul de mari începând cu întelegerea algoritmului de calcul, cu retinerea multitudinii de reguli proprii sintaxei limbajului C, a numarului extrem de mare de functii care pot fi utilizate.

Din pacate multi dintre cei care folosesc calculatorul se complac numai in postura de "specialisti" în utilizarea de soft, fiindu-le din diferite motive inaccesibila munca de creatie in acest domeniu.

Faptul ca exista multe firme specializate pe producere de soft-uri, unele extrem de inteligent elaborate, atractive si prietenoase nu înseamna ca trebuie

sa nu încercam si noi sa creem programe performante, în diferite domenii de activitate.

Multe lucruri sunt necesare pentru acest lucru: pasiune si interes în primul rand, cunostinte de matematica, imaginatie si spirit creativ, o gândire logica si poate nu în ultimul rând existenta unor probleme interesante, care sa merite sa fie rezolvate.

Un element indispensabil este si un manual în care sa se gaseasca aspectele cele mai importante, desigur ca nu vor fi suficiente, dar în urma parcurgerii acestuia se vor asimila niste cunostinte, pe baza carora se va putea merge mai departe. În acest manual se prezinta limbajul de programare C, cunoasterea acestuia fiind necesara pentru a trece la programarea în medii vizuale .

Autorii au inclus la fiecare capitol o multitudine de probleme rezolvate, care exemplifica modul de utilizare al regulilor limbajului C. De asemenea s-a introdus si un capitol continând probleme propuse, din diverse domenii: matematica, mecanica, mecanisme, vibratii, etc. Credem ca nu trebuie sa amintim ca un limbaj de programare se poate învata numai folosindu-l cât mai frecvent si rezolvând probleme din ce în ce mai complicate .

Bibliografia de la sfârsitul cartii cuprinde atât lucrari de programare în limbajul C cât si în C++, Visual C++, de algoritmi si metode numerice, grafica pe calculator, alte limbaje de programare. Recomandam consultarea acestor lucrari citate în bibliografie, scrise atât de autori romani cât si straini, a documentatiei firmelor care au scris compilatoare C (Borland, Microsoft, Watcom, s. a.), a informatiilor, inclusiv programe, existente în Help-urile disponibile în cadrul mediilor de programare precum si a unor site-uri pe Internet.

Autorii, prof. dr. ing. matem. Nicolae Ursu-Fischer si asist. dr. ing. Mihai Ursu, sunt cadre didactice la Catedra de Mecanica si Programare din Facultatea de Constructii de Masini de la Universitatea Tehnica din Cluj-Napoca.

Lucrarea se adreseaza studentilor de la sectiile de ingineri care au în planele de învatamânt cursuri de programare în limbajul C, celor de la studii aprofundate, participantilor la studii în cadrul învatamântului la distanta, cât si tuturor acelora care doresc sa dispuna de o carte, speram noi, accesibila si instructiva si sa o foloseasca cat mai mult.

Cluj-Napoca, noiembrie 2001

Nicolae Ursu-Fischer & Mihai Ursu

Cuprins

Prefaţă 3

Capitolul I Calculatoare numerice. Istoric şi perspective 9

Capitolul II Algoritmi fundamentali şi scheme logice 15

2.1 Etapele ce se parcurg în timpul rezolvării unei probleme tehnice cu calculatorul 15

2.2 Prezentarea unor algoritmi de calcul sub formă de scheme logice. 20

1. Calculul valorii unei funcţii având expresii diferite pe diverse intervale 20
2. Calculul valorilor unei funcţii într-un interval. Noţiunea de ciclu 21
3. Schimbarea între ele a valorilor a două variabile 23
4. Algoritmul lui Euclid 23
5. Rezolvarea unei ecuaţii cu metoda parcurgerii 24
6. Rezolvarea unei ecuaţii cu metoda înjumătăţirii intervalelor (a bisecţiei) 26
7. Operaţii cu şiruri. Calculul unor sume, produse, a termenilor maxim, minim şi a poziţiei acestora 30
8. Ordonarea unui şir 31
9. Inserarea unui element într-un şir ordonat 34
10. Formarea şirurilor prin recurenţă 37
11. Operaţii cu elementele unei matrice : suma, produsul elementelor ce respectă anumite condiţii impuse 41
12. Transpunerea unei matrice 42
13. Formarea unei matrice utilizând relaţii între elementele acesteia 45
14. Calculul valorilor unei funcţii utilizând dezvoltări în serie 48
15. Probleme aplicative din domeniul ingineriei mecanice 51

Capitolul III Scurt istoric al limbajelor C şi C++ 58

Capitolul IV Probleme generale privind programele scrise în C 62
4.1 Forma unui program scris în C 62
4.2 Mediul de programare Borland C++ 65
4.3 Directivele preprocesor 68
4.4 Fişiere "header" şi conţinutul acestora 70
4.5 Fazele prelucrării unui program 76

Capitolul V Unităţile lexicale ale limbajului C 77
5.1 Setul de caractere 77
5.2 Identificatori 78
5.3 Cuvinte-cheie (keywords) 79
5.4 Tipuri principale de date 80
5.5 Constante 81
5.6 Variabile simple, tablouri şi declaraţii 85
5.7 Operatori 90
5.8 Separatori 90

Capitolul VI Expresii, operanzi şi operatori. 92
6.1 Operanzi şi expresii 92
6.2 Operatori. Ordinea de prioritate a acestora 92
6.3 Operatorii aritmetici **+ - * /** şi **%** 94
6.4 Operatorii relaţionali **< <= > >=** 95
6.5 Operatorii de egalitate **= =** şi inegalitate **! =** 96
6.6 Operatorii logici **! && ¦¦** 96
6.7 Operatorii logici la nivel de bit **~ & ¦ ^ << >>** 98
6.8 Operatorul de atribuire (asignare) **=** 101
6.9 Operatori de atribuire multiplă 102
6.10 Operatorii de incrementare **++** şi decrementare **- -** 102
6.11 Operatorul de forţare a tipului 103
6.12 Operatorii paranteză. Operatori de apel de funcţie **( )** Operatori de indexare **[ ]** 103
6.13 Operatorul condiţional ternar **? :** 104
6.14 Operatorul virgulă 104
6.15 Operatorul dimensiune **sizeof** 105
6.16 Operatorii unari de adresare **&** şi de indirectare ***** 105
6.17 Operatorii punct **.** şi săgeată **->** 105

Capitolul VII Funcţii de I/O 106
7.1 Funcţii de I/O pentru caractere 106
7.2 Funcţii de I/O pentru şiruri 110
7.3 Funcţiile **scanf** şi **printf**. Specificatori de format 112
7.4 Funcţiile **cin** şi **cout** din C++ 119

Capitolul VIII Instrucţiuni. 122
8.1 Generalităţi 122
8.2 Instrucţiunea expresie, instrucţiunea vidă şi instrucţiunea compusă 122
8.3 Instrucţiuni de decizie şi de selecţie 123
8.4 Instrucţiunile **if, if-else** şi **if-else-if** 124
8.5 Instrucţiunea de selecţie **switch** 131
8.6 Instrucţiunea **break** 135
8.7 Instrucţiunea **goto** 136
8.8 Funcţia **exit** 137
8.9 Instrucţiuni de iteraţie 137
8.10 Instrucţiunile **for** şi **continue** 138
8.11 Instrucţiunea **while** 152
8.12 Instrucţiunea **do-while** 157
8.13 Aplicaţii 158

Capitolul IX Pointeri. 170
9.1 Introducere 170
9.2 Declararea variabilelor pointer 170
9.3 Operaţii cu pointeri 173
9.4 Pointeri şi tablouri unidimensionale 176
9.5 Pointeri şi tablouri multidimensionale 182
9.6 Tablouri de pointeri 186
9.7 Adresare indirectă 189

Capitolul X Funcţii 190
10.1 Probleme generale 190
10.2 Definire, apelul şi revenirea, transmiterea datelor 192
10.3 Pointeri la funcţii 219
10.4 Variabile globale şi locale 225
10.5 Recursivitate 229

Capitolul XI Biblioteca de funcţii standard 234
11.1 Funcţiile matematice standard. Fişierul math.h 234
11.2 Funcţiile pentru manipularea caracterelor, din fişierul ctype.h 238
11.3 Funcţiile pentru prelucrarea şirurilor de caractere din fişierele stdlib.h şi string.h 242
11.4 Funcţiile pentru determinarea timpului din fişierul time.h 249
11.5 Funcţiile pentru generarea numerelor aleatoare din fişierul stdlib.h 253

Capitolul XII Structuri, uniuni şi câmpuri. 257
12.1 Structuri 257
12.2 Uniuni 268
12.3 Câmpuri de biţi 269

Capitolul XIII Fişiere. 270
13.1 Probleme generale privind fişierele 270
13.2 Deschiderea şi închiderea unui fişier. Determinarea sfârşitului unui fişier 272
13.3 Operaţii de I/O în cazul fişierelor 274

Capitolul XIV Programarea în mod text. 289
14.1 Setarea modului text . Ferestre. Culori 290
14.2 Afişarea şi manipularea textelor pe ecran 292

Capitolul XV Programarea în mod grafic. 300
15.1 Setarea modului grafic. Gestiunea culorilor. Ferestre 301
15.2 Funcţii pentru desenare şi colorare 304
15.3 Funcţii pentru afişarea textelor 308
15.4 Realizarea deplasării pe ecran a imaginilor 311

Capitolul XVI Probleme de programare propuse pentru rezolvare 325
16.1 Operaţii cu şiruri 325
16.2 Operaţii cu matrice 332
16.3 Calculul unor expresii 341
16.4 Probleme de geometrie 349
16.5 Rezolvarea ecuaţiilor neliniare 354
16.6 Calculul unor funcţii utilizând dezvoltări în serii 357
16.7 Calculul integralelor definite 361
16.8 Operaţii cu polinoame 371
16.9 Sisteme de numeraţie 374
16.10 Sisteme de ecuaţii 379
16.11 Calculul derivatelor 382
16.12 Programarea în mod text şi mod grafic 383
16.13 Probleme diverse 384

XVII Anexe 390
17.1 Codul ASCII extins 390
17.2 Codurile ASCII extinse ale tastelor pentru ANSI.SYS 398
17.3 Prefixele sistemului metric 399
17.4 Puterile lui 2 399

XVIII Bibliografie 400

Capitolul I

Calculatoare numerice. Istoric şi perspective.

Lucrând la un calculator, fie ca sarcină de servici, fie ca student sau elev făcându-şi temele, fie din pasiune nu se poate să nu ne aducem aminte şi de cei câţiva oameni de ştiinţă, matematicieni, fizicieni, ingineri prin strădania cărora dispunem azi de această sculă cu performanţe absolut uimitoare, care poate soluţiona numeric o ecuaţie oricât de complicată, poate trimite un mesaj prin e-mail în câteva fracţiuni de secundă unui prieten sau rude din alt continent, poate accesa prin intermediul Internet-ului diverse pagini web ale oricărei instituţii de învăţământ superior sau de cercetare din Europa sau din altă parte, poate stabili contacte de afaceri, poate găsi oferte de locuri de muncă, burse, etc.

Dacă ar fi să începem cu **primul** calculator electronic, acesta a fost construit de **J. P**resper **Eckert şi J**ohn **W. Mauchly**, la Universitatea din Pennsylvania – Moore School of Engineering, în anul **1946**, iar denumirea cu care a intrat în istorie a fost **ENIAC** (**E**lectrical **N**umerical **I**ntegrator and **A**utomatic **C**omputer). Cu cele peste 19,000 tuburi electronice din componenţa sa acest calculator realiza o adunare în 200 μs (1 μs = 10^{-6} s), o înmulţire în 2.5 ms în cazul unor numere cu maximum 20 de cifre.

Din cele mai vechi timpuri s-au folosit diferite mijloace de calcul. Cu mii de ani în urmă s-a folosit **abacul**.

În secolele XVI-XIX au existat încercări de a construi diferite dispozitive de calcul pentru efectuarea unor operaţii elementare. Astfel matematicianul scoţian **John Napier** (1550-1617), inventatorul logaritmilor naturali, în **1614** construieşte un dispozitiv de calcul numit "vergelele" lui Napier, **Wilhelm Schickard** din Tübingen construieşte în **1623** prima maşină de calculat mecanică, fizicianul şi filosoful francez **Blaise Pascal** (1623-1662) inventează şi construieşte în **1642** un dispozitiv de calculat cu roţi dinţate, la care cifrele (zecimale) numerelor erau realizate prin poziţionarea corespunzătoare a unor roţi dinţate cu câte 10 dinţi, iar o rotaţie completă a unei roţi era urmată de 1/10 dintr-o rotaţie a roţii din stânga deci se asigura transportul unui unităţi spre o poziţie de rang superior. Cea mai performantă maşină de calculat a fost realizată de matematicianul german **Gottfried Wilhelm Leibniz** (1646-1716), în anul **1672**.

Cu toate aceste preocupări şi realizări, mecanizarea calculelor numerice nu s-a putut face decât la sfârşitul secolului trecut, când tehnologia destul de înaintată a permis fabricarea unor maşini mecanice suficient de sigure în funcţionare. Astfel, în anul 1885 **W. S. Burrougs** a produs primele maşini de adunat care s-au putut folosi efectiv în practică, câţiva ani mai târziu apărând pe piaţă şi primele maşini mecanice ce puteau efectua toate cele patru operaţii aritmetice iar după anul 1920 au apărut şi maşinile electromecanice.

Nu e lipsit de interes nici faptul că invenţia englezului **William Oughtred** (1621), **rigla de calcul**, a fost folosită până acum câteva zeci de ani, asigurând efectuarea unor calcule de

înmulţire sau împărţire cu două-trei cifre semnificative exacte. Dacă acum visul oricărui elev sau student este să aibă pe birou un calculator cât mai performant, în anii '50, '60 problema era mai simplu (şi mai ieftin) de rezolvat : se cumpăra o riglă de calcul !

Cu peste 150 de ani în urmă unele idei fundamentale, care şi în prezent stau la baza construcţiei de calculatoare au fost enunţate de matematicianul englez **Charles Babbage** (1791-1871), profesor la Universitatea din Cambridge. Acesta, în anul 1820 a proiectat o maşină de calculat mecanică pe care a numit-o **"Difference Engine"** pe care a şi realizat-o parţial iar apoi a început să proiecteze şi să construiască (şi nu a finalizat-o niciodată) un calculator mecanic "**Analytical Engine**", care avea conform proiectului trei părţi: **memorie** ("store" - depozit), realizată cu roţi dinţate cu câte 10 poziţii, pentru memorarea a 1000 de numere, fiecare cu 50 de cifre, **dispozitiv aritmetic** ("mill" – moară) , format din roţi dinţate, pârghii, diverse dispozitive mecanice, **dispozitiv de comandă**, care trebuia să funcţioneze pe baza unui program înregistrat pe cartele perforate, similare cu cele ce se utilizau la comanda războaielor de ţesut.

Trebuie amintit şi numele primei autoare a unui "program" pentru calculator, lady **Ada Augusta Byron**, contesă de Lovelace (1815-1852), colaboratoare a matematicianului Ch. Babbage, în cinstea acesteia limbajul de programare realizat la Ministerul Apărării al SUA, în anul 1979, fiind numit **ADA**.

Ca etape în dezvoltarea tehnicii de calcul ar trebui amintită şi inventarea cartelelor perforate de către **J. M. Jacquard**, la începutul secolului XIX, pentru comanda războaielor de ţesut şi utilizarea lor de către statisticianul **Hermann Holerith** pentru prelucrarea datelor recensământului efectuat în Statele Unite în anul 1880.

Introducerea datelor şi a programelor cu ajutorul cartelelor perforate s-a utilizat până prin anii '60, inclusiv la cele mai reuşite calculatoare aparţinând generaţiei a treia, **IBM 360**.

Abia în 1944 (construcţia a început în 1939) a fost realizat integral un calculator electro-mecanic (Automatic Sequence Controlled Calculator), cu numele **MARK 1**, de către **Howard Aiken**, profesor la Universitatea Harvard, cu sprijinul firmei IBM, dar deja era prea târziu, deoarece aşa cum s-a arătat, după doi ani, în 1946, apărea primul calculator electronic, ENIAC.

Funcţionarea acestui calculator electronic era îngreunată de faptul că de fiecare dată, împreună cu introducerea datelor trebuiau instalate şi instrucţiunile, prin cablare. Pentru a evita acest lucru matematicianul american **John von Neumann** (1903-1957), a introdus conceptul de **program memorat** , programul fiind introdus în calculator împreună cu datele. Primul calculator care s-a construit pe acest principiu a fost la Institute for Advanced Study – Princeton, în anul **1952** .

Primul calculator european, numit **EDSAC**, s-a construit la Universitatea din Cambridge, în anul **1949**.

Memorarea datelor se face sub formă binară, de asemenea toate operaţiile din calculator. Lucrându-se numai cu cifrele **0** şi **1**, se foloseşte algebra booleană, creată de matematicianul englez **George Boole**, (1815-1864), pentru prima dată expusă într-o lucrare publicată în anul **1854**: "An Investigation of the Law of Thought".

În domeniul calculatoarelor noţiunea de **hardware** se referă la partea de componente fizice iar cea de **software** la partea de programe .

În România au fost realizate începând din anul 1957 câteva calculatoare care din punct de vedere al construcţiei şi performanţelor aparţineau generaţiilor I şi II, la Institutul de Fizică Atomică din Bucureşti, la Institutul Politehnic din Timişoara şi la Filiala din Cluj-Napoca a Institutului de Calcul al Academiei Române .

Din **1970** s-a produs, pe bază de licenţă, un calculator de generaţia a treia, **FELIX C-256** asemănător cu calculatorul IBM 360 iar apoi şi minicalculatoare, compatibile cu cele de tip PDP-11.

În acest context trebuie să amintim numele acad. Grigore C. Moisil (1906-1973), profesor de matematică la Universitatea din Bucureşti, care a fost una din personalităţile ştiinţifice care a militat cel mai mult pentru introducerea tehnicii de calcul în toate domeniile de activitate.

Azi, în anul 2001, nu ne putem imagina cum ne-am putea descurca fără serviciile calculatorului, de la plata chitanţelor la telefon, cumpărarea biletelor de avion, tehnoredactarea pe calculator a unor lucrări şi până la proiectarea de către NASA a navetelor spaţiale. Însă cu 40-45 de ani în urmă cei mai mulţi nici nu auziseră de aşa ceva.

Cum s-a ajuns în aproximativ 50 de ani de la performanţele lui ENIAC (1946) la cele ale calculatoarelor care se produc şi folosesc în anul 2001 ?

În primul rând industia de componente electronice a avut un rol decisiv la trecerea de la o generaţie la alta de calculatoare.

Inventarea diodei, a tranzistorului, a circuitului integrat, a microprocesorului au fost cele câteva etape care au revoluţionat performanţele calculatoarelor.

Dacă **dioda** (inventată de John Ambrose Flemming în 1904) s-a folosit mult mai târziu, în schimb **tranzistorul** (inventat de Walter Trattain, William Shockley şi John Barden de la Laboratoarele Bell Telephone în 1948), **circuitul integrat** (produs prima dată de Fairchild Corp. şi Texas Instruments în 1960), **microprocesorul** (cele de prima generaţie – INTEL 4004, 8008, General Purpose Controller/Processor, Parallel Processor produse de firmele INTEL, National Semiconductor, Rockwell din 1971) au fost utilizate aproape imediat în construcţia calculatoarelor.

O tendinţă a ultimilor ani este concentrarea eforturilor din partea mai multor firme producătoare de microprocesoare asupra etapelor de proiectare şi testare, în multe cazuri acestea soldându-se cu fuzionarea firmelor sau laboratoarelor respective. În privinţa **chip**-urilor pentru calculatoare personale, se remarcă astăzi de la distanţă **AMD**, **Intel**, **SGS Thomson**, **SiS** şi **VIA Cyrix** (informaţii detaliate se pot găsi la adresele www.sandpile.org şi www.x86.org).

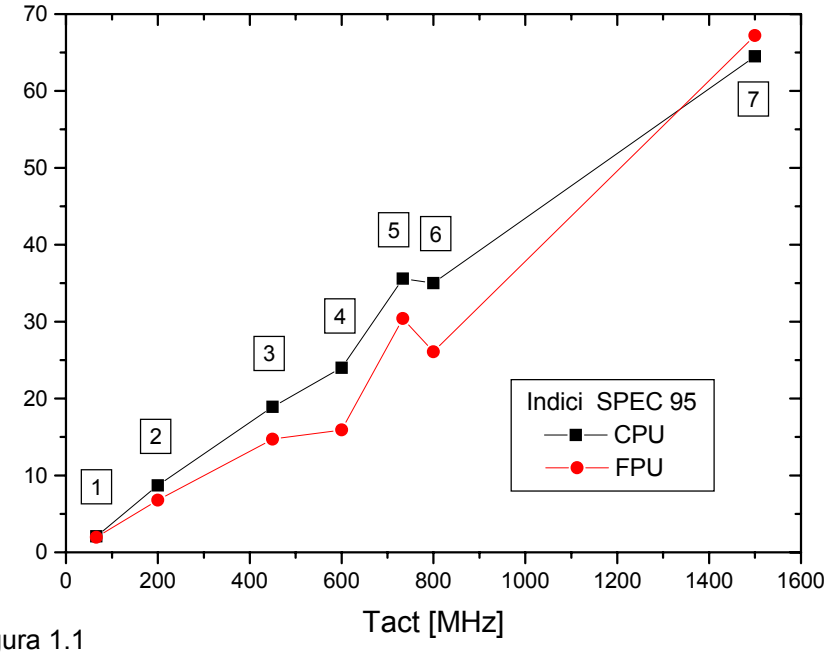


Figura 1.1

În figura 1.1 sunt daţi comparativ indicii de performanţă pentru unitatea centrală (**C**entral **P**rocessing **U**nit) şi cea în virgulă mobilă (**F**loating **P**oint **U**nit), utilizând sistemul **SPEC** (**S**tandard **P**erformance **E**valuation **C**orporation) pentru microprocesoarele mai reprezentative utilizate azi în PC-uri. Ele sunt numerotate conform tabelului 1.1, unde pot fi găsite şi câteva caracteristici ale lor.

Toate utilizează 32 biţi pentru magistrala de date, proiecte de anvergură orientate spre 64 biţi în arhitectura PC-urilor fiind demarate doar în anul 2000. Intel are cel puţin patru modele în vedere în cadrul **IA-64** (Itanium, McKinley, Madison şi Deerfield), iar AMD două în cadrul **K8-Hammer**.

Tabelul 1.1

Nr.	Familia	Chip	Data apariţiei / durata fabricaţiei	Tehnologia [μm]	Număr tranzistori [milioane]
1	Intel P5	Pentium	1993	0,8	3,1
2	Intel P6	Pentium Pro	1995-97	0,35	5,5
3		Pentium II Xeon	1998	0,25	7,5
4		Pentium III	1999	0,25	9,5
5		Pentium III Xeon	1999-2000	0,18	28
6	AMD K7	Athlon	1999-2000	0,18	22
7	Intel P7	Pentium 4	2000-	0,13	42

Utilizatorii orientaţi spre calcule ştiinţifice vor urmări de regulă sisteme performante din punct de vedere al unităţii în virgulă mobilă – marcată ***FPU***, aceasta fiind până la generaţia Intel P7 mai "înceată" decât unitatea dedicată aplicaţiilor generale, marcată ***CPU***. În general staţiile de lucru proiectate după arhitectură PC nu oferă suport pentru mai mult de 8 procesoare în paralel. Din punct de vedere al costurilor, achiziţia unui sistem de calcul raportată la necesităţile curente devine tot mai rentabilă, datorită scăderii continue a raportului preţ / performanţă.

Nu acelaşi lucru se poate spune şi despre super-calculatoare, care oferă la preţuri adecvate puteri de calcul de ordinul sutelor de ori mai mari faţă de cele ale PC-urilor. Bazate exclusiv pe procesare paralelă, beneficiind de arhitecturi mult mai extinse şi performante, sunt singurele capabile să rezolve cele mai complexe probleme tehnice. Având asigurate metode de stocare uriaşă a informaţiei (zeci de TeraBytes), dar şi de prelucrare a ei, sunt deocamdată standardul doar în cercetare şi industrie.

Datorită complexităţii şi exigenţelor în realizarea lor, numărul companiilor care le fabrică este extrem de restrâns. Cele mai cunoscute sunt : **HP**, **IBM**, **SGI** şi **Sun**. Un aspect particular al procesoarelor prezente în aceste sisteme este că ele pot fi optimizate pentru codul unui anumit limbaj de programare, cum ar fi C, Fortran, etc. Cu alte cuvinte, acelaşi program scris în diverse limbaje-sursă va fi favorizat spre timpi sensibil reduşi de rulare, pe aceeaşi maşină, dacă se compilează cu limbajul pentru care a fost "gândit" microprocesorul.

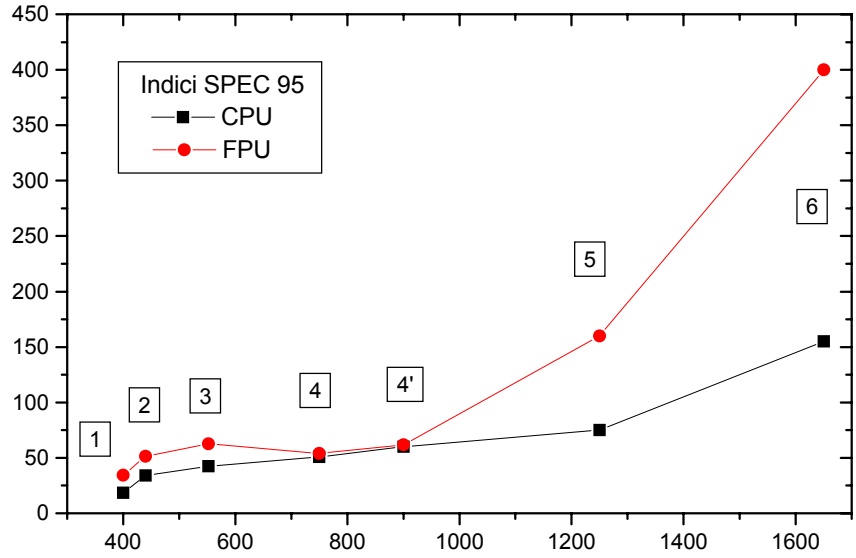


Figura 1.2

În figura 1.2 sunt prezentaţi indicii de performanţă ai unor microprocesoare folosite în serverele de supercomputing şi/sau staţiile grafice de ultimă generaţie.

Tabelul 1.2

Nr.	Firma	Chip	Data apariţiei	Tehnologia [μm]	Număr tranzistori [milioane]
1	SGI	MIPS R12000	1998	0,25	6,9
2	HP	PA-8500	1998	0,25	140
3		PA-8600	2000	0,25	140
4, 4'	Sun	UltraSPARC III	2000	0,18	29
5	Compaq	Alpha 21364	2001	0,18	100
6		Alpha 21464	2001	0,13	250

Spre deosebire de figura 1.1, unde performanţa creşte practic liniar cu frecvenţa, în super-calculatoare saltul de performanţă este dat de procesarea paralelă, respectiv de complexitatea arhitecturală. Doar de la frecvenţe de tact mai mari de 1 GHz este vizibil saltul în cazul procesoarelor Alpha. O altă observaţie faţă de figura 1.1 ar fi că indicii pentru unitatea în virgulă mobilă sunt permanent superiori unităţii centrale. Concluzia care se poate trage este că cine are la dispoziţie aşa o putere de calcul ar fi păcat doar să editeze texte sau să navigheze în Internet !

Sistemele de operare capabile să gestioneze asemenea resurse de calcul sunt toate derivate ale **UNIX**, sub diverse forme particularizate de firmele respective. Trebuie menţionat faptul că şi în familia PC-urilor există o versiune **POSIX** adaptată platformelor Intel 386-compatibile, numită **Linux**. Fiind pe zi ce trece tot mai răspândit, acest sistem de operare s-a impus acolo unde

siguranţa în funcţionare, suportul expres pentru lucrul în reţea şi multitaskingul "adevărat" sunt nişte priorităţi. În prezent sunt prezente pe piaţă câteva versiuni, cunoscute după numele redistribuitorilor: **RedHat**, **NetBSD**, **SuSE**, etc.

Întrucât electronica singură nu este în măsură să ne rezolve doleanţele, este nevoie de o anumită logică pentru gestiunea şi comanda resurselor de calcul. După cum se cunoaşte şi în cazul PC-urilor, principalele puncte cheie în vederea achiziţiei sunt ***viteza de calcul***, ***memoria***, ***capacitatea de stocare*** şi ***sistemul de operare***. Cât timp arhitectura internă nu cunoaşte mari variaţii la PC-uri, în general saltul de performanţă este echivalent cu un chip capabil să ruleze la frecvenţe mai mari pe o platformă uniprocesor. Ca sistem de operare, alegerea este a utilizatorului: **Linux** şi/sau **Windows**. În cadrul super-calculatoarelor, sistemul de operare este dinainte cunoscut, fiind unicul produs software livrat în măsură să ruleze pe respectiva arhitectură.

În tabelul următor se prezintă comparativ configuraţiile maximale ale serverelor High-End actuale livrate de cele mai cunoscute firme:

Tabelul 1.3

Sistem de calcul	Procesoare	Procesor / Tact [MHz]	Memorie [GB]	Capacitate de Stocare Internă [GB]	Sistem de Operare
Compaq AlphaServer SC	512	Alpha 21264A / 667	16	1228	Tru64 UNIX 5.0
HP 9000 V2600	32	PA-8600 / 552	32	288	HP-UX 11
IBM @server pSeries 680	24	RS 64 IV / 600	96	873	AIX 4.3.3
SGI Origin 3800	512	NUMAflex	1024	876	IRIX 6.5
Sun HPC 10000	64	UltraSPARC II / 400	64	318	Solaris 2.6

După cum se poate uşor deduce, utilizarea, întreţinerea, depanarea şi upgrade-ul unor asemenea resurse nu se face după modelul PC-urilor ! O cale de a accede la topologia şi unele principii funcţionale ar fi înţelegerea şi utilizarea intensivă a **Linux**-ului pe platforme Intel 386-compatibile, întrucât standardul POSIX face transparente anumite nivele hardware. Chiar dacă anevoie şi deloc prietenos în primele sale versiuni, **Linux** face eforturi vizibile în integrarea unor aplicaţii până nu demult tipic-**Windows**, cum ar fi Multimedia sau mediile integrate de dezvoltare vizuale (**Visual IDE** – **I**ntegrated **D**evelopment **E**nvironment). Relativ dificil de administrat, des evitat până şi de utilizatorii cu medie experienţă, accesibil la toate nivelele doar programatorilor în **C** avizaţi, **Linux** se poate considera numitorul comun pentru programator între un sistem de calcul ieftin, PC-ul, şi un sistem de calcul multiprocesor, extrem de complex, necesar problemelor tehnice critice sau extrem de pretenţioase.

Capitolul II

Algoritmi fundamentali şi scheme logice.

Experienţa a arătat că o etapă extrem de importantă în programarea unor probleme cu caracter tehnic, matematic sau de altă natură trebuie să fie consacrată studiului unor algoritmi fundamentali şi a unor modalităţi de prezentare cât mai intuitivă a acestora. Formarea unui anumit mod de gândire, propriu unui programator, nu se realizează odată cu cumpărarea unui calculator şi a unei cărţi de programare ci este rodul unei activităţi relativ mai îndelungate, proprii fiecărei persoane, mai lungă sau mai scurtă, depinzând de factori cum ar fi: gradul de inteligenţă, cunoştinţe de matematică, capacitate de memorare, etc.

2.1 Etapele ce se parcurg în timpul rezolvării unei probleme tehnice cu calculatorul.

A. Modelarea matematică. Orice problemă care urmează să fie soluţionată prin programare la un calculator trebuie să aibă un model matematic, să fie prezentată prin relaţii (formule matematice). Diversitatea acestor modele este extrem de mare, depinzând de cazul studiat. De la cel mai simplu model, care poate fi o anumită funcţie care descrie un fenomen şi care trebuie calculată într-un interval (de exemplu spaţiul, viteza şi acceleraţia pistonului unui mecanism manivelă-bielă, corespunzător unei rotaţii complete a manivelei) până la modele mai complexe care pot fi sisteme de ecuaţii diferenţiale ce descriu mişcarea vibratorie a unor sisteme mecanice, pentru soluţionarea cărora trebuie utilizate diverse metode numerice, orice se poate studia cu calculatorul, scriind programe adecvate.

Este evident faptul că parcurgerea cu succes a acestei etape este funcţie de cunoştinţele programatorului sau ale echipei în respectivul domeniu, de exemplu, abordarea studiului unor probleme de vibraţii se poate face numai după cunoaşterea unor discipline ca Mecanica teoretică, Analiza matematică, Metode numerice, Rezistenţa materialelor, Vibraţii mecanice.

Deoarece această carte se adresează în special studenţilor din primii ani, autorii au selectat ca exemple tehnice numai acele probleme care vor putea fi înţelese fără a fi necesar studiul unor discipline de specialitate, insistând în special pe probleme de matematici, cunoscute sau corespunzătoare nivelului de înţelegere al acestora.

B. Determinarea algoritmului (metodei) de calcul. Dacă o serie de probleme relativ simple, cum vor fi şi unele din cele prezentate în continuare, pot fi soluţionate având numai cunoştinţe elementare de matematică, altele vor necesita cunoaşterea unor probleme de calcul numeric (cum ar fi inversarea unor matrice patratice, rezolvarea unui sistem algebric liniar neomogen, rezolvarea numerică a unei ecuaţii diferenţiale ordinare cu metoda Runge-Kutta,

interpolare cu funcţii "spline", etc) fiind necesară consultarea unor cărţi de metode numerice în tehnică, autorii indicând la bibliografie câteva din titlurile cele mai reprezentative [Bucu73], [Carn69], [DeMa73], [Enge96], [Hopk88], [Kopc75], [Post94], [Pres90], [Rals65], [Roşc80], [Simi95], [Ursu00], [Vett90], [Vrac82], ş.a.

Este evident faptul că pentru a parcurge această etapă, programatorul, pe lângă cunoştinţe generale din domeniul matematicilor, la nivelul cursurilor predate în anii I şi II, va mai trebui să se pună la punct şi cu o serie de probleme legate de metodele numerice.

Algoritmul (metoda) de rezolvare va trebui să îndeplinească următoarele cerinţe:

- ***universalitate,*** deci să fie general valabilă pentru rezolvarea unei clase de probleme şi nu numai a unei probleme particulare. Dacă e vorba de rezolvarea unui sistem algebric liniar neomogen atunci se va utiliza un algoritm de calcul pentru un sistem de **n** ecuaţii cu **n** necunoscute, chiar dacă problema concretă ce trebuie soluţionată se referă la un sistem de 5 ecuaţii cu 5 necunoscute.

- ***claritate,*** adică trebuie să se ţină seama de absolut toate situaţiile posibile care ar putea apare în timpul desfăşurării calculelor. Calculatorul nu poate lua nici o decizie în afara celor care au fost precizate în cadrul programului. Pot apărea cazuri în care se cere efectuarea unor operaţii de împărţire, de exemplu, **a/b**, valoarea lui **b** rezultând ca urmare a unor calcule efectuate anterior. Programatorul trebuie să aibă în acest caz în vedere necesitatea ca să evite împărţiri cu zero sau cu valori numerice prea mici în valoare absolută.

- *eficienţă (finalitate)*, deci numărul de etape ce trebuie parcurse trebuie să fie finit. Enunţul unei probleme de programare de tipul: se cere să se calculeze valoarea funcţiei **sin(x)** pentru un **x** dat, utilizând dezvoltarea în serie

$$\sin x = \frac{x}{1!} - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots\ldots$$

evident că nu poate fi corect din acest punct de vedere: trebuie să se precizeze fie numărul de termeni din serie care se vor însuma, fie faptul că se vor lua în calcul termenii a căror valoare absolută este mai mare decât un număr dat ε, ştiind că această serie este absolut convergentă.

Trebuie precizat de asemenea faptul că atunci când se consideră că numărul de etape trebuie să fie finit se subînţelege ca acesta este rezonabil ca ordin de mărime.

Numele de **algoritm** se presupune ca derivă de la numele unui matematician arab din secolul al IX-lea, Abu Ja'far Mohammed Ibn Musâ **al Khowârizmi**.

În secolele trecute prin algoritm se înţelegea o regulă pe baza căreia se efectuau operaţii aritmetice.

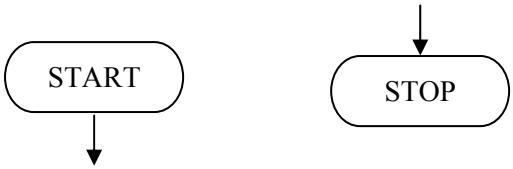
Cel mai cunoscut algoritm este cel al lui **Euclid**, pentru determinarea celui mai mare divizor comun a două numere întregi, care a apărut în "Elemente" , în secolul IV î. Ch.

Pentru o informare în detaliu asupra importantei probleme a algoritmilor de calcul se recomandă consultarea în special a lucrărilor lui **D. E. Knuth**, unele dintre acestea apărute şi în limba română, [Knut74], [Knut76].

C. Prezentarea algoritmului sub forma unei scheme logice. Algoritmul poate fi descris în diverse moduri, cea mai uzuală formă de prezentare fiind aceea sub forma de schemă logică (**flow chart** în limba engleză). Utilizând un număr foarte restrâns de simboluri geometrice, corespunzătoare etapelor ce se parcurg, aceste simboluri fiind legate prin linii orientate, orice algoritm, oricât ar fi de complex, se poate prezenta sub o formă foarte explicită. În faza de început, un programator începător va trebui să parcurgă în mod obligatoriu această etapă. Odată cu câştigarea unei experienţe în programare această etapă se va putea evita.

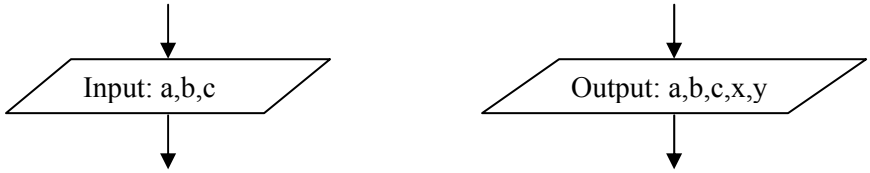
Există patru categorii de simboluri, corespunzătoare etapelor ce s-ar putea parcurge în cadrul algoritmului de calcul utilizat:

- pornirea şi **oprirea** algoritmului se precizează prin folosirea simbolurilor



dreptunghiuri cu laturile laterale rotunjite. Orice schemă logică va începe cu simbolul **START** şi se va termina cu **STOP**.

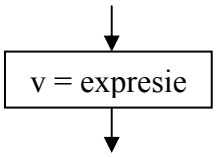
- paralelograme (sau eventual trapeze în unele lucrări) sunt folosite pentru a prezenta etapele în care trebuie **introduse şi/sau afişate date.**



Pentru a preciza sensul transferului de date , în cazul introducerii (de la tastatură) se va scrie cuvântul **Input** urmat de lista variabilelor cărora li se atribuie valorile introduse iar în cazul afişării cuvântul **Output** urmat de lista variabilelor ale căror valori se afişează.

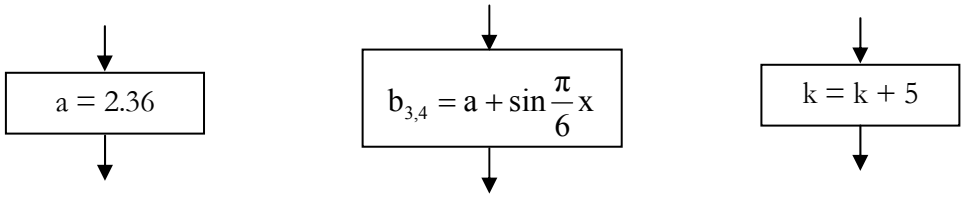
Dacă simbolul (blocul) corespunzător introducerii de date ar putea lipsi, cel de afişare este obligatoriu să existe.

- efectuarea unor **calcule** se prezintă în simbolurile de tip **dreptunghi**, de tipul



unde **v** este obligatoriu o variabilă simplă sau cu indici iar în cadrul **expresie** toate elementele care apar (funcţii, variabile, etc.) au valori cunoscute. Este unul din lucrurile cele mai importante în programare, indiferent de limbajul de programare ce va fi utilizat, să se reţină regula conform căreia funcţionează acest bloc : **se calculează valoarea expresiei** din dreapta semnului **=** , după care aceasta **se atribuie variabilei din stânga**.

Având în vedere această regulă se vor putea interpreta în felul următor cele trei exemple :



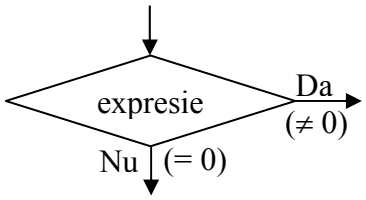
1. În primul caz valoarea constantei 2.36 se atribuie variabilei **a**. Este evident că dacă variabila **a** a avut anterior o anumită valoare, aceasta se va pierde.

2. Se presupune că cele două variabile **a** şi **x** care apar în cadrul expresiei din dreapta semnului **=** au valori cunoscute, se calculează respectiva expresie iar valoarea obţinută se atribuie unui element al unei matrice.

În aceste două cazuri se poate observa că semnul **=** are aceeaşi semnificaţie ca şi în matematică, indicând o egalitate între elementele din partea stângă şi din dreapta.

3. **k=k+5 ?** Se pare că aşa ceva nu este conform celor ştiute de la matematică. Dacă însă recitim cu atenţie regula după care funcţionează acest bloc atunci nu mai este nici o problemă. În expresia k+5 (din dreapta) variabila k are o valoare cunoscută, să presupunem 3. Conform regulii, se calculează valoarea expresiei k+5, rezultând valoarea 8, care se atribuie variabilei din stânga, deci tot lui k, această variabilă mărindu-şi deci valoarea cu 5. Acest tip de blocuri vor fi frecvent folosite în continuare, oridecâteori o anumită variabilă trebuie să-şi modifice valoarea, după o anumită lege.

- luarea unor **decizii** se prezintă prin folosirea unor **romburi** (în unele lucrări triunghiuri) în care intră o săgeată orientată (o intrare) şi ies două săgeţi (două ieşiri), în interiorul rombului (triunghiului) fiind scrisă o expresie.



Expresia poate fi de tip aritmetic, în urma evaluării acesteia rezultând o **valoare numerică**. De asemenea este posibil ca să se utilizeze o expresie logică sau relaţională, care poate avea valoare de **adevăr** sau de **fals**.

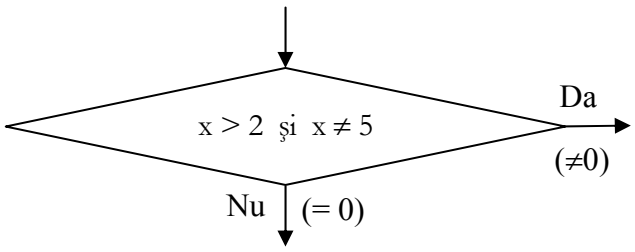
Dacă expresia este numerică şi are o valoare diferită de zero sau este logică (relaţională) şi are valoare de adevăr (căruia i se asociază valoarea **1** în cazul limbajului de programare C/C++) atunci se urmează una din căi respectiv cealaltă cale dacă expresia numerică are valoare nulă sau expresia logică (relaţională) are valoare de fals (valorii de fals i se asociază valoarea **0**).

Trebuie reamintite cele două legi ale lui **Augustus DeMorgan**, (1806-1871), [Chu_68],

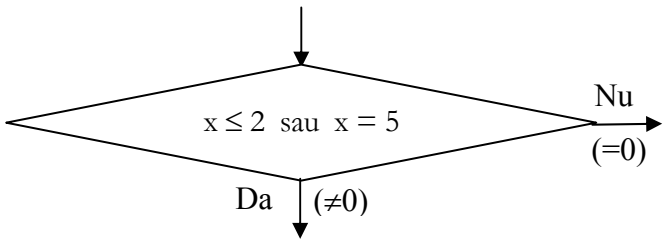
$$\overline{a \wedge b} = \overline{a} \vee \overline{b} \qquad \overline{a \vee b} = \overline{a} \wedge \overline{b}$$

unde **a** şi **b** sunt variabile (sau expresii) cu valori logice, de adevăr sau de fals.

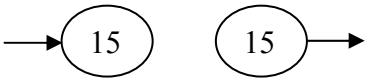
De aceste legi se va ţine seama dacă într-un bloc de decizie, cum ar fi următorul:



se doreşte inversarea celor două informaţii de pe săgeţile de ieşire. Pentru aceasta trebuie făcută inversarea logică (negaţia) a expresiei din interiorul rombului. În acest caz, utilizând prima lege a lui DeMorgan, rezultă :



Pentru a reuşi o prezentare cât mai simplă a schemelor logice, a reduce numărul de săgeţi orientate care eventual s-ar putea intersecta pe desen, îngreunând înţelegerea algoritmului, se folosesc **blocuri conectoare** , de forma



subînţelegându-se că între cele două blocuri, scrise în locuri diferite, ar exista o săgeată orientată, care însă nu se mai trasează.

O altă modalitate de a prezenta algoritmii este cea a **pseudo-codurilor** care pot fi considerate ca şi combinaţii ale unui limbaj de programare cu limbajul natural.

În acest capitol vor fi prezentate o serie de exemple de algoritmi, sub formă de scheme logice.

D. Scrierea programului sursă, într-unul din limbajele de programare folosite în prezent (C/C++, Fortran, Pascal, Basic, etc.) şi **crearea unui fişier,** utilizând un mediu de programare.

În anii '70 parcurgerea acestei etape era o adevarată problemă: scrierea programului era urmată de crearea pachetului de cartele perforate, câte o cartelă pentru fiecare rând din program, iar orice eroare se rezolva prin înlocuirea cartelei perforate care conţinea instrucţiunea sau data respectivă.

În prezent programatorul poate introduce direct programul în calculator, rezultând un fişier, cu un nume dat de autor iar extensia fiind obligatoriu **.c .cpp .for .pas .bas** funcţie de limbajul de programare utilizat.

Este evident că parcurgerea acestei etape implică studiul prealabil al unui limbaj de programare, pe baza unor manuale şi cărţi, de asemenea este foarte utilă şi consultarea informaţiilor disponibile în **HELP**-ul existent în fiecare mediu de programare.

Autorii speră că şi acest manual va constitui un ajutor preţios pentru depăşirea cu succes a acestei etape.

În scrierea programului sursă s-ar putea întâmpla să se facă unele greşeli, motiv pentru care la aceasta etapă se va reveni în mod repetat pentru a se face corecţiile necesare.

E. Compilarea programului sursă. Programul sursă se **"traduce"** obţinându-se un alt fişier, numit **fişier obiect** având acelaşi nume dar extensia **.obj** . În timpul acestei operaţii de compilare mediul de programare afişează toate **erorile** întâlnite (care trebuie corectate, intervenind în programul sursă) precum şi o serie de **avertismente**, care pot sau nu să fie luate în considerare.

O instrucţiune de următoarea formă :

$$y = (x+2)/x*x+2) ;$$

conţine una din cele mai frecvente **erori** de programare a unor expresii mai mult sau mai puţin complicate : necorespendenţa dintre numărul de paranteze deschise şi închise.

Tot la **erori** duce şi utilizarea în diferite instrucţiuni a unor funcţii care nu sunt definite sau cu nume greşit scris.

Dacă într-o unitate de program (funcţie) se declară o variabilă dar nu se foloseşte, atunci mediul de programare va semnala acest lucru ca un **avertisment**.

F. Execuţia programului. În cazul în care nu există erori de compilare se poate trece la execuţia programului . După comanda respectivă şi obţinerea unui fişier care va conţine programul executabil, fişier cu acelaşi nume dar având extensia **.exe ,** începe execuţia instrucţiunilor programului, în cadrul cărora utilizatorul trebuie să facă introducerea datelor, obţinându-se rezultatele cerute. În această fază de asemenea s-ar putea întâmpla ca să apară erori care să ducă la oprirea programului, cum ar fi : tentative de împărţire cu zero, extrageri de radicali din expresii cu valori negative, etc., erori care trebuie eliminate din programul sursă .

În scrierea programului autorul va trebui să aibă în vedere că datele vor trebui introduse în regim conversaţional, după introducerea acestora să fie afişate iar utilizatorul programului să confirme corectitudinea acestora, înainte de a trece la utilizarea acestor date în cadrul programului.

G. Utilizarea rezultatelor. Dacă în faza de învăţare acest lucru nu prezintă o importanţă deosebită să nu se uite că de fapt acesta este scopul final al muncii de programare: obţinerea unor rezultate care să poată fi folosite efectiv în proiectarea constructivă, a unor tehnologii, etc.

2.2 Prezentarea unor algoritmi de calcul sub formă de scheme logice.

1. Calculul valorii unei funcţii având expresii diferite pe diverse intervale.

Se consideră funcţia :

$$y = \begin{cases} x+5, & x \le 10, \\ \dfrac{x-18}{x^4+6}, & x > 10, \end{cases}$$

a cărei valoare trebuie calculată pentru un **x** dat. În cadrul schemei logice trebuie parcurse următoarele etape: pornire, introducerea valorii lui **x**, alegerea formulei de calcul, funcţie de îndeplinirea sau nu a condiţiei **x≤10** , calculul valorii lui **y** cu una din cele două formule, afişarea valorilor lui **x** şi **y,** oprire.

Schema logică este următoarea (fig. 2.1) :

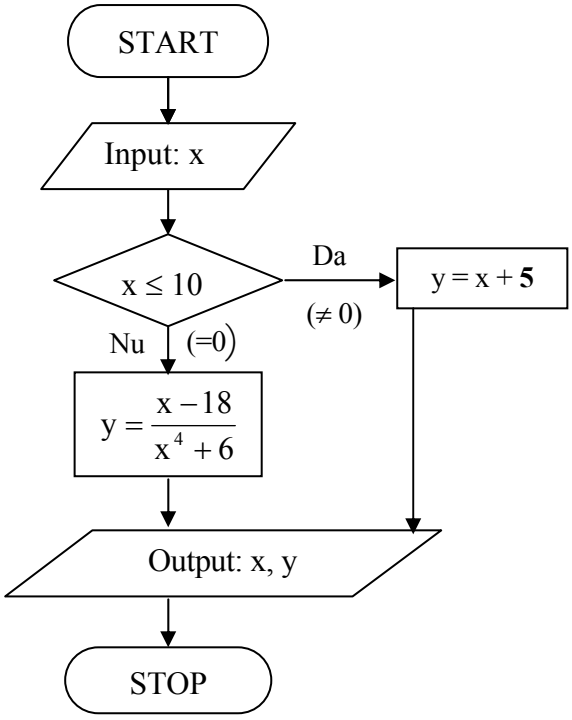


Fig. 2.1

Se observă că în acest caz au fost folosite toate cele patru tipuri de simboluri. Acest exemplu de schemă logică poate fi considerat printre cele mai simple posibile.

2. Calculul valorilor unei funcţii într-un interval. Noţiunea de ciclu.

Este evident faptul că pentru a rezolva probleme similare celei anterioare nu este necesară folosirea calculatorului. Dacă însă se cere să se calculeze valorile aceleiaşi funcţii **y = y(x)** corespunzătoare tuturor valorilor abscisei **x** în intervalul **[a, b]** , parcurs cu un pas **h**, numărul de puncte în care se face calculul fiind mare, atunci desigur că aceste operaţii se vor face pe baza unui program, al cărui schemă logică trebuie construită.

Vor trebui parcurse următoarele etape : pornire, introducerea valorilor lui **a**, **b** şi **h**, apoi parcurgerea intervalului, începând cu **x = a**, continuând cu abscisele a+h, a+2h, a+3h, ... până la valoarea **b** , pentru fiecare abscisă efectuându-se calculul valorii funcţiei şi afişarea valorilor lui **x** şi **y** , oprire.

Aici a apărut o problemă nouă : trebuie să se parcurgă succesiv şirul absciselor

a a+h a+2h a+3h a+4h ... ...

Acest lucru se poate realiza prin utilizarea unei iniţializări

x = a ;

iar trecerea de la o abscisă la cea următoare se va face prin parcurgerea repetată a blocului de calcul

x =x + h ;

ceea ce va avea ca efect incrementarea valorii lui **x** cu mărimea **h**, la fiecare parcurgere.

Deoarece valorile absciselor din şirul

a a+h a+2h a+3h a+4h ... ...

nu pot depăşi valoarea abscisei **b** , a capătului din dreapta al intervalului, va trebui să se verifice şi condiţia

x ≤ b

înainte de a se trece la calculul valorii funcţiei.

Schema logică este următoarea (fig. 2.2) :

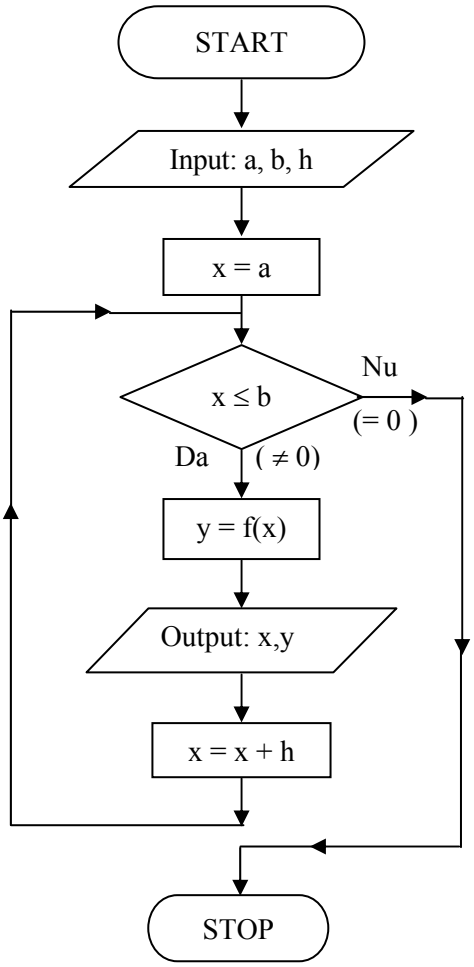


Fig. 2.2

Această schemă logică conţine un **ciclu** , care în mod obligatoriu va avea patru părţi: o **iniţializare** (blocul în care variabila care conduce ciclul primeşte o valoare iniţială), o **decizie**, un bloc în care se verifică o condiţie iar dacă aceasta este îndeplinită se execută **corpul ciclului** care este un bloc sau un grup de blocuri care se execută repetat şi blocul în care se face **modificarea valorii variabilei** care conduce ciclul, după o anumită lege. Este adevărat că de cele mai multe ori această lege este cea corespunzătoare formării termenilor unei progresii aritmetice dar acest lucru nu este obligatoriu.

Se va vedea în continuare că marea majoritate a schemelor logice şi apoi a programelor care vor urma vor conţine cicluri.

În această schemă logică s-a plasat decizia înaintea execuţiei corpului ciclului, deci acesta nu va fi parcurs decât în cazul în care condiţia testată este îndeplinită. În acest caz utilizând limbajul de programare C/C++ se vor folosi instrucţiunile de ciclare **for** şi **while** (vezi Capitolul VIII). Dacă decizia este plasată după corpul ciclului rezultă că acesta va fi parcurs cel puţin o dată, indiferent de faptul că este satisfăcută sau nu condiţia testată. Astfel de cicluri se programează cu instrucţiunea **do - while**.

3. Schimbarea între ele a valorilor a două variabile.

Se consideră două variabile, **a** şi **b** , cărora li se atribuie două valori numerice, de exemplu valoarea **1** variabilei **a** şi valoarea **5** variabilei **b**. Se cere ca să se efectueze schimbul de valori numerice între cele două variabile. Pentru a realiza acest lucru trebuie să se ţină seama că valoarea numerică a fiecărei variabile este reţinută într-o anumită zonă de memorie iar în urma atribuirii unei anumite valori unei variabile valoarea anterioară se va pierde.

Pentru a înţelege mai bine ceea ce trebuie făcut, să presupunem că avem două pahare, unul notat cu **a**, plin cu vin alb iar al doilea **b**, plin cu vin roşu. Oricine îşi dă seama că pentru a muta vinul roşu în paharul **a** şi pe cel alb în paharul **b** este necesar să mai existe un al treilea pahar **c** în care să se toarne de exemplu vinul alb din paharul **a** apoi în paharul **a** să se toarne vinul roşu din paharul **b** iar în final în paharul **b** să se toarne vinul alb care se găsea în paharul al treilea, **c**.

Rezultă deci că într-un program în care trebuie să se facă schimbul de valori între variabilele **a** şi **b** va trebui să existe un bloc cu următorul conţinut :

c = a ; a = b ; b = c ;

unde **c** este o variabilă intermediară.

Observaţie. Aceeaşi problemă s-ar putea rezolva şi fără folosirea variabilei intermediare **c**, prin efectuarea următoarelor operaţii :

(Valori iniţiale : a = 1 ; b = 5)

a = a + b ;

(a= 6 ; b = 5)

b = a – b ;

(a = 6 ; b = 1)

a = a – b ;

(Valori finale : a = 5 ; b = 1)

4. Algoritmul lui Euclid. Este unul din cei mai cunoscuţi algoritmi, poate şi prin vechimea sa de peste 2300 de ani, familiar oricărui absolvent de liceu.

Dându-se două numere întregi **a** şi **b,** pentru a determina **c**el **m**ai **m**are **d**ivizor **c**omun (**c. m. m. d. c.**) al celor două numere se fac următoarele operaţii

$$
\begin{aligned}
a &= b\ q_1 + r_1 && 1 < r_1 < b \\
b &= r_1\ q_2 + r_2 && 1 < r_2 < r_1 \\
r_1 &= r_2\ q_3 + r_3 && 1 < r_3 < r_2 \\
&\ldots\ldots\ldots\ldots && \ldots\ldots\ldots \\
r_k &= r_{k+1}\ q_{k+2} + r_{k+2} && 1 < r_{k+2} < r_{k+1} \\
r_{k+1} &= r_{k+2}\ q_{k+3} + r_{k+3} && r_{k+3} = \begin{cases} 0 & c.m.m.d.c = r_{k+2} \\ 1 & nr.\ prime \end{cases}
\end{aligned}
$$

deci se împarte **a** cu **b** obţinându-se câtul $\mathbf{q_1}$ şi restul $\mathbf{r_1}$, apoi se împarte **b** la rest şi se obţine câtul $\mathbf{q_2}$ şi restul $\mathbf{r_2}$ iar în continuare restul $\mathbf{r_1}$ se împarte la restul $\mathbf{r_2}$, obţinut anterior, rezultând câtul $\mathbf{q_3}$ şi următorul rest $\mathbf{r_3}$, etc. După un număr oarecare de paşi se va obţine fie un rest **0** (în acest caz restul obţinut anterior este **c.m.m.d.c.** al celor două numere) sau **1** (cele două **numere** sunt **prime**).

În continuare se prezintă aceste calcule, efectuate conform algoritmului lui Euclid, pentru două perechi de numere, obţinându-se cele două rezultate posibile: fie se determină **c.m.m.d.c.**, fie numerele sunt **prime**.

```
a=3333;b=2733;                a=3333;b=2732;

 3333 = 2733 ä  1 + 600        3333 = 2732 ä  1 + 601
 2733 =  600 ä  4 + 333        2732 =  601 ä  4 + 328
  600 =  333 ä  1 + 267         601 =  328 ä  1 + 273
  333 =  267 ä  1 +  66         328 =  273 ä  1 +  55
  267 =   66 ä  4 +   3         273 =   55 ä  4 +  53
   66 =    3 ä 22 +   0          55 =   53 ä  1 +   2
                                 53 =    2 ä 26 +   1

   (c.m.m.d.c. = 3)              (nr. prime)
```

În cadrul schemei logice trebuie efectuate următoarele operaţii: introducerea celor două numere **m** şi **n**, iar dacă n>m atunci trebuie schimbate între ele valorile, iniţializate valorile variabilelor a=m, b=n şi r=mod(a,b) şi efectuate în mod repetat operaţiile:

a = b ; b = r ; r = mod (a, b) ;

până când se obţine un rest nul sau egal cu 1.

Schema logică este prezentată în figura 2.3.

Observaţie. Valoarea celui mai mic multiplu comun (**c.m.m.m.c.**) a două numere **a** şi **b** se calculează în felul următor :

$$cmmmc(a,b) = \frac{a \cdot b}{cmmdc(a,b)}$$

5. Rezolvarea unei ecuaţii cu metoda parcurgerii.

Rezolvarea unor ecuaţii, determinând rădăcinile cu o precizie impusă, este una din cele mai des întâlnite probleme de programare. În cele ce urmează se va prezenta una din cele mai simple metode, cea a parcurgerii (a calculării valorilor funcţiei) utilizabilă pentru determinarea rădăcinilor reale dintr-un interval.

Se consideră ecuaţia

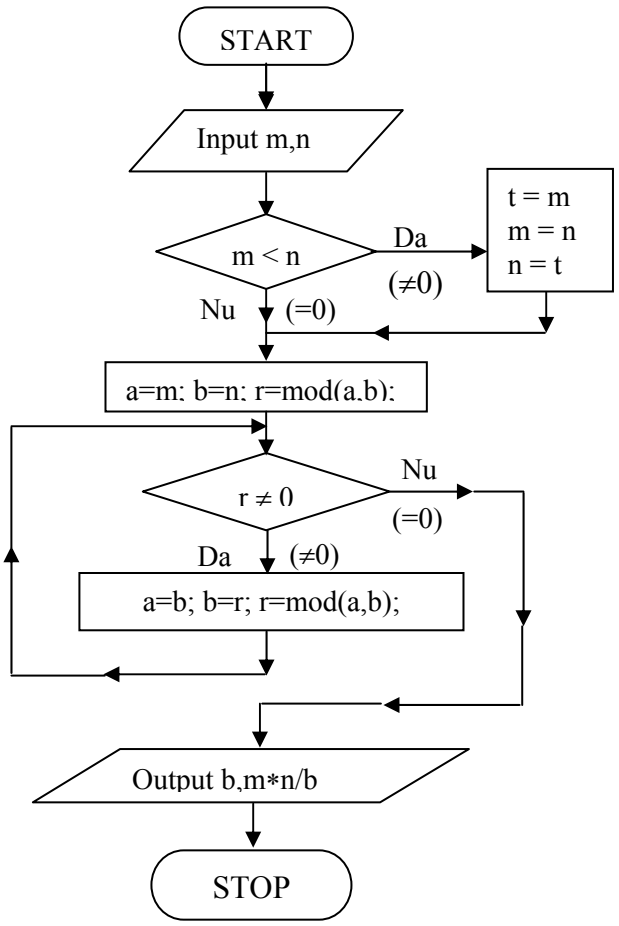


Fig. 2.3

f(x) = 0

şi se cere să se determine rădăcinile acesteia din intervalul **[a ; b]**, parcurs cu pasul **p**.

Există următoarele trei posibilităţi:

- în interval nu se găseşte nici o rădăcină,
- există o singură rădăcină,
- există mai multe rădăcini.

Identificarea poziţiei unei rădăcini se va face prin calcularea valorilor funcţiei **f(x)** pentru abscisele punctelor din intervalul **[a ; b],** parcurs cu pasul **p** , suficient de mic, (fig. 2.4).

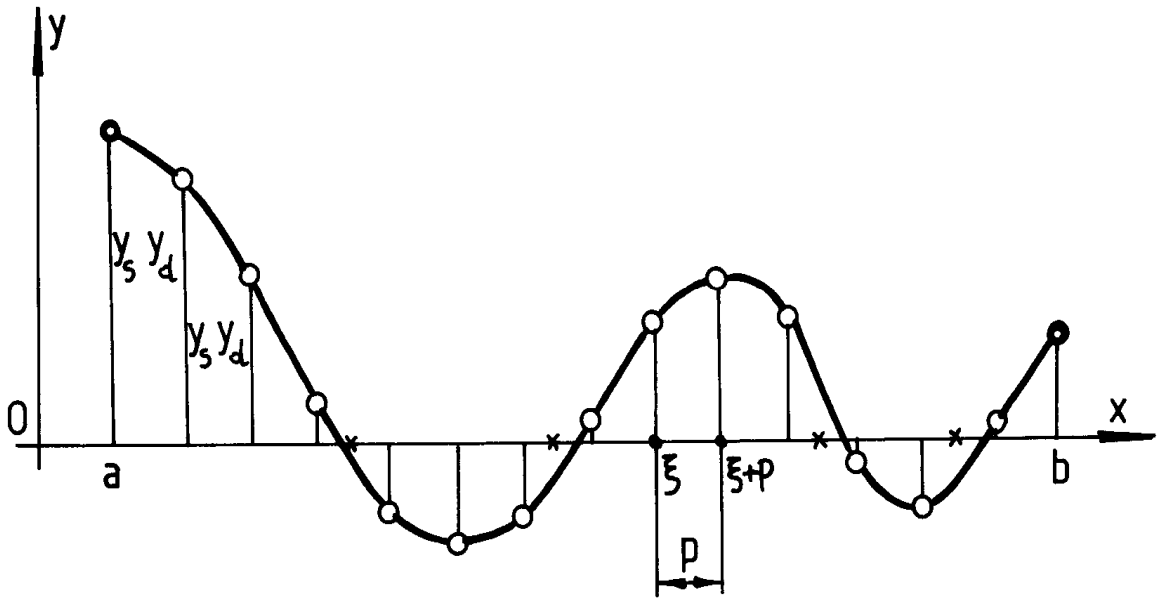


Fig. 2.4

În cazul fiecărui subinterval **[ξ ; ξ + p]** se compară cu zero produsul **f(ξ)*f(ξ+p),** dacă acesta este pozitiv atunci nu există o rădăcină în acel subinterval, dacă este negativ se va considera că rădăcina **r** se află în mijlocul subintervalului, deci **r = ξ + ½ p** iar dacă produsul este nul trebuie identificat care din cei doi factori se anulează, deci sau **r = ξ** sau **r = ξ + p .**

În urma parcurgerii complete a intervalului **[a ; b]** şi a afişării rădăcinilor determinate în fiecare subinterval, problema este rezolvată.

Trebuie remarcat faptul că singura condiţie impusă funcţiei **f(x)** este aceea de a fi definită şi continuă pe intervalul **[a ; b].**

Pe parcursul calculului se vor nota cu $\mathbf{y_s}$ respectiv $\mathbf{y_d}$ valorile funcţiei la capetele unui subinterval de mărime **p**. Se poate observa că prima valoare a lui $\mathbf{y_s}$ este **f(a)** iar la trecerea la următorul subinterval noua valoare a lui $\mathbf{y_s}$ este egală cu valoarea anterioară a lui $\mathbf{y_d}$.

Schema logică este prezentată în figura 2.5.

În [Ursu00] pg. 39-40 este prezentat programul, scris în C/C++, care rezolvă această problemă.

6. Rezolvarea unei ecuaţii cu metoda înjumătăţirii intervalelor (a bisecţiei).

Această metodă este aplicabilă pentru determinarea unei **singure rădăcini**, care se găseşte în intervalul **[a ; b]** , a ecuaţiei

$$f(x) = 0, \qquad r \in [a;b], \quad r - \textit{radacina unica}$$

singura condiţie care se impune funcţiei **f(x)** fiind cea de continuitate.

Metoda se poate înţelege foarte uşor, observând figura 2.6.

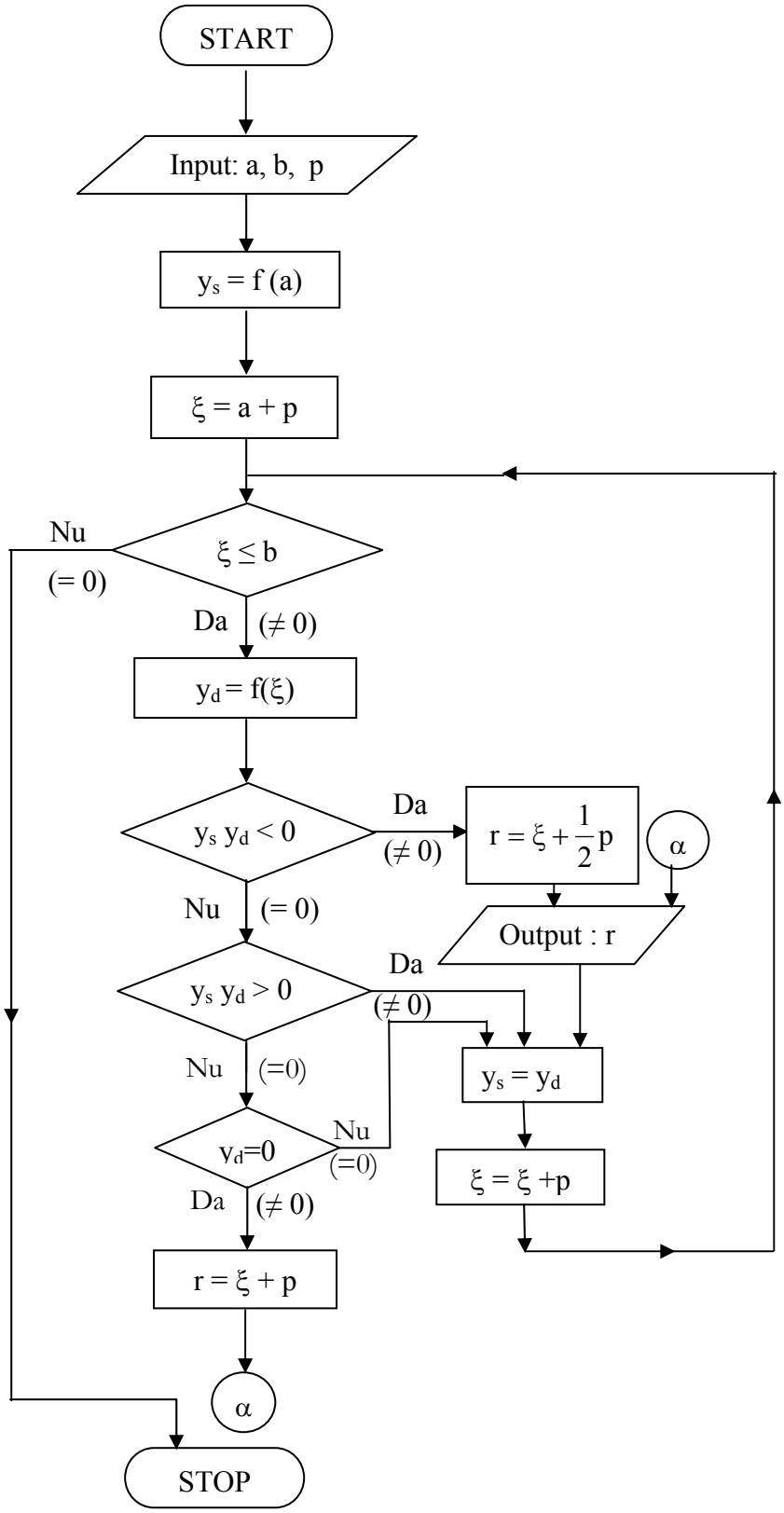

START
Input: a, b, p
$y_s = f(a)$
$\xi = a + p$
Nu
$\xi \le b$
(= 0)
Da
(≠ 0)
$y_d = f(\xi)$
$y_s\, y_d < 0$
Da
(≠ 0)
$r = \xi + \frac{1}{2}p$
α
Output : r
Nu
(= 0)
$y_s\, y_d > 0$
Da
(≠ 0)
$y_s = y_d$
Nu
(=0)
$y_d=0$
Nu
(=0)
$\xi = \xi + p$
Da
(≠ 0)
$r = \xi + p$
α
STOP

Fig.. 2.5

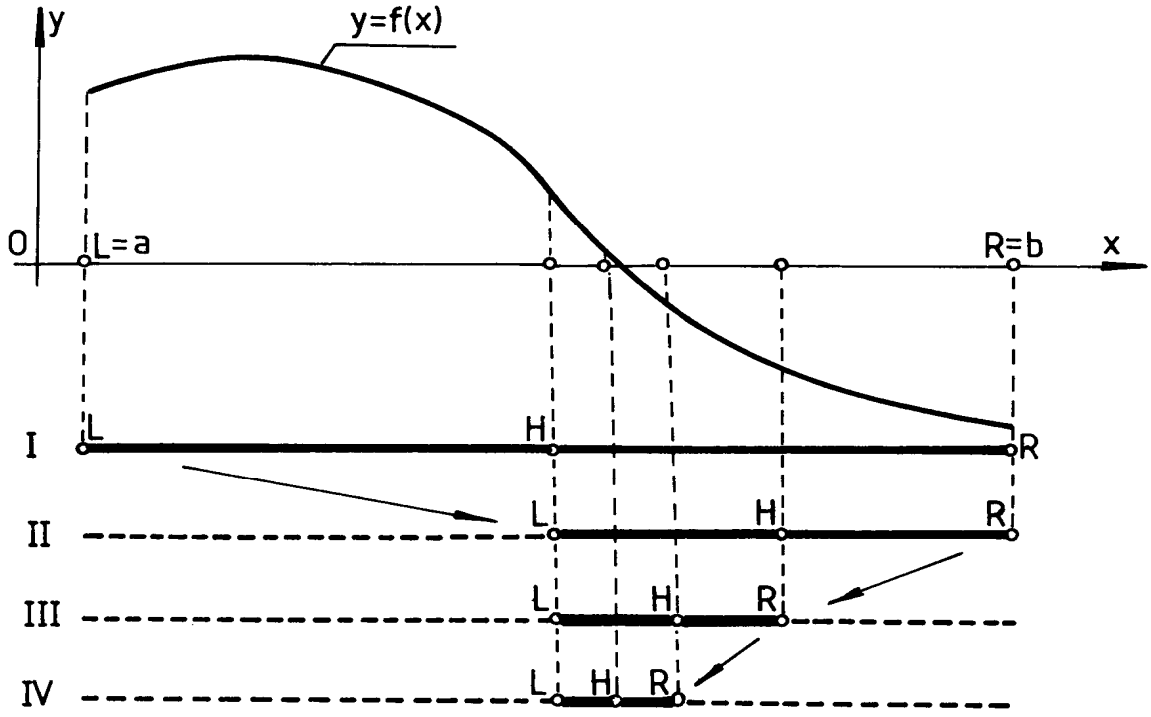


Fig. 2.6

La fiecare etapă abscisele punctelor din stânga şi din dreapta ale intervalului considerat se vor nota cu **L** respectiv **R** iar abscisa mijlocului intervalului cu **H** ,

$$H = \frac{L + R}{2}$$

valorile iniţiale ale acestora fiind **L = a** respectiv **R = b** .

Se vor nota cu $\mathbf{f_L}$ şi $\mathbf{f_H}$ valorile funcţiei **f(x)** calculate în abscisele **x=L** şi **x=H**, în etapa respectivă.

În cadrul acestei metode se determină, în fiecare etapă, unde se găseşte rădăcina **r**, în jumătatea din stânga **[L ; H)** sau în cea din dreapta **(H ; R]** a intervalului **[L ; R]** , un caz particular fiind situaţia **r=H**. Decizia respectivă se ia în funcţie de semnul produsului $\mathbf{f_L * f_H}$

$$f_L * f_H < 0, \qquad L \ neschimbat, \ R = H,$$
$$f_L * f_H > 0, \qquad L = H, \ R \ neschimbat,$$
$$f_L * f_H = 0, \qquad r = H,$$

în primele două cazuri se va trece la etapa următoare (numărul de etape **n** fiind impus) iar în cel de-al treilea caz s-a ajuns la un rezultat final **exact**.

Dacă după parcurgerea numărului impus de etape nu s-a ajuns la un rezultat exact atunci se va adopta ca valoare **aproximativă** a rădăcinii ultima valoare a abscisei mijlocului, deci $\mathbf{r \approx H}$.

Schema logică se prezintă în figura 2.7 :

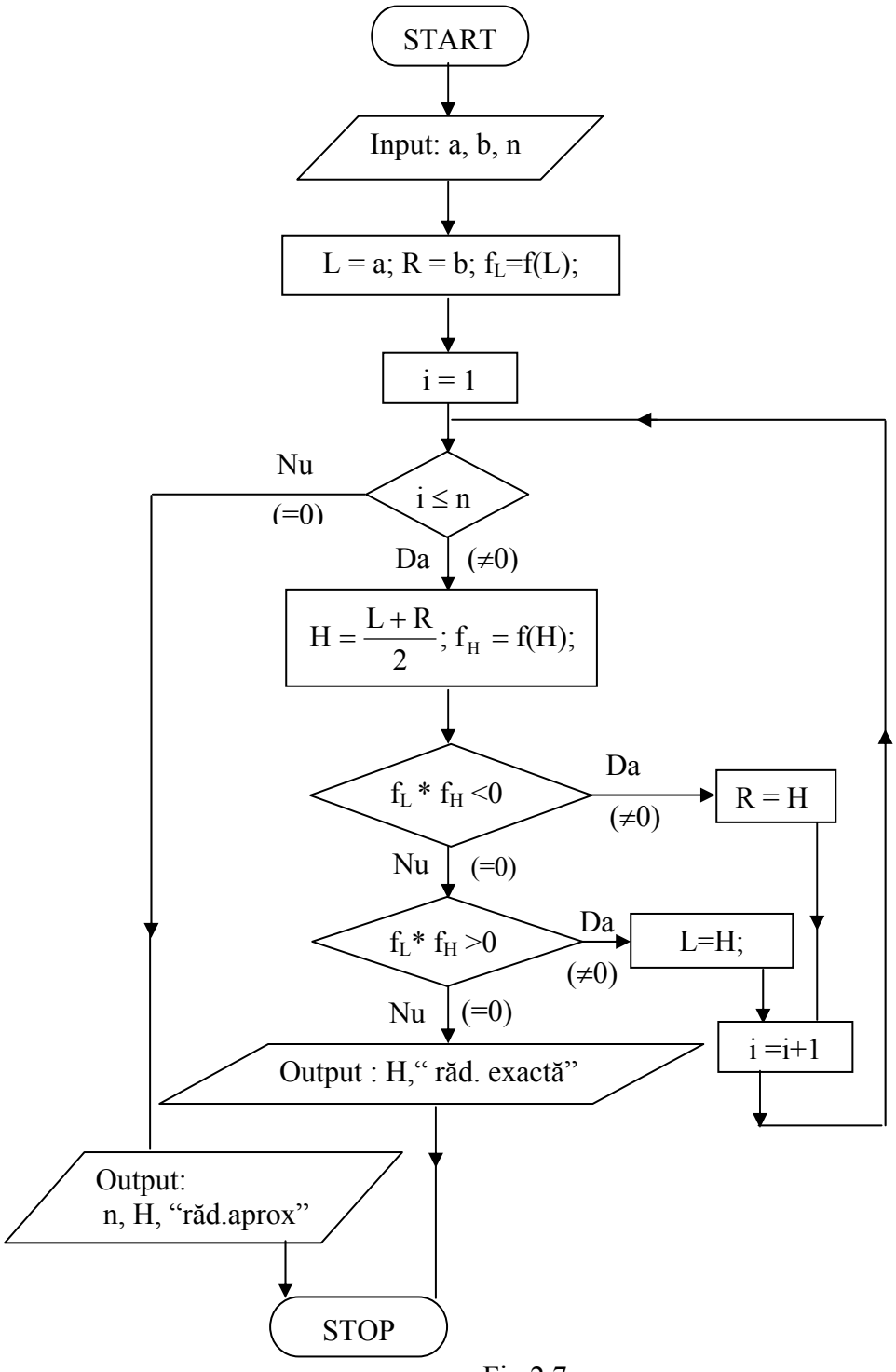


Fig.2.7

Programul scris în C/C++ se găseşte în [Ursu00] pg. 41-42.

7. Operaţii cu şiruri. Calculul unor sume, produse, a termenilor maxim, minim şi a poziţiei acestora.

Se consideră un şir format din **n** termeni

$$a_1,\ a_2,\ a_3,\ .....,\ a_i,.....,\ a_n$$

având valori reale şi se cere calculul sumei şi produsului acestor termeni precum şi determinarea valorii şi poziţiei termenului maxim.

Se vor folosi variabilele **sum** , **prod** , **vmax** , **imax** care au ca valori suma, produsul, valoarea maximă şi indicele termenului maxim, corespunzătoare celor determinate la un moment dat.

Ce valori iniţiale trebuie să primească aceste variabile ? Dacă se scrie

$$s = \underline{0} + a_1 + a_2 + ... + a_{i-1} + a_i + ... + a_n$$

$$p = \underline{1} \times a_1 \times a_2 \times ..... \times a_{i-1} \times a_i \times ..... \times a_n$$

se poate observa că iniţial **sum=0** iar **prod=1** iar pentru a realiza însumarea respectiv produsul vor trebui parcurse în mod repetat instrucţiunile

sum = sum + a_i ; **prod =prod × a_i ;**

deoarece acesta este modul în care se trece de la o anumită sumă (produs) subliniată la următoarea. Pentru a utiliza toţi termenii şirului indicele **i** va lua toate valorile între **1** şi **n**.

Pentru a determina valoarea termenului maxim şi poziţia acestuia, şirul se va parcurge începând cu primul element, acesta luându-se ca maxim, deci la început vom considera :

vmax = a_1 ; **imax = 1 ;**

iar apoi se va compara pe rând valoarea curentă a lui **vmax** cu cea a termenului şirului, dacă **vmax≥a_i** atunci **vmax** şi **imax** vor rămânea nemodificate, în caz contrar acestea vor primi noi valori:

vmax = a_i ; **imax = i ;**

în acest fel la terminarea parcurgerii şirului **vmax** şi **imax** vor conţine valoarea maximă respectiv valoarea indicelui termenului maxim.

Schema logică este prezentată în figura 2.8.

Dacă şirul conţine doi sau mai mulţi termeni cu valori egale cu cea maximă atunci utilizând această schemă logică se va determina valoarea indicelui **primului** element maxim.

Pentru a determina valoarea minimă **vmin** şi indicele **imin** al acestui termen va trebui modificată o condiţie şi două notaţii, **vmax** devine **vmin** respectiv **imax** devine **imin**.

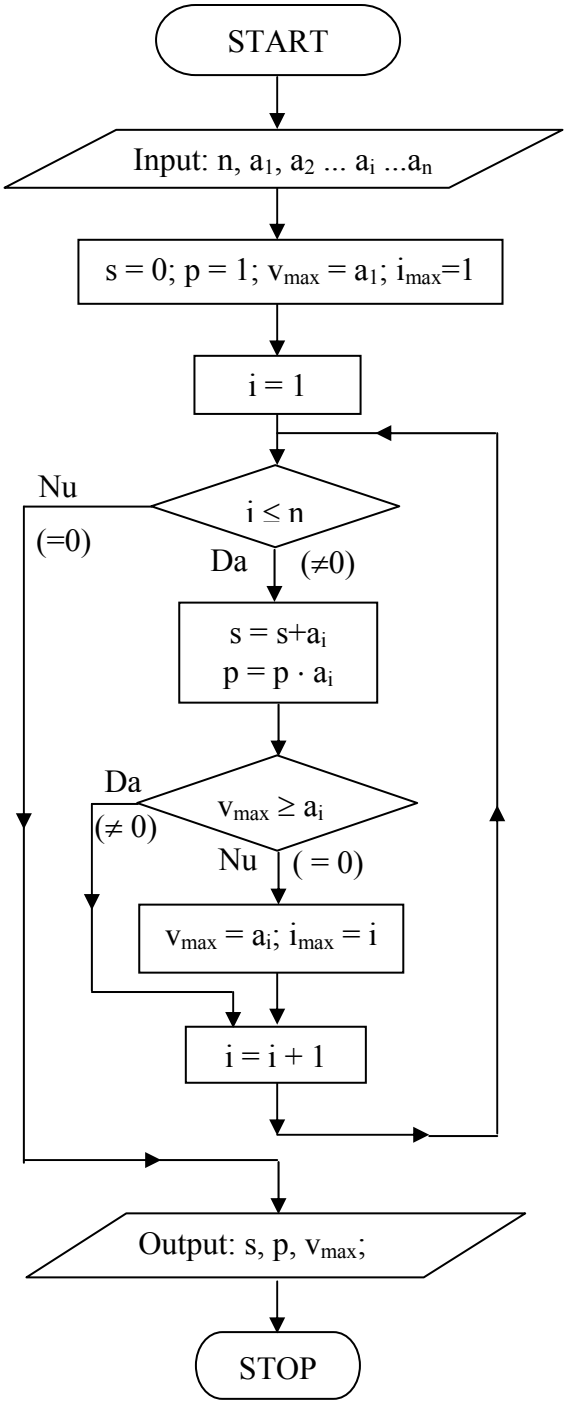


Fig. 2.8

8. Ordonarea unui şir.

Problema ordonării crescătoare (sau descrescătoare) a unui şir de numere este una dintre problemele cu caracter fundamental pe care trebuie să le cunoască orice programator. Există mai multe metode de ordonare (sortare), aplicabile în cazul unui şir cu **n** elemente,

$$a_1,\ a_2,\ a_3, \ldots\ldots\ldots, a_i, \ldots\ldots\ldots, a_n$$

dintre care se pot aminti următoarele, utilizate în cazul ordonării crescătoare, în mod asemănător procedând-se şi în cazul celei descrescătoare.

METODA I. Pentru a face ordonarea se procedează în felul următor: se parcurge şirul, care iniţial este neordonat, de la primul la ultimul element şi se determină poziţia elementului cu valoare minimă, după care se face schimbul de valori între acesta şi primul element din şir. Se parcurge apoi porţiunea neordonată din şir începând cu elementul al doilea până la ultimul, se determină poziţia elementului minim şi se schimbă valorile dintre acesta şi elementul al doilea. În continuare, la fiecare etapă se parcurge porţiunea neordonată din şir, se determină poziţia elementului minim şi se face schimbul de valori între acesta şi primul element din şirul neordonat. În acest fel, la fiecare etapă porţiunea ordonată crescător din şir se va mări cu un element . Această metodă de ordonare se numeşte **ordonare cu selecţie**.

Modul de lucru se poate vedea şi din exemplul următor,

```
 2.1   -4.0    23.9     15.9     0.2
        min                              (etapa 1)
-4.0    2.1    23.9     15.9     0.2

-4.0    2.1    23.9     15.9     0.2
                        min      (etapa 2)
-4.0    0.2    23.9     15.9     2.1

-4.0    0.2    23.9     15.9     2.1
                        min      (etapa 3)
-4.0    0.2     2.1     15.9     23.9

-4.0    0.2     2.1     15.9     23.9
                         min     (etapa 4)
-4.0    0.2     2.1     15.9     23.9
```

unde porţiunea ordonată este subliniată.

Se va nota cu **i** numărul etapei, această variabilă luând valori de la **1** la **n-1**, deoarece ultima ordonare se face cu şirul format din doi termeni, deci numărul de etape este mai mic cu o unitate decât numărul de termeni din şir. La începutul etapei numărul **i** primele **i-1** elemente sunt ordonate. În cadrul acesteia se determină poziţia elementului minim din porţiunea neordonată (elementele cu indicii de la **i** la **n**), utilizând un ciclu condus de indicele **j** care va lua valori în intervalul respectiv, elementul minim fiind $\mathbf{a_k}$. Etapa **i** se termină odată cu schimbarea între ele a valorilor elementelor $\mathbf{a_i}$ şi $\mathbf{a_k}$ astfel că la porţiunea de şir ordonat se mai adaugă un element, după cum se arată în continuare :

$$\underbrace{a_1 \quad a_2 \quad a_3 \ \ldots\ldots\ a_{i-1}}_{\textit{sir ordonat}} \ \vdots\ \underbrace{a_i \ \ldots\ a_k \ \ldots\ a_n}_{\textit{sir neordonat}}$$

$$\text{(min)} \qquad (\textit{etapa } i)$$

$$\underbrace{a_1 \quad a_2 \quad a_3 \ \ldots\ldots\ a_{i-1} \quad a_i}_{\textit{sir ordonat}} \ \vdots\ \underbrace{a_{i+1} \ \ldots\ a_k \ \ldots\ a_n}_{\textit{sir neordonat}}$$

Conform celor prezentate, schema logică va conține două cicluri, cel exterior fiind condus de indicele **i** (cu valori de la **1** la **n-1**) iar cel interior de indicele **j** (cu valori de la **i** la **n**).

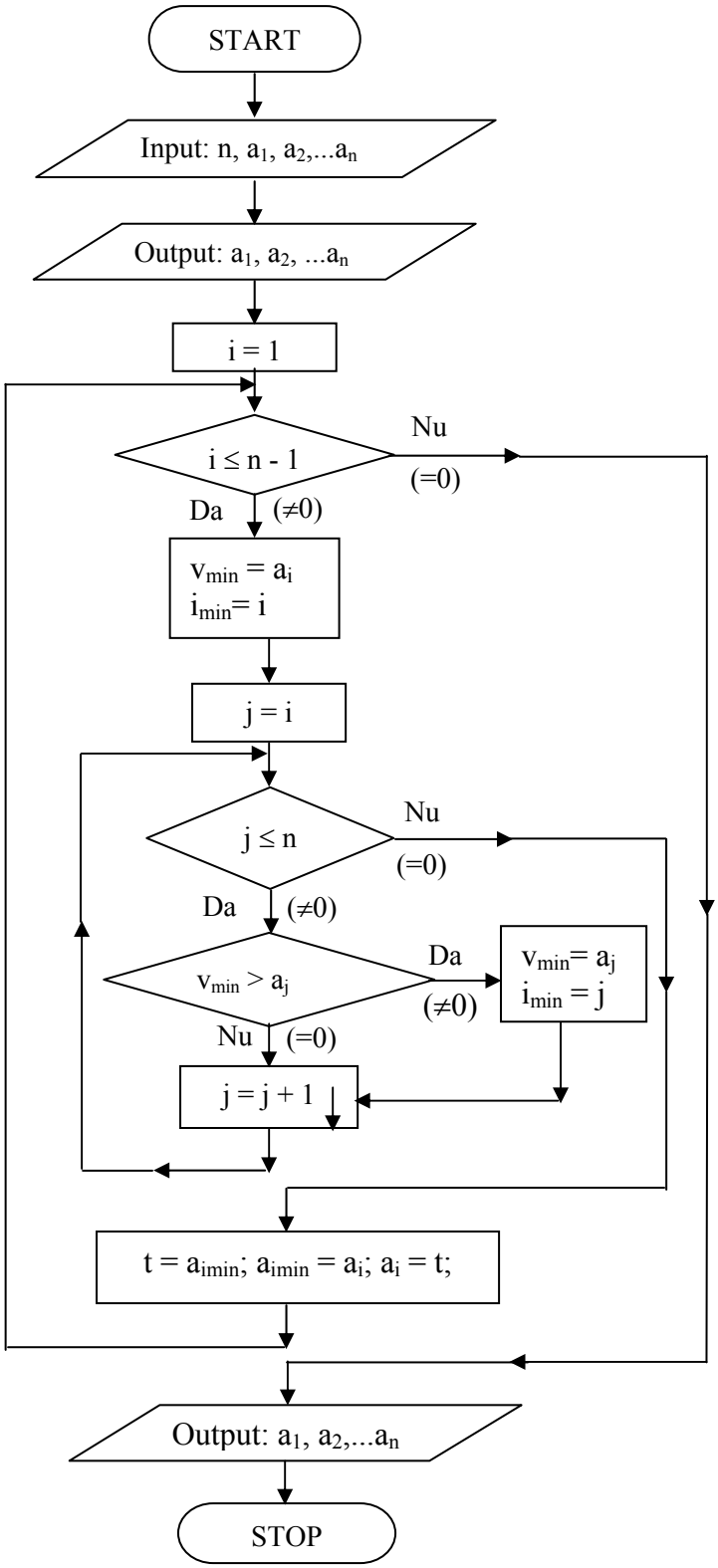


Fig. 2.9

După cum se observă, în cadrul schemei logice s-au prevăzut două afişări ale valorilor şirului, una la început (când şirul iniţial este neordonat) şi una la sfârşit, a şirului ordonat.

METODA II. O altă modalitate de a realiza ordonarea unui şir este aşa numita "**bubble sorting**" (metoda bulelor) care va fi prezentată în cadrul Exemplului nr. 18 de la Capitolul VIII. Metoda constă în parcurgerea repetată a şirului, compararea între ele a două elemente succesive şi eventualul schimb de valori între ele dacă nu sunt ordonate corespunzător. Şirul este ordonat dacă la o anumită parcurgere a sa nu mai este necesar să se facă nici un schimb de valori între elementele sale.

METODA III. **Ordonarea cu "inserare"** constă în următoarele : se parcurge şirul începând cu elementul a_2 până la elementul a_n , în cazul considerării unui element $\mathbf{a_i}$ porţiunea de şir

$$a_1 , \ a_2 , \ a_3 , \ ....., \ a_{i-1}$$

fiind ordonată, iar elementul considerat $\mathbf{a_i}$ va trebui inserat în acest şir ordonat, conform procedeului care va fi arăta în exemplul următor, nr. 9

Diferite alte metode de ordonare sunt prezentate în literatura de specialitate. Algoritmi foarte eficienţi de sortare au fost descoperiţi de Donald L. Shell în **1959** (sortarea **Shell**) şi de către C.A.R Hoare în **1962** (sortarea rapidă – **Quick sort**). Sunt folosiţi în special pentru realizarea sortării unui număr foarte mare de date.

9. Inserarea unui element într-un şir ordonat.

Se consideră un şir format din **n** termeni

$$a_1 , \ a_2 , \ a_3 , \ ....., \ a_i , ..., \ a_n$$

şi o variabilă **b**. Presupunând că şirul este ordonat, de exemplu crescător, se cere să se formeze un nou şir, de asemenea ordonat, format din **n+1** termeni, în şirul iniţial inserându-se elementul **b**, în poziţia corespunzătoare.

Sunt posibile **trei** situaţii:

- elementul **b** este mai mare (sau egal) decât ultimul element al şirului : $\mathbf{b \geq a_n}$, caz în care problema are cea mai simplă rezolvare. Valoarea elementului **b** se va atribui ultimului element al noului şir

$$\mathbf{a_{n+1} = b ;}$$

- elementul **b** este mai mic (sau egal) decât primul element al şirului : $\mathbf{b \leq a_1}$, deci primul element al noului şir trebuie să conţină valoarea elementului **b**. Pentru aceasta va trebui ca toate valorile elementelor vechiului şir să fie mutate spre dreapta cu un element

$$\begin{array}{cccccccccc} b & a_1 & a_2 & a_3 & .. & .. & a_{i-1} & a_i & .. & a_n \\ \downarrow & \downarrow & \downarrow & \downarrow & & & \downarrow & \downarrow & & \downarrow \\ a_1 & a_2 & a_3 & a_4 & .. & .. & a_i & a_{i+1} & .. & a_{n+1} \end{array}$$

Se poate observa că dacă aceste operaţii sunt efectuate în ordine de la stânga la dreapta (coloana din stânga) atunci toate elementele şirului se pierd, devenind egale cu valoarea lui **b.** Pentru a obţine în mod corect noul şir, aceste atribuiri se vor face de la dreapta la stânga (coloana din dreapta).

$a_1 = b;$ $a_{n+1} = a_n;$

$a_2 = a_1;$ $a_n = a_{n-1};$

$a_3 = a_2;$ .....

..... $a_{i+1} = a_i;$

$a_i = a_{i-1};$ $a_i = a_{i-1};$

$a_{i+1} = a_i;$ .....

..... $a_3 = a_2;$

$a_n = a_{n-1};$ $a_2 = a_1;$

$a_{n+1} = a_n;$ $a_1 = b;$

- cea de-a treia situație este atunci când valoarea elementului **b** este cuprinsă în intervalul **(a_1, a_n)**

$$a_1 < b < a_n$$

în acest caz în primul rând trebuie determinată poziția, dată prin indici, a celor două elemente între care este cuprins elementul **b,**

$$a_k \le b \le a_{k+1}$$

Pentru aceasta se va verifica condiția

$$a_i < b \wedge b \le a_{i+1}$$

pentru toate valorile indicelui **i** începând cu **1** până la **n-1**, atribuind lui **k** valoarea indicelui **i** pentru care expresia are valoare de adevăr.

Valorile primelor **k** elemente vor rămânea neschimbate, valoarea elementului **b** se va atribui lui $\mathbf{a_{k+1}}$, după ce în prealabil valorile tuturor elementelor începând cu $\mathbf{a_{k+1}}$ au fost deplasate spre dreapta, după cum se vede din cele ce urmează

$$\begin{array}{cccccccccccccc} & & & & & b & a_{k+1} & .. & .. & a_{i-1} & a_i & .. & a_{n-1} & a_n \\ a_1 & a_2 & .. & a_{k-1} & a_k & \downarrow & \downarrow & & & \downarrow & \downarrow & & \downarrow & \downarrow \\ & & & & & a_{k+1} & a_{k+2} & .. & .. & a_i & a_{i+1} & .. & a_n & a_{n+1} \end{array}$$

Schema logică este dată în figura 2.10. :

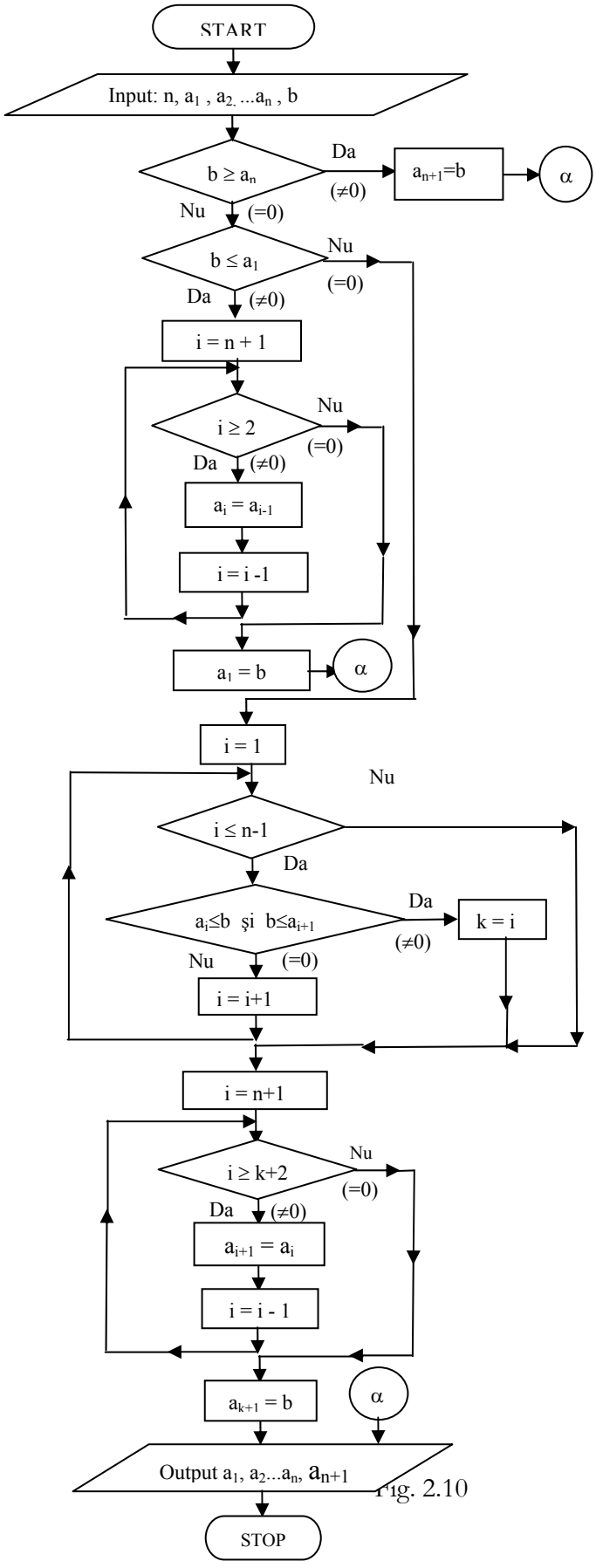

START
Input: n, a1 , a2, ...an , b
b ≥ an
Da
(≠0)
an+1=b
α
Nu
(=0)
b ≤ a1
Nu
(=0)
Da
(≠0)
i = n + 1
i ≥ 2
Nu
(=0)
Da
(≠0)
ai = ai-1
i = i -1
a1 = b
α
i = 1
Nu
i ≤ n-1
Da
ai≤b şi b≤ai+1
Da
(≠0)
k = i
Nu
(=0)
i = i+1
i = n+1
i ≥ k+2
Nu
(=0)
Da
(≠0)
ai+1 = ai
i = i - 1
ak+1 = b
α
Output a1, a2...an, an+1
STOP

Fig. 2.10

Observaţie : pentru ca inserarea să se facă mai rapid, în cel de-al treilea caz când locul elementului **b** va fi undeva în interiorul şirului, se poate iniţial face o comparare a lui **b** cu valoarea elementului care se află la mijlocul şirului pentru a "înjumătăţi" numărul de comparaţii ce urmează să fie efectuate, alegând din şir jumătatea în care va fi cuprins elementul.

10. Formarea şirurilor prin recurenţă.

Problemele anterioare, în care s-au considerat şiruri de numere (cu valori întregi sau reale) s-au rezolvat în ipoteza că valorile elementelor acestor şiruri sunt cunoscute, ele fiind introduse de utilizator de la tastatură.

Sunt multe probleme însă în care elementele şirurilor rezultă prin relaţii de recurenţă, astfel un element $\mathbf{a_i}$ se calculează fie în funcţie de valoarea elementului anterior $\mathbf{a_{i-1}}$ fie în funcţie de mai multe elemente anterioare.

Numerele lui Fibonacci . Secvenţa de numere care-i poartă numele a fost stabilită în anul 1202 (numele lui adevărat era **Leonardo Pisano** 1175 ? – 1250 ?), fiind următoarea :

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 .. ..

între trei numere succesive existând relaţia de recurenţă

$$f_0 = 0, \quad f_1 = 1, \qquad f_k = f_{k-1} + f_{k-2}, \qquad k \geq 2.$$

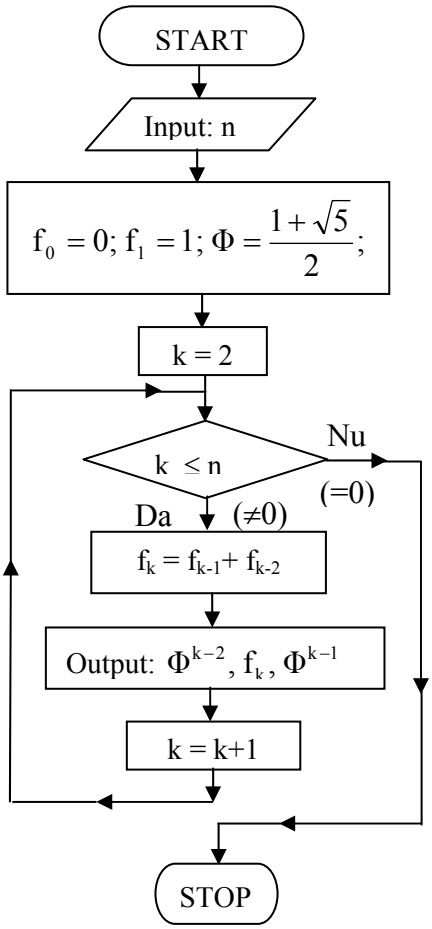


Fig. 2.11

Este cunoscută următoarea relaţie :

$$f_k = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^k - \left(\frac{1-\sqrt{5}}{2} \right)^k \right]$$

precum şi inegalitatea

$$\Phi^{k-2} \le f_k \le \Phi^{k-1} , \qquad \Phi = \frac{1+\sqrt{5}}{2} \approx 1.6180339887...$$

unde cu **Φ** s-a notat "**raportul de aur**", numit astfel deoarece se consideră că raportul **Φ/1** este cea mai plăcută proporţie estetică.

În figura 2.11 se prezintă schema logică care realizează calculul primelor 25 de numere ale lui Fibonacci, utilizând recurenţa precum şi formula de calcul prezentată iar inegalitatea se verifică începând cu k=2.

Observaţia nr. 1. Se poate arăta (Lucas – 1876) că în cazul a doi termeni oarecare $\mathbf{f_m}$ şi $\mathbf{f_n}$ ai şirului lui Fibonacci există proprietatea:

$$cmmdc(f_m, f_n) = f_{cmmdc(m,n)}$$

Observaţia nr. 2. Termenii şirului lui Fibonacci se pot exprima şi cu ajutorul unor determinanţi :

$$f_{k+2} = \begin{vmatrix} 1 & 1 & 0 & .. & 0 & 0 & 0 \\ -1 & 1 & 1 & .. & 0 & 0 & 0 \\ 0 & -1 & 1 & .. & 0 & 0 & 0 \\ .. & .. & .. & .. & .. & .. & .. \\ 0 & 0 & 0 & .. & 1 & 1 & 0 \\ 0 & 0 & 0 & .. & -1 & 1 & 1 \\ 0 & 0 & 0 & .. & 0 & -1 & 1 \end{vmatrix}_{(k \times k)} \qquad k \ge 0 .$$

Determinarea unor coeficienţi utilizaţi în cadrul formulelor lui Adams-Bashforth şi Adams-Moulton pentru rezolvarea numerică a ecuaţiilor diferenţiale. În cadrul cursurilor de **Metode numerice** unde sunt prezentate aceste metode se utilizează două şiruri

$$a_0 = 1, \quad a_1, \quad a_2, \quad a_3, \ldots, a_j, \ldots, a_{n-1}$$
$$b_0 = 1, \quad b_1, \quad b_2, \quad b_3, \ldots, b_j, \ldots, b_{n-1}$$

termenii cărora rezultă din următoarele relaţii de recurenţă [Henr62] pg. 193 şi 194,

$$\sum_{j=0}^{m} \frac{a_j}{m-j+1} = 1, \qquad \sum_{j=0}^{m} \frac{b_j}{m-j+1} = 0, \qquad m = \overline{1,\ n-1}.$$

Se pune problema de a determina coeficienţii acestor două şiruri, cunoscând valoarea primului element.

Dezvoltat, cele două relaţii de recurenţă se scriu sub forma:

$$\frac{1}{m+1}a_0 + \frac{1}{m}a_1 + \frac{1}{m-1}a_2 + \ldots + \frac{1}{m-j+1}a_j + \ldots + \frac{1}{3}a_{m-2} + \frac{1}{2}a_{m-1} + a_m = 1,$$

$$\frac{1}{m+1}b_0 + \frac{1}{m}b_1 + \frac{1}{m-1}b_2 + \ldots + \frac{1}{m-j+1}b_j + \ldots + \frac{1}{3}b_{m-2} + \frac{1}{2}b_{m-1} + b_m = 0,$$

Referindu-ne la elementele primului şir, se poate observa că vor trebui folosite succesiv relaţiile :

$m=1;\quad \frac{1}{2}a_0 + a_1 = 1;\quad a_1 = 1 - \frac{1}{2}a_0;$

$m=2;\quad \frac{1}{3}a_0 + \frac{1}{2}a_1 + a_2 = 1;\quad a_2 = 1 - \frac{1}{3}a_0 - \frac{1}{2}a_1;$

. . . . . .

$$m;\quad \frac{1}{m+1}a_0 + \frac{1}{m}a_1 + \frac{1}{m-1}a_2 + \ldots + \frac{1}{m-j+1}a_j + \ldots + \frac{1}{3}a_{m-2} + \frac{1}{2}a_{m-1} + a_m = 1,$$

$$a_m = 1 - \sum_{j=0}^{m-1} \frac{a_j}{m-j+1};$$

. . . . . .

deci elementul $\mathbf{a_1}$ depinde numai de $\mathbf{a_0}$, elementul $\mathbf{a_2}$ depinde de $\mathbf{a_0}$ şi $\mathbf{a_1}$, în sfârşit, un element curent $\mathbf{a_m}$ depinde de toate elementele anterior determinate.

Rezultă deci că în schema logică va trebui parcurs un ciclu, condus de indicele **m**, cu valori între **1** şi **n-1**, necesar pentru a trece de la un element la altul al şirului,

$$a_1,\ a_2,\ a_3, \ldots,\ a_m, \ldots,\ a_{n-1}$$

iar pentru calculul sumei care intră în expresia elementului $\mathbf{a_m}$, utilizând formula de însumare, va fi necesară parcurgerea unui ciclu interior, condus de indicele **j**, cu valori între **0** şi **m - 1**, deci schema logică va conţine două cicluri suprapuse.

În mod similar se determină şi valorile elementelor celui de-al doilea şir.

Schema logică este dată în figura 2.12.

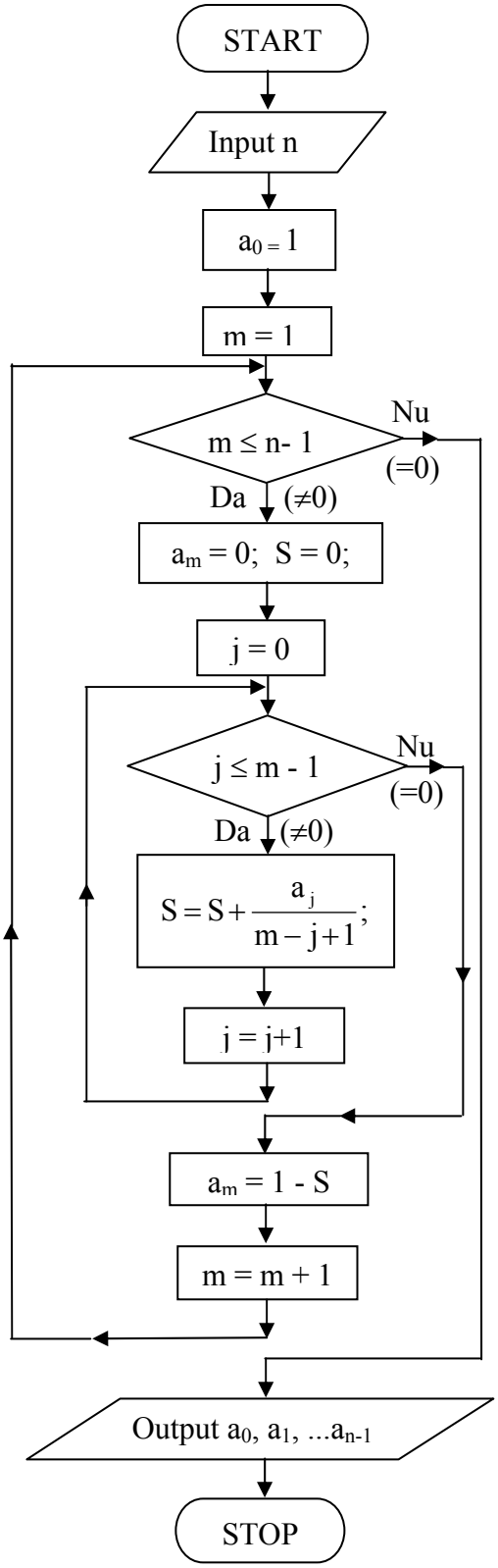

START
Input n
$a_0 = 1$
m = 1
$m \le n-1$
Nu
(=0)
Da
(≠0)
$a_m = 0;\ S = 0;$
j = 0
$j \le m-1$
Nu
(=0)
Da
(≠0)
$S = S + \frac{a_j}{m-j+1};$
j = j+1
$a_m = 1 - S$
m = m + 1
Output $a_0, a_1, \ldots a_{n-1}$
STOP

Fig. 2.12

11. Operaţii cu elementele unei matrice: suma şi produsul elementelor ce respectă anumite condiţii impuse.

Se consideră o matrice patratică cu **n** linii şi coloane. Se cere să se determine suma elementelor pozitive de sub diagonala principală şi produsul elementelor negative de deasupra diagonalei secundare.

Scriind de două ori aceeaşi matrice **B**, în primul caz punându-se în evidenţă diagonala principală iar în al doilea caz diagonala secundară, se poate observa că poziţia unui element $\mathbf{b_{i,j}}$ depinde de valorile indicilor acestora.

$$\begin{bmatrix}
b_{1,1} & b_{1,2} & .. & b_{1,j} & .. & b_{1,i} & .. & b_{1,k} & .. & b_{1,n} \\
b_{2,1} & b_{2,2} & .. & b_{2,j} & .. & b_{2,i} & .. & b_{2,k} & .. & b_{2,n} \\
.. & .. & \ddots & .. & .. & .. & .. & .. & .. & .. \\
.. & .. & .. & \ddots & .. & .. & .. & .. & .. & .. \\
.. & .. & .. & .. & \ddots & .. & .. & .. & .. & .. \\
b_{i,1} & b_{i,2} & .. & b_{i,j} & .. & b_{i,i} & .. & b_{i,k} & .. & b_{i,n} \\
.. & .. & .. & (i>j) & .. & .. & \ddots & (i<k) & .. & .. \\
.. & .. & .. & .. & .. & .. & .. & \ddots & .. & .. \\
.. & .. & .. & .. & .. & .. & .. & .. & \ddots & .. \\
b_{n,1} & b_{n,2} & .. & b_{n,j} & .. & b_{n,i} & .. & b_{n,k} & .. & b_{n,n}
\end{bmatrix}$$

$$\begin{bmatrix}
b_{1,1} & .. & b_{1,j} & .. & b_{1,n-i+1} & .. & b_{1,k} & .. & b_{1,n-1} & b_{1,n} \\
b_{2,1} & .. & b_{2,j} & .. & b_{2,n-i+1} & .. & b_{2,k} & .. & b_{2,n-1} & b_{2,n} \\
.. & .. & .. & .. & .. & .. & .. & \iddots & .. & .. \\
.. & .. & .. & .. & .. & .. & \iddots & .. & .. & .. \\
.. & .. & .. & .. & .. & \iddots & .. & .. & .. & .. \\
b_{i,1} & .. & b_{i,j} & .. & b_{i,n-i+1} & .. & b_{i,k} & .. & b_{i,n-1} & b_{i,n} \\
.. & .. & (i+j<n+1) & \iddots & .. & .. & (i+k>n+1) & .. & .. & .. \\
.. & .. & \iddots & .. & .. & .. & .. & .. & .. & .. \\
.. & \iddots & .. & .. & .. & .. & .. & .. & .. & .. \\
b_{n,1} & .. & b_{n,j} & .. & b_{n,n-i+1} & .. & b_{n,k} & .. & b_{n,n-1} & b_{n,n}
\end{bmatrix}$$

Astfel, dacă

a. **i < j** - $\mathbf{b_{i,j}}$ este **deasupra diagonalei principale**,
b. **i = j** - $\mathbf{b_{i,j}}$ este **pe diagonala principală,**
c. **i > j** - $\mathbf{b_{i,j}}$ este **sub diagonala principală,**
d. **i+j < n+1** - $\mathbf{b_{i,j}}$ este **deasupra diagonalei secundare**,
e. **i+j = n+1** - $\mathbf{b_{i,j}}$ este **pe diagonala secundară**,
f. **i+j > n+1** - $\mathbf{b_{i,j}}$ este **sub diagonala secundară.**

În cazul problemei propuse va trebui să iniţializăm suma şi produsul (**sum=0**; **prod=1**;) şi să parcurgem întreaga matrice într-o anumită ordine, de exemplu **linie după linie**. Indicele de linie va fi **i** iar cel de coloană **j**, ambii luând valori de la **1** la **n**, schema logică având două cicluri suprapuse.

Deoarece s-a presupus că parcurgerea matricei se face linie după linie, ciclul exterior va fi condus de indicele de linie **i** , iar pentru fiecare valoare a indicelui **i** va trebui parcursă complet o linie, deci indicele de coloană **j** va lua valori de la **1** la **n**, în cadrul unui ciclu interior.

Pentru fiecare element considerat, dacă este îndeplinită condiția

$$b_{i,j} > 0 \ \wedge i > j$$

atunci acesta se va însuma (**sum=sum + $b_{i,j}$** ;), fiind pozitiv şi găsindu-se sub diagonala principală. Dacă este verificată următoarea condiţie

$$b_{i,j} < 0 \ \wedge i + j < n + 1$$

(elementul este negativ şi se găseşte deasupra diagonalei secundare), atunci elementul va participa la formarea produsului (**prod=prod × $b_{i,j}$** ;). La terminarea parcurgerii celor două cicluri suma şi produsul cerut sunt calculate.

Schema logică se găseşte în figura 2.13.

În cazul problemei propuse a fost nevoie să se parcurgă întreaga matrice.

Dacă s-ar fi cerut să se considere numai elementele pozitive de sub diagonala principală, atunci ar fi trebuit verificată repetat numai condiția **$b_{i,j}$ > 0**, iar indicii care conduceau cele două cicluri ar fi trebuit să ia următoarele valori:

- în cazul ciclului exterior **i** va lua valori de la **2** la **n**,
- în cazul ciclului interior **j** va lua valori de la **1** la **i-1.**

În mod similar, dacă s-ar fi cerut să se considere numai elementele negative de deasupra diagonalei secundare, atunci ar fi trebuit verificată repetat condiția **$b_{i,j}$ < 0**, iar indicii care conduceau cele două cicluri ar fi trebuit să ia următoarele valori:

- în cazul ciclului exterior **i** va lua valori de la **1** la **n-1**,
- în cazul ciclului interior **j** va lua valori de la **1** la **n-i.**

12. Transpunerea unei matrice.

Se consideră o matrice patratică **B**, cu **n** linii şi coloane. Se cere să se determine transpusa acestei matrice, $\mathbf{B}^t$, în cadrul schemei logice utilizând aceeaşi notaţie **B** şi pentru matricea transpusă.

Pentru aceasta vor trebui ca să se schimbe între ele valorile elementelor $\mathbf{b_{i,j}}$ şi $\mathbf{b_{j,i}}$ care ocupă poziții simetrice față de diagonala principală, adică a elementelor care se află sub diagonala principală cu cele care se află deasupra acesteia. Deoarece elementele $\mathbf{b_{i,j}}$ se găsesc sub diagonala principală rezultă că pentru a parcurge porțiunea triunghiulară indicele de linie **i** (care conduce ciclul exterior) va lua valori de la **2** la **n** iar indicele **j** care determină poziția elementului curent pe linia **i** va lua valori de la **1** la **i-1**, după cum se vede în continuare

$$\begin{array}{l} \\ (i=2) \\ \\ \\ \\ (i) \\ \\ (i=n) \end{array} \begin{bmatrix} .. & .. & .. & .. & .. & .. & .. & .. \\ b_{2,1} & .. & .. & .. & .. & .. & .. & .. \\ b_{3,1} & b_{3,2} & .. & .. & .. & .. & .. & .. \\ .. & .. & .. & .. & .. & .. & .. & .. \\ .. & .. & .. & .. & .. & .. & .. & .. \\ b_{i,1} & .. & b_{i,j} & .. & b_{i,i-1} & .. & .. & .. \\ .. & .. & .. & .. & .. & .. & .. & .. \\ b_{n,1} & .. & b_{n,j} & .. & .. & .. & b_{n,n-1} & .. \end{bmatrix}$$

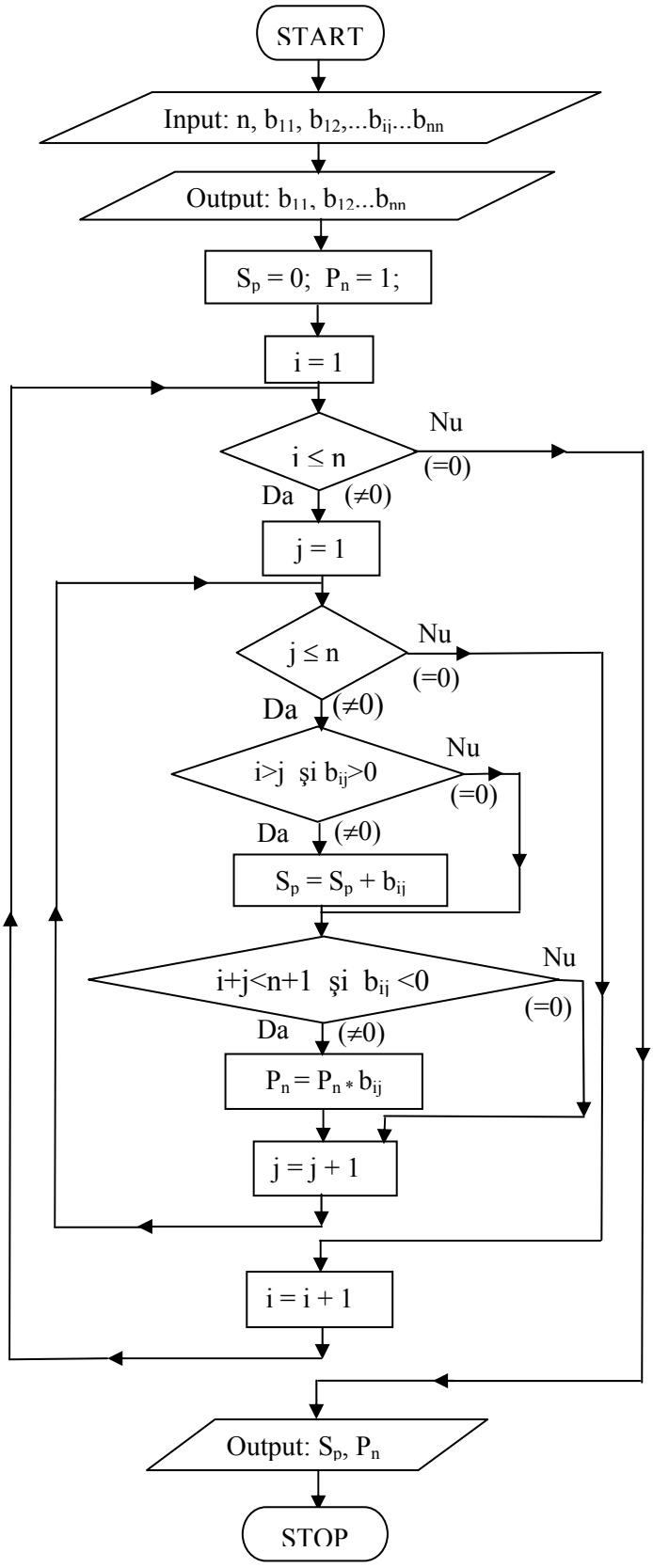

START
Input: n, b_{11}, b_{12},...b_{ij}...b_{nn}
Output: b_{11}, b_{12}...b_{nn}
$S_p = 0$; $P_n = 1$;
i = 1
i ≤ n
Nu
(=0)
Da
(≠0)
j = 1
j ≤ n
Nu
(=0)
Da
(≠0)
i>j şi b_{ij}>0
Nu
(=0)
Da
(≠0)
$S_p = S_p + b_{ij}$
i+j<n+1 şi b_{ij} <0
Nu
(=0)
Da
(≠0)
$P_n = P_n * b_{ij}$
j = j + 1
i = i + 1
Output: S_p, P_n
STOP

Fig. 2.13

Schema logică în care se prezintă operația de transpunere este următoarea:

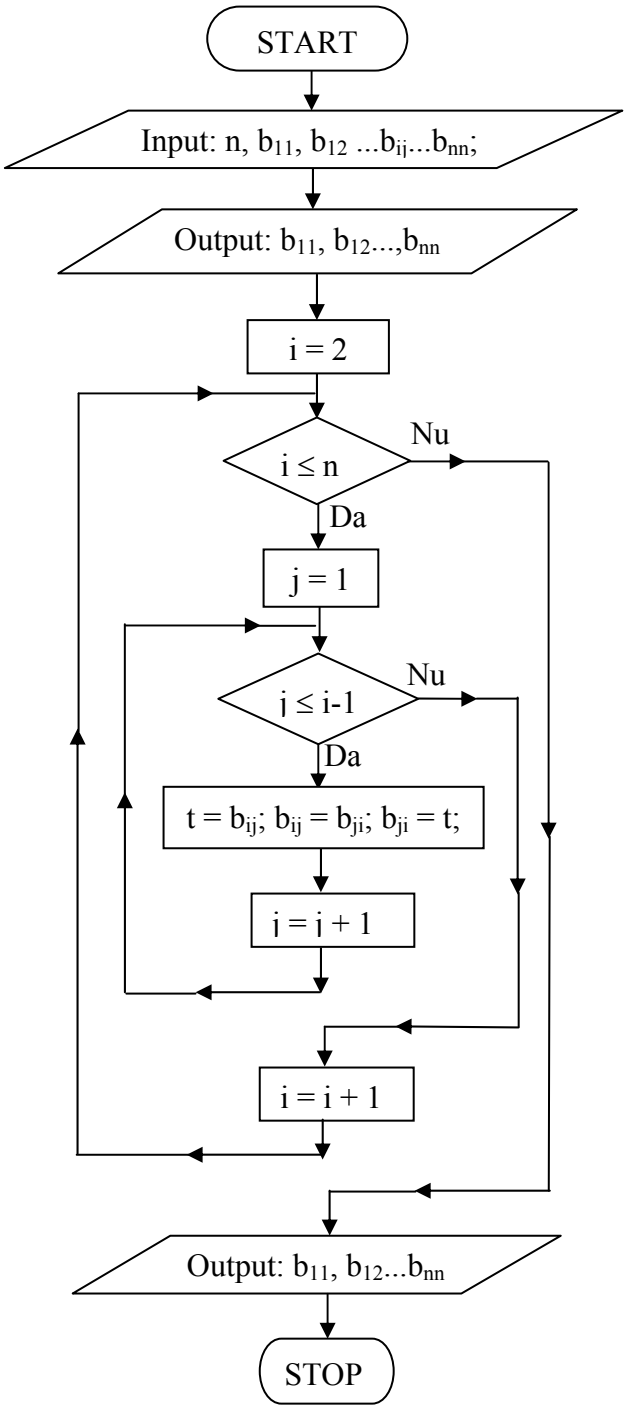


Fig. 2.14

Dacă nu s-ar fi impus condiţia ca să se păstreze aceeaşi notaţie, matricea iniţială fiind **B,** cu **m** linii şi **n** coloane, iar matricea transpusă s-ar fi notat cu $\mathbf{C} = \mathbf{B}^t$ (cu **n** linii şi **m** coloane) atunci ciclul exterior, condus de **i**, s-ar fi parcurs de **m** ori iar cel interior, condus de **j** , s-ar fi parcurs de **n** ori, executându-se de **m×n** ori atribuirea

$$c_{j,i} = b_{i,j}$$

13. Formarea unei matrice utilizând relaţii între elementele acesteia.

Una din cele mai bune exemplificări o constitue determinarea numerelor lui **Stirling** de **speţa I**. Acestea sunt coeficienţii polinoamelor prin care se exprimă **factorialele "generalizate"**,

$$x^{[i]} = x(x-1)(x-2)\ldots(x-i+1) = \sum_{j=1}^{i+1} S_{i,j}\ x^{j-1},$$

astfel,

$$\begin{aligned}
x^{[1]} &= x && = S_{1,1}\ x^0 + S_{1,2}\ x^1,\\
x^{[2]} &= x(x-1) && = S_{2,1}\ x^0 + S_{2,2}\ x^1 + S_{2,3}\ x^2\\
x^{[3]} &= x(x-1)(x-2) && = S_{3,1}\ x^0 + S_{3,2}\ x^1 + S_{3,3}\ x^2 + S_{3,4}\ x^3\\
x^{[4]} &= x(x-1)(x-2)(x-3) && = S_{4,1}\ x^0 + S_{4,2}\ x^1 + S_{4,3}\ x^2 + S_{4,4}\ x^3 + S_{4,5}\ x^4\\
&\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots
\end{aligned}$$

Efectuând calculele, coeficienţii acestor polinoame (numerele lui Stirling de speţa I) se vor putea scrie sub forma unei matrice :

$$\begin{bmatrix}
0 & 1 & & & & & & & \\
0 & -1 & 1 & & & & & & \\
0 & 2 & -3 & 1 & & & & & \\
0 & -6 & 11 & -6 & 1 & & & & \\
0 & .. & .. & .. & .. & 1 & & & \\
0 & .. & .. & S_{i-1,j-1} & S_{i-1,j} & .. & 1 & & \\
0 & .. & .. & .. & S_{i,j} & .. & .. & 1 & \\
0 & .. & .. & .. & .. & .. & .. & .. & 1
\end{bmatrix}$$

În [Hamm62] §1.7 se demonstrează următoarea relaţie de recurenţă :

$$S_{i,j} = S_{i-1,j-1} - (i-1)\,S_{i-1,j}$$

cu care se vor putea calcula valorile elementelor, începând cu cele de pe linia a doua.

Se vor calcula coeficienţii primelor **n** factoriale generalizate. În acest caz matricea care se va forma va avea **n** linii şi **n+1** coloane.

Schema logică va conţine două cicluri suprapuse, unul condus de **i** cu valori de la **1** la **n** (cel exterior) iar celălalt, ciclul interior, condus de **j** cu valori de la **1** la **n+1**, valoarea fiecărui element $\mathbf{S_{i,j}}$ determinându-se în felul următor:

$$S_{i,j} = \begin{cases} 0 & daca \quad j=1 \\ 1 & daca \quad j=i+1 \\ S_{i-1,j-1} - (i-1)\ S_{i-1,j} & daca \quad 2 \le j \le i \ \wedge i \ge 2 \\ 0 & daca \quad j > i+1 \ \wedge \ i < n \end{cases}$$

Schema logică este dată în figura 2.15.

Observaţia nr. 1. Cu ajutorul numerelor lui Stirling de speţa a doua se pot exprima puterile lui **x** în funcţie de factorialele generalizate

$$x^i = \sum_{j=1}^{i} S^*_{i,j}\ x^{[j]},$$

matricea formată de aceşti coeficienţi şi formula de recurenţă fiind următoarele :

$$\begin{bmatrix} 1 & & & & & & \\ 1 & 1 & & & & & \\ 1 & 3 & 1 & & & & \\ 1 & 7 & 6 & 1 & & & \\ 1 & 15 & 25 & 10 & 1 & & \\ 1 & 31 & 90 & 65 & 15 & 1 & \\ .. & .. & .. & .. & .. & .. & .. \end{bmatrix} \qquad S^*_{i,j} = S^*_{i-1,j-1} + j\, S^*_{i-1,j}$$

Observaţia nr. 2. Problema cea mai cunoscută de acest tip este cea a **triunghiului lui Pascal** (Blaise Pascal – Traité du triangle arithmétique, 1653), scrierea sub formă de matrice (triunghiulară) a coeficienţilor binomiali

$$\begin{bmatrix} 1 & & & & & & \\ 1 & 1 & & & & & \\ 1 & 2 & 1 & & & & \\ 1 & 3 & 3 & 1 & & & \\ 1 & 4 & 6 & 4 & 1 & & \\ 1 & 5 & 10 & 10 & 5 & 1 & \\ .. & .. & .. & .. & .. & .. & .. \end{bmatrix}$$

$$(a+b)^1 = a+b$$
$$(a+b)^2 = a^2 + 2ab + b^2$$
$$(a+b)^3 = a^3 + 3a^2 b + 3ab^2 + b^3$$
$$(a+b)^4 = a^4 + 4a^3 b + 6a^2 b^2 + 4ab^3 + b^4$$
$$(a+b)^5 = a^5 + 5a^4 b + 10a^3 b^2 + 10a^2 b^3 + 5ab^4 + b^5$$

începând cu linia a doua a matricei. Se poate observa că există următoarea relaţie de recurenţă

$$c_{i,j} = c_{i-1,j-1} + c_{i-1,j}$$

care rezultă din cunoscuta formulă din combinatorică,

$$C_i^j = C_{i-1}^{j-1} + C_{i-1}^j$$

legătura dintre combinări şi coeficienţii matricei triunghiulare fiind

$$c_{i,j} = C_{i-1}^{j-1}$$

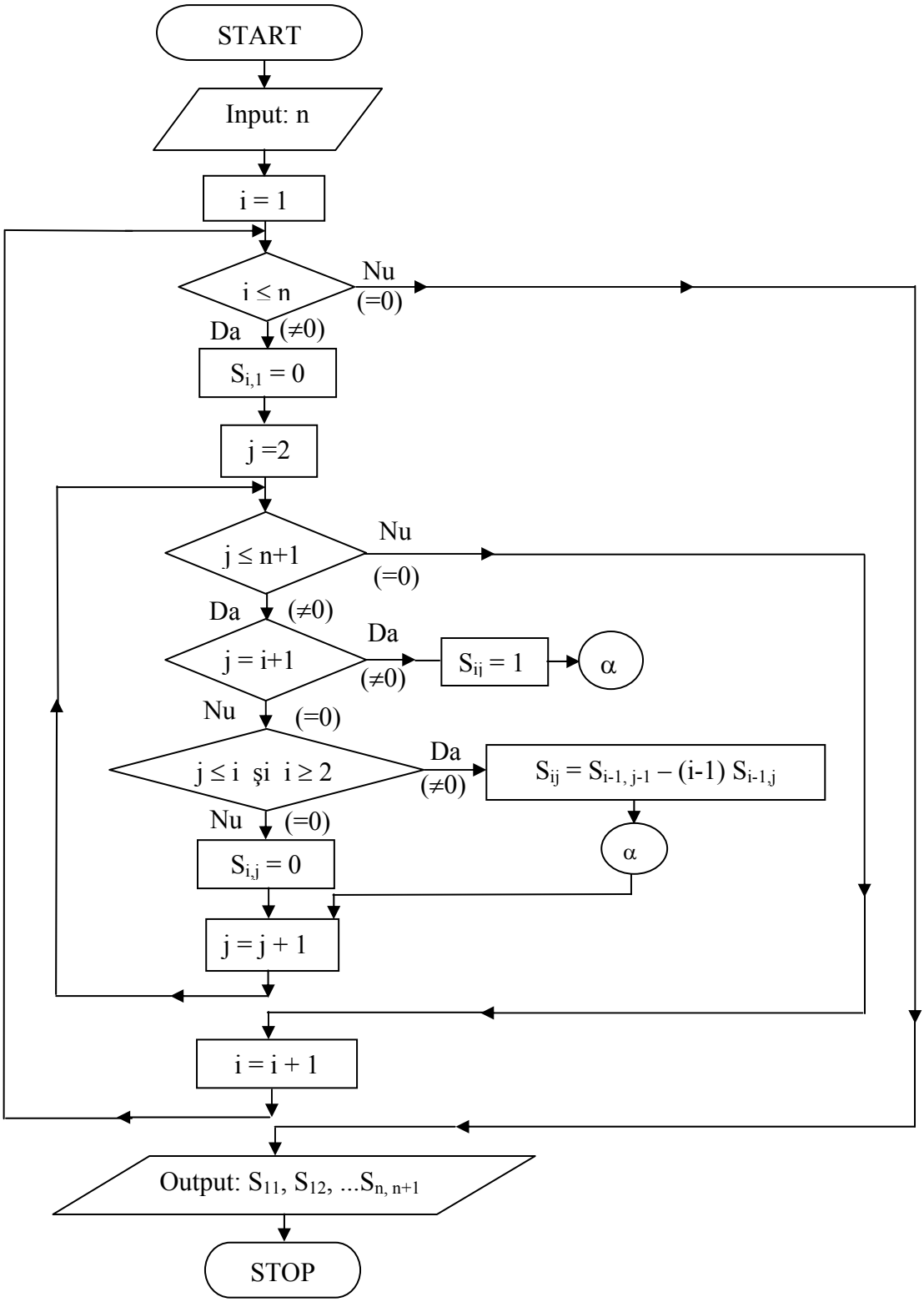

START
Input: n
i = 1
i ≤ n
Nu
(=0)
Da
(≠0)
$S_{i,1} = 0$
j =2
j ≤ n+1
Nu
(=0)
Da
(≠0)
j = i+1
Da
(≠0)
$S_{ij} = 1$
α
Nu
(=0)
j ≤ i şi i ≥ 2
Da
(≠0)
$S_{ij} = S_{i-1, j-1} – (i-1) S_{i-1,j}$
α
Nu
(=0)
$S_{i,j} = 0$
j = j + 1
i = i + 1
Output: $S_{11}, S_{12}, ...S_{n, n+1}$
STOP

Fig. 2.15

Un element oarecare al acestei matrice se calculează în felul următor

$$c_{i,j}=\begin{cases} 1 & daca \quad j=1 \vee j=i \\ 0 & daca \quad j>i \\ c_{i-1,j-1}+c_{i-1,j} & daca \quad 2\le j\le i-1 \wedge i\ge 3 \end{cases} \qquad i=\overline{1,n}$$

14. Calculul valorilor unei funcţii utilizând dezvoltări în serie.

Deoarece foarte multe funcţii admit dezvoltări în serii iar calculul sumelor se poate programa, prezintă interes acest tip de calcule, în special prin faptul că în cazul în care funcţia matematică utilizată este una uzuală aceasta se poate calcula şi direct, deci s-ar putea face o evaluare a preciziei.

Se consideră funcţia **sin(x)** a cărei dezvoltare în serie este

$$\sin x=\frac{x}{1!}-\frac{x^3}{3!}+\frac{x^5}{5!}-\frac{x^7}{7!}+\ldots..$$

Se cere determinarea valorii acestei funcţii pentru un **x** dat, utilizând primii **n** termeni ai dezvoltării.

Pentru a construi schema logică va trebui să observăm că de fapt avem de calculat suma unui şir,

$$\sin x \approx s = t_1 + t_2 + t_3 + \ldots.. + t_i + \ldots.. + t_n$$

deci iniţial **s = 0** iar însumarea se face parcurgând repetat **s = s + t$_i$** , termenul general având expresia

$$t_i = (-1)^{i+1}\ \frac{x^{2i-1}}{(2i-1)!}$$

şi ar trebui calculat la fiecare pas. Pentru micşorarea numărului de operaţii se poate stabili o relaţie de recurenţă între doi termeni succesivi, **t$_i$** şi **t$_{i+1}$** ,

$$t_{i+1} = (-1)^{i+2}\ \frac{x^{2i+1}}{(2i+1)!} \qquad t_{i+1} = -\ t_i\ \frac{x^2}{(2i+1)(2i)}$$

iar în cazul utilizării acestei relaţii de recurenţă se va efectua acelaşi număr de operaţii, pentru a trece de la un termen la altul, indiferent de termenul care se calculează.

Schema logică din fig. 2.16 are un singur ciclu, condus de variabila **i** , în cadrul corpului acestuia se face însumarea **s = s + t$_i$** şi calculul valorii noului termen, utilizând relaţia de recurenţă. Dacă se face convenţia că **t** reprezintă valoarea termenului curent, se poate renunţa la indice. Înainte de intrarea în ciclu se fac cele două iniţializări **s = 0** şi **t = x**

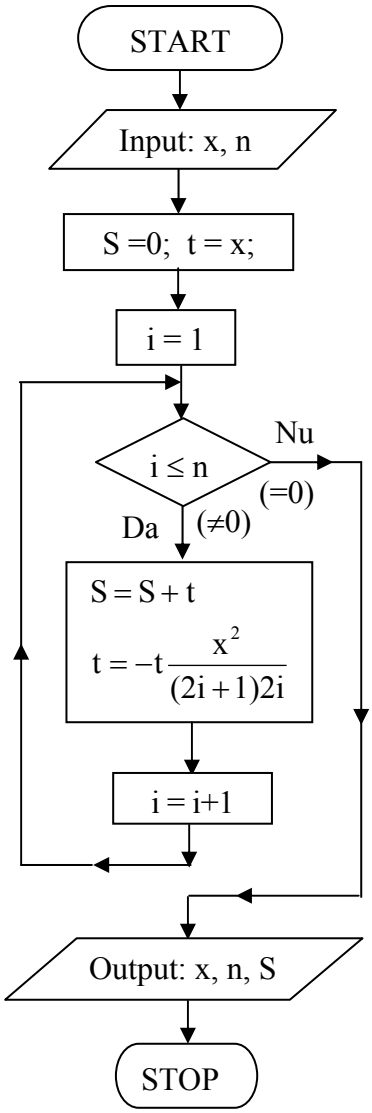


Fig. 2.16

Observaţia 1. În mod asemănător se pot calcula valorile altor funcţii. Dacă se consideră

$$\arcsin x = x + \frac{1}{2}\frac{x^3}{3} + \frac{1\cdot 3}{2\cdot 4}\frac{x^5}{5} + \frac{1\cdot 3\cdot 5}{2\cdot 4\cdot 6}\frac{x^7}{7} + \ldots . \qquad x \in (-1;1)$$

atunci se va scrie

$$\arcsin x \approx s = x + t_1 + t_2 + t_3 + \ldots . + t_i + t_{i+1} + \ldots . + t_{n-1}$$

şi se va deduce expresia termenului general şi relaţia de recurenţă

$$t_i = \frac{(2i-1)!!}{(2i)!!}\,\frac{x^{2i+1}}{2i+1} \qquad t_{i+1} = t_i\,\frac{(2i+1)^2\,x^2}{(2i+2)(2i+3)}$$

Se va putea folosi o schemă logică asemănătoare, cu modificările următoare: iniţializarea sumei este **s = x**, a primului termen este **t = x³/6**, ciclul se va parcurge de **n-1** ori iar formula de recurenţă se va modifica în mod corespunzător.

Observaţia 2. Utilizând acelaşi procedeu, se pot calcula şi unele integrale definite care se pot exprima cu serii convergente, cum ar fi

$$\int_0^a \frac{\sin x}{x}\,dx = \frac{a}{1\cdot 1!} - \frac{a^3}{3\cdot 3!} + \frac{a^5}{5\cdot 5!} - \frac{a^7}{7\cdot 7!} + \ldots + (-1)^{i+1}\,\frac{a^{2i-1}}{(2i-1)(2i-1)!} + \ldots$$

Această integrală se numeşte **sinus integral**. Cunoscându-se expresia termenului general se poate stabili şi relaţia de recurenţă

$$t_{i+1} = -\,t_i\,\frac{(2i-1)\,a^2}{(2i)(2i+1)^2}$$

În schema logică anterior prezentată se vor face modificările : **t = a** (iniţializarea valorii primului termen) şi se va introduce noua expresie de la recurenţă.

Observaţia 3. În mod similar s-ar putea calcula şi sume de combinări, de tipul

$$S = C_n^0 + C_n^3 + C_n^6 \quad +\ldots. + C_n^{3i-3} +\ldots. + \begin{cases} C_n^{n-2} \\ C_n^{n-1} \\ C_n^n \end{cases}$$

$$(t_1)\quad (t_2)\quad (t_3) \qquad\qquad (t_i)$$

suma fiind considerată iniţial egală cu zero, **S = 0**, primul termen fiind **t = 1** iar cunoscând expresiile a doi termeni succesivi,

$$t_i = C_n^{3i-3} = \frac{n!}{(n-3i+3)!\,(3i-3)!} \qquad t_{i+1} = C_n^{3i} = \frac{n!}{(n-3i)!\,(3i)!}$$

se poate determina şi relaţia de recurenţă :

$$t_{i+1} = t_i\,\frac{(n-3i+3)(n-3i+2)(n-3i+1)}{(3i)(3i-1)(3i-2)}$$

Această problemă diferă de cele anterioare prin faptul că numărul de termeni care trebuie însumat (numărul de parcurgeri al ciclului) nu este **n** ci **m = [n/3] + 1**.

Rezultatul obţinut se poate verifica deoarece valoarea sumei se poate calcula cu formula :

$$S = \frac{1}{3}\left(\,2^n + 2\,\cos\frac{\pi n}{3}\,\right)$$

15. Probleme aplicative din domeniul ingineriei mecanice.

Considerăm că ar fi deosebit de util ca începând chiar din acest capitol să se prezinte o serie de probleme din domeniul ingineriei (mecanice), rezolvarea cărora se poate face cu calculatorul.

Exemplul A. Pentru a realiza transformarea unei mişcări de rotaţie continuă a arborelui **1** într-o mişcare intermitentă a arborelui **2** se poate utiliza mecanismul tip "**Cruce de Malta**", reprezentat în figura 2.17.

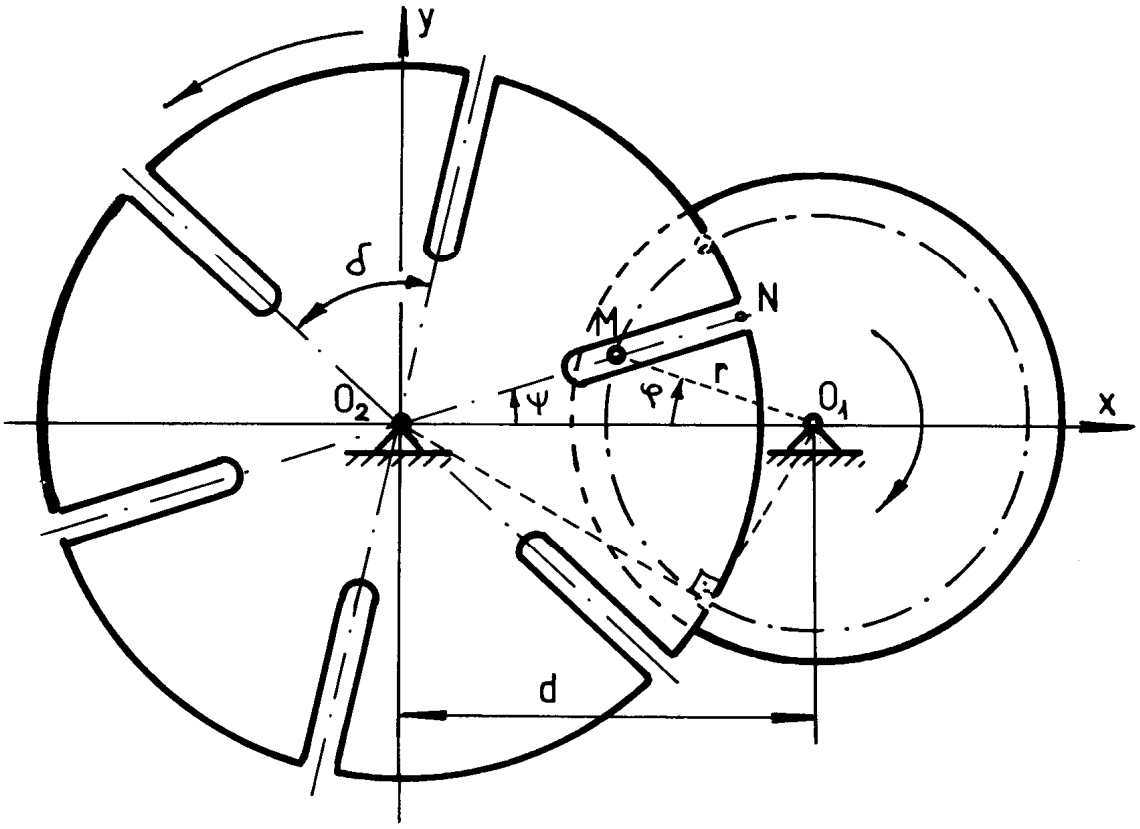


Fig. 2.17

Se poate observa că dacă iniţial elementul $\mathbf{O_1M}$ este în poziţie orizontală şi canalul $\mathbf{O_2N}$ din crucea de Malta este de asemenea orizontal. Dacă $\mathbf{O_1M}$ se roteşte atunci se va roti şi elementul condus, până când **M** părăseşte canalul. În continuare, $\mathbf{O_1M}$ se roteşte iar crucea va rămânea nemişcată, până când **M** va intra în canalul următor iar în momentul când $\mathbf{O_1M}$ a ajuns din nou în poziţie orizontală crucea de Malta s-a rotit cu un unghi egal cu cel dintre canale. Rezultă deci că pentru a obţine o rotaţie completă a elementului condus (crucea de Malta) este necesar ca elementul conducător (manivela $\mathbf{O_1M}$) să efectueze un număr de rotaţii complete egal cu numărul de canale.

Tot din figură rezultă că este necesar ca traiectoria circulară a ştiftului M să fie perpendiculară pe traiectoria, de asemenea circulară, a capătului **N** al canalului crucii de Malta, rezultând deci relaţia :

$$r = d \sin \frac{\delta}{2} .$$

Pe baza relaţiilor geometrice ce se pot stabili rezultă următoarele formule de calcul a unghiului de rotaţie ψ al crucii de Malta în funcţie de unghiul de rotaţie φ al manivelei $\mathbf{O_1M}$:

$$\psi = \begin{cases} arctg \dfrac{r \sin\varphi}{d - r\cos\varphi} & 0 \le \varphi \le \dfrac{\pi}{2} - \dfrac{\delta}{2} \\ \dfrac{\delta}{2} & \dfrac{\pi}{2} - \dfrac{\delta}{2} < \varphi < \dfrac{3\pi}{2} + \dfrac{\delta}{2} \\ \dfrac{\delta}{2} - arctg \dfrac{r \sin\varphi}{d - r\cos\varphi} & \dfrac{3\pi}{2} + \dfrac{\delta}{2} \le \varphi \le 2\pi \end{cases}$$

Ca date de intrare pentru acestă problemă vor fi: distanţa **d** dintre cele două axe, numărul de canale $\mathbf{n_c}$ şi pasul de variaţie $\mathbf{\Delta}$ (în grade) al unghiului φ, unghi care va lua valori în intervalul **[0; 360].** Va rezulta unghiul dintre canale $\mathbf{\delta = 2\pi/n_c}$.

Problema respectivă se încadrează într-o categorie de probleme foarte frecvent întâlnite în programare : calculul valorilor unei funcţii într-un interval

În cazul acestei probleme trebuie să se aibe în vedere faptul că unghiurile care apar ca argumente în funcţiile trigonometrice trebuie să fie date în radiani, de asemenea în radiani se exprimă şi rezultatul calculului funcţiei arctg (**atan** în toate limbajele de programare). Valorile unghiurilor care se afişează trebuie exprimate în grade. Din acest motiv în schema logică se vor utiliza două variabile pentru unghiul φ, anume φ_{grad} respectiv φ_{rad} şi tot două pentru unghiul ψ, ψ_{grad} respectiv ψ_{rad} . Reamintim că relaţia de legătură dintre un unghi exprimat în grade respectiv în radiani este următoarea:

$$\alpha_{radiani} = \alpha_{grade} \frac{\pi}{180} .$$

Schema logică este prezentată în figura 2.18.

S-a notat cu **i** numărul de ordine al punctului considerat din interval.

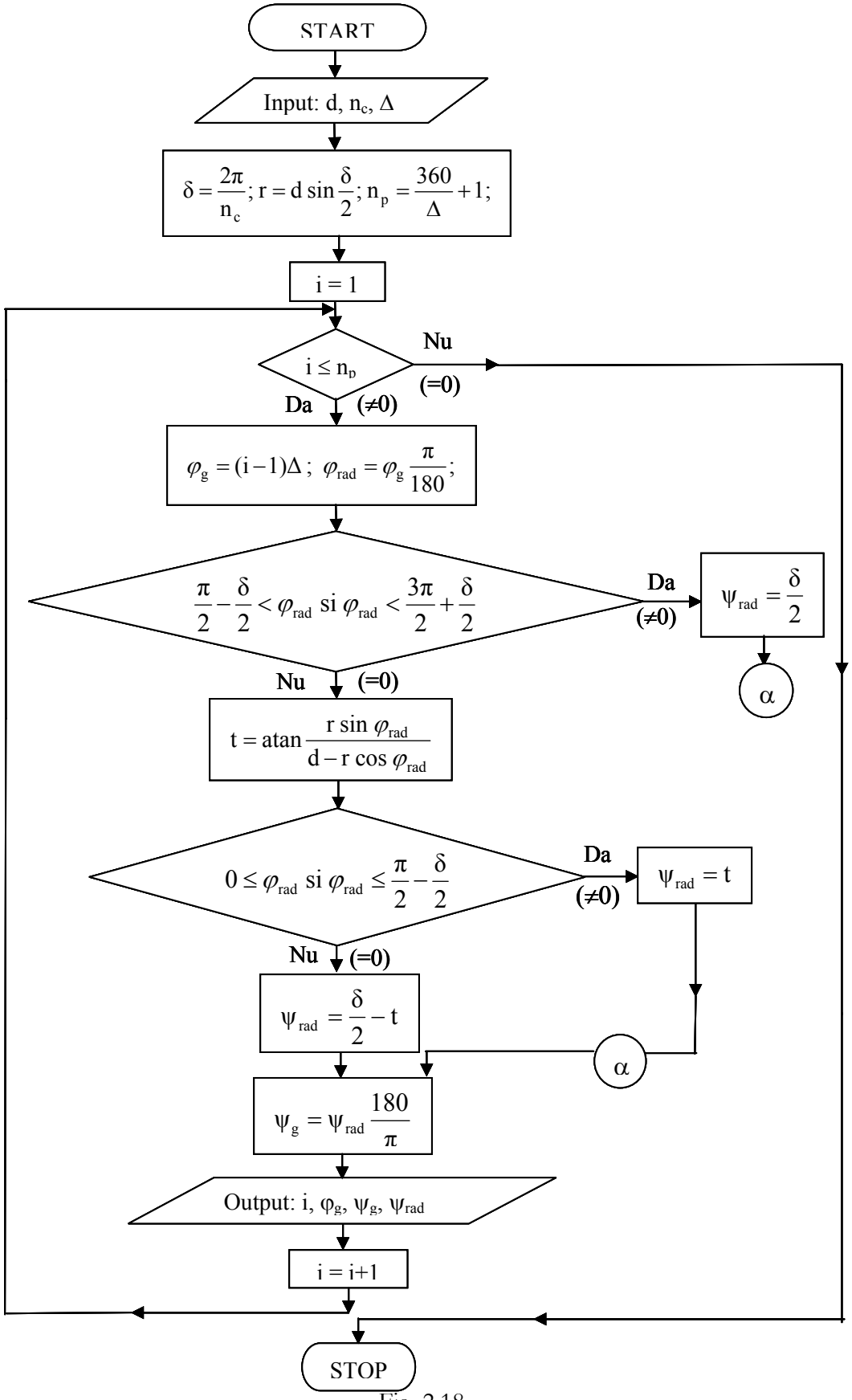

START
Input: d, n_c, Δ
$\delta = \frac{2\pi}{n_c}; r = d \sin\frac{\delta}{2}; n_p = \frac{360}{\Delta} + 1;$
i = 1
$i \le n_p$
Nu
(=0)
Da
(≠0)
$\varphi_g = (i-1)\Delta;\ \varphi_{rad} = \varphi_g \frac{\pi}{180};$
$\frac{\pi}{2} - \frac{\delta}{2} < \varphi_{rad}$ si $\varphi_{rad} < \frac{3\pi}{2} + \frac{\delta}{2}$
Da
(≠0)
$\psi_{rad} = \frac{\delta}{2}$
α
Nu
(=0)
$t = \text{atan}\frac{r \sin \varphi_{rad}}{d - r \cos \varphi_{rad}}$
$0 \le \varphi_{rad}$ si $\varphi_{rad} \le \frac{\pi}{2} - \frac{\delta}{2}$
Da
(≠0)
$\psi_{rad} = t$
Nu
(=0)
$\psi_{rad} = \frac{\delta}{2} - t$
α
$\psi_g = \psi_{rad} \frac{180}{\pi}$
Output: i, φ_g, ψ_g, ψ_{rad}
i = i+1
STOP

Fig. 2.18

Exemplul B. Roata dinţată cu z_1 dinţi, fixată pe arborele **1**, care se roteşte cu turaţia n_1 , este în angrenare cu roata dinţată cu z_2 dinţi, fixată pe arborele **2** (fig. 2.19). Cunoscându-se turaţia n_1 a arborelui **1** şi impunând condiţia ca arborele **2** să aibă turaţia n_2 , din intervalul [0.2 n_1 ; 5 n_1], se cere să se determine numerele de dinţi z_1 şi z_2 care vor realiza cel mai exact turaţia cerută a arborelui **2**.

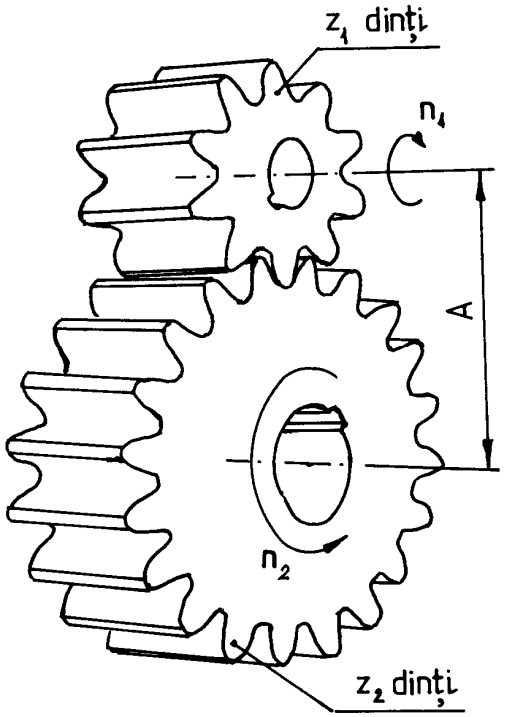


Fig. 2.19

Se ştie că $n_1/n_2 = z_2/z_1$.

În urma introducerii celor două valori numerice ale turaţiilor, n_1 şi n_2 , a căror raport **r** este deci cunoscut, $r = n_1/n_2$ se pune problema de a determina două numere întregi, z_2 şi z_1 , a căror raport să fie cât mai apropiat de **r**. Numerele întregi z_1 şi z_2 se vor căuta în intervalul **[20; 100]** .

Se va nota cu $z_{1\,OPTIM}$ şi $z_{2\,OPTIM}$ valorile optime găsite până la etapa respectivă a numerelor de dinţi care aproximează cel mai bine raportul **r** iar cu R_{OPTIM} raportul acestora, $R_{OPTIM} = z_{2\,OPTIM}/z_{1\,OPTIM}$. Iniţial se vor atribui acestor variabile valori arbitrare.

Schema logică va conţine două cicluri suprapuse, unul condus de z_1 care va lua valori de la **20** la **100** iar celălalt de z_2 care va lua valori în acelaşi interval.

Pentru fiecare combinaţie de valori a lui z_1 şi z_2 se va calcula raportul $R = z_2/z_1$ şi se va face verificarea inegalităţii

$$|r - R| < |r - R_{OPTIM}|$$

dacă aceasta este adevărată înseamnă că valorile curente ale lui z_1 şi z_2 sunt mai bune decât cele determinate anterior, $z_{1\,OPTIM}$ şi $z_{2\,OPTIM}$ deci acestea vor trebui înlocuite, de asemenea şi valoarea lui R_{OPTIM} , în caz contrar vor rămânea aceleaşi valori. După parcurgerea completă a celor două cicluri variabilele $z_{1\,OPTIM}$ şi $z_{2\,OPTIM}$ au valorile căutate ale numerelor de dinţi a roţilor dinţate care aproximează cel mai bine raportul **r**,

$$r = \frac{n_1}{n_2} \approx \frac{z_{2\,OPTIM}}{z_{1OPTIM}}$$

În continuare se prezintă schema logică corespunzătoare acestei probleme (fig. 2.20) :

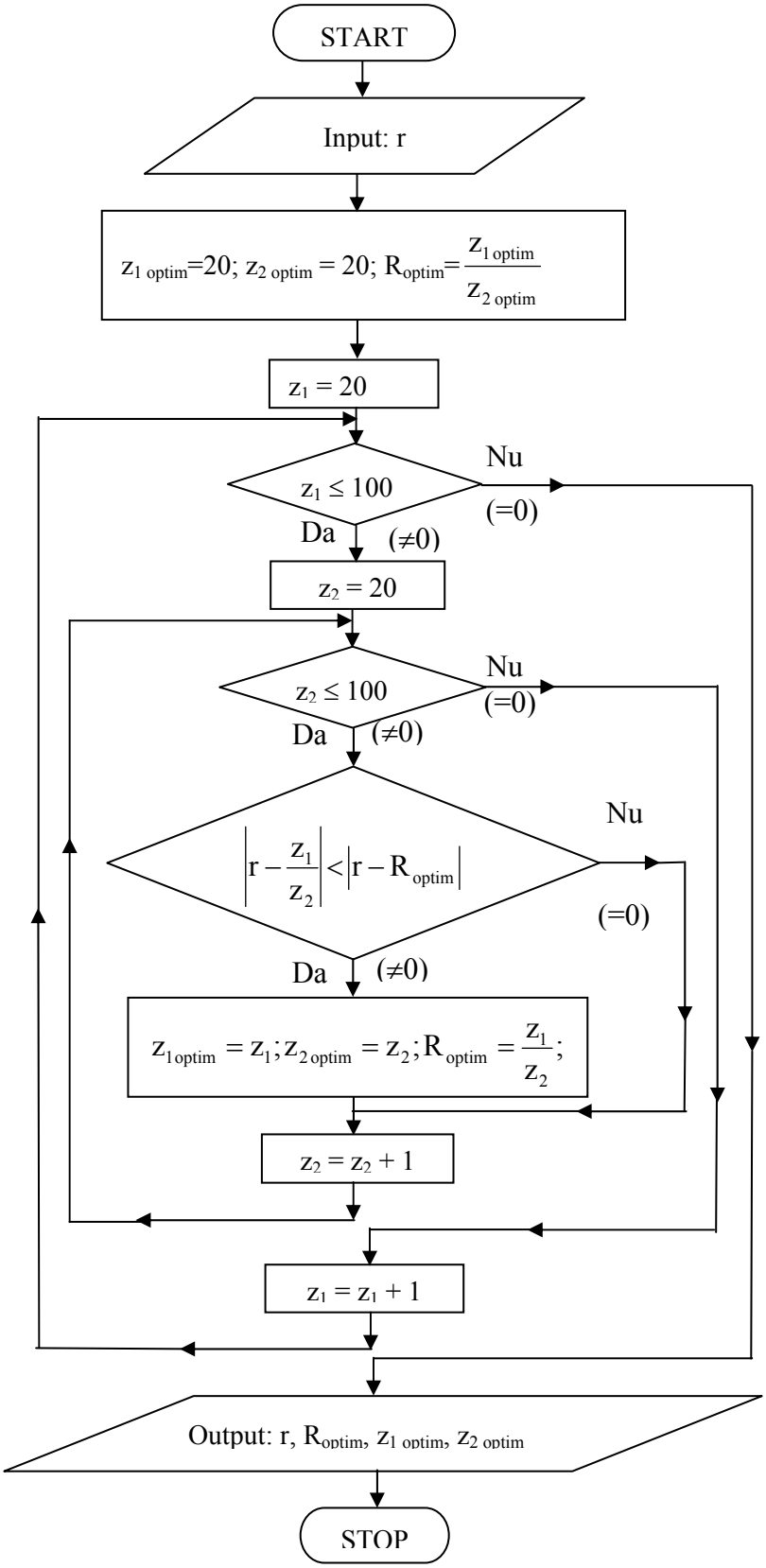

START
Input: r
$z_{1\,optim}$=20; $z_{2\,optim}$ = 20; $R_{optim}=\frac{z_{1\,optim}}{z_{2\,optim}}$
$z_1 = 20$
$z_1 \le 100$
Nu
(=0)
Da
(≠0)
$z_2 = 20$
$z_2 \le 100$
Nu
(=0)
Da
(≠0)
$\left|r-\frac{z_1}{z_2}\right| < \left|r-R_{optim}\right|$
Nu
(=0)
Da
(≠0)
$z_{1optim} = z_1; z_{2optim} = z_2; R_{optim} = \frac{z_1}{z_2};$
$z_2 = z_2 + 1$
$z_1 = z_1 + 1$
Output: r, R_{optim}, $z_{1\,optim}$, $z_{2\,optim}$
STOP

Fig. 2.20

Exemplul C. În cazul studiului vibraţiilor unui sistem mecanic cu elemente "în linie" se utilizează o matrice tridiagonală având următoarea formă [Ursu98a]:

$$D = \begin{bmatrix} \frac{k_1+k_2}{m_1} & -\frac{k_2}{m_1} & 0 & 0 & .. & 0 & 0 \\ -\frac{k_2}{m_2} & \frac{k_2+k_3}{m_2} & -\frac{k_3}{m_2} & 0 & .. & 0 & 0 \\ 0 & -\frac{k_3}{m_3} & \frac{k_3+k_4}{m_3} & -\frac{k_4}{m_3} & .. & 0 & 0 \\ .. & .. & .. & .. & .. & .. & .. \\ 0 & 0 & 0 & 0 & .. & \frac{k_{n-1}+k_n}{m_{n-1}} & -\frac{k_n}{m_{n-1}} \\ 0 & 0 & 0 & 0 & .. & -\frac{k_n}{m_n} & \frac{k_n+k_{n+1}}{m_n} \end{bmatrix}$$

numită "**matrice dinamică**" iar cele două şiruri, elementele cărora sunt utilizate pentru formarea elementelor matricei,

$$m_1 \quad m_2 \quad m_3 \quad \ldots\ldots \quad m_i \quad \ldots\ldots \quad m_n$$

$$k_1 \quad k_2 \quad k_3 \quad \ldots\ldots \quad k_i \quad \ldots\ldots \quad k_n \quad k_{n+1}$$

conţin valorile **maselor** sistemului mecanic respectiv a **constantelor elastice**.

Dacă se consideră un element $\mathbf{d_{i,\,j}}$ al acestei matrice, se poate observa că expresia acestuia este funcţie de valorile indicilor **i** şi **j**, după cum urmează

$$d_{i,j} = \begin{cases} 0 & daca \;\; 1 \le j < i-1 \\ -\frac{k_i}{m_i} & daca \;\; j = i-1 \\ \frac{k_i + k_{i+1}}{m_i} & daca \;\; j = i \\ -\frac{k_{i+1}}{m_i} & daca \;\; j = i+1 \\ 0 & daca \;\; j > i+1 < j \le n \end{cases} \qquad i = \overline{1,\, n}$$

În cadrul schemei logice se vor utiliza două cicluri suprapuse, unul pentru a trece de la o linie la alta a matricei, condus de **i** iar celălalt pentru a parcurge elementele fiecărei linii, condus de **j** , în cadrul acestuia alegându-se formula de calcul, conform relaţiei anterior prezentate.

Schema logică este prezentată în figura 2.21.

În cazul în care se elaborează un program pe baza acestei scheme logice şi se introduc date numerice, este evident că toate valorile elementelor şirului m_1 , m_2 , ... , m_n trebuie să fie strict pozitive iar cele ale şirului k_1 , k_2 , ... , k_n , k_{n+1} vor fi de asemenea strict pozitive, exceptând valorile elementelor k_1 şi k_{n+1} care pot fi eventual nule.

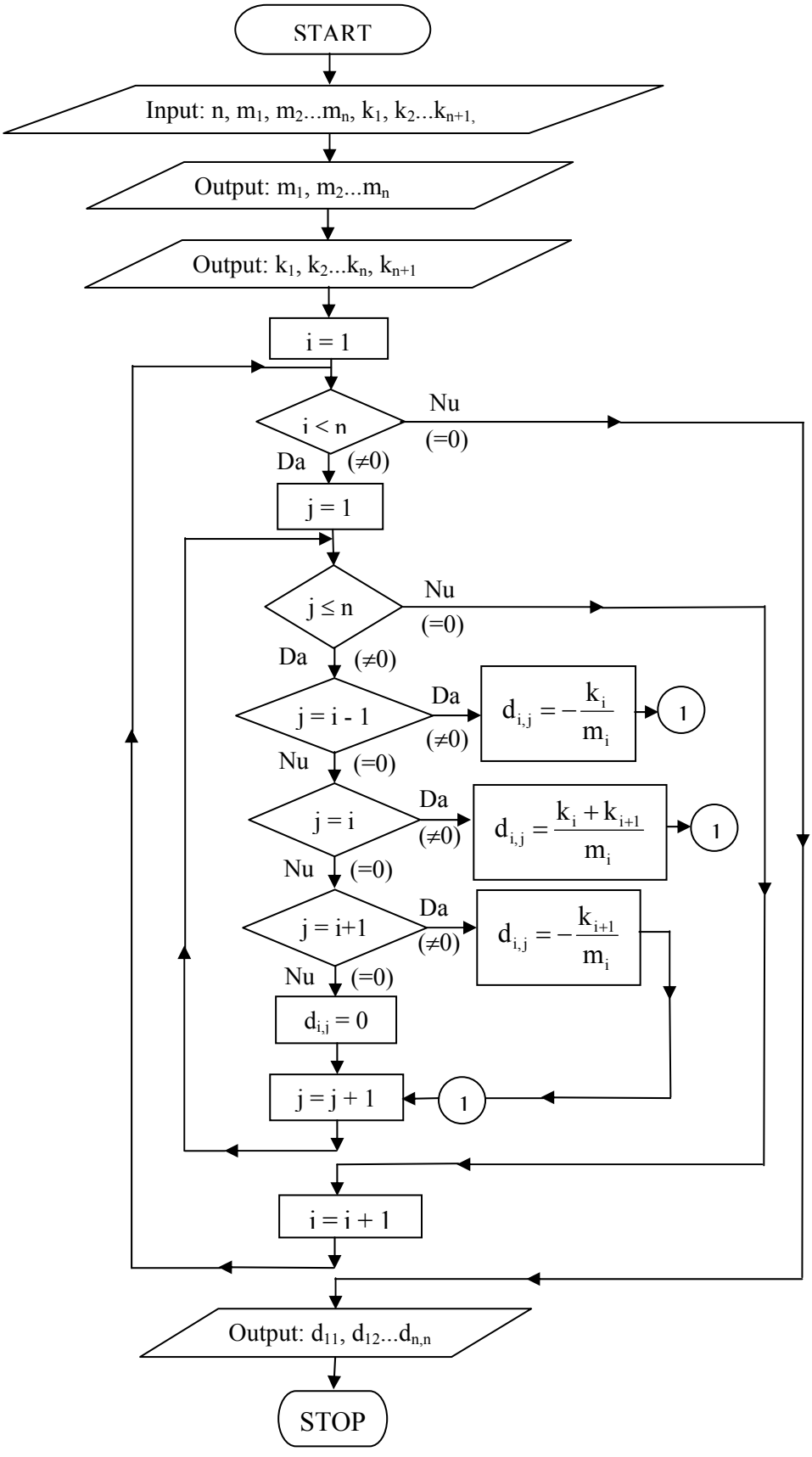

START
Input: n, m1, m2...mn, k1, k2...kn+1,
Output: m1, m2...mn
Output: k1, k2...kn, kn+1
i = 1
i < n
Nu
(=0)
Da
(≠0)
j = 1
j ≤ n
Nu
(=0)
Da
(≠0)
j = i - 1
Da
(≠0)
$d_{i,j} = -\frac{k_i}{m_i}$
1
Nu
(=0)
j = i
Da
(≠0)
$d_{i,j} = \frac{k_i + k_{i+1}}{m_i}$
1
Nu
(=0)
j = i+1
Da
(≠0)
$d_{i,j} = -\frac{k_{i+1}}{m_i}$
Nu
(=0)
di,j = 0
j = j + 1
1
i = i + 1
Output: d11, d12...dn,n
STOP

Fig. 2.21

Capitolul III

Scurt istoric al limbajelor C şi C++.

Apariţia **limbajului C** a fost marcată prin publicarea în **1978** (reeditată cu modificări în 1988) a cărţii **The C Programming Language**, în editura Prentice Hall, având ca autori pe **Brian W. Kernighan** şi **Dennis M. Ritchie** .

În prezent, după un număr relativ mic de ani, limbajul **C** a ajuns unul din cele mai populare limbaje profesionale de programare.

Cum a apărut acest limbaj nou de programare ?

Mai mulţi cercetători şi-au adus contribuţia la realizarea acestui limbaj, care "la origini" este o creaţie europeană.

Astfel, în anii '60 la Universitatea Cambridge a fost creat şi s-a utilizat limbajul **CPL** (**C**ambridge **P**rogramming **L**anguage).

În anul 1967 **Martin Richards**, tot de la Universitatea din Cambridge a creat o versiune, de fapt un nou limbaj, pe care l-a numit **BCPL** (**B**asic **CPL**). În cadrul acestui limbaj tipul variabilelor rezulta din context, transmiterea valorilor argumentelor se făcea numai prin valoare iar după părerea specialiştilor care l-au folosit gramatica şi structura lexicală erau destul de dificile. Prezentarea acestui limbaj s-a făcut într-o carte scrisă de Martin Richards şi Colin Whitby-Stevens.

Calculatoarele **PDP 7**, produse de firma **D**igital **E**quipment **C**orporation, în anul 1968 au fost dotate cu sistemul de operare UNIX, scris de către Ken Thompson în limbaj de asamblare, simţindu-se şi nevoia unui limbaj de nivel înalt. În această perioadă a apărut şi un limbaj TMG utilizat pentru scrierea compilatoarelor.

Pentru calculatoarele **PDP 11** , apărute în 1970, fiind nevoie să se rescrie sistemul de operare s-a creat şi utilizat de către **Ken Thompson** un nou limbaj de programare, numit **B** (se pare că după iniţiala prenumelui soţiei sale **B**onnie), bazat atât pe limbajul BCPL cât şi pe TMG .

Acestui limbaj i s-au adus succesiv o serie de îmbunătăţiri, s-a trecut şi printr-o fază intermediară cu limbajul **NB** (new B) iar în final s-a ajuns la varianta iniţială a limbajului **C** care a fost creată în **1972** de **Dennis Ritchie** şi **Ken Thompson**, acesta fiind utilizat pentru scrierea sistemului de operare **UNIX** pentru calculatoarele PDP-11.

Datorită succesului acestui limbaj, a faptului că tot mai mulţi programatori erau interesaţi în folosirea lui şi deoarece diferite firme de soft au început să producă diferite versiuni de compilatoare C, s-a simţit nevoia unei standardizări pentru o compatibilizare a versiunilor existente.

În cadrul **ANSI** (**A**merican **N**ational **S**tandards **I**nstitute) s-a creat un comitet de lucru numit X3J11, în anul 1983, care a realizat **limbajul ANSI C**, eliminând din versiunea iniţială anumite inconsistenţe şi scăpări. Versiunea finală, standardizată, a limbajului **C** a apărut în anul **1990** la care s-au făcut o serie de completări în anul **1995** .

Dacă la început limbajul **C** a fost considerat legat mai mult de sistemul de operare UNIX şi calculatoarele firmei DEC, odată cu apariţia cărţii mai sus citate acest limbaj a început să fie folosit şi la calculatoarele firmelor IBM, Honeywell ş. a.

Trecerea, după 1980, de la microprocesoarele pe 8 biţi la cele pe 16 biţi (Intel 8086, 8088) a permis accesarea unui RAM mai mare decât 64K, ceea ce a făcut ca limbajul **C** să-şi poată arăta mult mai bine calităţile.

Ce a făcut ca în prezent limbajul **C** să fie atât de folosit ?

Poate că în primul rând ar trebui prezentată părerea creatorilor acestui limbaj, aşa cum este exprimată în cartea de referinţă care a pus bazele limbajului C, înainte de standardizarea sa în ANSI C. Cei doi autori au caracterizat în felul următor limbajul pe care l-au creat :

The [C] language ... is not tied to any one operating system or machine; and although it has been called a "system programming language" because it is useful for writing operating systems, it has been used equally well to write major numerical, text-processing and database programs.

Desigur că un răspuns competent poate fi dat numai de o persoană care a mai "încercat" două-trei limbaje de programare, le cunoaşte posibilităţile şi s-a lovit şi de anumite inconveniente.

Limbajul de programare **C** este în prezent cel mai utilizat limbaj profesional de programare la nivel mondial. Este suficient ca să se urmărească numai titlurile cărţilor de programare care sunt editate atât în străinătate cât şi la noi ca să se observe acest lucru.

Limbajul **C** reuşeşte să combine cele mai reuşite elemente ale limbajelor de nivel înalt cum ar fi Pascal-ul, Fortran-ul şi altele cu flexibilitatea şi gradul de control oferite de limbajele de asamblare.

Un alt avantaj al limbajului **C** îl constitue numărul foarte mare de operatori existenţi precum şi facilităţile de reprezentare şi prelucrare a datelor.

Limbajul permite programarea structurată iar programele sunt portabile.

Acest limbaj permite operaţii cu biţi, octeţi, etc, ceea ce face să fie foarte utilizat în programarea la nivel de sistem.

Un avantaj, dar totodată şi dezavantaj al **C**-ului îl constitue flexibilitatea lui deosebită. Compilatorul **C** face mult mai puţine controale ale programului în faza de compilare, decât de exemplu compilatoarele realizate pentru limbajul Fortran sau Pascal, lucru care pe un programator începător câteodată îl poate deruta şi îngreunează depistarea erorilor.

Trebuie de asemenea amintit faptul că în prezent toate bibliotecile de programe ştiinţifice din domeniul matematicii, ştiinţelor exacte, etc., sunt transcrise şi în limbajul **C**.

Mulţi autori au publicat ediţii paralele de cărţi de metode numerice conţinând programe în Fortran, Pascal şi C, de exemplu [Pres90], [Vett90] , alţii în Fortran şi C [Enge96].

"Celălalt" limbaj de programare, **C++**, a apărut de asemenea ca urmare a publicării unei alte cărţi de referinţă, datorată unui tânăr cercetător , de origine daneză, **Bjarne Stroustrup**, de la Laboratoarele Bell, **The C++ Programming Language** , în anul **1986** la editura Addison Wesley, carte reeditată în 1991 şi 1997 .

Preocupat de probleme de simulare, care ar fi putut fi soluţionate şi cu programe scrise în limbajul Simula 67, B. Stroustrup a scris un compilator care întrunea anumite facilităţi ale C-ului şi mecanisme de modelare proprii limbajului Simula.

Denumirea aleasă, **C++**, sugerează faptul că de fapt este un **C** cu multiple noi facilităţi, putând fi considerat ca un nou limbaj, lucrând cu **clase** şi **obiecte**.

În cazul limbajului **C++** s-a făcut de asemenea o standardizare, de către comitetul X3J16, special creat în cadrul **ANSI.** Această standardizare, rezultatul căreia a fost **ANSI C++,** a fost finalizată în ianuarie **1994**.

Anteriorul limbaj standardizat, **ANSI C**, poate fi considerat ca un subset al limbajului **C++**, cu unele diferenţe.

Avantajul, şi în principal deosebirea, faţă de limbajul **C** îl constitue aşa numita **programare orientată pe obiecte** (**Object Oriented Programming – OOP**), care se poate face cu limbajul **C++**, admiţând de asemenea şi stilul tradiţional de programare, propriu limbajului C şi altor limbaje. Trebuie amintit faptul că există limbaje de programare, cum ar fi **Smalltalk** ş.a., care realizează numai programare orientată pe obiecte.

Trecerea de la **C**-ul obişnuit la **C++** a fost impusă de creşterea complexităţii unor programe, astfel încât erau foarte dificil de gestionat. Odată ce au fost atinse limitele la care s-a ajuns ca urmare a folosirii programării structurate a fost nevoie să se "inventeze" ceva nou adică să se adauge unui limbaj care şi-a dovedit pe deplin eficienţa, limbajul **C**, suportul care să permită şi programarea orientată pe obiecte.

Din păcate, nu putem să prezentăm aceste aspecte în cartea de faţă. Câteva cuvinte sunt totuşi necesare pentru a înţelege, măcar în principiu, despre ce e vorba.

Limbajul **C++** a fost creat în anul **1980**, fiind iniţial numit **C cu clase**, apoi numele lui modificându-se în **C++**, în **1983**.

Pe lângă tipurile de date predefinite, în cazul unor probleme mai complexe, se poate lucra cu "**tipuri abstracte de date – clase**" care sunt formate din date cu reprezentări similare şi pentru care sunt definite nişte operaţii specifice, iar "**obiectul**" este o dată de un tip definit printr-o clasă.

În plus, alte câteva noţiuni esenţiale sunt folosite în cazul acestui limbaj de programare:

- încapsularea (crearea obiectelor şi asigurarea unui nivel de protecţie al datelor),

- moştenirea (se pot defini ierarhii între tipurile abstracte de date, un obiect poate prelua prototipul altui obiect) şi

- **polimorfismul** .

Prima grijă a unui programator este de a-şi găsi un suport cât mai complet şi performant pentru sarcinile avute. Dacă programele le doreşte realizabile într-un timp cât mai scurt şi fără multă bătaie de cap, atunci este foarte posibil ca un simplu compilator să fie depăşit de varietatea aplicaţiilor-ţintă şi de situaţiile neprevăzute. După cum se ştie, există o diferenţă bine definită între erorile sintactice din cod şi cele de logică în conceperea unei proceduri, funcţii sau a altei părţi de program. Chiar dacă nu există aparent nici o eroare, rezultatele livrate pot fi total diferite de cele aşteptate. În acest scop sunt foarte utile programele de "depanare", **Debugger**-ele, care permit urmărirea "pe viu", în timpul rulării, a valorilor atribuite variabilelor, a conţinutului zonelor de memorie, a mesajelor de eroare întoarse, a legăturilor între diverse corpuri de program etc. Acest lucru este deosebit de util acolo unde complexitatea structurală depăşeşte posibilităţile de organizare într-un singur fişier cu întreg codul necesar, iar munca devine orientată spre **proiecte**. Devine astfel posibilă crearea de programe generate de mii sau chiar milioane de linii cu cod sursă, conţinute în mai multe fişiere ce alcătuiesc, grupat, pilonii pe care se clădeşte aplicaţia. Diversele funcţii conţinute în aceste fişiere vor da culoare fiecărei faţete a programului, de exemplu una axată strict pe calcul numeric, o alta pe interfaţa grafică cu utilizatorul, alta pe lucrul cu fişiere pentru citirea sau salvarea datelor folosite, o alta pentru managementul resurselor computaţionale ale staţiei de lucru etc.

Un alt punct important este dimensiunea şi viteza de execuţie a codului generat. Statistic, fiecare grup de programatori are o sferă favorită de aplicaţii, cazurile în care acelaşi om e nevoit să folosească toată gama de funcţii avute la dispoziţie în C++ fiind rarităţi. Ţinând cont de spectrul complet al aplicaţiilor scrise în C/C++ - **sisteme de operare**, **comandă la nivel hardware**, **compilatoare**, **aplicaţii pentru reţea**, **calcule ştiinţifice**, **grafică**, **baze de date**, **editoare de texte**, **jocuri**, etc. – astăzi e practic imposibilă cuprinderea într-un spaţiu restrâns a informaţiilor

detaliate referitoare la toate aceste teme. Pentru crearea de soft performant, munca în echipă şi buna organizare sunt noţiuni implicite. Fiecare programator va deveni profilat pe o anumită categorie de aplicaţii, schimbul de informaţii pe tema respectivă reducându-se în general la cei cu aceleaşi interese. Astfel devine posibilă optimizarea codului sursă pentru cele mai pretenţioase probleme, adăugând experienţa şi ideile celorlalţi. Pentru a rămâne în competiţie, toate firmele care doresc să-şi vândă marfa sunt nevoite să scrie aplicaţii pentru sistemele de operare cele mai răspândite, să livreze un cod cât mai mic, rapid şi fără bug-uri. De asemenea, pentru uşurarea muncii utilizatorului, s-a trecut exclusiv la programe capabile să interacţioneze prin intermediul controalelor : ferestre, cutii de dialog, butoane, liste etc. Indiferent de sistemul de operare, aplicaţiile care nu respectă aceste condiţii sunt astăzi practic scoase din scenă.

Toate cerinţele enumerate mai sus sunt îndeplinite de foarte puţine produse de firmă. Acestea au în comun caracteristica de **Visual Integrated Development Environment** : programarea vizuală, adică editarea "pe viu" a tuturor controalelor cu care interacţionează utilizatorul. Pentru Windows, cele mai cunoscute sunt **Microsoft Visual C++**, **Borland C++ Builder** şi **Watcom C/C++** (fostă PowerSoft). Pentru **Linux**, **KDevelop** depune eforturi pentru a ajunge din urmă complexitatea mediilor integrate Windows.

În continuare se va face o scurtă prezentare a primelor două produse, cu precădere din punctul de vedere al celor interesaţi de inginerie şi calcul ştiinţific.

Visual C++ (http://msdn.microsoft.com/visualc/) este dorit acolo unde viteza şi dimensiunea codului sunt priorităţi. Utilizat în special la crearea de aplicaţii orientate spre hardware, de exemplu sisteme de operare (Windows NT / ME /2000), de comandă şi achiziţie de date, asigură datorită bibliotecilor comune cu ale sistemului de operare Windows combinaţia optimă a utilizării şi gestionării resurselor. Totodată este foarte bine documentat, help-ul este extrem de complex, exemplele livrate de firmă acoperă practic întreg domeniul de aplicaţii iar compilatorul oferă suport pentru fiecare eroare detectată. Dispune de asemenea şi de un dezasamblor performant, posibil de utilizat inclusiv de către sistemul de operare (compatibil NT) pentru corectarea conflictelor generate de alte aplicaţii. Din 1997 este parte componentă a pachetului **MS Visual Studio**, iar în prezent e orientat şi spre dezvoltarea aplicaţiilor Internet.

Particulare îi sunt **MFC**-urile (MS Foundation Classes), clasele C++ specifice firmei. Un avantaj este că sunt relativ uşor de învăţat şi că oferă suport la zi pentru toată gama de aplicaţii Windows.

Un dezavantaj îl constituie lipsa formatului pentru date în virgulă mobilă de tipul **long double**, fiind mapat direct la **double**.

Borland C++ Builder (http://www.borland.com/bcppbuilder/) având toate cerinţele satisfăcute pentru un mediu de dezvoltare C++ complet, este în general preferat de către cei care urmăresc crearea de aplicaţii cu o bogată interfaţă grafică.

Şi acest pachet oferă propriile sale clase, cunoscute ca **OWL** (**O**bject **W**indows **L**ibrary). Fiind mult mai abstracte decât MFC, e posibil să nu fie totdeauna o alegere inspirată programatorului C++ începător. Mai puţin prietenoase datorită complexităţii excesive a arhitecturii lor, OWL sunt accesibile la un nivel înalt doar după mulţi ani de utilizare. Un aspect deloc neglijabil este că firma Borland a inclus în ultimele versiuni OWL şi codul pentru MFC, făcând astfel posibilă portarea aproape tuturor aplicaţiilor scrise în Visual C++.

Un dezavantaj este lipsa controlului asupra nivelului de link-editare, ce are urmări nu întotdeauna pozitive asupra codului generat.

Capitolul IV

Probleme generale privind programele scrise în limbajul C

Considerăm că un programator "începător" ar trebui să vadă cum arată un program scris în acest limbaj de programare, chiar dacă multe din cele prezentate vor face obiectul unor capitole viitoare.

De asemenea este deosebit de important să se cunoască posibilităţile mediului de programare ce se va utiliza, precum şi fazele care se vor parcurge în timpul prelucrării unui program.

4.1 Forma unui program scris în limbajul C

Se prezintă în continuare câteva exemple de programe scrise în C , cu scopul de a putea observa anumite probleme esenţiale privind forma lor generală de scriere.

Primul exemplu este următorul

fişierul ex01.cpp

```
/*  EXEMPLUL nr. 1
    Program pentru generarea unor sunete
    corespunzatoare notelor din octava a IV-a       */
#include <dos.h>
#define DO4 256
main()
{ sound(DO4);      delay(500); nosound();  //Nota do (IV)
  delay(250);
  sound(DO4*9/8);  delay(500); nosound();  //Nota re
  delay(250);
  sound(DO4*5/4);  delay(500); nosound();  //Nota mi
  delay(250);
  sound(DO4*4/3);  delay(500); nosound();  //Nota fa
  delay(250);
  sound(DO4*3/2);  delay(500); nosound();  //Nota sol
  delay(250);
  sound(DO4*5/3);  delay(500); nosound();  //Nota la
  delay(250);
  sound(DO4*15/8); delay(500); nosound();  //Nota si
  delay(250);
  sound(DO4*2);    delay(500); nosound();  //Nota do (V)
}
```

fiind un program conţinut într-un fişier cu numele specificat.

Ca elemente importante se poate observa că acesta este alcătuit dintr-o singură funcţie, a cărei nume este **main( ),** conţinutul acesteia fiind scris între cele două acolade **{ .... }** . În interiorul acestei funcţii se utilizează în mod repetat trei funcţii (***sound***, ***delay*** şi ***nosound***), care se găsesc într-un fişier cu numele **dos.h** (acest lucru este precizat în documentaţie şi se poate afla şi prin utilizarea HELP-ului mediului de programare). Pentru a se putea folosi aceste funcţii în programul respectiv s-a făcut includerea în programul sursă a fişierului **dos.h** prin "***directiva***"

#include <dos.h>

Un alt lucru care se poate observa este existenţa unor ***comentarii*** care au rolul de a prezenta conţinutul programului sau în alte cazuri mai complicate de a explica algoritmul de calcul utilizat.

Comentariile sunt cuprinse între succesiunea de caractere **/*** şi ***/** (după cum se vede în primele trei rânduri) sau pot fi scrise după succesiunea **//** până la sfârşitul rândului.

Comentariile pot fi plasate oriunde în programul sursă, dar în afara cuvintelor cheie şi a identificatorilor.

Nu s-au precizat argumentele funcţiei main, motiv pentru care aceasta s-ar mai fi putut scrie şi sub forma :

```
main(void)
{ ..... }
```

Deoarece funcţia **main** are ca rezultat al execuţiei sale o ***valoare întreagă*** acest lucru se poate preciza prin utilizarea unei declaraţii iar pentru a avea certitudinea că execuţia acesteia a fost corectă (completă), înainte de acolada finală se va scrie o instrucţiune care precizează valoarea returnată :

```
int main(void)
{  ..................
   ..................
   return 0;
}
```

Se mai poate observa că în acest program apare un al doilea rând care începe cu caracterul **#** (diez), explicaţiile urmând să fie date în cadrul §4.3.

O caracteristică generală a tuturor instrucţiunilor acestui program este că se termină cu **;** de asemenea şi declaraţiile referitoare la tipul unor variabile, cum se va vedea din exemplele următoare.

În cadrul exemplelor nr. 2 şi nr. 3 se prezintă două variante posibile ale aceluiaşi program, utilizabil pentru adunarea a doi vectori.

În exemplul nr. 2 există un singur fişier care conţine trei funcţii, dintre care una este funcţia **main( )**, a doua este o funcţie utilizată la introducerea datelor, **inp_vect(...)** iar a treia, **out_vect(...)**, realizează afişarea rezultatelor.

Se poate observa că în cadrul funcţiei **main( )** se utilizează cele două funcţii **inp_vect(...),** de două ori iar **out_vect(...)** o dată. De asemenea se poate observa existenţa unor *"declaraţii"* de funcţii, înainte de funcţia **main( ).**

Fişierul ex02.cpp

```
//  EXEMPLUL nr. 2

#include <conio.h>
#include <stdio.h>

void inp_vect(int n, double V[]); // Declaratie de functie
void out_vect(int n, double V[]); // Declaratie de functie

int main(void)                        // Functia  main
{ int i, n; double A[25], B[25], C[25];
  clrscr();
  printf("\n\t\t  n = "); scanf("%d", &n);
  inp_vect(n, A); inp_vect(n, B);
  for(i=0; i<n; i++) C[i]=A[i]+B[i];
  out_vect(n, C); getch(); return 0;
}

void inp_vect(int n, double V[])   // Functia nr. 2
{ int i; printf("\n\t Indice  Valoare element");
  for(i=0; i<n; i++)
     { printf("\n\t %2d\t\t", i); scanf("%le", &V[i]); }
}

void out_vect(int n, double V[])   // Functia nr. 3
{ int i; printf("\n\t Indice  Valoare element");
  for(i=0; i<n; i++) printf("\n\t %2d      %14.6le", i, V[i]);
}
```

În cel de-al treilea exemplu se arată un alt mod posibil de a scrie un astfel de program : fiecare funcţie este constituită ca un fişier separat iar în fişierul care conţine funcţia **main ( )** există "directive" de includere a celor două fişiere care conţin funcţiile.

Fişierul ex03.cpp

```
//  EXEMPLUL nr. 3

#include <conio.h>
#include <stdio.h>

#include "inp_vect.cpp"
#include "out_vect.cpp"

int main(void)
{ int i, n; double A[25], B[25], C[25];
  clrscr(); printf("\n\t\t  n = "); scanf("%d", &n);
  inp_vect(n, A); inp_vect(n, B);
  for(i=0; i<n; i++) C[i]=A[i]+B[i];
  out_vect(n, C); getch(); return 0;
}
```

Fişierul inp_vect.cpp

```
void inp_vect(int n, double V[])
{ int i; printf("\n\t Indice  Valoare element");
  for(i=0; i<n; i++)
     { printf("\n\t %2d\t\t",i); scanf("%le", &V[i]); }
}
```

Fişierul out_vect.cpp

```
void out_vect(int n, double V[]) { int i; printf("\n\t Indice
Valoare  element");     for(i=0;  i<n;  i++)  printf("\n\t  %2d
%14.6le", i, V[i]); }
```

În Capitolul **X** – Funcţii – vor fi prezentate toate regulile ce trebuie aplicate în cazul când un program este scris sub una din formele prezentate în exemplele 2 şi 3.

În exemplele nr. 2 şi 3 , în cadrul funcţiei **main( )** cât şi a celorlalte două, se folosesc o serie de funcţii : ***clrscr( ), getch( ), printf(...), scanf(...)*** care aparţin unor fişiere şi anume fişierelor **conio.h** şi **stdio.h**, motiv pentru care acestea (similar cu cazul fişierului **dos.h** din primul exemplu) trebuie să fie "incluse" în programul sursă, prin directivele

```
#include <conio.h>
#include <stdio.h>
```

plasate în faţa funcţiei **main( )**.

O nouă situaţie apare în exemplul nr. 3, unde se lucrează cu trei fişiere, în primul fiind funcţia **main( )** iar în celelalte două cele două funcţii care se utilizează. Pentru a face o "reunire" a celor trei fişiere, în fişierul care conţine funcţia **main( )** s-au introdus două directive de preprocesare

```
#include "inp_vect.cpp"
#include "out_vect.cpp"
```

care realizează acest lucru. În acest caz s-a presupus că aceste două fişiere se află în acelaşi director cu primul. Dacă fişierele se află în alţi directori atunci trebuie specificată calea.

O constatare valabilă pentru toate funcţiile utilizate este că acestea se compun din

- o **linie de definiţie** (antet), care conţine ***tipul valorii care se returnează*** de funcţie în urma execuţiei acesteia, un ***nume*** şi o **listă de argumente**,
- un **corp** al funcţiei, scris între două acolade.

4.2 Mediul de programare Borland C++ 3.1 pentru DOS

Toate firmele producătoare de software în domeniul limbajelor de programare au elaborat medii interactive de dezvoltare a programelor **IDE** (**I**ntegrated **D**evelopment **E**nvironment) pentru a se putea realiza într-un mod cât mai comod operaţiile de ***editare*** (crearea fişierului conţinând programul sursă), ***compilare*** (transformarea fişierelor sursă în fişiere obiect, simultan cu verificarea corectitudinii scrierii programului sursă), ***link-editare*** (combinarea unuia sau a mai multor module obiect într-un fişier executabil), ***depanare*** şi ***rulare*** (execuţie) a programelor, de asemenea existând şi posibilitatea de a obţine informaţii privind orice funcţie utilizată prin folosirea HELP-ului existent.

Lansarea în execuţie a mediului de programare Borland C++ 3.1 se face prin comanda:

bc Enter

iar lucrul efectiv cu acest mediu se va face prin intermediul celor ***11 menu-uri*** care sunt afişate pe ecran, în partea sa superioară:

≡ File Edit Search Run Compile Debug Project Options Window Help

fiecare din aceste componente ale menu-ului principal având submenu-uri.

În urma apăsării tastei F10 este activat acest menu iar trecerea de la o componentă la alta şi de asemenea la sub-menu-uri se face cu tastele săgeţi → ↑ ← ↓ urmată de apăsarea tastei Enter sau cu **mouse**-ul şi un clic pe componenta selectată (cu butonul din stânga).

Părăsirea menu-ului principal şi a sub-menu-urilor se face în urma apăsării tastei Esc.

Sub linia cu menu-uri există ***fereastra de editare***, în partea de jos a ecranului ***fereastra de dialog*** (conţine mesaje ca urmare a operaţiilor de compilare, etc.) şi ***linia de stare***, care corespunde operaţiei curente ce se execută. Dacă se face editarea programului sursă atunci această linie de stare are următorul conţinut:

F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

Autorii nu au intenţia să prezinte pe larg aceste probleme deoarece este evident faptul că orice programator va învăţa să utilizeze componentele menu-ului nu prin citirea unei cărţi ci lucrând efectiv cu acestea la calculator.

Prezentăm în continuare (pe scurt) acţiunile care se pot realiza prin utilizarea menu-urilor principale şi conţinutul acestora :

≡ (menu-ul SYSTEM) – conţine o serie de comenzi generale ale sistemului şi o listă de programe instalate cu comanda *Transfer* din menu-ul Options. Are următoarele componente:

Repaint desktop	**G**REP	**T**urbo Assembler	Turbo **D**ebbuger
Turbo **P**rofiler	**Re**source Compiler	**I**mport Librarian	

File (Alt+F) – crearea unui fişier nou, deschiderea unuia existent, salvarea unui fişier, ieşirea în DOS şi ieşirea din mediul integrat de programare

New	**O**pen F3	**S**ave F2	S**a**ve as	Save a**l**l
Change dir	**P**rint	**D**OS shell	**Q**uit Alt+X	

Edit (Alt+E) – conţine comenzi pentru mutarea unui bloc de text într-o zonă tampon, numită Clipboard, cu sau fără ştergerea lui din fereastra de editare, aducerea acestui bloc într-un alt loc în cadrul programului sursă, pentru anularea ultimei comenzi de editare, permite deschiderea unei ferestre Clipboard pentru a vizualiza sau edita conţinutul acesteia preluat din Message, Output sau Help.

Undo (Alt+BkSp) Redo (Shift+Alt+BkSp) Cut Shift+Delete
Copy Ctrl+Insert Paste Shift+Insert Clear (Ctrl+Del)
Copy example About … **S**how clipboard …

Search (Alt+S) - Conţine comenzi pentru căutarea unui text, a unor declaraţii de funcţii şi poziţia erorilor în fişierele sursă

Find … **R**eplace … **S**earch again (Ctrl+L)
Go to line number … Previous error (Alt+F7) Next error (Alt+F8)
Locate function …

Run (Alt+R) – Furnizează comenzi pentru realizarea execuţiei programului precum şi pentru începutul şi sfârşitul sesiunilor de depanare

Run Ctrl+F9 Program reset (Ctrl+F2) **G**o to cursor (F4)
Trace into (F7) **S**tep over (F8) **A**rguments …

Compile (Alt+C) – Comenzi pentru compilarea, link-editarea programului sursă din fereastra activă, crearea fişierului executabil.

Compile Alt+F9 **M**ake F9 **L**ink **B**uild all **I**nformation
Remove messages

Debug (Alt+D) – Comenzile existente permit depanarea programului

Inspect (Alt+F4) **E**valuate/Modify (Ctrl+F4) Call stack(Ctrl+F3)
Watches (**A**dd watch,**D**elete watch,**E**dit watch,**R**emove all watches) **T**oggle Breakpoints (Ctrl+F8)
Breakpoints (Delete, View, Clear all, Edit)

Project (Alt+P) – Conţine comenzi pentru realizarea unui proiect (program cu surse multiple) : crearea sau deschiderea, adăugarea sau ştergerea de fişiere, setarea unor opţiuni, etc.

Open project ... Close project Add item ... Delete item
Local options ... Include files ...

Options (Alt+O) – Comenzile din acest menu permit cunoaşterea setărilor existente precum şi setări noi, efectuate de utilizator. Trei dintre comenzi (**C**ompiler, **L**inker şi **E**nvironment) au sub-menu-uri iar celelalte duc la ferestre de dialog.

Application... **C**ompiler (**C**ode generation..., **A**dvanced code generation..., **E**ntry/Exit Code..., C**++** options..., Ad**v**anced C++ options, **O**ptimizations..., **S**ource..., **M**essages, **N**ames...)
Transfer **M**ake... **L**inker (**S**ettings..., **L**ibraries...)
Librarian... De**b**ugger... **D**irectories ...
Environment (**P**references..., **E**ditor..., **M**ouse..., **D**esktop..., **S**tartup..., **C**olors...)
Save...

Window (Alt+W) – Conţine comenzi pentru lucrul cu ferestre, deschidere şi închidere, precizarea mărimii, a poziţiei, etc.

Size/Move (Ctrl+F5)	**Z**oom F5	C**a**scade
Tile	Ne**x**t (F6)	**C**lose (Alt+F3)
Clos**e** all	**M**essage	**O**utput
Watch	**U**ser screen (Alt+F5)	**R**egister
Project	Project notes	**L**ist all (Alt+O)

Help (Alt+H) - Componentele acestui menu permit cunoaşterea unor informaţii referitoare la mediul de programare, la toate noţiunile folosite, la instrucţiunile limbajului, la funcţiile de bibliotecă.

Contents	**I**ndex Shift+F1	**T**opic search (Ctrl+F1)
Previous topic (Alt+F1)	**H**elp on help	Active **f**ile...
About...		

Este evident faptul că într-o primă fază, de învăţare a limbajului C/C++ şi de lucru cu mediul de programare, nu se vor utiliza decât o mică parte a componentelor submenu-urilor disponibile, o listă cu cele mai folosite fiind următoarea :

- New, Open, Save, Save all, Print, Quit (din File),
- Cut, Copy, Paste (din Edit),
- Run (din Run),
- Compile, Make (din Compile),
- Zoom (din Window),
- componentele de la Help.

În afara manualelor editate de firma Borland International, descrieri detaliate privind mediile de programare se pot găsi în lucrările [Căpr91] pg. 11-95 (**Borland Turbo C++ 2.0** – apărut în 1991), [Pope99] pg. 247-270, 284-288, [Swan] pg. 33-47 (**Borland C++3.1**) iar despre versiunea **Borland C++ 4.1** în [Bark98] pg. 31-89.

4.3 Directivele preprocesor din Borland C++ 3.1

În primul paragraf al acestui capitol în toate cele trei exemple de programe au fost folosite aşa numitele ***directive de preprocesare*** prin care în programul sursă s-au inclus o serie de alte fişiere, conţinând anumite funcţii care erau utilizate de către programator. Aceste includeri se fac **înainte de compilare** . S-au făcut includeri ale unor **fişiere antet** cu extensia **h** (**h**eader),

```
#include <dos.h>
#include <conio.h>
#include <stdio.h>
```

precum şi ale unor fişiere conţinând funcţii (programe sursă), elaborate de programator

```
#include "inp_vect.cpp"
#include "out_vect.cpp"
```

Utilizând astfel de includeri de fişiere se pot construi programe sursă practic oricât de mari folosind ***preprocesorul***. Acesta acţionează înainte de operaţia de compilare, urmărind liniile programului sursă care încep cu **#** (diez) şi efectuează operaţiile de **includere** (ca în exemplele anterioare) **definirea unor macrouri**, **substituiri** de şiruri de caractere şi **compilare condiţionată**.

Conform standardului **ANSI C** există următoarele **12** directive de preprocesare:

```
#define          #ifdef
#elif            #ifndef
#else            #include
#endif           #line
#error           #pragma
#if              #undef
```

În afară de directiva **#include**, cea mai folosită este directiva **#define,** forma sa generală fiind :

```
#define nume sir_caractere
```

ca urmare a acesteia, în programul sursă, începând de la instrucţiunea scrisă pe rândul următor, identificatorul **nume** se va înlocui cu **sir_caractere** (cu excepţia cazurilor când **nume** apare într-un şir de caractere sau într-un comentariu), de exemplu în secvenţa de instrucţiuni care urmează, peste tot unde apare **PI** acesta va fi înlocuit cu şirul de caractere corespunzător,

```
.....................
#define PI 3.1415926
.....................
u=sin(PI/6); v=cos(PI/6); w=tan(PI/6);
...........
```

Pentru a se renunţa la definirea făcută anterior există directiva de preprocesare

```
#undef nume
```

prin care se anulează definirea anterioară referitoare la identificatorul **nume**.

Astfel, dacă într-un rând al funcţiei din programului sursă ar apare

```
#undef PI
```

înseamnă că din acel loc identificatorul **PI** poate fi utilizat în alte scopuri, acesta având alte valori, conform atribuirilor care se vor face.

Cu directiva de preprocesare **#define** se pot defini şi **macrouri**, care au următoarea formă generală :

```
#define  nume_macrou(p1, p2, ... , pn)  text
```

de exemplu, pentru a realiza determinarea valorii absolute a unei variabile (expresii) se poate folosi următorul macrou :

```
#define ABS(a) ((a)>=0?(a):-(a))
```

aici folosindu-se operatorul condiţional ternar (vezi Capitolul VI) iar pentru a calcula radicalul din suma patratelor a două numere se poate folosi

```
#define IPOT(x,y)  (sqrt((x)*(x)+(y)*(y)))
```

În cazul compilării condiţionate se folosesc directivele :

```
#else   #endif   #if   #ifdef   #ifndef
```

Pentru a sista compilarea se utilizează directiva **#error**, directiva **#line** modifică conţinutul unor identificatori predefiniţi ai compilatorului iar directiva **#pragma** permite emiterea de diverse instrucţiuni către compilator.

4.4 Fişierele "header" şi conţinutul acestora

Un element specific limbajului C/C++ îl constitue utilizarea fişierelor header (fişiere antet), din care unele trebuie incluse în programul sursă. Aceste fişiere conţin fiecare din ele o serie de funcţii, mai mult sau mai puţin folosite, motiv pentru care din fişierele care se vor enumera numai la o parte se va prezenta şi conţinutul (lista funcţiilor componente).

În standardul **ANSI C** din anul **1990** au fost definite 15 fişiere antet iar în **1995** s-au mai adăugat trei, numărul ***fişierelor standardizate*** ajungând deci la **18** . La acestea s-au mai adăugat foarte multe fişiere nestandardizate, depinzând de firma care produce compilatorul.

Fişierele standardizate sunt următoarele :

assert.h , ***ctype.h*** , ***errno.h*** , ***float.h*** , ***iso646.h*** , ***limits.h*** , ***locale.h*** , ***math.h*** , ***setjmp.h*** , ***signal.h*** , ***stdarg.h*** , ***stddef.h*** , ***stdio.h*** , ***stdlib.h*** , ***string.h*** , ***time.h*** , ***wchar.h*** şi ***wctype.h*** .

În continuare se prezintă lista fişierelor antet şi conţinutul acestora, în cazul fişierelor standardizate precum şi a celor mai frecvent folosite :

alloc.h – conţine funcţii pentru gestiunea memoriei,

assert.h (ANSI C) – defineşte macrocomanda assert şi conţine fişierele **stdio.h** şi **stdlib.h**,

bcd.h – declară clasa **bcd** din C++

bios.h – funcţii specifice BIOS-ului

_bios_disk	_bios_equip	_bios_keybrd	_bios_memsize
_bios_printer	_bios_serialcom	_bios_timeofday	bioscom
biosdisk	diosequip	bioskey	biosmemory
biosprint	biostime		

complex.h – funcţii matematice complexe,

abs	acos	arg	asin	atan	conj	cos	cosh	exp
imag	log	log10	norm	polar	pow	pow10	real	sin
sinh	sqrt	tan	tanh					

conio.h - funcţii nestandard de I/O

cgets	clreol	clrscr	cprintf
cputs	cscanf	delline	getch
getche	getpass	gettext	gettextinfo
gotoxy	highvideo	insline	inp
inport	inportb	inpw	kbhit

lowvideo	movetext	normvideo	outp
outport	outportb	outpw	putch
puttext	_setcursortype	textattr	textbackground
textcolor	textmode	ungetch	wherex
wherey	window		

ctype.h (ANSI C) – funcţii şi macrouri pentru conversia caracterelor,

_ftolower	_ftoupper	isalnum	isalpha	isascii	iscntrl
isdigit	isgraph	islower	isprint	ispunct	isspace
isupper	isxdigit	toascii	tolower	toupper	

dir.h - conţine structuri, macrouri şi funcţii pentru lucrul cu directoare,

direct.h - conţine structuri, macrouri şi funcţii pentru lucrul cu directoare

dirent.h – declară funcţii şi structuri pentru operaţii cu directoare,

dos.h – constante şi funcţii sub DOS

absread	abswrite	allocmen	bdos
bdosptr	chain_intr	country	ctrlbrk
delay	disable	_dos_allocmem	_dos_close
_dos_creat	_dos_creatnew	_dosexterr	_dos_findfirst
_dos_findnext	_dos_freemem	_dos_getdate	_dos_getdiskfree
_dos_getdrive	_dos_getfileattr	_dos_getftime	_dos_gettime
_dos_getvect	_dos_keep	_dos_open	_dos_read
_dos_setblock	_dos_setdate	_dos_setdrive	_dos_setfileattr
_dos_setftime	_dos_settime	_dos_setvect	_dostounix
_dos_write	_emit_	enable	FP_OFF
FP_SEG	freemem	geninterrupt	getcbrk
getdate	getdfree	getdrive	getdta
getfat	getfatd	getftime	getpsp
gettime	getvect	getverify	_harderr
harderr	_hardresume	hardresume	_hardretn
hardretn	inp	inport	inportb
int86	int86x	intdos	intdosx
intr	keep	MK_FP	nosound
outp	outport	outportb	_OvrInitEms
_OvrInitExt	parsfnm	peek	peekb
poke	pokeb	randbrd	randbwr
segread	setblock	setcbrk	setdate
setdta	settime	setvect	setverify
sleep	sound	unixtodos	unlink

errno.h (ANSI C) – permite semnalarea erorilor date de funcţiile de bibliotecă şi conţine următoarele patru macrouri :

EDOM EILSEQ ERANGE errno

fcntl.h (17) - defineşte constante simbolice,

float.h (ANSI C) – defineşte parametrii reprezentărilor în virgulă mobilă, conţine 29 de macrouri.

iso646.h (ANSI C) - defineşte alte modalităţi de scriere a unor operatori,

fstream.h – defineşte stream-uri pentru operaţii de I/O cu fişiere,

generic.h – conţine macrouri pentru declaraţii generice de clase

graphics.h – conţine funcţii pentru grafică

arc	bar	bar3d
circle	cleardevice	clearviewpoint
closegraph	detectgraph	drawpoly
ellipse	fillellipse	fillpoly
floodfill	getarccoords	getaspectratio
getbkcolor	getcolor	getdefaultpalette
getdrivername	getfillpattern	getfillsettings
getgraphmode	getimage	getlinesettings
getmaxcolor	getmaxmode	getmaxx
getmaxy	getmodename	getmoderange
getpalette	getpalettesize	getpixel
gettextsettings	getviewsettings	getx
gety	graphdefaults	grapherrormsg
_graphfreemem	_graphgetmem	graphresult
imagesize	initgraph	installuserdriver
installuserfont	line	linerel
lineto	moverel	moveto
outtext	outtextxy	pieslice
putimage	putpixel	rectangle
registerbgidriver	registerfarbgdriver	registerbgifont
registerfargifont	restorecrtmode	sector
setactivepage	setallpalette	setaspectratio
setbkcolor	setcolor	setfillpattern
setfillstyle	setgraphbufsize	setgraphmode
setlinestyle	setpalette	setrgbpalette
settextjustify	settextstyle	setusercharsize
setviewpoint	setvisualpage	setwritemode
textheight	textwidth	

io.h – conţine structuri şi declaraţii pentru operaţii de I/O,

iomanip.h – declară manipulatori de stream-uri de I/O,

iostream.h (8 classes) – se declară rutine de I/O proprii limbajului C++

ios	iostream	iostream	_withassign
istream	istream	_withassign	ostream
ostream	_withassign	streambuf	

limits.h (ANSI C) – defineşte diferite limite pentru valori de tip întreg,

char_bit	char_max	char_min	int_max	int_min
long_max	long_min	mb_len_max	schar_max	schar_min
shrt_max	shrt_min	uchar_max	uint_max	ulong_max
ushrt_max				

locale.h (ANSI C) – asigură localizarea, adaptarea la specificul local,

malloc.h – funcţii pentru gestiunea memoriei,

math.h (ANSI C) – funcţii matematice,

abs	…	acos	acosl	asin	asinl
atan	atanl	atan2	atan2l	atof	_atold
cabs	cabsl	ceil	ceill	cos	cosl
cosh	coshl	exp	expl	fabs	fabsl
floor	floorl	fmod	fmodl	frexp	frexpl
hypot	hypotl	labs	…	ldexp	ldexpl
log	logl	log10	log10l	matherr	_matherrl
modf	modfl	poly	polyl	pow	powl
pow10	pow10l	sin	sinl	sinh	sinhl
sqrt	sqrtl	tan	tanl	tanh	tanhl

mem.h – funcţii pentru gestiunea memoriei,

memory.h – funcţii pentru manipularea memoriei,

new.h – acces la operatorul new

process.h – conţine structuri şi declaraţii

search.h – funcţii pentru căutare şi sortare

setjmp.h (ANSI C) – asigură salturi

jmp_buf	longjmp	setjmp

share.h – defineşte parametri utilizaţi în funcţii care folosesc file-sharing.

signal.h (ANSI C) – asigură manipularea semnalelor

raise	signal

stdarg.h (ANSI C) – asigură liste de argumente cu număr variabil,

va_arg	va_end	va_start

stddef.h (ANSI C) – defineşte constante şi macrouri frecvent folosite

NULL	offsetof	ptrdiff_t	size_t	wchar_t

stdio.h (ANSI C) – defineşte funcţii şi macrouri (utilizate şi în versiunea Kernighan & Ritchie) pentru realizarea operaţiilor de I/O,

clearerr	fclose	fcloseall	fdopen	feof	ferror
fflush	fgetc	fgetchar	fgetpos	fgets	fileno
flushall	fopen	fprintf	fputc	fputchar	fputs
fread	freopen	fscanf	fseek	fsetpos	ftell
fwrite	getc	getchar	gets	getw	perror
printf	putc	putchar	puts	putw	remove
rename	rewind	rmtmp	scanf	setbuf	setvbuf
sprintf	sscanf	strerror	_strerror	tempnam	tmpfile
tmpnam	ungetc	unlink	vfprintf	vfscanf	vprintf
vscanf	vsprintf	vsscanf			

stdiostr.h - declară clase de stream-uri utilizate în cadrul unor structuri,

stdlib.h (ANSI C) – se definesc funcţii uzuale, pentru conversii, sortări, etc.

abort	abs	atexit	atof	atoi
atol	bsearch	calloc	div	ecvt
exit	_exit	fcvt	free	_fullpath
gcvt	getenv	itoa	labs	ldiv
lfind	_lrotl	_lrotr	lseach	ltoa
_makepath	malloc	max	mblen	mbtowc
mbstowcs	min	putenv	qsort	rand
random	randomize	realloc	_rotl	_rotr
_searchenv	_splitpath	srand	strtod	strtol
_strtold	strtoul	swab	system	time
ultoa	wctomb	wcstombs		

string.h (ANSI C) – funcţii pentru prelucrarea şirurilor

_fmemccpy	_fmemchr	_fmemcmp	_fmemcpy	_fmemicmp
_fmemset	_fstrcat	_fstrchr	_fstrcmp	_fstrcpy
_fstrcspn	_fstrdup	_fstricmp	_fstrlen	_fstrlwr
_fstrncat	_fstrncmp	_fstrnicmp	_fstrncpy	_fstrnset
_fstrpbrk	_fstrrchr	_fstrrev	_fstrset	_fstrspn
_fstrstr	_fstrtok	_fstrupr	memccpy	memchr
memcmp	memcpy	memicmp	memmove	memset
movedata	movmem	setmem	stpcpy	strcat
strchr	strcmp	strcmpi	strcpy	strcspn
strdup	_strerror	strerror	stricmp	strlen
strlwr	strncat	strncmp	strncmpi	strncpy
strnicmp	strnset	strpbrk	strrchr	strrev
strset	strspn	strstr	strtok	strxfrm
strupr				

strstrea.h – declară clase de stream-uri utilizate în cazul tablourilor de octeţi din memorie,

sys\locking.h – definiţii legate de parametrii unor funcţii de închidere,

sys\stat.h – defineşte constante simbolice utilizate în deschiderea şi crearea fişierelor,

sys\timeb.h – defineşte funcţia ftime şi structura timeb

sys\types.h – declară tipul time_t utilizat cu funcţii de timp,

time.h (ANSI C) – funcţii de timp pentru sistem,

asctime	clock	ctime	difftime	gmtime
localtime	mktime	stime	strftime	_strdate
_strtime	time	tzset		

utime.h – declară funcţiile utime şi structura utimbuf,

values.h – se definesc constante

varargs.h – definiri de macrouri

wchar.h (ANSI C) – funcţii pentru operaţii de I/O cu "wide streams"

wctype.h (ANSI C) – funcţii pentru operaţii şi prelucrarea "wide characters"

În majoritatea programelor care se vor scrie se vor folosi în special următoarele fişiere header : **conio.h** (funcţiile *clrscr, getch, getche*, cele pentru lucrul în mod text – vezi Capitolul XIV), **dos.h** (funcţiile *delay, nosound, sound*), **graphics** (foarte multe funcţii – vezi Capitolul XV), **iostream.h** (*cin, cout* ş. a.), **math.h** (toate funcţiile uzuale necesare în calculele matematice), **stdio.h** (funcţiile *gets, printf, fprintf, puts, scanf, fscanf* ş. a.), **stdlib.h** (funcţiile *exit, rand, random, time* ş. a.), **string.h** (funcţiile *strcat, strcmp, strcpy, strlen* ş. a.), **time.h** (funcţia *time*)

Astfel, în programul următor, cu care se calculează coordonatele carteziene ale punctelor unei evolvente de cerc, s-au utilizat funcţii care aparţin la patru fişiere header : **conio.h**, **dos.h**, **math.h** şi **stdio.h** precizându-se în cazul fiecărei funcţii, prin comentarii, cărui fişier îi aparţine.

```
/*   The computing of an involute coordinates
     x = r * [ cos(phi) + phi * sin(phi) ]
     y = r * [ sin(phi) - phi * cos(phi) ]
   An involute of a circle is a plane curve generated
   by a point on a tangent which rolls on the circle
   without slipping, or by a point on a taut string
   which is unwrapped from a reel.                  */

#include <conio.h>
#include <dos.h>
#include <math.h>
#include <stdio.h>

#define PI 3.1415926

main()
{ int i, np;
  float c, phi_fin, phid, phir, rb, s, stepd, x, y;
  printf("\n The radius of base circle     [mm]   = ");// stdio.h
  scanf("%f", &rb);                                    // stdio.h
  printf("\n The final value of angle phi (< 285) = ");
  scanf("%f", &phi_fin);
  printf("\n The number of calc. points (< 500 )  = ");
  scanf("%d", &np);
  getch();                              // conio.h
  stepd=phi_fin/(np-1);
  for(i=0; i<np; i++)
     { phid=i*stepd; phir=phid*PI/180.0;
       s=sin(phir); c=cos(phir);        // math.h
       x=rb*(c+phir*s); y=rb*(s-phir*c);
       printf("\n  %5d   %7.3f   %12.5e   %12.3e",i,phid,x,y);
       delay(100);                      // dos.h
     }
  getch();                              // conio.h
}
```

4.5 Fazele prelucrării unui program

Orice mediu integrat de programare asigură parcurgerea tuturor etapelor de realizare a unui program, începând cu **editarea** acestuia (singurul lucru pe care trebuie să-l facă utilizatorul, folosind reguli mult mai simple decât la editoarele profesionale de texte cu care este obişnuit), rezultând un fişier **nume.cpp**, un fişier de tip text ASCII, urmat de faze ce vor fi realizate în mod automat, acestea fiind :

- **preprocesarea** şi **compilarea**,
- **link-editarea** şi **execuţia,**

care sunt iniţiate în urma unor comenzi adresate mediului de programare C/C++.

Anterior s-au precizat funcţiile preprocesorului, acestea fiind : includeri de fişiere, sursă sau header, definirea unor macrouri, substituiri de nume cu şiruri de caractere precum şi comenzi pentru realizarea unei compilări condiţionate.

În timpul fazei de compilare se face o verificare a sintaxei programului sursă şi se crează un modul obiect, un fişier cu acelaşi nume, având extensia **.obj** , fiecare instrucţiune din programul sursă fiind interpretată, generându-se o succesiune de instrucţiuni în cod maşină. Va rezulta un fişier **nume.obj** .

În timpul compilării se utilizează (din punct de vedere logic) un număr de **patru zone de memorie** :

- pentru înregistrarea codului programului,
- pentru înregistrarea valorilor variabilelor globale (vezi §10.4),
- o "**stivă**" (stack – stivă), pentru înregistrarea valorilor argumentelor funcţiilor, a variabilelor locale (vezi §10.4), a unor adrese,
- o zonă de manevră ("**heap**") care este o zonă de memorie liberă.

În faza de link-editare modulul obiect care rezultă în urma compilării trebuie "completat" cu modulele obiect ale funcţiilor standard, a celor create de utilizator şi a altor module obiect iar ca urmare a acţiunii link-editorului rezultă **programul executabil**, care este un fişier cu extensia **.exe** deci fişierul **nume.exe**.

$$\left\{\begin{array}{c} Fisierul\ sursa \\ nume.cpp \\ + \\ Bibl.\ de\ functii \\ a\ compilatorului \end{array}\right\} \begin{array}{c} compilator \\ \Rightarrow \end{array} \left\{\begin{array}{c} Fisierul\ obiect \\ nume.obj \\ + \\ Bibl.\ de\ functii \\ a\ link-editorului \end{array}\right\} \begin{array}{c} link-editor \\ \Rightarrow \end{array} \left\{\begin{array}{c} Fisierul \\ executabil \\ nume.exe \end{array}\right\}$$

Capitolul V

Unităţile lexicale ale limbajului C

În limbajul **C** există **şase tipuri** de unităţi lexicale : **identificatori**, **cuvinte-cheie** (proprii limbajului), **constante** (de diverse tipuri), **şiruri**, **operatori** şi **separatori**. Acest limbaj este deosebit de bogat mai ales în ceea ce priveşte numărul de operatori disponibili, dintre care unii sunt utilizaţi pentru a face diferite operaţii la nivel de bit.

5.1 Setul de caractere

În cadrul funcţiilor care compun programele sursă C/C++ se pot folosi următoarele tipuri de caractere:

- **52 litere, mari şi mici** (ale alfabetului englez)

```
A  B  C  D  E  F  G  H  I  J  K  L  M
N  O  P  Q  R  S  T  U  V  W  X  Y  Z
a  b  c  d  e  f  g  h  i  j  k  l  m
n  o  p  q  r  s  t  u  v  w  x  y  z
```

- cele **10 cifre**

```
0  1  2  3  4  5  6  7  8  9
```

- **simboluri speciale** (30)

```
!  blanc  "   #   %   &   '   (   )   *

+   ,   -   .   /   :   ;   <   =   >

?   [   \   ]   ^   _   ~   {   |   }
```

- simboluri negrafice sau **caractere de control** (8)

```
\0  \a  \b  \t  \n  \v  \f  \r
```

În Anexa 1 din Capitolul XVII se prezintă setul celor 256 caractere **ASCII** (setul extins de caractere), din care în cadrul limbajului de programare C/C++ se utilizează o parte din primele 128 de caractere.

Fiecare caracter se reprezintă pe un **octet** (**byte** în l. engleză), compus din opt biţi de memorie (**bit** – **bi**nary digi**t**) şi se exprimă de obicei fie printr-un număr zecimal fie hexazecimal.

Caracterele litere mari şi mici şi cele 10 cifre arabe au următorul mod de reprezentare în codul ASCII:

		(hexa.)	(zec.)			(hexa.)	(zec.)
A	0100 0001	41	65	**a**	0110 0001	61	97
B	0100 0010	42	66	**b**	0110 0010	62	98
C	0100 0011	43	67	**c**	0110 0011	63	99
.	....	..	..	.	....	..	..
Y	0101 1001	59	89	**y**	0111 1001	79	121
Z	0101 1010	5A	90	**z**	0111 1010	7A	122

		(hexa.)	(zec.)
0	0011 0000	30	48
1	0011 0001	31	49
2	0011 0010	32	50
.	....	..	..
8	0011 1000	38	56
9	0011 1001	39	57

O parte din caracterele limbajului C/C++ care au reprezentări în primele 32 de poziţii ale codului ASCII se numesc ***caractere de control***. Acestea sunt următoarele:

Caracter	Cod hexa.	Cod zec.	Nume	Descriere
\0	00	0	NUL	null character
\a	07	7	BEL	audible alert
\b	08	8	BS	backspace
\t	09	9	HT	horizontal tab
\n	0A	10	NL	new line
\v	0B	11	VT	vertical tab
\f	0C	12	FF	form feed
\r	0D	13	CR	carriage return

Modul cum sunt reprezentate celelalte 30 de caractere speciale (semne matematice, de punctuaţie, etc) se poate vedea în Anexa 1.

Trebuie remarcat faptul că reprezentarea ASCII a caracterului **blank** (spaţiu liber) este $0010\ 0000_2 = 20_{16} = 32_{10}$, acesta fiind înaintea literelor, cifrelor şi semnelor de punctuaţie, motiv pentru care acest caracter va apărea la începutul listelor sortate alfabetic.

5.2 Identificatori

Într-un program sursă C/C++ se folosesc identificatori pentru a denumi diverse variabile, funcţii, tipuri de date care se utilizează în respectivul program.

Un identificator este o secvenţă de caractere care conţine litere, cifre şi semnul _

Numărul caracterelor dintr-un identificator nu este limitat dar de obicei numai primele opt sunt semnificative, acest lucru depinzând de compilator.

Exemple de identificatori :
- int double - definesc *tipul datelor*
- main printf scanf getch - *denumiri de funcţii*
- do while for - *denumiri de instrucţiuni*
- a_b sum35 produs07 - *denumiri* date unor *variabile*

În limbajul C/C++ un identificator poate începe fie cu o literă fie cu semnul _, de exemplu: aaa AAA _a22 , în compunerea sa intrând litere , cifre sau/şi semnul _

Un alt lucru deosebit de important se referă la faptul că tipul literei (mare sau mică) utilizate în scrierea identificatorului este luat în considerare de compilator, motiv pentru care identificatorii

aaaa aAaa AAAA AAAa Aaaa

vor putea reprezenta diferite variabile în cadrul funcţiei în care vor fi utilizaţi. Referitor la acest lucru se poate spune că în cazul limbajului C/C++ acesta este **"case sensitive"** .

De asemenea trebuie reţinut faptul că toţi identificatorii care reprezintă denumiri de funcţii ale limbajului precum şi cei care se utilizează pentru denumirea unor instrucţiuni se vor scrie utilizând **litere mici** (şi eventual caracterul _)

Identificatorii cu care se notează diferitele variabile în cadrul funcţiilor trebuie să fie diferiţi de cuvintele-cheie şi de denumirile funcţiilor din biblioteca limbajului.

În standardul ANSI C nu se precizează un număr maxim de caractere ale unui identificator. În cazul variabilelor globale (vezi Capitolul X) numărul de caractere este limitat la şase iar în cazul unor variabile obişnuite se pot folosi până la 32 de caractere, dar este de preferat folosirea unor identificatori mai simpli.

5.3 Cuvinte-cheie ("keywords")

În limbajul **ANSI C** se utilizează un număr de **32** de cuvinte cheie (scrise cu caractere îngroşate), acest set de cuvinte cheie extinzându-se la un număr de **83** în cazul limbajului C++ . Cuvintele cheie sunt următoarele :

asm	_asm	__asm	**auto**
break	**case**	cdecl	_cdecl
__cdecl	**char**	class	**const**
continue	_cs	__cs	**default**
delete	**do**	**double**	_ds
__ds	**else**	**enum**	_es
__es	_export	__export	**extern**
far	_far	__far	_fastcall
__fastcall	**float**	**for**	friend
goto	huge	_huge	__huge
if	inline	**int**	interrupt
_interrupt	__interrupt	_loadds	__loadds
long	near	_near	__near
new	operator	pascal	_pascal

__pascal	private	protected	public
register	**return**	_saveregs	__saveregs
_seg	__seg	**short**	**signed**
sizeof	_ss	__ss	**static**
struct	**switch**	template	this
typedef	**union**	**unsigned**	virtual
void	**volatile**	**while**	

Aceste cuvinte-cheie sunt identificatori proprii limbajului şi pot fi utilizaţi numai conform regulilor sintactice ale acestuia. După cum s-a precizat anterior cuvintele-cheie se vor scrie numai cu litere mici şi eventual cu caracterul _ .

De asemenea cuvântul **main** (numele dat funcţiei principale) nu va fi utilizat decât în scopul respectiv.

5.4 Tipuri principale de date

În limbajul **C** există **cinci tipuri** principale de date : **char** (caracter), **int** (întregi) , **float** (reale în simplă precizie), **double** (reale în dublă precizie) şi **void** (fără valoare) .

Modul cum se reprezintă în memorie, valorile minime şi maxime, sunt prezentate în tabelul următor.

În ceea ce priveşte tipul **void** acesta se foloseşte pentru a declara funcţii care nu returnează nici o valoare.

Tipurile de date **int** , **char** , **float** , **double** pot fi precedate de **modificatori** (**signed** , **unsigned** , **short** , **long**) , rolul acestora fiind de a specifica dimensiunea zonei de memorie pe care se face înregistrarea valorilor.

După cum se poate observa din tabel, în cazul tipului **int** valoarea numerică întreagă se înregistrează pe doi octeţi (16 biţi) iar dacă se foloseşte modificatorul **long** atunci se vor utiliza patru octeţi (32 de biţi), în consecinţă va putea creşte valoarea maximă înregistrabilă în memorie, de la **32767** = $2^{15} - 1$ la **2147483647** = $2^{31} - 1$.

În cazul tipurilor de date prezentate, în afară de numărul de octeţi necesar pentru înregistrarea în memorie şi valorile minime/maxime, mai este necesar să se cunoască şi precizia de înregistrare. Astfel, în cazul valorilor întregi toate cifrele sunt exacte, în cazul celor de tip **float** primele 6 - 7 cifre semnificative sunt exacte iar la cele de tip **double** primele 14 - 16 cifre semnificative au valori exacte.

În ceea ce priveşte modalitatea de reprezentare în memorie, indiferent de faptul că data este de tip **int**, **float** sau **double**, primul bit este cel care reţine semnul (0 dacă numărul este pozitiv respectiv 1 dacă numărul este negativ). Datele de tip **float** au exponentul reprezentat pe 8 biţi iar mantisa pe 23 de biţi iar în cazul celor de tip **double** exponentul pe 11 biţi iar mantisa pe 52 de biţi.

Tip	Octeţi (Bytes)	Mod de reprezentare	Intervalul de valori
unsigned char	1	În codul ASCII	[0 ; 255]
char	1	Întreg reprezentat prin complement faţă de 2	[- 128 ; 127]
int	2		[**- 32,768** ; **32,767**]
long int	4		[**- 2,147,483,648** ; **2,147,483,647**]
__int64 (specific Microsoft)	8		[- 9,223,372,036,854,775,808 ; 9,223,372,036,854,775,807]
unsigned int	2	Întreg fără semn	[0 ; 65,535]
unsigned long	4		[0 ; 4,294,967,295]
float	4	Reprezentare în virgulă mobilă în simplă precizie	($\mathbf{-3.4*10^{38}}$; $-3.4*10^{-38}$) **»** {0 } **»** ($3.4*10^{-38}$; $\mathbf{3.4*10^{38}}$)
double	8	Reprezentare în virgulă mobilă în dublă precizie	($\mathbf{-1.7*10^{308}}$; $-1.7*10^{-308}$) **»** {0 } **»** ($1.7*10^{-308}$; $\mathbf{1.7*10^{308}}$)
long double	10		($\mathbf{-1.1*10^{4932}}$; $-3.4*10^{-4932}$) **»** {0 } **»** ($3.4*10^{-4932}$; $\mathbf{1.1*10^{4932}}$)

5.5 Constante

Constantele în limbajul de programare C/C++ sunt ***numere***, ***caractere*** sau ***şiruri de caractere***, **tipul** acestora rezultând din modul în care sunt scrise iar **valoarea** lor nu se modifică în timpul execuţiei programului.

Se poate face următoarea clasificare a constantelor :

- constante **întregi** – scrise în sistemele de numeraţie **zecimal**, **octal** şi **hexazecimal**,
- constante **reale** (scrise numai în sistemul de numeraţie zecimal),
- constante **caracter**,
- constante **şir de caractere**,
- constante **simbolice**

Constantele întregi pot fi scrise în cele trei sisteme de numeraţie: zecimal, octal şi/sau hexazecimal, în primul caz utilizându-se modul obişnuit de scriere iar în celelalte două cazuri se va utiliza cifra **0** înaintea constantei (în scrierea căreia se vor folosi cifrele 0,1, ..., 7) respectiv cifra **0** urmată de **x** sau **X** în cazul celor hexazecimale (utilizând cifrele 0, 1, 2, ..., 8, 9, A, B, .., F), de exemplu,

12 sau **0**14 sau **0x**C

127 sau **0**177 sau **0X**7F

Valoarea oricărei constante întregi exprimate în zecimal se poate afişa în celelalte două sisteme de numeraţie, octal şi hexazecimal (vezi Capitolul VII).

Tipul implicit al acestor constante poate fi : **int** , **long int** , **unsigned long int**, funcţie de valoarea pe care o au. Astfel constanta **20000** va fi de tip **int** , constanta **45000** de tip **long int** (deoarece este mai mare decât 32767) iar constanta **3000000000** va fi de tipul **unsigned long int.**

Este posibil să se declare în mod explicit că o constantă va fi de tip **long** sau **unsigned** chiar dacă acest lucru nu este necesar din punctul de vedere al valorii acesteia, prin adăugarea literelor **l** sau **L** respectiv **u** sau **U** , de exemplu : 12**L** va fi o constantă care se va înregistra pe patru octeţi, cu toate că nu sunt necesari decât doi octeţi pentru aceasta.

Constantele reale (în virgulă mobilă) sunt de tip **double**, deci se înregistrează pe opt octeţi. Se pot scrie în următoarele feluri :

123.902 9.23 .54 -34.7609 (forma naturală de scriere)

1.23902e+2 0.00923E3 5400.0e-4 -3.47609e1 (forma exponenţială de scriere)

Se poate observa că se va folosi caracterul **.** (punct) pentru a separa partea întreagă de cea zecimală, numerele sunt scrise fără spaţii între cifre, prin convenţie în cazul scrierii exponenţiale se consideră că numărul care urmează după caracterul **e** (sau **E**), cu semnul său, reprezintă exponentul lui 10, cu care se înmulţeşte numărul zecimal anterior scris, deci

$$0.00923E3 = 0.00923 \cdot 10^3$$

Şi în cazul acestor constante (al căror tip implicit este **double**) se poate schimba tipul, acesta devenind **float** sau **long double**, prin adăugarea literelor **f** sau **F** respectiv **l** sau **L**, astfel dacă se scrie

45.9**f** -234.89**L**

prima constantă se va înregistra pe patru octeţi (în loc de opt) iar a doua pe zece octeţi.

Constante caracter. Prin convenţie acestea se scriu între două apostroafe, care mărginesc fie o literă, o cifră sau un semn de punctuaţie ,

'a' 'S' '2' '-'

valoarea lor numerică fiind cea corespunzătoare codurilor ASCII, astfel constanta caracter **'2'** are valoarea 50 .

După cum se va vedea în continuare, în instrucţiuni notaţiile 'A' şi 65 vor fi echivalente deoarece 65 este valoarea în zecimal a constantei caracter A.

Tot din categoria constantelor caracter fac parte şi aşa numitele secvenţe de evitare (ESC-ape) utilizate pentru reprezentarea unor caractere cum ar fi apostroful, ghilimelele, backslash-ul,

'\'' '\"' '\\'

în care, înaintea caracterului respectiv, se scrie caracterul **** (backslash).

Caracterele negrafice **\a** (BEL-audible alert), **\b** (BS-backspace), **\t** (HT-horizontal tab), **\n** (NL-new line), **\v** (VT-vertical tab), **\f** (FF-form feed), **\r** (CR-carriage return) se reprezintă de asemenea prin secvenţe ESC-ape:

```
'\a'  '\b'  '\t'  '\n'  '\v'  '\f'  '\r'
```

o altă secvenţa ESC-ape extrem de folosită fiind caracterul nul, scris sub forma **'\0'** .

Pentru a putea folosi şi caracterele care aparţin codului ASCII extins se pot folosi aşa numitele "numeric escape sequences" :

'\123' '\237' '\x8A' '\xFF'

după backslash scriindu-se fie un număr octal, fie un număr hexazecimal

Şiruri de caractere. Un şir de caractere este o secvenţă de litere, cifre şi alte simboluri, scrise între ghilimele,

```
"AaA"   "Universitatea Tehnica"   "s"
```

fiecare caracter al şirurilor fiind reprezentat pe câte un octet iar la sfârşitul acestuia, deci pe ultimul octet, se înregistrează caracterul nul **'\0'** (cu valoare zero conform codului ASCII).

În consecinţă, şirul **"AaA"**, format din **trei** caractere se va înregistra pe **patru** octeţi, sub forma

0100 0001 0110 0001 0100 0001 0000 0000 (binar)
4 1 6 1 4 1 0 0 (hexazecimal)

iar şirul **"s"** , format dintr-un **singur** caracter se va înregistra pe **doi** octeţi, sub forma **73** 00 (în hexazecimal).

În consecinţă, din punct de vedere al reprezentării interne nu este acelaşi lucru **'s'** şi **"s"** deoarece constanta caracter **'s'** se reprezintă pe un octet pe când şirul **"s"** se reprezintă pe doi octeţi, cel de-al doilea octet fiind ocupat de caracterul nul **'\0'** .

Caracterul nul **'\0'** este plasat automat la sfârşitul şirurilor pentru ca în timpul execuţiei programelor să poată fi detectat sfârşitul şirurilor .

În cazul în care într-un şir trebuie să intervină anumite caractere cum ar fi ghilimele, apostroafe, backslash-uri acestea sunt precedate de caracterul backslash, după cum se vede în următoarele exemple :

Şirul de caractere	Scrierea în C/C++
Sala **"**Nicolae Iorga**"** are 250 de locuri	"Sala **\"**Nicolae Iorga**\"** are 250 de locuri"
Caracterul **** este numit backslash	"Caracterul **** este numit backslash"

Dacă un şir de caractere care trebuie afişat pe ecran este lung, de exemplu şirul

Universitatea Tehnica din Cluj-Napoca, Catedra de Mecanica si Programare

acesta poate fi scris ca o succesiune de două sau mai multe şiruri, următoarele funcţii de afişare având acelaşi efect

```
printf( "Universitatea Tehnica din Cluj-Napoca, Catedra de Mecanica si Programare");
```

```
printf("Universitatea Tehnica din Cluj-Napoca,"
       " Catedra de Mecanica si Programare");

printf("Universitatea Tehnica din "
       "Cluj-Napoca, Catedra de "
       "Mecanica si Programare");
```

Funcţia **printf(...)** va fi prezentată în Capitolul VII – Funcţii de I/O. Prin folosirea caracterului **\n** în cadrul unui şir care se afişează acesta va apare pe linii succesive. Astfel dacă se doreşte afişarea următorului text (începutul Sonetului 55 al lui W. Shakespeare) :

Not marble, nor the gilded monument,
Of princes, shall outlive this powerful rhyme

cu ajutorul funcţiei **printf(...)**, şirul se va scrie sub forma :

```
printf("\n\t Not marble, nor the gilded monument,\n\t Of princes, "
       "shall outlive this powerful rhyme");
```

Caracterul **\t** s-a inclus deoarece s-a dorit afişarea fiecărui rând lăsând spaţii de la marginea din stânga.

O altă regulă privind scrierea şirurilor într-un program C/C++ se referă la modul cum se face continuarea unui şir pe rândul următor: rândul anterior se termină cu un caracter **** şi se continuă scrierea şirului pe rândul următor, astfel funcţia de afişare

```
printf( "Universitatea Tehnica din Cluj-Napoca, Catedra de Mecanica si Programare");
```

se mai poate scrie şi în felul următor :

```
printf( "Universitatea Tehnica din Cluj-Napoca, Catedra de \
Mecanica si Programare");
```

În cazul în care se doreşte ca într-un şir să se introducă caractere aparţinând codului ASCII extins acestea se vor scrie ca secvenţe escape numerice. Astfel, cu programul care se prezintă în continuare

```
#include <conio.h>
#include <stdio.h>

int main(void)
{clrscr();
 printf("\n Technische Universit\204t Braunschweig\n");
 printf("\n K\224ln   M\201nchen\n");
 printf("\n Bruel&Kj\221r\n");
 printf("\n \343 = 3.1415926535\n");
 printf("\n sin\375\340 + cos\375\340 = 1\n");
getch(); return 0;
}
```

se realizează afişarea următoarelor :

```
Technische Universität Braunschweig
Köln    München
Bruel&Kjær
```

p = 3.1415926535
$\sin^2\alpha + \cos^2\alpha = 1$

în care există şi astfel de caractere. După cum se poate vedea, în cadrul şirurilor care apar în cadrul funcţiilor **printf(....)** s-au scris astfel de secvenţe :

Caracterul care se afişează	Secvenţa escape numerică
ä	\204
ö	\224
ü	\201
æ	\221
p	\343
^{2}(ridicare la patrat)	\375
α	\340

Constante simbolice. Prin construcţii de tipul

```
#define nume succesiune_de_caractere
```

există posibilitatea de a defini identificatori cu valori de constante, caz în care aceşti identificatori se numesc **constante simbolice**.

Astfel, în cazul

```
#define E 2.718281828
#define PI 3.141592654
#define SQRT_2 1.414213562
```

constantele simbolice **E , PI** şi **SQRT_2** vor fi înlocuite, peste tot unde apar în programul sursă, cu **2.718281828** , **3.141592654** respectiv **1.414213562** , deci în acest caz cu valori numerice.

Această înlocuire nu se face dacă constanta simbolică apare în compunerea unui şir sau într-un identificator mai lung. Astfel, ca urmare a execuţiei instrucţiunii

```
printf("\n  Valoarea lui PI =  %4.2f", PI);
```

va apare

Valoarea lui PI = 3.14

deci apariţia lui PI în cadrul şirului nu a dus la înlocuirea acestuia cu valoarea numerică definită.

Avantajul utilizării constantelor simbolice constă în posibilitatea de a simplifica scrierea programului, scriind mai simplu **E**, **PI** sau **SQRT_2** în locul numerelor care reprezintă valoarea acestora. Vor fi folosite în acele cazuri când apar frecvent în programul sursă.

5.6 Variabile simple, tablouri şi declaraţii

O definiţie posibilă a variabilei poate fi considerată următoarea : o locaţie cunoscută de memorie utilizată pentru a înregistra o valoare care poate fi modificată în cursul execuţiei programului. Fiecărei variabile i se atribuie un nume, care este un identificator.

În limbajul de programare C/C++ toate **variabilele sunt declarate** înainte de folosire. Prin aceste declaraţii este precizat **tipul** acestora şi **"clasa de memorie"** (cea de-a doua noţiune se va preciza în Capitolul X).

Tipul variabilelor se poate declara în trei locuri :

- în interiorul funcţiilor,
- în cadrul definirii parametrilor funcţiilor (vezi Capitolul X),
- în exteriorul tuturor funcţiilor (de asemenea în Capitolul X).

La început se va prezenta definirea variabilelor în interiorul funcţiilor.

Forma generală a instrucţiunilor cu care se face declararea variabilelor este :

tip lista_de_variabile ;

de exemplu:

```
int a,b,i,k,m,n;
double prod,suma01,xxxx;
char c;
```

unde, după cum se vede, se utilizează unul din cuvintele cheie urmat de o listă de identificatori, separaţi prin virgule, instrucţiunea fiind terminată cu semnul **;**

Aceste declarări a tipului variabilelor se vor face de obicei la **începutul funcţiei main** şi la **începutul instrucţiunilor compuse** .

În exemplul următor, calculul valorilor funcţiei $y=2.5*x^2-12.55*x+100.21$ în intervalul [a; b] cu pasul h, se fac declaraţii atât la începutul funcţiei **main**, declarându-se tipul variabilelor **i**, **n** , **a** , **b** şi **h** , variabile care sunt utilizate în toată funcţia , cât şi în interiorul instrucţiunii compuse (care urmează după instrucţiunea **for** şi este scrisă între acolade) variabilele **x** şi **y** care sunt utilizate numai în această instrucţiune compusă.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int i, n;         // Declaratii facute la
  double a, b, h;   // inceputul corpului functiei
  clrscr();
  printf("\n a = "); scanf("%le", &a);
  printf("\n b = "); scanf("%le", &b);
  printf("\n h = "); scanf("%le", &h);
  n=(b-a)/h+1;
  for(i=0; i<n; i++)
    { double x, y;      // declaratii valabile in
                        // instructiunea compusa
      x=a+i*h;
      y=2.5*x*x-12.55*x+100.21;
      printf("\n\t %16.6le \t %16.6le", x, y);
      if(i%20==0) getch();
     }
 getch(); return 0;
}
```

Dacă în programul sursă în afară de funcţia **main** mai există şi alte funcţii atunci declararea variabilelor se poate face şi în exteriorul funcţiilor precum şi în cadrul definirii parametrilor acestor funcii, situaţii care vor fi prezentate în Capitolul X.

În afară de declararea tipului variabilelor se poate face simultan şi iniţializarea valorilor acestora,

tip nume1=expresie_1,nume_2=expresie_2,....,nume_n=expresie_n;

după cum se arată în exemplul următor :

```
int a, b=234, c, d=-1234;
double e, f=12.34+b;
char g='c';
```

Dacă în **expresie_i** apar alte tipuri de constante sau variabile decât cele precizate de **tip** atunci se face în mod implicit conversia la tipul care se declară.

În afară de acest mod de a defini tipul unor variabile, în limbajul de programare C/C++ s-a mai introdus şi o facilitate suplimentară pentru realizarea acestui lucru prin folosirea cuvântului cheie **typedef** .

În afara celor câteva cuvinte cheie pentru specificarea tipului (**int** , **float** , **double** , **char** , etc) cu ajutorul declaraţiei **typedef** se mai pot crea de programator "cuvinte cheie" pentru precizarea unor tipuri.

Se utilizează sub următoarea formă :

typedef tip *NUME* **;**
.....
NUME ident_1 , ident_2 , ... , ident_n ;

unde **tip** este unul din tipurile definite, *NUME* este noua denumire a tipului **tip** care va putea fi utilizată iar ***ident_1*** , ***ident_2*** , ... , ***ident_n*** reprezintă identificatorii variabilelor care vor avea valori de tipul **tip** .

Astfel, în cazul unei funcţii care conţine următoarele instrucţiuni :

```
{ ...........
  typedef  double  MASE ;
  MASE  x, y, z ;
  ............
}
```

variabilele **x** , **y** şi **z** vor fi de tip **double** , deoarece declaraţia de tip **MASE** este ***"similară"*** cu declaraţia **double** , lucru care este realizat cu **typedef** .

După cum se poate observa, prin intermediul acestui cuvânt cheie **typedef** nu se crează tipuri noi de date dar în unele situaţii există posibilitatea de a folosi unele denumiri mai sugestive în funcţia ce se scrie. Astfel într-o funcţie în care se utilizează variabile ale căror valori reprezintă mase (deci valori care sunt de tip real şi nu întreg) s-ar putea folosi instrucţiunile de mai sus .

Acest mod de a realiza declararea tipului se mai poate folosi şi în cazul pointerilor (vor fi prezentaţi în Capitolul IX) precum şi al structurilor (Capitolul XII) .

În cazul funcţiilor în care se lucrează cu variabile cu indici (şiruri de variabile, matrice, tablouri cu mai mult decât două dimensiuni) acestea de asemenea se declară, în afara tipului precizându-se şi dimensiunile, date prin valorile scrise între parantezele drepte ,

```
tip nume[max_ind_1][max_ind_2]. . . [max_ind_n] ;
```

cu precizarea că acestea sunt cu o unitate mai mari decât valorile maxime ale indicilor, datorită faptului că numerotarea indicilor începe de la zero şi nu cu unu, aşa cum suntem familiarizaţi din matematici .

Astfel, dacă se consideră declaraţia de tip şi dimensiune,

```
int a[15], b[10][20];
```

rezultă că în funcţia căreia îi aparţine această declaraţie se va putea lucra cu elementele

```
a[0], a[1], a[2], . . . . , a[13], a[14]
```

ale unui şir de 15 variabile şi de asemenea cu elementele

```
b[0][0], b[0][1], b[0][2], . . . , b[0][19],
b[1][0], b[1][1], b[1][2], . . . , b[1][19],
. . . . . .. . . . . . . . . . . . . . . . . ,
b[9][0], b[9][1], b[9][2], . . . , b[9][19],
```

ale unei matrice cu 10 linii şi 20 de coloane, diferenţa faţă de scrierea obişnuită din matematică fiind aceea că indicele de linie al primei linii este 0, de asemenea şi indicele de coloană al primei coloane, deci numerotarea lor începe cu 0 şi nu cu 1.

În cazul folosirii acestor declaraţii a dimensiunilor unui tablou, programul compilator va aloca (va reţine) în memorie o zonă pentru a păstra aceste valori, deci 15*2 = 30 de octeţi pentru elementele şirului, respectiv 10*20*2 = 400 de octeţi pentru reţinerea elementelor matricei.

În cazul vectorului **a** respectiv al matricei **b** ca valori care exprimă numărul de elemente ale acestora s-au folosit constante întregi.

Este posibil de asemenea să se folosească şi constante simbolice, de exemplu instrucţiunile următoare

```
 . . . . . .
#define nr_elem_A 15;
#define lin_B 10;
#define col_B 20;
 . . . . . .
int a[nr_elem_A], b[lin_B][col_B];
```

vor avea acelaşi efect ca şi declaraţia anterior utilizată.

Ca şi în cazul variabilelor simple se poate face şi o iniţializare a elementelor tablourilor, cu una, două sau mai multe dimensiuni, simultan cu definirea tipului şi dimensiunilor.

Astfel, pentru a iniţializa valorile unui şir de variabile se poate scrie,

```
double s[5] = { 12.901, -0.89, 345.43, 9.1, 0.0023 } ;
```

ceea ce este echivalent cu următoarele

```
s[0]=12.901; s[1]=-0.89; s[2]=345.43; s[3]=9.1; s[4]=0.0023;
```

toate variabilele fiind de tip **double**. Dacă în lista de constante dintre acolade numărul acestora este mai mic decât numărul de variabile al şirului atunci ultimele variabile, cărora nu le corespund constante vor primi valori nule, deci în cazul instrucţiunii

```
double s[5] = { 12.901, -0.89, 345.43 };
```

se vor face următoarele iniţializări:

```
s[0]=12.901  s[1]=-0.89  s[2]=345.43  s[3]=0.0  s[4]=0.0
```

Declaraţia de tip şi iniţializarea elementelor unui şir se poate face şi în felul următor :

```
double s[ ] = { 12.901, -0.89, 345.43, 9.1, 0.0023 } ;
```

fără să se specifice în mod explicit dimensiunea şirului.

În cazul iniţializării elementelor unei matrice scrierea valorilor elementelor unei linii (separate prin virgule) se face între acolade, aceste acolade fiind la rândul lor scrise într-o acoladă,

```
int v[3][2]= { { 23, -5 }, {-90, 546} , { 43, 10 } } ;
```

elementele matricei **v** primind următoarele valori :

```
v[0][0] =  23;     v[0][1] =   -5;
v[1][0] = -90;     v[1][1] =  546;
v[2][0] =  43;     v[2][1] =   10;
```

Se poate deci observa că un tablou cu două dimensiuni este de fapt un tablou cu o dimensiune, ale cărui elemente sunt la rândul lor tablouri cu o dimensiune.

Este posibilă de asemenea şi următoarea formă a precedentei instrucţiuni,

```
int v[3][2]= {23,-5,-90,546,43,10} ;
```

în care nu se utilizează acoladele intermediare, ţinând cont că atribuirea valorilor elementelor din lista cuprinsă între acolade, elementelor matricei, se face în ordinea **linie după linie**.

Dacă numărul de elemente din acolade este mai mic decât cel necesar atunci ultimele elemente ale liniei sunt iniţializate cu valori nule, deci în cazul instrucţiunii

```
int w[4][4]= {{3,-5,2,12},{-8,6},{3,1,210},{23}} ;
```

matricea se va iniţializa cu următoarele valori :

```
 3   -5    2   12
-8    6    0    0
 3    1  210    0
23    0    0    0
```

Declararea tipului şi iniţializarea valorilor elementelor unei matrice se mai poate face şi în felul următor :

```
int v[][2]= { { 23, -5 }, {-90, 546} , { 43, 10 } } ;
```

fără a fi nevoie să se precizeze numărul de linii al matricei.

În cazul şirurilor de caractere, acestea se declară în felul următor :

```
char sir_1[dim_1], sir_2[dim_2], ....., sir_n[dim_n] ;
```

de exemplu :

```
char surname[25], name[25] ;
```

Se pot face şi în cazul şirurilor iniţializări, aşa cum se arată în continuare :

```
char sir[] = "Programming in C/C++" ;
```

5.7 Operatori

Aşa cum s-a precizat anterior, unul din avantajele limbajului C/C++ este şi numărul mare şi diversitatea operatorilor disponibili. Lista acestora este următoarea :

[ ]	**( )**	**.**	**- >**	**++**	**- -**
&	*	+	-	**~**	**!**
sizeof	/	%	**< <**	**> >**	**<**
>	**< =**	**> =**	**= =**	**! =**	**^**
|	**&&**	**||**	**? :**	**=**	* =
/ =	% =	+ =	- =	**< < =**	**> > =**
& =	**^ =**	**| =**	,	**#**	**# #**
: :	. *	**- >** *			

În capitolul următor se vor prezenta în detaliu toate problemele privind tipul acestor operatori, ordinea de prioritate precum şi numărul de operanzi la care se aplică, în corelaţie cu modul cum se scriu expresiile în C/C++ .

5.8 Separatori.

Separatorii sunt cea de-a şasea unitate lexicală a limbajului C/C++ .

Sunt utilizaţi pentru a separa unităţile lexicale, fiind formaţi fie dintr-un singur caracter fie din două.

Principalii separatori sunt următorii :

- **blank**-ul şi caracterele de comandă **\n** , **\t** şi altele,

- parantezele rotunde **(** şi **)** care încadrează lista argumentelor funcţiilor şi se folosesc şi în expresiile aritmetice pentru a defini ordinea efectuării operaţiilor,

int func **(** int a, int b **)** ; a = **(((** b + 1 **)** * b + 2 **)** * b + 3 **)** * b + 4 ;

- parantezele drepte **[** şi **]** care mărginesc indicii unor tablouri cu una sau mai multe dimensiuni :

a**[**12**]** b**[**5**][**2**][**6**]**

- acoladele **{** şi **}** care mărginesc corpul funcţiilor şi al instrucţiunilor compuse,

{ x = pow(a,4) + 13.22 ; y = x/(x*x+10) ; **}**

- ghilimelele **"** şi **"** care mărginesc un şir de caractere,

"anul I Inginerie economica**"**

- apostroafele **'** şi **'** care mărginesc un caracter,

'a**'**

- caracterul **;** cu care se termină o instrucţiune

x = y+3 **;**

Un separator este un caracter sau un şir de caractere care separă unităţile lexicale într-un program scris in C/C++.

Tot în categoria separatorilor se încadrează şi caracterele **/*** ****/*** respectiv **//** utilizate pentru scrierea comentariilor,

```
.........................
/*   comentariul nr. 1
     comentariul nr. 1 – continuare   */
.........................
//  comentariul nr. 2
//  comentariul nr. 3
.........................
```

Capitolul VI

Expresii, operanzi şi operatori.

6.1 Operanzi şi expresii

Ca operanzi pot fi utilizaţi următorii: constante, variabile (simple sau cu indici), şiruri, nume de funcţii, expresii indiciale, o reuniune de operanzi şi operatori, scrişi între paranteze.

Expresiile sunt combinaţii de operanzi şi operatori, ca urmare a evaluării acestora rezultând o singură valoare.

6.2 Operatori. Ordinea de prioritate a acestora

Lista completă a operatorilor limbajului C/C++ este următoarea:

[ ]	**( )**	**.**	**- >**	**++**	**- -**
&	*	**+**	**-**	**~**	**!**
sizeof	**/**	**%**	**< <**	**> >**	**<**
>	**< =**	**> =**	**= =**	**! =**	**^**
|	**&&**	**||**	**? :**	**=**	*** =**
/ =	**% =**	**+ =**	**- =**	**< < =**	**> > =**
& =	**^ =**	**| =**	**,**		
#	**# #**	**: :**	**. ***	**- > ***	

iar aceştia pot fi **unari** (se aplică unui singur operand), **binari** (leagă doi operanzi) sau **ternari** (leagă trei operanzi).

În continuare se va pune accentul în special pe operatorii specifici limbajului C.

Ordinea de prioritate a operatorilor este prezentată în continuare

Prioritatea şi modul de notare		Semnificaţia
1.	**( )**	Apel de funcţie
	[ ]	Expresie indicială
	- >	Selector indirect de componente
	: :	Acces la membrul unei clase (doar în C++)
	.	Selector direct de componente,
2. Unari	**!**	Negaţie logică
	~	Negaţie la nivel de bit
	+	Plus
	-	Minus
	++	Incrementare
	- -	Decrementare
	&	Referenţiere
	*****	Indirectare
	sizeof	Dimensiune
	new	Crearea unui obiect (în C++)
	delete	Ştergerea unui obiect (în C++)
3. Acces la membru	**. ***	C++ dereferenţiere
	- > *	C++ dereferenţiere
4. Multiplicativi	*****	Multiplicare
	/	Împărţire
	%	Impărţire modulo
5. Aditivi	**+**	Plus
	-	Minus
6. Deplasare (shift-are)	**<<**	Deplasare spre stânga la nivel de biţi
	>>	Deplasare spre dreapta la nivel de biţi
7. Relaţionali	**<**	Mai mic decât
	<=	Mai mic sau egal
	>	Mai mare decât
	>=	Mai mare sau egal
8. Egalitate	**= =**	Egal
	! =	Neegal
9.	**&**	ŞI logic la nivel de bit
10.	**^**	SAU exclusiv (XOR) la nivel de bit
11.	**|**	SAU la nivel de bit
12.	**&&**	ŞI logic
13.	**||**	SAU logic
14. Condiţional	**? :**	
15. Atribuire	**=**	Atribuire simplă
	*** =**	Atribuirea unui produs
	/ =	Atribuirea unui cât
	% =	Atribuirea unui rest
	+ =	Atribuirea unei sume
	- =	Atribuirea unei diferenţe
	& =	Atribuirea unui produs la nivel de bit
	^ =	Atribuirea unui SAU exclusiv la nivel de bit
	| =	Atribuirea unui SAU la nivel de bit

	<< =	Atribuirea unei depl. spre stânga
	>> =	Atribuirea unei depl. la dreapta
16. Virgulă	,	Calculează

6.3 Operatorii aritmetici + - * / şi %

Operatorii + şi - pot fi consideraţi operatori **unari**, atunci când se aplică unui singur operand : +a sau -b .

De obicei operatorii aritmetici leagă doi operanzi, deci sunt operatori **binari,** fiind utilizaţi pentru realizarea celor patru operaţii aritmetice elementare,

```
a+b   a-b   a*b   a/b
```

adunare, scădere, înmulţire şi împărţire.

În cazul utilizării numai a operatorilor + şi - într-o expresie aritmetică operaţiile se fac de la stânga la dreapta, deci expresia **a + b – c + d + e – f** se calculează în următoarea ordine :

((((a + b) – c) + d) + e) – f

Aceeaşi regulă funcţionează şi atunci când într-o expresie apare o succesiune de operaţii * / şi %, deci a * b / c * d se calculează în ordinea dată de paranteze : ((a * b) / c) * d

O atenţie deosebită trebuie să se acorde cazului în care ambii operanzi **a** şi **b** sunt întregi. Astfel, într-un program în care există următoarele instrucţiuni:

```
. . . . . .
int a=20000, b=25000, c;
. . . . . .
c=a+b;
d=a*b;
. . . . . .
```

şi se afişează valorile obţinute ale variabilelor întregi **c** şi **d,** în loc să se obţină rezultatele aşteptate (45000 respectiv 500000000), se obţin alte valori, incorecte (–20536 respectiv 25856 !) , deoarece nu se face o detectare a depăşirii valorilor maxime în cazul numerelor întregi de tip **int**. S-ar obţine valori corecte numai în cazul în care rezultatele ar aparţine intervalului [**-32768** ; **32767**] sau s-ar face o declaraţie de forma

```
long int a=20000, b=25000, c, d;
```

caz în care intervalul de valori posibile va deveni [**-2147483648** ; **2147483647**].

Tot în cazul când cei doi operanzi sunt de tip întreg se pot obţine rezultate "surpriză" ca urmare a operaţiei de împărţire. Astfel după operaţiile de împărţire

```
12/13       13/12       51/2        25/5
```

se obţin rezultatele 0 , 1, 25 şi 5 (valori de asemenea întregi) deci numai în cazul patru, când numărătorul este divizibil la numitor rezultatul este cel corect. Aici funcţionează regula că tipul rezultatului este acelaşi cu tipul operanzilor, când aceştia au acelaşi tip.

Pentru a obţine rezultate corecte şi în primele trei cazuri, acestea se vor scrie sub forma:

```
12./13      13/12.      51./2.
```

deci unul din operanzi sau amândoi se vor considera de tip real.

Cel de-al cincilea operator aritmetic al limbajului C/C++ este **%**, care se poate aplica în cazul a doi operanzi de tip întreg, ca rezultat obţinându-se restul împărţirii primului operand cu al doilea :

```
34%12  =  10          23%2 = 1          15%3 = 0
(34 = 2*12 + 10)    (23 = 11*2 + 1)   (15 = 5*3 + 0)
```

În multe programe prin folosirea operatorului **%** se poate verifica dacă un număr întreg este par sau impar,

$$a \begin{cases} par & daca \ \ a\%2 = 0 \\ impar & daca \ \ a\%2 = 1 \end{cases}$$

În cazul unor operanzi de tip întreg, **a** şi **b**, cu operatorii **/** şi **%** se poate determina câtul şi restul

```
. . . . . .
int a, b, quotient, remainder;    // quotient  – cat
                                  // remainder - rest
. . . . . .
quotient=a/b;
remainder=a%b;
. . . . . .
```

6.4 Operatorii relaţionali < , <= , > şi >=

Operatorii relaţionali au aproape aceeaşi formă de scriere ca şi în relaţiile matematice,

$< \rightarrow <$ $\quad \leq \rightarrow <=$ $\quad > \rightarrow >$ $\quad \geq \rightarrow >=$

O expresie de forma

expresie1 **operator_relational** *expresie2*

va avea valoarea **1** dacă condiţia care se verifică este îndeplinită şi **0** în caz contrar, spre deosebire de alte limbaje, cum ar fi de exemplu limbajul FORTRAN, unde există constantele logice **.TRUE.** şi **.FALSE.** şi expresiile relaţionale primesc una din aceste valori.

Astfel, ca urmare a evaluării expresiilor următoare se obţin rezultatele

1 <= 4	**1**	(inegalitatea este satisfăcută),
1+4<= 4	**0**	(operatorul + este prioritar faţă de operatorul relaţional <= , deci 5<=4, inegalitate falsă),
1+(3<=4)	**2**	(se evaluează expresia din paranteze, a cărei valoare este 1 iar apoi se face adunarea),
(1<2)+(3<4)	**2**	(cele două expresii relaţionale din paranteze au valori de 1, deci rezultatul este 2)

În cazul unei succesiuni de doi sau mai mulţi operatori relaţionali aceştia acţionează de la stânga la dreapta, obţinându-se următoarele rezultate ca urmare a evaluării expresiilor:

$1 < 2 < 3 = \mathbf{1}$

deoarece la început se evaluează expresia 1<2 obţinându-se valoarea 1 iar apoi 1<3 rezultând tot **1** iar în cazul expresiei

15>10>5 = **0**

s-a obţinut **0** deoarece 15>10 are ca rezultat **1** iar apoi 1>5 are ca rezultat **0**.

În cazul expresiei

1<2+3<4

la început se face suma 2+3, rezultând astfel **1<5<4** , care se evaluează în ordinea (1<5)<4 obţinându-se rezultatul final **1** .

Aceşti operatori relaţionali, după cum s-a arătat în §6.2, au o prioritate mai mică decât operatorii aritmetici dar mai mare decât cei de egalitate, inegalitate, logici şi logici la nivel de bit.

6.5 Operatorii de egalitate == şi inegalitate !=

Sunt utilizaţi pentru a verifica dacă două expresii sunt sau nu egale, având următoarea sintaxă

expresie1 **operator** *expresie2*

obţinându-se rezultatul **1** (în caz de adevăr) sau **0** (dacă nu este verificată condiţia).

Astfel

$$a==b \;=\; \begin{cases} 0 & daca\ a \neq b \\ 1 & daca\ a = b \end{cases} \qquad a\ !=\ b \;=\; \begin{cases} 0 & daca\ a = b \\ 1 & daca\ a \neq b \end{cases}$$

De exemplu, expresia **a%2==0** (unde **a** este de tip **int**) are valoarea **1** dacă **a** este divizibil cu 2 şi **0** în caz contrar.

Expresia **sir[i] != '\0'** (unde **sir** este un şir de caractere) va avea valoarea **1** dacă **sir[i]** nu conţine elementul nul **'\0'** respectiv valoarea **0** dacă este elementul nul .

6.6 Operatorii logici ! && şi ||

Operatorii logici au fost introduşi de matematicianul englez **George Boole** în lucrarea "An Investigation of the Law of Thought" (1854).

În algebra booleană operanzii pot avea doar două valori, **0** respectiv **1** iar operatorii sunt **NU logic** (complementul, negaţia) **, SAU logic** (reuniunea, suma logică) şi **ŞI logic** (intersecţia, produsul logic). Primele denumiri ale acestor operatori corespund interpretării diagramelor **Venn** [Chu_68], în care operanzii sunt reprezentaţi ca mulţimi, iar celelalte denumiri, mai frecvent folosite, corespund utilizării tabelelor de adevăr.

În cazul când se folosesc doi operanzi se pot imagina un număr de 16 funcţii booleene diferite. Dintre acestea , pe lângă cele trei amintite :

$$\overline{a}\ \ (negatie) \qquad a \vee b\ \ (SAU\ \ logic) \qquad a \wedge b\ \ (SI\ \ logic)$$

mai sunt folosite şi următoarele :

$$a \oplus b = \bar{a} \wedge b \ \vee \ a \wedge \bar{b} \quad (SAU\ exclusiv)$$
$$a \equiv b \ = a \wedge b \vee \bar{a} \wedge \bar{b} \quad (coincidenta)$$

În limbajul de programare C/C++ se pot folosi cei trei operatori logici fundamentali, **negaţia**, **SAU logic** şi **ŞI logic**, care se notează în felul următor:

$$\bar{a} \ \rightarrow \ !a \qquad a \vee b \ \rightarrow \ a \,||\, b \qquad a \wedge b \ \rightarrow \ a \,\&\&\, b$$

Rezultatele care se obţin utilizând aceşti operatori se prezintă în tabel:

a	b	$\bar{a}$ $!a$ **NEGATIE**	$a \vee b$ $a \,\|\|\, b$ **SAU logic**	$a \wedge b$ $a \,\&\&\, b$ **SI logic**	$a \oplus b$	$a \equiv b$
0	0	1	0	0	0	1
0	1 (≠0)	1	1	0	1	0
1 (≠0)	0	0	1	0	1	0
1 (≠0)	1 (≠0)	0	1	1	0	1

Dacă în algebra booleană valorile posibile ale variabilelor logice şi ale expresiilor erau **0** şi/sau **1** în cazul limbajului de programare C/C++ toate valorile diferite de zero ale operanzilor se vor asimila cu valoarea **1** iar valorile ce rezultă ca urmare a evaluării expresiilor sunt **0** şi/sau **1**.

Este important de amintit că există două formule importante, datorate lui **DeMorgan**, utilizate deja în Capitolul II,

$$\overline{a \vee b} = \bar{a} \ \wedge \ \bar{b} \qquad \overline{a \wedge b} = \bar{a} \ \vee \ \bar{b}$$

care sunt frecvente folosite.

Adesea operatorul de negaţie **!** se foloseşte pentru a scrie într-un alt mod o expresie relaţională, astfel următoarele expresii au aceeaşi valoare

a<5 şi **! (a>=5)**

Dacă trebuie verificat faptul că valoarea unei variabile **x** este cuprinsă în intervalul [**a**; **b**],

$$x \in [a; b] \qquad sau \qquad a \le x \le b$$

în limbajul C/C++ se va evalua expresia

a <=x && x <= b

care trebuie să aibă valoarea **1** în cazul în care condiţia este îndeplinită respectiv **0** în caz contrar.

Ţinând cont de legile lui DeMorgan, se poate scrie o expresie logică echivalentă cu cea anterioară :

! (a > x || x > b)

Cele două operaţii logice, **SAU exclusiv** şi **coincidenţa** , cărora în limbajul C/C++ nu le corespund operatori, se pot scrie sub forma

$$a \oplus b \ \rightarrow \ !\,a \,\&\&\, b \,||\, a \,\&\&\, !b$$
$$a \equiv b \ \rightarrow \ a \,\&\&\, b \,||\, !a \ \&\&\, !\,b$$

operaţiile efectuându-se în următoarea ordine : **((!a) && b) ‖ (a&&(!b))** respectiv **(a&&b) ‖ ((!a)&&(!b))** , conform ordinii de prioritate cunoscute: **1**- operatorul **!**, **2** – operatorul **SI logic**, **3** – operatorul **SAU logic**.

În cazul unor expresii logice de tipul **0 && operand** respectiv **1 ‖ operand** nu se mai evaluează al doilea operand, valoarea expresiei rezultând pe baza valorii primului operand, fiind **0** respectiv **1**.

6.7 Operatorii logici la nivel de bit ~ | & ^ << şi >>

Cei doi operanzi (un operand în cazul operatorului unar ~) pot fi de tipul **int**, **long int**, **short**, **unsigned** şi **char.** Se fac operaţii cu cifrele binare ale reprezentării interne a acestor operanzi.

Există şase operatori logici la nivel de bit, dintre care pentru primii patru sunt prezentate notaţia (atât în scrierea matematică cât şi în C/C++) şi semnificaţia în tabelul următor:

a	b	$\bar{a}$ $\sim a$ **Complement faţă de unu**	$a \vee b$ $a \mid b$ **SAU logic** la nivel de bit	$a \wedge b$ $a \,\&\, b$ **SI logic** la nivel de bit	$a \oplus b$ $a \,\hat{}\, b$ **SAU exclusiv** La nivel de bit
0	0	1	0	0	0
0	1	1	1	0	1
1	0	0	1	0	1
1	1	0	1	1	0

În cazul operatorului **~** (negaţie la nivel de bit, complementare faţă de unu) se face o inversare a cifrelor binare, **0** devenind **1** iar **1** devenind **0**.

În cazul celorlalţi trei operatori (binari) se fac respectivele operaţii cu fiecare pereche de cifre binare ale reprezentării în baza 2 a operanzilor, cifre binare care sunt pe aceleaşi poziţii în cadrul numerelor.

Se consideră variabilele cu valori întregi, de tip **int** :

```
a = 0000 0000 1000 0111₂ = 207₈ = 135₁₀ = 87₁₆
b = 0000 0000 0101 0111₂ = 127₈ =  87₁₀ = 57₁₆
c = 0000 0000 0001 1110₂ =  36₈ =  30₁₀ = 1E₁₆
```

ale căror valori sunt scrise în sistemele de numeraţie binar, octal, zecimal şi hexazecimal, înregistrarea lor în memorie făcându-se pe câte doi octeţi.

Se obţin următoarele rezultate în cazul aplicării operatorului unar **~** de complementare faţă de unu:

```
 a = 0000 0000 1000 0111₂ =    207₈ =  135₁₀ =   87₁₆
     ~~~~ ~~~~ ~~~~ ~~~~
~a = 1111 1111 0111 1000₂ = 177570₈ = -136₁₀ = FF78₁₆
```

În cazul celorlalţi trei operatori (binari) la nivel de bit – **SAU** , **SI** şi **SAU exclusiv** – se obţin rezultatele :

```
  b = 0000 0000 0101 0111₂ = 127₈ =  87₁₀ = 57₁₆
(SAU) |||| |||| |||| ||||
  c = 0000 0000 0001 1110₂ =  36₈ =  30₁₀ = 1E₁₆

b|c = 0000 0000 0101 1111₂ = 137₈ =  95₁₀ = 5F₁₆
```

```
  b = 0000 0000 0101 0111₂ = 127₈ =  87₁₀ = 57₁₆
(SI)  &&&& &&&& &&&& &&&&
  c = 0000 0000 0001 1110₂ =  36₈ =  30₁₀ = 1E₁₆

b&c = 0000 0000 0001 0110₂ =  26₈ =  22₁₀ = 16₁₆


  b = 0000 0000 0101 0111₂ = 127₈ =  87₁₀ = 57₁₆
(SAU excl)^^^^ ^^^^ ^^^^ ^^^^
  c = 0000 0000 0001 1110₂ =  36₈ =  30₁₀ = 1E₁₆

b^c = 0000 0000 0100 1001₂ = 111₈ =  73₁₀ = 49₁₆
```

toate operaţiile făcându-se, aşa cum se poate cu uşurinţă observa, cu cifrele binare care ocupă aceleaşi poziţii în cadrul scrierii binare a numerelor.

Semnul ^ folosit pentru notarea operatorului SAU exclusiv se numeşte ***caret*** (în limba engleză).

Alţi operatori la nivel de bit sunt cei **doi operatori de deplasare** **>>** şi **<<** ,

a >> b **a << b**

în primul caz toate cifrele binare ale reprezentării numărului **a** se deplasează spre dreapta cu un număr de unităţi egal cu valoarea lui **b** (valoarea rezultatului este mai mică decât **a**, pierzându-se ultimii **b** biţi şi completându-se cu **b** zerouri în stânga) iar în al doilea caz deplasarea se face spre stânga (valoarea rezultatului este mai mare decât **a**, obţinându-se $\mathbf{a} \cdot 2^{\mathbf{b}}$, cu condiţia să nu dispară primii biţi ai numărului **a**, la dreapta numărului binar adăugându-se **b** zerouri).

Dacă primul operand este

$\mathbf{a} = 0000\ 0000\ 0\mathbf{101\ 0111}_2 = \mathbf{87}_{10} = \mathbf{57}_{16}$

iar al doilea, **b**, va lua valori în intervalul [0 ; 7] atunci se obţin următoarele valori ale expresiei **a>>b :**

b	**a>>b**		
	Reprezentarea binară	**Repr. zecimală**	**Repr. hexazecimală**
0	0000 0000 0**101 0111**	87	57
1	0000 0000 00**10 1011**	43	2B
2	0000 0000 000**1 0101**	21	15
3	0000 0000 0000 **1010**	10	A
4	0000 0000 0000 0**101**	5	5
5	0000 0000 0000 00**10**	2	2
6	0000 0000 0000 000**1**	1	1
7	0000 0000 0000 0000	0	0

Dacă se consideră expresia **a<<b** unde **a** are aceeaşi valoare iar **b** ia valori în intervalul [0 ; 12] se obţin rezultatele :

b	a<<b		
	Reprezentarea binară	Repr. zecimală	Repr. hexazecimală
0	0000 0000 **0101 0111**	87	57
1	0000 0000 **1010 111**0	174=87·2	AE
2	0000 000**1 0101 11**00	348=87·2^2	15C
3	0000 00**10 1011 1**000	696=87·2^3	2B8
4	0000 **0101 0111** 0000	1392=87·2^4	570
5	0000 **1010 111**0 0000	2784=87·2^5	AE0
6	000**1 0101 11**00 0000	5568=87·2^6	15C0
7	00**10 1011 1**000 0000	11136=87·2^7	2B80
8	**0101 0111** 0000 0000	22272=87·2^8	5700
9	**1010 111**0 0000 0000	-20992	AE00
10	**0101 11**00 0000 0000	23552	5C00
11	**1011 1**000 0000 0000	-18432	B800
12	**0111** 0000 0000 0000	28672	7000

În unele situaţii se va prefera să se programeze calculul expresiei **a<<b** decât **a*2^b** deoarece prima operaţie se face mult mai rapid, bineînţeles atunci când nu se obţin rezultate eronate, aşa cum se poate vedea şi din tabelul anterior prezentat.

Observaţia nr. 1 . Cunoscând faptul că ultimul bit al unui ***număr par*** trebuie să fie **0** iar al unui ***număr impar*** **1** se poate determina dacă un număr întreg **a** este par sau impar prin calculul expresiei :

$$a \& 1 = \begin{cases} 0 & (numarul\ este\ par) \\ 1 & (numarul\ este\ impar) \end{cases}$$

Observaţia nr. 2 . Folosind operatorii la nivel de bit se poate determina valoarea cifrei din reprezentarea binară a numărului, care se găseşte pe o anumită poziţie. Dacă se consideră un număr de tip **int** (care se reprezintă pe doi octeţi = 16 biţi) , de exemplu numărul **n** = 13215, cifra binară care se găseşte pe poziţia **i** = 5 va putea fi determinată în felul următor :

- aplicând operatorul de deplasare la dreapta facem ca această cifră să ajungă pe poziţia ultimă (a 16-a),

- determinarea ultimei cifrei a numărului se face ca urmare a efectuării înmulţirii la nivel de bit dintre numărul anterior obţinut şi cifra 1, utilizându-se deci următoarea secvenţă de instrucţiuni :

```
dig= numar>>(16-i) & 1;
```

Dacă se doreşte determinarea tuturor cifrelor binare ale respectivului număr se va folosi următorul program, în care apar evident instrucţiuni care vor fi prezentate în capitolele următoare :

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
int main(void)
{ int dig, i, numar=13215;
  char str[25];
  clrscr();
```

```
  printf("\n\n\t\t      1234567890123456");       //Cap.VII
  itoa(numar, str, 2);                            //Cap.XI
  printf("\n\n\t %d (10) = %16s (2)\n",numar,str);//Cap.VII
  for(i=1; i<=16; i++)                            //Cap.VIII
     { dig= numar>>(16-i) & 1;
       printf("\n\t digit[%2d] = %d ", i, dig);   //Cap.VII
     }
 getch(); return 0;                               //Cap.VII
}
```

Rezultatele obţinute cu acest program sunt următoarele :

```
                    1234567890123456

          13215 (10) =   11001110011111 (2)

          digit[ 1] = 0
          digit[ 2] = 0
          digit[ 3] = 1
          digit[ 4] = 1
          digit[ 5] = 0
          digit[ 6] = 0
          digit[ 7] = 1
          digit[ 8] = 1
          digit[ 9] = 1
          digit[10] = 0
          digit[11] = 0
          digit[12] = 1
          digit[13] = 1
          digit[14] = 1
          digit[15] = 1
          digit[16] = 1
```

6.8 Operatorul de atribuire (asignare) =

Acest operator este notat cu semnul **=** , ca şi în alte limbaje de programare, şi este utilizat în expresii de forma

v = ***expresie ;***

unde **v** poate fi o variabilă simplă sau cu indici iar ***expresie*** conţine numai elemente cunoscute în momentul evaluării acesteia, valoarea expresiei atribuindu-se variabilei, după cum se arată în exemplele următoare:

x = a + b ; c = getch( ) ; t[2][4] = 57.98 ;

Valoarea expresiei formate **v=expresie** este egală cu valoarea atribuită lui **v**.

În cazul în care tipul lui ***expresie*** şi al lui **v** nu coincid, valoarea expresiei se converteşte la tipul variabilei.

Operatorul de atribuire **=** poate fi precedat de unul din următorii operatori aritmetici şi operatori logici la nivel de bit :

+ - * / % | & ^ << >>

această convenţie fiind introdusă cu scopul de a simplifica scrierea în cazurile în care se face modificarea valorii unei variabile, următoarele expresii fiind echivalente :

```
x=x+p        x+=p                m=m|12     m|=12
a=a-5.5      a-=5.5              k=k&3      k&=3
v=v*2.5  şi  v*=2.5              l=l^23 şi  l^=23
d=d/3.91     d/=3.91             i=i<<1     i<<=1
n=n%4        n%=4                j=j>>2     j>>=2
```

unde variabilele i, j, k, l, m, n sunt de tip **int**.

Astfel de combinaţii dintre operatorul de atribuire şi unul din operatorii aritmetici sau logici la nivel de bit se numesc **"shorthand"** fiind specifice limbajului de programare C/C++.

Prioritatea operatorilor de atribuire este redusă, numai operatorul virgulă având o prioritate inferioară acestora.

6.9 Operatori de atribuire multiplă

În limbajul C/C++ este posibilă şi atribuirea multiplă, care conţine mai mult decât un semn egal,

v1 = v2 = ... = vn = ***expresie***

ordinea în care se fac atribuirile fiind ***de la dreapta spre stânga***, deci atribuirile se vor face în ordinea indicată de paranteze :

v1 = (v2 = (v3 = (..... = (vn = ***expresie*** **) .....)))**

Aceeaşi ordine se păstrează şi în cazul unor expresii de forma :

```
v1= 10*(v2=12-(v3=15))-2
(3)       (2)     (1)
```

ca urmare a evaluării acesteia obţinându-se, în ordine v3 = 15, v2 = -3, v1 = -32 .

6.10 Operatorii de incrementare ++ şi decrementare --

Tot cu scopul de a simplifica scrierea unor expresii s-au introdus operatorii unari **++** şi **--** utilizabili atunci când o variabilă trebuie să-şi mărească sau să-şi micşoreze valoarea cu o unitate.

Diferenţa dintre semnificaţiile următoarelor forme de scriere a expresiilor

u = ++ i v=i++

constă în faptul că ;

- în primul caz (operatorul este **prefixat**) valoarea iniţială a variabilei **i** se măreşte cu o unitate iar apoi această ***valoare modificată*** se atribuie lui **u** iar

- în al doilea caz (operatorul este **postfixat**) ***valoarea iniţială*** a lui **i** se atribuie lui **v** iar apoi se modifică valoarea lui **i** , crescând cu o unitate.

Deci, dacă s-ar considera instrucţiunile :

u = ++ i ; **v = i ++ ;**

acestea ar fi echivalente cu următoarele :

i = i + 1 ; **v = i ;**
u = i ; **i = i + 1 ;**

Explicaţii similare se pot da şi în cazul operatorului de decrementare **- -,**

w = - - i ; **z = i - - ;**

Dacă valoarea iniţială a lui **i** , în toate cele patru cazuri, ar fi 5 atunci în primele două cazuri s-ar obţine **i=6, u=6, v=5** iar în ultimele două cazuri s-ar obţine **i=4, w=4, z=5**, după evaluarea expresiilor.

6.11 Operatorul de forţare a tipului

Este un operator unar, care are rolul de a modifica tipul rezultatului unei expresii, având forma generală :

(tip) ***expresie***

o astfel de formă numindu-se expresie **"cast"**. Utilizarea unor astfel de expresii este necesară atunci când funcţiile au parametri de un anumit tip, care trebuie respectat sau atunci când se doreşte ca rezultatul obţinut ca urmare a calculului să fie de un alt tip decât cel care ar rezulta în mod normal (implicit).

```
double func(int a);
..............
int i, n; double x, y, z;
x=(double)i/n;  // in absenta operatorului (double) lui x i se
                // va atribui partea intreaga a catului i/n
y=func((int)z); // Functia func trebuie sa aiba argument de
                // tip int, conform definitiei sale
..............
```

După cum se poate observa, ca urmare a acestui operator se face o conversie explicită a tipului.

6.12 Operatorii paranteză. Operatorii de apel de funcţie ( ). Operatorii de indexare [ ]

Prin utilizarea parantezelor **(** respectiv **)** se modifică prioritatea de acţiune a operatorilor iar în cazul funcţiilor acestea includ lista parametrilor , în acest caz numindu-se operatori de apel de funcţie.

Parantezele drepte **[** şi **]** se numesc operatori de indexare deoarece includ expresii indiciale.

În următoarea secvenţă de program sunt prezentate cele trei cazuri

```
.....................
int i, n, n_eq;
double a, b, c, h, t[35], yD, yR, yS;
.....................
h=(b-a)/(n-1);
func(a, n_eq, &yR, &yD, &yS);
c=t[n+3*i];
.....................
```

6.13 Operatorul condiţional ternar ? :

Acest operator reuneşte **trei expresii**, fiind un operator ternar, sintaxa **expresiei condiţionale** fiind următoarea:

expresie1 **?** *expresie2* **:** *expresie3*

Se evaluează ***expresie1*** care poate fi diferită de zero (adevărată, dacă se verifică o condiţie) sau egală cu zero (neadevărată dacă condiţia nu este îndeplinită). În primul caz ca rezultat se consideră valoarea lui ***expresie2*** iar în al doilea al lui ***expresie3*** , întotdeauna fiind evaluată numai una din cele două expresii.

Astfel, dacă se consideră secvenţele de instrucţiuni:

```
.....................
double a, b, c, d, e, v_max, v_min, abs_e;
.....................
v_max = a>b ? a : b ;        // (1)
v_min = c<d ? c : d ;        // (2)
abs_e = e>0 ? e : -e ;       // (3)
.....................
```

în cadrul instrucţiunii (1) se determină care din variabilele **a** şi **b** are valoare maximă iar în cadrul instrucţiunii (2) care din variabilele **c** şi **d** are valoare minimă, aceste valori atribuindu-se lui **v_max** respectiv **v_min**. În instrucţiunea (3) apare o expresie condiţională a cărei valoare este chiar valoarea absolută a variabilei **e** .

Utilizând operatorul condiţional ternar se pot face şi diferite operaţii cu variabile de tip **char**. Astfel, dacă un caracter este ***literă mare*** se poate transforma în ***literă mică*** sau să rămână nemodificat (dacă nu este literă mare), conform următoarei secvenţe de instrucţiuni:

```
............
char c1, c2;
............
c2 = c1>='A' && c1<='Z' ? c1+'a'-'A' : c1 ;
............
```

6.14 Operatorul virgulă

Operatorul virgulă este utilizat sub forma

(expresie_1 , expresie_2)

sau

(expresie_1 , expresie_2 , ... , expresie_n)

expresiile fiind **evaluate** în ordine **de la stânga la dreapta**, valoarea întregii expresii fiind egală cu valoarea ultimei expresii (din dreapta).

Operatorul virgulă are cea mai redusă prioritate, motiv pentru care aceste expresii separate prin virgule se scriu între paranteze.

Un exemplu ar fi următorul, în care se calculează lungimea ipotenuzei unui triunghi dreptunghic cu catetele **b** şi **c**, cărora li se atribuie în expresiile nr. 1 şi nr. 2 valori iniţiale:

a = **(** b=10 **,** c=5 **,** sqrt(b*b+c*c) **)** ;

rezultatul obţinut în urma evaluării ultimei expresii (11.180339) fiind atribuit lui **a**.

6.15 Operatorul dimensiune sizeof

Operatorul **sizeof** se utilizează pentru a determina dimensiunea, în octeţi, a unei date (variabilă simplă sau cu indici, nume de tablou, nume de structură) sau a unui anumit tip de variabilă. Se scrie sub forma :

sizeof(data**)** sau **sizeof(**tip**)**

în cazul datelor parantezele nefiind necesare.

Considerând următoarea secvenţă de program :

```
.....................
int i, n1, n2, n3, n4; double a, b[10];
.....................
n1=sizeof(i); n2=sizeof(a); n3=sizeof(b[2]); n4=sizeof(b);
.....................
```

variabilele întregi n1, n2, n3 şi n4 vor primi următoarele valori :

n1=2 n2=8 n3=8 n4=80

deoarece valoarea lui **i** este înregistrată pe doi octeţi, a lui **a** şi **b[2]**, fiind de tip **double**, pe câte 8 octeţi iar elementele tabloului **b** , care are 10 elemente, pe 80 de octeţi.

6.16 Operatorii unari de adresare & şi de indirectare *

Aceşti operatori vor fi utilizaţi în Capitolul IX - **Pointeri**.

Considerând o variabilă **v** de un anumit tip prin **&v** se înţelege **"adresa"** acestei variabile, adică un număr care reprezintă numărul de ordine ataşat primului octet (byte) dintre cei în care este reprezentată în memorie variabila respectivă.

Dacă **p** este o variabilă care conţine adresa unei locaţii de memorie (**p** este numit **pointer**) prin aplicarea operatorului de indirectare ***** acestei variabile pointer, ***p**, se va obţine valoarea care este înregistrată începând cu locaţia respectivă.

6.17 Operatorii punct . şi săgeată ->

Aceşti operatori sunt specifici **structurilor** şi **uniunilor** şi se vor prezenta în Capitolul XII, primul, adică operatorul "punct", fiind folosit atunci când se lucrează cu un element al unei structuri sau uniuni iar al doilea, "săgeata", atunci când se utilizează un pointer de structură sau uniune.

Capitolul VII

Funcţii de I/O

Operaţiile de intrare/ieşire (**I**nput/**O**utput) în cazul limbajului C/C++ nu se realizează ca urmare a unor comenzi date prin cuvinte instrucţiune (cum e de exemplu în limbajul Fortran : read, write, print, accept) ci prin utilizarea unor funcţii aparţinând fişierelor header **conio.h** şi **stdio.h** (**stdio.h** este cuprins în standardul **ANSI C**) .

În acest capitol se vor prezenta funcţiile de **I/O** care realizează introducerea datelor prin **tastatură** şi obţinerea rezultatelor pe **monitor**.

Există şi operaţii de I/O în cazul fişierelor, acestea urmând să fie prezentate în Capitolul XIII - Fişiere.

7.1 Funcţii de I/O pentru caractere

Există cinci funcţii care pot realiza astfel de operaţii : **getch, getche, putch** (din ***conio.h***), **getchar, putchar** (din ***stdio.h***).

Funcţiile **getch( )** şi **getche( )** sunt utilizate pentru introducerea unui caracter, deosebirea dintre acestea fiind aceea că în cazul funcţiei **getch( )** caracterul tastat ***nu va apare*** pe ecran pe când în cazul funcţiei **getche( )** acesta ***va apare*** pe ecran.

Aceste funcţii au următoarele prototipuri :

```
int getch(void);   int getche(void);
```

Dacă este introdus un caracter al tastaturii care are codificarea în ASCII atunci valoarea returnată de aceste funcţii este cea corespunzătoare codului ASCII. Astfel, dacă se consideră următoarea secvenţă de instrucţiuni :

```
...........
int a, b, c;
...........
a=getch(); b=getch(); c=getch();
...........
```

iar utilizatorul va tasta caracterele u, v şi W (care nu apar pe ecran) atunci variabilelor întregi a, b şi c li se vor atribui valorile întregi (în zecimal) 117, 118 şi 87, corespunzătoare codificării ASCII a acestor caractere.

Pentru a introduce caracterele corespunzătoare unor taste speciale : F1 F2 ... , F10 → ↑ Scroll Lock Page Down Enter ş. a. funcţia **getch( )** sau **getche( )** se va scrie de două ori în

program,

```
..........
getch(); getch();
...........
```

prima dată returnând valoarea **zero** iar a doua oară o **valoare specifică** respectivei **taste**, astfel dacă se apasă F1 se returnează valoarea **0** ; **59** (59 corespunde caracterului **:**) , dacă se apasă F8 se returnează **0** ; **66** (66 este codificarea caracterului **B**) iar dacă se apasă Enter atunci rezultă 13 (**CR**).

Valorile returnate în cazul apăsării diferitelor taste funcţionale sunt prezentate în Capitolul XVII – Anexa nr. 2.

Pentru a realiza afişarea unui caracter se utilizează funcţia **putch** , care are următorul prototip:

```
char putch(int expresie);
```

Se returnează (afişează) un caracter, care este funcţie de valoarea expresiei utilizată ca argument. Dacă aceasta este cuprinsă în intervalul [32, 127] atunci se afişează un caracter ASCII imprimabil.

În locul unei valori numerice sau a expresiei se pot folosi şi caractere de control, cum ar fi **'\a'** (BEL - audible alert), **'\b'** (BS - backspace), **'\n'** (NL - new line), **'\r'** (CR - carriage return) ş. a., următoarele instrucţiuni având acelaşi efect :

```
putch('\n');   sau   putch(10);
putch('\r');   sau   putch(13);
putch('\b');   sau   putch(8);
```

unde în partea din dreapta s-au înlocuit caracterele de control cu valoarea acestora conform codificării ASCII. Aceste instrucţiuni vor avea efect asupra modului cum se face afişarea. Astfel, succesiunea :

```
putch('\n');  putch('\r');
```

va avea ca efect trecerea la un rând nou, la începutul acestuia.

În continuare se prezintă un program în cadrul căruia se utilizează cele trei funcţii anterior prezentate, de asemenea se mai foloseşte şi funcţia **printf(...)** (vezi §7.3) pentru a apare nişte texte explicative pe ecran. Considerăm comentariile care sunt intercalate în programul sursă suficiente pentru înţelegerea modului de utilizare al tuturor instrucţiunilor.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{char c1, c2;

     // Afisarea datelor incepe pe un ecran nou
 clrscr();

     // Functia putch() este folosita de 5 ori, pentru
     // tiparirea caracterelor C , / , C , +  si +.
 putch('C'); putch('/'); putch('C'); putch('+'); putch('+');
       // Se trece la un inceputul unui rand nou,
       // lasand un rand liber.
```

```
 putch('\n'); putch('\n'); putch('\r');

     // Functia putch() este folosita de 5 ori, fiecare caracter
     // fiind afisat pe un rand nou, pe pozitii succesive.
 putch('C'); putch('\n');    putch('/'); putch('\n');
 putch('C'); putch('\n');    putch('+'); putch('\n');
 putch('+');

     // Se trece la un inceputul unui rand nou,
     // lasand un rand liber.
 putch('\n'); putch('\n'); putch('\r');

     // Functia putch() este folosita de 5 ori, fiecare caracter
     // fiind afisat pe un rand nou, pe prima pozitie.
 putch('C'); putch('\n'); putch('\b');
 putch('/'); putch('\n'); putch('\b');
 putch('C'); putch('\n'); putch('\b');
 putch('+'); putch('\n'); putch('\b');
 putch('+'); putch('\n'); putch('\b');

     // Aceasta functie afiseaza textul "Tastati trei caractere".
 printf("\n\t Tastati trei caractere\n");
     // Se apeleaza de trei ori functia getche(). Caracterele
     // tastate apar pe ecran.
 getche(); getche(); getche();

     // Se trece la inceputul unui rand nou
 putch('\n'); putch('\r');

       // Aceasta functie afiseaza textul "Tastati un caracter".
 printf("\n\t Tastati un caracter\n");
     // Un caracter introdus cu functia getch() se
     // afiseaza cu functia putch().
 putch(getch());

     // Se trece la inceputul unui rand nou
 putch('\n'); putch('\r');

     // Aceasta functie afiseaza un text
 printf("\n\n\t Apasati o tasta functionala F1 .. F10 sau Enter\n");
     // In urma apasarii unei taste functionale (F1 ... F10  s.a.)
     // se va obtine codificarea acesteia.
 c1=getch(); c2=getch();
     // Se afiseaza valoarea numerica coresp. tastei functionale
 printf("\n %d %d", c1, c2);

     // Ecranul ramane nemodificat pana la apasarea unei taste.
 getch(); return 0;
}
```

Ca urmare a execuţiei primelor trei grupuri de instrucţiuni, pe ecran va apare :

```
C/C++

C
 /
  C
   +
    +
```

```
C
/
C
+
+
```

iar apoi utilizatorul va trebui să respecte cerinţele programului, aşa cum apar specificate pe ecran.

S-a putut observa că la începutul programului s-a apelat funcţia **clrscr** (**cl**ea**r scr**een), care de asemenea aparţine fişierului header ***conio.h***.

Funcţia

```
getch();
```

se utilizează frecvent pentru a "opri" momentan execuţia programului şi derularea pe ecran a rezultatelor, până la apăsarea unei taste.

Alte funcţii ale limbajului C/C++ care realizează operaţii de I/O cu caractere sunt **getchar** şi **putchar**, care au următoarele prototipuri :

```
int getchar(void);      int putchar(int c);
```

Funcţia **putchar** este asemănătoare cu funcţia **putch**, cu deosebirea că în cazul folosirii ca argument a caracterului **'\n'** (new line), aceasta realizează trecerea la începutul unui rând nou, pe când **putch('\n')** face trecerea la un rând nou pe poziţia următoare faţă de cea de pe rândul anterior.

Funcţia **getchar** realizează citirea unui caracter de la tastatură (urmat de apăsarea tastei Enter), returnând o valoare întreagă care este corespondentul în codul ASCII al caracterului introdus.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int c; clrscr();   // se poate declara si char c;
  c = getchar(); putchar(c);
  getch(); return 0;
}
```

Dacă se tastează caracterul U, urmat de Enter , atunci pe ecran va apărea

```
U
U
```

primul U apărând în urma tastării (cu ecou) iar al doilea apare ca urmare a afişării cu funcţia **putchar**.

Dacă s-ar tasta mai multe caractere : Uvwxyz urmate de Enter atunci pe ecran apare numai primul caracter ca urmare a afişarii cu **putchar**.

```
Uvwxyz
U
```

În programul următor se utilizează de asemenea aceste funcţii, cu primele două grupe de funcţii se afişează în două feluri caracterele C , + , + .

```
#include <conio.h>
#include <stdio.h>

int main(void)
{char c; clrscr();

     // Se afiseaza caracterele  'C'  '+'  si  '+' cu macroul
     // putchar, in doua feluri, pe linie si pe coloana.
 putchar('C'); putchar('+');  putchar('+'); putchar('\n');
 putchar(10);

 putchar('C'); putchar('\n'); putchar('+');
 putchar(10);  putchar('+');  putchar(10);

     // Se introduc min. 10 caractere (cu ecou), urmate de <Enter>
     // Primele sase caractere vor fi afisate pe o linie noua.
 putchar(10);
 putchar(getchar()); putchar(getchar());  putchar(getchar());
 putchar(getchar()); putchar(getchar());  putchar(getchar());

 getch(); return 0;
}
```

iar în continuare dacă utilizatorul tastează

```
abcdefghijklmno
```

urmat de 

, atunci pe ecran va apare :

```
abcdefghijklmno
abcdef
```

acest lucru fiind explicabil prin faptul că ceea ce se tastează, până la 

, se introduce într-o zonă tampon (acest lucru se numeşte ***line-buffered input***), formându-se un şir de caractere care urmează să fie preluate, fiind afişate primele şase datorită succesiunii (de şase ori) a funcţiilor **putchar(getchar( )).**

7.2 Funcţii de I/O pentru şiruri

Operaţiile de **I/O** cu şiruri de caractere se pot realiza cu funcţiile **gets** şi **puts** din fişierul header **stdio.h**.

Aceste funcţii au următoarele prototipuri :

```
int puts(const char *str);   char *gets(char *str);
```

în cazul funcţiei **puts** şirul utilizat ca argument se afişează la monitor iar cu funcţia **gets** se realizează introducerea de la tastatură a unui şir (până la apăsarea tastei 

).

În funcţiile programului sursă în care vor fi utilizate **gets** şi **puts** vor trebui să existe declaraţii referitoare la dimensiunile şirurilor de caractere care se vor introduce, de exemplu :

```
char sir1[50], sir2[50];
```

Este cunoscut faptul că în cazul unui şir compus din **n** caractere acesta ocupă în memorie un număr de **n+1** octeţi, în ultimul înregistrându-se caracterul nul (**'\0'**). Acest ultim caracter, atunci

când şirul se afişează cu funcţia **puts** se transformă în NL (new line – **'\n'**), motiv pentru care în cazul unei succesiuni de funcţii **puts (sir)** fiecare şir se afişează pe câte un rând nou, de la începutul acestuia, astfel ca urmare a execuţiei programului

```
#include <stdio.h>
int main()
{ puts("A"); puts("ABC"); puts("ABCDE"); puts("ABCDEFG");
  return 0;
}
```

şirurile vor apărea pe ecran în felul următor :

```
A
ABC
ABCDE
ABCDEFG
```

Exemple foarte simple de utilizare a celor două funcţii se prezintă în continuare .

Primul program necesită introducerea numelui de familie al utilizatorului, urmat apoi şi de prenume

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ char name[35], surname[35]; clrscr();
  puts("\n\t Please keyboard your surname");
                                          //surname-nume de familie
  gets(surname);
  puts("\n\t Please keyboard your name"); //name   -prenume
  gets(name);
  puts("\nHello"); puts(name); puts(surname);
  getch(); return 0;
}
```

iar ca rezultat se obţine

```
Hello
Ioan
Popescu
```

dacă aşa-l cheamă pe utilizator.

Al doilea program cere introducerea anului curent, **2001**, şi a lunii, de exemplu **January**.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ char year[5], month[15]; clrscr();
  puts("\n\t Year  :"); gets(year);
  puts("\n\t Month :"); gets(month);
  putch(month[0]); putch(month[1]); putch(month[2]);
  putch(' '); putch('.'); putch(' ');
  putch(year[2]); putch(year[3]);
  getch(); return 0;
}
```

Cu funcţiile **puts** se face afişarea **Year :** respectiv **Month :** iar cu funcţiile **gets** se realizează prin introducere de la tastatură cele două şiruri, 2001 având următoarele reprezentări (în hexazecimal), conform codificării ASCII

sir	**2**	**0**	**0**	**1**	'\0'
repr. în mem.	32	30	30	31	00
nr. octet	(0)	(1)	(2)	(3)	(4)

respectiv

J	**a**	**n**	**u**	**a**	**r**	**y**	'\0'
4A	61	6E	75	61	72	79	00
(0)	(1)	(2)	(3)	(4)	(5)	(6)	(7)

Cu ajutorul funcţiei **putch** (folosită repetat) care are ca argumente caracterele nr. 0, 1 şi 2 din **month** şi caracterele 2 şi 3 din **year** se afişează :

```
Jan . 01
```

7.3 Funcţiile *scanf* şi *printf*. Specificatori de format

Cele mai utilizate funcţii de **I/O** sunt **printf** şi **scanf** , deoarece cu ajutorul acestora se poate realiza toată gama de operaţii de **I/O**, pentru caractere, şiruri şi pentru toate tipurile de date, în plus, modul cum trebuie făcută introducerea şi afişarea este sub controlul celui care a scris programul, acesta stabilind **formatul** datelor.

Ambele funcţii aparţin fişierului standardizat **stdio.h** .

Prototipurile acestor funcţii sunt :

int printf (**sir** [, **lista_argumente**]) ; int scanf (**sir**, **lista_pointeri**) ;

În cadrul **sir** pot interveni : (şiruri de) caractere care se afişează, caractere de control negrafice şi specificatori de format.

Funcţia printf(...). Pentru a afişa următorul text :

Universitatea Tehnica din Cluj-Napoca
Catedra de Mecanica si Programare

se va folosi următoarea funcţie :

```
printf("\n\t Universitatea Tehnica din Cluj-Napoca\n\t Catedra de "
       "Mecanica si Programare\n");
```

După cum se vede, şirul utilizat conţine caracterele de control **'\n'** şi **'\t'** precum şi un text care se va reproduce pe monitor, prin folosirea acestei funcţii rezultând acelaşi efect ca şi în cazul funcţiei **puts,** care ar avea ca argument pe **sir,**

puts("\n\t Universitatea Tehnica din Cluj-Napoca\n\t Catedra de "

```
"Mecanica si Programare");
```

singura deosebire constând în faptul că în cazul funcţiei **puts** se trece în mod automat pe rândul următor, după afişarea textului, fără să fie necesară utilizarea caracterului **'\n'** .

Dacă această funcţie are ca argument şi o **lista_argumente** care conţine cel puţin un element (constantă, variabilă simplă sau cu indici cu valoare numerică sau de tip caracter, şir de caractere, expresie, apel de funcţie, etc) corespunzător fiecărui element al acesteia, în cadrul **sir** trebuie să existe câte un **specificator** (descriptor) **de format**.

Aceşti specificatori de format sunt funcţie de tipul datei care se afişează,

Specificator	*Tipul datei*
%d	int
%u	unsigned int
%lu	unsigned long
%ld	long int
%x, %X	int (în hexazecimal)
%o	int (în octal)
%f	float (forma naturală)
%e	float (forma exponenţială)
%g	float (forma normală sau exponenţială)
%lf, %Lf	long double
%le, %Le	long double
%c	char
%s	şir de caractere
%p	pointer
%n	pointer de variabilă întreagă a cărei valoare este egală cu numărul de caractere citite până în acel moment.

rolul lor fiind de a preciza modul cum se realizează afişarea datelor (formatul datelor). Este obligatoriu ca numărul specificatorilor de format să fie identic cu cel al argumentelor din **lista_argumente**, existând o coincidenţă şi în ce priveşte tipul valorii care se afişează şi specificatorul de format utilizat, după cum se vede şi din următoarea secvenţă de instrucţiuni :

```
..............
int a=4567, b; float c; char d = 'x';
..............
printf("\n a = %d  b = %d\n  c = %e\n d = %c",   a,   b,   c,   d);
            (spec.f.1) (spec.f.2)   (spec.f.3)   (spec.f.4) (var.1)(var.2)(var.3)(var.4)
..............
printf("\n a = %o (oct.) = %d (zec.) = %X (hexa.)\n", a, a, a);
..............
printf("\n %c (char) = %d (zec.) = %X (hexa.)\n", d, d, d);
..............
```

Primei şi celei de-a doua variabile, **a** şi **b**, care sunt de tip **int**, le corespund specificatorii **%d**, celei de-a treia, **c**, de tip **float** îi corespunde specificatorul **%f** iar celei de-a patra, **d**, de tip **char** îi corespunde **%c** . Prin intercalarea caracterului de comandă **'\n'** tipărirea valorilor variabilelor de diferite tipuri se face pe rânduri succesive.

În cadrul celei de-a doua folosiri a funcţiei **printf** se face afişarea de trei ori a valorii

variabilei întregi **a**, sub formă octală, zecimală şi hexazecimală, folosind specificatorii **%o**, **%d** şi **%X** , obţinându-se rezultatele :

```
a = 10727 (oct.)  = 4567 (zec.) = 11D7 (hexa.)
```

Pentru ca valorile numerice întregi, exprimate în octal şi hexazecimal, să fie afişate în forma în care se scriu, conform regulilor cunoscute ale limbajului, se poate folosi caracterul **#** între **%** şi specificatorul de format. Dacă se va scrie :

```
printf("\n a = %#o = %d (zec) = %#X\n", a, a, a);
```

atunci datele se vor afişa în următoarea formă :

```
a = 010727 = 4567 (zec.) = 0X11D7
```

La a treia folosire se face afişarea unei variabile de tip **char**, cu specificatorii **%c** , **%d** şi **%X**, rezultând valorile acesteia în zecimal şi hexazecimal, conform codului ASCII,

```
x (char) = 120 (zec.) = 78 (hexa.)
```

După cum s-a precizat, fiecărei variabile din lista de variabile îi corespunde câte un specificator de format, lista de variabile fiind parcursă de la stânga la dreapta, de asemenea aceeaşi fiind şi ordinea în care se consideră specificatorii de format. Astfel, în cazul instrucţiunilor :

```
int  m=16 ;
printf("\n\n %d  %d  %d", m, m/2, m/4);
```

se vor afişa rezultatele

```
16  8  4
```

Atunci când se face afişarea unor valori numerice, caractere sau şiruri se poate preciza în mod explicit numărul de poziţii pe care se va face afişarea şi dacă e cazul şi numărul de cifre al părţii subunitare.

Dacă de exemplu se folosesc specificatorii **%6d** , **%10.2f** în cazul tipului **int** datele numerice se afişează pe şase poziţii (aliniate la dreapta) iar în cazul celor de tip **float** pe 10 poziţii, pe ultimele două fiind partea subunitară a numărului real.

Dacă se doreşte alinierea la stânga a valorilor numerice se vor folosi aceşti specificatori în felul următor: **%-6d** , **%-10.2f** .

După execuţia secvenţei de instrucţiuni :

```
.................
double v = 234.5678;
.................
printf("\n x = |%f|  x = |%13.5f|  x = |%-13.5f|",  x, x, x);
printf("\n x = |%e|  x = |%13.5e|  x = |%-13.5e|",  x, x, x);
.................
```

s-au obţinut rezultatele :

```
x = |234.567795|    x = |     234.5678|  x = |234.5678     |
x = |2.345678e+02|  x = |  2.34568e+02|  x = |2.34568e+02  |
```

observându-se că în cazul utilizării lui **%f** sau **%e** se afişează câte şase cifre după punctul zecimal iar dacă numărul de poziţii corespunzător părţii de după punctul zecimal este mai mic decât cel al

cifrelor atunci ultima cifră afişată este rotunjită.

Trebuie specificat că se poate folosi descriptorul de format de tip **%f** atât în cazul datelor de tip **float** cât şi în cazul celor de tip **double** deoarece orice argumente de tip **float** sunt în mod automat convertite în tipul **double**.

Dacă se utilizează specificatorul **%g** atunci numărul real se va afişa fie în formă naturală fie exponenţială, astfel ca să se ocupe un număr minim de poziţii.

Pentru a afişa date de tipul **long int** sau **long double** se vor folosi specificatori de tipul **%ld** respectiv **%lf**, **%le** .

În anumite situaţii specificatorii de format pot fi folosiţi într-un mod mai "general". În toate exemplele anterioare s-a specificat în mod explicit atât numărul total de poziţii pe care se face afişarea, cât şi numărul de poziţii corespunzător părţii subunitare, în cazul celor de tip **float** sau **double**, de exemplu

```
.................
int a; double b, c;
.................
printf("\n a = %5d    b = %15.6f    c = %17.8e", a, b, c);
.................
```

Există posibilitatea ca în scrierea şirului care conţine şi specificatorii de format să se utilizeze caracterul generic ***** care să înlocuiască una sau ambele valori specificate.

Astfel, instrucţiunile următoare sunt echivalente cu cele anterioare

```
.....................
int a, i, j, k, l; double b, c;
.....................
i=5; j=15; k=6; l=8;
.....................
printf(\n a = %*d   b = %*.*f   c = %17.*e", i, a, j, k, b, l, c);
.....................
```

După cum se poate observa, în acest caz primul asterisc va fi înlocuit cu valoarea lui **i**, rezultând specificatorul de format **%5d**, al doilea şi al treilea asterisc vor fi înlocuite cu valorile lui **j** şi **k**, rezultând **%15.6f** , etc.

În cazul specificatorului **%s** utilizat în cazul afişării unui şir acesta va apare în întregime. Dacă s-ar scrie **%14.8s** ar însemna că şirul se va scrie pe ultimele 8 poziţii din totalul celor 14 afectate pentru aceasta iar dacă s-ar folosi **%-14.10s** se va afişa şirul pe primele 10 poziţii. În cazul instrucţiunilor următoare

```
.................
char sir[ ] = "Programare in C/C++";
.................
printf("\n |%s|  |%14.8s|  |%-14.10s|", sir, sir, sir);
.................
```

se obţin rezultatele :

```
|Programare in C/C++|  |      Programa|  |Programare    |
```

Aceleaşi rezultate se pot obţine şi în felul următor :

```
.................
printf("\n  |%s|    |%14.8s|    |%-14.10s|",  "Programare  in  C/C++",
        "Programare in C/C++", "Programare in C/C++");
.................
```

Specificatorul de format **%n** (relativ mai rar folosit) permite determinarea numărului de caractere care au fost parcurse/afişate din şirul folosit ca argument al funcţiei **printf**, în cazul datelor reţinându-se numărul de caractere afişat, luându-se în considerare caracterele de comandă şi caracterele care se afişează direct, inclusiv caracterele ***blank***.

Ca rezultat al execuţiei următorului program

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int a, b = 2000;
  clrscr();
  printf("\n\n\n Programare in C%n/C++", &a);
  printf("\n %d", a);
  printf("\n\n %d Programare in C%n/C++", b, &a);
  printf("\n %d", a);
  getch(); return 0;
}
```

se obţin următoarele rezultate ;

```
Programare in C/C++
19


2000 Programare in C/C++
23
```

Funcţia **printf (...)** returnează o valoare întreagă, egală cu numărul de caractere care se afişează sau **-1** în caz de eroare.

Astfel, ca urmare a execuţiei programului

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int a=1376, n;  float x=0.00987;
  clrscr();
  n=printf("\n a = %6d  x = %10.6f", a, x);
  printf("\nNr. car. = %d", n);
getch(); return 0;
}
```

se obţin următoarele rezultate,

```
 a =   1376  x =   0.009870
Nr. car. = 28
```

în cadrul celor 28 de caractere incluzându-se atât caracterele de comandă cât şi blank-urile.

Funcţia scanf este utilizată pentru introducerea datelor de la tastatură, respectând formatele datelor aşa cum sunt specificate prin descriptorii utilizaţi în cadrul **sir** , având forma generală :

int scanf (**sir**, **lista_pointeri**) ;

În cadrul **sir** se pot utiliza specificatorii de format (aceiaşi ca şi la funcţia **printf**) între care pot exista blank-uri sau virgule.

În **lista_pointeri** se scriu ***adresele*** variabilelor de tip numeric sau caracter cărora li se atribuie valorile tastate, utilizând operatorul unar de adresare **&v** iar în cazul şirurilor se scrie numai **identificatorul** acestora, deoarece (se vor prezenta aceste probleme în Capitolul IX – Pointeri) sub această formă este notată adresa din memorie a primului element al şirului.

Programul următor conține o serie de instrucțiuni de **I/O**, **printf (...)** şi **scanf (...)**, prin utilizarea cărora se exemplifică modul lor de folosire în cazul diferitelor tipuri de date.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int a, b, c, d, e, f, g, h, i, j; float u, v, w;
  double x, y; char c1, c2, sir1[15], sir2[15];
  clrscr();

  printf("\n\n int a b c\n");
  scanf("%d %d %d", &a, &b, &c);       //12 23 4567
  printf("a = %d  b = %d  c = %d", a, b, c);
  //a = 12  b = 23  c = 4567

  printf("\n\n int d, e, f\n");
  scanf("%d, %d, %d", &d, &e, &f);     //12, 23, 4567
  printf("d = %d  e = %d  f = %d", d, e, f);
  //d = 12  e = 23  f = 4567

  printf("\n\n int (oct.) g, h\n");
  scanf("%o, %o, %o", &g, &h);         //120, 771
  printf("g = %o (oct.) = %d (zec.)  h = %o (oct.) = %d (zec.)",
       g, g, h, h);
  //g = 120 (oct.) = 80 (zec.)  h = 771 (oct.) = 505 (zec.)

  printf("\n\n int i, j (hex.)\n");
  scanf("%X, %X", &i, &j);             //23A, 5FE
  printf("i = %X (hex.) = %d (zec.)  j = %X (hex.) = %d (zec.)",
       i, i, j, j);
  //i = 23A (hex.) = 570 (zec.)  j = 5FE (hex.) = 1534 (zec.)

  printf("\n\n float u, v, w\n");
  scanf("%f, %f, %f", &u, &v, &w);     //32.89, -90.008, 1678.8
  printf("u = %e  v = %e  w = %e", u, v, w);
  //u = 3.289000e+01  v = -9.000800e+01  w = 1.678800e+03

        printf("\n\n double x, y\n");
  scanf("%le, %le", &x, &y);           //12.005, 90.09e-12
  printf("x = %e  y = %e", x, y);
  //x = 1.200500e+01  y = 9.009000e-11

  printf("\n\n sir1 sir2\n");          //Limbajul C/C++
  scanf("%s", sir1);
  scanf("%s", sir2);
  printf("\n%s", sir1);
  printf("\n%s", sir2);
  //Limbajul
  //C/C++

  getch(); return 0;
}
```

În comentariile din dreptul funcţiilor **scanf** sunt scrise datele în forma în care se tastează la terminal iar în comentariile care urmează după **printf** este prezentat modul cum apar rezultatele pe monitor.

În cazul introducerii datelor, dacă specificatorii de format sunt scrişi sub forma **%d %d %d** atunci şi datele numerice întregi trebuie scrise separate prin spaţii iar dacă sunt scrişi sub forma **%d, %d, %d** sau **%f, %f** atunci valorile numerice se scriu separate prin virgule.

Când se introduc valori numerice întregi scrise în sistemul octal sau hexazecimal acestea se scriu utilizând numai cifrele octale sau hexazecimale, nefiind necesar ca înaintea acestora să se scrie **0** respectiv **0X**, cum este cazul când aceste numere se scriu în cadrul instrucţiunilor programului sursă.

În cazul introducerii unui şir, la întâlnirea caracterului nul acesta se termină. Din acest motiv, în **şir1** cu 15 elemente, vor fi înregistrate caracterele 'L', ' i', 'm', 'b', 'a', 'j', 'u', 'l', '\0' în sir1[0], sir1[1], . . . , sir1[8] iar **sir2** va conţine caracterele 'C', '/', 'C', '+', '+', '\0' în sir2[0], sir2[1], ..., sir2[5].

Dacă în cadrul execuţiei următorului program

```
#include <conio.h>
#include <stdio.h>

int main (void)
{ char name[25];
  printf("\n What is your name ?\n");
  scanf("%s", name);                    // Mihai Ursu
  printf("\n Hello, %s !\n", name ) ;   // Hello, Mihai !
  getch(); return 0;
}
```

se tastează prenumele şi numele, separate printr-un spaţiu, atunci în cadrul şirului **name** se va înregistra numai prenumele.

Ca şi funcţia **printf** şi funcţia **scanf** returnează o valoare întreagă, care în acest caz este egală cu numărul câmpurilor citite corect, acest lucru exemplificându-se în programul următor :

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int a, b, c, n; char sir[25];
  clrscr();
  printf("\n Keyboard three integer numbers and a string !\n");
  n = scanf("%d, %d, %d, %s", &a, &b, &c, sir);  //1,2,3,qwerty
  printf("\n n = %d", n);                         // n = 4
  getch(); return 0;
}
```

Se poate consulta HELP-ul corespunzător mediului de programare Borland C++3.1 pentru a cunoaşte şi alte probleme legate de scrierea şirului de caractere din cadrul funcţiei **scanf**, pe "traseul" ***HELP Æ scanf Æ Format specifiers Æ Format specifiers conventions Æ Search Set Conversion .***

O problemă de o importanţă deosebită în ceea ce priveşte introducerea datelor este legată de realizarea unui ***"regim conversaţional"*** , pe monitor trebuind să apară suficiente indicaţii ca utilizatorul să nu poată greşi în această etapă.

Să comparăm următoarele două variante :

```
//  Varianta  I
#include <conio.h>
#include <stdio.h>

int main(void)
{ int a, b, c; float d; double e;
  scanf("%d, %d, %d", &a, &b, &c);
  scanf("%f, %le", &d, &e);
  ..............
  ..............
  getch( ); return 0;
}
```

```
//  Varianta II
#include <conio.h>
#include <stdio.h>

int main(void)
{ int a, b, c; float d; double e;
  printf("\n a [int]   = "); scanf("%d", &a);
  printf("\n b [int]   = "); scanf("%d", &b);
  printf("\n c [int]   = "); scanf("%d", &c);
  printf("\n d [float] = "); scanf("%f", &d);
  printf("\n e [double]= "); scanf("%le",&e);
  ..............
  ..............
  getch( ); return 0;
}
```

Se poate vedea că în primul caz introducerea datelor trebuie făcută numai după ce s-a consultat programul sursă, observând că trebuie introduse trei valori întregi, scrise pe o linie, cu virgule între ele iar pe al doilea rând trebuie scrise două valori reale, de asemenea cu o virgulă între ele.

În cazul variantei II acest lucru nu este necesar, deoarece autorul programului a prevăzut ca pe ecran să apară denumirea şi tipul datelor care vor fi introduse, deci utilizatorul trebuie să ştie doar că în cazul unei valori de tip **int** nu va folosi punctul zecimal, pe care-l va folosi însă în cazul datelor de tip **float** sau **double**.

De asemenea este indicat ca să se precizeze şi unităţile de măsură, tot cu scopul de a evita erori la introducerea datelor :

```
.....................
double masa, mom_iner, const_el ;
.....................
printf("\n Masa corpului      [kg]     = "); scanf("%le", &masa);
printf("\n Moment de inertie  [kg*m*m] = "); scanf("%le", &mom_iner);
printf("\n Constanta elastica [N/m]    = "); scanf("%le", &const_el);
.....................
```

7.4 I/O standard *cin* şi *cout* din C++

Aceste două **stream**-uri ("**stream**" – flux) au fost introduse în C++ şi ele aparţin fişierului header **iostream.h**.

Pentru a se înţelege problemele esenţiale privind noţiunea de stream se poate consulta [Negr_2] pg. 592 - 630, [Schi97] pg. 204 - 206, 416 - 447.

În cazul operaţiilor de ieşire (cu stream-ul **cout – c**onsole **out**put) se foloseşte operatorul **<<**, numit ***operator de inserare*** iar în cazul celor de intrare (cu stream-ul **cin** – **c**onsole **in**put) operatorul **>>**, numit ***operator de extragere***.

Dacă se folosesc instrucţiunile

cout << a ; cin >> b ;

acestea vor avea următorul efect : pe ecran va fi afişată valoarea variabilei **a**, modul cum apare fiind funcţie de tipul ei (**int**, **float**, **double**, **char**) iar apoi se va introduce utilizând tastatura o valoare care se va atribui lui **b**. Este evident că modul cum trebuie tastată valoarea lui **b** este de asemenea funcţie de tipul declarat al acestei variabile.

Există următoarea echivalenţă în cazul stream-ului **cout** cu cele anterior prezentate.

```
int a;
cout << a;      sau     printf("%d", a);

double b;
cout << b;      sau     printf("%g", b);

char c;
cout << c;      sau     printf("%c", c);      sau      putch(c);

char sir[ ] = "Programare"
cout << sir;    sau     printf("%s", sir);    sau      puts(sir);

double d;
cout <<'\n'<<" d = "<<d<<'\n';     sau        printf("\n d = %d\n", d);
```

Dacă s-ar folosi **cin** atunci se pot scrie echivalenţele :

```
int a;
cin >> a ;     sau    scanf("%d", &a);

double b;
cin >> b ;     sau    scanf("%le", &b);

char c;
cin >> c ;     sau  scanf("%c", &c);
```

Pentru a exemplifica utilizarea acestor stream-uri s-a elaborat programul următor, în care se fac operaţii de **I/O** pentru date de tip **int**, **double**, şiruri de caractere.

```
#include <conio.h>
#include <iostream.h>

int main(void)
{ int a=231, b, c;
  char sir1[] = "Technical University", sir2[50];
  double d=239.09887654e-1, e=239.098e-7, f;
  clrscr();
```

```
  cout << " Valoarea lui a = ";
  cout << a;
  cout << "\n Valoarea lui a = " << a ;
  cout << "\n              d = " << d ;
  cout << "\n              f = " << e ;
  cout << "\n b [int]    = ";
  cin >> b;              // Se tasteaza o valoare de tip int: 7890
  cout << " " << b;
  cout << "\n c [int]    = ";
  cin >> c;              // Se tastează o valoare de tip int: 0XAA2
  cout << " " << c;
  cout << "\n f [double] = ";
  cin >> f;              // Se tastează o valoare de tip double:
                         //               23.56e12
  cout << " " << f;
  cout << "\n Sirul 1 : " << sir1;
  cout << "\n Sirul 2 :\n";
  cin  >> sir2;          // Tastam textul următor :
                         // Department of Mechanics and Programming
  cout << "\n " << sir2;
  getch(); return 0;
}
```

Rezultatele obţinute sunt următoarele:

```
 Valoarea lui a = 231
 Valoarea lui a = 231
           d = 23.909888
           e = 2.39098e-05
 b [int]    = 7890
 7890
 c [int]    = 0XAA2
 2722
 f [double] = 23.56e12
 2.356e+13
 Sirul 1 : Technical University
 Sirul 2 :
Department of Mechanics and Programming
 Department
```

Se pot observa următoarele :

- în cazul introducerii datelor de tip **int** acestea pot fi scrise în oricare din cele trei sisteme de numeraţie (zecimal, octal sau hexazecimal),

- în cazul datelor de tip **double** afişate sub formă naturală apar şase cifre după punctul zecimal iar în cazul formei "exponenţiale" se face o afişare cu şase cifre semnificative, de asemenea după punctul zecimal,

- în cazul introducerii unui şir de la tastatură acesta se consideră terminat când apare caracterul blank.

Capitolul VIII

Instrucţiunile limbajului C

8.1 Generalităţi.

Instrucţiunile unui program controlează execuţia acestuia. Numai în cazuri cu totul particulare ordinea de execuţie a tuturor instrucţiunilor este cea **secvenţială**. De obicei numai anumite grupuri de instrucţiuni se execută secvenţial, altele se execută în **mod repetat** iar altele **alternativ**.

Pe lângă aceste trei modalităţi (secvenţială, repetitivă şi alternativă) mai este permisă şi structura **selectivă**.

În ceea ce priveşte structura repetitivă, aceasta poate fi condiţionată ***anterior*** sau ***posterior***.

Instrucţiunile se pot clasifica, conform standardului **ANSI C**, în următoarele grupe :

- instrucţiuni ***expresie***,
- instrucţiuni ***compuse*** (bloc),
- instrucţiuni de ***selecţie*** (if, if - else, if - else - if, switch),
- instrucţiuni de ***salt*** (goto, break, continue, return),
- instrucţiunile de ***ciclare*** (for, while, do - while),
- instrucţiuni ***etichetă*** (case, default).

Instrucţiunile sunt formate din cuvinte cheie, expresii şi alte instrucţiuni. Cuvintele cheie folosite în construcţia instrucţiunilor sunt următoarele:

break	**case**	**continue**	**default**	**do**	**else**
for	**goto**	**if**	**return**	**switch**	**while**

Instrucţiunea **return** va fi prezentată în cadrul Capitolului X - **Funcţii**.

8.2 Instrucţiunea expresie, instrucţiunea vidă şi instrucţiunea compusă (bloc).

Instrucţiunea expresie este o **expresie** urmată de semnul **;**

expresie ;

iar acestea pot fi atribuiri sau apeluri de funcţii, cum ar fi:

```
.............
y = (x*x - 100)/(x*x + 5) ;
k++ ;
```

```
.............
scanf(" %le", &z);
puts(" The first program in C/C++") ;
line ( 24,35, 345,78) ;
clrscr() ;
..............
```

Se poate deci observa că în limbajul C semnul **;** este un terminator de instrucţiune. În exemplele prezentate instrucţiunile au fost scrise fiecare separat pe o linie, lucru care face ca programul să fie mai uşor de urmărit şi de înţeles, dar care nu este obligatoriu (cum este de exemplu în limbajul Fortran). Ordinea în care se face evaluarea unei expresii este cea dată de ordinea de prioritate a diferiţilor operatori din componenţa acesteia. Pentru a respecta ordinea dorită se vor folosi parantezele, numărul cărora (deschise respectiv închise) este limitat la 32.

Instrucţiunea vidă este de forma

;

fiind necesară în acele situaţii când din punct de vedere sintactic trebuie într-un anumit loc să existe o instrucţiune, care însă nu are nici o importanţă din punct de vedere al logicii programului. Aceste situaţii pot fi întâlnite la instrucţiunile de ciclare, în special în cazul instrucţiunii **for**. O altă posibilă folosire a instrucţiunii vide poate fi întâlnită la efectuarea salturilor, când instrucţiunea care se etichetează şi la care se face saltul poate fi vidă (vezi instrucţiunea **goto**).

Instrucţiunea compusă (blocul) reprezintă o reuniune de declaraţii şi instrucţiuni de orice tip închise între două acolade.

A fost necesară introducerea în limbaj a acestui tip de instrucţiune deoarece în multe situaţii când din punct de vedere sintactic se poate folosi o singură instrucţiune, din punct de vedere al modului de soluţionare al algoritmului este necesar un număr mai mare de instrucţiuni, însoţite în prealabil eventual şi de declaraţii "locale" de tip.

Forma generală este următoarea:

```
{
  [ declaratie_1; ] [ declaratie_2 ; ] .... [ declaratie_k ; ]
  instructiune_1 ;
  [ instructiune_2 ; ] [ instructiune_3 ; ] .... [ instructiune_n ; ]
}
```

folosindu-se convenţia că ceea ce este cuprins între parantezele drepte **[** şi **]** este opţional.

În interiorul unei instrucţiuni compuse se pot utiliza alte instrucţiuni compuse, numărul maxim al instrucţiunilor compuse incluse una în alta fiind de 15.

8.3 Instrucţiuni de decizie şi de selecţie.

După cum s-a precizat anterior modul secvenţial de parcurgere al instrucţiunilor poate fi utilizat numai în anumite porţiuni ale programelor. În general, se întâlnesc o serie de decizii conform rezultatelor cărora se urmează o anumită cale. Uneori ca urmare a unei decizii se trece la executarea sau nu a unei instrucţiuni, alteori se execută una din cele două instrucţiuni posibile sau se pot lua în mod repetat mai multe decizii şi în mod corespunzător să se execute una din mai multe instrucţiuni existente. În aceste situaţii se vor folosi instrucţiunile **if**, **if - else** , **if - else - if**.

Este posibilă de asemenea şi utilizarea unei **structuri de selecţie** (instrucţiunea **switch**), extrem de eficientă atunci când trebuie să se aleagă între mai multe variante.

8.4 Instrucţiunile if , if - else şi if - else - if .

Instrucţiunea **if** are următoarea formă generală:

```
.................
if ( expresie )
            instructiune ;
.................
```

iar funcţionarea ei este următoarea:

- se evaluează **expresie** din paranteză,
- dacă expresia este nenulă (în cazul unei condiţii, aceasta este verificată) atunci se execută **instrucţiune**, după care se trece la instrucţiunea de pe rândul următor,
- dacă **expresie** este nulă atunci nu se execută **instrucţiune** ci se trece la instrucţiunea de pe rândul următor.

Ceea ce s-a notat prin **instrucţiune** poate fi o instrucţiune simplă sau o instrucţiune compusă (bloc).

EXEMPLUL nr. 8.1. Instrucţiunea **if** se poate întâlni în cadrul programului în care se calculează $\mathbf{C_n^k}$ cu o formulă de recurenţă, pornind de la $C_n^0 = 1$,

$$1 = C_n^0 \to C_n^1 \to .... \to C_n^{i-1} \to C_n^i = \frac{n-i+1}{i} C_n^{i-1} \to ... \to C_n^k$$

Pentru ca numărul de utilizări al formulei de recurenţă să fie minim se va avea în vedere formula cunoscută din combinatorică

$$C_n^k = C_n^{n-k}$$

deci dacă k>n/2 se va înlocui **k** cu **n-k**, în programul respectiv existând următoarea secvenţă de instrucţiuni:

```
.............
int k,n; double C;
printf ("\n\t n = " ); scanf ( "%d ", &n );
printf ("\n\t k = " ); scanf ( "%d ", &k );
if( k>n/2 )
    k = n-k;
C = 1.0;
.............
```

Se poate observa că dacă **expresie** (în acest caz **k > n/2**) este diferită de zero (adevărată) atunci se execută **instrucţiune** , adică atribuirea **k = n - k**, după care se trece la iniţializarea **C = 1**. În caz contrar, dacă **k ≤ n/2**, se va trece direct la iniţializarea **C=1**, valoarea iniţială a variabilei întregi **k** rămânând neschimbată.

Instrucţiunea **if** se poate folosi şi în situaţia în care ar trebui să se aleagă între execuţia a două instrucţiuni de atribuire.

EXEMPLUL nr. 8.2. Dacă se pune problema calculării valorii funcţiei

$$y=\begin{cases}\dfrac{x-1}{x^4+1}, & x<3,\\ x^2, & x\geq 3,\end{cases}$$

pentru o valoare dată a variabilei independente **x** se poate utiliza următorul program:

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

int main(void)
{  double x, y;
   printf ("\n\t x = ");
   scanf ("%le", &x);
   y = x*x;
   if( x<3 )
      y = (x-1)/(pow(x,4)+1);
   printf("\n\n\t y = %le", y);
   getch(); return 0;
}
```

Se poate observa că instrucţiunea de atribuire y = x*x; se execută în mod inutil atunci când x ≥ 3, motiv pentru care modul acesta de programare nu este recomandat.

Instrucţiunea **if - else.**

După cum s-a văzut, în cazul folosirii instrucţiunii **if** , ca urmare a verificării unei condiţii (**expresie** este diferită de zero sau nu) se trece sau nu la execuţia lui **instrucţiune**. În destul de multe cazuri însă trebuie luată în considerare o alternativă: dacă **expresie** este nenulă se execută **instrucţiune_1** iar în caz contrar **instrucţiune_2**.

Instrucţiunile **if - else** realizează acest lucru, forma lor generală fiind următoarea:

```
.....
if ( expresie )
        instructiune_1 ;
else
        instructiune_2 ;
.....
```

Se evaluează **expresie** din paranteză.

Dacă expresia este nenulă (în cazul unei condiţii aceasta este verificată) atunci se execută **instrucţiune_1**, după care se trece la execuţia instrucţiunii de pe rândul următor celui care conţine **instrucţiune_2**.

Dacă expresia este nulă (condiţia nu este verificată) atunci se execută **instrucţiune_2** după care se trece la instrucţiunea de pe rândul următor.

instrucţiune_1 şi **instrucţiune_2** pot fi instrucţiuni simple sau compuse.

EXEMPLUL nr. 8.3. Programul din cadrul Exemplului nr. 8.2 se poate relua şi modifica astfel ca nici o instrucţiune să nu se execute în mod inutil. Pentru aceasta se vor folosi instrucţiunile **if - else.**

```
#include <conio.h>
#include <math.h>
#include <stdio.h>
```

```
int main(void)
{  double x, y;
   printf("\n\t  x =  ");
   scanf("%le", &x);
   if( x<3 )
        y = (x-1)/(pow(x,4)+1);
   else
        y = x*x;
   printf("\n\n\t y = %le", y);
   getch(); return 0;
}
```

EXEMPLUL nr. 8.4. Se consideră integrala definită

$$v = \int_a^b x^n \, dx, \qquad 0 < a \le b, \quad n \in Z,$$

şi se cere să se calculeze aceasta, pentru valori date ale lui **a**, **b** şi **n**, care respectă condiţiile de mai sus.

Valoarea integralei se va calcula cu una din următoarele relaţii,

$$v = \begin{cases} \ln\dfrac{b}{a}, & n = -1, \\ \dfrac{b^{n+1} - a^{n+1}}{n+1}, & n \ne -1, \end{cases}$$

în funcţie de valoarea lui **n**.

În programul care urmează, după ce s-au scris funcţiile care realizează introducerea datelor, se utilizează instrucţiunea **if - else** pentru a alege formula corectă de calcul, în funcţie de valoarea expresiei **n = = - 1** care se calculează. Dacă valoarea acesteia este nenulă (atunci când n = - 1) se va folosi formula de sus iar în caz contrar (expresia are valoare nulă) se va folosi cea de-a doua formulă.

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

int main(void)
{int n; double a, b, v;
printf("\n\t\t n = "); scanf("%d", &n);
printf("\t\t a = "); scanf("%le", &a);
printf("\t\t b = "); scanf("%le", &b);
//
if( n==-1 ) v=log(b/a);
else        v=(pow(b,n+1)-pow(a,n+1))/(n+1);
//
printf("\n\n\t Lim. inf. = %8.3f  Lim. sup. = %8.3f"
       "\n\t Functia: pow(x,%d)   Val. integralei = "
       "%le\n\n", a, b, n, v);
getch(); return 0;
}
```

.

În program s-a folosit funcţia **pow(…)** care aparţine fişierului **math.h.** Cu această funcţie expresia $\mathbf{u}^{\mathbf{v}}$ se scrie sub forma **pow (u , v)** .

Observaţie . Prin folosirea operatorului condiţional ternar **? :** se poate scrie altfel secvenţa de instrucţiuni din programul anterior, cuprinsă între cele două rânduri libere, considerate comentarii :

```
//
   v = n==-1  ? log(b/a) : (pow(b,n+1)-pow(a,n+1))/(n+1);
//
```

Instrucţiunea **if - else - if**

Forma generală este următoarea:

```
.....................
if (  expresie_1  )
                    instructiune_1;
else if (  expresie_2  )
                    instructiune_2;
else if (  expresie_3  )
                    instructiune_3;
.....................
else if (  expresie_n  )
                    instructiune_n;
else
                    instructiune;
 ..............  (rândul nr. k)
```

Se începe cu evaluarea **expresie_1**.

Dacă valoarea este diferită de zero atunci se execută **instrucţiune_1** , după care se trece la execuţia instrucţiunii de pe rândul k al programului, de la sfârşitul instrucţiunii **if - else - if**.

Dacă valoarea lui **expresie_1** este nulă atunci se trece la evaluarea **expresie_2**, după care se execută **instrucţiune_2** dacă valoarea determinată a fost nenulă , urmând apoi execuţia instrucţiunii de pe rândul k al programului sau se trece la evaluarea lui **expresie_3** în caz contrar, continuând în acelaşi mod mai departe.

Dacă nici una din valorile lui **expresie_1**, **expresie_2**, … **expresie_n** nu sunt diferite de zero atunci se trece la execuţia lui **instrucţiune** care urmează după ultimul **else** .

EXEMPLUL nr. 8.5. În continuare se va prezenta un program foarte simplu pentru calculul valorii funcţiei

$$y=\begin{cases} x^3+1, & x<-10, \\ x+100, & -10\le x<2, \\ \dfrac{x-9}{x-5}, & 2\le x<15, \\ \dfrac{x-100}{x^4+8}, & x\ge 15, \end{cases}$$

pentru un **x** dat. Deoarece există diferite formule de calcul pe diferite intervale este necesar să se verifice mai multe condiţii în mod succesiv, utilizând instrucţiunile **if - else - if.**

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

int main(void)
{double x, y;
printf("\n\n\t x = "); scanf("%le", &x);

if( x<-10 )
  { y=pow(x,3)+1;
    printf("\n\t x = %le  y = %le", x,y); }
else if( -10<=x && x<2 )
  { y=x+100;
   printf("\n\t x = %le  y = %le", x,y); }
else if( 2<=x && x<15 && fabs(x-5) > 1.e-15 )
  { y=(x-9)/(x-5);
    printf("\n\t x = %le  y = %le", x,y); }
else if( 15<=x )
  { y=(x-100)/(pow(x,4)+8);
    printf("\n\t x = %le  y = %le", x,y); }
else
  printf("\n\t x = %le  y undetermined", x);
getch(); return 0;
}
```

Se poate observa că a fost necesară verificarea a patru condiţii, dacă **x** aparţine unuia din intervalele $(-\infty ; -10)$, $[-10 ; 2)$, $[2 ; 15)$ şi $[15 ; \infty)$, în cazul intervalului nr. 3 verificându-se şi condiţia ca funcţia să fie definită (calculabilă), trecându-se la execuţia instrucţiunilor corespunzătoare iar dacă funcţia nu se poate calcula (x=5) se trece la execuţia instrucţiunii care urmează după **else** simplu, neurmat de **if**.

EXEMPLUL nr. 8.6. Un exemplu sugestiv privind aplicarea instrucţiunilor **if - else - if** este programul următor, unde se atribuie variabilei **c** de tip caracter o valoare tip caracter iar în continuare se verifică dacă aceasta este o cifră, o literă (mică sau mare) sau nu aparţine acestor categorii, tipărindu-se textul corespunzător.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ char c;
  printf("Press a character key:"); c=getch();
  printf("\n Character %c is ", c);
  if ( c >='0' && c<= '9' )
     puts("a digit. ");
  else if ( c>='a' && c<='z' )
           puts("a lowercase letter.");
  else if ( c>='A' && c<='Z' )
           puts("an uppercase letter.");
  else
```

```
            puts("not a digit or letter.");          // 129
  getch(); return 0;
}
```

Instrucţiunile **if** imbricate

În afară de formele anterior prezentate sunt posibile şi situaţiile prezentate mai jos, în care există instrucţiuni imbricate ("suprapuse"), în număr de maximum opt,

```
if ( expresie_1)
      { if ( expresie_2 )
            instructiune_1;
        else
            instructiune_2;
      }
```

```
if( expresie_1 )
      { if( expresie_2 )
          instructiune_1;
      }
      else
          instructiune_2;
```

EXEMPLUL nr. 8.7. Se consideră un sistem de forţe , înregistrate în raport cu un sistem cartezian **Oxyz**. În urma reducerii acestui sistem în raport cu punctul **O**, originea sistemului cartezian, se obţine vectorul rezultant **R** (***the resultant force***) şi vectorul moment rezultant $\mathbf{M_O}$ (***the resultant moment***), care se calculează cu următoarele formule matriceale:

$$R = \begin{bmatrix} R_x \\ R_y \\ R_z \end{bmatrix} = \sum_{i=1}^{n} F_i = \begin{bmatrix} \sum_{i=1}^{n} F_{ix} \\ \sum_{i=1}^{n} F_{iy} \\ \sum_{i=1}^{n} F_{iz} \end{bmatrix},$$

$$M_O = \begin{bmatrix} M_{Ox} \\ M_{Oy} \\ M_{Oz} \end{bmatrix} = \sum_{i=1}^{n} \begin{bmatrix} 0 & -z_i & y_i \\ z_i & 0 & -x_i \\ -y_i & x_i & 0 \end{bmatrix} \begin{bmatrix} F_{ix} \\ F_{iy} \\ F_{iz} \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^{n} (y_i F_{iz} - z_i F_{iy}) \\ \sum_{i=1}^{n} (z_i F_{ix} - x_i F_{iz}) \\ \sum_{i=1}^{n} (x_i F_{iy} - y_i F_{ix}) \end{bmatrix},$$

unde cu $\mathbf{x_i}$, $\mathbf{y_i}$, $\mathbf{z_i}$ s-au notat coordonatele punctelor de aplicaţie a forţelor. Se presupune că aceste calcule s-au efectuat şi sunt cunoscute componentele lui **R** şi $\mathbf{M_O}$ pe cele trei axe.

Normele euclidiene (modulele vectorilor) ale acestor două matrice coloană se calculează cu formulele:

$$\|R\| = \sqrt{R_x^2 + R_y^2 + R_z^2}, \qquad \|M_O\| = \sqrt{M_{Ox}^2 + M_{Oy}^2 + M_{Oz}^2},$$

iar expresia aşa numitului **trinom invariant** este următoarea:

$$T = R_x M_{Ox} + R_y M_{Oy} + R_z M_{Oz} .$$

În funcţie de valorile obţinute ale lui $\|\mathbf{R}\|$, $\|\mathbf{M_O}\|$ şi **T** se pot distinge următoarele cazuri de reducere ale unui sistem de forţe:

- $\|\mathbf{R}\| = 0$, $\|\mathbf{M_O}\| = 0$; sistemul de forţe este în ***echilibru,***
- $\|\mathbf{R}\| = 0$, $\|\mathbf{M_O}\| \neq 0$; sistemul de forţe se reduce la un ***cuplu***,

- $\|\mathbf{R}\| \neq 0$, $\|\mathbf{M_O}\| = 0$; sistemul de forţe se reduce la o ***rezultantă unică***, reducerea este canonică, ***O este pe axa centrală***,

- $\|\mathbf{R}\| \neq 0$, $\|\mathbf{M_O}\| \neq 0$, $\mathbf{T} = 0$; sistemul de forţe se reduce la o ***rezultantă unică***, reducerea nu este canonică, ***O nefiind pe axa centrală***,

- $\|\mathbf{R}\| \neq 0$, $\|\mathbf{M_O}\| \neq 0$ şi $\mathbf{T} \neq 0$, sistemul de forţe se reduce la un ***torsor***, cu ***ambele elemente nenule***.

Se poate observa că în cel de-al cincilea caz există două posibilităţi: dacă componentele lui $\mathbf{R}$ sunt proporţionale cu cele ale lui $\mathbf{M_O}$, fiind valabile relaţiile,

$$\begin{cases} R_y M_{Oz} - R_z M_{Oy} = 0, \\ R_z M_{Ox} - R_x M_{Oz} = 0, \\ R_x M_{Oy} - R_y M_{Ox} = 0, \end{cases}$$

atunci punctul **O** este pe axa centrală (torsorul se numeşte **torsor minimal**) iar dacă proporţionalitatea nu este îndeplinită atunci punctul **O** este în afara axei centrale.

Programul ce se prezintă în continuare va realiza afişarea unuia din cele cinci texte posibile privind cazurile de reducere, în funcţie de verificarea condiţiilor.

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

int main(void)
{double M, Mx, My, Mz, R, Rx, Ry, Rz, T;
clrscr();
printf("\n\n The resultant force components :");
printf("\n\t Rx = "); scanf("%le", &Rx);
printf("\t Ry = "); scanf("%le", &Ry);
printf("\t Rz = "); scanf("%le", &Rz);
printf("\n The resultant moment components :");
printf("\n\t Mox = "); scanf("%le", &Mx);
printf("\t Moy = "); scanf("%le", &My);
printf("\t Moz = "); scanf("%le", &Mz);
R=sqrt(Rx*Rx+Ry*Ry+Rz*Rz); M=sqrt(Mx*Mx+My*My+Mz*Mz);
T=Rx*Mx+Ry*My+Rz*Mz;
printf("\n\n |R| = %le  |Mo| = %le  T = %le\n\n",R, M, T);

if( R==0 )
  { if( M==0 )
      printf("\n\t System in equilibrium\n");
    else
      printf("\n\t The simplest resultant is a couple."
                "\n\t The screwdriver has no force.\n");
  }
else { if( M==0 )
         printf("\n\t The simplest resultant is a force."
                "\n\t The point  O  is on the screwdriver line.\n");
       else {  if( T==0 )
```

```
          printf("\n\t The simplest resultant is"
                 " a force.\n\t  The point O is "
                 "not on the screwdriver line\n");
             else
                printf("\n\t General case.\n\t The"
                       " screwdriver has the both "
                       "elements different from zero.\n");
           }
      }
getch(); return 0;
}
```

Se pot observa cu uşurinţă instrucţiunile **if - else - if** imbricate, scrise cu bold.

8.5 Instrucţiunea de selecţie switch.

Limbajul de programare C/C++ admite o instrucţiune de selecţie cu ramuri multiple. Forma generală este următoarea:

```
.............
switch ( expresie )
{
    case const_1:  instructiune_1;
                   break;
    case const_2:  instructiune_2;
                   break;
 ..................................
   case const_n:  instructiune_n;
                   break;
  default:         instructiune;
}
.............
```

unde ***const_1, const_2, ... , const_n*** sunt constante, întregi sau de tip caracter, ***instrucţiune_1, instrucţiune_2, ..., instrucţiune_n*** sunt instrucţiuni simple sau secvenţe de instrucţiuni (instrucţiuni compuse) .

Funcţionarea este următoarea:

- se evaluează ***expresie***, care poate fi de tip **char** sau **int**,

- valoarea obţinută se compară pe rând cu valorile constantelor ***const_1, const_2, ... , const_n*** . Dacă aceasta coincide cu valoarea ***const_i*** atunci se trece la execuţia instrucţiunii simple sau a secvenţei de instrucţiuni ***instrucţiune_i*** , după care se trece la execuţia instrucţiunii care urmează după acolada instrucţiunii **switch**, datorită faptului că a urmat instrucţiunea **break**. Dacă valoarea lui ***expresie*** nu coincide cu nici una din constantele ***const_i*** atunci se execută ***instrucţiune*** care urmează după keyword-ul **default** şi se trece la execuţia instrucţiunii care urmează după acolada instrucţiunii **switch**.

În cazul în care lipseşte instrucţiunea **break** de după ***instrucţiune_i*** atunci se trece la execuţia următoarei instrucţiuni ***instrucţiune_i+1***. În cazul când nu există nici o instrucţiune

break atunci după execuţia ***instrucţiune_i*** se vor executa pe rând toate cele care urmează după aceasta.

Din cadrul formei generale prezentate poate lipsi

default: instructiune;

caz în care dacă ***expresie*** nu coincide cu nici una din ***const_1, const_2, ... , const_n*** instrucţiunea **switch** nu are nici un efect.

Se poate observa că dacă în cazul instrucţiunii **if,** anterior prezentate, decizia se lua în funcţie de faptul că ***expresie*** este diferită de zero sau nu, în acest caz se trece la execuţia unei ***instrucţiune_i*** numai în cazul când se obţine o ***coincidenţă*** a valorii expresiei cu a constantei care urmează după **case**.

Dacă în cadrul unei instrucţiuni **switch** se folosesc constante caracter atunci acestea în mod automat vor fi convertite la tipul **int**.

Este permis ca după o instrucţiune **case** să nu urmeze nici o instrucţiune sau secvenţă de instrucţiuni.

Există de asemenea posibilitatea de a folosi instrucţiuni **switch** imbricate,

```
.............
switch ( expresie_1 )
{  case const_1:  instructiune_1;
                  break;
   case const_2:  switch ( expresie_2 )
                  {  case: const_21:  instructiune_21;
                                      break;
                     case: const_22:  instructiune_22;
                                      break;
                     default: instructiune_2;          }
   ..................................
   case const_n:  instructiune_n;
                  break;
   default:       instructiune;    }
   .............
```

EXEMPLUL nr. 8.8. Cu programul următor se poate afişa înălţimea celor mai înalte clădiri din lume (existente în anul 2001, după 11 septembrie).

În urma introducerii uneia din valorile întregi 1, 2, 3, 4, 5, 6 sau 7 şi a atribuirii acestei valori variabilei **n,** ca urmare a folosirii instrucţiunii **switch** se alege tipărirea unuia din cele şapte texte, corespunzătoare celor mai înalte şapte clădiri din lume iar dacă lui **n** i se atribuie o altă valoare atunci se afişează textul corespunzător instrucţiunii **printf** care urmează după instrucţiunea **default,** text care conţine lista următoarelor clădiri şi construcţii, în ordinea descrescătoare a înălţimii acestora.

```
//  The highest buildings in the world
//
#include <conio.h>
#include <stdio.h>
```

```
int main(void)
{int n; clrscr();
 printf("\n\n\t The first seven highest buildings in “
        ”the world (in 2001)");
 printf("\n\n\t  n = "); scanf("%d", &n);
 switch ( n )
   { case 1 :
       printf("\n\t First: Petronas-452 m; Kuala Lumpur-Malaysia");
       break ;
     case 2 :
       printf("\n\t Second: Sears Tower-442 m; Chicago - USA");
       break ;
     case 3 :
       printf("\n\t Third: Jin Mao Building-421 m; Shanghai");
       break ;
     case 4 :
       printf("\n\t Fourth: Empire State Building-381 m; New York");
       break ;
     case 5 :
       printf("\n\t Fifth: Central Plaza-374 m; Hong Kong");
       break ;
     case 6 :
       printf("\n\t Sixth: Bank of China Building-369 m;”
              ” Hong Kong");
       break ;
     case 7 :
       printf("\n\t Seventh: Post & Communications”
              ” Building-365 m; Xiamen-China"); break ;
     default :
       printf("\n\t  8 - Tuntex Tower     (348 m) –“
              ” Taiwan\n\t 9 - Amoco Building   “
              ”(346 m) - Chicago\n\t 10 - Shun Hung”
              ” Square (325 m) - Shentzen, China\n\t”
              ” 11 - Tour Eiffel      (320 m) - Paris");
       break ;
     }
 getch(); return 0;
}
```

În cadrul programului prezentat la Exemplul nr. 8.8 alegerea secvenţei de instrucţiuni se făcea în urma comparării unei **valori întregi** cu valoarea constantei (de asemenea întregi) care urma după instrucţiunea **case**.

În legătură cu acest program se poate observa că dacă s-ar renunţa la toate instrucţiunile **break**, acest lucru va avea următorul efect : în urma tastării unui anumit număr, de exemplu 3, vor fi afişate toate informaţiile începând cu clădirea care este a treia ca înălţime.

În anul 2005 acestui program va trebui să i se facă o corectură : cea mai înaltă clădire va fi de **465 m** (cu 107 etaje) şi se va găsi în portul **Pusan** din Coreea (de sud).

Cea de-a patra clădire ca înălţime, existentă în 2001 (World Trade Center din New York – 415 m) a fost distrusă în 11 septembrie 2001 ca urmare a unui atac terorist.

EXEMPLUL nr. 8.9. Programul care urmează utilizează de asemenea instrucţiunea de selecţie **switch**, deosebirea faţă de cel anterior constând în faptul că ***expresie*** este de tip **char** iar constantele folosite sunt de asemenea de tip **char** .

```
#include <conio.h>
#include <stdio.h>

int main(void)
{char c; double a, b, max, min;
 printf("\n\n\t a = "); scanf("%le", &a);
 printf("\t b = "); scanf("%le", &b);
 printf("\n\n To find max. value - press  M"
        "\n To find min. value - press  m"
        "\n To find average    - press  A");
 c=getch();
 switch ( c )
 { case 'M' :
       max = a<b ? b : a ;
        printf("\n\t The maximum value is: %le", max);
        break ;
   case 'm' :
       min = a<b ? a : b ;
        printf("\n\t The minimum value is: %le", min);
        break ;
   case 'A' :
       printf("\n\t The average is: %le", (a+b)/2 );
        break;
   default :
       printf("\n\t Mistake. Select among  M, m or A !");
 }
 getch(); return 0;
}
```

În cadrul programului se face introducerea a două valori (reale) care se atribuie variabilelor **a** şi **b** iar în urma introducerii unuia din caracterele **M**, **m** sau **A** se afişează valoarea maximă, minimă sau media celor două variabile. Dacă se introduce un alt caracter atunci va apărea un mesaj de eroare.

Pentru calculul valorilor maxime respectiv minime s-a folosit operatorul condiţional ternar **? :** .

EXEMPLUL nr. 8.10. În cadrul celor două exemple anterioare 8.8 şi 8.9 a fost folosită instrucţiunea **break** după fiecare parcurgere a ***instrucţiune_i.*** Programul care urmează va conţine o singură instrucţiune **break**.

Se numerotează zilele săptămânii cu cifre de la 1 la 7 (Luni - 1,…, Duminică - 7). Programul va realiza următoarele: dacă se va tasta o valoare între 1 şi 7 va apare lista denumirilor zilelor săptămânii începând cu ziua respectivă sau un mesaj de eroare în caz contrar. Programul este următorul:

```
#include <conio.h>
#include <stdio.h>
```

```
int main(void)
{ int z;
  printf("\n\t z  [ 1, …, 7] = "); scanf("%d", &z);
  switch( z )
  { case 1 : puts("Monday");               // Luni
    case 2 : puts(" Tuesday");             // Marti
    case 3 : puts("  Wednesday");          // Miercuri
    case 4 : puts("   Thursday");          // Joi
    case 5 : puts("    Friday");           // Vineri
    case 6 : puts("     Saturday");        // Sambata
    case 7 : puts("      Sunday"); break ; // Duminica
    default : puts(" E r r o r !");
  }
  getch(); return 0;
}
```

Trebuie reamintit faptul că în cazul utilizării funcţiei **puts(…)** în mod automat se adaugă la sfârşitul şirului caracterul **\n ,** motiv pentru care denumirile zilelor săptămânii vor apărea pe ecran una ***sub*** alta.

EXEMPLUL nr. 8.11. S-a precizat faptul că este posibil ca după unele constante de la **case** să nu apară instrucţiuni.

Cu programul care urmează se afişează numele unui oraş (Bucureşti, Cluj-Napoca, Timişoara sau Iaşi) în funcţie de caracterul care se tastează, **B** sau **b**, **C** sau **c**, **T** sau **t** respectiv **I** sau **i**.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ char c; c=getchar();
  switch( c )
  {  case 'B' : case 'b' :
            printf("\n\t Bucharest"); break;
    case 'C' : case 'c' :
            printf("\n\t Cluj-Napoca"); break;
    case 'T' : case 't' :
            printf("\n\t\Timisoara"); break;
    case 'I' : case 'i' :
            printf("\n\t Iasi"); break ;
  }
  getch(); return 0;
}
```

Deoarece nu există în cadrul acestui program **default: instrucţiune** rezultă că dacă s-a introdus un alt caracter decât cele precizate atunci instrucţiunea **switch** este inoperantă.

8.6 Instrucţiunea break.

Forma generală a acestei instrucţiuni este:

break ;

O primă utilizare a instrucţiunii **break** s-a făcut în cadrul instrucţiunii **switch,** rolul acesteia fiind de a realiza ieşirea din structura selectivă, în momentul întâlnirii acesteia şi continuarea execuţiei programului de la prima instrucţiune care urmează după **switch { …. }**.

De asemenea, instrucţiunea **break** se mai foloseşte şi pentru a părăsi corpul unor cicluri, în urma verificării unei anumite condiţii.

Această a doua modalitate de folosire se va prezenta în cadrul programării structurilor repetitive utilizând instrucţiunile **for**, **while** şi **do - while**.

8.7 Instrucţiunea goto.

Instrucţiunea **goto** întrerupe execuţia secvenţială a instrucţiunilor unui program C/C++ realizând trecerea (forţată) de la linia pe care este poziţionată la o altă instrucţiune aparţinând aceleiaşi funcţii în care se găseşte **goto**, instrucţiune marcată cu o ***etichetă,***

Forma generală a acestei instrucţiuni de salt necondiţionat este următoarea:

```
.....
label:
 ..... // (rândul i)
 .....
 .....
goto label; // (rândul k)
.....
```

unde **label** este un identificator care este urmat de **:** şi reprezintă o ***etichetă*** care marchează instrucţiunea scrisă pe rândul **i.**

În cazul întâlnirii instrucţiunii **goto** (scrisă pe rândul nr. **k**, de exemplu) se trece la execuţia instrucţiunii de pe rândul **i**, prefixată prin eticheta scrisă după **goto**. Utilizarea acestei instrucţiuni **goto** este necesară în special atunci când se doreşte părăsirea unor cicluri suprapuse. Aceste situaţii se vor exemplifica în continuare, odată cu prezentarea instrucţiunilor **for**, **while** şi **do-while**.

O altă situaţie ar fi atunci când trebuie reluată execuţia programului pentru un alt set de valori iniţiale, în acest sens a se vedea Exemplul nr. 8.12.

De asemenea instrucţiunea **goto** s-ar putea utiliza pentru realizarea programării ciclurilor, împreună cu instrucţiunea **if**, dar acest lucru trebuie evitat deoarece există alte posibilităţi mult mai eficiente pentru aceasta .

EXEMPLUL nr. 8.12. În programul următor se calculează, în mod repetat, media aritmetică a două numere reale **a** şi **b**, după fiecare introducere a celor două valori, calculul şi afişarea mediei, utilizatorul fiind întrebat dacă doreşte să continue.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{char c;  double a, b, A; clrscr();

 LAB:
 printf("\n\t a = "); scanf("%le", &a);
 printf("\t b = ");   scanf("%le", &b);
 A=(a+b)/2;
 printf("\t Average = %le \n\n\t\t\t Do you want "
        "to continue ?  (Y, y) \n\n ", A);
 c=getch();
 if(c=='Y' || c=='y') goto LAB ;
 getch(); return 0;
}
```

După cum se poate observa, dacă se tastează **Y** sau **y,** unul din aceste caractere 'Y' sau 'y' fiind atribuit variabilei **c** de tip **char**, atunci expresia din paranteza lui **if** va avea valoarea 1 şi se va trece la execuţia lui **goto LAB,** deci se va relua introducerea datelor.

8.8 Funcţia exit.

Funcţia **exit** se foloseşte pentru a realiza ieşirea dintr-un program. Prototipul acestei funcţii este următorul:

```
void exit ( int  cod_eroare );
```

Dacă programul s-a terminat normal atunci **cod_eroare** va avea valoarea 0, deci această funcţie se va folosi sub forma:

```
exit ( 0 );
```

iar în cazul unei terminări anormale **cod_eroare** va avea o valoare diferită de zero, de exemplu

```
exit ( 1 );
```

Exemple de utilizare a acestei funcţii se vor găsi în unele din programele următoare. În cazul folosirii acestei funcţii trebuie inclusă directiva de preprocesare

#include <stdlib.h>

această funcţie aparţinând fişierului ***stdlib.h*** .

8.9 Instrucţiuni de iteraţie.

Deoarece în majoritatea problemelor de calcul care se rezolvă cu calculatorul intervin relaţii sau grupuri de relaţii care trebuie parcurse în mod repetat, de fiecare dată pentru alte valori ale unor variabile din componenţa acestora, variabilele respective modificându-şi valorile după anumite legi, în toate limbajele de programare s-au introdus o serie de instrucţiuni speciale pentru a realiza programarea acestor cicluri.

În limbajul C/C++ există trei astfel de instrucţiuni:

for **while** şi **do - while**

Se poate reaminti că în cazul unui ciclu porţiunea corespunzătoare de program trebuie să realizeze următoarele patru obiective :

- o iniţializare a variabilei care conduce ciclul,
- parcurgerea repetată a instrucţiunii sau a instrucţiunii compuse,
- o verificare a unei condiţii în funcţie de care se decide sau nu continuarea calculelor în ciclu,
- o modificare, după o anumită lege (de cele mai multe ori conform unei progresii aritmetice) , a valorii variabilei care conduce ciclul.

În funcţie de faptul că verificarea condiţiei se cere să fie făcută **înainte** sau **după** parcurgerea corpului ciclului (a instrucţiunii simple sau compuse) se pot folosi fie instrucţiunile **for** şi **while** în primul caz respectiv **do - while** în al doilea.

8.10 Instrucţiunile for şi continue.

Forma generală a instrucţiunii **for** este următoarea:

```
.....
for ( [ expresie_1 ] ; [ expresie_2 ] ; [ expresie_3 ] )
      instructiune ;
..... // (rândul i)
```

unde ***instrucţiune*** (care poate fi simplă sau compusă) este executată în mod repetat până când ***expresie_2*** este diferită de zero.

Înaintea primei iteraţii este evaluată ***expresie_1*** în cadrul căreia se face iniţializarea variabilelor utilizate în cadrul ciclului.

Se evaluează ***expresie_2***, dacă aceasta este diferită de zero atunci se execută ***instrucţiune*** iar în caz contrar (***expresie_2*** are valoare nulă) se trece la execuţia instrucţiunii de pe rândul următor (notat cu rândul **i**)

După fiecare parcurgere a corpului ciclului din cadrul lui ***instrucţiune*** se evaluează ***expresie_3*** care se foloseşte de obicei pentru a realiza reiniţializarea (modificarea) variabilei care conduce ciclul.

Conform convenţiei, faptul că ***expresie_1*** , ***expresie_2*** , ***expresie_3*** sunt scrise între parantezele drepte [şi] are semnificaţia că aceste elemente sunt opţionale, deci poate exista această instrucţiune şi sub forma:

```
for ( ; ; )
```

În cazul când lipseşte ***expresie_2*** se consideră că valoarea acesteia este 1, deci ciclul s-ar putea executa de o infinitate de ori.

Utilizarea instrucţiunii de ciclare **for** se va face în special atunci când se lucrează cu şiruri, matrice şi în alte cazuri când se cunosc valorile iniţiale, finale şi pasul de variaţie al variabilelor care conduc ciclul.

Astfel, pentru a realiza calculul sumei primilor 100 de termeni ai unui şir x[0], x[1], ..., x[99] se pot folosi instrucţiunile:

```
.....
for( i=0, sum=0.0 ; i<100 ; i++ ) sum+=x[i];
.....
```

EXEMPLUL nr. 8.13. Unul din cele mai simple exemple de utilizare a instrucţiunii de ciclare **for** este în cadrul unui program de calcul a valorilor unei funcţii, de exemplu $y = 2.1x^3 + 52.34x^2 - 12.5$, pentru toate valorile variabilei **x** dintr-un interval [**a**, **b**], parcurs cu un pas **h**.

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

int main(void)
{ double a, b, h, x;
  printf("\n\n\t a = "); scanf("%le", &a);
  printf("\t b = "); scanf("%le", &b);
  printf("\t h = "); scanf("%le", &h);
  for( x=a ; x<=b ; x+=h )
      printf("\t\t x = %le   y = %le\n",
             x, 2.1*pow(x,3)+52.34*x*x-12.5);
  getch(); return 0;
}
```

Instrucţiunea **for** utilizată a fost de forma:

```
for ( x=a ; x<=b ; x+=h )
    printf(.......);
```

observându-se că în acest caz simplu ***instrucţiune*** este o instrucţiune simplă.

Dacă valorile numerice ale lui **a**, **b** şi **h** au fost astfel alese încât numărul de linii pe care se afişează rezultatele este mic (max. 25) atunci valorile lui **x** şi **y,** prezentate pe două coloane, vor rămâne pe ecran, în caz contrar vor rămâne pe ecran numai ultimele 25 de linii.

EXEMPLUL nr. 8.14. Pentru a remedia acest impediment va trebui ca în programul anterior să se facă o serie de completări (scrise cu **caractere îngroşate**).

```
int main(void)
{int k; double a, b, h, x;
 printf("\n\n\t a = "); scanf("%le", &a);
 printf("\t b = "); scanf("%le", &b);
 printf("\t h = "); scanf("%le", &h);

 for( x=a, k=1; x<=b; x+=h, k++ )
    { printf("\t\t %d  x = %le   y = %le\n",
             k, x, 2.1*pow(x,3)+52.34*x*x-12.5);
     if(k%22==0) getch();
    }
 getch(); return 0;
}
```

Ca urmare, se va mai folosi o variabilă **k** care se va utiliza pentru a număra liniile pe care se afişează datele, iniţializarea **k=1** introducându-se în cadrul ***expresie_1*** iar incrementarea **k++** în cadrul ***expresie_3.*** Cu ajutorul instrucţiunii

```
if ( k % 22 == 0 )  getch( ) ;
```

inclusă în ***instrucţiune*** se realizează oprirea calculării şi afişării datelor până la apăsarea unei taste, după fiecare grup de 22 de linii afişate.

Se poate de asemenea observa că în acest caz ***instrucţiune*** este o instrucţiune compusă.

EXEMPLUL nr. 8.15. Să se elaboreze programul cu care se vor afişa pe ecran caracterele **litere mari** şi **litere mici**, pe rânduri separate.

Se va utiliza o variabilă **c** de tip **char** care va lua toate valorile între **'A'** şi **'Z'**, cu pasul unitar, fiecare afişându-se la distanţă de un caracter după cel anterior. Acest lucru se realizează cu prima instrucţiune de ciclare **for** , în mod asemănător făcându-se şi afişarea literelor mici :

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ char c; clrscr();
  printf("\n\tThe capital letters :\n\n");
  for(c='A'; c<='Z'; c++) printf(" %c", c);
  printf("\n\n\n\tThe lower letters :\n\n");
  for(c='a'; c<='z'; c++) printf(" %c", c);
  getch(); return 0;
}
```

Se poate observa că în cazul ambelor instrucţiuni **for** corpul ciclurilor este alcătuit dintr-o singură instrucţiune .

Dacă se presupun cunoscute valorile în zecimal a codificării ASCII pentru caracterele **'A'** , **'Z'** , **'a'** şi **'z'** atunci s-ar putea utiliza următorul program :

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int k; clrscr();
  printf("\n\n\n\tThe capital letters :\n\n");
  for(k=65; k<=90; k++) printf(" %c", k);
  printf("\n\n\n\tThe lower letters :\n\n");
  for(k=97; k<=122; k++) printf(" %c", k);
  getch(); return 0;
}
```

Următoarele exemple sunt programe care efectuează operaţii cu şiruri de date numerice: însumarea elementelor şirului şi calculul produsului acestora, calculul unor medii (valoare medie, valoare medie absolută, valoare medie patratică, valoare efectivă, eroare medie patratică), determinarea valorii şi poziţiei elementului maxim (în mod similar se rezolvă şi problema pentru minim), ordonarea descrescătoare a termenilor unui şir.

EXEMPLUL nr. 8.16. Un exemplu simplu de utilizare a instrucţiunii **for** se poate găsi în programul următor, unde se calculează suma şi produsul elementelor unui şir de numere reale.

```
#include <conio.h>
#include <stdio.h>
```

```
int main(void)
{ int i, n;  double A[50], sum=0, prod=1;
  printf("\n\n\t The number of elements   n = "); scanf("%d", &n);

  // First cycle      (Primul ciclu)
  for( i=0 ; i<n ; i++ )
    { printf("\n\t A[%2d] = ",i); scanf("%le",&A[i]); }

  // Second cycle          (Al doilea ciclu)
  for( i=0 ; i<n ; i++ )
    { sum+=A[i]; prod*=A[i]; }

  printf("\n\n\t\t  Sum = %le  Prod = %le", sum, prod);
  getch(); return 0;
}
```

Instrucţiunea **for** a fost folosită de două ori, odată pentru a programa introducerea datelor (a elementelor şirului) iar a doua oară pentru a realiza operaţiile de însumare şi înmulţire, în mod repetat. Este evident că s-ar fi putut folosi un singur ciclu, în care să se facă atât introducerea datelor cât şi calculele necesare :

```
  ..................
  for( i=0 ; i<n ; i++ )
    { printf("\n\t A[%2d] = ",i); scanf("%le",&A[i]);
      sum+=A[i]; prod*=A[i]; }
  ..................
```

EXEMPLUL nr. 8.17. Calculul mediilor şi a deviaţiilor utilizate în prelucrarea statistică a şirurilor de date se face cu următoarele formule:

- valoarea medie ("arithmetic mean", "average" – l. engleză) şi valoarea medie absolută,

$$v_m = \frac{\sum_{i=1}^{n} a_i}{n}, \qquad v_A = \frac{\sum_{i=1}^{n} |a_i|}{n},$$

- valoarea medie patratică şi valoarea efectivă ("root mean square"),

$$\overline{v^2} = \frac{\sum_{i=1}^{n} a_i^2}{n}, \qquad v_{RMS} = \sqrt{\frac{\sum_{i=1}^{n} a_i^2}{n}},$$

- media valorilor absolute a deviaţiilor ("mean absolute deviation"),

$$v_{MAD} = \frac{\sum_{i=1}^{n} |a_i - v_m|}{n},$$

- eroarea medie patratică sau deviaţia standard ("standard deviation"),

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(v_m - a_i)^2} = \sqrt{v_{RMS}^2 - v_m^2}$$

Programul va conţine mai multe cicluri succesive, necesare pentru introducerea datelor, calculul unor sume (numărătorii de la valoarea medie, valoarea medie absolută şi valoarea medie patratică) iar apoi a sumei valorilor absolute a deviaţiilor :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

int main(void)
{ int i, n;
  double a[100],d,er,s1=0,s2=0,s3=0,vA,vm,vp,vMAD,vRMS;
  printf("\n\n\t The number of elements n = "); scanf("%d",&n);

  for(i=0; i<n; i++)
     { printf("\t\t A[%2d] = ", i); scanf("%le", &a[i]); }

  for(i=0; i<n; i++)
     { s1+=a[i]; s2+=fabs(a[i]); s3+=a[i]*a[i]; }

  vm=s1/n; vA=s2/n; vp=s3/n; vRMS=sqrt(vp);
  printf("\n\t Average (arithmetic mean)= %le"
         "\n\t Mean of absolute values  = %le"
         "\n\t Average of square values = %le"
         "\n\t Root mean square value   = %le", vm,vA,vp,vRMS);

  for(s1=0, i=0; i<n; i++) s1+=fabs(a[i]-vm);
  vMAD=s1/n; er=sqrt(vRMS*vRMS-vm*vm);
  printf("\n\t Mean absolute deviation  = %le"
         "\n\t Standard deviation       = %le", vMAD,er);

  getch();  return 0;
}
```

În continuare se prezintă rezultatele obţinute în cazul datelor introduse

```
 The number of elements = 8
                A[ 0] = 12.03
                A[ 1] = 10.14
                A[ 2] = 11.67
                A[ 3] = 13.08
                A[ 4] = 12.08
                A[ 5] = 13.04
                A[ 6] = 12.7
                A[ 7] = 13.05
           Average (arithmetic mean)= 1.222375e+01
           Mean of absolute values  = 1.222375e+01
           Average of square values = 1.502970e+02
           Root mean square value   = 1.225957e+01
           Mean absolute deviation  = 7.437500e-01
           Standard deviation       = 9.364686e-01
```

EXEMPLUL nr. 8.18. Determinarea valorii şi poziţiei elementului maxim (minim) dintr-un şir este una din problemele "clasice" de programare, care nu poate lipsi din nici un

manual de acest gen. Trebuie precizat că o astfel de problemă apare de obicei ca parte componentă în multe alte probleme mai complicate.

Iniţial se consideră ca maxim (notat cu **v_max**) chiar primul element, se parcurge şirul şi se compară valoarea fiecărui element cu valoarea curentă a maximului. Dacă un element este mai mare decât **v_max** atunci valoarea lui **v_max** se modifică, atribuindu-i-se valoarea elementului respectiv iar lui **i_max** valoarea indicelui acestui element. În mod similar se rezolvă şi problema determinării minimului **v_min** şi a poziţiei acestuia **i_min**. La început, înainte de a începe parcurgerea şirului se va considera că

v_min = v_max = A[0] i_min = i_max = 1 .

În cazul prelucrării statistice a unui şir de date, la care s-au determinat **v_max** şi **v_min**, se poarte calcula **intervalul** ("the range"), care este diferenţa dintre mărimea maximă şi cea minimă din şir :

r = v_max – v_min .

Programul este următorul :

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int i, i_max, i_min, n; double A[50], v_max, v_min;
  printf("\n\n\t The number of elements   n = ");
  scanf("%d", &n);
  //  First loop
  for( i=0 ; i<n ; i++ )
     { printf("\n\t A[%2d] = ",i); scanf("%le",&A[i]); }
  //  Second loop
  v_min=v_max=A[0]; i_min=i_max=1;
  for( i=0 ; i<n ; i++ )
     { if(v_max < A[i] ) { v_max=A[i]; i_max=i; }
       if(v_min > A[i] ) { v_min=A[i]; i_min=i; }
     }
  printf("\n\t  v_min = A[%2d] = %le", i_min, v_min);
  printf("\n\t  v_max = A[%2d] = %le", i_max, v_max);
  printf("\n\t  The range = %le", v_max-v_min);
  getch(); return 0;
}
```

EXEMPLUL nr. 8.19. Ordonarea descrescătoare (crescătoare) a termenilor unui şir se poate face prin mai multe metode. Una dintre acestea, aşa numita "**bubble sorting**" , va fi prezentată în continuare. Dacă se consideră şirul format din **n = 5** elemente reale

2.1 -4.0 23.9 15.9 0.2

acesta va fi parcurs de mai multe ori, luând câte două elemente vecine şi comparând valorile acestora : dacă $\mathbf{a}_i \geq \mathbf{a}_{i+1}$ atunci valorile elementelor nu se schimbă între ele deoarece respectiva pereche de elemente are valori descrescătoare iar în caz contrar se vor schimba între ele valorile elementelor. Faptul că s-a făcut un schimb de valori a două elemente vecine va fi înregistrat prin atribuirea valorii **1** unei variabile **k**, a cărei valoare iniţială este **0** şi va rămâne bineînţeles tot **0** în absenţa schimburilor.

Manual, operaţiile vor fi efectuate în următoarea ordine:

Prima parcurgere a şirului (k=0):

2.1	-4.0	23.9	15.9	0.2	(nu se schimbă valorile)
2.1	-4.0	23.9	15.9	0.2	(se schimbă, k=1)
2.1	23.9	-4.0	15.9	0.2	(se schimbă, k=1)
2.1	23.9	15.9	-4.0	0.2	(se schimbă, k=1)
2.1	23.9	15.9	0.2	-4.0	k = 1

A doua parcurgere a şirului (k=0):

2.1	23.9	15.9	0.2	-4.0	(se schimbă, k=1)
23.9	2.1	15.9	0.2	-4.0	(se schimbă, k=1)
23.9	15.9	2.1	0.2	-4.0	(nu se schimbă valorile)
23.9	15.9	2.1	0.2	-4.0	(nu se schimbă valorile)
23.9	15.9	2.1	0.2	-4.0	k = 1

A treia parcurgere a şirului (k=0):

23.9	15.9	2.1	0.2	-4.0	(nu se schimbă valorile)
23.9	15.9	2.1	0.2	-4.0	(nu se schimbă valorile)
23.9	15.9	2.1	0.2	-4.0	(nu se schimbă valorile)
23.9	15.9	2.1	0.2	-4.0	(nu se schimbă valorile)
23.9	15.9	2.1	0.2	-4.0	k = 0

Este evident faptul că deoarece la a treia parcurgere (în acest caz) a şirului nu s-a făcut nici un schimb de valori la elementele vecine înseamnă că şirul obţinut după parcurgerea anterioară este ordonat iar indicatorul **k** a rămas cu valoarea **0** atribuită înainte de a începe parcurgerea şirului.

În cazul unui şir care a fost ordonat (crescător sau descrescător) se poate găsi **mediana**, adică valoarea elementului aflat la mijlocul şirului ordonat, dacă numărul elementelor acestuia este impar, respectiv media aritmetică a celor două elemente de la mijlocul şirului dacă numărul de elemente este par :

$$mediana = \begin{cases} A[k], & k=\dfrac{n-1}{2}, \quad n-impar, \\ \dfrac{A[k]+A[k+1]}{2}, & k=\dfrac{n-2}{2}, \quad n-par, \end{cases}$$

considerându-se că primul element al şirului are indicele 0 : A[0], A[1], ... , A[n-1] .

Programul este următorul:

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int i, k, n;  double A[50], median, t;
  printf("\n\n\t The number of elements   n = ");
  scanf("%d", &n);
  //  First loop
  printf("\n\n\t\t The initial string\n");
  for( i=0 ; i<n ; i++ )
     { printf("\n\t A[%2d] = ", i); scanf("%le",&A[i]); }
  // **********************
```

```
  //  Second loop
  label:
  k=0;
  for( i=0 ; i<n-1 ; i++ )
     { if(A[i]<A[i+1])
       { t=A[i]; A[i]=A[i+1]; A[i+1]=t; k=1; }
     }
  if(k != 0) goto label;
  // **********************
  //  Third loop
  printf("\n\n\t\t The final form of the string in"
         " descending order\n");
  for( i=0 ; i<n ; i++ )
     printf("\n\t A[%2d] = %le", i, A[i]);

  if(n%2 != 0) { k=(n-1)/2; median = A[k]; }
  else { k=(n-2)/2; median=(A[k]+A[k+1])/2; }
  printf("\n\n\n\t The median = %le", median );
  getch(); return 0;
}
```

În cadrul acestui program s-au folosit instrucţiunile **for** de două ori (pentru introducerea datelor şi pentru parcurgerea şirului), instrucţiunea **if** , de asemenea de două ori (pentru a verifica dacă **A[i]<A[i+1]** şi valoarea lui k obţinută după parcurgerea şirului), precum şi instrucţiunea **goto** , necesară pentru a relua iniţializarea **k=0** şi parcurgerea şirului.

Pentru determinarea imparităţii sau parităţii lui **n** s-a verificat condiţia **n%2 != 0**.

Instrucţiunea **continue** se poate folosi numai în cadrul corpului unui ciclu, când ca urmare a verificării unei condiţii trebuie să se treacă la execuţia următoarei iteraţii, abandonând execuţia iteraţiei curente. Se va utiliza în cadrul instrucţiunii **for** cât şi în celorlalte două instrucţiuni de ciclare (**while** şi **do - while**) care urmează să fie prezentate.

Astfel, dacă ar trebui calculată suma şi produsul termenilor pozitivi ai unui şir se poate folosi una din secvenţele următoare de instrucţiuni:

```
.............                  .............
s_poz=0; p_poz=1;              s_poz=0; p_poz=1;
for( i=0 ; i<n ; i++ )            for( i=0 ; i<n ; i++ )
   { if(A[i]>0)                      { if(A[i]<= 0) continue;
      {s_poz+=A[i];                    s_poz+=A[i]; p_poz*=A[i];
       p_poz*=A[i];}                 }
   }                           .............
.............
```

Se poate observa (la instrucţiunile din dreapta) că în cazul în care A[i] este negativ sau nul atunci se va trece la următorul termen al şirului, la următoarea iteraţie, ca urmare a execuţiei instrucţiunii **continue**.

Programarea ciclurilor suprapuse (imbricate) cu instrucţiunea **for** .
Forma generală este următoarea:

```
……..
for ( expresie_11 ; expresie_12 ; expresie_13 )
   { ……..
     for ( expresie_21 ; expresie_22 ; expresie_23 )
        { ……..
          for ( expresie_31 ; expresie_32 ; expresie_33 )
             { ……..
               ……..
              }
            ……..
           }
        ……..
     }
 ……...
```

Conform standardului **ANSI C** numărul maxim admis de cicluri suprapuse este **15**. O multitudine de operaţii, dintre care cele mai uzuale sunt cele cu **matrice**, utilizează doi sau mai mulţi indici, care iau valori între anumite limite bine precizate, pentru rezolvarea unor astfel de probleme fiind necesară folosirea ciclurilor suprapuse.

EXEMPLUL nr. 8.20. Una din cele mai frecvente operaţii care trebuie efectuate atunci când se lucrează cu matrice este introducerea elementelor acestora, în cadrul unui regim conversaţional. Dacă matricea **A** are dimensiunile **m** (linii) şi **n** (coloane), atunci pe ecran ar trebui să apară în mod repetat următoarele date (sunt reprezentate bold), urmând ca utilizatorul să introducă datele numerice (sunt reprezentate cu italic):

……..
A [val_i] [val_j] = *val_elem_*$a_{i,j}$
……..

Secvenţa de instrucţiuni care realizează aceste operaţii este prezentată în continuare

```
..............
int i, j, m, n, ..... ;
double  A[20][20], v, ..... ;
..............
//
//  The input of the matrix dimensions  m, n  and of the elements
//  A[0][0], A[0][1], ….. , A[m-1][n-2], A[m-1][n-1]
//
printf("\n\t  m (number of rows)    = "); scanf ("%d", &m) ;
printf("\n\t  n (number of columns) = "); scanf ("%d", &n ) ;
for( i=0 ; i<m ; i++ )
  { for( j=0 ; j<n ; j++ )
       { printf("\n  A[%2d][%2d] = ",i,j);
         scanf("%le", &v); A[i][j] = v;
       }
  }
```

```
. . . . . .
```

EXEMPLUL nr. 8.21. Pentru a face suma a două matrice $\mathbf{A_{m,n}}$ şi $\mathbf{B_{m,n}}$ şi a obţine elementele celei de-a treia $\mathbf{C_{m,n}}$ se va parcurge în mod repetat instrucţiunea de atribuire

$$c_{i,j} = a_{i,j} + b_{i,j}, \qquad i = \overline{1,m}, \quad j = \overline{1,n},$$

pentru toate valorile indicelui de linie **i** între **1** şi **m** şi a indicelui de coloană **j** între **1** şi **n**.

Dacă se face calculul elementelor matricei **C** în ordinea liniilor iar fiecare linie se parcurge începând cu elementul din stânga, se poate observa că indicele **i** va trebui să aibă o valoare constantă, începând cu **1**, apoi **2, 3… m**, în timp ce se parcurge linia respectivă iar pentru a trece de la un element la altul (pe linie) indicele de coloană **j** va trebui să ia valori de la **1** la **n**.

Rezultă deci că va fi nevoie de două cicluri suprapuse, cel exterior condus de indicele de linie **i** iar în interiorul acestuia va fi un alt ciclu, condus de indicele **j**.

Formula de calcul a elementelor matricei sumă s-a scris ca şi în cursurile de matematici. În cazul când se va trece la programarea în C/C++ se va ţine seama că prima valoare a indicilor este **0** şi nu **1**.

Suma a două matrice se calculează în urma executării următoarelor instrucţiuni:

```
. . . . . .
//
// The sum of two matrices A[m][n]+B[m][n] --> C[m][n]
//
for( i=0 ; i<m ; i++ )
  { for( j=0 ; j<n ; j++ ) C[i][j]=A[i][j]+B[i][j];
  }
. . . . . .
```

EXEMPLUL nr. 8.22. În cazul produsului a două matrice $\mathbf{A_{m,n}}$ şi $\mathbf{B_{n,l}}$, elementele matricei produs $\mathbf{C_{m,l}}$ se calculează cu formula :

$$c_{i,j} = \sum_{k=1}^{n} a_{i,k}\, b_{k,j}, \qquad i = \overline{1,m}, \quad j = \overline{1,l},$$

pentru parcurgerea elementelor matricei produs **C** cu **m** linii şi **l** coloane fiind necesare două cicluri, cel exterior fiind condus de variabila întreagă **i** (indicele de linie) iar cel interior de variabila întreagă **j** (indice de coloană).

În plus, pentru fiecare combinaţie de valori a indicilor **i** şi **j** trebuie calculată valoarea elementului $\mathbf{c_{i,j}}$, care rezultă printr-o însumare de produse, deci mai este necesar încă un ciclu, al treilea, plasat cel mai interior.

S-au scris cele trei matrice conform schemei lui **Falk**, [Falk51], observându-se că elementul $\mathbf{c_{i,j}}$ se obţine ca urmare a însumării produselor elementelor de pe linia **i** a matricei **A** cu elementele de pe coloana **j** a matricei **B**

$$\begin{bmatrix} a_{1,1} & a_{1,2} & . & . & . & a_{1,n} \\ a_{2,1} & a_{2,2} & . & . & . & a_{2,n} \\ . & . & . & . & . & . \\ a_{i,1} & a_{i,2} & . & a_{i,k} & . & a_{i,n} \\ . & . & . & . & . & . \\ a_{m,1} & a_{m,2} & . & . & . & a_{m,n} \end{bmatrix} \cdot \begin{bmatrix} b_{1,1} & b_{1,2} & . & . & b_{1,j} & . & . & b_{1,l} \\ b_{2,1} & b_{2,2} & . & . & b_{2,j} & . & . & b_{2,l} \\ . & . & . & . & . & . & . & . \\ . & . & . & . & b_{k,j} & . & . & . \\ . & . & . & . & . & . & . & . \\ b_{n,1} & b_{n,2} & . & . & b_{n,j} & . & . & b_{n,l} \end{bmatrix} = \begin{bmatrix} c_{1,1} & c_{1,2} & . & . & . & . & . & c_{1,l} \\ c_{2,1} & c_{2,2} & . & . & . & . & . & c_{2,l} \\ . & . & . & . & . & . & . & . \\ . & . & . & . & c_{i,j} & . & . & . \\ . & . & . & . & . & . & . & . \\ c_{m,1} & c_{m,2} & . & . & . & . & . & c_{m,l} \end{bmatrix}$$

Formula de calcul a elementelor matricei produs precum şi scrierea elementelor celor trei matrice sub forma schemei lui Falk s-a făcut şi în acest caz ca şi în cursurile de matematici. Atunci când se va trece la scrierea programului se va ţine seama că prima valoare a indicilor este **0**.

Secvenţa de instrucţiuni este următoarea:

```
//
// The product of two matrices A[m][n]*B[n][l] --> C[m][l]
//
.............
int i, j, k, l, m, n, .... ;
double A[15][15], B[15][15], C[15][15], ...... ;
.............
for( i=0 ; i<m ; i++ )
    { for( j=0 ; j<l ; j++ )
         { C[i][j]=0;
           for( k=0 ; k<n ; k++ ) C[i][j]+=A[i][k]*B[k][j];
         }
    }
.............
```

Cele trei cicluri suprapuse se mai pot scrie mai simplu, renunţând la acolade şi introducând o atribuire în cadrul instrucţiunii **for** din interior :

```
.............
for(i=0;i<m;i++)
   for(j=0;j<l;j++)
      for(k=0,C[i][j]=0.0;k<n;k++) C[i][j]+=A[i][k]*B[k][j];
.............
```

EXEMPLUL nr. 8.23. O altă operaţie foarte des întâlnită este afişarea unei matrice, existentă în orice program C/C++ care lucrează cu matrice. În cele ce urmează se prezintă secvenţa de instrucţiuni care realizează afişarea pe ecran a unei matrice de dimensiuni relativ mici (numărul de coloane este de maximum 6),

```
...........
int i, j, m, n, .......;
double A[20][20], .....;
............
//    The output of the matrix  A  with m rows and n columns
printf("\n\n");
for( i=0 ; i<m ; i++ )
   { for( j=0 ; j<n ; j++ ) printf("%12.4le", A[i][j]);
     printf("\n");
   }
............
```

Primul ciclu **for** este folosit pentru a trece de la o linie la alta a matricei **A**. În cadrul celui de-al doilea ciclu (interior) se face afişarea elementelor liniei **i**, până la terminarea acestora. În corpul ciclului exterior existând şi funcţia **printf("\n")** aceasta va comanda trecerea la rândul următor, realizându-se astfel afişarea matricei, linie după linie, dacă $n \le 6$. Dacă $n>6$ atunci o linie a matricei se va afişa pe mai multe rânduri pe ecran sau se poate modifica specificatorul de format, în loc de **%12.4le** folosind de exemplu **%10.3le** .

EXEMPLUL nr. 8.24. În Capitolul II s-au prezentat o serie de exemple în care elementele unor matrice rezultau ca urmare a utilizării unor formule de recurenţă.

O astfel de situaţie se întâlneşte şi atunci când pe liniile succesive ale matricei se găsesc coeficienţii unor polinoame "ortogonale", după următoarea regulă: **m[i][j]** este coeficientul cu care este multiplicată variabila $\mathbf{x}^{\mathbf{j}}$ de la polinomul de gradul **i.**

Se cere să se determine elementele unei matrice care să conţină pe linii coeficienţii tuturor ***polinoamelor lui Cebâşev***, până la gradul 11 inclusiv.

Polinoamele lui Cebâşev se definesc în felul următor :

$$T_n(x) = \cos(n \arccos x), \qquad |x| \le 1,$$

primele două polinoame având expresiile :

$$T_0(x) = 1, \quad T_1(x) = x.$$

Aceste polinoame ale lui Cebâşev se numesc ortogonale deoarece îndeplinesc condiţia :

$$\int_{-1}^{1} \frac{1}{\sqrt{1-x^2}} T_m(x)\, T_n(x)\, dx = \begin{cases} 0, & m \ne n, \\ \dfrac{\pi}{2}, & m = n \ne 0 \\ 0, & m = n = 0. \end{cases}$$

Ţinând cont de identitatea trigonometrică

$$\cos(n+1)\varphi \; + \; \cos(n-1)\,\varphi \; = 2 \cos\varphi \; \cos n\varphi$$

şi înlocuind

$$\varphi = \arccos x$$

rezultă o relaţie între trei polinoame Cebâşev de grade diferite, care se va putea scrie sub forma,

$$T_{n+1}(x) + T_{n-1}(x) = 2\, x\, T_n(x),$$

obţinând astfel o formulă de recurenţă :

$$T_{n+1}(x) = 2\,x\,T_n(x) - T_{n-1}(x),$$

care va putea fi utilizată pentru a determina expresiile polinoamelor începând cu cele de gradul doi:

$$\begin{aligned}
T_2(x) &= -1 + 2\,x^2,\\
T_3(x) &= -3\,x + 4\,x^3,\\
T_4(x) &= 1 - 8\,x^2 + 8\,x^4,\\
T_5(x) &= 5\,x - 20\,x^3 + 16\,x^5,\\
&\ldots\ldots\ldots
\end{aligned}$$

Se va putea astfel începe completarea elementelor matricei care conţine pe linii coeficienţii acestor polinoame:

$$\begin{bmatrix}
1 & & & & & & \\
0 & 1 & & & & & \\
-1 & 0 & 2 & & & & \\
0 & -3 & 0 & 4 & & & \\
1 & 0 & -8 & 0 & 8 & & \\
0 & 5 & 0 & -20 & 0 & 16 & \\
.. & .. & .. & .. & .. & .. & ..
\end{bmatrix}$$

Pentru a determina expresia unui anumit element **t[i][j]** al matricei, care este coeficientul lui $\mathbf{x}^{\mathbf{j}}$ din polinomul de gradul **i** se va considera relaţia de recurenţă

$$T_i(x) = 2\,x\,T_{i-1}(x) - T_{i-2}(x),$$

scrisă dezvoltat :

$$(\ldots\ldots + t_{i,j}\,x^j + \ldots\ldots) = 2x(\ldots + t_{i-1,j-1}\,x^{j-1} + \ldots) - (\ldots + t_{i-2,j}\,x^j + \ldots)$$

de unde va rezulta relaţia de recurenţă căutată între coeficienţii polinoamelor lui Cebâşev:

$$t_{i,j} = 2\,t_{i-1,j-1} - t_{i-2,j} \qquad i = 2,3,\ldots,n; \quad 1 \le j \le i.$$

Se poate observa că toate elementele de pe primele două linii sunt nule, cu excepţia elementelor **t[0][0]** şi **t[1][1]** care au valori unitare, de asemenea sunt nule toate elementele de deasupra diagonalei principale.

Particularizând relaţia de recurenţă dedusă pentru elementele primei coloane, rezultă

$$t_{i,0} = -t_{i-2,0}, \qquad i = 2,4,6,8,\ \ldots$$

În continuare se prezintă programul, în cadrul căruia s-au utilizat în mod repetat instrucţiunile de ciclare **for** simple şi imbricate (suprapuse).

```
#include <conio.h>
#include <math.h>
```

```
#include <stdio.h>

int main(void)
{ int t[20][20], i, j, n=12;
  for(i=0; i<n; i++)
     for(j=0; j<n; j++) t[i][j]=0.0;
  t[0][0]=t[1][1]=1;
  for(i=2; i<n; i+=2) t[i][0]=-t[i-2][0];
  for(i=2; i<n; i++)
     for(j=1; j<=i; j++) t[i][j]=2*t[i-1][j-1]-t[i-2][j];
  clrscr();
  printf("\n\t Definition :  T{n}(x) ="
         " cos [ n * arccos ( x ) ]\n");
  printf("\n\t Recurrence formula for Chebyshev's polynomials :"
        "\n\t\t T{0}(x) = 1 \t T{1}(x) = x"
        "\n\t\t T{i}(x) = 2*x*T{i-1}(x) - T{i-2}(x)   i>1\n");
  printf("\n\t Recurrence formula for the coefficients :"
        "\n\t\t t[i][j]=2*t[i-1][j-1]-t[i-2][j]"
         "    i >= j > 0 \n");
  printf("\n\t\t     The Chebyshev's polynomial coefficients\n\n");
  printf("Degree");
  for(i=0; i<n; i++) printf("   t%2d", i);
  printf("\n\n");
  for(i=0; i<n; i++)
     { printf(" %2d  ", i);
       for(j=0; j<n; j++) printf(" %5d", t[i][j]);
       printf("\n");
     }
  getch(); return 0;
}
```

Rezultatele obţinute cu acest program sunt următoarele :

Definition : T{n}(x) = cos [n * arccos (x)]

Recurrence formula for Chebyshev's polynomials :
T{0}(x) = 1 T{1}(x) = x
T{i}(x) = 2*x*T{i-1}(x) - T{i-2}(x) i>1

Recurrence formula for the coefficients :
t[i][j]=2*t[i-1][j-1]-t[i-2][j] i >= j > 0

The Chebyshev's polynomial coefficients

Degree	C 0	C 1	C 2	C 3	C 4	C 5	C 6	C 7	C 8	C 9	C10	C11
0	1	0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0	0
2	-1	0	2	0	0	0	0	0	0	0	0	0
3	0	-3	0	4	0	0	0	0	0	0	0	0
4	1	0	-8	0	8	0	0	0	0	0	0	0
5	0	5	0	-20	0	16	0	0	0	0	0	0
6	-1	0	18	0	-48	0	32	0	0	0	0	0
7	0	-7	0	56	0	-112	0	64	0	0	0	0
8	1	0	-32	0	160	0	-256	0	128	0	0	0
9	0	9	0	-120	0	432	0	-576	0	256	0	0
10	-1	0	50	0	-400	0	1120	0	-1280	0	512	0
11	0	-11	0	220	0	-1232	0	2816	0	-2816	0	1024

Observaţie. Într-un mod asemănător se pot calcula şi coeficienţii polinoamelor ortogonale ale lui **Hermite**, ştiind că :

$$H_0(x) = 1\,, \quad H_1(x) = 2\,x\,,$$
$$H_i(x) = 2\,x\,H_{i-1}(x) - 2(i-1)\,H_{i-2}(x), \quad i = 2,3,4,.....$$

Propunem cititorului să determine relaţia de recurenţă între elementele matricei în care se pot înregistra coeficienţii polinoamelor lui Hermite şi efectuarea unor modificări în programul anterior pentru a-l utiliza la determinarea acestor coeficienţi.

8.11 Instrucţiunea while.

Sintaxa acestei instrucţiuni este:

```
.....................
while ( expresie )
      instructiune ;
..................... // linia k
```

unde **instrucţiune** este o instrucţiune simplă sau compusă (bloc).

Funcţionarea este următoarea: se evaluează **expresie**, dacă aceasta este nenulă atunci se execută **instrucţiune** după care se revine la evaluarea **expresie** din paranteza instrucţiunii **while**. În cazul când se obţine o valoare nulă pentru **expresie** atunci se trece la următoarea instrucţiune, scrisă pe linia **k**.

Rezultă ca verificarea condiţiei se face **înainte** de fiecare execuţie a corpului ciclului.

Se poate observa că instrucţiunea **while** este echivalentă cu următoarea instrucţiune **for** :

```
for ( ; expresie ; )
   { instructiune ; }
```

utilizând instrucţiuni vide pe locul **expresie_1** şi **expresie_3** de la forma generală a instrucţiunii **for**.

Invers, dacă se doreşte înlocuirea instrucţiunii **for**, de forma generală

```
.....................
for( expresie_1 ; expresie_2 ; expresie_3 )
   { instructiune ; }
.....................
```

cu instrucţiunea **while**, aceasta va fi de forma:

```
.....
expresie_1 ;
while ( expresie_2 ; )
        { instructiune ;
          expresie_3 ;
        }
.....
```

EXEMPLUL nr. 8.25. Pentru a calcula suma şi produsul primilor **n** termeni ai unui şir A[0], A[1], ... , A[n-1] se poate folosi următoarea secvenţă de instrucţiuni :

```
...........
i=0; sum=0; prod=1.0;
while( i<n )
     { sum+=A[i]; prod*=A[i]; i++; }
...........
```

Se poate observa că până când este îndeplinită condiţia **i<n** se vor parcurge cele trei instrucţiuni componente ale instrucţiunii compuse.

EXEMPLUL nr. 8.26. În multe situaţii corectitudinea datelor introduse trebuie verificată.

De exemplu, dacă într-un program trebuie calculate valorile unei funcţii **f(x)** pentru **x** luând valori în intervalul **[a , b]** , care este conţinut în intervalul **[-100.0; 150.0],** astfel încât **-100≤a** şi **a<b≤150**, se va putea folosi de două ori instrucţiunea **while** , prima dată pentru a verifica dacă **a** este cuprins în intervalul **[-100.0 ; 150.0]** iar a doua oară pentru a verifica dacă **b** este cuprins între **a** şi **150.0** .

Acest lucru este realizat în următoarea secvenţă de instrucţiuni.

```
...........
printf("\n\n\t The interval [a,b] belongs to"
       " [-100.0; 150.0]");
a=b=-1000;

while( a < -100 || a > 150 )
     { printf("\n\t a = "); scanf("%le", &a); }

while( b<=a || b > 150 )
     { printf("\n\t b = "); scanf("%le", &b); }
...........
```

EXEMPLUL nr. 8.27. Dacă s-ar relua Exemplul nr. 8.19, unde se rezolvă problema ordonării descrescătoare a elementelor unui şir, porţiunea cuprinsă între cele două linii de comentariu s-ar putea scrie într-un alt mod, renunţând la instrucţiunea de salt necondiţionat **goto** prin utilizarea instrucţiunii **while** :

```
//  ************
k=1;
while(k == 1)
    { for(k=0,i=0; i<n-1; i++)
         { if(A[i]<A[i+1]){ t=A[i];A[i]=A[i+1];A[i+1]=t;k=1; }
         }
    }
```

```
//  *************
```

EXEMPLUL nr. 8.28. Utilizând instrucţiunea **while** se poate face introducerea unui număr oarecare de valori (reale de exemplu) până la întâlnirea unui anumit caracter, valori cu care să se facă anumite operaţii, de exemplu calculul mediei aritmetice şi determinarea valorii efective $\mathbf{v_{RMS}}$ pentru elementele unui şir .

Se reaminteşte faptul că valoarea returnată de funcţia **scanf(…)** este un număr întreg, egal cu numărul câmpurilor (zonelor) citite corect.

```
#include <conio.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{int n=0; double s1=0, s2=0, t, v, v_RMS; clrscr();
 printf("\n\t Enter the numbers !\n");
 while( scanf("%le", &t) )
      { n++; s1+=fabs(t); s2+=t*t; }
 if(n==0) exit(1);
 v=s1/n; v_RMS=sqrt(s2/n);
 printf("\n\t There were introduced %d numbers", n);
 printf("\n\t Average of abs. values = %le"
        "\n\t v_RMS                  = %le", v, v_RMS);
 getch(); return 0;
}
```

În cazul în care în loc de o valoare numerică de tip întreg sau real, scrisă corect, se introduce un alt caracter sau eventual o valoare numerică scrisă incorect (23,89 de exemplu) expresia din paranteza de la **while** devine nulă şi ***instrucţiune*** nu se mai execută.

EXEMPLUL nr. 8.29. În cadrul unor probleme cu caracter tehnic se pune problema determinării deplasărilor, vitezelor şi acceleraţiilor elementelor unor mecanisme, în funcţie de poziţia unui anumit element conducător (motor). Unul din cele mai cunoscute şi utilizate mecanisme este mecanismul manivelă - bielă, reprezentat în figura 8.1.

Pe baza figurii se poate deduce expresia spaţiului $\mathbf{s_1}$

$$s_1 = r\cos\varphi + \sqrt{b^2 - (r\sin\varphi + e)^2}\ ,$$

iar apoi, prin derivare în raport cu timpul, expresiile vitezei şi acceleraţiei,

$$v = \frac{d\,s_1}{d\,t} = -r\,\omega\left[\ \sin\varphi + \frac{(r\sin\varphi + e)\cos\varphi}{\sqrt{b^2 - (r\sin\varphi + e)^2}}\right],$$

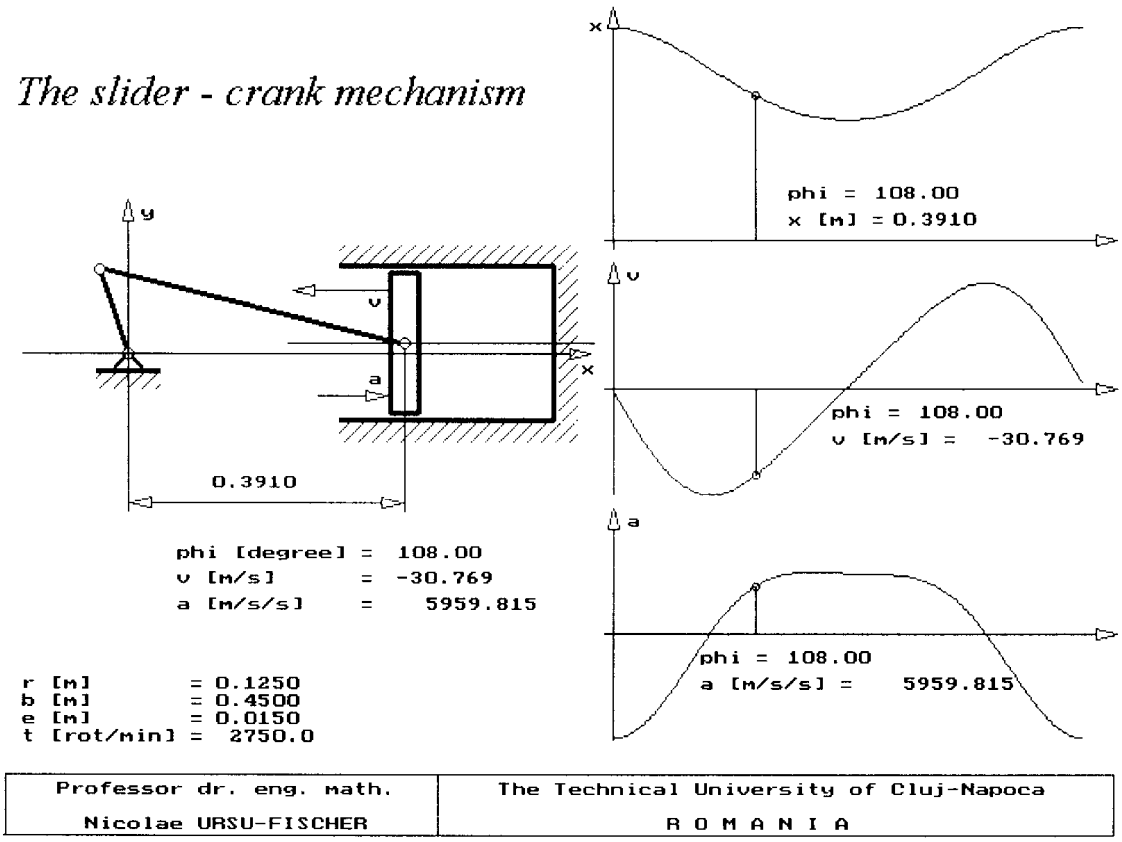


Figura 8.1

$$a = -r\omega^2\left\{\cos\varphi + \frac{\left[b^2-(r\sin\varphi+e)^2\right]\left[r\cos^2\varphi-(r\sin\varphi+e)\sin\varphi\right]+ +r\cos^2\varphi(r\sin\varphi+e)^2}{\left[b^2-(r\sin\varphi+e)^2\right]^{3/2}}\right\}$$

considerând că manivela face o mişcare de rotaţie cu viteză unghiulară constantă,

$$\varphi = \omega\, t.$$

Dacă se calculează şi spaţiul notat cu $\mathbf{s}_2$, măsurat de la ***punctul mort interior***, expresia acestuia este

$$s_2 = s_{1\max} - s_1 = \sqrt{(r+b)^2 - e^2} - s_1,$$

$\mathbf{s}_{1\max}$ obţinându-se atunci când manivela şi biela sunt în prelungire.

Un astfel de mecanism se numeşte mecanism manivelă-bielă excentric, în cazul celui obişnuit, când $\mathbf{e} = 0$, evident că formulele se vor simplifica.

Se pune problema de a determina valorile deplasării, vitezei şi acceleraţiei pistonului, pentru toate valorile unghiului φ corespunzătoare unei rotaţii complete.

Datele iniţiale ale problemei sunt : **r, b, e** [mm] - lungimile manivelei, bielei şi mărimea excentricităţii, **t** [rot/min] - turaţia manivelei şi **n -** numărul de puncte în care se face calculul.

Deoarece în formulele prezentate anterior unghiul φ trebuie considerat în radiani iar valorile afişate ale unghiului de rotaţie vor fi în grade, în cadrul programului se vor folosi variabilele $\varphi_{radiani}$ şi φ_{grade} , legate prin relaţia

$$\varphi_{radiani} = \varphi_{grade} \frac{\pi}{180}.$$

În programul următor se va folosi o instrucţiune de ciclare **for**, ciclul fiind condus de o variabilă întreagă, care are ca semnificaţie numărul de ordine al punctului considerat.

Observând că funcţiile şi expresiile

$$\sin\varphi, \quad \cos\varphi, \quad \sqrt{b^2-(r\sin\varphi+e)^2}$$

apar de mai multe ori în relaţiile ce se vor programa este evident faptul că la fiecare pas acestea se vor calcula în prealabil, atribuind valorile lor unor variabile.

Deoarece din punct de vedere practic lungimile **r**, **b**, **e** ce se introduc şi de asemenea turaţia **tur** trebuie să fie în anumite limite , în program s-au prevăzut mai multe instrucţiuni **while** care nu vor permite decât introducerea unor date numerice care respectă condiţiile impuse. De exemplu, în cazul mecanismelor bielă manivelă de la motoarele cu ardere internă este respectată condiţia :

$$\mathbf{3\,r < b < 5\,r},$$

turaţia este limitată superior, etc.

Programul este următorul :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

#define PI 3.1415926

int main(void)
{int i, n;
 double a,b,c,delta,e,omega,phid,phir,r,R,s,tur,v,x;
 clrscr(); r=-1; b=-1; e=-1000; tur=-1; n=-1;

while( r<=0 || r>499 )
     { printf("\n\n 0 < r < 500 [mm] = "); scanf("%le", &r); }
while( b<3*r || b>5*r )
     { printf("\t %5.1f [mm] < b < %5.1f [mm] = ", 3*r, 5*r);
       scanf("%le", &b); }
while( fabs(e)>0.25*r )
     { printf("\t\t 0.0 <= e < %5.1f [mm] = ", 0.25*r);
       scanf("%le", &e); }
while( tur<0 || tur>20000 )
     { printf("\t\t\t 0 < n < 20000 [RPM] = ");scanf("%le",&tur );}
while( n<10 || n>700 )
     { printf("\t\t\t\t The number of points  10 "
              "<= n <= 700; n = "); scanf("%d",&n); }
r/=1000; b/=1000; e/=1000; omega=PI*tur/30.0; delta=360./(n-1);
printf("\n\t     Nr. of point  Angle phi     x [m]"
       "      v [m/s]     a[m/s/s]\n");
```

```
for( i=0 ; i<n ; i++ )
    { phid=i*delta; phir=phid*PI/180;
      s=sin(phir); c=cos(phir); R=sqrt(b*b-(r*s+e)*(r*s+e));
      x=r*c+R; v=-r*omega*(s+c*(r*s+e)/R);
      a=-r*omega*omega*(c+(R*R*(r*c*c-(r*s+e)*s)+
        r*c*c*(r*s+e)*(r*s+e))/R/R/R);
      printf("\n\t\t %3d        %7.3f   %9.6f   "
             "%9.4f  %12.5e", i, phid, x, v, a);
      if((i+1)%20==0)
        { getch(); clrscr();
          printf("\n\t     Nr. of point  Angle phi     "
                 "x [m]       v [m/s]     a [m/s/s]\n");
        }
    }
 getch(); return 0;
}
```

În interiorul ciclului s-a introdus instrucţiunea

if ((i+1) % 20 == 0) { getch(); ….. }

pentru ca pe ecran să rămână câte 20 de linii cu rezultate până la apăsarea unei taste. Ca urmare a executării instrucţiunilor **r/=1000; b/=1000; e/=1000;** valorile exprimate în [mm] ale elementelor mecanismului au fost transformate în [m] pentru ca rezultatele să fie conforme cu sistemul internaţional de unităţi.

8.12 Instrucţiunea do - while.

Sintaxa este următoarea:

```
.....................
do
  instructiune ;
while ( expresie ) ;
.....................
```

Funcţionarea: ***instrucţiune*** este executată (eventual în mod repetat) până când ***expresie*** are valoare nenulă, deci verificarea condiţiei, în cadrul ***expresie*** se face **după** fiecare execuţie a corpului ciclului

Instrucţiunea **do - while** se poate înlocui cu o instrucţiune **while** sau cu o instrucţiune **for**, acestea având următoarele forme:

```
.....................
instructiune ;
while ( expresie )
      instructiune ;
.....................
```

```
.....................
instructiune ;
for ( ; expresie ; )
     instructiune ;
.....................
```

EXEMPLUL nr. 8.30. Se poate relua problema calculării valorii sumei şi produsului termenilor unui şir cu **n** termeni, rezolvând-o prin folosirea instrucţiunilor **do - while,** conform următoarelor instrucţiuni:

```
..........
i=0; sum=0; prod=1.0;
do
 { sum+=A[i]; prod*=A[i]; i++; }
while( i<n ) ;
..........
```

EXEMPLUL nr. 8.31. Secvenţa de instrucţiuni din cadrul Exemplului nr. 8.26 de la instrucţiunea **while** este reluată în cele ce urmează, utilizând instrucţiunile **do – while** :

```
.................
printf("\n\nThe interval [a; b] belongs to [-100.0;150.0]\n");
a=b=-1000;

do
  { printf("\n\t a = "); scanf("%le", &a); }
while( a <-100.0 ¦¦ a > 150.0 ) ;

do
  { printf("\n\t b ="); scanf("%le", &b); }
while( b <= a ¦¦ b > 150.0; ) ;
.................
```

8.13 Aplicaţii

EXEMPLUL nr. 8.32. Determinarea faptului că nişte variabile de tip întreg au valori pare sau impare se poate face în două feluri, ambele prezentate în programul care urmează. În primul rând se poate verifica dacă restul împărţirii cu 2 (**a%2**) este nul sau nu iar în al doilea rând se poate compara cu zero rezultatul obţinut ca urmare a calculului expresiei **b&1**, utilizând deci operatorul ***ŞI logic*** la nivel de bit.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int a,b; clrscr();
  printf("\n\t a = "); scanf("%d", &a);
  printf("\t b = ");   scanf("%d", &b);
  if(a%2==0) printf("\n\n  a = %d  is even", a); //even - par
  else       printf("\n\n  a = %d  is odd", a);  //odd  - impar
  if(b&1)    printf("\n\n b = %d  is  odd", b);
  else       printf("\n\n b = %d  is even", b);
  getch(); return 0;
}
```

În acest program s-au folosit de două ori instrucţiunile **if - else** .

O altă modalitate de rezolvare a problemei ar fi cu ajutorul operatorului condiţional ternar **? :**, după cum urmează:

```
.............
a%2 ? printf("\n a - odd"); : printf("\n a - even");
b&1 ? printf("\n b - odd"); : printf("\n b - even");
.............
```

EXEMPLUL nr. 8.33. Considerând trei valori de tip **double**, a **radicalului din 2** , a numărului **e** şi a lui **p** , se va elabora un program în care să se facă afişarea acestor trei valori cu un număr variabil de cifre a părţii subunitare, de la **1** la **10** .

Afişarea repetată se va face utilizând instrucţiunea **for** iar în cadrul şirului din interiorul parantezei funcţiei **printf** se foloseşte de trei ori caracterul generic ***** care se înlocuieşte la fiecare execuţie a ciclului cu valoarea curentă a variabilei **i** .

Programul este următorul :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

int main(void)
{ int i; double e, pi, r;
  e=exp(1.0); pi=4*atan(1.0); r=sqrt(2.0); clrscr();
  printf("\n         sqrt(2)              e              \343\n");
  for(i=1; i<=10; i++)
     printf("\n %15.*f %15.*f %15.*f", i,r,i,e,i,pi);
 getch(); return 0;
}
```

Datele se afişează în forma următoare :

```
     sqrt(2)              e              p

         1.4            2.7            3.1
        1.41           2.72           3.14
       1.414          2.718          3.142
      1.4142         2.7183         3.1416
     1.41421        2.71828        3.14159
    1.414214       2.718282       3.141593
   1.4142136      2.7182818      3.1415927
  1.41421356     2.71828183     3.14159265
 1.414213562    2.718281828    3.141592654
1.4142135624   2.7182818285   3.1415926536
```

Caracterul **p** s-a afişat ca urmare a introducerii secvenţei **\343** în şirul de la instrucţiunea **printf** , valoarea octală **343** corespunzând acestui caracter în codul **ASCII** extins.

EXEMPLUL nr. 8.34. Pentru a calcula valorile factorialelor numerelor întregi de la **1** la **n** se utilizează instrucţiunile de ciclare **for**.

În programul care urmează se face calculul factorialului de trei ori, odată considerându-l că are o valoare de tip **long int** (max. 2147483647), pentru **n=12**, a doua oară că are o valoare de tip **float** (max. 10^{38}). pentru **n= 34** iar a treia oară că are o valoare de tip **double** (max. 10^{308}), pentru **n=53** .

```
#include <conio.h>
#include <stdio.h>
int main(void)
{ int i; long int fi=1; float ff=1.0; double fd=1.0; clrscr();
  for(i=1; i<=12; i++) { fi*=i; printf("\n  %2d! = %14ld", i, fi); }
  getch(); clrscr();
  for(ff=1.0, i=1; i<35; i++)
     { ff*=i; printf("\n  %2d! = %28.20e", i, ff);
       if(i%15==0) getch();      }
  getch(); clrscr();
  for(fd=1.0, i=1; i<71; i++)
     { fd*=i; printf("\n  %2d! = %28.20le", i, fd);
       if(i%15==0) getch();      }
 getch(); return 0;
}
```

În cazul celor trei instrucţiuni de ciclare **for** de fiecare dată corpul ciclului era o instrucţiune compusă. La ultimele două cicluri s-au introdus instrucţiunile

```
if(i%15==0)  getch();
```

pentru ca să se oprească execuţia programului, după afişarea a câte 15 rânduri, până la apăsarea unei taste.

Se poate observa că în cazul factorialului **12 !** valorile obţinute în cele trei cazuri sunt identice (12 ! = 479001600) dar dacă se compară valorile corespunzătoare lui **30 !** , calculate considerându-l de tip **float**, **double** cu valorile **exacte** (modul de calcul exact se va prezenta într-un exemplu următor),

$$30! = \begin{cases} \underline{\underline{2.652528}}\ 89961724358000 \circ 10^{32} & (val.tip\ float - 7\ cifre\ \ exacte) \\ \underline{\underline{2.652528598121910}}\ 32000 \circ 10^{32} & (val.\ tip\ \ double - 16\ cifre\ exacte) \\ 265252859812191058636308480000000 & (val.\ exacta - 33\ cifre) \end{cases}$$

vor apare diferenţe datorită acumulării erorilor de rotunjire pe parcursul înmulţirilor repetate, acest lucru fiind explicabil prin modul cum se face înregistrarea în memorie a valorilor numerice de tip **float** respectiv **double**.

EXEMPLUL nr. 8.35. Una din posibilităţile de a calcula valoarea unei funcţii de forma:

$$y = \sin(\sin(\sin(\ldots.\sin(\sin(x))\ldots)))$$

este de a folosi instrucţiunea de selecţie **switch**, fără a utiliza şi instrucţiunea **break**, după cum se face în programul următor. Se vor putea folosi maximum opt perechi de paranteze.

```
//  sin(sin(..sin(sin(x))..)) = y
#include <conio.h>
#include <math.h>
#include <stdio.h>
int main(void)
{ int i, n; double x, y; clrscr();
  printf("\n\t sin(sin(...sin(sin(x))..)) = y");
  printf("\n\n\t x = "); scanf("%le", &x);
  printf("\t n [number of function] ( n<9 )  = ");
  scanf("%d", &n); printf("\n"); y=sin(x);
```

```
    switch(n)
         {   case 8:  y=sin(y);
             case 7:  y=sin(y);
             case 6:  y=sin(y);
             case 5:  y=sin(y);
             case 4:  y=sin(y);
             case 3:  y=sin(y);
             case 2:  y=sin(y);
         }
     for(i=1; i<=n; i++) printf("sin(");
     printf("%6.3f", x);
     for(i=1; i<=n; i++) printf(")");
     printf(" = %le", y);
     getch(); return 0;
}
```

Se poate desigur observa că această problemă se poate rezolva şi într-un mod mai simplu, utilizând instrucţiunea de ciclare **for**.

EXEMPLUL nr. 8.36. În figura 8.2 este reprezentat un compresor de tip **Roots**, al cărui rotori au profile formate din ***epicicloide*** şi ***hipocicloide***, generate de un punct aparţinând unui cerc de rază **r/4** care se rostogoleşte în exteriorul respectiv în interiorul unui cerc de rază **r**.

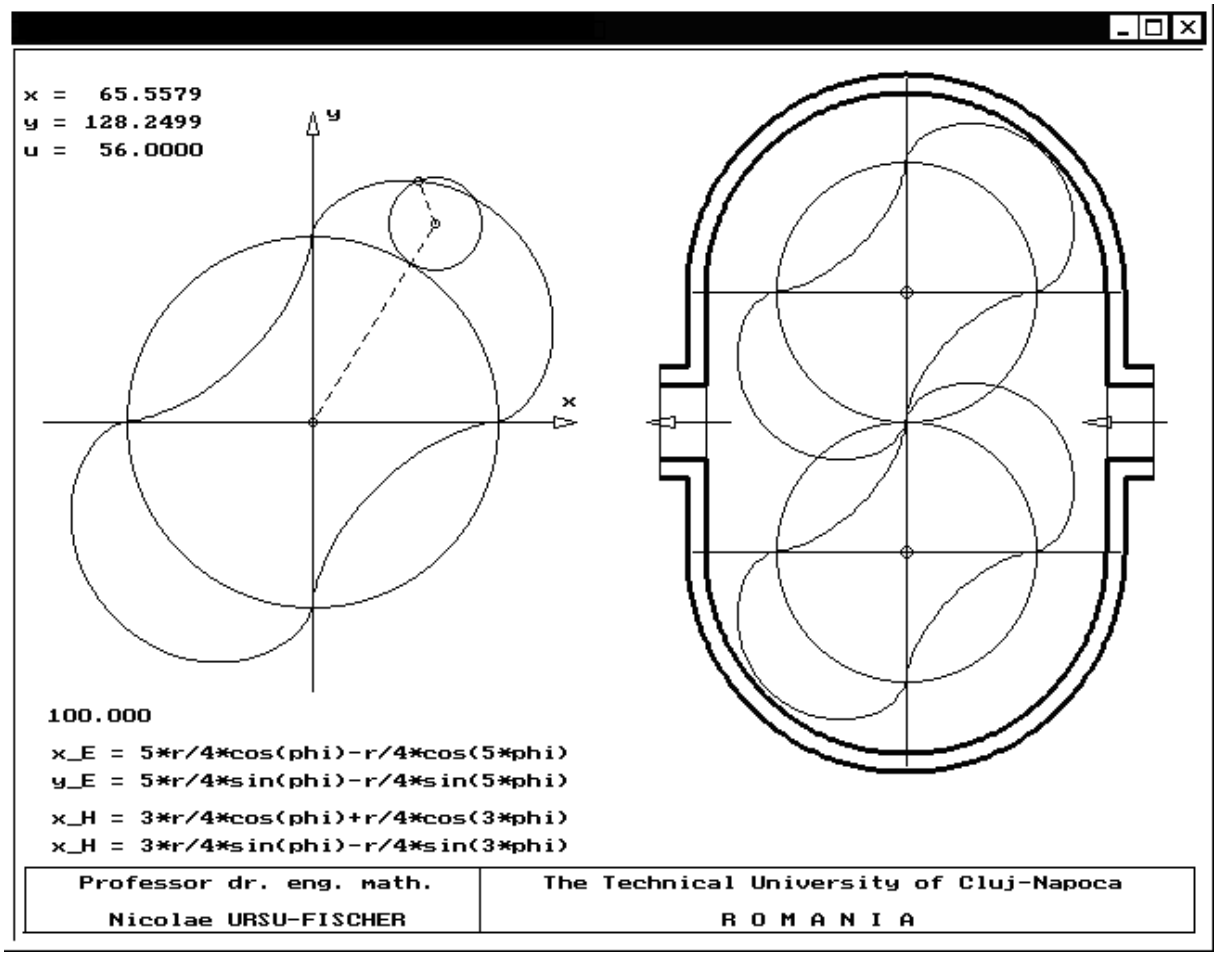


Figura 8.2

În cadranele I şi III profilul rotorului este de tip epicicloidă normală, ecuaţiile parametrice ale acestuia fiind:

$$(E)\begin{cases} x = \dfrac{5r}{4}\cos\varphi - \dfrac{r}{4}\cos 5\varphi, \\ y = \dfrac{5r}{4}\sin\varphi - \dfrac{r}{4}\sin 5\varphi, \end{cases} \qquad 0 \le \varphi \le \frac{\pi}{2} \;\; sau \;\; \pi \le \varphi \le \frac{3\pi}{2}$$

iar în cadranele II şi IV profilul este de tip hipocicloidă normală, având următoarele ecuaţii parametrice:

$$(H)\begin{cases} x = \dfrac{3r}{4}\cos\varphi + \dfrac{r}{4}\cos 3\varphi, \\ y = \dfrac{3r}{4}\sin\varphi - \dfrac{r}{4}\sin 3\varphi, \end{cases} \qquad \frac{\pi}{2} < \varphi < \pi \;\; sau \;\; \frac{3\pi}{2} < \varphi \le 2\pi .$$

Parametrul φ este unghiul dintre axa Ox şi segmentul care uneşte centrul cercului fix de rază **r** cu centrul cercului mobil de rază **r/4**, care se rostogoleşte în exteriorul sau în interiorul cercului mare.

În cadrul programului care se prezintă în continuare sunt folosite o mare parte din instrucţiunile din acest capitol.

Deoarece datele ce se introduc trebuie să aibă nişte valori care să corespundă unor situaţii din practică s-a folosit de două ori instrucţiunea **while** care face să se reia introducerea valorilor lui **r** respectiv **n** dacă nu se încadrează în nişte intervale date.

Pentru a realiza programarea ciclului, condus de o variabilă întreagă **i** s-a folosit instrucţiunea **for** iar pentru alegerea formulelor de calcul ale coordonatelor **x** şi **y** s-a folosit instrucţiunea de decizie **if -else**.

Instrucţiunea **if** s-a folosit în două scopuri: odată pentru a verifica dacă funcţia **scanf** returnează valoarea zero (atunci când nu s-a introdus o dată corectă) iar a doua oară pentru a compara cu zero restul obţinut prin operaţia **i % 20**.

În program se realizează afişarea pe ecran a câte 20 de linii cu rezultate, după care este necesară apăsarea unei taste.

```
#include <conio.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#define PI 3.1415926

int main(void)
{ int i, k1, k2, n; double delta, r, phid, phir, x, y;
clrscr(); r=-1.0;
while( r<=0 || r>1500.0 )
     { printf("\n\n\t The radius  r [mm]      = ");
       if( scanf("%le", &r)==0 ) exit(1);
     }
n=0;
while( n<1 || n>2500 )
     { printf("\t The number of points  n = ");
       if( scanf("%d", &n)==0 ) exit(1);
     }
```

```
delta=360.0/(n-1);
printf("\n\t     Nr. of point  Angle phi     x [mm]"
       "      y [mm]\n");
for( i=1; i<=n; i++ )
   { phid=(i-1)*delta; phir=phid*PI/180;
     if( phid>=0 && phid<=90 || phid>=180 && phid<=270 )
       { x=5*r/4*cos(phir)-r/4*cos(5*phir);
         y=5*r/4*sin(phir)-r/4*sin(5*phir);
       }
     else
       { x=3*r/4*cos(phir)+r/4*cos(3*phir);
         y=3*r/4*sin(phir)-r/4*sin(3*phir);
       }
     printf("\n\t\t %3d        %7.3f   %9.3f   %9.3f",i,phid,x,y);
     if(i%20==0)
       { getch(); clrscr();
          printf("\n\t     Nr. of point  Angle "
                 "phi     x [mm]      y [mm]\n");
       }
   }
getch(); return 0;
}
```

S-a putut observa şi în acest program că pentru a nota unghiul φ s-au folosit doi identificatori **phid** (valori în grade) şi **phir** (valori în radiani), deoarece în cazul utilizării funcţiilor **sin** şi **cos** argumentele acestora trebuie să fie exprimate în radiani iar în cazul afişării pe ecran aceasta trebuie să se facă pentru valori în grade.

EXEMPLUL nr. 8.37. În Exemplul nr. 8.34 s-a prezentat un program pentru calculul factorialelor. S-a observat că în cazul unor numere mari nu se pot obţine rezultatele numerice cu toate cifrele exacte ci numai cu primele 15-17 cifre în cazul când se fac calcule cu numere de tip **double**. Se pune problema de a scrie un program cu care să se poată determina toate cifrele exacte, indiferent de valoarea lui **n** . De exemplu, dacă **n = 53** valoarea factorialului este un număr (întreg) cu 70 de cifre :

53 ! = 4274883284 0600255642 9801375338 9399649690 3437883668 1372467200 0000000000

Este evident că acest lucru nu se poate obţine dacă s-ar considera **n!** ca o valoare numerică. Se va considera că **n!** se va scrie sub forma unui şir de numere întregi, fiecare cu câte **m** cifre, scrise alăturat.

Dacă se doreşte determinarea tuturor valorilor de la **1 !** la **53 !** atunci se va folosi şirul **T** cu 14 elemente T[0], T[1], ..., T[13], fiecare conţinând câte un număr întreg cu 5 cifre. Pentru a prezenta modul de rezolvare al problemei (trecerea de la şirul **T** corespunzător lui **k !** la cel corespunzător lui **(k+1) !**) se va considera de exemplu cum se trece de la şirul **T** ce corespunde lui **22 !** care are următoarea valoare

22 ! = 11 **24000** 72777 **76076** 80000

iar şirul **T** este următorul :

...., T[8] = 00000, T[9] = 00011, T[10] = 24000, T[11] = 72777, T[12]=76076, T[13]=80000,

la **23 !** care are valoarea :

23 ! = 258 **52016** 73888 **49766** 40000

iar elementele şirului **T** sunt

...., T[8] = 00000, T[9] = 00258, T[10] = 52016, T[11] = 73888, T[12]=49766, T[13]=40000,

După cum se poate vedea din schema de mai jos, fiecare număr (cu cinci cifre) T[13], T[12], T[11], ... se înmulţeşte cu 23 obţinându-se numere cu minimum cinci cifre, ultimele cinci cifre obţinute rămânând pe "poziţia" iniţială iar primele "transferându-se" la elementul anterior, noua valoare a unui element T[i] obţinându-se ca suma dintre numărul "transferat" şi ultimele cinci cifre obţinute prin înmulţirea valorii anterioare a lui T[i] cu 23 :

```
   ....  T[7]  T[8]  T[9] T[10] T[11] T[12] T[13]

                                      18 40000 = 80000 µ 23
                                   17 49748    = 76076 µ 23
                              16 73871         = 72777 µ 23
                            5 52000            = 24000 µ 23
                     0 00253                   = 00011 µ 23
                        -------------------------
          23! = S = 258 52016 73888 49766 40000
```

Programul următor soluţionează problema calculării factorialelor care conţin până la 70 de cifre .

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int i, j, transf; long int T[14];

clrscr();
  for(j=0; j<14; j++) T[j]=0;
  T[13]=1;
  for(i=1; i<=53; i++)
     { transf=0;
       for(j=13; j>=0; j--)
          { T[j]=T[j]*i+transf;
            transf=T[j]/100000;
             T[j]%=100000;
          }
       printf("\n %2d!=", i);
       for(j=0; j<=13; j++) printf("%05ld", T[j]);
       if(i%20==0) getch();
     }
 getch(); return 0;
}
```

În acest program s-a specificat că elementele şirului **T** vor fi de tip **long int** (deoarece pot fi mai mari decât 32767). S-a notat cu **transf** valoarea care se transferă elementului anterior al şirului **T**.

S-au folosit două cicluri suprapuse, ambele programate cu instrucţiuni **for** , în primul trecându-se de la un factorial la altul iar în al doilea se calculează valorile elementelor şirului **T** , de la elementul T[13] la elementul T[0], corespunzătoare fiecărui factorial. Se poate observa că în cadrul instrucţiunii

```
printf("%05ld", T[j]);
```

s-a folosit specificatorul de format **%05d** pentru ca să se tipărească totdeauna câte cinci cifre , inclusiv cifrele zero, la început, dacă numărul întreg are mai puţin de cinci cifre.

Este evident că şirul **T** se poate considera conţinând valori numerice întregi şi cu un alt număr de cifre decât cinci. Dacă se va considera că acesta conţine numere cu maximum 7 cifre atunci şirul va avea numai 10 elemente iar ciclul interior se va scrie sub forma :

```
.....................
for(j=9; j>=0; j--)
   { T[j]=T[j]*i+transf;
     transf=T[j]/10000000;
      T[j]%=10000000;
   }
.....................
```

Cu acest program, mărind valoarea lui **n** şi dimensiunea şirului **T** se pot determina cifrele exacte practic pentru factorialul oricărui număr.

EXEMPLUL nr. 8.38. Problema următoare, în care se vor căuta care sunt operaţiile ce trebuie făcute pentru ca rezultatul calculului să fie **N,**

```
( ( ( ( ( ( ( 1 o 2 ) o 3 ) o 4 ) o 5 ) o 6 ) o 7 ) o 8 ) o 9 =  N
```

se poate rezolva numai în urma utilizării unor cicluri suprapuse. Se pot utiliza cei patru operatori aritmetici, **+** , **-** , ***** şi **/** , care vor trebui puşi în opt poziţii în cadrul expresiei din stânga semnului egal, în locul operatorilor notaţi cu **o** , deci vor trebui verificate 4**µ**4**µ**4**µ**4**µ**4**µ**4**µ**4**µ**4 posibilităţi. Acest lucru se va putea face prin utilizarea a **opt** cicluri suprapuse, fiecare ciclu parcurs de câte patru ori, în acest fel pentru un **N** impus vor rezulta operatorii aritmetici ce trebuie folosiţi.

Astfel, dacă s-ar considera N = **111** una din combinaţiile posibile va fi :

```
( ( ( ( ( ( 1 + 2 ) + 3 ) + 4 ) / 5 ) + 6 ) + 7 ) * 8 ) - 9 =  111
```

În cadrul programului se va lucra cu două şiruri de caractere, **S** şi **Q**, în primul, **S**, introducându-se caracterele corespunzătoare necesare pentru formarea egalităţii care se verifică iar în al doilea, **Q**, se introduc în poziţiile corespunzătoare cele patru caractere, **'+'** , **'-'** , **'*'** şi **'/'** , corespunzătoare operatorilor aritmetici, afişarea rezultatului, adică a succesiunii de caractere care reprezintă operatorii efectuându-se numai dacă este îndeplinită condiţia

$$\left|N-111\right| < \varepsilon \ =10^{-10}$$

În cadrul celor opt cicluri suprapuse (realizate cu instrucţiunea ***for***) se utilizează un şir numeric **O** al cărui elemente **O[1]** , **O[2]** , ... , **O[8]** iau valori de la **1** la **4** iar pentru fiecare combinaţie se introduc în şirul **Q** caracterele corespunzătoare operatorilor aritmetici, cu ajutorul instrucţiunii ***switch*** , în care se face de asemenea şi calculul valorii curente a expresiei, corespunzătoare operatorului identificat.

Programul este următorul :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

#define PLUS '+'
#define MINUS '-'
#define PROD '*'
#define SLASH '/'

int main(void)
{ int i, j, k, O[10]; double eps=1.e-10, n;
  char blank=' ', S[70], Q[70];
  clrscr();
  for(i=0; i<70; i++) { S[i]=blank; Q[i]=blank; }
  for(i=0; i<=12; i+=2) S[i]='(';
  for(i=20; i<=56; i+=6) S[i]=')';
  S[14]='1'; S[62]='=';
  S[64]='1'; S[65]='1'; S[66]='1';     //   N = 111
//      ***        ***        ***              ***
  for(k=1, i=18; i<=60; i+=6, k++) S[i]='1'+k;
  S[67]=S[68]=S[69]=blank;
  for(i=16; i<=58; i+=6) { S[i]='o'; Q[i]='?'; }
  printf("\n");
  for(i=0; i<67; i++) printf("%c", S[i]);
  printf("\n");
  for(i=0; i<67; i++) printf("%c", Q[i]);
  getch();
  k=1;
  for(O[1]=1; O[1]<5; O[1]++)
   for(O[2]=1; O[2]<5; O[2]++)
    for(O[3]=1; O[3]<5; O[3]++)
     for(O[4]=1; O[4]<5; O[4]++)
      for(O[5]=1; O[5]<5; O[5]++)
       for(O[6]=1; O[6]<5; O[6]++)
        for(O[7]=1; O[7]<5; O[7]++)
         for(O[8]=1; O[8]<5; O[8]++)
          {n=1.0;
           for(i=2; i<10; i++)
            {switch(O[i-1])
             { case 1: { n+=i; Q[16+6*(i-2)]=PLUS;  } break;
               case 2: { n-=i; Q[16+6*(i-2)]=MINUS; } break;
               case 3: { n*=i; Q[16+6*(i-2)]=PROD;  } break;
               case 4: { n/=i; Q[16+6*(i-2)]=SLASH; } break;
             }
            }
          if(fabs(n-111)<eps)
//                 ***
           { printf("\n\n Case number %2d\n", k);
             for(j=0; j<67; j++) printf("%c", S[j]);
             printf("\n");
             for(j=0; j<67; j++) printf("%c", Q[j]);
             k++; getch();
```

```
            }
        }
  getch(); return 0;
}
```

Ca urmare a execuţiei acestui program, pentru **N=111** se obţin în total 10 combinaţii posibile de operatori aritmetici, tipărindu-se în mod repetat expresia sub formă generală (şirul **S**) iar sub aceasta operatorii în succesiunea necesară (şirul **Q**). Conform rezultatelor obţinute, sunt valabile următoarele egalităţi :

```
( ( ( ( ( ( 1 + 2 ) + 3 ) + 4 ) / 5 ) + 6 ) + 7 ) * 8 ) - 9 =  111
( ( ( ( ( ( 1 + 2 ) + 3 ) - 4 ) * 5 ) + 6 ) * 7 ) + 8 ) - 9 =  111
( ( ( ( ( ( 1 + 2 ) * 3 ) - 4 ) + 5 ) + 6 ) * 7 ) + 8 ) - 9 =  111
( ( ( ( ( ( 1 + 2 ) / 3 ) + 4 ) + 5 ) + 6 ) * 7 ) + 8 ) - 9 =  111
( ( ( ( ( ( 1 + 2 ) / 3 ) - 4 ) + 5 ) + 6 ) + 7 ) * 8 ) - 9 =  111
( ( ( ( ( ( 1 - 2 ) / 3 ) * 4 ) + 5 ) * 6 ) - 7 ) * 8 ) - 9 =  111
( ( ( ( ( ( 1 * 2 ) + 3 ) + 4 ) + 5 ) - 6 ) + 7 ) * 8 ) - 9 =  111
( ( ( ( ( ( 1 * 2 ) * 3 ) + 4 ) / 5 ) + 6 ) + 7 ) * 8 ) - 9 =  111
( ( ( ( ( ( 1 * 2 ) * 3 ) - 4 ) * 5 ) + 6 ) * 7 ) + 8 ) - 9 =  111
( ( ( ( ( ( 1 / 2 ) / 3 ) * 4 ) * 5 ) - 6 ) + 7 ) + 8 ) * 9 =  111
```

Dacă se doreşte obţinerea rezultatelor corespunzătoare unei alte valori a lui **N** se vor face modificări în program, în părţile subliniate cu caracterele ******* .

Observaţie. Un alt program care necesită de asemenea folosirea a opt cicluri suprapuse este cel care soluţionează "problema celor opt regine", adică determinarea poziţiilor posibile pe care le pot ocupa pe o tablă de şah opt piese care fac mişcările permise "reginei", în aşa fel încât să nu se "atace" una pe alta.

EXEMPLUL nr. 8.39. O altă categorie de probleme care se pot rezolva utilizând programe cu multe cicluri suprapuse sunt cele în care se determină numere întregi zecimale care au anumite proprietăţi mai speciale, de exemplu cum ar fi :

- numere cu **n** cifre a căror valoare este egală cu suma puterilor de ordinul **n** a cifrelor componente, aceste numere fiind numite "**numerele lui Armstrong**",

$$\overline{a_{n-1}\ a_{n-2}\ .....\ a_2\ a_1\ a_0} = a_{n-1}^n + a_{n-2}^n\ + \ldots . + a_2^n\ + a_1^n\ + a_0^n\ , \qquad a_{n-1} \geq 1,$$

astfel de numere fiind de exemplu **153** (cu trei cifre), **1634** (cu patru cifre),

$$153 = 1^3 + 5^3 + 3^3\,, \quad 1634 = 1^4 + 6^4 + 3^4 + 4^4$$

şi altele (370, 371, 407, 8208, 9474),

- numere a căror valoare este egală cu suma divizorilor acestora (aceste **numere** se numesc "**perfecte**") , cum ar fi : 6, 28, 496, 8128, ... , astfel de exemplu :

```
  6 = 1 + 2 + 3 ,
 28 = 1 + 2 + 4 + 7 + 14 ,
496 = 1 + 2 + 4 + 8 + 16 + 31 + 62 + 124 + 248 ,
...........
```

- numere a căror valoare să fie egală cu suma factorialelor cifrelor componente,

$$\overline{a_{n-1}\ a_{n-2}\ .....\ a_2\ a_1\ a_0} = a_{n-1}! + a_{n-2}! + \ldots . + a_2! + a_1! + a_0!, \qquad a_{n-1} \geq 1,$$

cum ar fi de exemplu 1, 2, 145, 40585.

$$1=1! \quad 2=2! \quad 145=1!+4!+5! \quad 40585=4!+0!+5!+8!+5!$$

- numere cu diverse numere de cifre la care valoarea radicalilor de diverse ordine este egală cu suma cifrelor,

$$\sqrt[m]{\overline{a_{n-1}\,a_{n-2}\,....\,a_2\,a_1\,a_0}} = a_{n-1} + a_{n-2} + ... + a_2 + a_1 + a_0 , \qquad a_{n-1} \geq 1,$$

cum ar fi

$$\sqrt{81} = 8+1,$$

$$\sqrt[3]{512}=5+1+2,\ \sqrt[3]{4913}=4+9+1+3,\ \sqrt[3]{5832}=5+8+3+2,$$

$$\sqrt[3]{17576}=1+7+5+7+6\ ,\ \sqrt[3]{19683}=1+9+6+8+3$$

$$\sqrt[4]{2401}=2+4+0+1,\ \sqrt[4]{234256}=2+3+4+2+5+6,\ \sqrt[4]{614656}=6+1+4+6+5+6$$

- numere prin ridicarea cărora la patrat să se obţină numere ale căror cifre finale sunt identice cu cifrele numărului care se ridică la patrat,

$$\overline{a_{n-1}\,a_{n-2}\,...a_2\,a_1\,a_0}^{\,2} = \overline{b_{m-1}\,b_{m-2}\,...\,b_{n+1}\,b_n\,a_{n-1}\,a_{n-2}\,...\,a_2\,a_1\,a_0}$$

astfel de numere sunt următoarele :

$$5^2 = 2\underline{5}\ , \qquad 6^2 = 3\underline{6}$$

$$25^2 = 6\underline{25}\ , \qquad 76^2 = 57\underline{76},$$

$$376^2 = 141\underline{376}\ , \qquad 625^2 = 390\underline{625}\ ,$$

$$9376^2 = 8790\underline{9376}$$

Aceste tipuri de probleme se pot rezolva în două feluri, unul din ele fiind acela al folosirii unui număr de cicluri suprapuse (imbricate), verificând astfel toate numerele care au un anumit număr de cifre.

Dacă se cere determinarea numerelor lui Armstrong cu trei cifre $\mathbf{a_2}$, $\mathbf{a_1}$, $\mathbf{a_0}$

$$\overline{a_2\,a_1\,a_0} = a_2^3 + a_1^3 + a_0^3$$

în program se vor folosi trei cicluri, conduse de $\mathbf{a_2}$ care va lua valori de la **1** la **9** (ciclul exterior), $\mathbf{a_1}$ care va lua valori de la **0** la **9** (ciclul intermediar) şi $\mathbf{a_0}$ care va lua de asemenea valori de la **0** la **9** (ciclul interior), pentru fiecare combinaţie de valori a acestora verificându-se dacă este îndeplinită condiţia.

Programul este următorul :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>
```

```
int main(void)
{int a0, a1, a2, n;  clrscr();
 for(a2=1; a2<10; a2++)
  {for(a1=0; a1<10; a1++)
    {for(a0=0; a0<10; a0++)
      {if(100*a2+10*a1+a0==pow(a2,3)+pow(a1,3)+pow(a0,3))
         {printf("\n  %1d%1d%1d", a2, a1, a0); getch(); }
      }
    }
  }
  getch(); return 0;
}
```

Ca urmare a execuţiei acestui program se obţin cele patru numere care îndeplinesc condiţia impusă : **153**, **370**, **371** şi **407** .

Al doilea mod de rezolvare al acestor tipuri de probleme, mai "bun" în sensul generalităţii acestuia va fi prezentat în cadrul unui exemplu din Capitolul X – Funcţii .

Capitolul IX

Pointeri

9.1 Introducere.

În limbajul de programare **C**, pe lângă multe elemente noi care au apărut faţă de alte limbaje care l-au precedat, s-au introdus şi aşa numiţii **pointeri (*to point*** – a indica**)**, în literatura tehnică română preluându-se termenul din limba engleză. Aceştia sunt nişte variabile care au ca valori **adrese** ale altor variabile, mai exact spus adrese ale zonelor din **RAM** (**R**andom **A**ccess **M**emory) unde se găsesc valorile respectivelor variabile.

După cum s-a arătat, diferitele tipuri de date se pot înregistra fie într-un singur octet (pe opt biţi), fie pe doi, patru sau opt octeţi. Prin **adresă** a variabilei se înţelege **adresa primului octet** din care începe înregistrarea.

Adresa se exprimă printr-un număr scris în sistemul de numeraţie hexazecimal.

Se fac următoarele ***convenţii*** privind notaţiile:

- dacă o variabilă se notează, de exemplu, cu **v** atunci adresa unde se găseşte valoarea acestei variabile se notează cu **&v** , această notaţie numindu-se **pointer**.

Invers,

- dacă se notează cu **p** un pointer, deci o adresă a unei variabile, atunci ***p** reprezintă valoarea respectivei variabile.

Se poate deci observa că dacă pointerul **p** conţine adresa variabilei **v** atunci următoarele notaţii sunt echivalente:

- **p** şi **&v** ,
- ***p** şi **v ,**
- ***&v** şi **v**.

În anumite cazuri prin utilizarea pointerilor se simplifică scrierea programelor C/C++.

În expresiile **&v** şi ***p** operatorii **&** şi ***** sunt ***operatori unari***.

Operatorul **&** se numeşte **operator adresă** (de referenţiere) iar ***** **operator de indirectare** (sau de dereferenţiere).

9.2 Declararea variabilelor pointer.

Pointerii se declară în cadrul funcţiilor **C** ca şi celelalte variabile, cu deosebirea că numele dat pointerului este precedat de operatorul *** ,** forma generală a declaraţiei fiind:

tip *nume ;

unde **nume** este identificatorul care defineşte o ***variabilă pointer***. Astfel, ca urmare a declaraţiei :

```
int *p ;
```

în memorie se va reţine un număr de octeţi în care se va înregistra adresa (valoarea pointerului) iar prin existenţa declaraţiei **int** rezultă că această adresă va fi a unei variabile de tip întreg .

Dacă s-ar considera următoarele declaraţii :

int n ; şi **int *p ;**

în primul caz variabila de tip întreg **n** are o valoare neprecizată. În ceea ce priveşte "valoarea" pointerului **p** , aceasta este dată de macroul **NULL**, corespunzător faptului că în această etapă pointerul nu conţine încă nici o adresă . El este iniţializat cu o astfel de valoare de adresă care nu corespunde cu nici una din adresele posibile ale unor variabile, tablouri, funcţii , structuri (vezi Capitolul XII) pe parcursul execuţiei programului .

Un astfel de pointer se numeşte **pointer nul** , ceea ce nu înseamnă în mod implicit că adresa pe care o conţine este o adresă nulă .

Aici trebuie precizată deosebirea între **pointerul nul** şi **caracterul nul '\0'** acesta din urmă având valoarea zero în reprezentarea ASCII, pe când în cazul pointerului nul nu este obligatoriu ca adresa pe care o conţine să fie nulă .

Unei variabile **n** i se poate atribui o valoare în modul "obişnuit", cum s-a procedat până acum în programele din capitolele anterioare sau prin intermediul pointerilor, după cum se arată în cele două secvenţe de instrucţiuni :

```
......                  ......
int n, ... ;            int n, ... , *p, ... ;
......                  ......
n=100;                  p=&n; *p=100;
......                  ......
printf("\n %d", n);     printf("\n n = %d  &n = %p", *p, p);
......                  ......
```

În cadrul secvenţei cu pointer, după declararea acestuia, i se atribuie valoarea adresei variabilei **n** iar în continuare, fiind stabilită relaţia dintre variabila **n** şi pointerul **p** în care este conţinută adresa variabilei, se poate introduce valoarea variabilei la adresa respectivă prin instrucţiunea

```
*p = 100 ;
```

În această secvenţă s-a folosit atât operatorul adresă **&** (de referenţiere) cât şi operatorul de indirectare ***** (operatorul de dereferenţiere) .

S-a utilizat specificatorul de format **%p** care realizează afişarea adresei (a conţinutului pointerului) în formatul folosit de către calculator.

Să considerăm următoarea secvenţă de instrucţiuni :

```
...........
int v1, v2, *p1, *p2;
float a,  *b;
char c, *d;
...........
```

Conform secvenţei prezentate, **p1** şi **p2** sunt variabile pointer corespunzătoare unor variabile întregi, deci valorile lor sunt adrese ale unor zone din memorie în care se vor afla date de tip întreg, **b** este o variabilă pointer a unei variabile cu valoare reală (**float**) iar **d** este o variabilă pointer a unei variabile de tip caracter (**char**). Este evident faptul că variabilele **p1**, **p2**, **b** şi **d** care apar în cazul declaraţiei pointerilor sunt de tip întreg, deoarece ele conţin adrese.

Declaraţiile utilizate introduc deci în cazul pointerilor noi tipuri de declaraţii:

int* float * char *

În interiorul funcţiei care conţine declaraţiile de mai sus pot exista următoarele instrucţiuni:

```
.............
v1=123;  v2= -2789; a = 90.67; c= 'x' ;
p1=&v1;
p2=&v2;
b=&a;
d=&c;
printf("\n Value = %d   Address = %p", v1, p1);
printf("\n Value = %d   Address = %p", v2, p2);
printf("\n Value = %f   Address = %p", a, b);
printf("\n Value = %c   Address = %p", c, d);
............
```

Se poate vedea din instrucţiunile de atribuire că a existat o corespondenţă între tipul variabilei şi cel declarat pentru pointer, pointerii conţinând adrese de zone de memorie în care au fost înregistrate valori de acelaşi tip cu ale pointerului. Deci instrucţiuni de forma:

```
..........
p1=&a;  b=&v1;  // incorect !!
..........
```

nu sunt corecte.

Pentru a înţelege mai bine această problemă a pointerilor să considerăm un exemplu extrem de simplu şi anume atribuirea valorii unei variabile **a** (deci **a** are o valoare cunoscută, presupusă reală) unei alte variabile, notate cu **b.** Evident că acest lucru se rezolvă prin parcurgerea instrucţiunii de atribuire, după ce în prealabil s-a făcut declararea tipului,

```
double a=5.25, b ;
b = a ;
```

Pentru a "complica" lucrurile şi a folosi pointerii se poate proceda în felul următor : se vor face declaraţiile

```
double a=5.25, b, *p ;
```

deci mai apare şi declararea unui pointer la o variabilă de tip **double** iar apoi se parcurg următoarele instrucţiunile de atribuire :

```
p = &a ;
b =  *p ;
```

prima realizând atribuirea adresei lui **a** variabilei pointer **p** iar a doua făcând atribuirea valorii spre care pointează **p** , variabilei **b**.

În cazul când se doreşte utilizarea unui acelaşi pointer pentru a înregistra adresele unor date de tipuri diferite, în diferite instrucţiuni ale funcţiei, se foloseşte o declaraţie de forma:

```
.................
void *p, *b, *d;
.................
```

în acest caz trebuind de fiecare dată să se precizeze în mod "forţat" printr-un operator **cast** (**to cast** - a trimite, a întoarce) care este tipul în momentul respectiv.

Exemplul următor prezintă astfel de situaţii:

```
...........
int a = 12, b = - 43;
```

```
float c = 1.24, d = 342.08;
void *p1, *p2, *p3, *p4;
p1=&a;
p2=&c;
* (int * ) p3 = b;
* (float * ) p4 = d;
...........
```

Prin instrucţiunile **p1=&a ;** şi **p2=&c ;** pointerilor **p1** şi **p2** li se atribuie adrese ale unor zone din memorie în care se găsesc valori ale unor variabile de tipuri diferite, acest lucru fiind posibil deoarece pointerii au fost declaraţi cu **void.**

Nu vor fi corecte atribuirile:

```
*p3=b;
*p4=d;
```

deoarece prin utilizarea declaraţiei **void** nu se defineşte tipul datelor spre care pointează **p3** şi **p4**, motiv pentru care se vor folosi atribuirile aşa cum au fost scrise în exemplul de mai sus, adică

```
*(int *) p3 = b ;
*( float *) p4 = d ;
```

9.3 Operaţii cu pointeri.

Numărul de operaţii care sunt admise în cazul pointerilor este foarte redus în comparaţie cu cel admis în cazul datelor de tip întreg sau real.

Cu pointerii se pot face următoarele tipuri de operaţii:

- ***atribuiri***,
- ***adunări*** (pointer şi o constantă întreagă), inclusiv ***incrementări*** **++p** sau **p++ ,**
- ***scăderi*** (pointer şi o constantă întreagă sau pointer şi pointer), inclusiv ***decrementări*** **- - p** sau **p - - ,**
- ***comparări*** (între doi pointeri).

În afară de aceste operaţii toate celelalte sunt interzise .

Atribuirea. În cazul operaţiei de atribuire nu este nici o deosebire între operaţiile de atribuire obişnuite şi cele în care intervin pointeri.

EXEMPLUL nr. 9.1 În programul următor se prezintă un exemplu de operaţie de atribuire:

```
#include <conio.h>
#include <stdio.h>
int main(void)
{ int a, *p1, *p2 ;
  printf("\n\t  a = "); scanf("%d", &a);
  p1= &a ;
  p2 = p1 ;
  printf("\n\t a = %d;\n\t p1 = %p  p2 = %p;"
         "\n\t *p1 = %d   *p2 = %d;",a, p1, p2, *p1, *p2);
  getch(); return 0;
}
```

La o execuţie a acestui program s-au obţinut rezultatele:

```
a = 1307
p1 = 8FC0:0FFE   p2 = 8FC0:0FFE ;
*p1 = 1307   *p2 = 1307 ;
```

Adunare, scădere, comparare. În ceea ce priveşte aceste operaţii, trebuie să se aibă în vedere că valoarea unei variabile de tip pointer reprezintă adresa primului octet al zonei de memorie unde se găseşte valoarea variabilei căreia îi corespunde pointerul.

Ştiind că variabilele de tip **int** se reprezintă pe **doi** octeţi (16 biţi), cele de tip **float** pe **patru** octeţi (32 biţi), cele de tip **double** pe **opt** octeţi (64 biţi) , este evident faptul că dacă pointerul corespunzător unei variabile de tip **int** are valoarea **n** atunci valoarea pointerilor următoarelor variabile de tip **int** vor fi **n+2**, **n+4**, **n+6**, … iar conform aceluiaşi raţionament, în cazul variabilelor de tip **float** adresele a mai multor variabile succesive vor fi **m**, **m+4**, **m+8**, **m+12**, …. iar în cazul **double** vor fi **k**, **k+8**, **k+16**, **k+24**, …

Numai în cazul pointerilor de tip caracter, ştiind că variabilele de tip **char** ocupă câte un octet, adresele a mai multor variabile succesive de tip caracter vor fi **i**, **i+1**, **i+2**, **i+3**, …

Adunarea şi scăderea. Rezultatul obţinut ca urmare al operaţiei de adunare (scădere) a unui pointer **p** cu o valoare întreagă **j** reprezintă adresa variabilei de acelaşi tip care urmează (precede) după **j** zone de memorie, deci valoarea adresei se va calcula cu formula

p + j * sizeof(tip)

EXEMPLUL nr. 9.2. În programul următor se consideră cele patru tipuri de variabile, int **i**, float **f**, double **d** şi char **c** , cărora le corespund pointerii **p2**, **p6**, **p10** şi **p14**.

În cadrul instrucţiunilor scrise cu bold se fac operaţii de adunare şi de scădere cu pointeri, valorile găsite ale pointerilor afişându-se.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int   i=15, *p1, *p2, *p3, *p4;
  float f=0.988, *p5, *p6, *p7, *p8;
  double d=-345.678, *p9, *p10, *p11, *p12;
  char c='A', *p13, *p14, *p15, *p16;
  clrscr();
  p2=&i;
  p1=p2-1; p3=p2+1; p4=p2+2;
  p6=&f;
  p5=p6-1; p7=p6+1; p8=p6+2;
  p10=&d;
  p9=p10-1; p11=p10+1; p12=p10+2;
  p14=&c,
  p13=p14-1; p15=p14+1; p16=p14+2;
  printf("\n int i = %d   p = &i"
       "\n p -1 = %p; p = %p; p+1 = %p; p+2 = %p;",
       i, p1, p2, p3, p4);
  printf("\n\n float f = %f   q = &f"
         "\n q -1 = %p; q = %p; q+1 = %p; q+2 = %p;",
         f, p5, p6, p7, p8);
  printf("\n\n double d = %le   r = &d"
        "\n r -1 = %p; r = %p; r+1 = %p; r+2 = %p;",
```

```
        d, p9, p10, p11, p12);
  printf("\n\n char c = %c   s = &c"
       "\n s -1 = %p; s = %p; s+1 = %p; s+2 = %p;",
       c, p13, p14, p15, p16);
  getch(); return 0;
}
```

Rezultatele obţinute ca urmare a execuţiei programului sunt următoarele:

```
int i = 15   p = &i
p -1 = 8FB8:0FFC; p = 8FB8:0FFE; p+1 = 8FB8:1000; p+2 = 8FB8:1002;

float f = 0.988000   q = &f
q -1 = 8FB8:0FE6; q = 8FB8:0FEA; q+1 = 8FB8:0FEE; q+2 = 8FB8:0FF2;

double d = -3.456780e+02   r = &d
r -1 = 8FB8:0FCA; r = 8FB8:0FD2; r+1 = 8FB8:0FDA; r+2 = 8FB8:0FE2;

char c = A   s = &c
s -1 = 8FB8:0FC0; s = 8FB8:0FC1; s+1 = 8FB8:0FC2; s+2 = 8FB8:0FC3;
```

În cazul pointerilor la variabile de tip **char** (ultimul rând) se poate observa că adresele sunt succesive, diferind printr-o unitate.

La ceilalţi pointeri se observă că valorile lor cresc cu câte două unităţi în cazul pointerilor de tip **int**, cu câte 4 unităţi la pointerii de tip **float**

E6 + 4 = EA, EA + 4 = EE , EE + 4 = F2,

şi cu câte opt unităţi la pointerii de tip **double**,

CA + 8 = D2, D2 + 8 = DA, DA + 8 = E2 ,

utilizându-se sistemul de numeraţie hexazecimal.

În cazul pointerilor sunt utilizabile de asemenea şi instrucţiunile

p2--; --p2; p2++; ++p2;

pentru a realiza decrementarea sau incrementarea pointerului cu o "unitate", adică de a obţine valoarea pointerului (adresa) anterior sau următor, care diferă de cea a celui curent cu o unitate, două, patru sau opt.

Scăderea pointerilor. În afară de adunarea (scăderea) unei constante întregi la un pointer sau de operaţii de incrementare sau decrementare mai este definită numai operaţia de scădere a doi pointeri. Prin scăderea a doi pointeri se determină numărul de variabile de acelaşi tip care separă cei doi pointeri.

Astfel, dacă în programul anterior, înainte de **getch();** s-ar mai folosi următoarea funcţie

```
printf("\n\n\t p16-p13 = %d", p16-p13);
```

aceasta ar realiza afişarea valorii **3** iar dacă se foloseşte specificatorul de format **%p** atunci se afişează **0000:0003**

Compararea pointerilor. Doi pointeri pot fi comparaţi cu ajutorul unor operatori relaţionali (**<** , **<=** , **>=** , **>**) sau de egalitate şi inegalitate (**==** , **!=**) .

Referindu-ne la exemplul de program anterior prezentat, la acesta s-ar putea adăuga:

```
..............
if( p1<p3 )
   printf("\n\t Adresa %p este anterioară adresei %p", p1, p3);
..............
```

unde în cadrul instrucţiunii **if** se face compararea a doi pointeri.

Operatorii **= =** şi **! =** se pot folosi în cadrul unor expresii de forma

p = = NULL sau/şi **p ! = NULL**

pentru a verifica dacă pointerul conţine sau nu o adresă, adică dacă este sau nu un **pointer nul**.

Se poate observa că următoarele expresii sunt echivalente:

p = = NULL cu **! p**

p ! = NULL cu **p**

9.4 Pointeri şi tablouri unidimensionale.

Definirea pointerilor, aşa cum s-a utilizat anterior, se referă la variabile simple. De o importanţă deosebită este legătura dintre un tablou de variabile cu o dimensiune şi pointerul acestuia. Prin convenţie se consideră că **denumirea unui tablou** este chiar **pointerul** acestuia. Deci dacă într-o funcţie se operează cu un şir (tablou unidimensional) a[0], a[1], ... , a[n-1], se notează cu **a** pointerul acestui şir, **a** fiind adresa primului element al acestui tablou, adică a lui **a[0]**, notaţiile următoare

a &a &a[0]

sunt **echivalente**, reprezentând pointerul şirului **a** adică adresa primului său element.

Deoarece pointerul **a** al unui tablou este o constantă (**pointer constant**) evident că nu sunt corecte instrucţiuni de felul

a+= 5 ; a++ ;

Pentru a putea face astfel de operaţii cu pointerul unui tablou acesta trebuie declarat ca şi pointer variabil, după cum se arată în continuare:

```
.............
double a[50], *pa;
pa=a;       //  sau   pa=&a[0];
.............
pa++;
.............
```

în acest caz pointerul **pa** fiind o variabilă pointer .

Ţinând cont de definirea pointerului unui tablou există două posibilităţi de a ne referi la un element al tabloului : fie utilizând indicele, scriind de exemplu **a[10]**, deci al 11-lea element al tabloului, fie scriind ***(a+10)** , adică referindu-ne la conţinutul zonei de memorie care are ca adresă pe **a+10**. Rezultă deci că notaţiile

a[i] *(a+i)

sunt echivalente.

În cazul în care se face declararea unui pointer variabil **pa** pentru şirul **a,**

```
.........
double a[50], *pa;
.........
pa=a;
.........
```

atunci există următoarele patru posibilităţi de referire la valoarea unui anumit element:

a[i] *(a+i) *(pa+i) pa[i]

aici **pa** fiind un pointer variabil iar **a** este un pointer constant, ca o noutate intervenind faptul că dacă **pa** este un pointer atunci **pa[i]** va reprezenta un element al şirului şi anume elementul **a[i]**.

EXEMPLUL nr. 9.3 Pentru a exemplifica, se poate face calculul sumei termenilor unui şir şi al produsului termenilor strict pozitivi, utilizând atât modul obişnuit de lucru cât şi cel cu pointeri.

În programul următor, unde se consideră un şir de numere reale, se fac următoarele operaţii:

- se tipăresc adresele **a, &a** şi **&a[0]** observând coincidenţa acestora,
- se tipăresc valorile elementelor utilizând notaţiile **a[i]**, ***(a+i)** şi adresele acestora **&a[i]**, **a+i** observând că adresele cresc cu câte 8 unităţi, deoarece elementele şirului sunt de tipul **double**,
- se calculează suma şi produsul utilizând operaţiile clasice, de tipul

sum_n+=a[i]; prod_n*=a[i];

- se calculează suma şi produsul utilizând pointeri,

sum_p+=*(a+i); prod_p*=*(a+i);

rezultatele obţinute cu ambele metode fiind evident identice,

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int i, n=8;
  double sum_n, sum_p, prod_n, prod_p,
       a[8] = {12.1, -2.9, 0.0, 18.34, 230.8, 0.0, -9.32, 3.9};
  clrscr(); sum_n=sum_p=0; prod_n=prod_p=1;
  //
  printf("\n\n\t  a = %p;  &a = %p;  &a[0] = %p;\n",
         a, &a, &a[0]);
  //
  for(i=0; i<n; i++)
     printf("\n a[%d] = %7.2f;  &a[%d] = %p;  a+%d = %p;    "
            " *(a+%d) = %7.2f;",
            i, a[i], i, &a[i], i, a+i, i, *(a+i));
  //
  for(i=0; i<n; i++)
     { sum_n+=a[i];
       if(a[i]!=0) prod_n*=a[i];     }
  //
  for(i=0; i<n; i++)
     { sum_p+=*(a+i);
       if(*(a+i)!=0) prod_p*=*(a+i); }
  //
  printf("\n\n\t     sum_n = %10.3f   prod_n = %le"
       "\n\t     sum_p = %10.3f   prod_p = %le",
        sum_n, prod_n, sum_p, prod_p);
  getch(); return 0;
}
```

Rezultatele obţinute cu acest program au fost următoarele:

```
          a = 8FB9:0FA0;  &a = 8FB9:0FA0;  &a[0] = 8FB9:0FA0;

a[0] =  12.10;  &a[0] = 8FB9:0FA0;  a+0 = 8FB9:0FA0; *(a+0) =   12.10;
a[1] =  -2.90;  &a[1] = 8FB9:0FA8;  a+1 = 8FB9:0FA8; *(a+1) =   -2.90;
a[2] =   0.00;  &a[2] = 8FB9:0FB0;  a+2 = 8FB9:0FB0; *(a+2) =    0.00;
a[3] =  18.34;  &a[3] = 8FB9:0FB8;  a+3 = 8FB9:0FB8; *(a+3) =   18.34;
a[4] = 230.80;  &a[4] = 8FB9:0FC0;  a+4 = 8FB9:0FC0; *(a+4) =  230.80;
a[5] =   0.00;  &a[5] = 8FB9:0FC8;  a+5 = 8FB9:0FC8; *(a+5) =    0.00;
a[6] =  -9.32;  &a[6] = 8FB9:0FD0;  a+6 = 8FB9:0FD0; *(a+6) =   -9.32;
a[7] =   3.90;  &a[7] = 8FB9:0FD8;  a+7 = 8FB9:0FD8; *(a+7) =    3.90;

            sum_n =    252.920   prod_n = 5.398822e+06
            sum_p =    252.920   prod_p = 5.398822e+06
```

Din coloanele 1 şi 4 se poate observa echivalenţa notaţiilor **a[i]** şi ***(a+i)** iar din coloanele 2 şi 3 echivalenţa notaţiilor **&a[i]** şi **a+i**.

EXEMPLUL nr . 9.4 În continuare se prezintă un alt program, cu care se determină lungimea unui şir de caractere, lucrând atât cu variabile cu indici cât şi cu pointeri. În toate cele cinci variante prezentate se verifică dacă s-a ajuns la ultimul caracter al şirului care este caracterul nul **'\0'** (caracterul nul conform notaţiei din codificarea ASCII – vezi Anexa 1)

Toate explicaţiile necesare se găsesc în cadrul programului, ca şi comentarii.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int i, i1, i3, i4, i5;
  char sir1[50], sir2[50], sir3[50], sir4[50], sir5[50],
       *psir2, *psir4, *psir5;
// ============================================
//                        The classic variant
//
clrscr();
puts("\n Write a character string, finished with"
     " <ENTER>\n First variant.\n");
gets(sir1);
i1=0;
while(sir1[i1] != '\0')
     i1++;
printf("\n The character string has %d characters.\n", i1);
getch();
//
//                        The variants with pointers.
// ============================================
psir2=sir2;
puts("\n Write a character string, finished with"
     " <ENTER>\n Second variant.\n");
gets(sir2);
//
/* Se foloseste instructiunea  while  pentru parcurgerea sirului.
   Variabila pointer  psir2  este initializata cu valoarea
   pointerului sirului sir2. La început este  evident ca  *psir2
   are ca valoare primul caracter din  sir, deci pe  sir2[0]. Pana
```

```
   cand nu se ajunge la caracterul '\0' variabila pointer  psir2
   isi tot mareste valoarea cu cate o unitate. In final se afiseaz
   valoarea diferentei adreselor ultimului element din sir si a
   primului, care in cazul caracterelor este chiar numarul acestora.
*/
while(*psir2 != '\0')
     psir2++;
printf("\n The character string has %d characters.\n",
        psir2 - sir2);
getch();
// ============================================
puts("\n Write a character string, finished with"
     " <ENTER>\n Third variant.\n");
gets(sir3);
//
/* Se foloseşte instrucţiunea  while  pentru parcurgerea sirului.
   Variabila de tip intreg  i3  va contine numarul de caractere ale
   sirului, gasite pana la etapa respectiva, incrementandu-se la
   fiecare caracter nou gasit. In loc sa se compare '\0' cu sir3[i3]
   se compara acest caracter cu *(sir3+i3). In final se afiseaza i3,
   deci numarul de caractere gasite pana la '\0'.               */
//
i3=0;
while(*(sir3+i3) != '\0')
      i3++;
printf("\n The character string has %d characters.\n", i3);
getch();
//  ============================================
puts("\n Write a character string, finished with <ENTER>"
       "\n Fourth variant.\n");
scanf("%s", sir4);
//
/* Se foloseste instructiunea  for  pentru parcurgerea sirului.
   Variabilei pointer psir4  i se atribuie pointerul constant sir4.
   In cazul intalnirii unui caracter diferit de '\0' se face
   incrementarea variabilei i4  de tip int care contine numarul de
   caractere al sirului. Pentru a trece la elementul urmator al
   sirului se face incrementarea variabilei pointer  psir4 , deci se
   trece la urmatorul element prin marirea adresei.          */
//
psir4=sir4;
for(i4=0; *psir4 != '\0'; psir4++)
    i4++;
printf("\n The character string has %d characters.\n", i4);
getch();
// ============================================
puts("\n Write a character string, finished with"
     " <ENTER>\n Fifth variant.\n");
scanf("%s", sir5);
//
/* Pentru parcurgerea sirului se foloseste instructiunea  for.
   La intalnirea caracterului '\0'  se iese fortat din ciclu prin
```

```
   intermediul instructiunii break. Variabila  i  de tip int are ca
   valoare numarul de caractere al sirului. Trecerea de la un
   element la altul se face prin incrementarea variabilei pointer
   psir5.                                                          */
//
psir5=sir5;
for(i=0, i5=0; i5<50; i5++)
   { if(*psir5 == '\0') { break; }
     psir5++; i++;
   }
printf("\n The character string has %d characters.\n", i);
// ==========================================
getch(); return 0;
}
```

EXEMPLUL nr. 9.5. Se consideră un şir de caractere, introdus de la tastatură. Se cere să se determine frecvenţa de apariţie în cadrul şirului a caracterelor ASCII utilizate.

Evident că şi această problemă, care se poate rezolva şi în alt fel, în acest caz se va rezolva utilizând pointerii.

În program se va folosi pointerul constant **str** (denumirea şirului de caractere) şi un pointer variabil **p** de tip **char** .

Ca urmare a atribuirii **p = str ;** la început pointerul **p** pointează spre octetul care conţine caracterul **str[0]** .

Pentru a determina pentru fiecare caracter ASCII de la 32 la 127 de câte ori apare în cadrul şirului, acesta se parcurge începând cu primul element până când se întâlneşte caracterul nul **'\0'** , prin intermediul unei structuri repetitive realizate cu **while**, notaţia ***p** reprezentând valoarea ASCII a caracterului din şir spre care pointează pointerul variabil **p** . Rezultă deci că instrucţiunea compusă care urmează după **while** va fi parcursă până când se va întâlni caracterul nul care are o valoare **zero**. În cadrul acestei instrucţiuni compuse se fac două operaţii : valoarea elementului din şirul **char_freq** care corespunde frecvenţei de apariţie a caracterului spre care pointează **p** se măreşte cu o unitate, de asemenea se incrementează şi valoarea pointerului **p** pentru ca să se treacă la următorul element al şirului de caractere .

Programul este următorul :

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int char_freq[127], i; char str[80], *p;
  clrscr();
  for(i=0; i<127; i++) char_freq[i]=0;
  printf("\n\t Input the string of characters :\n");
  gets(str);
// ****************************
  p=str;
  while(*p)
     { char_freq[*p]++;  p++; }
// ****************************
  printf("\n The character              Occurrence\n");
  for(i=32; i<127; i++)
     { printf("\n   %3d  %c                   %2d",
             i, i, char_freq[i]);
```

```
      if(i%15==0) getch();
    }
  getch(); return 0;
}
```

Observaţie. Parcurgerea unui şir de caractere şi efectuarea diferitelor operaţii cu caracterele componente, se poate face mai simplu, în următoarele două feluri, conform Exemplelor 9.3 şi 9.4 :

```
.........
char ..., str[100], ... , *p, ... ;
.........
p=str;
while(*p != '\0') { .....; p++; }
.........
```

sau

```
.........
char ..., str[100], ... , *p, ... ;
.........
p=str;
while(*p) { .....; p++; }
.........
```

EXEMPLUL nr. 9.6. S-a precizat anterior că notaţiile

a[i] şi ***(a+i)**

sunt echivalente, deci dacă se doreşte atribuirea unei anumite valori unui element al tabloului unidimensional **a** (presupus de tip **int**) s-ar putea scrie :

a[i] = 150; sau ***(a+i) = 150;**

Deoarece ***(a+i)** şi ***(i+a)** reprezintă valoarea aceluiaşi element **a[i]** al tabloului unidimensional **a** ar rezulta că ne-am putea referi la elementul **a[i]** şi sub forma **i[a]** .

În programul următor se atribuie la şirurile de caractere **A** şi **B** două prenume, caracter cu caracter, afişând apoi conţinutul celor două şiruri.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ char A[5], B[5]; int i;
  A[0]='N';     // *(A+0) = 'N';
  A[1]='I';     // *(A+1) = 'I';
  A[2]='C';     // *(A+2) = 'C';
  A[3]='K';     // *(A+3) = 'K';
  0[B]='M';     // *(0+B) = 'M';
  1[B]='I';     // *(1+B) = 'I';
  2[B]='K';     // *(2+B) = 'K';
  3[B]='E';     // *(3+B) = 'E';
  clrscr();
  printf("\n\n\t %10.4s", A); printf("\n\t %10.4s", B);
  getch(); return 0;
```

```
}
```

Se observă că în cazul şirului **A** s-au facut atribuirile în mod "normal", aşa cum s-au făcut şi în cadrul altor programe iar în cazul atribuirilor de caractere elementelor şirului **B** s-a ţinut cont de observaţia anterioară . Ca şi comentarii s-a prezentat un al doilea mod posibil de a realiza atribuirile, utilizând pointerii .

Ca urmare a execuţiei acestui program se afişează :

```
NICK
MIKE
```

În Capitolul V § 5.6 s-a arătat cum se poate iniţializa un şir de caractere , în cadrul declaraţiei prin care se definea că respectivul şir este de tipul **char** :

```
char str[50] = "Programming in C";
```

sau

```
char str[] = "Programming in C";
```

Conform celor ştiute rezultă că în primul caz se reţin 50 de octeţi pentru înregistrarea elementelor şirului, din care vor fi efectiv ocupate numai 17 (cele 16 caractere dintre ghilimele la care se va adăuga caracterul nul **'\0'**) iar în al doilea caz se vor reţine exact numărul necesar de octeţi, adică 16 + 1. În ambele cazuri **str** este un ***pointer constant*** a cărui valoare este egală cu adresa octetului în care se înregistrează primul caracter din şir .

Utilizând pointerii, este admisă şi următoarea formă de iniţializare :

```
char *STR = "Programming in C";
```

unde **STR** este o ***variabilă pointer*** . În acest caz se înregistrează cele 16+1 caractere şi în plus mai este nevoie de un număr de octeţi (de obicei doi sau eventual patru) pentru a înregistra valoarea ***pointerului variabil*** **STR**.

9.4 Pointeri şi tablouri multidimensionale.

În cazul declarării unor tablouri cu două dimensiuni

```
.............
char C[2][5], *pC;
int M[3][3], *pM;
double  A[3][2], *pA;
.............
```

pentru înregistrarea acestora în memorie vor fi rezervate zone contigue în care valorile variabilelor vor fi plasate în ordinea:

…., A[0][0], A[0][1], A[1][0], A[1][1], A[2][0], A[2][1], …..
…., M[0][0], M[0][1], M[0][2], M[1][0], M[1][1], M[1][2], M[2][0], M[2][1], M[2][2], ….
…., C[0][0], C[0][1], C[0][2], C[0][3], C[0][4], C[1][0], C[1][1], C[1][2], C[1][3], C[1][4], ….

deci linie după linie, cele de tip **char** ocupând câte un octet, cele de tip **int** câte doi octeţi, cele de tip **float** câte patru octeţi iar cele de tip **double** câte opt octeţi.

Se observă că s-a declarat şi câte un pointer de tip **char**, **int** şi **double**.

Ca şi în cazul tablourilor unidimensionale adresa primului element al tabloului bidimensional se notează cu **A**, **M** respectiv **C**, aceştia fiind ***pointeri constanţi***, deci notaţiile

&A[0][0] şi **A** , **&M[0][0]** şi **M** , **&C[0][0]** şi **C**

sunt echivalente.

În cazul unui element al tabloului, care se află pe linia **i** şi coloana **j** (numerotarea începând de la zero), de exemplu **A[i][j],** adresa acestuia este **&A[i][j].**

Dacă se consideră declaraţia :

```
char T[6], A[4][6];
```

T reprezintă pointerul tabloului unidimensional de caractere iar **A** pointerul tabloului bidimensional, fiind echivalente notaţiile

T şi **&T[0]** respectiv **A** şi **&A[0][0]**

În cazul tabloului bidimensional se poate face o legătură cu cel unidimensional în sensul următor :

Denumire (pointer)	Număr de elemente
T	[6]
A[4]	[6]

Conform acestei convenţii rezultă că matricea poate fi considerată ca un tablou unidimensional de **patru pointeri A[0] , A[1] , A[2]** şi **A[3]** , fiecare dintre aceştia fiind pointerul câte unui şir de şase caractere .

Adresa primului element al liniei **i** (numerotarea începe de la 0) deci a lui **A[i][0],** se notează cu **A[i]** iar a elementului **A[i][j]** este **A[i]+j**.

Aceşti pointeri **A[0]**, **A[1]** , **A[2]** şi **A[3]** se mai pot scrie şi sub forma: ***A**, ***(A+1)** , ***(A+2)** şi ***(A+3)**.

Conform acestor notaţii rezultă următoarele variante de a face referire la valoarea şi la adresa unui element $\mathbf{A_{i,j}}$ al unei matrice :

Valoarea lui $\mathbf{A_{i,j}}$	Adresa lui $\mathbf{A_{i,j}}$
A[i][j]	**&A[i][j]**
***(A[i] + j)**	**A[i] + j**
***(pA + k)**	**pA + k**
***(*(A+ i)+ j)**	***(A+i)+j**

unde **pA** este ***pointerul variabil*** de tip **char** al matricei **A** iar **k** este un număr întreg care reprezintă numărul de ordine al elementului $\mathbf{A_{i,j}}$ în şirul de valori din zona contiguă de memorie în care este înregistrată matricea **A**, valoarea acestuia fiind

k = i * n + j

n fiind numărul de coloane al matricei **A**.

EXEMPLUL nr. 9.7. În programul următor se consideră o matrice **A** cu două linii şi trei coloane şi se afişează pentru fiecare element atât valoarea acestuia cât şi adresa, utilizând toate modalităţile de scriere ale acestora.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int i, j, k, m=2, n=3;
  double A[2][3]={11., 12., 13., 21., 22., 23.}, *pA;
  clrscr(); pA=&A[0][0];
  printf("\n\n A = %p; &A[0][0]= %p; A[0] = %p; “
         ”pA = %p\n\n",A, &A[0][0], A[0], pA);
  k=-1;
  for(i=0; i<m; i++)
     { for(j=0; j<n; j++)
        { k++;
         printf("\n A[%d][%d] = %7.3f;", i, j, A[i][j]);
         printf("  *(A[%d]+%d) = %7.3f;", i, j, *(A[i]+j));
         printf("  *(pA+%d) = %7.3f;", k, *(pA+k));
         printf("\n\t\t\t\t\t\t   *(*(A+%d)+%d) = %7.3f;",
                 i, j, *(*(A+i)+j));
         printf("\n &A[%d][%d] = %p; A[%d]+%d = %p; pA+%d = %p;",
                i, j, &A[i][j], i, j, A[i]+j, k, pA+k);
         printf("\n\t\t\t\t\t\t   *(A+%d)+%d = %p;",i, j, *(A+i)+j);
         getch();
        }
       printf("\n");
     }
     getch(); return 0;
}
```

Rezultatele obţinute sunt următoarele:

```
A = 8FBB:0FAE;  &A[0][0] =  8FBB:0FAE;  A[0] = 8FBB:0FAE; pA = 8FBB:0FAE


A[0][0] = 11.000; *(A[0]+0) = 11.000; *(pA+0) = 11.000;
                                      *(*(A+0)+0) = 11.000;
&A[0][0] = 8FBB:0FAE; A[0]+0 = 8FBB:0FAE; pA+0 = 8FBB:0FAE;
                                      *(A+0)+0 = 8FBB:0FAE;
A[0][1] = 12.000; *(A[0]+1) = 12.000; *(pA+1) = 12.000;
                                      *(*(A+0)+1) = 12.000;
&A[0][1] = 8FBB:0FB6; A[0]+1 = 8FBB:0FB6; pA+1 = 8FBB:0FB6;
                                      *(A+0)+1 = 8FBB:0FB6;
A[0][2] = 13.000; *(A[0]+2) = 13.000; *(pA+2) = 13.000;
                                      *(*(A+0)+2) = 13.000;
&A[0][2] = 8FBB:0FBE; A[0]+2 = 8FBB:0FBE; pA+2 = 8FBB:0FBE;
                                      *(A+0)+2 = 8FBB:0FBE;
A[1][0] = 21.000; *(A[1]+0) = 21.000; *(pA+3) = 21.000;
                                      *(*(A+1)+0) = 21.000;
&A[1][0] = 8FBB:0FC6; A[1]+0 = 8FBB:0FC6; pA+3 = 8FBB:0FC6;
                                      *(A+1)+0 = 8FBB:0FC6;
A[1][1] = 22.000;  *(A[1]+1) = 22.000;  *(pA+4) = 22.000;
                                      *(*(A+1)+1) = 22.000;
&A[1][1] = 8FBB:0FCE; A[1]+1 = 8FBB:0FCE; pA+4 = 8FBB:0FCE;
                                      *(A+1)+1 = 8FBB:0FCE;
A[1][2] = 23.000; *(A[1]+2) = 23.000; *(pA+5) = 23.000;
                                      *(*(A+1)+2) = 23.000;
&A[1][2] = 8FBB:0FD6; A[1]+2 = 8FBB:0FD6; pA+5 = 8FBB:0FD6;
                                      *(A+1)+2 = 8FBB:0FD6;
```

Deoarece matricea 2 **µ** 3 este completă s-a putut observa la tipărirea rezultatelor că elementele acesteia au adrese care diferă cu câte opt unităţi

8FBB:0F**AE**, 8FBB:0F**B6**, 8FBB:0F**BE**, 8FBB:0F**C6**, 8FBB:0F**CE**, 8FBB:0F**D6**
AE+8 = B6 B6+8 = BE BE+8 = C6 C6+8 = CE CE+8 = D6

elementele fiind înregistrate în memorie în ordinea linie după linie.

Programul prezentat afişează automat valoarea adresei unui element al matricei A.

Valoarea adreselor unor elemente ale unor matrice, declarate în felul următor:

char A[mc][nc]; int B[mi][ni]; float C[mf][nf]; double D[md][nd];

unde **mc**, **nc**, **mi**, **ni**, **mf**, **nf**, **md**, **nd** sunt constante întregi şi pozitive, se pot calcula cu următoarele formule:

Element	**Adresă**
A[i][j]	**&A[0][0] + i * nc + j**
B[i][j]	**&B[0][0] + 2 * (i * ni + j)**
C[i][j]	**&C[0][0] + 4 * (i * nf + j)**
D[i][j]	**&D[0][0] + 8 * (i * nd + j)**

sau în general, când se face o declaraţie de forma

tip M[m][n];

m şi **n** fiind de asemenea constante întregi şi pozitive, atunci avem

M[i][j] **&M[0][0] + sizeof(tip) * (i * m + j)**

Dacă se consideră un tablou cu mai multe dimensiuni (**n** de exemplu), declarat după cum urmează

tip M[d1][d2][d3]…[dn] ;

atunci adresa unui element se determină cu formula [Pope99] pg. 164:

$$\&M[i_1][i_2]\ldots[i_n] = \&M[0][0]\ldots[0] + sizeof(tip) * (i_1 * d2 * d3 * d4 * \ldots * dn + i_2 * d3 * d4 * \ldots * dn + i_3 * d4 * \ldots * dn + \ldots\ldots\ldots\ldots + i_{n-1} * dn + i_n)$$

Se poate observa că valoarea **d1** (limita superioară a primei dimensiuni) nu intervine în formulele de calcul a adreselor. Din acest motiv, după cum se va vedea în Capitolul X, unde se vor trata funcţiile în C/C++ , dacă un tablou este utilizat ca argument al unei funcţii nu este necesară declararea primei sale dimensiuni.

9.6 Tablouri de pointeri.

În limbajul de programare C/C++ există posibilitatea de a declara tablouri conţinând pointeri, prin instrucţiuni de forma:

char *A[da] ; int *B[db] ; float *C[dc] ; double *D[dd] ;

unde **da**, **db**, **dc**, **dd** sunt constante întregi şi pozitive.

Tablourile de pointeri se pot utiliza în special atunci când se prelucrează succesiuni de şiruri de caractere.

EXEMPLUL nr. 9.8. În cazul următoarei declaraţii:

```
char *days[]={ "Monday","Tuesday","Wednesday","Thursday",
               "Friday","Saturday","Sunday" }
```

days este un tablou de pointeri la caracter iar **days[0]**, **days[1]**, …, **days[6]** sunt pointeri corespunzători începutului fiecărui şir de caractere.

În programul următor se utilizează un tablou de pointeri cu elementele căruia se fac o serie de operaţii, toate explicaţiile necesare fiind scrise ca şi comentarii în cadrul programului.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ char *days[] = {"Monday   ", "Tuesday  ", "Wednesday",
                  "Thursday ", "Friday   ", "Saturday ",
                  "Sunday   " };
               /* S-a declarat un tablou de pointeri  days[ ]
                  elementele caruia au fost initializate cu
                  adresele celor sapte siruri de caractere. */
  int i; clrscr();
  printf("\n\n\t\t  *days = %p;\n", *days);
               // S-a afisat adresa tabloului de pointeri days

  for(i=0; i<7; i++)
  printf("\n\t  days[%d] = %s; days[%d] = %p",
         i, days[i], i, days[i]);
  getch();
               /* S-a afisat continutul fiecaruia din cele
                  sapte siruri de caractere si adresele
                  acestor siruri. */

  printf("\n");
  for(i=0; i<10; i++)
     printf("\n\t  i = %d;  days[1]++ = %p; "
         "*(days[2]++) = %c",i, days[1]++, *(days[2]++));
  getch();
```

```
              /* Pentru elementele sirului nr. 2 s-au
                 afisat adresele elementelor succesive iar
                 pentru sirul nr. 3 s-au afisat pe rand
                 caracterele componente. Se observa ca
                 ultimul element este caracterul nul */

  printf("\n\n");
  for(i=0; i<6; i++)
       printf(" (*days[0]++) = %c          ",
              (*days[0])++);
              /* S-au afisat caracterele ce se obtin in urma
                 incrementării primului caracter al primului
                 sir. */
  getch(); return 0;
}
```

Rezultatele obţinute sunt următoarele:

```
                        *days = 8F91:001C;

          days[0] = Monday   ; days[0] = 8F91:001C
          days[1] = Tuesday  ; days[1] = 8F91:0026
          days[2] = Wednesday; days[2] = 8F91:0030
          days[3] = Thursday ; days[3] = 8F91:003A
          days[4] = Friday   ; days[4] = 8F91:0044
          days[5] = Saturday ; days[5] = 8F91:004E
          days[6] = Sunday   ; days[6] = 8F91:0058

          i = 0;  days[1]++ = 8F91:0026; *(days[2]++) = W
          i = 1;  days[1]++ = 8F91:0027; *(days[2]++) = e
          i = 2;  days[1]++ = 8F91:0028; *(days[2]++) = d
          i = 3;  days[1]++ = 8F91:0029; *(days[2]++) = n
          i = 4;  days[1]++ = 8F91:002A; *(days[2]++) = e
          i = 5;  days[1]++ = 8F91:002B; *(days[2]++) = s
          i = 6;  days[1]++ = 8F91:002C; *(days[2]++) = d
          i = 7;  days[1]++ = 8F91:002D; *(days[2]++) = a
          i = 8;  days[1]++ = 8F91:002E; *(days[2]++) = y
          i = 9;  days[1]++ = 8F91:002F; *(days[2]++) =

 (*days[0]++) = M    (*days[0]++) = N    (*days[0]++) = O
 (*days[0]++) = P    (*days[0]++) = Q    (*days[0]++) = R
```

Se poate observa că adresele succesive ale celor şapte şiruri diferă cu 10 unităţi, deoarece şirurile s-au scris pe 9 poziţii în cadrul declaraţiei, la care s-a mai adăugat în mod automat caracterul nul (**'\0'**) .

EXEMPLUL nr. 9.9. În continuare se prezintă un program cu care se introduce şirul **STR** care conţine câteva prenume, separate prin caracterul '_' , se determină poziţia caracterelor de la începutul fiecărui prenume, determinându-se valorile elementelor şirului de tip întreg **T**, se determină valorile unui tablou conţinând pointeri, fiecare pointând spre zona în care este reţinut câte un prenume şi apoi se afişează aceste prenume.

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ char STR[100], *pS[10], *p; int i, j, n_char, T[10];
  clrscr();
  printf("\n\t The input of the string of characters."
         "\n\t Between two successive names will be the"
         " character  _  \n\n");
  scanf("%s", STR);
         // Se determina numarul de caractere din sir.
  n_char=0;
  while(STR[n_char] != '\0') n_char++;
  printf("\n\n Number of characters = %d", n_char);
  getch();
         // Se determina pozitiile in sir a primelor
         // caractere din nume, retinute în sirul  T[].
  T[0]=0;
  for(i=1, j=0; i<n_char; i++)
      if(STR[i]=='_') { j++; T[j]=i+1; }
         // Se determina valorile pointerilor corespunzatori
         // fiecarui nume din sir.
  for(i=0; i<j; i++)
      { printf("\n T[%d] = %d;", i, T[i]); getch();
        pS[i]=STR+T[i];
      }
         // Se afiseaza numele din sir, fiecare pe cate o
         // linie, spre care pointeaza pointerul variabil  p.
  for(i=0, p=STR; *p; p++)
      { if(p==pS[i]) { printf("\n"); i++; }
         printf("%c", *p);
      }
  getch(); return 0;
}
```

Pentru a realiza afişarea prenumelor se utilizează pointerul variabil **p** care se compară cu pointerii corespunzători fiecărui prenume.

În continuare se arată rezultatele care se obţin în urma introducerii unui şir de prenume.

```
    The input of the string of characters.
    Between two successive names will be a character _

Shirley_Nick_Michael_Ann_Jane_Gabriel_William_Monica_

 Number of characters = 53
 T[0] = 0;
 T[1] = 8;
 T[2] = 13;
 T[3] = 21;
 T[4] = 25;
 T[5] = 30;
 T[6] = 38;
```

```
 T[7] = 46;
```

```
Shirley_
Nick_
Michael_
Ann_
Jane_
Gabriel_
William_
Monica_
```

9.7 Adresare indirectă.

Sunt situaţii când un pointer poate conţine adresa unui alt pointer, acesta conţinând la rândul său adresa unei variabile de un anumit tip.

Declararea acestor **pointeri la pointeri** se face prin scrierea în faţa pointerului a două asteriscuri.

Pentru a tipări conţinutul zonei de memorie spre care pointează un pointer obişnuit se foloseşte notaţia ***a** iar în acest caz, al **pointerului la pointer**, se foloseşte notaţia ****b** .

În cazul următoarei secvenţe de program

```
.............
double v, *a, **b ;
.............
v = 12.780934 ;
a = &v ;
b = &a ;
.............
```

pointerului (obişnuit) **a** i se atribuie adresa valorii variabilei **v** de tip **double** iar pointerului la pointer **b** i se atribuie valoarea adresei pointerului **a**. Conform acestor notaţii este evident că notaţiile ***b** şi **a** au aceleaşi semnificaţii, reprezentând aceeaşi adresă.

Dacă se execută instrucţiunea:

```
printf("\n v = %le; a = %p; b = %p; **b = %le;", v, a, b, **b);
```

se va obţine afişată valoarea lui **v**, a celor doi pointeri **a** şi **b** precum şi a conţinutului zonei spre care pointează pointerul de pointer, adică din nou valoarea lui **v**.

Capitolul X

Funcţii

10.1 Probleme generale.

După cum s-a putut observa, problemele de programare care au fost prezentate ca exemple în capitolele anterioare aveau un grad redus de complexitate, motiv pentru care rezolvarea s-a putut realiza prin scrierea doar a **funcţiei principale** (a funcţiei **main**), forma generală a acestor programe fiind:

```
#include <conio.h>
  ……….
#include <stdio.h>
  ……….
tip main( )
 {  ……….
    ……….
 }
```

Anumite operaţii, cum au fost cele de afişare sau de introducere de date, s-au executat cu ajutorul unor **funcţii standard** ale limbajului, cele mai utilizate fiind **printf(….)**, **scanf(….)**, **getch( )** şi altele. De asemenea, au fost utilizate frecvent o serie de funcţii matematice, care aparţin fişierului **math.h** .

În anumite situaţii, dacă algoritmul de rezolvare este mai complicat, este mult mai comod să se facă o descompunere a acestui algoritm în părţi componente, care eventual să se poată regăsi şi la alte probleme similare.

Să considerăm următoarea problemă: se dau două matrice patratice de dimensiuni egale, **A** şi **B**, trebuie calculate matricele **C=A+B** şi **D= A*B** şi afişate valorile elementelor matricelor care rezultă.

Programul, în cazul în care se va scrie numai funcţia **main( )**, va conţine următoarele:

- declaraţii de tip şi de dimensiuni (1),
- introducerea valorii lui **n**, dimensiunea matricelor patratice (2),
- instrucţiuni pentru introducerea valorilor elementelor matricei **A** (3),
- instrucţiuni pentru introducerea valorilor elementelor matricei **B**, care sunt asemănătoare cu cele precedente (4),
- instrucţiuni pentru realizarea însumării matricelor **A** şi **B**, obţinând matricea **C**, (5),
- instrucţiuni pentru realizarea produsului matricelor **A** şi **B**, obţinând matricea **D**, (6),
- instrucţiuni pentru afişarea elementelor matricei **C**, (7),

- instrucţiuni pentru afişarea elementelor matricei **D**, care sunt asemănătoare cu cele anterioare (8).

Se poate observa că parcurgerea etapelor 3 şi 4 respectiv 7 şi 8 va presupune parcurgerea de două ori a unor secvenţe de instrucţiuni asemănătoare, care diferă numai prin nişte notaţii.

Ca şi alte limbaje de programare, limbajul C/C++ permite folosirea nu numai a unei singure unităţi de program ci a mai multor, care în acest caz se numesc **funcţii** (subprograme de tip **function** şi **subroutine** în Fortran, proceduri în Pascal).

În consecinţă, se vor putea scrie în acest caz două funcţii, una pentru realizarea introducerii valorilor elementelor unei matrice, cu numele **inp_matr(…)** iar cealaltă pentru afişarea acestora, cu numele **out_matr(…)**, fiecare din aceste funcţii fiind **apelată** de două ori, mai rămânând de rezolvat modul cum se va face transmiterea informaţiilor dintre funcţia **main( )** şi funcţiile care se apelează.

Trebuie de asemenea observat că aceste operaţii vor trebui făcute în foarte multe programe, în cursul activităţii de programare, astfel că aceste funcţii vor putea fi folosite şi cu alte ocazii.

Etapele 5 şi 6 vor fi programate cu secvenţe de instrucţiuni care realizează adunarea respectiv înmulţirea a două matrice, iarăşi operaţii extrem de frecvente. Din acest motiv, cu toate că în acest caz aceste secvenţe vor fi parcurse numai câte o singură dată (în cadrul acestui program) totuşi ele vor fi scrise sub forma unor funcţii separate, cu numele **add_matr(…)** şi **multmatr(…),** care vor putea fi utilizate şi în alte programe.

În cazul când cele patru funcţii sunt în acelaşi director cu funcţia **main( )** programul în **C** va avea următoarea formă generală:

```
#include <conio.h>
#include <stdio.h>

#include "inp_matr.cpp"
#include "out_matr.cpp"
#include "add_matr.cpp"
#include "multmatr.cpp"

main()

{..............
 inp_matr(...);
 inp_matr(...);
 ..............
 add_matr(...);
 multmatr(...);
 ..............
 out_matr(...);
 out_matr(...);
 ..............
}
```

iar aşa cum s-a precizat, în directorul respectiv trebuie să se găsească şi cele patru funcţii care se apelează.

Scrierea şi testarea unui astfel de program, compus din mai multe funcţii este mult mai uşoară pentru că se lucrează cu unităţi mai mici şi se poate folosi experienţa acumulată în timp,

orice programator dispunând de un număr de funcţii pentru rezolvarea unor probleme care apar frecvent în practica programării.

Un alt avantaj deosebit al folosirii mai multor funcţii care vor intra în compunerea unui program este acela că permite lucrul în echipă, singurul mod în care în prezent se pot scrie soft-uri de mare complexitate, ajungându-se la scrierea unor programe sursă cu zeci sau chiar sute de mii de instrucţiuni .

10.2 Definirea, apelul şi revenirea, transmiterea datelor.

Din cele anterior prezentate a rezultat că în multe cazuri este necesară existenţa în program şi a altor funcţii, create de utilizator, în afara funcţiei **main( )**.

Trebuie să fie cunoscute următoarele:

- forma generală a unei funcţii,
- cum se foloseşte această funcţie (cum se apelează),
- cum se transmit valorile între funcţia în care se face apelul şi funcţia apelată.

Forma generală a unei funcţii este următoarea:

```
tip nume( lista_param_formali )      // antetul funcţiei
{
        corpul funcţiei
}
```

unde **tip** poate fi **void** dacă funcţia **nume(…)** nu returnează nici o valoare sau una din declaraţiile de tip cunoscute dacă se face o returnare de valoare, caz în care în corpul funcţiei va exista instrucţiunea **return,** urmată de o expresie. Denumirea **nume** a funcţiei este urmată în paranteze de o listă de ***parametri formali***, care este opţională.

Tipul, numele şi lista parametrilor formali formează **antetul funcţiei**.

În cazul apelului unei funcţii se utilizează numele funcţiei urmat în paranteze de o listă de ***parametri efectivi*** (în cazul când există parametri formali).

Problema cea mai importantă, în cazul folosirii funcţiilor, este modul cum se transmite informaţia între funcţia unde se face apelul şi funcţia apelată.

Există două modalităţi de a realiza acest lucru:

- **apel prin valoare**, adică valorile parametrilor efectivi se transferă (se atribuie) parametrilor formali care se găsesc pe poziţii similare în lista parametrilor, acest lucru se face numai atunci când valorile parametrilor formali cărora li s-au atribuit valori, în momentul apelului, nu trebuie returnate cu modificări înapoi în funcţia în care se face apelul, în funcţia în care s-a făcut apelul valorile variabilelor care au fost transmise funcţiei apelate rămânând nemodificate,

- **apel prin referinţă**, acesta utilizându-se atunci când se transmit între două funcţii valorile elementelor unor tablouri, cu una sau mai multe dimensiuni sau valorile unuia sau mai multor parametri formali, calculate în funcţia apelată, trebuie transmise înapoi în funcţia în care se face apelul. În ambele situaţii între cele două funcţii se vor transmite valori de adrese. Pentru a se realiza acest lucru, apelul prin referinţă, va trebui ca ***parametrul efectiv*** să aibă ca valoare o ***adresă*** iar **parametrul formal** de pe pozitia corespunzătoare să fie un **pointer**.

În ceea ce priveşte existenţa sau inexistenţa listei de parametri formali precum şi a tipului funcţiei în cazul celor la care se face apelul prin valoare, se pot întâlni următoarele situaţii:

A. Funcţia **are parametri formali** şi **returnează o valoare** de un anumit tip. Ca exemplu de funcţie care returnează o valoare se poate considera funcţia:

```
double v_max( double x, double y )
{ return ( x>y ? x : y ); }
```

care **returnează (întoarce) valoarea** maximă a parametrilor **x** şi **y**, deci tipul funcţiei trebuie să fie **double** iar argumentele sale (parametrii formali) sunt **double x** şi **double y**.

B. Funcţia **are parametri formali** dar **nu returnează nici o valoare**. Următoarea funcţie, căreia i se transmit valorile a două variabile realizează afişarea acestora,

```
void displ_ab( double first, double second )
{ printf("\n First value = %le; Second value = %le;\n",
         first, second ); }
```

fără a returna vreo valoare, motiv pentru care tipul acesteia este **void.**

În cazul acestor două exemple foarte simple în corpul funcţiei s-a lucrat numai cu variabilele care apăreau în lista parametrilor formali şi al căror tip era precizat. Evident că dacă este nevoie de alte variabile atunci tipul acestora se va declara în corpul funcţiei. Aceste variabile se numesc **variabile locale** sau **automatice** existând numai pe perioada când funcţia este apelată. Prima funcţie dată ca exemplu s-ar putea rescrie în felul următor:

```
double v_max(double x,double y)
{ double t ;
  t = x>y ? x : y ; return t;
}
```

complicând în mod inutil scrierea funcţiei .

C. Este de asemenea posibil şi cazul când funcţia **nu are nici un parametru formal** şi **nu returnează nici o valoare**, cum este exemplul următor când în urma apelului funcţiei se tipăreşte un nume:

```
void nick(void)
{ printf("\n\t Prof. dr. eng. math. Nicolae URSU-FISCHER"
    "\n\t Department of Mechanics and Computer Programming");   }
```

D. Este posibilă şi cea de-a patra situaţie, când funcţia **nu are o listă de parametri formali** dar **returnează o valoare**.

Astfel, funcţia următoare va returna un număr aleator întreg cuprins în intervalul [0,100] :

```
int frand100(void)
{ time_t lt; int number;
  srand((unsigned)time(&lt));
  number = rand()%101;
  return number;
}
```

Este de reţinut faptul că în acest ultim caz trebuie să se facă includerea fişierelor **stdlib.h** şi **time.h** .

În cazul tuturor funcţiilor anterior prezentate utilizarea acestora se face în acelaşi mod: în funcţia în care se face apelul se scrie numele funcţiei urmat în paranteze de lista **parametrilor efectivi** (dacă aceştia există). În cazul când există parametri formali pe locul acestora se folosesc parametri efectivi, respectându-se ordinea de scriere şi corespondenţa tipului. În cazul când tipul parametrului efectiv nu coincide cu cel al parametrului formal de pe aceeaşi poziţie în lista parametrilor, se face o conversie forţată de tip.

EXEMPLUL nr. 10.1. Dacă în cadrul unui program trebuie să se facă următoarele:
- afişarea numelui autorului,
- introducerea a două date de tip **double,** notate cu **a** şi **b**,
- alegerea valorii maxime dintre cele două date,
- afişarea acesteia,
- afişarea unui număr aleator din intervalul 0 - 100,

atunci acesta va fi următorul:

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#include "displ_ab.cpp"
#include "frand100.cpp"
#include "nick.cpp"
#include "v_max.cpp"

int main()
{ double a, b;
  nick();
  printf("\n a = "); scanf("%le", &a);
  printf("\n b = "); scanf("%le", &b);
  displ_ab( a, b ) ;
  printf("\n\n\t Val. max = %le", v_max( a, b ));
  printf("\n\n The random number between 0 and 100 = %d",
         frand100( ));
  getch(); return 0;
}
```

unde funcţiile

```
  void displ_ab( double first, double second)
  int frand100(void)
  void nick(void)
  double v_max(double a, double b)
```

sunt conţinute în fişierele

```
displ_ab.cpp,
frand100.cpp,
  nick.cpp,
  v_max.cpp,
```

care **sunt în acelaşi director** ca şi fişierul care conţine funcţia **main( ),** în cadrul programului efectuându-se includerea acestor fişiere în programul sursă, prin directivele de preprocesare.

În cazul în care se scrie un program sursă care conţine toate cele cinci funcţii, cea principală şi cele patru care sunt utilizate, acesta va arăta în felul următor:

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
//-----------------------------------------------
void displ_ab( double first, double second );
     //  declaraţie de funcţie
int frand100( void );
     // declaraţie de funcţie
void nick( void );
     // declaraţie de funcţie
double v_max( double x, double y );
     // declaraţie de funcţie
//-----------------------------------------------
int main(void)
{ double a, b;
  nick( ) ;
  printf("\n a = "); scanf("%le", &a);
  printf("\n b = "); scanf("%le", &b);
  displ_ab( a, b ) ;
  printf("\n\n\t Val. max = %le\n\n", v_max(a, b) );
  printf("\n The random number between 0 and 100 = %d",
        frand100( ) );
  getch(); return 0;
}
//-----------------------------------------------
void displ_ab( double  first, double second )
{ printf("\n   First value =  %le;  Second value = %le;\n",
         first, second ) ;  }
//-----------------------------------------------
int frand100(void)
  { time_t lt; int number;
  srand((unsigned) time(&lt));
  number=rand()%101;
  return number;
}
//-----------------------------------------------
void nick ( void )
{ printf("\n\n\t Professor dr. eng. math. Nicolae "
         "URSU-FISCHER\n\t Department of Mechanics"
         " and Computer Programming\n\n" ); }
//-----------------------------------------------
```

```
double v_max ( double x, double y )
{ return ( x>y ? x : y ); }
```

În acest al doilea caz înaintea funcţiei **main( )** s-au introdus declaraţiile de funcţii, care constau din scrierea **antetului** urmat de **;** Se poate observa că în al doilea caz datorită existenţei declaraţiilor de funcţii, în care se precizează lista de argumente (parametri formali), vor fi evitate erorile privind eventuale necorespondenţe între tipul şi ordinea parametrilor efectivi şi formali.

Dacă programatorul are toate funcţiile din directorul curent listate şi se pot cunoaşte listele de parametri formali, atunci este de preferat prima variantă, în care se face includerea fişierelor necesare, programul sursă devenind mult mai scurt.

În ambele variante modul de apelare al funcţiilor a fost identic :

- nick( ) în cazul funcţiei **void nick(void),**

- displ_ab(a, b) în cazul funcţiei **void displ_ab(double first, double second),**

- v_max(a, b) în cazul funcţiei **double v_max(double x, double y)** respective

- frand100( ) în cazul funcţiei **int frand100(void),**

observând că parametrii efectivi **a** şi **b** s-au scris în locul parametrilor formali **double x** şi **double y** sau **double first** şi **double second**, numărul şi tipul parametrilor efectivi coincizând cu numărul şi tipul parametrilor formali. În urma folosirii funcţiei **v_max(a, b)** s-a returnat o valoare de tip **double**, care s-a afişat iar în urma folosirii funcţiei **frand100( )** s-a returnat o valoare de tip **int**.

Observaţie. În locul funcţiei **v_max(...)** utilizate pentru determinarea valorii maxime a două variabile s-ar putea defini o macroinstrucţiune în cadrul preprocesării, după cum urmează :

```
#define v_max(x,y) (((x)>(y)) ? (x) : (y))
```

EXEMPLUL nr. 10.2. Se consideră o variabilă de tip întreg **n** şi se pune problema calculării factorialelor

$$1!\ \ 2!\ \ 3!\ \ 4!\ \ \ldots..\ \ n!$$

precum şi a aranjamentelor şi combinărilor

$$\mathbf{A_1^1;\ A_2^1, A_2^2;\ A_3^1, A_3^2, A_3^3;\ \ldots..\ ;\ A_n^1, A_n^2, \ldots., A_n^k, \ldots., A_n^n;}$$

$$\mathbf{C_1^0, C_1^1;\ C_2^0, C_2^1, C_2^2;\ C_3^0, C_3^1, C_3^2, C_3^3;\ \ldots..\ ;\ C_n^0, C_n^1, \ldots., C_n^k, \ldots., C_n^n;}$$

Această problemă poate fi considerată ca una din cele mai bune exemplificări a utilizării funcţiilor pentru realizarea unui program.

Se vor scrie trei funcţii **fact(int m)**, **arrang(int m, int k)** şi **comb(int m, int k**) care returnează valori de tip **double,** în cadrul cărora se calculează valorile factorialului, a aranjamentelor şi a combinărilor. În fiecare dintre acestea există câte un ciclu, programat cu ajutorul instrucţiunii **for,**

```
//  calculul factorialului
for( i=1,f=1.0; i<=n; i++ ) f*=i;

//  calculul aranjamentelor
for( a=1.0,i=n; i>=n-k+1; i-- ) a*=i;
```

```
//   calculul combinarilor
for( c=1.0, i=1; i<=k; i++ )
   { if(c>1.e250) return -1.0;
     c*=(n-i+1.0)/i; }
```

utilizându-se relaţiile de recurenţă

$$\mathbf{i! = i\,(i-1)!} \qquad \mathbf{A_n^i = (n-i+1)\,A_n^{i-1}} \qquad \mathbf{C_n^i = \frac{n-i+1}{i}\,C_n^{i-1}}$$

Programul sursă a fost scris în ipoteza că cele trei funcţii au fost scrise şi există în cadrul fişierelor **fact.cpp** , **arrang.cpp** şi **comb.cpp** care se includ în programul sursă.

În funcţia **main( )** se iniţializează valoarea lui **n=35** şi se trece la calculul factorialelor (în cadrul unui ciclu) şi al aranjamentelor şi combinărilor (fiecare în cadrul a câte două cicluri suprapuse).

S-au folosit caracterele bold pentru a arăta modul cum se apelează aceste trei funcţii.

```
#include <conio.h>
#include <stdio.h>

#include "fact.cpp"   // long double fact(int m)
#include "arrang.cpp" // long double arrang(int n, int k)
#include "comb.cpp"   // long double comb(int n, int k)

int main(void)
{ int k, m, n=35;  clrscr();
for( m=1; m<=n; m++ )
   { printf("\n\t\t %3d! = %23.15Le", m, fact( m ) );
     if(m%20==0) getch(); }
getch(); clrscr();
for( m=1; m<=n; m++ )
   { for(k=1; k<=m; k++)
        { printf("\n\t\t A(%3d,%3d) = %23.15Le", m, k,
                  arrang( m, k ));
          if(k%20==0) getch(); }
     printf("\n"); getch();
   }
getch(); clrscr();
for( m=1; m<=n; m++ )
   { for(k=0; k<=m; k++ )
        { printf("\n\t\t C(%3d,%3d) = %23.15Le", m, k,
                  comb( m, k ) );
          if(k%20==0) getch(); }
     printf("\n"); getch();
   }
getch(); return 0;
}

//      Fisierul   fact.cpp
//
long double fact( int n )
{ int i; long double f;
  if( n<0 || n>150 ) return -1.0;
```

```
  for( i=1,f=1.0; i<=n; i++ ) f*=i;
  return f;
}

//       Fisierul  arrang.cpp
//
long double arrang( int n,  int k )
{ int i; long double a;
  if( n<1 || n>150 ) return -1.0;
  if( k<1 || k>n )   return -1.0;
  for( a=1.0,i=n; i>=n-k+1; i-- ) a*=i;
  return a;
}

//         Fisierul   comb.cpp
//
long double comb( int n, int k )
{ int i; long double c;
  if( n<1 || n>1000 ) return -1.0;
  if( k<0 || k>n ) return -1.0;
  if( k>n/2 ) k=n-k;
  for( c=1.0,i=1; i<=k; i++ )
     { if(c>1.e250) return -1.0;
       c*=(n-i+1.0)/i; }
  return c;
}
```

EXEMPLUL nr. 10.3. Funcţiile care returnează o valoare sunt frecvent utilizate în cazul programelor în care se calculează valorile unor integrale definite. Dacă se consideră integrala definită

$$\mathbf{I = \int_a^b f(x)\,dx}$$

valoarea aproximativă a acesteia se poate calcula cu una din următoarele metode :

- metoda dreptunghiurilor,

$$\mathbf{\int_a^b f(x)\,dx \approx h \sum_{i=1}^{n} f\left(a - \frac{h}{2} + i\,h \right),}$$

- metoda trapezelor,

$$\mathbf{\int_a^b f(x)\,dx \approx \frac{h}{2}\left[f(a) + 2\sum_{i=1}^{n} f(a + i\,h) - f(b) \right]}$$

- metoda lui Simpson,

$$\mathbf{\int_a^b f(x)\,dx \approx \frac{h}{6}\left[f(a) + 4\sum_{i=1}^{n} f\left(a - \frac{h}{2} + i\,h \right) + 2\sum_{i=1}^{n} f(a + i\,h) - f(b) \right]}$$

unde

$$\mathbf{h = \frac{b-a}{n}},$$

este mărinea subintervalelor în care se face împărţirea intervalului **[a ; b]**. Este evident faptul că exactitatea calculelor va creşte cu mărirea lui **n**.

În cadrul programului se va calcula valoarea **constantei lui Catalan,** [Fiht_2] pg. 430-431, [Jank60] cap. I, care rezultă ca valoare a următoarei integrale definite :

$$\mathbf{G = \int_0^1 \frac{arctg\ x}{x}\,dx = 0.915\ 965\ 594\ ,}$$

utilizând toate cele trei formule de calcul, pentru următoarele valori ale lui **n** : 5, 10, 15, 20, 25, 30, 35, 40, 45 şi 50.

Se poate observa că în toate cele trei formule se utilizează **f(a)** unde a=0, ceea ce necesită ca funcţia de sub semnul integralei să se calculeze în următorul fel :

$$\mathbf{f(x) = \begin{cases} 1, & x = 0, \\ \dfrac{arctg\ x}{x}, & x \neq 0, \end{cases}}$$

deoarece în x=0 apare o nedeterminare de tipul 0/0,

$$\mathbf{\lim_{x \to 0} \frac{arctg\ x}{x} = 1.}$$

Programul sursă este format din funcţia **main( )** şi funcţia **double func (double x)** în care se calculează valoarea funcţiei de sub semnul integralei returnându-se valoarea acesteia :

```
//  catalan.cpp
//
#include <conio.h>
#include <math.h>
#include <stdio.h>

double func( double x );

int main(void)
{ int i, n; double a=0, b=1, G=0.915965594, h, sc, sm, D, T, S;
  clrscr();
  printf("\n\t\t    G = %11.9f\n\n\t  n    Met. dr.     Met. tr."
         "   Met. Simpson\n", G);
  for( n=5; n<=50; n+=5 )
     { h=(b-a)/n;
       for( sm=0,sc=0,i=1; i<=n; i++ )
          { sm+=func( a-0.5*h+i*h ); sc+=func( a+i*h ); }
       D=h*sm;
       T=0.5*h*(func( a )+2*sc-func( b ));
       S=h/6*(func( a )+4*sm+2*sc-func( b ));
       printf("\n\t %3d  %11.9f  %11.9f  %11.9f",
              n, D, T, S);
     }
  getch(); return 0;
}
```

```
double func( double x )
{ if( x != 0.0 ) return atan(x)/x;
  else return 1.0;
}
```

După cum s-a putut observa s-a făcut declaraţia de funcţie, înaintea funcţiei **main( )** unde se foloseşte respectiva funcţie.

Ca urmare a execuţiei acestui program s-au obţinut următoarele rezultate numerice :

```
G = 0.915965594

n  Met. dr.     Met. tr.     Met. Simpson

 5  0.916441811  0.915013634  0.915965752
10  0.916084545  0.915727723  0.915965604
15  0.916018453  0.915859883  0.9159655 96
20  0.915995325  0.915906134  0.9159655 95
25  0.915984622  0.915927540  0.915965594
30  0.915978807  0.915939168  0.915965594
35  0.915975302  0.915946179  0.915965594
40  0.915973027  0.915950730  0.915965594
45  0.915971467  0.915953849  0.915965594
50  0.915970351  0.915956081  0.915965594
```

Se poate observa (cifrele exacte s-au tipărit cu caractere bold) că în cazul utilizării metodei lui Simpson se obţin cele mai precise rezultate, pentru aceleaşi valori ale lui **n**.

EXEMPLUL nr. 10.4. Calculul valorilor **polinomului de interpolare al lui Lagrange**. (Joseph Louis de Lagrange 1736-1813). Se consideră cunoscute coordonatele unui număr de **n** puncte din planul **Oxy**,

$$(x_1, y_1); (x_2, y_2); \ldots\ldots; (x_i, y_i); \ldots\ldots; (x_{n-1}, y_{n-1}); (x_n, y_n);$$

presupunându-se că

$$x_i \neq x_k, \quad \forall i,k \in \overline{1,n}$$

care pot fi conţinute de reprezentarea grafică a unui polinom de gradul **n-1** (care are **n** coeficienţi),

$$L_{n-1}(x) = c_1 x^{n-1} + c_2 x^{n-2} + \ldots\ldots + c_{n-2} x^2 + c_{n-1} x + c_n$$

numit polinomul de interpolare al lui Lagrange.

Impunând condiţiile ca reprezentarea grafică a acestui polinom să treacă prin cele **n** puncte vor rezulta un număr de **n** ecuaţii liniare din care se pot determina coeficienţii polinomului, acest sistem fiind compatibil şi determinat deoarece matricea coeficienţilor necunoscutelor este de tip Vandermonde.

În cazul în care se cere determinarea unor valori ale acestui polinom de interpolare, se utilizează formula:

$$L_{n-1}(x) = \sum_{j=1}^{n} \left(y_j \prod_{\substack{i=1 \\ i \neq j}}^{n} \frac{x - x_i}{x_j - x_i} \right)$$

programarea căreia se face cu uşurinţă, utilizând de două ori instrucţiunea de ciclare **for**.

Deoarece calculul valorilor acestui polinom de interpolare intervine în multe probleme cu caracter matematic şi tehnic este oportun să scriem o funcţie în care să se facă acest calcul, care să poată fi apelată atunci când este nevoie.

Această funcţie se prezintă în continuare iar fişierul care o conţine s-a denumit **f_lagr.cpp**:

```
//  --------  Fisierul  f_lagr.cpp --------
//
double f_lagr( int n, double X[ ], double Y[ ], double xc )
{ int i, j; double L, P;
  for( L=0,j=0; j<n; j++ )
     { for( P=Y[j],i=0; i<n; i++ )
          { if(i != j) P*=(xc-X[i])/(X[j]-X[i]); }
       L+=P;
     }
  return L;
}
```

După cum se poate observa, această funcţie are patru parametri formali, dintre care doi, **int n** şi **double xc,** se vor înlocui cu parametrii efectivi ale căror valori se atribuie lui **n** şi **xc**, în cazul acestora utilizându-se **apelul prin valoare**. Ceilalţi doi parametri formali, **double X[ ]** şi **double Y[ ]**, sunt tablouri unidimensionale elementele cărora trebuie să aibă ca valori coordonatele punctelor, care vor fi "transmise" de către funcţia care apelează funcţia **f_lagr(…),** în acest caz funcţia **main( )**.

În această situaţie transmiterea valorilor va fi înlocuită prin transmiterea adreselor la care se găsesc primele elemente din fiecare şir, ca parametri efectivi utilizându-se notaţiile **X** şi **Y** adică adresele (pointerii) şirurilor. Acest mod de transmitere al datelor între funcţia în care se face apelul şi funcţia apelată se numeşte **apel prin referinţă**.

În continuare se prezintă programul sursă în care se calculează valorile polinomului de interpolare Lagrange.

```
//       lagrange.cpp
//
#include <conio.h>
#include <math.h>
#include <stdio.h>

#include "f_lagr.cpp"  //(int np,double X[],double Y[],double xc)

int main(void)
{ int i, n, np;
  double X[25], xL[250], Y[25], yL[250];
  double a, b, step,  xc, xmax, xmin, yc,  ymax, ymin;
```

```
  clrscr();
  printf("\n\t\t The Lagrange interpolation polynom\n");
  printf("\n\t The number of points  np = ");scanf("%d", &np);
  printf("\n\n\t The coordinates of %d points:  "
         "X[0] < X[1] < ... < X[%d]\n", np, np-1);
  printf("\n\t X[0] = "); scanf("%le", &X[0]);
  printf("\t Y[0] = "); scanf("%le", &Y[0]);
  for( i=1; i<np; i++ )
     { X[i]=X[i-1]-1;
       while( X[i]< X[i-1] )
           { printf("\n\t X[%d] = ",i); scanf("%le",&X[i]);}
       printf("\t Y[%d] = ", i); scanf("%le", &Y[i]);
      }
  printf("\n\n\t The absciss of the left point of the"
         " interval  x_init = "); scanf("%le", &a); b=a-1;
  while( b<a )
     { printf("\n\t The absciss of the right point of"
            " the interval x_fin  = ");scanf("%le", &b); }
  printf("\n\t The number of points in which the"
         " Lagrange's\n\t     polynomial function"
        " will be computed n = ");  scanf("%d", &n);
  step=(b-a)/(n-1);
  for( i=0; i<n; i++ )
     { xc=a+i*step; xL[i]=xc; yc=f_lagr(np, X, Y, xc);
       yL[i]=yc;
       printf("\n\t\t  %4d   %15.6le   %15.6le", i, xc, yc);
       if(i%22==0) getch();
     } getch();
  xmin = xL[0]<X[0] ? xL[0] : X[0];
  xmax = xL[n-1] > X[np-1] ? xL[n-1] : X[np-1];
  ymin = ymax=yL[0];
  for( i=0; i<n; i++ )
     { if(ymin>yL[i]) ymin=yL[i];
       if(ymax<yL[i]) ymax=yL[i]; }
  for( i=0; i<np; i++ )
     { if(ymin>Y[i]) ymin=Y[i];
       if(ymax<Y[i]) ymax=Y[i]; }
  printf("\n\n\t x_min = %le;  x_max = %le;", xmin, xmax);
  getch();
  printf("\n\t y_min = %le;  y_max = %le;", ymin, ymax);
  getch(); return 0;
}
```

Privind acest program se pot face următoarele observaţii :

1. După introducerea valorilor coordonatelor primului punct utilizatorul este obligat ca valorile introduse ale absciselor să fie crescătoare.

2. Se introduc valorile **a** , **b** şi **n** reprezentând valorile absciselor punctelor din stânga şi din dreapta intervalului în care se vor calcula valorile precum şi numărul de puncte în care se va face acest calcul. Evident că şi aici utilizatorul va fi obligat ca valoarea introdusă a lui **b** să fie mai mare decât cea a lui **a.**

3. După calculul şi afişarea valorilor absciselor curente şi a polinomului de interpolare Lagrange se trece la determinarea valorilor minime şi maxime atât a absciselor cât şi a ordonatelor, urmată de afişarea acestor valori.

4. Reţinerea valorilor absciselor curente cât şi a valorilor calculate ale polinomului de interpolare ca elemente ale şirurilor **xL** şi **yL** este necesară în cazul în care ar urma şi o reprezentare grafică a polinomului de interpolare Lagrange, aşa cum este în figura următoare :

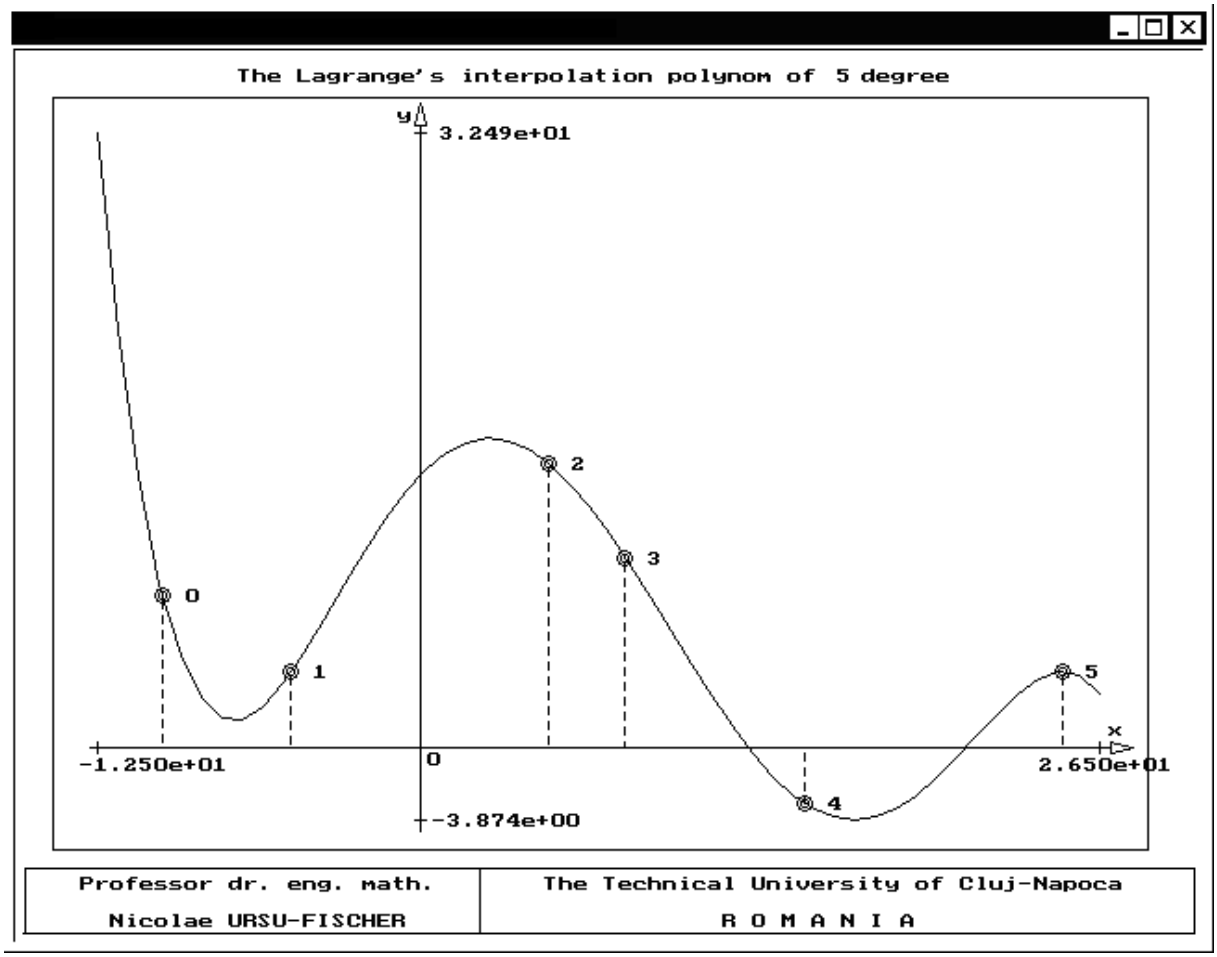


Fig. 10.1

Coordonatele celor şase puncte, prin care trece polinomul de interpolare Lagrange de gradul cinci, au fost următoarele: [-10; 8], [-5; 4], [5; 15], [8; 10], [15; -3] şi [25; 4] .

În cadrul exemplelor anterior prezentate s-au întâlnit următoarele situaţii:

- În funcţia unde se face apelul se utiliza numele funcţiei apelate, valorile parametrilor efectivi a acesteia atribuindu-se parametrilor formali, care erau variabile simple, realizându-se apelul prin valoare iar în funcţia care făcea apelul se returna o valoare (sau nu se returna nici o valoare dacă tipul funcţiei apelate era void). Acest mod de lucru s-a putut observa în cadrul Exemplelor nr. 10.1, 10.2 şi 10.3.

- În Exemplul nr. 10.4 s-a întâlnit o altă situaţie posibilă: trebuiau transmise funcţiei apelate atât valori care să se atribuie unor parametri formali variabile simple (deci apel prin valoare) cât şi valorile unor elemente de tablouri unidimensionale, transmiţându-se valori de

adrese, realizându-se deci apelul prin referinţă. Se returna o singură valoare ca şi în cadrul Exemplelor nr. 10.1, 10.2 şi 10.3.

O caracteristică comună a Exemplelor 10.1, 10.2, 10.3 şi 10.4 era aceea că valorile atribuite parametrilor formali nu se modificau.

Sunt însă multe situaţii când în cadrul funcţiei care se apelează ***valorile unor parametri formali care au primit valori în momentul apelului trebuie modificate şi returnate în funcţia care apelează***. Acest lucru este posibil numai prin utilizarea **apelului prin referinţă**, un exemplu foarte simplu fiind prezentat în continuare.

EXEMPLUL nr. 10.5 Se vor introduce două valori numerice care se vor atribui la două variabile **a** şi **b** iar apoi se vor schimba între ele aceste valori, afişându-se atât valorile iniţiale ale celor două variabile cât şi valorile schimbate.

Programul va conţine funcţia **main( )** şi încă două funcţii, cea care realizează afişarea **void displ_ab(double first, double second)** , care este conţinută în fişierul **displ_ab.cpp** şi funcţia **void exchange (double *x, double *y)** aparţinând efectiv programului sursă, unde ca parametri formali se utilizează două variabile de tip **pointer** deci în cazul folosirii (apelului) acestei funcţii ca parametri efectivi vor fi utilizate adresele celor două variabile, în interiorul funcţiei **exchange(…)** făcându-se un schimb de adrese. Acest lucru este necesar deoarece în funcţia **exchange(…)**, care se apelează, nu este posibil să modificăm valorile a doi parametri efectivi în cazul când aceştia se prezintă ca variabile de un anumit tip şi nu ca pointeri.

Se poate deci reţine următoarea regulă : ***în cadrul funcţiei unde se face apelul trebuie să se specifice adresa variabilei care se va modifica în funcţia apelată iar în funcţia apelată parametrul formal care va fi scris pe poziţia respectivă va fi declarat ca pointer.***

Programul sursă este prezentat în continuare.

```
//      schimb.cpp
#include <conio.h>
#include <stdio.h>
#include “displ_ab.cpp”  //Se include fisierul conţinând funcţia
void exchange(double *x,double *y);      //Declaraţia de funcţie
int main()
{ double a,b;
  printf(“\n\t The initial values:\n\n\t\t a = “); scanf(“%le”, &a);
  printf(“\t\t b = “); scanf(“%le”, &b);
  displ_ab( a, b ) ;  exchange( &a, &b ) ;
  printf(“\n\n\t The final (changed) values:\n”);
  displ_ab( a, b ) ;  getch(); return 0;
}
void exchange(double *x, double *y)  // x şi y  -  pointeri
{ double v;
  v=*x;     // *x şi *y - valorile variabilelor de tip double
  *x=*y;    //            spre care pointează  x  şi  y.
  *y=v; }
```

EXEMPLUL nr. 10.6. Un alt exemplu care este oarecum similar cu cel anterior este cel al unor funcţii care transformă un caracter – literă mare – în literă mică sau invers.

În programul care urmează, această operaţie se va face în două feluri, din punct de vedere al modului cum se face transmiterea datelor, între funcţia **main( )** unde se face apelul şi funcţiile apelate.

Se va ţine seama că un caracter literă mare **A, B, ... , Z** are reprezentări numerice (în zecimal) conform codificării ASCII în intervalul **[65 ; 90]** iar literele mici **a, b, ... , z** în intervalul **[97 ; 122]**. Pentru a transforma o literă mare în literă mică va trebui să se adune valoarea **32** la reprezentarea numerică ASCII a literei mari. În cazul când se doreşte transformarea unei litere mici în litera mare corespunzătoare va trebui să se scadă aceeaşi valoare **32** din reprezentarea numerică ASCII a literei mici.

Funcţiile ce se vor scrie vor acţiona numai asupra literelor, motiv pentru care înainte de a face operaţiile amintite se va verifica dacă respectivul caracter este literă mare (sau mică).

```
#include <conio.h>
#include <stdio.h>

int UPPER_1(int c);
int LOWER_1(int c);

void UPPER_2(char *c);
void LOWER_2(char *c);

int main(void)
{char c1='A', c2='a';
 clrscr();
 printf("\n\n UPPER_1(%c) = %c", c2, UPPER_1(c2));
 printf("\n\n LOWER_1(%c) = %c", c1, LOWER_1(c1));
 printf("\n\n %c  ---> ", c2); UPPER_2(&c2); printf("%c", c2);
 printf("\n\n %c  ---> ", c1); LOWER_2(&c1); printf("%c", c1);
 getch(); return 0;
}

int UPPER_1(int c)
{ if(97<=c && c<=122) c-=32;
  return c;  }

int LOWER_1(int c)
{ if(65<=c && c<=90) c+=32;
  return c;  }

void UPPER_2(char *c)
{ if(97<=*c && *c<=122) *c-=32; }

void LOWER_2(char *c)
{ if(65<=*c && *c<=90) *c+=32;  }
```

În funcţiile **UPPER_1(...)** şi **LOWER_1(...)** se foloseşte ca parametru formal o variabilă de tip **int** returnându-se o valoare de tip **int** . În cazul celor două funcţii cu toate că se modifică valoarea variabilei **c** de tip **char**, acest lucru nu are nici o influenţă asupra valorilor variabilelor **c2** respectiv **c1**, care sunt folosite ca parametri efectivi, acestea rămânând nemodificate .

Ca urmare a execuţiei primelor două instrucţiuni **printf(...)** se vor afişa următoarele :

```
UPPER_1(a) = A
LOWER_1(A) = a
```

În funcţiile **UPPER_2(...)** şi **LOWER_2(...)** parametrii formali sunt ***pointeri*** ai unor variabile de tip **char** iar parametrii efectivi (în cazul apelului funcţiilor în **main( )**) sunt ***adrese*** ale acestor variabile de tip **char** .

După cum se poate observa, între două tipăriri ale "aceleiaşi" variabile se intercalează apelul funcţiilor **UPPER_2(...)** respectiv **LOWER_2(...)** , acestea modificând caracterul care se găseşte la adresa respectivă :

```
a  --->  A
A  --->  a
```

Se va arăta în continuare, în Capitolul XI, că există funcţii standard care rezolvă această problemă, a transformării caracterelor tip literă mare în literă mică şi invers.

EXEMPLUL nr. 10.7 O problemă care apare în multe situaţii este cea privind formarea unui **şir** de tip **char** al cărui elemente să fie cifrele unui număr întreg introdus de la tastatură.

În acest scop s-a scris funcţia

```
void i_to_str(long int n, int n_dig, char *strdec)
```
[valori cunoscute în momentul apelării] [se returnează şirul determinat]

căreia i se transmite valoarea unui număr întreg **n** (de fapt de tip **long int** pentru ca problema să aibă un grad de generalitate mai mare), precum şi numărul de elemente **n_dig** ale şirului . Determinarea cifrelor componente ale numărului începe evident de la cele mai puţin semnificative, atribuirea lor elementelor şirului începând cu ultimele.

Dacă numărul de elemente al şirului este mai mare decât cel al cifrelor numărului atunci primele elemente vor fi caracterele **0**.

În urma execuţiei următoarelor instrucţiuni :

```
for(i=0; i<n_dig; i++) { strdec[n_dig-i-1]=n%10;
                         n-=strdec[n_dig-i-1];
                         n/=10; }
```

se realizează determinarea cifrelor componente ale numărului **n** .

Astfel, dacă s-ar considera n = 245 iar n_dig=5, atunci se vor parcurge următoarele :

```
i=0;  strdec[4]=245%10=5; n=245-5=240; n=240/10=24;
i=1;  strdec[3]= 24%10=4; n= 24-4= 20; n= 20/10= 2;
i=2;  strdec[2]=  2%10=2; n=  2-2=  0; n=  0/10= 0;
i=3;  strdec[1]=  0%10=0; n=  0-0=  0; n=  0/10= 0;
i=4;  strdec[0]=  0%10=0; n=  0-0=  0; n=  0/10= 0;
```

În funcţia **main( )** se face introducerea datelor (valorile lui **n** şi **n_dig**) , apelul funcţiei **i_to_str (...)** , ca urmare a acestui apel obţinându-se şirul de caractere **strdec** care se afişează (cu un spaţiu între caractere).

Programul este următorul:

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

void i_to_str(long int n, int n_dig, char *strdec);

int main(void)
{ int i, n_dig; long int n, N=0; char strdec[20];
  clrscr();
  printf("\n\n   The integer number [1 ... 2147483647] = ");
  scanf("%ld", &n);
  while(N<n)
     { printf("\n   The length of the string [1 ... 20] = ");
       scanf("%d", &n_dig);
       for(i=0; i<n_dig; i++) N+=9*pow(10, i);
     }
  i_to_str(n, n_dig, strdec);
  printf("\n   The string is the following : ");
  for(i=0; i<n_dig; i++) printf("%2d", strdec[i]);
  getch(); return 0;
}

void i_to_str(long int n, int n_dig, char *strdec)
{ int i;
  for(i=0; i<n_dig; i++) strdec[i]=0;
  for(i=0; i<n_dig; i++)
     { strdec[n_dig-i-1]=n%10; n-=strdec[n_dig-i-1]; n/=10; }
}
```

S-a putut observa că în funcţia **main( )** s-a impus condiţia ca valoarea introdusă pentru **n_dig** să fie suficient de mare pentru ca toate cifrele numărului să se poată înregistra în cadrul şirului.

Rezultatele se prezintă în felul următor :

```
The integer number [1 ... 2147483647] = 2456709

The length of the string [1 ... 20] = 3

The length of the string [1 ... 20] = 5

The length of the string [1 ... 20] = 10

The string is the following :  0 0 0 2 4 5 6 7 0 9
```

EXEMPLUL nr. 10.8 În Capitolul VIII s-au prezentat o serie de probleme privind sistemele de numeraţie şi unele proprietăţi ale unor numere zecimale, probleme rezolvate prin utilizarea în unele cazuri , a unui număr mare de cicluri suprapuse.

Acest lucru (folosirea unui număr mare de cicluri suprapuse) poate fi evitat prin determinarea cifrelor care intră în componenţa numărului, cu funcţia **i_to_str(...)** utilizată în programul anterior.

Se propune rezolvarea următoarei probleme : determinarea tuturor numerelor cu trei, patru şi cinci cifre la care suma cifrelor este egală cu valoarea radicalului de ordinul trei din respectivul număr :

$$\sqrt[3]{\overline{s_2 s_1 s_0}} = s_2 + s_1 + s_0 \ , \quad \sqrt[3]{\overline{s_3 s_2 s_1 s_0}} = s_3 + s_2 + s_1 + s_0 \ .$$

$$\sqrt[3]{\overline{s_4 s_3 s_2 s_1 s_0}} = s_4 + s_3 + s_2 + s_1 + s_0$$

În funcţia **main ( )** se parcurge intervalul de la **100** la **99999** şi pentru fiecare din numerele respective se determină şirul cifrelor componente, se face suma acestora şi se compară cu valoarea radicalului de ordinul trei din **n** , afişându-se în caz de egalitate.

Programul este următorul :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

void i_to_str(long int n, int n_dig, char *strdec);

int main(void)
{ long int n; int i, s; char strdec[20];
  clrscr();
  for(n=100; n<=99999; n++)
     { i_to_str(n, 5, strdec);
       for(s=0, i=0; i<5; i++) s+=strdec[i];
       if( fabs(pow(n, 1./3)-s) < 1.e-5)
        printf("\n %ld", n);
     }
  getch(); return 0;
}

void i_to_str(long int n, int n_dig, char *strdec)
{ int i;
  for(i=0; i<n_dig; i++) strdec[i]=0;
  for(i=0; i<n_dig; i++)
     { strdec[n_dig-i-1]=n%10; n-=strdec[n_dig-i-1]; n/=10; }
}
```

Se poate observa cum s-a făcut verificarea condiţiei :

```
if( fabs(pow(n, 1./3) - s) < 1.e-5 ) printf("\n\n %ld", n);
```

în loc să se verifice existenţa egalităţii ,

```
if( pow(n, 1./3) == s ) printf("\n\n %ld", n);
```

acest lucru fiind motivat de faptul că cele două valori care se compară nu sunt ambele de tip întreg iar din cauza unor erori de rotunjire a rezultatelor egalitatea ar putea să nu fie îndeplinită chiar şi în cazurile când ea există .

S-au obţinut următoarele rezultate :

512
4913
5832

17576
19683

În majoritatea exemplelor anterioare, parametrii formali, dacă au existat, au fost variabile simple, într-un caz, cel al polinomului de interpolare Lagrange (Exemplul nr. 10.4) un parametru formal a fost un tablou unidimensional, valorile elementelor căruia au fost utilizate în funcţia apelată, dar fără a fi modificate iar în alte cazuri (Exemplele nr. 10.7 şi 10.8) în interiorul funcţiei apelate s-a determinat un tablou unidimensional, un şir de caractere, în funcţia unde s-a făcut apelul transmiţându-se adresa acestui şir.

Este posibil şi cazul când

- **valorile tablourilor unidimensionale** care apar ca parametri efectivi **sunt modificate în funcţia care se apelează (cazul A)**.

De asemenea, în cazul în care se utilizează funcţii cu ajutorul cărora se fac diferite operaţii cu matrice mai sunt posibile şi situaţiile când

- **se transmit funcţiei** apelate **valorile elementelor unei matrice**, care **nu se modifică** în cadrul funcţiei (**cazul B**),
- **se returnează valorile elementelor unei matrice** care au fost **determinate în cadrul funcţiei apelate (cazul C)**,
- s**e modifică valorile unei matrice** care apare ca **parametru efectiv în cazul apelului funcţiei** respective (**cazul D**).

În toate aceste situaţii se face apelul prin referinţă, utilizându-se pointerii respectivelor tablouri.

În cele ce urmează se prezintă aceste situaţii ce se pot întâlni la transferul datelor între două funcţii.

Cazul A. Dacă într-o funcţie **func1(…)** se determină (sau se modifică) elementele unui tablou unidimensional **T** de tip **double**, având maximum 25 de elemente, care se returnează în funcţia care a făcut apelul, atunci cele două funcţii vor fi scrise sub următoarea formă :

```
int main( )
{ ……..
  double  ….. , T[25], ….  ;      // Se declară dimensiunea tabloului
  ……..
  func1 ( … , T , … ) ;  // În momentul apelului funcţiei func1 ( ... ) fie elementele
  ……..                   //  tabloului T sunt cunoscute şi urmează să fie modificate
                         // în funcţia ce se apelează, fie sunt nedeterminate şi urmează
                         // să fie calculate în funcţia func1 ( … )
 }

void func1 ( … , double  *T, … )  // Se declară pointerul T corespunzător tabloului
{ ……..                            // unidimensional
  ……..
  T[ k ]= ….. ;                   // Se determină valorile elementelor tabloului
  ……..
}
```

Cazurile B, C şi **D**. Dacă în funcţia **main( )** sau în altă funcţie se face apelul unei funcţii **func2(…)** între acestea trebuind să se facă un transfer de valori ale elementelor unei matrice **M,** atunci se va utiliza următoarea secvenţă de instrucţiuni:

```
int main( )
{ ........
  double  ... , M[10][15], .... ;   // Se declară dimensiunea matricei M
  ........
  func2(  ... ,  M , ... );         // Se utilizează notaţia M care reprezintă
   ........                         // pointerul corespunzător matricei (adresa
   ........                         // elementului M[0][0])
 }

void func2 ( ... , double M[ ] [15] , ... )  // În lista parametrilor formali
{ .......                                     // se face declaraţia corespunzătoare
  .......                                     // matricei, fără să se specifice numărul
  .......                                     // de linii al acesteia
}
```

EXEMPLUL nr. 10.9. Se va considera un program mai complex, dar relativ uşor de înţeles, care pe lângă funcţia **main( )** mai foloseşte un număr de funcţii, având următoarele utilizări :

- void **addmatr(** int **rows**, int **columns**, double **X**[ ][20], double **Y**[ ][20], double **Z**[ ][20]**)** - adunarea a două matrice **X** şi **Y** cu **rows** linii şi **columns** coloane, obţinând matricea **Z**,

- void **decompLU(** int **n**, double **A**[ ][20], double **L**[ ][20], double **U**[ ][20], int ***kod** **)** - determinarea a două matrice **L** şi **U** (triunghiulară jos şi triunghiulară sus) obţinute prin factorizarea matricei patrate **A** cu **n** linii şi coloane. Dacă factorizarea este posibilă atunci se va atribui variabilei spre care pointează pointerul **kod** valoarea **0** respectiv **1** în caz contrar,

- void **diffmatr (** int **rows**, int **columns**, double **X**[ ][20], double **Y**[ ][20], double **Z**[ ][20] **)** - scăderea a două matrice **X** şi **Y** cu **rows** linii şi **columns** coloane, obţinând matricea **Z**,

- void **inp_matr (** int **m**, int **n**, double **A**[ ][20] **)** - introducerea de la tastatură a valorilor elementelor matricei **A** cu **m** linii şi **n** coloane,

- void **multmatr (** int **m**, int **n**, int **l**, double **X**[ ][20], double **Y**[ ][20], double **Z**[ ][20]**)** - înmulţirea unei matrice **X** cu **m** linii şi **n** coloane cu o matrice **Y** cu **n** linii şi **l** coloane şi obţinerea unei matrice **Z** (cu **m** linii şi **l** coloane),

- void **names ( )** - afişarea unor nume,

- void **out_matr (** int **m**, int **n**, double **A**[ ][20] **)** - afişarea valorilor unei matrice **A** cu **m** linii şi **n** coloane,

- void **sparsemv (** int **m**, int **n**, double **S**[ ][20], int ***INDEX**, double ***VECT**, int ***kk)** - formarea a două şiruri, **INDEX** şi **VECT**, conţinând indicii echivalenţi ai elementelor nenule respectiv valorile acestor elemente ale unei matrice "rare" (**sparse** în l. engleză), notate cu **S,** având **m** linii şi **n** coloane iar **kk** este adresa variabilei întregi reprezentând numărul de elemente al şirurilor **INDEX** şi **VECT**,

- void **sparsevm (** int **m**, int **n**, int **kk**, int **INDEX**[ ], double **VECT**[ ], double **S**[ ][20]**)** - formarea unei matrice **S** cu **m** linii şi **n** coloane, cunoscându-se cele **kk** elemente ale şirurilor **INDEX** şi **VECT** conţinând indicii echivalenţi respectiv valorile elementelor nenule ale matricei **S**, presupusă matrice rară,

-void **tranmatr (** int **n**, double **A**[ ][20] **)** - transpunerea unei matrice **A** patrate, cu **n** linii şi coloane, matricea ce rezultă fiind păstrată de asemenea în **A**.

Se consideră că toate aceste funcţii sunt conţinute în fişiere, având acelaşi nume cu cel al funcţiei, aceste fişiere găsindu-se în acelaşi director cu funcţia **main( ).**

În cadrul acestui program se vor realiza următoarele:

- afişarea numelui autorului, cu funcţia **names( ),**
- introducerea valorii lui **m=n**,
- introducerea valorilor matricelor **A** şi **B**, utilizând de două ori funcţia **inp_matr(…)** şi afişarea acestora utilizând de două ori funcţia **out_matr(…),**
- calculul sumei matricelor **A+B** şi obţinerea matricei **C**, cu funcţia **addmatr(…),** urmată de afişarea matricei **C**, cu funcţia **out_matr(…),**
- transpunerea matricei patrate **A**, cu funcţia **tranmatr(…),** urmată de afişarea matricei transpuse, cu funcţia **out_matr(…),**
- transpunerea matricei **A** anterior transpuse, cu funcţia **tranmatr(…)** urmată de afişarea acesteia cu funcţia **out_matr(…)**,
- descompunerea matricei **A** într-un produs de două matrice **L** şi **U**, cu funcţia **decompLU(…)**, urmată de afişarea celor două matrice, utilizând de două ori funcţia **out_matr(…)**, dacă respectiva descompunere s-a putut efectua,
- efectuarea produsului dintre matricele **L** şi **U** obţinând matricea **P**, cu funcţia **multmatr(…)**, urmată de afişarea matricei **P**,
- efectuarea diferenţei dintre matricea **P** şi matricea iniţială **A** şi obţinerea matricei **Q**, cu ajutorul funcţiei **diffmatr(…)**, urmată de afişarea matricei **Q** (elementele căreia vor fi nule sau foarte apropiate de zero),
- introducerea şi afişarea unei matrice dreptunghiulare **A** cu **m** linii şi **n** coloane,
- determinarea a două şiruri, primul conţinând indicii echivalenţi iar al doilea conţinând valorile elementelor nenule ale matricei **A**, cu funcţia **sparsemv(…)**,
- afişarea celor două şiruri,
- formarea unei matrice dreptunghiulare **A** cunoscând elementele a două şiruri, primul conţinând indicii echivalenţi iar al doilea valorile elementelor nenule, cu funcţia **sparsevm(…)**, urmată de afişarea acestei matrice.

Programul sursă conţine directivele de preprocesare şi funcţia **main( )** şi este prezentat în continuare :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

#include "addmatr.cpp"   // (m,n,X[][20],Y[][20],Z[][20])
#include "decompLU.cpp"  // (n,A[][20],L[][20],U[][20],*kod)
#include "diffmatr.cpp"  // (m,n,X[][20],Y[][20],Z[][20])
#include "inp_matr.cpp"  // (m,n,A[][20])
#include "multmatr.cpp"  // (m,n,l,A[][20],B[][20],C[][20])
```

```
#include “names.cpp”     // ()
#include “out_matr.cpp”  // (m,n,A[][20])
#include “sparsemv.cpp”  // (m,n,S[][20],*INDEX,*VECT,*kk)
#include “sparsevm.cpp”  // (m,n,kk,INDEX[],VECT[],S[][20])
#include “tranmatr.cpp”  // (n,A[][20])

int main(void)
{int i, INDEX[100], j, kod, m, n, kk;
 double  A[20][20], B[20][20], L[20][20], P[20][20],
         Q[20][20], U[20][20], VECT[100];
names() ; getch(); clrscr();
//
printf(“\n\t The number of rows and column = “);
scanf(“%d”,&m); n=m;
//
printf(“\n\t The matrix  A”);
inp_matr( m, n, A ) ;  out_matr( m, n, A ) ; getch();
//
printf(“\n\t The matrix  B”);
inp_matr( m, n, B ) ; out_matr( m, n, B ) ; getch();
//
printf(“\n\t The sum  C = A + B”);
addmatr( m, n, A, B, U ) ; out_matr( m, n, U ) ; getch();
//
printf(“\n\n\t The matrix  A_t”);
tranmatr( n, A ) ; out_matr( n, n, A ) ; getch();
//
printf(“\n\t The initial matrix  A”);
tranmatr( n, A ) ; out_matr( n, n, A ) ; getch();
//
decompLU( n,  A,  L,  U,  &kod ) ;
printf(“\n\t The decomposition  A = L * U”
       “\n\t\t kod = %d”, kod); getch();
if(kod==0)
  { printf(“\n\t Matrix  U”); out_matr( n, n, U ) ; getch();
    printf(“\n\t Matrix  L”);  out_matr( n, n, L ) ; getch();
    printf(“\n\t The product   L * U”);
    multmatr( n, n, n, L, U, P );out_matr( n, n, P ); getch();
    printf(“\n\t The difference   L*U - A”);
    diffmatr( n, n, P, A, Q ); out_matr( n, n, Q ) ; getch();
  }
//
clrscr();
printf(“\n\t Input of sparse matrix\n”
       “\n\t Number of rows      m = “); scanf(“%d”, &m);
printf(“\n\t Number of columns  n = “); scanf(“%d”, &n);
inp_matr( m, n, A ) ; out_matr( m, n, A ) ; getch();
//
sparsemv( m, n, A, INDEX, VECT, &kk ) ;
for( i=0; i<=kk;  i++ )
    printf(“\n\t %3d\t %10.4le”, INDEX[i], VECT[i]);
```

```
getch();
//
sparsevm(m,n,kk,INDEX,VECT,A); out_matr(m,n,A ); getch(); return 0;
}

//  -------- Fisierul  addmatr.cpp --------
//
void addmatr( int rows, int columns, double X[][20],
              double Y[][20], double Z[][20] )
{ int i, j;
  for( i=0; i<rows; i++ )
     {for( j=0; j<columns; j++ )Z[i][j]=X[i][j]+Y[i][j];}
}

// -------- Fisierul  decompLU.cpp --------
//
void decompLU( int n, double A[][20], double L[][20],
                      double U[][20], int *kod )
{ int i, j, k; double s;
  if(A[0][0]==0.0) { *kod=1; return; }
  for( i=0; i<n; i++ )
     { for(j=0; j<n; j++ ) L[i][j]=U[i][j]=0.0; }
  for( i=0; i<n; i++ )
     { L[i][0]=A[i][0]; U[0][i]=A[0][i]/L[0][0]; U[i][i]=1.0;}
  for( j=1; j<n; j++ )
     { for( i=1; i<n; i++ )
          { if(i<j)
               { if(L[i][i]==0.0 ) { *kod=1; return; }
                 for(s=0.0, k=0; k<=i-1; k++)
                      s+=L[i][k]*U[k][j];
                 U[i][j]=(A[i][j] - s)/L[i][i]; }
            else
               { for(s=0.0,k=0;k<=j-1;k++) s+=L[i][k]*U[k][j];
                 L[i][j]=A[i][j]-s; }
          }
     }
*kod=0; return;
}

//  -------- Fisierul  diffmatr.cpp --------
//
void diffmatr( int rows, int columns, double X[][20],
                      double Y[][20], double Z[][20] )
{ int i, j;
  for( i=0; i<rows; i++ )
    {for(j=0;j<columns;j++)Z[i][j]=X[i][j]-Y[i][j];}
}
```

```
//  -------- Fisierul  inp_matr.cpp --------
//
void inp_matr( int m, int n, double A[][20] )
{ int i, j; double v;
  printf("\n\n\t The matrix has %d rows and %d columns.",m,n);
  printf("\n   Line   Column   Element value\n");
  for( i=0; i<m; i++ )
     { for(j=0; j<n; j++ )
       { printf("   %2d    %2d   =   ",i,j);
         scanf("%le",&v); A[i][j]=v; }
     }
}
//  -------- Fisierul   multmatr.cpp --------
//
void multmatr( int m, int n, int l, double X[][20],
                      double Y[][20], double Z[][20] )
{ int i, j, k;
  for(i=0; i<m; i++)
     {for(j=0; j<l; j++)
         {for(Z[i][j]=0.0,k=0;k<n;k++) Z[i][j]+=X[i][k]*Y[k][j];}
     }
}

//  --------  Fisierul  names.cpp --------
//
void names(void)
{ clrscr(); printf("\n\n Nick  &  Mike\n\n"); }
//  -------- Fisierul  out_matr.cpp --------
//
void out_matr( int m, int n, double A[][20] )
{ int i, j; printf("\n\n");
  for( i=0; i<m; i++ )
      { for( j=0; j<n; j++ )  printf("%12.4le", A[i][j]);
        printf("\n");
      }
  printf("\n\n");
}

//  -------- Fisierul  sparsemv.cpp --------
//
void sparsemv( int m, int n, double S[][20], int *INDEX,
               double *VECT, int *kk )
{ int i, j, k=-1;
  for( i=0; i<m; i++ )
     { for( j=0; j<n; j++ )
          { if( S[i][j] != 0.0 )
              {k++; INDEX[k]=i*n+j; VECT[k]=S[i][j]; *kk=k;}
          }
     }
return;
}
```

```
//  -------- Fisierul  sparsevm.cpp --------
//
void sparsevm( int m,  int n,  int kk,  int INDEX[],
               double VECT[], double S[][20] )
{ int i, r, c;
  for( i=0; i<= kk; i++ )
     { r=INDEX[i]/n; c=INDEX[i]-r*n; S[r][c]=VECT[i];}
}

//  -------- Fisierul  tranmatr.cpp --------
//
void tranmatr( int n, double A[][20] )
{ int i, j; double t;
for( i=0; i<n-1; i++ )
   { for( j=i+1; j<n; j++ )
        { t=A[i][j]; A[i][j]=A[j][i]; A[j][i]=t; }
   }
}
```

Observaţia nr. 1. În acest program se pot întâlni aproape toate situaţiile care intervin în cazul transmiterii valorilor între funcţia unde se face apelul şi funcţia apelată şi reciproc.

1. Valorile unor variabile, expresii sau constante se transmit funcţiei apelate :

```
int main( )
{ .....
  inp_matr(m, n, A) ;                          //  apelul funcţiei în main( )
  .....
}

void inp_matr ( int m, int n, double  A[ ][20] )     //  antetul funcţiei apelate
{  .....  }
```

în acest caz făcându-se apelul prin valoare.

2. Valorile unor variabile determinate în funcţia apelată se transmit (se returnează) funcţiei în care s-a făcut apelul :

```
int main( )
{ .....
  decompLU ( n, A, L, U, &kod ) ;                 // apelul funcţiei în main( )
   .....
}

void decompLU( int n, double A[ ][20], double L[ ][20], double U[ ][20], int *kod )
{ .....                                           // antetul funcţiei apelate
   .....  }
```

În funcţia apelată, în antet există o declarare a unui pointer la o variabilă, aici de tip **int**. În interiorul acestei funcţii se atribuie o valoare variabilei care are adresa **kod**, deci lui ***kod**. În cazul apelului acestei funcţii se utilizează o adresă pe locul parametrului formal de tip pointer, la acestă adresă găsindu-se valoarea determinată în funcţia apelată.

3. Valorile elementelor unui tablou unidimensional se transmit funcţiei apelate:

```
int main( )
{  ..........................................................
   int … , INDEX[100], … ;  double … , VECT[100], … ;
                                          // dimensiunile tablourilor se declară în main( )
   ..........................................................
   sparsevm ( m, n, INDEX, VECT, A ) ; // apelul funcţiei în main( )
   ..........................................................
}

void sparsevm(int m, int n, int INDEX[ ], double VECT[ ], double A[ ][20] )
{ .....................................          // antetul funcţiei apelate
  .....................................  }
```

Dimensiunile tablourilor se declară în funcţia unde se face apelul iar în instrucţiunile de apel se utilizează denumirile tablourilor, deci pointerii constanţi corespunzători primelor elemente ale acestora. În antetul funcţiei apelate nu trebuie precizate dimensiunile tablourilor, corespondenţa stabilindu-se prin transmiterea valorilor adreselor primelor elemente.

4. Valorile elementelor unui tablou unidimensional, calculate (sau modificate) în funcţia apelată, se transmit (se returnează) funcţiei în care s-a făcut apelul :

```
int main( )
{  ..........................................................
   int … , INDEX[100], … ;  double … , VECT[100], … ;
   ..........................................................          // dimensiunile tablourilor se declară în main( )
   sparsemv(m, n, A, INDEX, VECT, &kk ) ; // apelul funcţiei în main( )
   ..........................................................
}

void sparsemv(int m, int n, double S[ ][20], int *INDEX, double *VECT, int *kk )
                                          // antetul funcţiei apelate
{  ..................................................
   .....................................................  }
```

În funcţia care se apelează, în antet se declară variabile de tip pointer pe locul cărora în lista parametrilor efectivi apar pointerii constanţi corespunzători tablourilor unidimensionale, dimensiunile cărora se declară în funcţia în care se face apelul.

Este posibilă şi scrierea parametrilor formali ca şi în cazul 3, deci sub forma :

```
void sparsemv ( int m, int n, double S[ ][20], int INDEX[ ], double VECT[ ], int *kk )
                                          // antetul funcţiei apelate
```

5. Valorile elementelor unor matrice se transmit funcţiei apelate iar în respectiva funcţie se determină valorile elementelor unei matrice care trebuie transmise (returnate) funcţiei care face apelul :

```
double … , A[20][20], B[20][20], U[20][20], … ;
                                    // dimensiunea matricelor se declară în main( )
…….
addmatr ( m, n, A , B , U );        // apelul funcţiei în main( )

void addmatr ( int rows, int columns, double X[ ][20], double Y[ ][20],
               double Z[ ][20] )    // antetul funcţiei apelate
```

Dimensiunile şi tipul matricelor se declară în funcţia în care se face apelul. Ca parametri efectivi se folosesc pointerii constanţi corespunzători primelor elemente ale matricelor, adică numele acestor matrice. Ca parametri formali se utilizează de asemenea pointerii matricelor, precizându-se numai numărul coloanelor matricelor, deoarece după cum s-a arătat anterior (vezi Capitolul IX - Pointeri) adresa unui element al unui tablou multidimensional nu depinde de mărimea maximă a primului indice (în acest caz de numărul de linii al matricelor). După cum se poate observa şi în acest caz se face o transmitere de adrese.

Observaţia nr. 2. În continuare se arată numărul de utilizări al fiecărei funcţii :

Numele funcţiei	*Nr. de utilizări*
addmatr(…)	1
decompLU(…)	1
diffmatr(…)	1
inp_matr(…)	3
multmatr(…)	1
nick( )	1
out_matr(…)	11
sparsemv(…)	1
sparsevm(…)	1
tranmatr(…)	2

Cu toate că o mare parte din aceste funcţii s-au utilizat o singură dată în acest program, deoarece sunt frecvent folosite în diverse programe, a fost util ca să se scrie sub forma unor funcţii independente, pentru ca să se poată utiliza şi în alte programe.

Observaţia nr. 3. Pentru a înţelege algoritmul de calcul programat în cadrul funcţiei **decompLU(…)**, utilizată pentru a realiza factorizarea unei matrice patratice, se pot consulta lucrările: [Carn69], [DeMa73] cap. VIII, [Pres94] pg. 43-48, [Ursu00], pg. 73-80 ş. a.

Observaţia nr. 4. Matricele rare (sparse matrix) se utilizează frecvent în probleme de calcul legate de **metoda elementului finit**.

EXEMPLUL nr. 10.10. Se consideră un sistem de **n** ecuaţii liniare neomogene cu **n** necunoscute şi se cere elaborarea unui program pentru rezolvarea acestui sistem cu metoda

pivotării parţiale Gauss-Jordan (algoritmul se poate găsi prezentat în lucrările [Carn69], [Enge96], [Pres90], [Ursu79], [Ursu00] şi altele).

Se pleacă de la sistemul de ecuaţii scris sub formă matriceală ,

$$\begin{bmatrix} a_{1,1} & a_{1,2} & .. & a_{1,n} \\ a_{2,1} & a_{2,2} & .. & a_{2,n} \\ .. & .. & .. & .. \\ a_{n,1} & a_{n,2} & .. & a_{n,n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ .. \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ .. \\ b_n \end{bmatrix}, \qquad \mathbf{A\ X = B\ ,}$$

care în urma unor operaţii elementare de echivalenţă:

- împărţirea unei ecuaţii cu o constantă diferită de zero,
- adunarea unei ecuaţii înmulţită cu o constantă la o altă ecuaţie din sistem,
- modificarea ordinii de scriere a ecuaţiilor în sistem,

va putea fi scris sub forma :

$$\begin{bmatrix} 1 & 0 & .. & 0 \\ 0 & 1 & .. & 0 \\ .. & .. & .. & .. \\ 0 & 0 & .. & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ .. \\ x_n \end{bmatrix} = \begin{bmatrix} s_1 \\ s_2 \\ .. \\ s_n \end{bmatrix}, \qquad \mathbf{I\ X = S\ ,}$$

Se poate observa că în urma acestor operaţii se va modifica atât matricea **A** a coeficienţilor necunoscutelor, aceasta devenind matrice unitate, cât şi vectorul **B** conţinând iniţial termenii liberi, pe locul cărora vor fi soluţiile determinate.

Acest algoritm (rezolvarea sistemelor de ecuaţii algebrice liniare cu metoda **"pivotării parţiale"**) se va putea programa sub forma unei funcţii ([Ursu00] pg. 196-203), prezentată în continuare :

```
//       Fisierul   gauss.cpp
//
void gauss(int n, double A[ ][20], double *B, double eps, int *kod)
{ int i, imax, ip, j; double piv, t, tmax;
  for( ip=0; ip<n; ip++ )
      {// Step I
       tmax=fabs(A[ip][ip]); imax=ip;
       for( i=ip; i<n; i++ )
          { if( fabs(A[i][ip] ) >=  tmax)
               { tmax=fabs( A[i][ip] ); imax= i; }
          }
       // Step II
       if( tmax < eps ) { *kod=1; return; }
       // Step III
       for(j=ip; j<n; j++)
           { t=A[ip][j]; A[ip][j]=A[imax][j]; A[imax][j]=t; }
       t=B[ip]; B[ip]=B[imax]; B[imax]=t;
       // Step IV
       piv=A[ip][ip]; B[ip]=B[ip]/piv;
       for( j=ip; j<n; j++ ) A[ip][j]=A[ip][j]/piv;
```

```
      // Step V
      for( i=0; i<n; i++ )
         { if ( i != ip )
              { t=A[i][ip];
                for( j=ip; j<n; j++ ) A[i][j]-=A[ip][j]*t;
                B[i]=B[i]-B[ip]*t;
              }
         }
     }
*kod=0; return;
}
```

iar utilizarea acestei funcţii se va putea face în cadrul unei alte funcţii, după cum urmează:

```
int main( )
{ int n, kod, ....... ;
   double  A[20][20], B[20], eps = 1.e-15, ... ;
   ..........................
   gauss( n, A, B, eps, &kod );
   ..........................
}
```

În cazul parametrilor formali **int n** şi **double eps** aceştia primesc ca valori pe cele ale variabilelor **n** şi **eps** (se transferă valori) din funcţia **main( )**, în cazul parametrului formal pointer **int *kod** se atribuie o valoare întreagă (**0** sau **1**) variabilei care are adresa dată de pointerul **kod** iar în funcţia **main( )** ca parametru efectiv se utilizează adresa variabilei, **&kod**, transferându-se din funcţia apelată în cea în care s-a făcut apelul o adresă.

În ceea ce priveşte matricea **A** şi vectorul **B**, în instrucţiunea de apel se folosesc pointerii constanţi (denumirile matricei şi ale vectorului) iar în antetul funcţiei apelate în cazul matricei **A** se declară tipul matricei şi numai cea de-a doua dimensiune (numărul de coloane) iar în cazul tabloului unidimensional (vectorul **B**) se declară pointerul acestuia.

10.3 Pointeri la funcţii.

În cadrul Capitolului IX tratând despre pointeri, cât şi în paragrafele anterioare din acest capitol s-au folosit pointerii pentru a indica adresele primilor bytes (octeţi) de memorie unde se înregistrează valorile unor caractere, variabile simple sau valorile unor elemente de tablouri.

Similar cu cazul tablourilor, numele unei funcţii este un pointer spre funcţia respectivă. Dacă în cazul tabloului pointerul conţinea adresa primului element, în cazul funcţiilor acesta se referă la locul (adresa) din memorie unde începe ***codul executabil*** corespunzător funcţiei.

Dispunând de pointerii la funcţii, denumirile acestora se pot utiliza ca argumente în alte funcţii.

Pentru a obţine adresa unei funcţii se utilizează numele acesteia, neurmat de paranteze şi argumente.

Declararea unui pointer la o funcţie se face în felul următor :

tip (*nume_pointer)(lista_de_parametri_formali) ;

unde **tip** poate fi **void** dacă funcţia nu returnează nici o valoare sau unul din tipurile **char, int**, **float**, **double** dacă se returnează o valoare.

Pentru a apela funcţia a cărei adresă este **nume_pointer** se va scrie instrucţiunea de apel în felul următor :

(*nume_pointer)(listă_de_parametri_efectivi) ;

Este deosebit de important să se respecte sintaxa scrierii corecte în cazul declaraţiei şi a folosirii pointerilor la funcţii, astfel dacă în loc de **(*nume_pointer)** s-ar scrie ***nume_pointer**, fără paranteze, s-ar înţelege că e vorba de o funcţie care returnează o valoare de tip pointer şi nu de un pointer la funcţie **!**

EXEMPLUL nr. 10.11. Pentru a exemplifica aceste probleme, se consideră un program foarte simplu, cu care se vor tipări valorile a trei funcţii trigonometrice, **sin(x)**, **cos(x)** şi **tan(x)**, pentru anumite valori ale argumentelor, utilizând pointerii la funcţii.

Aceşti pointeri se notează cu **ps**, **pc** şi **pt**, declarându-se conform regulii enunţate şi cu specificarea tipului **double**, deoarece toate cele trei funcţii returnează valori de acest tip.

Programul se prezintă în continuare :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

#define PI 3.1415926

int main()
{ double (*ps)(double), (*pc)(double), (*pt)(double);
  ps=sin; pc=cos; pt=tan;
  printf("\n\t Pointer la sin = %p", ps);
  printf("\n\t Pointer la cos = %p", pc);
  printf("\n\t Pointer la tan = %p", pt);
  printf("\n\t\t\t sin(PI/6) = %le", (*ps)(PI/6));
  printf("\n\t\t\t cos(PI/6) = %le", (*pc)(PI/6));
  printf("\n\t\t\t tan(PI/4) = %le", (*pt)(PI/4));
  getch(); return 0;
}
```

Se poate observa că după ce s-au atribuit celor trei pointeri adresele funcţiilor şi s-au afişat, în cadrul celor trei funcţii **printf(...)** s-au folosit apelurile de funcţii, dar nu în modul cu care am fost familiarizaţi până acum,

sin (PI/6) cos (PI/6) tan(PI/4)

ci folosind pointerii la funcţie:

(*ps) (PI/6) (*pc) (PI/6) (*pt) (PI/4)

Rezultatele obţinute sunt următoarele :

```
Pointer la sin = 8B6F:0ECB
Pointer la cos = 8B6F:104C
Pointer la tan = 8B6F:0F2A

           sin(PI/6) = 5.000000e-01
           cos(PI/6) = 8.660254e-01
           tan(PI/4) = 1.000000e+00
```

Este evident că în cazul acestui program nu se observă nici un avantaj al folosirii pointerilor la funcţii, importanţa acestora fiind observabilă în exemplele următoare.

EXEMPLUL nr. 10.12. Un alt program relativ simplu, prezentat în continuare, utilizează de asemenea pointerii la funcţii iar spre deosebire de cel anterior, în acesta pointerii la funcţii vor fi utilizaţi ca parametri formali în cadrul unei funcţii.

Şi în acest program se vor folosi pointerii **ps** şi **pc** corespunzători funcţiilor matematice **sin** şi **cos** . Se va calcula valoarea expresiei

$$\mathbf{E = \left|\sin(x)\right| + \left|\cos(x)\right|}$$

pentru valori ale lui **x** din intervalul **[0 , π]**, cu pasul **π/10**.

Se poate observa că modul cum se declară pointerii la funcţii şi atribuirea adreselor funcţiilor **sin** şi **cos** este identic ca şi în cazul Exemplului nr. 10.11.

Pentru a calcula valoarea expresiei s-a elaborat funcţia **calcul (…)** care are ca argumente formale pointerii la funcţii.

În cadrul comentariilor sunt date toate informaţiile necesare pentru a înţelege funcţionarea acestui program.

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

double calcul(double x, double (*ps)(double),double (*pc)(double));
                                        // Declaratia functiei

int main(void)
{ int i;
  double PI, x, (*ps)(double), (*pc)(double) ;
                        // Declaratiile pointerilor la functii
  PI=4*atan(1.0);
  ps=sin; pc=cos;      // Valorile adreselor functiilor se
                       // atribuie pointerilor la functii
  printf("\n\n   i     x       |sin(x)| + |cos(x)|\n");
  for( i=0,x=0; x<=PI; x+=PI/10 )
     { i++;
       printf("\n %3d  %6.2f  %24.16le",
              i, x*180/PI, calcul( x, ps, pc ) );
                  // La apelul functiei ca parametri
                  // efectivi se folosesc pointerii la functii
     }
  getch(); return 0;
}

double calcul( double x, double (*ps)(double),
               double (*pc)(double) )
        // Ca parametri formali se utilizeaza pointeri
        // la functii, precizandu-se tipul valorii returnate
        // de functie si tipul argumentelor
```

```
{ double v;
  v=fabs((*ps)(x)) + fabs((*pc)(x));
     // In calcule functiile sunt inlocuite cu pointerii la functii
  return v;
}
```

Ca urmare a execuţiei acestuia s-au obţinut rezultatele:

```
 i     x       |sin(x)| + |cos(x)|

 1    0.00    1.0000000000000000e+00
 2   18.00    1.2600735106701009e+00
 3   36.00    1.3968022466674206e+00
 4   54.00    1.3968022466674206e+00
 5   72.00    1.2600735106701011e+00
 6   90.00    1.0000000000000000e+00
 7  108.00    1.2600735106701011e+00
 8  126.00    1.3968022466674204e+00
 9  144.00    1.3968022466674206e+00
10  162.00    1.2600735106701011e+00
11  180.00    1.0000000000000002e+00
```

EXEMPLUL nr. 10.13. În programul următor se vor calcula valorile radicalilor

$$\mathbf{r_2 = \sqrt{2}\,;\ r_3 = \sqrt[3]{2}\,;\ r_4 = \sqrt[4]{2}\,;\ r_5 = \sqrt[5]{2}\,;\ r_6 = \sqrt[6]{2}\,;\ r_7 = \sqrt[7]{2}\,;\ r_8 = \sqrt[8]{2}\,;}$$

rezolvând pe rând ecuaţiile

$$\mathbf{f(x) = x^m - 2 = 0, \qquad m = 2, 3, \ldots, 8}$$

cu metoda bisecţiei (a înjumătăţirii intervalelor), aplicată de **n** ori.

Fără a utiliza pointerii la funcţii acest program s-ar putea elabora în felul următor: în funcţia **main ( )** să fie incluse funcţiile pentru introducerea datelor, să fie programat algoritmul de rezolvare al ecuaţiei cu metoda bisecţiei iar alegerea funcţiei dorite să se facă cu ajutorul instrucţiunii **switch**

Utilizând pointerii la funcţii putem face ca un astfel de program să aibă un grad de generalitate mai mare. Astfel, se va elabora o funcţie **bisect (…)** în care se va programa algoritmul de rezolvare cu metoda bisecţiei, la care unul din parametrii formali va fi un pointer la funcţie, care în cazul apelului acesteia va fi înlocuit cu adresa funcţiei care se doreşte a fi utilizată.

În funcţia **main ( )** se poate observa că funcţia **bisect (…)** este apelată de 7 ori, de fiecare dată utilizându-se un alt pointer la funcţie, deci transmiţându-se adresa funcţiei care conţine expresia ecuaţiei ce trebuie rezolvată, în cadrul programului sursă fiind incluse toate cele şapte funcţii utilizate.

Programul se prezintă în continuare :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

double bisect( double a, double b, int n, double (*func)(double) );
double func02( double x );
double func03( double x );
```

```
double func04( double x ) ;
double func05( double x ) ;
double func06( double x ) ;
double func07( double x ) ;
double func08( double x ) ;

int main(void)
{ int n; double a=1.0, b=2.0, r2, r3, r4, r5, r6, r7, r8;
  printf("\n\n\t\t  n = "); scanf("%d", &n);
  r2=bisect(a, b, n, func02);
       // Se foloseste pointerul functiei ca param. efectiv
  printf("\n\t  2 ** 0.5     = %12.10f", r2);
  r3=bisect( a, b, n, func03 ) ;
  printf("\n\t  2 ** (1/3)   = %12.10f", r3);
  r4=bisect( a, b, n, func04 ) ;
  printf("\n\t  2 ** 0.25    = %12.10f", r4);
  r5=bisect( a, b, n, func05 ) ;
  printf("\n\t  2 ** 0.2     = %12.10f", r5);
  r6=bisect( a, b, n, func06 ) ;
  printf("\n\t  2 ** (1/6)   = %12.10f", r6);
  r7=bisect( a, b, n, func07 ) ;
  printf("\n\t  2 ** (1/7)   = %12.10f", r7);
  r8=bisect( a, b, n, func08 ) ;
  printf("\n\t  2 ** 0.125   = %12.10f", r8);
  getch(); return 0;
}

double bisect( double a, double b, int n, double (*func)(double) )
                 // Ca parametru formal se utilizeaza un pointer
                 // la functie, aceasta avand un parametru de tip
                 // double si returnand o valoare de tip double.
{ int i; double d, m, s, vm, vs;
  s=a; d=b; vs=(*func)(s);
  for( i=0; i<n; i++ )
     { m=0.5*(s+d); vm=(*func)(m);
       if(vs*vm < 0 ) { d=m; continue; }
       if(vs*vm > 0 ) { s=m; vs=vm; continue; }
       if(vs*vm == 0) { return m; }
     }
  return m;
}

double func02( double x )
{ return  x*x-2; }

double func03( double x )
{ return x*x*x-2; }

double func04( double x )
{ return pow(x, 4)-2; }

double func05( double x )
{ return pow(x, 5)-2; }
```

```
double func06( double x )
{ return pow(x, 6)-2; }

double func07( double x )
{ return pow(x, 7)-2; }

double func08( double x )
{ return pow(x, 8)-2; }
```

Este evident că acest mod de a concepe un astfel de program este mult mai favorabil, deoarece funcţia **bisect (…)** va putea fi folosită pentru rezolvarea oricărei ecuaţii care este scrisă într-un fişier de forma celor utilizate

```
double funcmn( double x )
{ ..................... }
```

Utilizând acest program, de două ori, s-au obţinut rezultatele :

```
          n = 15

2 ** 0.5     = 1.4142150879
2 ** (1/3)   = 1.2599182129
2 ** 0.25    = 1.1891784668
2 ** 0.2     = 1.1487121582
2 ** (1/6)   = 1.1224670410
2 ** (1/7)   = 1.1040954590
2 ** 0.125   = 1.0904846191

          n = 25

2 ** 0.5     = 1.414213580
2 ** (1/3)   = 1.259921041
2 ** 0.25    = 1.189207068
2 ** 0.2     = 1.148698397
2 ** (1/6)   = 1.122462040
2 ** (1/7)   = 1.104089283
2 ** 0.125   = 1.090507159
```

S-au scris cu caractere bold cifrele exacte din cadrul rezultatelor, cu creşterea numărului de aplicări al metodei crescând precizia rezultatelor obţinute.

Un alt exemplu de folosire al pointerilor la funcţii se poate găsi în [Negr_1] pg. 361-363, privind calculul valorii unei integrale definite cu metoda trapezelor, utilizând diferite funcţii sub semnul integralei, exemplu existent şi în alte cărţi.

10.4 Variabile globale şi locale.

După cum s-a observat, în toate programele sursă anterioare, fie compuse dintr-o singură funcţie (ca în Capitolele IV ~ IX) sau din mai multe funcţii (Capitolul X şi următoarele), pentru toate variabilele care erau utilizate era obligatorie o declarare a acestora, în care se specifica **tipul** valorilor pe care le vor putea lua.

Aceste declaraţii s-au făcut la începutul fiecărei funcţii şi valabilitatea lor era numai în funcţia care le conţinea.

Variabilele declarate în acest mod se numesc **locale** iar valorile lor se alocă pe **stivă** (o zonă din RAM) ***în momentul execuţiei***, aceste variabile numindu-se şi **automatice** . Acest tip de variabile pot fi declarate fie la începutul unei funcţii iar la revenirea din funcţie valorile acestora se pierd, fie la începutul unei instrucţiuni compuse iar la terminarea acesteia de asemenea valorile lor se pierd. În ambele cazuri stiva revine la situaţia iniţială, de la începutul apelului sau de la începutul execuţiei instrucţiunii compuse. În diferite funcţii se pot utiliza variabile cu nume identice, fără ca acestea să aibă legături una cu alta.

Un alt lucru deosebit de important este aşa numita "durată de viaţă" a respectivelor variabile, determinată prin **clasa de memorie**.

După cum s-a văzut, în cazul variabilelor **automatice**, acestea aveau valori determinate **numai** în funcţia sau în instrucţiunea compusă în care exista respectiva declaraţie de tip.

În anumite situaţii este de preferat ca să se poată stabili legături între aceleaşi variabile folosite în funcţii diferite, în astfel de situaţii utilizându-se clasele de memorare **static** şi **extern** .

Dacă se utilizează acelaşi mod de declarare, dar în faţa instrucţiunii respective se mai scrie cuvântul **static,** atunci valorile respectivelor variabile nu se mai alocă pe stivă, ci ***în faza de compilare***, într-o zonă de memorie special alocată acestora.

Declaraţia unei **variabile statice** poate fi făcută fie în interiorul unei funcţii, fie în afară, la începutul fişierului care conţine mai multe funcţii, caz în care variabila aceasta este utilizabilă în toate funcţiile pe care le conţine fişierul.

Printr-o astfel de declaraţie se rezolvă accesul şi transmiterea de valori între mai multe funcţii care compun un fişier.

EXEMPLUL nr. 10.14 Să considerăm un exemplu extrem de simplu: în funcţia **main**( ) se va face introducerea a două valori numerice întregi **a** şi **b**, se va calcula suma acestora **c**=a+b, iar afişarea datelor introduse şi a rezultatului **c** se va face cu o altă funcţie având numele **func**.

În continuare se prezintă cele două variante posibile de elaborare ale acestui program: fără folosirea variabilelor statice (var. I) respectiv cu folosirea acestora (var. II) :

```
// ======================
//   Varianta I
//
#include <conio.h>
#include <stdio.h>

void func(int a,int b,int c);


int main(void)
{ int a,b,c;
  clrscr();
  printf("\n\t a = ");
  scanf("%d", &a);
  printf("\t b = ");
  scanf("%d", &b);
  c=a+b;
  func(a,b,c);
  getch(); return 0;
}
```

```
// ==================
// Varianta II
//
#include <conio.h>
#include <stdio.h>

void func(int c);

static int a,b;
int main(void)
{ int c;
  clrscr();
  printf("\n\t a = ");
  scanf("%d", &a);
  printf("\t b = ");
  scanf("%d", &b);
  c=a+b;
  func(c);
  getch(); return 0;
}
```

```
void func(int a,int b,int c)     void func(int c)
{ printf("\n a = %d  b "         { printf("\n a = %d  b"
  "= %d  c = %d", a,b,c); }        "= %d  c = %d", a,b,c);}
```

Să ne îndreptăm atenţia asupra variabilelor **a** şi **b** utilizate în cele două variante prezentate: în prima, acestea se declară atât în funcţia **main( )** cât şi în antetul funcţiei **func(...)** care se apelează, aşa cum eram obişnuiţi din exemplele anterioare, pe când în varianta a doua prin utilizarea declaraţiei **static** (la începutul fişierului sursă care conţine funcţiile) nu mai este nevoie să fie declarate nicăieri în cadrul funcţiilor (nici în corp şi nici în antet) iar în cadrul funcţiilor **main( )** şi **func( )** avem acces la aceste variabile.

În ceea ce priveşte variabila **c** în cazul acesteia s-a utilizat modul "clasic" de transmitere al valorilor între cele două funcţii, în ambele variante.

Sunt situaţii (au apărut în paragrafele anterioare din acest capitol) când se doreşte accesul şi transmiterea de valori între funcţii care aparţin atât la **acelaşi fişier sursă** cât şi la **fişiere sursă diferite**.

În acest caz se face o definire a **variabilelor globale** (**variabile externe**) care este posibil să fie utilizate în tot programul sursă.

În cazul când funcţiile C/C++ aparţin **aceluiaşi fişier sursă** pentru ca anumite variabile să fie definite ca variabile globale se scriu declaraţiile acestora la începutul fişierului şi în consecinţă nu mai este nevoie de declararea lor în funcţiile din respectivul fişier.

EXEMPLUL nr. 10.15. Pentru exemplificare, se poate relua exemplul anterior (varianta a II-a) la care va fi folosită o declarare a unor variabile (în acest caz ale lui **a** şi **b**) înaintea funcţiilor pe care le conţine respectivul fişier sursă:

```
// ================================================
//  Fisier sursa [contine functia  main() si func()]
// ================================================
#include <conio.h>
#include <stdio.h>

void func(int);

int a, b;

int main(void)
{ int c; clrscr();
  printf("\n\t a = "); scanf("%d", &a);
  printf("\t b = "); scanf("%d", &b);
  c=a+b;
  func(c); getch(); return 0;
}
void func(int c)
{ printf("\n a = %d   b = %d   c = %d", a, b, c); }
```

În cazul programului prezentat variabilele **a** şi **b** sunt **variabile globale**. Se poate observa că s-a obţinut în acest caz acelaşi efect ca şi în cazul utilizării declaraţiei **static**, deoarece în acest caz funcţiile aparţin aceluiaşi fişier sursă.

Declararea unor variabile ca şi globale este eficientă atunci când programul sursă este format din **funcţii care aparţin la două sau mai multe fişiere sursă**.

În aceste situaţii în unul din fişierele sursă se declară că anumite variabile sunt globale (externe) iar în celelelte fişiere sursă trebuie făcută o declarare a acestor variabile precedată de cuvântul **extern**.

EXEMPLUL nr. 10.16. Se va relua acelaşi exemplu foarte simplu. În acest caz cele două funcţii aparţin la două **fişiere sursă diferite**, după cum se vede în continuare.

```
// ================================
//   Fisierul sursa - fisier_1.cpp
// ================================

#include <conio.h>
#include <stdio.h>

#include "fisier_2.cpp"

int a, b;

int main(void)
{ int c; clrscr();
  printf("\n\t a = "); scanf("%d", &a);
  printf("\t b = ");   scanf("%d", &b);
  c=a+b;
  func(c); getch(); return 0;
}
```

```
// ================================
//    Fisierul sursa  -  fisier_2.cpp
// ================================

extern int a, b;

void func(int c)
{ printf("\n a = %d  b = %d  c = %d", a, b, c); }
```

În unul din fişiere (fisier_1.cpp în acest caz) s-a făcut declararea globală a celor două variabile **a** şi **b** iar în celălalt fişier (fisier_2.cpp), pentru a se putea folosi aceste variabile s-a reluat declararea acestora, precedată de cuvântul **extern**.

Aceste programe, foarte simple, prezentate în diverse variante, au avut rolul de a exemplifica modul de utilizare al variabilelor **automatice**, **statice** şi **globale**. Datorită simplităţii problemei poate că nu au rezultat cu claritate avantajele deosebite ale folosirii variabilelor statice respectiv globale. Acest lucru va fi mult mai evident în cazul unor programe care folosesc diferite funcţii care operează cu tablouri cu una sau mai multe dimensiuni, transmiterea cărora între funcţii prin intermediul stivei fiind în unele situaţii imposibilă sau consumând un timp exagerat. În astfel de situaţii, prin considerarea unor şiruri sau matrice ca variabile statice (sau globale) transmiterea valorilor se face mult mai simplu şi mai rapid.

Cea de-a patra clasă de memorare se referă la variabilele a căror valori sunt alocate într-un registru al calculatorului, acestea fiind variabile automatice de tip **int**, **char** şi **pointer** în faţa cuvântului cheie care fixează tipul adăugându-se şi cuvântul **register** . Prin această declaraţie se "indică" compilatorului că respectiva variabilă **va fi folosită foarte des** în cadrul funcţiei, deci

valorile lor vor trebui plasate în regiştrii interni ai microprocesorului şi nu în RAM, pe stivă, reducându-se deci timpul de execuţie al programelor.

Numărul de variabile declarate **register** în fiecare funcţie este limitat la trei. O altă regulă se referă la faptul că dacă o variabilă a fost declarată **register** este interzisă referirea la adresa acesteia.

EXEMPLUL nr. 10.17. Ca exemplu se poate considera un program în care se calculează valorile unei funcţii

$$y = 10.25\sqrt{x^6 + 1} + 3.5\,x + 2.35$$

în intervalul [a ; b] cu pasul h.

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

int main(void)
{ int i; double a, b, h;
  register double x;
  clrscr();
  printf("\n\t a = "); scanf("%le", &a);
  printf("\t b = ");   scanf("%le", &b);
  printf("\t h = ");   scanf("%le", &h);
  for(x=a, i=0; x<=b; x+=h)
    { printf("\n %5d   %13.5le   %13.5le",
            ++i, x, 10.25*sqrt(pow(x,6)+1)+3.5*x+2.35);
      if(i%20==0) getch();
    }
  getch(); return 0;
}
```

Se poate observa că în acest program variabila **x** este utilizată în permanenţă, motiv pentru care ea este declarată ca **register**.

În final, este de reţinut faptul că în limbajul **C** există **patru clase de memorie** : ***automatică***, ***statică***, ***externă*** şi ***registru***.

10.5 Recursivitate

Problema recursivităţii este o facilitate oferită de limbajul C/C++ care în anumite situaţii aduce avantaje.

În exemplele anterioare totdeauna o **funcţie apela o altă funcţie.**

În limbajul C/C++ **o funcţie se poate autoapela**, introducerea acestei facilităţi fiind datorată faptului că într-o serie de algoritmi se poate face o descriere prin referirea la acelaşi algoritm, caz în care algoritmul se numeşte **recursiv,** acelaşi termen folosindu-se şi în cazul funcţiilor care se autoapelează. Prin folosirea funcţiilor recursive de obicei nu se obţine o îmbunătăţire în ceea ce priveşte execuţia programului, în unele situaţii aceasta este chiar îngreunată, dar se poate face o scriere mult mai simplă şi elegantă a funcţiilor respective.

La fiecare apel al funcţiei sunt alocate noi zone de memorie (pe stivă) pentru înregistrarea valorilor parametrilor formali ai funcţiei precum şi pentru variabilele de tip **automatic** şi **register**.

Pentru ca o astfel de funcţie, în care se programează o procedură recursivă, să funcţioneze corect este necesar ca apelul funcţiei să apară într-o instrucţiune care se execută ca urmare a unei decizii sau ciclări iar condiţia să poată fi modificată de funcţie pentru ca să se realizeze terminarea apelării repetate.

În cazul acestor proceduri recursive calculul se face ca urmare a unui număr de apelări a funcţiei, când stiva se încarcă, urmat de un număr de reveniri, stiva descărcându-se, în această etapă efectuându-se calculele propriu zise ale valorii funcţiei.

Recursivitatea poate fi **directă** şi **indirectă,** primul tip fiind cel obişnuit, când o funcţie se autoapelează. În cazul recursivităţii indirecte în interiorul funcţiei **f(...)** se apelează funcţia **g(...)** unde se face apelul funcţiei **f(...)** .

EXEMPLUL nr. 10.18. Ca exemplu (cel mai simplu posibil) de utilizare a funcţiilor recursive se va prezenta la început o problemă "clasică", existentă în toate cărţile de programare în C/C++ şi anume calculul factorialului. Programul sursă, format din două funcţii, este următorul :

```
#include <conio.h>
#include <stdio.h>

double fact_rec(int i);

int main(void)
{ int n;
  clrscr(); n=-1;
  while(n<0 || n>170)
      { printf("\n\t n = "); scanf("%d", &n); }
  printf("\n\n\t\t %3d! = %24.16le", n, fact_rec(n));
  getch(); return 0;
}

double fact_rec(int k)
{ double t;
  if(k==0 || k==1) t=1.0;
  else             t=fact_rec(k-1)*k;
  return t;
}
```

Se poate observa că în funcţia **main( )** se face introducerea valorii lui **n** , apelul funcţiei recursive **fact_rec(...)** şi afişarea rezultatului.

În funcţia recursivă se foloseşte relaţia de recurenţă

$$k! = k\ (k-1)!$$

dintre doi factoriali succesivi, care se utilizează până când **k** este mai mare sau egal cu **1**, observându-se că s-a introdus verificarea unei condiţii, cu ajutorul instrucţiunii de decizie **if**.

EXEMPLUL nr. 10.19. Acest exemplu se referă la calculul valorii combinărilor, utilizând relaţia de recurenţă :

$$C_n^k = \frac{n-k+1}{k}\, C_n^{k-1}$$

Şi în acest caz programul sursă este format din două funcţii, cea de-a doua **comb_rec(...)** realizând programarea prin recurenţă a valorii combinărilor, utilizând relaţia anterioară. Se poate observa şi în acest caz că s-a introdus o condiţie conform căreia dacă se ajunge la $\mathbf{C_n^0}=1$ atunci nu se va mai face apelul recursiv.

```
#include <conio.h>
#include <stdio.h>

double comb_rec(int n, int k);

int main(void)
{ int k, n;
  clrscr(); n=0;
  while(n<1 || n>150)
       { printf("\n\n\t n = "); scanf("%d", &n); }
  k=n+1;
  while(k<1 || k>n)
       { printf("\t k = "); scanf("%d", &k); }
  printf("\n\n The recurrence formula :\n\t"
         " C(n,k) = (n-k+1)/k*C(n,k-1)");
  printf("\n\n n = %4d  k = %4d  C(n,k) = %24.16le",
           n, k, comb_rec(n,k));
  getch(); return 0;
}

double comb_rec(int n, int k)
{ double t=1.0;
  if(k==0) t=1.0;
  else     t=(n-k+1)*comb_rec(n,k-1)/k;
  return t;
}
```

În cazul calculării acestor două funcţii, factorialul şi combinările, nu se poate spune că prin folosirea recursivităţii programul devine mai simplu. Se poate face o comparaţie cu modul cum s-ar scrie aceste două funcţii fără a folosi recursivitatea :

```
double fact(int n)
{ int i; double t=1.0;
  for(i=1; i<=n; i++)
      t*=i;
  return t;
}
```

```
double comb(int n,int k)
{ int i; double t=1.0;
  for(i=1; i<=k; i++ )
      t*=(n-i+1.0)/i;
  return t;
}
```

EXEMPLUL nr. 10.20. În cele două exemple anterioare s-au folosit funcţii recursive în care practic s-au programat relaţiile de recurenţă corespunzătoare calculării factorialului şi a combinărilor.

În cadrul exemplului următor se vor folosi funcţii recursive pentru realizarea următoarelor: calculul **sumei**, **produsului** şi **numărului de termeni pozitivi** ai unui şir precum şi a **valorii maxime** a termenilor acestuia.

Programul va fi alcătuit din funcţia **main( )** şi patru funcţii, în cadrul cărora se vor programa procedurile de calcul recursiv :

```
#include <conio.h>
#include <stdio.h>

double sumposA(double A[], int n);
double prodposA(double A[], int n);
int n_posA(double A[], int n);
double maxstrA(double A[], int n);

int main(void)
{ int n=7;
  double A[50]={ -1.0, 2.0, -7.0, -4.0, 5.0, -6.0, -3.0 };
  clrscr();
  printf("\n The sum of positive elements     = %13.6le",
         sumposA(A,n-1));
  printf("\n The product of positive elements = %13.6le",
         prodposA(A,n-1));
  printf("\n The number of positive elements  =%3d",
         n_posA(A,n-1));
  printf("\n The max. value of elements       = %13.6le",
         maxstrA(A,n-1));
  getch(); return 0;
}

double sumposA( double A[], int n )
{ if( n==0 ) return ( A[n] > 0.0 ? A[n] : 0.0 ) ;
  else return ( A[n]>0 ? A[n] : 0 ) + sumposA(A,n-1); }

double prodposA( double A[], int n )
{ if ( n==0 ) return ( A[n]>0 ? A[n] : 1.0 );
  else return ( A[n]>0 ? A[n] : 1 ) * prodposA(A,n-1); }

int n_posA( double A[], int n )
{ if( n==0 ) return (A[n] > 0.0) ;
  else return (A[n]>0) + n_posA(A, n-1); }

double maxstrA( double A[], int n )
{ if( n == 0 ) return A[n];
  else return (A[n]<maxstrA(A,n-1) ? maxstrA(A,n-1): A[n]); }
```

Se poate observa că în cazul celor patru funcţii valoarea care se returnează se iniţializează la pornire (când **n=0**, deci atunci când se consideră primul element al şirului), fie prin evaluarea unei expresii condiţionate (**? :**) în cazul primelor două funcţii (suma numerelor pozitive şi produsul acestora) fie prin considerarea valorii **1** sau **0** care rezultă ca urmare a evaluării expresiei **A[n]>0** (în funcţia **n_posA**) fie prin considerarea ca maxim a primului element , în ultima funcţie.

În cazul unor alte valori ale lui **n** decât **0** va funcţiona procedeul recursiv :

- la suma anterior calculată a termenilor pozitivi se adună fie elementul pozitiv găsit fie o valoare nulă dacă elementul este negativ sau nul,

- produsul anterior calculat al termenilor pozitivi se înmulţeşte fie cu termenul curent dacă acesta este pozitiv fie cu **1** în caz contrar,

- la numărul anterior găsit de termeni pozitivi se adună fie valoarea **1** fie **0**, în funcţie de faptul că expresia **A[n]>0** este adevărată sau falsă,

- valoarea maximă anterior determinată se înlocuieşte cu termenul curent dacă acesta este mai mare decât maximul sau rămâne nemodificată.

În cadrul celor patru funcţii recursive **n** reprezintă valoarea indicelui ultimului element al şirului. Din acest motiv deoarece în **main( )** **n** reprezintă numărul de elemente al şirului atunci când se face apelul celor patru funcţii ca parametri efectivi s-au folosit expresiile **n-1** pe poziţia parametrului formal **n** .

EXEMPLUL nr. 10.21. Cel de-al patrulea exemplu se referă la algoritmul de determinare al celui mai mare divizor comun a două numere întregi (algoritmul lui **Euclid**), prezentat anterior, în Capitolul II, sub formă de schemă logică.

```
#include <conio.h>
#include <stdio.h>

int Euclid_r(int m, int n);

int main()
{ int aux, m, n; clrscr();
  printf("\n\n The Euclid's algorithm\n");
  printf("\n\t m = "); scanf("%d", &m);
  printf("\t n = "); scanf("%d", &n);
  if(m<n) { aux=m; m=n; n=aux; }
  printf("\n  The greatest common divider of %d and %d = %d",
         m, n, Euclid_r(m,n));
  getch(); return 0;
}

int Euclid_r(int m, int n)
{ if(n==0) return m;
  else return Euclid_r(n, m%n);
}
```

În acest caz scrierea funcţiei care determină **c.m.m.d.c.** este mult mai simplă sub forma recursivă decât cea obişnuită.

Se poate observa, din exemplele 10.18, 10.19 , 10.20 şi 10.21, că în cadrul funcţiei recursive s-a făcut de fiecare dată **o singură apelare** a aceleiaşi funcţii. Acest lucru nu este însă obligatoriu, un exemplu în care se vor face două apelări fiind următorul.

EXEMPLUL nr. 10.22. Să se determine primele **n** numere ale lui Fibonacci, utilizând o funcţie recursivă.

Se ştie că

$$f_0 = 0, \quad f_1 = 1, \quad f_k = f_{k-2} + f_{k-1}, \quad k = 2,3,4, ...$$

S-a scris funcţia recursivă **fibo_rec(...)** în care prin intermediul instrucţiunii de selecţie **switch** se atribuie valoarea **0** dacă indicele este zero, valoarea **1** dacă indicele este unu respectiv se utilizează formula de recurenţă în celelalte cazuri.

Programul este următorul :

```
#include <conio.h>
#include <stdio.h>
```

```
long int fibo_rec(int k);

int main(void)
{ int i, n;
  clrscr();
  printf("\n\n\t The Fibonacci's numbers computing"
       "\n\n\t\t n [2 , 3 , ... 29 , 30] = "); scanf("%d", &n);
  printf("\n\t\t F[%2d] = %8ld", 0, 0l);
  printf("\n\t\t F[%2d] = %8ld", 1, 1l);
  for(i=2; i<=n; i++)
     { printf("\n\t\t F[%2d] = %8ld", i, fibo_rec(i));
       if(i%15==0) getch();
     }
  getch(); return 0;
}

long int fibo_rec(int k)
{ switch(k)
      { case 0  : return 0;
        case 1  : return 1;
        default : return fibo_rec(k-2)+fibo_rec(k-1);
      }
}
```

Prezentatea acestui program a fost făcută în special în scop "didactic". Realitatea este că din punct de vedere al rapidităţii în execuţie acest program este extrem de deficitar, motiv pentru care, dacă se poate, acest mod de a programa o astfel de problemă, care are o rezolvare mult mai simplă, trebuie evitat .

În [Negr_1] pg. 373-376, [Pătr98] pg. 57-60 se poate găsi un exemplu interesant, un program în care se rezolvă problema "**turnurilor din Hanoi**" utilizând funcţii recursive, de asemenea în [Pătr98] pg. 64-65 este prezentată, în varianta cu funcţii recursive, problema celor "**opt regine**".

Capitolul XI

Biblioteca de funcţii standard.

11.1 Funcţiile matematice standard. Fişierul math.h.

În programele care au fost prezentate în capitolele anterioare s-au utilizat o serie de funcţii matematice, aparţinând fişierului standardizat **math.h**, ca de exemplu :

$$|x| \rightarrow fabs(x) \qquad \sqrt{x} \rightarrow sqrt(x) \qquad x^y \rightarrow pow(x,y)$$
$$\sin x \rightarrow \sin(x) \qquad \cos x \rightarrow \cos(x) \qquad tg\,x \rightarrow \tan(x)$$
$$arctg\,x \rightarrow a\tan(x)$$

fără a le preciza tipul argumentelor şi tipul valorilor returnate.

Pe lângă aceste funcţii matematice există multe altele, care vor fi prezentate în continuare.

Majoritatea acestor funcţii au argumente de tip **double** iar valorile returnate sunt de asemenea de tip **double**. Prezentăm aceste funcţii ;

Funcţii trigonometrice directe :

double cos(double x) **double sin(double x)** **double tan(double x)**

Funcţii trigonometrice inverse :

double acos(double x) **double asin(double x)** **double atan(double x)**

Funcţiile radical şi ridicare la putere :

double sqrt(double x) **double pow(double x, double y)** **(x^y)**

Funcţiile exponenţiale şi logaritmice :

double exp(double x) **double log(double x)** **double log10(double x)**
($\mathbf{e^x}$) ($\mathbf{\log_e x}$) ($\mathbf{\log_{10} x}$)

Funcţiile hiperbolice :

double cosh(double x) **double sinh(double x)** **double tanh(double x)**

Funcţii (macrouri) pentru determinarea valorii absolute :

double fabs(double x) **int abs(int x)** **long int labs(long int x)**

Funcţii pentru determinarea întregilor imediat inferior şi superior :

double floor(double x) **double ceil(double x)**

În cazul funcţiilor **sin** , **cos** şi **tan** argumentele trebuie date în radiani, de asemenea în radiani rezultă şi valoarea funcţiilor trigonometrice inverse **asin**, **acos** şi **atan**.

Se vor respecta regulile cunoscute din matematică în cazul utilizării funcţiilor **sqrt** (argumentul trebuie să fie nenegativ), **log** şi **log10** (argumentul trebuie să fie strict pozitiv).

Există posibilitatea de a calcula şi valorile absolute ale unor variabile complexe,

$$z = x + j\ y, \quad j = \sqrt{-1}\ , \qquad |z| = \sqrt{x^2 + y^2}\ ,$$

în acest caz utilizându-se funcţia :

double cabs(struct complex z)

a cărei argument este o "***structură***", definită în felul următor :

```
struct complex { double real;
                 double imag;
               }
```

problemele privind structurile urmând să fie prezentate în Capitolul XII .

În fişierul **math.h** mai există funcţia **poly** pentru calcularea valorii unui polinom:

double poly(double x, int n, double c[ ])

unde **x** este argumentul funcţiei polinomiale, **n** este gradul polinomului iar **c** este un şir, conţinând coeficienţii polinomului,

$$c[0] + c[1]*x + c[2]*x^2 + ... + c[n-1]\ x^{n-1} + c[n]\ x^n$$

Cu ajutorul funcţiei **double frexp(double x, int *p)** , care aparţine de asemenea lui **math.h** , se poate realiza scrierea lui **x** sub forma

$$x = m \cdot 2^p \qquad 0.5 < m \le 1$$

unde **m** este o mantisă a cărei valoare este returnată de funcţie iar pentru exponentul **p** se transmite adresa acestuia.

Funcţia **double ldexp(double x, int p)** returnează valoarea lui $\mathbf{x} \circ \mathbf{2^p}$.

În afară de funcţia **atan** a cărei valori (în radiani) sunt în intervalul **[-π/2 , π/2]** şi care are un singur argument, mai există şi funcţia

double atan2(double y, double x)

care returnează valori în intervalul **[-π, π]** acestea fiind valorile (în radiani) ale unghiului pe care-l face axa **Ox** cu dreapta care uneşte originea **O** cu punctul de coordonate **{ x , y },** unghiul fiind pozitiv dacă punctul se află în cadranele **I** sau **II** şi negativ în cazul cadranelor **III** sau **IV**. Funcţia **atan2** nu este definită dacă x = y = 0 .

EXEMPLUL nr. 11.1 Se consideră un punct **M** de coordonate **x** şi **y** cunoscute, în raport cu un sistem cartezian **Oxy,** şi se cere determinarea lungimii segmentului **OM** şi a unghiului **φ** (măsurat în sens trigonometric) dintre axa **Ox** şi segmentul **OM** . Segmentul **OM** şi unghiul **φ** se vor calcula în felul următor :

$$OM = \sqrt{x^2 + y^2}, \qquad \varphi_{rad} = \begin{cases} a\tan 2(y,x), & y \geq 0, \\ 2\pi + a\tan 2(y,x), & y < 0, \end{cases}$$

În acest program se vor folosi funcţiile **sqrt** şi **atan2** precum şi funcţia **floor** (de două ori) pentru a determina numărul (întreg) de grade şi de minute (de asemenea întreg) al unghiului φ.

Programul este următorul :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

int main(void)
{ double a_degree, a_rad, dist, min, pi=3.1415926, sec, x, y;
  clrscr();
  printf("\n\n\t x = "); scanf("%le", &x);
  printf("\t y = "); scanf("%le", &y);
  if(x!=0 || y!=0)
    {dist=sqrt(x*x+y*y);
     a_rad=atan2(y,x);
     if(a_rad<0) a_rad+=2*pi;
     a_degree=a_rad*180/pi;
     min=(a_degree-floor(a_degree))*60;
     sec=(min-floor(min))*60;
     printf("\n\t OM = %le", dist);
     printf("\n\t <(Ox,OM) = %e (rad) = %3d\370 %2d' %6.3f\"",
          a_rad, (int)floor(a_degree), (int)floor(min), sec);
    }
  getch(); return 0;
}
```

S-a putut observa că în cadrul directivelor de preprocesare s-a folosit :

#include <math.h>

deoarece au fost utilizate trei funcţii, **sqrt** , **atan2** şi **floor**, aparţinând acestui fişier.

În ultima instrucţiune **printf(...)** deoarece s-a dorit ca valorile funcţiilor **floor(a_degree)** şi **floor(min)** (care sunt de tip **double** conform definirii funcţiei) să fie afişate ca numere întregi, în faţa acestor expresii s-au scris operatorii de forţare a tipului,

(int) floor(a_degree) **(int)** floor(min)

Datele introduse şi rezultatele obţinute sunt următoarele ;

```
x = 23.67
y = -56.91

OM = 6.163617e+01
<(Ox,OM) = 5.106544e+00 (rad) = 292º 35' 0.224"
```

EXEMPLUL nr. 11.2 Se consideră polinomul :

$$P_n(x) = c_0 + c_1\, x + c_2\, x^2 + ... + c_{n-1}\, x^{n-1} + c_n\, x^n$$

şi se cere să se calculeze valorile acestuia în intervalul **[a ; b]** cu pasul **h**.

După ce se introduc valorile lui **n**, a coeficienţilor polinomului şi a variabilelor **a**, **b** şi **h** , calculul valorii polinomului pentru o anumită valoare a lui **x** se poate face în următoarele trei feluri :

- considerând polinomul ca o sumă de termeni şi programând operaţia de însumare :

```
............
for(i=0, p=0; i<=n; i++) p+=c[i]*pow(x,i);
............
```

aceasta fiind calea cea mai ineficientă, datorită numărului mare de operaţii aritmetice ce trebuie efectuate,

- considerând că polinomul ar fi scris sub aşa numita **formă a lui Horner**,

$$P_n(x) = c_0 + x\,(c_1 + x(\,c_2 + ... + x\,(c_{n-2} + x\,(c_{n-1} + x\,c_n\,)\,)\;....\;)\,)$$

caz în care numărul necesar de operaţii este minim (**n** înmulţiri şi **n** adunări), programarea acestor calcule făcându-se în felul următor :

```
.............
for(p=c[n], i=1; i<=n; i++) p=p*x+c[n-i];
.............
```

iar pentru calculul valorii polinomului se va putea utiliza funcţia :

```
double POL(double x, int n, double c[ ])
{ int i; double p;
  for(p=c[n], i=1;  i<=n;  i++) p=p*x+c[n-i];
  return p; }
```

- a treia posibilitate este de a folosi funcţia **poly(...)** , ce aparţine fişierului **math.h,** a cărei argumente au fost precizate anterior. În programul următor, în corpul ciclulului condus de variabila **x** se face apelul funcţiei **poly** cu argumentele **x**, **n** şi tabloul unidimensional **coef** , obţinându-se valoarea **v** a polinomului, care se returnează de către funcţie. Valoarea lui **v** se va afişa în forma obişnuită şi sub forma :

$$v = m \cdot 2^e \qquad 0.5 < m \le 1$$

mantisa **m** rezultând ca urmare a valorii returnate de funcţia **frexp(...)** iar valoarea exponentului **e** se obţine prin transmiterea adresei.

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

int main(void)
{ int e, i, n;
  double a, b, coef[25], h, m, v, x;
  clrscr();
  printf("\n\n\t The degree of the polynom = "); scanf("%d", &n);
  printf("\n\t The polynomial coefficients :\n\n");
  for(i=0; i<=n; i++)
     { printf(" coef[%2d] = ", i); scanf("%le", &coef[i]); }
  printf("\n\n a = "); scanf("%le", &a);
  printf(" b = "); scanf("%le", &b);
  printf(" h = "); scanf("%le", &h);
```

```
  printf("\n\n\t x       poly(x, n, coef)\n");
  for(i=0, x=a; x<=b; x+=h)
     { v=poly(x, n, coef);  m=frexp(v, &e);
       printf("\n   %10.3f   %15.6le  =  %le * 2 power %d",
              x, v, m, e);
       if(i%15==0) getch();
       i++;
     }
  getch(); return 0;
}
```

11.2 Funcţiile pentru manipularea caracterelor, din fişierul ctype.h .

Limbajul de programare **C** are definite în ***fişierul standardizat*** **ctype.h** o serie de funcţii utilizabile în manipularea caracterelor, care pot realiza următoarele: transformarea unui caracter literă mare în literă mică şi invers, identificarea dacă un caracter este cifră sau nu, dacă este sau nu un semn de punctuaţie şi altele.

În continuare sunt prezentate principalele funcţii şi macrouri din aceste categorii, precizând **argumentele** acestora, ce se **returnează** şi cărui **fişier** aparţin:

int tolower(int c) - returnează codul ASCII corespunzător literei mici dacă **c** este o literă mare sau acelaşi cod ASCII dacă **c** nu este o literă mare (**ctype.h**),

int toupper(int c) - returnează codul ASCII corespunzător literei mari dacă **c** este o literă mică sau acelaşi cod ASCII dacă **c** nu este o literă mică (**ctype.h**),

int toascii(int c) - returnează codul ASCII corespunzător reprezentării binare de pe ultimii 7 biţi, ceilalţi setându-se la valoarea 0 (**ctype.h**),

int isalpha(int c) - returnează o valoare diferită de 0 dacă **c** este o literă (mare sau mică) respectiv 0 dacă este un alt caracter (**ctype.h**),

int isdigit(int c) - returnează o valoare diferită de 0 dacă **c** este o cifră (între 0 şi 9) respectiv 0 dacă este un alt caracter (**ctype.h**),

int isxdigit(int c) - returnează o valoare diferită de 0 dacă **c** este o cifră hexazecimală (cifre între 0 şi 9, litere mari între A şi F, litere mici între a şi f) respectiv 0 dacă este un alt caracter (**ctype.h**),

int islower(int c) - returnează o valoare diferită de zero dacă **c** este o literă mică respectiv 0 dacă este un alt caracter (**ctype.h**),

int isupper(int c) - returnează o valoare diferită de zero dacă **c** este o literă mare respectiv 0 dacă este un alt caracter (**ctype.h**),

EXEMPLUL nr. 11.3 În continuare se prezintă un program în cadrul căruia se utilizează funcţiile definite anterior, precum şi rezultatele obţinute.

```
#include <conio.h>
#include <ctype.h>
#include <stdio.h>
```

```
int main(void)
{ char c;
  clrscr();
  printf("\n\n\t To stop the program press space bar\n");
  for( ; ; )
     {c=getch();
      if(c==' ') break;
      printf("\n\n %c -> tolower  ->  %c", c, tolower(c));
      printf("\n %c -> toupper  ->  %c", c, toupper(c));
      printf("\n %c -> toascii  ->  %X (hex.)",
                                        c, toascii(c));
      printf("\n %c -> isalpha  ->  %d", c, isalpha(c));
      printf("\n %c -> isdigit  ->  %d", c, isdigit(c));
      printf("\n %c -> isxdigit ->  %d", c, isxdigit(c));
      printf("\n %c -> islower  ->  %d", c, islower(c));
      printf("\n %c -> isupper  ->  %d\n", c, isupper(c));
     }
  getch(); return 0;
}
```

În program s-a folosit un ciclu infinit, în cadrul căruia se face introducerea repetată a câte unui caracter, până la apăsarea barei de spaţiere, pentru introducerea unui caracter blanc **' '**. Se prezintă rezultatele obţinute în cazul introducerii unui caracter literă mică (litera **u**), a unei cifre (cifra **3**) şi a caracterului **&** :

```
u  ->  tolower  ->  u
u  ->  toupper  ->  U
u  ->  toascii  -> 75 (hex.)
u  ->  isalpha  ->  8
u  ->  isdigit  ->  0
u  ->  isxdigit ->  0
u  ->  islower  ->  8
u  ->  isupper  ->  0

3  ->  tolower  ->  3
3  ->  toupper  ->  3
3  ->  toascii  -> 33 (hex.)
3  ->  isalpha  ->  0
3  ->  isdigit  ->  2
3  ->  isxdigit ->  2
3  ->  islower  ->  0
3  ->  isupper  ->  0

&  ->  tolower  ->  &
&  ->  toupper  ->  &
&  ->  toascii  -> 26 (hex.)
&  ->  isalpha  ->  0
&  ->  isdigit  ->  0
&  ->  isxdigit ->  0
&  ->  islower  ->  0
&  ->  isupper  ->  0
```

Alte macrouri sunt **isalnum**, **isgraph**, **iscntrl, isspace**, **isprint, ispunct,** toate aparţinând fişierului **ctype.h**.

În multe probleme legate de şiruri de caractere se cere eliminarea din şir a unor caractere, inserarea altor caractere, înlocuiri ale unor caractere cu altele sau a unui caracter cu un grup de caractere, etc. Cele două exemple care urmează prezintă astfel de probleme.

EXEMPLUL nr. 11. 4. Se consideră un şir de caractere, **str**, conţinând litere mari şi mici, cifre şi alte caractere ASCII, şir care se introduce de la tastatură. Se cere eliminarea din acest şir a tuturor caracterelor care nu sunt litere.

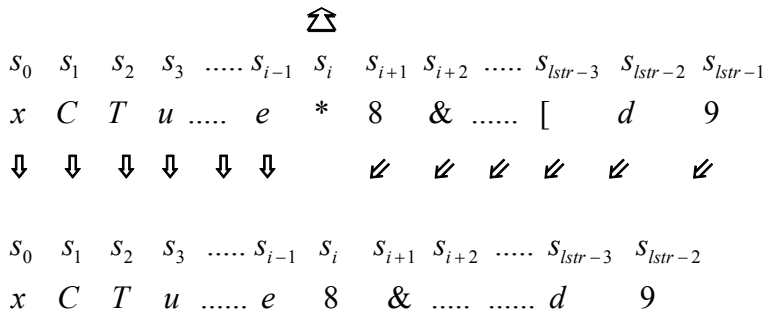
Se face includerea fişierului antet **ctype.h** deoarece se va folosi funcţia **isalpha(...)** în cadrul programului.

Cu instrucţiunile

```
.......
lstr=0;
while(str[str] != '\0' ) lstr++;
.......................
```

se determină numărul de caractere **lstr** din şirul introdus, instrucţiunea care urmează după **while** executându-se până când se ajunge la ultimul element al şirului, adică la caracterul nul ('\0') .

În cazul în care un element oarecare al şirului este altceva decât literă (mare sau mică) atunci acesta se elimină, toate elementele următoare deplasându-se la stânga iar numărul de elemente al şirului , **lstr** , se micşorează cu o unitate , după cum se arată în continuare :



Cu ajutorul funcţiei **isalpha(...)** se determină dacă un anumit caracter este literă (mică sau mare). Programul este următorul :

```
#include <conio.h>      //  clrscr(), getch()
#include <ctype.h>      //  isalpha(..)
#include <stdio.h>      //  gets(..), printf(..)

int main()
{ int i, j, lstr;
  char str[80];
  clrscr();
  printf("\n\n Input the string of characters : \n\n");
  gets(str);
  lstr=0;
  while(str[lstr]!='\0') lstr++;

//  Anterioarele 2 randuri s-ar putea inlocui cu
//             lstr=strlen(str);
//  utilizand functia  strlen(..)  din string.h
```

```
  printf("\n The length of the string = %d", lstr);
  i=0;
  while(str[i]!='\0')
       { if(!isalpha(str[i]))
           { for(j=i; j<lstr; j++) str[j]=str[j+1];
             lstr--; i--;
           }
         i++;
       }
  printf("\n\n The new form of the string \n"
         " (contains only capital and small letters) : ");
  printf("\n\n %s\n", str);
  printf("\n The length of the new string = %d", lstr);
  getch(); return 0;
}
```

Rezolvarea propriu zisă a problemei – eliminarea caracterelor care nu sunt litere şi deplasarea spre stânga a caracterelor din dreapta celui eliminat - se face cu instrucţiunile următoare :

```
i=0;
  while(str[i]!='\0')
       { if(!isalpha(str[i]))
         { for(j=i; j<lstr; j++) str[j]=str[j+1];
           lstr--; i--;
         }
        i++;
       }
```

EXEMPLUL nr. 11.5. Se consideră un şir de caractere, **str** , introdus de la tastatură, care conţine litere, cifre şi alte caractere ASCII. Se cere inserarea în acest şir a câte două blank-uri după fiecare literă mare şi a unui blank după fiecare literă mică.

În acest program se folosesc funcţiile **isupper(..)** şi **islower(..)** pentru a identifica care caractere sunt litere mari respectiv mici.

Programul este următorul :

```
#include <conio.h>
#include <ctype.h>
#include <stdio.h>

int main()
{ int i, j, lstr; char str[80];
  clrscr();
  printf("\n\n Input the string of characters : \n\n");
  gets(str);
  lstr=0; while(str[lstr]!='\0') lstr++;
  printf("\n The length of the string = %d", lstr);
// ************************************************
  i=0;
  while(str[i]!='\0')
       { if(isupper(str[i]))
           { for(j=lstr+1; j>=i+3; j--) str[j]=str[j-2];
             str[i+1]=' '; str[i+2]=' '; lstr+=2; }
        if(islower(str[i]))
```

```
            { for(j=lstr; j>=i+2; j--) str[j]=str[j-1];
              str[i+1]=' '; lstr+=1;              }
          i++;
        }
// ***********************************************
  printf("\n\n The new form of the string\n\376");
  for(i=0; i<lstr; i++) printf("%c", str[i]); printf("\376");
  printf("\n The length of the string = %d", lstr);
  getch(); return 0;
}
```

Se poate observa că rezolvarea propriu zisă a problemei este realizată în urma parcurgerii instrucţiunilor cuprinse între cele două rânduri de comentarii.

Dacă se identifică un caracter – literă mare – atunci toate caracterele din dreapta acestuia se deplasează spre dreapta cu două poziţii în cadrul şirului, cu observaţia că atribuirile se fac în ordine de la dreapta la stânga, pentru a nu se pierde toate valorile caracterelor cu care se lucrează iar apoi se atribuie valoarea caracterului blank la cele două caractere care urmează după litera mare.

În mod similar se procedează şi la întâlnirea unei litere mici, cu deosebirea că în acest caz deplasarea este spre dreapta, cu o singură poziţie.

11.3 Funcţiile pentru prelucrarea şirurilor de caractere din fişierele stdlib.h şi string.h.

În fişierul standardizat **stdlib.h** există o serie de funcţii care realizează transformarea unor **şiruri de caractere ASCII** în **numere** şi **invers**.

Unele din acestea realizează determinarea unui număr ca urmare a considerării unui şir care va fi interpretat ca şi conţinând cifrele respectivului număr. Acestea sunt următoarele:

int atoi (const char *str) - se returnează o valoare întreagă de tipul **int** care rezultă considerând elementele şirului **str** ca şi cifre ale respectivului număr, în scrierea şirului utilizând numai cifrele zecimale,

long atol (const char *str) - se returnează o valoare întreagă de tipul **long int** care rezultă considerând elementele şirului **str** ca şi cifre ale respectivului număr, în scrierea şirului utilizând numai cifrele zecimale,

double atof (const char *str) - se returnează o valoare reală de tipul **double** care rezultă considerând elementele şirului **str** ca şi cifre ale respectivului număr, şirul fiind scris cu utilizarea tuturor caracterelor corespunzătoare numerelor de tip **double**.

Deoarece parametrul **str** care apare în cadrul definirii funcţiilor defineşte zona în care se află şirul iar acesta nu se modifică ca urmare a acţiunii funcţiilor, se va utiliza modificatorul **const** .

A doua categorie de funcţii (**itoa**, **ltoa** şi **ultoa**) efectuează operaţia inversă, adică plecând de la o anumită valoare numerică (de tipul **int** , **long int** sau **unsigned long**) se determină cifrele respectivului număr sub forma unui şir. Aceste funcţii sunt :

char *itoa (int value, char *str, int radix) - returnează adresa unui şir **str** cu maximum **17** caractere, în care sunt înregistrate, conform codificării ASCII, cifrele în sistemul de numeraţie **radix** (baza acestuia poate fi de la **2** la **36**) corespunzătoare numărului **value** de tip **int** ,

char *ltoa (long value, char *str, int radix) - returnează adresa unui şir **str** cu maximum **33** caractere, în care sunt înregistrate, conform codificării ASCII, cifrele în sistemul de numeraţie **radix** (de la **2** la **36**) corespunzătoare numărului **value** de tip **long int** ,

char *ultoa (unsigned long value, char *str, int radix) - returnează adresa unui şir **str** cu maximum **33** caractere, în care sunt înregistrate, conform codificării ASCII, cifrele în sistemul de numeraţie **radix** (de la 2 la 36) corespunzătoare numărului **value** de tip **unsigned long** .

Caracterele utilizate pentru notarea cifrelor în sistemul de numeraţie impus sunt cifrele de la **0** la **9** şi literele de la **a** la **z**, care sunt integral folosite dacă se impune utilizarea sistemului de numeraţie cu baza **36**. Dacă baza sistemului de numeraţie va fi, de exemplu, 25 atunci se vor folosi cifrele de la **0** la **9** şi primele 15 litere, de la **a** la **o**.

EXEMPLUL nr. 11.6 În continuare se prezintă un program în care se folosesc patru din funcţiile prezentate : **atoi**, **atof** (pentru conversia unui şir într-un număr), **itoa** şi **ltoa** (pentru conversia unui număr întreg într-un şir, mai precis determinarea şirului format din cifrele respectivului număr scris într-o anumită bază de numeraţie). Deoarece aceste funcţii aparţin fişierului **stdlib.h** acesta va apărea în cadrul directivelor de preprocesare.

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{ int n, radix; long int nl;
  double v; char string[35];
  clrscr();
  printf("\n\n The function atoi (string --> integer number)");
  puts("\n\n\t Enter a string composed by decimal digits:\n");
  gets(string);
  n=atoi(string);
  printf("\n string = %s   number = %d", string, n);
  printf("\n\n The function atof (string --> floating "
         "point number)");
  puts("\n\n\t Enter a string containing the floating point"
       " number characters:\n");
  gets(string);
  v=atof(string);
  printf("\n string = %s   fl. point number = %f", string, v);
  getch(); clrscr();
  printf("\n\n  The function  itoa  (int -->  string of "
         " digits in radix base)");
  n=-1; radix=1;
  while(n<0 || n>32766)
       { printf("\n\n\t  int value = "); scanf("%d", &n); }
  while(radix<2 || radix>36)
       { printf("\n\t  int radix = "); scanf("%d", &radix); }
  itoa(n, string, radix);
  printf("\n\t\t %d  -->  %s (%2d)", n, string, radix);
  printf("\n\n\n  The function  ltoa  (long int -->  "
```

```
          "string of digits in radix base)");
  nl=-1; radix=1;
  while(nl<0 || nl>2147483647)
       { printf("\n\n\t  long int value = "); scanf("%ld", &nl);}
  while(radix<2 || radix>36)
       { printf("\n\t  int radix = "); scanf("%d", &radix);}
  ltoa(nl, string, radix);
  printf("\n\t\t %ld  -->  %s (%2d)", nl, string, radix);
  getch(); return 0;
}
```

Datele se introduc în regim conversaţional, în plus se impun condiţii (utilizând instrucţiunea de ciclare **while**) privind ordinul de mărime al datelor numerice ce pot fi introduse, cele de tip **int** fiind în intervalul lor de existenţă, de asemenea şi cele de tip **long int** iar în cazul bazei considerate a sistemului de numeraţie aceasta poate fi de la **2** la **36**.

Ca urmare a execuţiei acestui program se obţin următoarele rezultate :

```
The function atoi (string --> integer number)

      Enter a string composed by decimal digits:

1234

 string = 1234    number = 1234

The function atof (string --> floating point number)

      Enter a string containing the floating point number characters:

123.45e5

  string = 123.45e5   fl. point number = 12345000.000000


The function  itoa  (int  -->  string of digits in radix base)

  int value = 12345
  int radix = 23

  12345  -->  107h (23)


The function  ltoa  (long int -->  string of digits in radix base)

  long int value = 1234567890
  int radix = 23

  1234567890  -->  87ifcgi (23)
```

Funcţiile **itoa** şi **ltoa** sunt extrem de utile deoarece cu ajutorul acestora se rezolvă problema conversiei unor numere de tip **int** sau **long int** din baza **10** în orice bază, inclusiv baza **2**, utilizându-le sub forma :

```
.................
int n; long int m;
char str1[50], str2[50];
```

```
n=23456; m=234567890;
itoa(n, str1, 2);  ltoa(m, str2, 2);
printf("\n\n  %d  --- >  %s (2)", n, str1);
printf("\n\n  %ld  --- >  %s (2)", m, str2);
...............
```

Se obţin rezultatele :

```
  23456  --- > 101101110100000 (2)

  234567890 --- > 1101111110110011100011010010 (2)
```

După cum s-a văzut, cele şase funcţii prezentate realizează conversii de la un **şir** la **valori numerice** şi invers.

Alte trei funcţii, care aparţin de asemenea fişierului **stdlib.h**,

double strtod (const char *nptr, char **endptr)

long strtol (const char *nptr, char *endptr, int radix)

unsigned long strtoul (const char *nptr, char *endptr, int radix)

realizează conversia unui şir de caractere într-o valoare în dublă precizie (**strtod** – **str**ing **to d**ouble), întreg lung (**strtol** – **str**ing **to l**ong int) şi întreg lung fără semn (**strtoul** – **str**ing **to u**nsigned **l**ong int). O prezentare detaliată a acestora se poate găsi în [Petr93] pg. 174-176.

Există o serie de funcţii, aparţinând fişierului **string.h** care operează cu şiruri, făcând operaţii de **determinare a lungimii** şirului, **comparare**, **copiere**, **concatenare** a acestora.

Aceste funcţii sunt :

unsigned strlen (const char *string) - returnează numărul de caractere (fără caracterul terminal nul **'\0'**) al şirului (constant) **string** , utilizat ca parametru, (**str**ing **len**gth)

int strcmp (const char *string1, const char *string2) - returnează o valoare întreagă care poate fi zero dacă şirurile sunt identice (aceeaşi lungime şi aceleaşi caractere componente), o valoare negativă dacă primul şir este "mai mic" decât al doilea, adică primele caractere ale celor două şiruri sunt identice (sau eventual nici unul) iar primul caracter de după acestea la primul şir este mai mic decât cel corespunzător al celui de-al doilea şir, conform codificării ASCII sau o valoare pozitivă dacă primul şir este "mai mare" decât al doilea , (**str**ing **c**o**mp**arison) ,

int strncmp (const char * string1, const char *string2, unsigned n) - returnează o valoare întreagă (zero, negativă sau pozitivă) funcţie de rezultatul comparaţiei celor două şiruri, dar cu referire numai la primele **n** elemente ale acestora,

int strincmp (const char * string1, const char *string2, unsigned n) - returnează o valoare întreagă (zero, negativă sau pozitivă) funcţie de rezultatul comparaţiei celor două şiruri, dar cu referire numai la primele **n** elemente ale acestora şi fără să se facă deosebire între literele mari şi mici,

char *strcpy (char *string1, const char *string2) – şirul **string2** se copiază în memorie în zona unde era plasat şirul **string1** şi se returnează adresa primului byte de unde începe înregistrarea şirului **string1**, deci valoarea pointerului **string1**, (**str**ing **c**o**py**),

char *strncpy (char *string1, const char *string2, unsigned n) - această funcţie are o funcţionare asemănătoare cu cea a precedentei, cu deosebirea că se face copierea numai a primelor **n** caractere ale şirului **string2**. Dacă **n** este mai mare decât numărul de caractere al şirului **string2** atunci se va copia întregul şir **string2**,

char *strcat (char *string1, const char *string2) - şirul **string2** (fără caracterul terminal nul) se concatenează (se adaugă) la şirul **string1** iar funcţia returnează adresa primului element al şirului final obţinut, **string1** (**str**ing con**cat**enation)

char *strncat (char *string1, const char *string2, unsigned n) – primele **n** caractere ale şirului **string2** (sau eventual toate dacă **n** este mai mare decât lungimea şirului **string2**) se concatenează (se adaugă) la şirul **string1** iar funcţia returnează adresa primului element al şirului final obţinut, **string1.**

EXEMPLUL nr. 11.7 Se consideră următoarele şiruri :

s1 - "Programming in C/C++"
s2 - "Programming in C/C++"
s3 - "Programming in C"
s4 - "Programming in FORTRAN"
s5 - " and "
s6 - "Programming in Ada"

şi se cere să se elaboreze un program în cadrul căruia să fie utilizate funcţiile **strlen**, **strcmp**, **strcat** şi **strcpy** folosind şirurile de mai sus. Programul esrte următorul :

```
#include <conio.h>
#include <stdio.h>
#include <string.h>

int main(void)
{ int n1, n3, n12, n13, n14;
  char s1[]  ="Programming in C/C++",
       *s2   ="Programming in C/C++",
       s3[25]="Programming in C",
       s4[]  ="Programming in FORTRAN",
       s5[]  =" and ",
       s6[]  ="Ada";
  clrscr(); n1=strlen(s1); n3=strlen(s3);
  printf("\n\n\t The function  unsigned strlen(const "
         "char *str)\n");
  printf("\n Length of the string \"%s\" = %d", s1, n1);
  printf("\n Length of the string \"%s\" = %d", s3, n3);
  getch();

  printf("\n\n\t The comparison function"
         "\n\t\t int strcmp(const char *str1, const char *str2)");
  n12=strcmp(s1, s2); n13=strcmp(s1, s3); n14=strcmp(s1, s4);
  printf("\n\n \"%s\" ~ \"%s\"    %3d", s1, s2, n12);
  printf("\n \"%s\" ~ \"%s\"    %3d", s1, s3, n13);
  printf("\n \"%s\" ~ \"%s\"    %3d", s1, s4, n14);
  getch();
```

```
  printf("\n\n\t The concatenation function"
  "\n\t\t char *strcat(char *dest, const char * source)");
  printf("\n\n The destination string : \"%s\"", s4);
  printf("\n The source string      : \"%s\"", s5);
  strcat(s4, s5);
  printf("\n The resulting string   : \"%s\"", s4);
  printf("\n\n The destination string : \"%s\"", s4);
  printf("\n The source string      : \"%s\"", s6);
  strcat(s4, s6);
  printf("\n The resulting string   : \"%s\"", s4);
  getch();

  printf("\n\n\t The function\n\t\t char *strcpy(char *dest, "
      "const char * source)");
  printf("\n\n The destination string : \"%s\"", s1);
  printf("\n The source string      : \"%s\"", s6);
  strcpy(s1, s6);
  printf("\n The resulting string   : \"%s\"", s1);
  getch(); return 0;
}
```

La începutul programului se poate observa că declararea şi iniţializarea şirurilor de caractere s-a făcut în trei feluri.

La toate afişările şirurilor s-au pus înaintea acestora şi la sfârşitul lor caracterul **"** realizat prin folosirea secvenţei escape **'\"'.**

Programul a fost astfel realizat încât să se poate observa cu uşurinţă modul cum se folosesc aceste funcţii, rezultatele obţinute ca urmare a execuţiei fiind următoarele :

```
        The function  unsigned strlen(const char *str)

  Length of the string "Programming in C/C++" = 20
  Length of the string "Programming in C" = 16


        The comparison function
              int strcmp(const char *str1, const char *str2)

  "Programming in C/C++" ~ "Programming in C/C++"    0
  "Programming in C/C++" ~ "Programming in C"   47
  "Programming in C/C++" ~ "Programming in FORTRAN"   -3


        The concatenation function
              char *strcat(char *dest, const char * source)

  The destination string : "Programming in FORTRAN"
  The source string      : " and "
  The resulting string   : "Programming in FORTRAN and "

  The destination string : "Programming in FORTRAN and "
  The source string      : "Ada"
  The resulting string   : "Programming in FORTRAN and Ada"


        The function
              char *strcpy(char *dest, const char * source)
```

```
The destination string : "Programming in C/C++"
The source string      : "Ada"
The resulting string   : "Ada"
```

EXEMPLUL nr. 11.8 Considerând un interval de numere de tip **long int** de la $\mathbf{n_1}$ la $\mathbf{n_2}$ se cere să se determine acele numere care au următoarea proprietate :

$$\overline{a_{n-1}\, a_{n-2}\, a_{n-3} \ldots .\, a_1\, a_0} = a_{n-1} + a_{n-2}^2 + a_{n-3}^3 \ldots . + a_1^{n-1} + a_0^n$$

O astfel de problemă s-ar putea rezolva într-un mod asemător cu cel din Exemplul nr. 10.8 din Capitolul X, utilizând funcţia **i_to_str (...)** . Se va folosi, de această dată, funcţia de bibliotecă **ltoa** aparţinând fişierului **stdlib.h** ,

char *ltoa (long value , char *string , int radix) ;

apelând-o sub forma :

ltoa (n , string , 10) ;

unde **n** este numărul , **10** este baza sistemului de numeraţie utilizat iar **string** este pointerul unui şir de caractere care conţine cifrele numărului **n** , ***codificate ca şi caractere***, deci având o reprezentare în codul ASCII de la **48** (caracterul **0**) până la **57** (caracterul **9**). Prin aceasta se va deosebi deci funcţia de bibliotecă **ltoa** de funcţia **i_to_str** scrisă şi folosită în exemplele 10.7 şi 10.8 din Capitolul X, ca urmare a apelării funcţiei **i_to_str** rezultând un şir numeric al cărui elemente sunt cifrele de la **0** la **9** .

Programul elaborat este următorul :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{ int i, n_cifre, s[10];
  long int n1, n2, n, sum;
  char string[10];
  clrscr();
  printf("\n\t n_init  = "); scanf("%ld", &n1);
  printf("\n\t n_fin   = "); scanf("%ld", &n2);
  printf("\n\t n_cifre = "); scanf("%d", &n_cifre);
  for(n=n1; n<=n2; n++)
     { ltoa ( n, string, 10 );
       for(i=0; i<n_cifre; i++)  s[i]=string[i]-48;
       for(sum=0, i=0; i<n_cifre; i++) sum+=pow(s[i],i+1);
       if(n==sum)
        { printf("\n n = %ld ", n); getch(); }
     }
  getch(); return 0;
}
```

Se poate observa că în urma execuţiei repetate a instrucţiunii

```
s[i] = string[i] – 48 ;
```

se determină elementele unui şir numeric care va conţine cifrele numărului **n** . Astfel, dacă de exemplu `string[i]` este caracterul 5 (care are valoarea 53 conform codificării ASCII) atunci prin scăderea efectuată (53 – 48) va rezulta chiar valoarea numerică 5 , care se atribuie unui element al unui şir de tip **int** .

Ca urmare a utilizării acestui program au rezultat următoarele numere care au proprietatea respectivă :

- numere cu o cifră : 1, 2, 3, 4, 5, 6, 7, 8, 9,
- numere cu două cifre : 89 ($89 = 8 + 9^2$) ,
- numere cu trei cifre : 135, 175, 518, 598 ($135 = 1 + 3^2 + 5^3$, etc)
- numere cu patru cifre : 1306, 1676, 2427 ($1306 = 1 + 3^2 + 0 + 6^4$, etc) .

11.4 Funcţiile pentru determinarea timpului din fişierul time.h .

În cadrul programelor scrise în C/C++ se poate utiliza **timpul curent** măsurat în secunde începând de la **00:00:00 GMT** (**G**reenwich **M**ean **T**ime) **1 ianuarie 1970**, (sau **timpul curent local**) de asemenea se poate face afişarea datei, orei, minutului şi secundei curente (sub forma unui şir de caractere) şi diverse alte operaţii. Funcţiile necesare pentru a obţine, memora şi converti timpul curent se găsesc în fişierul **time.h** .

Acest fişier conţine 13 funcţii, dintre care ***nouă*** corespund standardului **ANSI C**, după cum urmează (cele standardizate sunt tipărite cu bold) :

asctime (ANSI C)	**clock** (ANSI C)	**ctime** (ANSI C)
difftime (ANSI C)	**gmtime**(ANSI C)	**localtime** (ANSI C)
mktime (ANSI C)	stime	**strftime** (ANSI C)
_strdate	_strtime	**time** (ANSI C)
tzset		

De asemenea, acest fişier mai conţine şi o serie de constante (**CLK_TCK** sau CLOCKS_PER_SEC în varianta standardizată, **timezone** – diferenţa în secunde dintre timpul **GMT** şi timpul local), definiţii de tipuri de date (**clock_t**, **time_t**, ş. a.) .

În continuare se prezintă definirea şi scurte explicaţii privind unele din funcţiile din **time.h** , care vor fi folosite în exemplele care urmează :

```
clock_t clock(void);
     //  Se returneaza numarul de unităţi de timp corespunzătoare
     //  CPU, care au fost utilizate de la inceputul executiei
     //  programului. Pentru a fi transformate in secunde trebuie
     //  impartite cu constanta  CLK_TCK.
     //  Se returneaza  -1  daca acest timp al CPU  nu poate fi
     //  determinat.
     //  clock_t – este un declarator de tip de date corespunzător
     //  timpului unitatii centrale CPU

char *ctime(const time_t *time);
     //  Se returneaza un pointer la un sir continand 26 de caractere
     //  care precizeaza data si ora curenta, ca urmare a conversiei
     //  timpului, memorat ca o valoare de tip  time_t.
     //  Sirul va avea urmatoarea forma:
     //             DDD MMM dd HH MM SS YYYY\n\0
     //  unde
     //     DDD  este ziua din saptamana (Mon, Tue, Wed, Thu, Fri,
```

```
     //          Sat,Sun),
     //     MMM  este luna (Jan, Feb, Mar, .... ),
     //     dd   este data (1, 2, 3, .... 31),
     //     HH   este ora (1, 2, 3, ..., 24),
     //     MM   este minutul (1, 2, 3,..., 60),
     //     SS   este secunda (1, 2, 3,..., 60),
     //     YYYY este anul.
     //  Sirul se termina cu caracterele  \n  si  \0
     //  Functia returneaza pointerul  NULL  daca data este anterioara
     //  anului 1980.
     //  time_t  este un tip de variabila corespunzator functiilor de
     //  timp .

double difftime(time_t t2, time_t t1);
     //  Se returneaza valoarea diferentei  t2-t1, t1  si  t2  fiind
     //  variabile de tipul  time_t.

time_t time(time_t *timer);
     //  Returneaza timpul curent masurat de la 00:00:00 GMT din
     //  1 Ianuarie 1970, acest timp (in secunde) pastrandu-se la
     //  adresa  *timer.
     //  In cazul in care se foloseste pointerul  NULL  ca argument,
     //  timpul nu se inregistreaza la adresa respectiva.
```

Aceste funcţii pot fi utilizate pentru ca în cadrul datelor care se afişează să apară şi data la care se execută respectivul program sau în unele programe pentru a determina durata de execuţie a unor calcule. Exemplele care urmează vor prezenta aceste două situaţii.

EXEMPLUL nr. 11.9 . În programul următor se va afişa timpul curent (în secunde), sub forma unui număr de tip **long int**, apoi sub forma unui şir, folosind funcţiile **time** şi **ctime,** iar în continuare, utilizând funcţia **difftime,** se va determina timpul necesar pentru calculul repetat al valorii funcţiei **exp(1.)** de un anumit număr de ori.

Programul se prezintă în continuare :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>
#include <time.h>

int main(void)
{ long int i; time_t DT, T, t, t1, t2;

     //  Se determina si se afiseaza timpul curent (in secunde)
  t=time(NULL);
  printf("\n\n The number of seconds since January, 1 1970 = %ld", t);

     //  Se determina timpul curent si se inregistreaza la adresa
     //  &T iar apoi data, ora, minutul si secunda se afiseaza sub
     //  forma unui sir de caractere.
  time(&T); printf("\n Data & Hour ~ %s", ctime(&T) );

     //  Se determina de doua ori timpul curent, inainte de
     //  efectuarea calculelor si dupa, se determina diferenta si se
     //  afiseaza.
  t1=time(NULL);
```

```
  for(i=0; i<2500000; i++) exp(1.);
  t2=time(NULL);
  DT=difftime(t2,t1); printf(" Diff. time = %ld", DT);
  getch(); return 0;
}
```

Rezultatele obţinute au fost următoarele (valabile la data şi ora când programul a fost executat) :

```
The number of seconds since January, 1 1970 = 995239181
Data & Hour ~ Sun Jul 15 19:19:41 2001
Diff. time  = 8
```

EXEMPLUL nr. 11.10 . În cazul când o anumită problemă se poate rezolva cu mai multe metode, orice programator va alege acea metodă în care se utilizează un număr minim de operaţii, respectiv timpul necesar de calcul este cât mai mic posibil. În programul următor se calculează în trei feluri valoarea funcţiei **sin x** , pentru un **x** dat, în primul caz utilizând funcţia **sin(x)** din fişierul **math.h** iar în cazurile doi şi trei se foloseşte dezvoltarea în serie,

$$\sin x = \frac{x}{1!} - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + ...+ (-1)^{i+1}\frac{x^{2i-1}}{(2\,i-1)\,!} + (-1)^{i+2}\,\frac{x^{2i+1}}{(2\,i+1)\,!} + ...$$

considerând un număr **n** de termeni.

După cum se ştie, calculul sumei primilor **n** termeni se poate face în două feluri :

- la fiecare pas al ciclului se face însumarea câte unui termen urmată de calcularea valorii termenului următor, folosind o formulă de recurenţă stabilită între doi termeni succesivi, valoarea primului termen fiind **x**,

- la fiecare pas al ciclului se face calculul valorii termenului din serie (**x** se ridică la putere şi se împarte la valoarea factorialului, care se calculează pentru fiecare termen), termenul adăugându-se la valoarea curentă a sumei.

Este evident faptul că în cel de-al treilea caz numărul de operaţii (şi deci şi timpul necesar de calcul) va fi mai mare.

Cu ajutorul programului următor ne putem convinge de acest lucru, apelând în mod repetat funcţiile **clock()**, înainte şi după terminarea calculelor corespunzătoare fiecărei metode. Pentru a obţine durata în secunde diferenţa obţinută se va împărţi la constanta **CLK_TCK** .

```
#include <conio.h>
#include <math.h>
#include <stdio.h>
#include <time.h>

double fact(int);

int main(void)
{ int i, n; long int j, j_max=100000; clock_t t;
  double arg_d, arg_r, term, s1, s2, s3, T1, T2, T3;
  printf("\n\n Argument [degree] = "); scanf("%le", &arg_d);
  printf("\n Number of terms   = "); scanf("%d", &n);
  arg_r=arg_d*M_PI/180;
            //     Metoda  I
  t=clock();
  for(j=1; j<=j_max; j++) s1=sin(arg_r);
  T1=(clock()-t)/CLK_TCK;
            //     Metoda II
```

```
  t=clock();
  for(j=1; j<=j_max; j++)
     { for(s2=0.0,term=arg_r,i=1; i<=n; i++)
         { s2+=term; term=-term*arg_r*arg_r/2/i/(2*i+1);}
     }
  T2=(clock()-t)/CLK_TCK;
          //     Metoda III
  t=clock();
  for(j=1; j<=j_max; j++)
     { for(s3=0.0,i=1; i<=n; i++)
       s3+=pow(-1,i+1)*pow(arg_r,2*i-1)/fact(2*i-1);
     }
  T3=(clock()-t)/CLK_TCK;
  //
  printf("\n\n Argument               = %8.2lf [degree]"
         "\n Number of terms        = %8d"
         "\n Number of calculations = %8ld", arg_d, n, j_max);
  printf("\n The computed values :          %11.8f  %11.8f  %11.8f",
          s1, s2, s3);
  printf("\n The necessary computing time : %11.8f  %11.8f  %11.8f",
          T1, T2, T3);
  getch(); return 0;
}
double fact(int n)
{ int i; double v=1;
  for(i=1; i<=n; i++) v*=i;
  return v; }
```

În cazul când argumentul este 30± şi se consideră 10 termeni din dezvoltarea în serie s-au obţinut rezultatele :

```
 Argument               =    30.00 [degree]
 Number of terms        =       10
 Number of calculations =   100000
 The computed values :          0.50000000  0.50000000  0.50000000
 The necessary computing time : 0.10989016  0.93406593  3.40659345
```

După cum s-a văzut, pentru a pune în evidenţă mai bine timpii de calcul necesari pentru efectuarea calculelor respective, în cadrul fiecărei metode, acestea au fost repetate de j_max = **100000** ori. Rezultate identice, obţinute cu cele trei metode, se obţin dacă numărul de termeni este mai mare decât cinci. În ceea ce priveşte timpul necesar de calcul, dacă se execută acest program pe alte calculatoare, mai performante sau mai puţin performante, se va păstra raportul dintre timpii obţinuţi, adică 0.109 : 0.934 : 3.406 .

11.5 Funcţiile pentru generarea numerelor aleatoare din fişierul stdlib.h .

Utilizând limbajul de programare C/C++ se pot apela ***patru funcţii*** pentru generarea numerelor aleatoare, toate aparţinând fişierului **stdlib.h**, ale căror prototipuri se prezintă în continuare:

```
int rand(void);
      // Se returneaza un numar aleator din intervalul  0 ~ RAND_MAX,
      // pornind de la un multiplu al numarului 346  cu o perioada de
      // 232
      // Este o functie conforma cu standardul ANSI C.

int random(int num);
      // Se returneaza un numar aleator din intervalul  0 ~ (num-1).
      // Nu este o functie standardizata conform ANSI C.

void randomize(void);
      // Se initializeaza generatorul de numere aleatoare cu un numar
      // aleator.
      // Necesita includerea fisierului  time.h  deoarece macroul
      // randomize  utilizeaza functia  time.
      // Nu este o functie standardizata conform ANSI C.

void srand(unsigned seed);
      // Generatorul de numere aleatoare este initializat cu un numar
      // "seed", care de obicei este o functie de timpul curent,
      // determinat cu functia  time.
      // Este o functie standardizata conform ANSI C.
```

Pentru a asigura portabilitatea programelor care folosesc funcţii pentru generarea numerelor aleatoare se vor utiliza funcţiile **srand** şi **rand**, prima pentru iniţializarea generatorului de numere aleatoare iar a doua pentru generarea efectivă a acestora.

EXEMPLUL nr. 11.11 . În cazul folosirii funcţiei **rand()**, fără a avea în prealabil realizată iniţializarea generatorului (fie cu **randomize** fie cu **srand**) se vor obţine rezultate identice, indiferent de momentul execuţiei programului.

Dacă se doreşte obţinerea unor numere aleatoare din intervalul **0 ~ 345** atunci se va calcula şi afişa în mod repetat valoarea expresiei **rand()%346** .

Programul este următorul :

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
int main(void)
{ int i;
  printf("\n Fifteen random numbers from  0  to  345\n");
  for(i=0; i<15; i++) printf("%4d", rand()%346);
  getch(); return 0;
}
```

Ca urmare a execuţiei acestui program se vor obţine rezultatele :

```
   0 130 256  52 238 197 295 187 112 332   8  26 111  43 154
```

Se poate observa că deoarece se face calculul restului împărţirii numărului aleator generat de funcţia **rand()** la **346** (acest număr aleator fiind iniţial un multiplu de 346) totdeauna primul rezultat este **0** iar apoi numerele aleatoare care se generează au perioada 232, rezultând celelalte valori.

EXEMPLUL nr. 11.12 . Un câştig la Loto (***6*** din ***49***) , oricât de mic, este util fiecăruia. Folosirea rezultatelor obţinute cu programul care urmează ar fi un mijloc posibil pentru a realiza acest lucru.

Pentru a obţine seturi de numere diferite, la fiecare execuţie, se va face iniţializarea generatorului de numere aleatoare în funcţie de timpul curent, utilizând ca argument al funcţiei **srand** timpul curent. Cele şase numere căutate (şi poate câştigătoare la o extragere) rezultă ca urmare a calculării de şase ori a expresiei **rand()%49+1** obţinându-se valori de la ***1*** la ***49***.

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{ int i; time_t t;
  srand((unsigned)time(&t));
  printf("\n\n Six random numbers from  1  to  49\n");
  for(i=0; i<6; i++) printf("%4d", rand()%49+1);
  getch(); return 0;
}
```

Se poate observa că s-a făcut includerea şi a fişierului **time.h** deoarece s-a folosit funcţia **time**.

Ca urmare a parcurgerii ciclului **for** de şase ori se vor afişa cele şase numere generate. S-ar putea întâmpla ca cele şase numere să nu fie distincte. Din acest motiv acest program ar trebui modificat pentru ca să se formeze un **şir** cu şase termeni distincţi.

Cu acest program autorii încă nu au obţinut câştiguri la LOTO, nu din cauză că programul ar fi greşit ci din cauză că nu l-au folosit la "momentul **t**" potrivit, pentru ca generatorul de numere să furnizeze secvenţa de numere câştigătoare

O utilizare posibilă a numerelor aleatoare este cea pentru determinarea ariilor unor figuri geometrice care se găsesc în interiorul unor arii de valori cunoscute.

EXEMPLUL nr. 11.13 . În primul cadran al unui sistem cartezian Oxy se consideră un sector circular de rază unitară, cu centrul în originea O, care este cuprins într-un patrat având latura de lungime 1, suprafaţa căruia este **1**. Dacă se parcurge segmentul **[0 ; 1]** de pe axa absciselor Ox, cu un pas constant, considerând **n** puncte iar pentru fiecare punct de abscisă **x** se calculează valoarea expresiei **rand()%1001/1000.0** va rezulta un număr aleator **y** din intervalul **[0 ; 1] .** Dacă din cele **n** puncte considerate **k** aparţin sectorului circular iar **n-k** nu aparţin, atunci suprafaţa sectorului s-ar putea calcula cu formula **k/n** . Suprafaţa întregului cerc (a cărei valoare este în acest caz **π**) este de patru ori mai mare, deci valoarea numărului **π** ar putea fi calculată cu formula **4*k/n**.

Programul este următorul :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{ long int i, k, n=50000; double PI, x, y; time_t t;
```

```
  srand((unsigned)time(&t));
  for(k=0,i=0; i<n; i++)
     { x=(double)i/n; y=rand()%1001/1000.0;
       if(sqrt(x*x+y*y)<=1.0) k++; }
  PI=4.0*k/n; printf("\n\n \343 = %6.4f", PI);
  getch(); return 0;
}
```

Ca urmare a mai multor execuţii a acestui program s-au obţinut următoarele valori numerice pentru numărul π

π = **3.1**529; **3.14**78; **3.14**63; **3.1**528; **3.14**99; **3.1**377; ...

destul de apropiate de valoarea exactă (**3.1415926...**).

EXEMPLUL nr. 11.14 . Calculul valorilor integralelor definite

$$v = \int_a^b f(x)\, dx,$$

se poate face de asemenea cu un program asemănător cu cel anterior prezentat, cu condiţia ca graficul funcţiei **f(x)** de sub semnul integralei să fie cuprins într-un dreptunghi de înălţime cunoscută, baza căruia are lungimea **b-a** .

Ca exemplu se va considera o integrală definită a cărei valoare este cunoscută,

$$v = \int_0^{\pi} \sin x \ dx = 2.$$

Se va parcurge intervalul [**0** ; π] cu un pas constant, considerând **n** puncte, se va calcula pentru fiecare punct o valoare a ordonatei (o valoare aleatoare între **0** şi **1**), care dacă este mai mică decât **sin(x)** valoarea unui "contor" **k** se va mări cu câte o unitate.

Valoarea ariei cuprinse între axa Ox şi graficul sinusoidei din intervalul [**0** ; π] , care este valoarea numerică a integralei, se va calcula cu formula :

$$v \approx \frac{k}{n} \pi \times 1.$$

Programul este următorul :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{ int i, k, n; double aria, x, y; time_t t;
  srand((unsigned)time(&t));
  for(n=100; n<=1000; n+=100)
     { for(k=0,i=1; i<=n; i++)
          { x=M_PI*i/n; y=rand()%1001/1000.0;
            if(y<=sin(x)) k++;}
       aria=M_PI*k/n;
       printf("\n  n = %4d  Aria = %6.4f", n, aria);
     }
```

```
  getch(); return 0;
}
```

Rezultatele numerice obţinute au fost următoarele :

```
 n =  100  Aria = 2.0420
 n =  200  Aria = 1.9949
 n =  300  Aria = 1.9687
 n =  400  Aria = 2.0420
 n =  500  Aria = 2.0358
 n =  600  Aria = 2.0368
 n =  700  Aria = 2.0061
 n =  800  Aria = 2.0263
 n =  900  Aria = 2.0316
 n = 1000  Aria = 2.0043
```

Se poate observa că s-au obţinut rezultate suficient de exacte începând chiar de la n = 100 .

Observaţia nr. 1. Această metodă de determinare fie a valorilor ariilor unor figuri geometrice, fie a valorilor integralelor definite, utilizând numere aleatoare, se numeşte **metoda Monte Carlo**.

Observaţia nr. 2 . Rezultatele numerice obţinute la ultimele trei exemple, 11.12, 11.13 şi 11.14 nu sunt reproductibile. Dacă respectivele programe se vor utiliza se vor obţine alte rezultate, evident foarte apropiate de cele din lucrare.

Capitolul XII

Structuri, uniuni şi câmpuri

Tablourile cu o dimensiune sau cu mai multe, utilizate în capitolele anterioare, conţineau totdeauna **acelaşi tip de valori**. Astfel în cazul unor declaraţii de forma :

```
{ ..............
  int a[100], b[25][10];
  double  c[15][15];
  char  d[80];
  ..............  }
```

în funcţia respectivă se utilizau două şiruri , şirul **a** conţinând valori numerice întregi iar şirul **d** conţinând caractere ASCII şi două matrice **b** şi **c**, prima conţinând valori întregi de tip **int** iar a doua conţinând valori reale de tip **double** .

Sunt situaţii, care însă nu au intervenit până acum, când ar fi extrem de util să se poată lucra cu ***mai multe tipuri de valori***, considerate ca o **entitate**, căreia să i se ataşeze un nume şi să fie reprezentată în memorie într-o zonă compactă.

De exemplu, **datele personale** [***numele*** şi ***prenumele*** – două şiruri de caractere, ***data naşterii*** – an (număr), lună (şir de caractere sau număr), zi (număr) , ***locul naşterii*** – şir de caractere, ***domiciliu*** – şiruri de caractere şi numere, etc] pot fi reprezentate în programe C/C++ printr-o **structură** aceasta fiind denumirea unei astfel de entităţi.

Din punct de vedere al calculelor, atunci când se utilizează numere complexe, de asemenea se va lucra cu o entitate - **numărul complex**, compus din două elemente, ***partea reală*** şi ***partea imaginară***.

12.1 Structuri

Forma generală a unei structuri (declaraţia) este următoarea :

```
struct  etic_str {
                   tip nume_elem_1;
                   tip nume_elem_2;
                   ..............
                   tip nume_elem_n;
                  } nume_str_1, nume_str_2, ..., nume_str_n ;
```

unde **struct** este un cuvânt instrucţiune, **etic_str** este o etichetă (un marcaj) al structurii, care în unele cazuri poate lipsi, în interiorul celor două acolade se găseşte lista membrilor (elementelor,

câmpurilor) structurii, ***nume_elem_1, nume_elem_2, ..., nume_elem_n***, la fiecare precizându-se tipul şi dimensiunile, dacă este un tablou. Numărul acestor membri ai structurii este limitat la **127**. Declaraţia de structură se continuă (opţional) cu o listă de variabile **nume_str_1, nume_str_2, ..., nume_str_n,** denumirile unor structuri de acelaşi tip cu cea definită, care vor fi utilizate în programul sursă. Declaraţia de structură se termină cu **;**

S-a precizat că lista de variabile **nume_str_1, nume_str_2, ..., nume_str_n,** poate să lipsească. În acest caz este obligatoriu să existe **etic_str** . În cazul când această listă lipseşte vor trebui să existe declaraţii de variabilă de tip **struct** în funcţia unde se foloseşte această structură.

În cele ce urmează se dau două exemple privind aceeaşi structură, corespunzătoare unui număr complex, utilizând două modalităţi de a efectua definirea variabilelor tip structură :

```
//         Varianta I
struct complex {
                double x ;
                double y ;
               } ;

int main()
{ ..............
  struct complex a, b, c, d ;
  ..............
  ..............
}

//         Varianta II
struct complex {
                double x ;
                double y ;
               } a, b, c, d ;

int main()
{ ..................
  ..................
  ..................
}
```

În prima variantă se face declararea structurii **complex**, având ca şi componente partea reală (**double x**) şi cea imaginară (**double y**) ale unui număr complex iar în funcţia **main( )**, unde se foloseşte structura, se declară că patru variabile **a**, **b**, **c** şi **d** vor fi de tip **struct complex**.

În cea de-a doua variantă, în cadrul declaraţiei de structură, se precizează că variabilele **a**, **b**, **c** şi **d** vor fi de tip **struct complex**, motiv pentru care nu mai este necesară o altă declarare a tipului acestora în funcţia **main** .

Există şi o a treia variantă, în care se foloseşte declaraţia **typedef** prin care tipul de structură primeşte un nume, în acest caz **complex**, care va fi folosit la definirea variabilelor cu care se vor nota structurile de tipul respectiv :

```
//         Varianta III
typedef struct { double x ; double y ;
               } complex ;
```

```
int main()
{ ..............
  complex a, b, c, d ;
  ..............
  ..............
}
```

În continuare se prezintă câteva exemple de structuri mai frecvent folosite :

- structura care conţine ***coordonatele unui punct din spaţiu*** :

```
struct punct3D { double x ; double y ; double z ; } ;
```

- structura care defineşte o: ***dată calendaristică***

```
struct data_calend { int an ;
                     char luna[30];
                     int zi ;
                   } ;
```

- structura care conţine ***elementele de identificare ale unui student*** :

```
struct student {
                 char nume[30] ;
                 char prenume[30] ;
                 char sex ;
                 int  an_nastere ;
                 char luna_nast[20] ;
                 int  ziua_nast ;
                 char localit_nast[50] ;
                 char facultatea[35] ;
                 char specializarea[25] ;
                 int  grupa ;
                 char domiciliu_permanent[100] ;
                 char domiciliu_Cluj_Napoca[100] ;
                } ;
```

Asemănător cu cazul variabilelor simple şi al tablourilor, cu una sau mai multe dimensiuni, cărora li se asociază pointeri care conţin adresele primilor octeţi din care încep înregistrările, şi în cazul structurilor se pot folosi pointeri.

Dacă se consideră funcţia **main( )** de la ultimul exemplu, se pot defini pointerii **pa** şi **pb** , cărora apoi li se atribuie adresele primelor două structuri **a** şi **b** .

```
int main()
{ ..............
  complex a, b, c, d , *pa, *pb;
  ..............
  pa=&a; pb = &b ;
  ..............
}
```

Utilizarea pointerilor la structuri este necesară în special atunci când se folosesc funcţii între care trebuie să se transmită structuri, acest lucru fiind mult mai eficient atunci când se face transmiterea valorilor pointerilor la structuri .

Iniţializarea elementelor unei structuri se poate face în faza de declarare a structurii, folosind o formă de scriere similară cu a structurii, elementele structurii fiind scrise între acolade, separate prin virgule. În secvenţa următoare de instrucţiuni, în funcţia **main** se face iniţializarea elementelor structurilor cu numele **complex** (care are două elemente cu valori numerice reale) , **punct3D** (care are trei elemente cu valori numerice reale) şi **data_calend** (care are trei elemente, primul cu o valoare numerică întreagă, al doilea va fi un şir de caractere iar al treilea va avea o valoare numerică întreagă) .

```
int main()
{ ..................
  struct complex z = { 12.78 , -0.23 };
  struct punct3D p = { 90.9, 34.8, -23.4 } ;
  struct data_calend A = { 1973, "October", 12 };
  ..................
}
```

Este evident că în acest caz s-a presupus că există făcute anterior declaraţiile celor trei structuri care se iniţializează.

Dacă un element al structurii este un tablou unidimensional sau bidimensional (matrice) atunci în cazul iniţializării respectivului element valorile şirului (matricei) se vor scrie între acolade , ca în exemplul următor :

```
struct  oper { double x[4]; double v[2][3]; } ;

int main()
{ ..................
  struct oper  m = { { 1.2, 0.9, -9.2, 2.9} ,
                     { 11., 12., 13., 21., 22., 23. } };
  ..................
}
```

Elementelor componente ale unei structuri li se pot atribui valori şi pe parcursul execuţiei programului. Există trei posibilităţi de a ne referi la elementele unei structuri .

Prin una din construcţiile

etic_str . nume_elem_i *sau* (*p) . nume_elem_i
sau p **->** nume_elem_i

unde **p** este pointerul asociat structurii cu marcajul **etic_str**, ne putem referi la elementul ***nume_elem_i*** al structurii , să-i atribuim valori şi să facem diverse operaţii cu el. După cum se vede, aici s-au utilizat operatorii punct **.** (selector direct de componente) şi săgeată **->** (selector indirect de componente). Se poate observa că **(*p).** este echivalent cu **p->**

Ca exemplu s-ar putea prezenta modul cum se atribuie valori elementelor structurii **data_calend** :

```
struct data_calend { int an ; char luna[30]; int zi ; };

int main()
{ ..................
  struct data_calend ..., A ,.... ;
  ..................
```

```
    A.an=1973;                    //  Cazul I
    strcpy(A.luna, "October");    //  Cazul II
    A.zi=12;                      //  Cazul I
    ..................
  }
```

După cum se poate observa, primului şi celui de-al treilea element al structurii (cazul I) li s-au atribuit valori întregi utilizând în partea stângă a semnului egal primul mod posibil de referire la elementul structurii, folosind operatorul direct de selectare **.** (punct).

În cazul celui de-al doilea element al structurii, care este un şir de caractere (denumirea unei luni), nu se poate face o atribuire de forma :

```
A.luna = "October" ;     // GRESIT !!!
```

deoarece în stânga semnului egal este un pointer iar în dreapta un şir .

Pentru a realiza atribuirea şirului "October" celui de-al doilea element al structurii în acest caz se foloseşte funcţia

```
char *strcpy (char *string1, const char *string2) ;
```

din **string.h** cu care se va face copierea şirului.

În ceea ce priveşte "***vizibilitatea***" structurilor în cadrul unui program sursă, acestea pot fi, aşa cum sunt şi variabilele,

- globale (se declară în afara funcţiilor) ,

- automatice (se declară în interiorul unei anumite funcţii) şi

- **statice** (declaraţiile acestor structuri vor începe cu cuvântul **static**) .

EXEMPLUL nr. 12.1. Programul care urmează (este deosebit de simplu având ca scop numai înţelegerea problemelor anterior prezentate) va realiza declararea unei structuri corespunzătoare numerelor complexe, atribuirea de valori iniţiale elementelor a două structuri pentru numere complexe, realizarea însumării celor două numere complexe şi afişarea rezultatelor :

```
#include <conio.h>
#include <stdio.h>

struct complex { double x; double y; } ;

int main()
{ struct complex *p2, *p3, z1, z2, z3;
  clrscr();
  p2=&z2; p3=&z3;
  z1.x=1.2; z1.y=-0.87;
  p2->x=-2.34; p2->y=0.99;
  z3.x=z1.x+z2.x; z3.y=z1.y+z2.y;
  printf("\n\n The real part      = %14.7le = %14.7le ="
         " %14.7le\n The imaginary part = %14.7le "
         "= %14.7le = %14.7le\n",
          z3.x, p3->x, (*p3).x, z3.y, p3->y, (*p3).y);
  getch(); return 0;
}
```

Se poate observa că în interiorul funcţiei **main( )** se declară cele trei structuri **z1**, **z2** şi **z3** care vor fi utilizate, precum şi doi pointeri la acelaşi tip de structură, **p2** şi **p3** , aceşti pointeri fiind asociaţi structurilor **z2** şi **z3**, lucru care rezultă în urma atribuirilor:

p2 = &z2 ; p3 = &z3 ;

Valorile elementelor structurilor se atribuie în două moduri: odată folosind scrierea elementului structurii utilizând operatorul punct

z1.x = 1.2 ; z1.y = -0.87 ;

iar a doua oară folosind operatorul săgeată,

p2->x = -2.34 ; p2->y = 0.99 ;

Afişarea rezultatelor obţinute se face în trei moduri, scriind în cele trei feluri posibile elementele structurii,

z3.x , p3->x , (*p3).x (partea reală – primul element al structurii),
z3.y , p3->y , (*p3).y (partea imaginară – al doilea element al structurii)

observând coincidenţa datelor afişate.

EXEMPLUL nr. 12.2 . Se cere să se determine numărul complex **z** (partea reală, imaginară, modulul şi argumentul) care rezultă ca urmare a calculului următoarei expresii

$$z = x + i\,y = (1+3i)\,(-2+5i) + \frac{1+7i}{2-3i} - (20-6i)$$

Operaţiile cu numere complexe care trebuie programate sunt următoarele :

- adunarea (şi scăderea) a două numere complexe

$$x + i\,y = (x_1 + i\,y_1) \pm (x_2 + i\,y_2) = (x_1 \pm x_2) + i(y_1 \pm y_2),$$

- produsul a două numere complexe

$$x + i\,y = (x_1 + i\,y_1) \times (x_2 + i\,y_2) = (x_1 x_2 - y_1 y_2) + i(x_1 y_2 + x_2 y_1),$$

- câtul a două numere complexe

$$x + i\,y = \frac{x_1 + i\,y_1}{x_2 + i\,y_2} = \frac{x_1 x_2 + y_1 y_2}{x_2^2 + y_2^2} + i\,\frac{x_2 y_1 - x_1 y_2}{x_2^2 + y_2^2}, \qquad x_2^2 + y_2^2 > 0,$$

- valoarea absolută a unui număr complex

$$\left| x + i\,y \right| = \sqrt{x^2 + y^2} ,$$

- argumentul unui număr complex

$$\arg(x + i\,y) = \begin{cases} atan2(y, x), & atan2(y,x) \geq 0, \\ atan2(y,x) + 2\pi, & atan2(y,x) < 0. \end{cases}$$

Rezolvarea acestei probleme se poate face în două feluri, în ambele folosind funcţii pentru realizarea operaţiilor cu numerele complexe – adunare, scădere, înmulţire, împărţire, determinarea modulului şi a argumentului, în primul caz folosind modul "clasic" de lucru, fără a utiliza structuri, aşa cum s-a procedat în cuprinsul Capitolului X iar în al doilea caz, utilizând structuri.

În continuare se prezintă prima variantă , în scrierea căreia s-au utilizat numai cunoştinţele prezentate în Capitolul X :

```
//      Varianta I (nu se utilizeaza structuri)
//
#include <conio.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#define PI 3.1415926

void add_com (double,double,double,double,double *,double *);
void diff_com(double,double,double,double,double *,double *);
void mult_com(double,double,double,double,double *,double *);
void div_com (double,double,double,double,double *,double *,int *);
double abs_com (double,double);
double arg_com (double,double);

int main(void)
{ int cod;
  double x1=1., y1=3., x2=-2., y2=5., x3=1., y3=7., x4=2., y4=-3.,
         x5=20., y5=-6., x6, y6, x7, y7, x8, y8, x9, y9;
  clrscr();
  mult_com(x1, y1, x2, y2, &x6, &y6);
  div_com(x3, y3, x4, y4, &x7, &y7, &cod);
  if(cod != 0) exit;
  add_com(x6, y6, x7, y7, &x8, &y8);
  diff_com(x8, y8, x5, y5, &x9, &y9);
  printf("\n\n The real part         = %14.7le"
         "\n The imaginary part    = %14.7le", x9, y9);
  printf("\n The absolute value    = %14.7le", abs_com(x9, y9));
  printf("\n The argument (degree) = %8.3f",
 arg_com(x9, y9)*180/PI);
  getch(); return 0;
}

void add_com(double x1, double y1, double x2, double y2,
             double *x, double *y)
{ *x=x1+x2; *y=y1+y2; }
```

```
void diff_com(double x1, double y1, double x2, double y2,
              double *x, double *y)
{ *x=x1-x2; *y=y1-y2; }

void mult_com(double x1, double y1, double x2, double y2,
              double *x, double *y)
{ *x=x1*x2-y2*y1; *y=x1*y2+x2*y1; }

void div_com(double x1, double y1, double x2, double y2,
             double *x, double *y, int *cod)
{ double delta;
  delta=x2*x2+y2*y2;
  if(delta==0) { *cod=-1; return;}
  *x=(x1*x2+y1*y2)/delta; *y=(x2*y1-x1*y2)/delta;
  *cod=0;  }

double abs_com(double x, double y)
       { return sqrt(x*x+y*y); }

double arg_com(double x, double y)
{ double v;
  v=atan2(y,x);
  if(v<0) v+=2*PI;
  return v;
}
```

Se poate observa că în funcţiile care fac cele patru operaţii s-au folosit câte şase argumente – parametri formali respectiv parametri efectivi, lucru care evident că îngreunează scrierea programului .

În cazul celei de-a doua variante, se utilizează (se declară) o structură **complex** pentru numerele complexe, toate numerele complexe care se vor utiliza (atât cele date iniţial cât şi cele care vor rezulta prin calcul) sunt declarate ca structură, ca urmare numărul de parametri al funcţiilor se va reduce la trei (structuri) în loc de şase, aşa cum a fost în cazul anterior.

```
// Varianta II  (se utilizeaza o structura pentru numere complexe)
//
#include <conio.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

#define PI 3.1415926

struct complex { double x ; double y ; };

void   ADD_com (complex *, complex *, complex *);
void   DIFF_com(complex *, complex *, complex *);
void   MULT_com(complex *, complex *, complex *);
void   DIV_com (complex *, complex *, complex *, int *);
double ABS_com (complex *);
double ARG_com (complex *);
```

```
int main()
{ int cod;
  struct complex *p, z1, z2, z3, z4, z5, z6, z7, z8, z9 ;
  clrscr();
  z1.x=1.; z1.y= 3.; z2.x= -2.; z2.y= 5.;  z3.x=1.; z3.y=7.;
  z4.x=2.; z4.y=-3.; z5.x= 20.; z5.y=-6.;
  p=&z9;
  MULT_com(&z1, &z2, &z6);
  DIV_com(&z3, &z4, &z7, &cod);
  if(cod!=0) exit;
  ADD_com(&z6, &z7, &z8);
  DIFF_com(&z8, &z5, &z9);
  printf("\n\n The real part         = %14.7le = %14.7le"
         "\n The imaginary part    = %14.7le = %14.7le",
         z9.x, p->x, z9.y, p->y);
  printf("\n The absolute value    = %14.7le", ABS_com(&z9));
  printf("\n The argument [degree] = %8.3f",ARG_com(&z9)*180/PI);
  getch(); return 0;
}

void ADD_com(complex *z1, complex *z2, complex *z3)
{ z3->x = z1->x + z2->x; z3->y = z1->y + z2->y; }

void DIFF_com(complex *z1, complex *z2, complex *z3)
{ z3->x = z1->x - z2->x; z3->y = z1->y - z2->y; }

void MULT_com(complex *z1, complex *z2, complex *z3)
{ z3->x= z1->x * z2->x - z1->y * z2->y ;
  z3->y= z1->x * z2->y + z2->x * z1->y ; }

void DIV_com(complex *z1, complex *z2, complex *z3, int *cod)
{ double delta;
  delta = z2->x * z2->x + z2->y * z2->y ;
  if(delta==0) { *cod=-1; return;}
  z3->x=(z1->x * z2->x + z1->y * z2->y)/delta;
  z3->y=(z2->x * z1->y - z1->x * z2->y)/delta;
  *cod=0; }

double ABS_com(complex *z)
{ return sqrt( z->x * z->x + z->y * z->y ); }

double ARG_com(complex *z)
{ double angle;
  angle = atan2( z->y , z->x );
  if(angle < 0) angle+=2*PI;
  return angle;  }
```

Observând de exemplu cum se face operaţia de înmulţire

```
.......................
MULT_com(&z1, &z2, &z6);
.......................
```

ca urmare a apelului funcţiei

```
void MULT_com(complex *z1,complex *z2,complex *z3)
{ ................................ }
```

rezultă că transferul de valori ale elementelor structurilor se face prin intermediul valorilor pointerilor respectivelor structuri.

În cadrul programului, în funcţia **main( )**, s-a folosit şi un pointer **p** , asociat structurii **z9**. Se poate observa în cazul listei de variabile de la funcţia **printf** că se va obţine acelaşi rezultat în urma afişării lui **z9.x** sau **p->x** respectiv **z9.y** sau **p->y** .

Pentru datele numerice introduse s-au obţinut rezultatele :

```
z = -3.8461538e+01 + i 6.3076923e+00
|z| = 3.8975337e+01
arg z = 170.686404
```

EXEMPLUL nr. 12.3 . Un număr de **n** candidaţi (identificaţi prin ***nume*** şi ***prenume***) care s-au prezentat pentru ocuparea unui post, ca urmare a unui examen, au primit fiecare câte o ***notă*** .

Programul care urmează realizează următoarele :

- declararea unei structuri cu marcajul **applicant** (applicant – candidat, solicitant) care conţine trei elemente : ***surname*** (surname – nume de familie), ***name*** (name – prenume) şi ***mark*** (mark – notă), primele două fiind şiruri de caractere iar al treilea un număr real (deoarece nota poate fi o medie a mai multor probe sau o medie a câte unei note acordate de fiecare examinator),
- declararea unui şir **A** conţinând structuri de tipul declarat,
- introducerea valorii numărului **n** al candidaţilor,
- introducerea datelor corespunzătoare fiecărui candidat (nume, prenume şi notă), în cadrul unui ciclu realizat cu instrucţiunea **for** , concomitent cu actualizarea numărului de ordine al candidatului (al structurii) care a obţinut nota maximă ,
- afişarea listei candidaţilor şi a notelor (deci a elementelor celor **n** structuri),
- afişarea structurii care conţine datele candidatului reuşit,

```
#include <conio.h>
#include <ctype.h>
#include <stdio.h>
#include <string.h>

struct applicant{ char surname[15] ;
                  char name[15] ;
                  double mark;
                } ;

int main()
{ int i, i_max, n; double mark, max_mark=0.0; char str_char[15];
  struct applicant A[50];
  clrscr();
  printf("\n The number of applicants : "); scanf("%d", &n);
  for(i=1; i<=n; i++)
     { printf("\n %3d  SURNAME : ", i); scanf("%s", str_char);
       strcpy(A[i].surname, str_char);
```

```
       printf("      NAME    : "); scanf("%s", str_char);
       strcpy(A[i].name, str_char);
       printf("      The obtained MARK = ");scanf("%le", &mark);
       A[i].mark=mark;
       if(mark>max_mark) { max_mark=mark; i_max=i; }
     }
  for(i=1; i<=n; i++)
     { printf("\n %2d %15s %15s %6.3f",
       i, A[i].surname, A[i].name, A[i].mark);
       if(i%10==0) getch();
     }
  printf("\n\n The winner is %s %s with %6.3f\n",
 A[i_max].surname, A[i_max].name, A[i_max].mark);
  getch(); return 0;
}
```

Se poate observa modul cum s-a făcut atribuirea numelui şi prenumelui fiecărui candidat celor două elemente ale structurii: de fiecare dată s-au parcurs două etape, în prima s-a citit un şir de caractere care s-a notat cu **str_char** iar apoi cu instrucţiunea **strcpy (...)** s-a copiat respectivul şir fie în elementul **A[i].surname** fie în **A[i].name** .

Dacă s-ar pune şi problema aranjării candidaţilor în ordinea descrescătoare a notelor obţinute şi afişarea elementelor structurilor, atunci în program ar trebui să se facă următoarele adăugiri :

- se va adăuga declararea variabilei întregi **cod** cu valoare iniţială 1,

```
int cod=1, i, i_max, n;
```

- se va adăuga porţiunea de program care urmează

```
.................
//     The "bubble sort" method
while(cod != 0)
       { cod=0;
         for(i=0; i<n-1; i++)
            { if(A[i].mark<A[i+1].mark)
                 { strcpy(str_char, A[i].surname);
                   strcpy(A[i].surname, A[i+1].surname);
                   strcpy(A[i+1].surname, str_char);
                   strcpy(str_char, A[i].name);
                   strcpy(A[i].name, A[i+1].name);
                   strcpy(A[i+1].name, str_char);
                   mark=A[i].mark; A[i].mark=A[i+1].mark;
                   A[i+1].mark=mark;
                   cod=1;
                 }
            }
       }
  printf("\n\n The list of the applicants\n");
  for(i=0; i<n; i++)
     { printf("\n %2d %15s %15s %6.3f",
               i+1, A[i].surname, A[i].name, A[i].mark);
       if((i+1)%10==0) getch();
     }
.................
```

înainte de ultimul rând al funcţiei **main( )** care conţine

```
getch();  return 0;
```

utilizându-se ***"bubble sorting"*** – metoda bulelor (prezentată în cadrul unui program anterior) pentru realizarea ordonării descrescătoare. Schimbarea între ele a conţinutului elementelor a două structuri succesive, în timpul unor parcurgeri repetate a şirului de structuri continuă până când variabila **cod** rămâne cu valoarea **0** .

12.2 Uniuni.

Problema "economiei" de memorie este rezolvată în cazul limbajului C/C++ prin utilizarea uniunilor, prin folosirea cărora este posibil ca în aceeaşi zonă de memorie să se înregistreze diferite tipuri de date, în diverse faze ale execuţiei programului, "suprapunându-se" înregistrările.

Declararea uniunilor este similară cu cea a structurilor, diferind doar semnificaţia pe care o au elementele uniunii

```
union etic_union {tip nume_elem_1;
                  tip nume_elem_2;
                  ..............
                  tip nume_elem_n;
                 } nume_union_1,nume_union_2,..,nume_union_n ;
```

Se foloseşte cuvântul instrucţiune **union**, urmat de eticheta (marcajul) cu care se notează uniunea, în interiorul acoladelor fiind o listă a **elementelor** uniunii, variabile simple, tablouri cu una sau mai multe dimensiuni, şiruri de caractere, a căror înregistrare se va face în aceleaşi locaţii de memorie. Este evident ca din faza de compilare se va reţine o zonă de memorie corespunzătoare în care este posibilă înregistrarea elementului cu cea mai mare dimensiune. Ca şi în cazul declaraţiei de structură poate exista o listă (opţională) de denumiri de uniuni de acelaşi tip cu cea definită iar declaraţia de uniune se termină cu **;** În cazul când există această listă de denumiri de uniuni numele acestora nu va mai trebui declarat ca uniune.

Astfel, dacă într-o funcţie se va lucra cu un şir **A** de numere reale cu 100 de elemente , cu o matrice **B** cu 15 linii şi 15 coloane şi cu o variabilă **c** de tip caracter, la momente diferite, se va utiliza următoarea secvenţă de instrucţiuni :

```
union abc {double A[100];
           double B[15][15];
           char c;
          } ;
.................
int main( )
{ .................
  union abc u1, u2;
  .................
}
```

A fost necesară declararea în cadrul funcţiei a uniunilor **u1** şi **u2** de tipul **abc** deoarece la declararea respectivei uniuni nu s-a făcut acest lucru.

Variabilele **u1** şi **u2** de tip **union** vor fi suficient de mari pentru ca să poată păstra cea mai mare valoare (din punct de vedere al ocupării memoriei) dintre elementele componente.

În cazul uniunilor se vor putea face toate operaţiile care au fost definite în cazul structurilor: referirea la un element al uniunii prin folosirea operatorului punct, utilizarea unui pointer la uniune şi referirea la un element al uniunii cu ajutorul operatorului săgeată, folosirea uniunii ca parametru sau a pointerului corespunzător în cazul apelului unor funcţii, etc.

O instrucţiune care este similară cu cea prezentată există şi în limbajul **Fortran**, aceasta fiind **equivalence.**

12.3 Câmpuri de biţi

În limbajul de programare C/C++ este posibilă şi accesarea unor zone din memorie în care se află înregistraţi unul sau mai mulţi biţi (**bit-field**), ca urmare a utilizării unei structuri speciale, care grupează câmpurile de biţi.

Forma generală este următoarea :

```
struct etic_camp { tip nume_camp_1 : nr_biti_1 ;
                   tip nume_camp_2 : nr_biti_2 ;
                   ..........................
                   tip nume_camp_n : nr_biti_n ;
                 } nume_1, nume_2, ..., nume_k ;
```

unde **tip** este de obicei **unsigned**, **nr_bit_i** este numărul de biţi pe care se reprezintă în memorie valoarea variabilei **nume_camp_i** iar **nume_1**, **nume_2**, ... , **nume_k** este o listă de denumiri de variabile de tip câmp, care se vor utiliza în program.

Variabilele câmp de biţi sunt înregistrate în memorie pe 16 biţi (dacă aceştia sunt suficienţi) sau pe un multiplu de 16 biţi , primele câmpuri ocupând partea cea mai puţin semnificativă (din dreapta).

Referirea la un anumit câmp de biţi se face la fel ca la un element al unei structuri

```
etic_camp . nume_camp_i
```

utilizând operatorul punct .

Dacă s-a declarat câmpul de biţi :

```
struct campbit { unsigned a : 1 ;
                 unsigned b : 4 ;
                 unsigned c : 6 ;
               } AAA ;
```

rezultă că în program se va folosi variabila de tip câmp **AAA** care conţine câmpurile **a** de un bit, **b** de 4 biţi şi **c** de 6 biţi .

Modul cum sunt aranjate în memorie cele trei câmpuri de biţi este următorul :

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					câmpul c						câmpul b				a

Atribuirea unei valori unuia din aceste câmpuri se face în felul următor :

```
AAA.a = 1 ;
```

Utilizarea variabilelor tip câmp de biţi trebuie făcută cu precauţie, deoarece multe probleme depind de arhitectura calculatorului.

Capitolul XIII

Fişiere

13.1 Probleme generale privind fişierele

În cadrul acestui capitol se vor folosi două noţiuni: cea de **fişier** şi cea de **flux** (**file** respectiv **stream** în limba engleză) .

Un **fişier** este o colecţie ordonată de înregistrări (articole) aflată pe un suport magnetic, de obicei harddisk–ul. În cazul când datele se introduc de la tastatură înregistrarea va conţine datele tastate pe un rând. Dacă datele se afişează la imprimantă atunci înregistrarea este compusă din totalitatea datelor de pe un rând .

Un **stream** este "canalul (dispozitivul) logic" prin care se realizează operaţiile de **I/O**. S-a introdus această noţiune cu scopul de a face şi de a putea trata diferitele funcţii care realizează operaţiile de **I/O** în mod independent de dispozitivul utilizat : o funcţie de scriere (înregistrare) a datelor pe disc este similară (identică) cu cea care face acelaşi lucru pe monitor.

Ca urmare a execuţiei programelor prezentate în capitolele anterioare s-a lucrat cu trei feluri de fluxuri (aceste fluxuri apar în mod automat): fluxul de intrare standard **stdio** şi cel de ieşire standard **stdout**, primul fiind asociat terminalului de la care se introduc datele – ***tastatura***, iar al doilea terminalului unde se prezintă rezultatele – ***monitorul***. Mai există şi un al treilea flux standard de editare a erorilor – **stderr**. În momentul apelului unui program în **C** aceste trei fluxuri sunt în mod automat definite şi deschise.

Există două feluri de **stream**-uri : **stream binar** şi **stream text** .

Stream-ul **binar** este o secvenţă ordonată de caractere (octeţi) care se înregistrează fără să se facă nici o conversie, astfel încât la citire, în cazul aceleiaşi implementări, se va obţine acelaşi lucru.

Stream-ul **text** conţine caractere organizate linie după linie, după fiecare linie urmând **NL** (new line). La înregistrare caracterele **CR** (carriage return) şi **FF** (form feed) vor fi traduse ca new line. În cazul acestor stream-uri nu există o corespondenţă biunivocă între caracterele transmise şi cele care apar pe dispozitivul extern.

În cadrul programelor se calculează diverse valori care se atribuie fie unor variabile simple fie elementelor unor tablouri uni sau multidimensionale, aceste valori fiind disponibile numai în timpul execuţiei programului iar la terminarea execuţiei se distrug.

Sunt situaţii când anumite date calculate ca urmare a execuţiei unui program se doreşte să se reţină în memoria calculatorului (pe harddisk) sub forma unor fişiere, de exemplu la o anumită dată, cu un anumit program s-a creat un fişier iar la o altă dată, ulterioară, cu un alt program datele din acest fişier sunt prelucrate sau modificate .

În momentul când un fişier este deschis de către o funcţie standard de bibliotecă i se asociază în mod automat un stream .

Numele atribuit fişierului este cel al unui pointer de tip **FILE** .

Conform documentaţiei, **FILE** este o structură definită pentru **stream**-uri, având următoarea formă :

```
typedef struct {short           level ;
                unsigned        flags ;
                char            fd ;
                unsigned char   hold ;
                short           bsize ;
                unsigned char   *buffer, *curp ;
                unsigned        istemp ;
                short           token ;
               } FILE ;
```

Această structură nu se crează în mod explicit prin program, ea se generează şi utilizează în mod automat prin funcţiile de bibliotecă.

Dacă într-un program se va utiliza un fişier, atunci va trebui să existe o declaraţie de forma :

```
FILE  *pf ;
```

unde **pf** este un pointer la o zonă de memorie care conţine toate informaţiile legate de fişier : nume, stare, poziţie curentă, etc.

Legat de problema fişierelor trebuie să se rezolve următoarele lucruri: fişierele trebuie **declarate**, aşa cum s-au declarat variabilele simple, tablourile, structurile, înaintea folosirii lor fişierele trebuie **deschise** (programul C/C++ care realizează acest lucru se ***conectează*** la fişier, un stream se asociază la fişier), în fişiere trebuie să se **introducă informaţii**, din ele să se **extragă informaţii**, să se **modifice** sau să se **şteargă informaţiile** existente, după folosire fişierele trebuie **închise**, (programul C/C++ se ***deconectează*** de la fişier, stream-ul se disociază de fişier), în unele situaţii fişierele, în întregime , trebuie **şterse**.

Principalele funcţii specializate de gestiune a fişierelor, care aparţin fişierului standardizat **stdio.h** , pentru realizarea diferitelor operaţii cu fişierele sunt următoarele :

Denumire	*Semnificaţie*
fattr(...)	Modificarea atributelor unui fişier
fclose(...)	Închide un fişier
feof(...)	Returnează "adevărat" dacă se atinge sfârşitul fişierului
ferror(...)	Returnează "adevărat" dacă survine o eroare
fflush(...)	Goleşte conţinutul unui fişier
fget(...)	Citeşte un articol
fgetc(...)	Citeşte un caracter dintr-un fişier
fgets(...)	Citeşte un şir de caractere
fmkdl(...)	Marcarea fişierului pentru ştergere
fopen(...)	Deschide un fişier
fprintf(...)	Analogul pt. fişiere al funcţiei printf
fput(...)	Scrie un articol
fputc(...)	Scrie un caracter într-un fişier
fputs(...)	Scrie un şir de caractere
fread(...)	Citeşte un articol din fişier
fscanf(...)	Analogul pt. fişiere al funcţiei scanf

fseek(...)	Caută un anumit octet într-un fişier
fskip(...)	Salt peste un articol în fişier
ftell(...)	Găseşte poziţia curentă în fişier
fwrite(...)	Scrie un articol în fişier
getc(...)	Citeşte un caracter dintr-un fişier
putc(...)	Scrie un caracter într-un fişier
remove(...)	Şterge un fişier
rewind(...)	Iniţializează indicatorul de poziţie al fişierului la începutul acestuia
ungetc(...)	Repunerea caracterului citit în exces înapoi în fluxul de intrare
unlink(...)	Şterge un fişier

13.2 Deschiderea şi închiderea unui fişier. Determinarea sfârşitului unui fişier .

Deschiderea şi închiderea unui fişier se face cu funcţiile **fopen(...)** şi **fclose(...)**, care au următoarele prototipuri :

```
FILE *fopen(const char *name_file, const char *mod);

int fclose(FILE *pf);
```

Funcţia **fopen(...)** returnează un pointer spre tipul **FILE** sau pointerul nul (**NULL**) în caz de eroare.

În cazul funcţiei **fopen(...)** primul argument este denumirea fişierului care se va deschide iar al doilea este un indicator privind tipul fişierului şi operaţiile ce urmează să se facă cu acesta, conform următoarelor :

Mod	*Semnificaţie*
"r"	Deschide un fişier text în vederea citirii
"w"	Crează un fişier text în vederea scrierii. Dacă fişierul există atunci vechiul conţinut se pierde
"a"	Adaugă la sfârşitul unui fişier text. Dacă fişierul nu există atunci el este creat
"rb"	Deschide un fişier binar în vederea citirii
"wb"	Crează un fişier binar în vederea scrierii
"ab"	Adaugă la sfârşitul unui fişier binar
"r+"	Deschide un fişier text pentru citire/scriere
"w+"	Crează un fişier text pentru citire/scriere
"a+"	Adaugă la sfârşit sau crează un fişier text pentru citire/scriere
"r+b"	Deschide un fişier binar pentru citire/scriere
"w+b"	Crează un fişier binar pentru citire/scriere
"a+b"	Adaugă la sfârşit sau crează un fişier binar

Se ştie că valoarea unui pointer asociat unui fişier inexistent este **NULL**. În consecinţă, pentru a păstra controlul asupra execuţiei programului, după utilizarea funcţiei **fopen(...)** va trebui să verificăm dacă respectivul fişier a fost deschis, deci dacă pointerul asociat nu are valoarea **NULL** .

În orice funcţie în care se fac operaţii cu fişiere, după deschiderea unui fişier este indicat să se facă o astfel de verificare, conform următoarei secvenţe de instrucţiuni :

```
{ .....
  FILE *pf ;
  .....
  pf = fopen("f001.ttt", "w");   //Deschiderea fisierului
  if(pf == NULL)
    { printf("\n\n\t The file  f001.ttt cannot be "
             "opened !");
      exit(1);
    }
  .....
  fclose(pf);                    //Inchiderea fisierului
  .....
}
```

În această secvenţă de instrucţiuni se presupune că fişierul **f001.ttt** , care se deschide, se află în acelaşi director ca şi programul **C** care-l deschide. Evident că în cazul în care fişierul se găseşte într-un alt director atunci trebuie precizată calea , de exemplu :

d:\curs_cpp\f001.ttt

iar instrucţiunea se va scrie sub forma :

```
pf=fopen("d:\\curs_cpp\\f001.ttt", "w");
```

în şirul conţinut între ghilimele, conform regulii cunoscute, caracterul \ se va scrie ca o secvenţă escape \\.

După ce s-au făcut diverse operaţii cu elementele fişierului, acesta trebuie închis. Prototipul funcţiei de închidere este următorul :

```
int fclose(FILE *pf);
```

iar modul de folosire se poate vedea în secvenţa de instrucţiuni prezentată anterior, ca argument al acestei funcţii utilizându-se pointerul asociat fişierului.

Funcţia **fclose(...)** returnează valoarea **0** la o ***închidere normală*** a fişierului respectiv **-1** în caz de ***eroare***.

Este obligatoriu să se facă închiderea fişierelor deoarece numărul de fişiere care pot fi simultan deschise este limitat.

Determinarea sfârşitului unui fişier se face cu ajutorul funcţiei **feof(...)** a cărei prototip este următorul :

```
int feof(FILE *pf);
```

Această funcţie returnează o valoare diferită de zero dacă s-a ajuns la sfârşitul fişierului , **EOF**, după ultima operaţie efectuată, respectiv zero în caz contrar.

În secvenţa de instrucţiuni următoare :

```
......
while ( ! feof(pf))
     { .....  }
......
```

instrucţiunea compusă care urmează după **while** se va parcurge repetat, până când se ajunge la sfârşitul fişierului al cărui pointer este **pf** .

Ştergerea unui fişier se face cu funcţia **remove (...)** care are următorul prototip:

```
int remove(const char *nume_fis);
```

Se returnează valoarea **0** dacă operaţia de ştergere s-a realizat, respectiv o valoare diferită de zero în caz de eroare .

13.3 Operaţii de I/O în cazul fişierelor

Se pot distinge mai multe situaţii, funcţie de tipul informaţiei care se prelucrează.

Astfel, cazul cel mai simplu este cel al scrierii şi citirii unor ***caractere***, cazuri în care se folosesc funcţiile **fputc(...)** şi **fgetc(...)**. Scrierea şi citirea unor ***şiruri de caractere***, considerate ca entităţi, se face cu funcţiile **fputs(...)** şi **fgets(...)**

Scrierea şi citirea ***datelor cu formate*** se realizează cu funcţiile **fprintf(...)** şi **fscanf(...)**, foarte asemănătoare ca mod de utilizare cu funcţiile **printf(...)** şi **scanf(...)** folosite la operaţiile de I/O obişnuite (vezi Capitolul VII), avantajul utilizării acestora fiind acela că sunt folosite pentru I/O atât în cazul caracterelor, al şirurilor de caractere cât şi al informaţiei numerice.

Prezintă de asemenea interes funcţiile **fwrite(...)** şi **fread(...)** utilizate în cazul operaţiilor de I/O la ***fişiere binare*** .

Scrierea /citirea unui singur caracter . Aceste operaţii se realizează cu funcţiile **fputc(...)** şi **fgetc(...)** , prototipurile acestora fiind prezentate în continuare :

```
int fputc(int ch, FILE *fp) ;

int fgetc(FILE *fp) ;
```

În cazul operaţiei de scriere (înregistrare), cu funcţia **fputc(...)** ca argumente se utilizează caracterul care se înregistrează (considerat ***unsigned char***), acesta scriindu-se în poziţia curentă în fişier şi pointerul asociat fişierului iar funcţia returnează caracterul scris (ca un ***int***) respectiv **EOF** în caz de eroare.

Pentru a face citirea unui caracter, de pe poziţia curentă dintr-un fişier, se utilizează funcţia **fgetc(...)** cu care se citeşte caracterul, ca un ***unsigned int*** şi se converteşte în ***int*** care se returnează. Ca parametru se utilizează pointerul spre o structură **FILE**, pointer asociat fişierului. În cazul când se returnează **EOF** înseamnă fie că s-a ajuns la sfârşitul fişierului, fie eroare. În cazul utilizării repetate a funcţiei **fgetc(...)** se face de fiecare dată citirea caracterului următor, deoarece la fiecare utilizare pointerul asociat fişierului este incrementat.

EXEMPLUL nr. 13.1. Se doreşte rezolvarea următoarei probleme: crearea unui fişier text, conţinând un număr neprecizat de caractere, utilizând un program sursă, caracterele fiind scrise cu funcţia **fputc(...)** iar apoi utilizând un alt program sursă citirea caracterelor din fişier, cu funcţia **fgetc(...)** şi afişarea lor pe monitor.

În ambele programe vor exista declaraţiile pointerului unei structuri de tip **FILE**, se vor folosi funcţiile **fopen(...)**, referitoare la acelaşi fişier **file.$$$** , cu diferenţa că în primul caz, la crearea fişierului, această funcţie va conţine ca argument **"w"** (se deschide un fişier gol pentru scriere) iar în al doilea program argumentul va fi **"r"** (se deschide un fişier pentru citire) .

După cum s-a impus, numărul de caractere care trebuie introduse (scrise în fişierul **file.$$$** respectiv citite din acelaşi fişier, cu al doilea program, nu este precizat .

Terminarea operaţiei de preluare a datelor de la tastatură şi scrierea lor în fişier este dată de apăsarea tastei Enter . Trebuie reamintit că tastei respective îi corespunde valoarea octală 15, motiv pentru care în cadrul parantezei de la instrucţiunea **while** se face verificarea dacă caracterul tastat nu are valoarea 15, în octal. În cazul citirii datelor din fişier faptul că s-a ajuns la sfârşitul de fişier este dat de întâlnirea lui EOF, identificat ca urmare a folosirii funcţiei **feof(...).**

Cele două programe sursă sunt prezentate în continuare :

Programul nr. 1

```
#include <conio.h>
#include <stdio.h>

int main(void)
{ int c;
  FILE *f;
  f=fopen("file.$$$", "w");
  clrscr();
  while(c != '\015')      //  CR ~ 015 (octal)
       { c=getche(); fputc(c, f); }
  fclose(f);
  getch(); return 0;
}
```

Programul nr. 2

```
#include <conio.h>
#include <dos.h>
#include <stdio.h>

int main(void)
{ char c;
  FILE *f;
  f=fopen("file.$$$", "r");
  clrscr();
  printf("\n\n The characters of the file will be "
         "displayed :\n\n");
  do
     { c=fgetc(f); delay(200);
       printf(" %c", c);
     }
  while(!feof(f));
  fclose(f);
  printf("\n\n All characters have been displayed . ");
  getch(); return 0;
}
```

În cele două programe sursă s-au tipărit cu caractere bold instrucţiunile specifice operaţiilor cu fişiere.

Scrierea /citirea unui şir de caractere se face cu funcţiile **fputs(...)** respectiv **fgets(...)** care au următoarele prototipuri :

```
int fputs( const char *sir, FILE *pf ) ;

char *fgets( char *str, int lungime, FILE *pf ) ;
```

În cazul funcţiei **fputs(...)** ca prim argument se utilizează identificatorul şirului de caractere care se "scrie" în fişier, al doilea argument fiind pointerul asociat fişierului. Această funcţie returnează o valoare nulă sau pozitivă dacă operaţia s-a putut efectua, respectiv **EOF** în caz contrar.

Funcţia **fgets(...)** are trei argumente, primul fiind numele atribuit şirului de caractere care se "citeşte", al doilea fiind un întreg al cărui valoare este egală cu numărul de caractere al şirului citit, inclusiv caracterul nul **'\0'** iar al treilea pointerul asociat fişierului. Trebuie specificat că în cazul în care în cadrul numărului de caractere specificat se întâlneşte caracterul **'\n'** atunci citirea şirului se întrerupe. Funcţia **fgets(...)** returnează un pointer spre şirul citit, respectiv **NULL** în cazul unei erori sau la întâlnirea sfârşitului de fişier.

EXEMPLUL nr. 13.2 . Se cere să se creeze un fişier, cu numele ***file_str*** , în care să se scrie următoarele trei şiruri de caractere :

```
God said, "Let there be light",\n   ( şirul  A )
and there was light.\n              ( şirul  B )
The Bible - Genesis 1.3             ( şirul  C )
```

la primele două adăugându-se şi caracterul **'\n'** pentru ca tipărirea celor trei şiruri să se facă la distanţe mai mari.

Fiind vorba de trei şiruri de caractere, care vor fi notate cu **A**, **B** şi **C**, fiecare cu câte 50 de caractere declarate, dar mai puţine efectiv, se pot arăta trei posibilităţi de a le iniţializa, înaintea scrierii lor în fişier. Primul şir, **A**, este iniţializat în cadrul declaraţiei de tip **char**, unde după cum se vede pentru a introduce caracterul **"** s-a folosit secvenţa **\".**

Al doilea şir, **B**, este iniţializat cu ajutorul funcţiei de bibliotecă **strcpy(...)** , în el copiindu-se conţinutul şirului utilizat ca al doilea argument al acestei funcţii .

Atât în cazul şirului **A** cât şi al şirului **B** se cunoaşte numărul de caractere introduse : 32 în cazul lui **A** respectiv 26 în cazul lui **B**.

Cel de-al treilea şir, **C**, se introduce de la tastatură, iar pentru a nu fi nevoiţi să numărăm câte caractere s-au introdus, numărul **nC** al acestora se va determina cu o altă funcţie de bibliotecă, **strlen(...)** .

În programul care urmează s-au tipărit cu caractere bold instrucţiunile care se referă la operaţiile de ***deschidere*** şi ***închidere*** ale fişierului precum şi cele de ***I/O*** pentru şiruri.

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
int main(void)
{ int nC;
  char A[50]={"God said, \"Let there be light\",\n"},  B[50], C[50];
                      // A contine 32 caractere + '\0'
  FILE *pf; clrscr();
  pf=fopen("file_str", "a");
  if(pf==NULL)
    { printf("\n The file file_str cannot be opened\n");exit(1);}
  fputs(A, pf);
  strcpy(B, "     and there was light.\n");
                      // B contine 26 caractere + '\0'
  fputs(B, pf);
  printf("\n Keyboard the string of characters : ");
  gets(C);
  nC=strlen(C);       // C contine nC caractere + '\0'
  fputs(C, pf);
  fclose(pf);
//
  fopen("file_str", "r");
  fgets(A, 33, pf);
                // Se citesc 33 (32+1) caractere din fisier
  printf("\n\n\n\t%-50s", A);
  fgets(B, 27, pf);
                // Se citesc 27 (26+1) caractere din fisier
  printf("\n\t%-50s", B);
  fgets(C, nC+1, pf);
                // Se citesc nC+1 caractere din fisier
  printf("\n\t%-50s", C);
  fclose(pf);
  remove("file_str");
  getch(); return 0;
}
```

Se poate observa că după ce fişierul ***file_str*** s-a deschis (s-a utilizat parametrul **"a"** corespunzător unui fişier text la care se face ***adăugarea*** unor înregistrări) şi s-a făcut verificarea că operaţia a reuşit, se face scrierea în fişier a celor trei şiruri, după care fişierul este închis.

În partea a doua a programului, fişierul creat anterior este deschis (se utilizează parametrul **"r"** corespunzător ***citirii*** din fişier) şi se fac trei citiri ale celor trei şiruri introduse, de fiecare dată specificându-se numărul de caractere din care este compus şirul respectiv, inclusiv caracterul nul **'\0'** . După fiecare citire din fişier se face o afişare a şirului la monitor.

Rezultatele sunt următoarele :

```
  Keyboard the string of characters : The Bible - Genesis 1.3


       God said, "Let there be light",

            and there was light.

       The Bible - Genesis 1.3
```

După ce s-au făcut cele trei citiri din fişier, acesta se închide iar apoi se şterge.

Observaţie . Dacă s-ar fi renunţat la includerea la sfârşitul şirurilor **A** şi **B** a caracterului **'\n'** şi s-ar fi considerat un alt şir **D** de tip **char** cu 75 de elemente , atunci s-ar fi putut face o citire din fişier urmată de o afişare

```
.....................
fopen("file_str", "r");
fgets(D, 75, pf);
printf("\n\t%-75s", D);
fclose(pf);
.....................
```

astfel încât să se obţină :

```
God said, "Let there be light",  and there was light.The Bible - Genesis 1.3
```

Scrierea /citirea cu format, cu funcţiile **fprintf (...)** şi **fscanf(...)** . După cum s-a precizat anterior, utilizarea acestor funcţii este asemătoare cu a funcţiilor **printf(...)** şi **scanf(...)**. Atât scrierea cât şi citirea se face sub controlul unor formate. În cazul fişierelor text se face o conversie din format intern în format ASCII şi invers, ceea ce duce la o micşorare a vitezei de prelucrare a fişierelor, conţinutul fişierului nefiind identic cu imaginea sa din memorie .

Prototipurile acestor funcţii sunt următoarele :

```
int fprintf(FILE *fp, const char *sir_control, ...) ;

int fscanf(FILE *fp, const char *sir_control, ... ) ;
```

diferind de cele ale funcţiilor **printf(...)** şi **scanf(...)** prin faptul că aici trebuie utilizat ca argument şi pointerul asociat fişierului unde se face scrierea respectiv de unde se face citirea datelor .

Astfel, dacă trebuiau afişate nişte valori sau introduse de la tastatură (de tip **int**, **double**, etc) s-au folosit instrucţiuni de tipul :

```
.................
int m, n=15; double x=12.345, y;
.................
printf("\n\n\t  n = %d   x = %le\n", n, x);
printf("\n\n m [int]    = "); scanf("%d", &m);
printf("\n y [double] = "); scanf("%le", &y);
.................
```

acelaşi efect avându-l şi utilizarea următoarelor instrucţiuni, unde s-au folosit funcţiile **fprintf(...)** respectiv **fscanf(...)**, ca prime argumente ale acestora figurând pointerii de fişiere **stdout** şi **stdin**, asociaţi monitorului respectiv tastaturii :

```
.................
int m, n=15; double x=12.345, y;
.................
fprintf(stdout, "\n\n\t  n = %d   x = %le\n", n, x);
fprintf(stdout, "\n\n m [int]    = "); fscanf(stdin,"%d",&m);
fprintf(stdout, "\n y [double] = "); fscanf(stdin,"%le",&y);
.................
```

În cazul în care se face o scriere sau citire în fişier se foloseşte ca prim argument pointerul asociat fişierului cu care se lucrează .

În următoarele două exemple sunt utilizate aceste două funcţii **fprintf(...)** şi **fscanf(...)**, la crearea şi la citirea fişierului utilizându-se aceleaşi formate (aceiaşi specificatori de format).

EXEMPLUL nr. 13.3 . În programul următor se crează un fişier cu numele **ABC**, care conţine trei elemente: o valoare de tip **int**, o valoare de tip **double** şi un **şir de caractere**.

Fişierul se deschide în mod scriere (**"w"**) valorile variabilelor de tip **int** şi **double** se introduc de la tastatură , în cazul primei folosind funcţiile **printf(...)** şi **scanf(...)** iar în cazul celei de-a doua funcţiile **fprintf(stdout, ...)** şi **fscanf(stdin, ...)**, şirul de caractere fiind iniţializat în cadrul declarării acestuia . Scrierea în fişier a celor două variabile şi a şirului se face cu următoarea funcţie :

```
fprintf(pf,"%d %le %s", n, x, str);
```

După acestă operaţie şi închiderea fişierului, acesta se va deschide şi închide de două ori, în primul caz citindu-se deodată toate elementele fişierului iar a doua oară efectuându-se trei citiri. Este evident că se vor obţine aceleaşi rezultate, afişate pe ecranul monitorului .

Programul este următorul :

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{int n; double x; char *str="Programming_in_C";
 FILE *pf; clrscr();
 if((pf=fopen("ABC", "w"))==NULL)
   { printf("\n\n\t The file  ABC  cannot be opened !");
     exit(1); }
 printf("\n n [int]    = "); scanf("%d", &n);
 fprintf(stdout,"\n x [double] = "); fscanf(stdin,"%le",&x);
 fprintf(pf,"%d %le %s", n, x, str);
 fclose(pf);
//
 if((pf=fopen("ABC", "r"))==NULL)
   { printf("\n\n\t The file  ABC  cannot be opened !");
     exit(1); }
 fscanf(pf, "%d %le %s", &n, &x, str);
 printf("\n\n %d  %e  %s", n, x, str);
 fclose(pf);
//
 if((pf=fopen("ABC", "r"))==NULL)
   { printf("\n\n\t The file  ABC  cannot be opened !");
     exit(1); }
 fscanf(pf,"%d",&n); fscanf(pf,"%le",&x); fscanf(pf,"%s",str);
 printf("\n\n %d  %e  %s", n, x, str);
 fclose(pf);
 getch(); return 0;
}
```

Se poate observa că după fiecare apel al funcţiei de deschidere **fopen(...)** a fişierului se face o verificare privind succesul acestei operaţii .

EXEMPLUL nr. 13.4 . De cele mai multe ori crearea şi apoi utilizarea datelor din fişier nu se face în cadrul aceluiaşi program. În exemplul următor, în cadrul primului program, se crează un fişier cu numele **IXY** care va conţine la început o valoare întreagă egală cu numărul de puncte în care s-au calculat valorile unei funcţii iar pe înregistrările următoare, pentru fiecare punct, numărul de ordine (valoare de tip **int**) şi valorile abscisei **x** şi a ordonatei **y** a funcţiei (două valori de tip **double**) .

S-au tipărit cu caractere bold acele instrucţiuni care efectuează diverse operaţii legate de fişiere.

Programul este următorul :

Programul 1

```
#include <conio.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

double f(double);

int main(void)
{ int i, n; double a, b, h, x, y;
  FILE *pf;
  clrscr();
  printf("\n\n\t a = "); scanf("%le", &a);
  printf("\t b = "); scanf("%le", &b);
  printf("\t h = "); scanf("%le", &h);
  pf=fopen("IXY", "w");
  if(pf==NULL) { printf("\n The file IXY can not be open !");
                 exit(1); }
  n=(b-a)/h+1; fprintf(pf, "%d", n);
  for(i=0, x=a; x<=b; i++, x+=h)
     { y=f(x);
       printf("\n %4d  %10.4f  %14.6e", i, x, y);
       fprintf(pf, "%4d %10.4lf %14.6le", i, x, y);
       if(i%20==0) getch();
     }
  fclose(pf);
  printf("\n\n\n The file  IXY  was created and contains "
         "the coordinates of %d points.\n\n", i);
  getch(); return 0;
}

double f(double x)
{ return sin(x)*sin(5*x+0.125)+1.55; }
```

Ca urmare a execuţiei acestui program se crează fişierul **IXY**.

Cu cel de-al doilea program se face citirea înregistrărilor acestui fişier, afişarea valorilor citite pe ecranul monitorului şi o serie de operaţii cu caracter elementar (determinarea valorii minime şi maxime a funcţiei în intervalul considerat şi calculul mediei aritmetice a valorilor funcţiei) .

Programul este următorul :

Programul 2

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{ int i, n;
  double x, y, y_min, Y_MAX, a;
  FILE *pf; clrscr();
  if((pf=fopen("IXY", "r"))==NULL)
    { printf("\n\n The file  IXY  can not be opened "
             "for reading\n"); getch(); exit(1); }
  fscanf(pf, "%d", &n);
  printf("\n\n\t\ The number of coordinates = %d\n\n", n);
  for(i=0; i<n; i++)
       { fscanf(pf, "%d %lf %le", &i, &x, &y);
         if(i==0) a=y_min=Y_MAX=y;
         if(y_min > y) y_min=y;
         if(Y_MAX < y) Y_MAX=y;
         a+=y;
         printf("\n %4d %10.4lf %14.6le", i, x, y);
         if(i%20==0) getch();
       }
  fclose(pf);
  a/=n;
  printf("\n\n\n  y min = %le  Y_MAX = %le  Average = %le",
         y_min, Y_MAX, a);
  getch(); return 0;
}
```

Şi în cazul celui de-al doilea program s-au tipărit cu caractere bold toate instrucţiunile legate de operaţiile cu fişierul **IXY** .

Se poate observa că la început se face citirea din fişier a valorii numărului **n** de puncte în care s-au calculat valorile funcţiei iar apoi în interiorul unui ciclu parcurs de **n** ori se face citirea câte unei înregistrări, afişarea datelor şi compararea cu valoarea minimă şi maximă curentă, urmată eventual de modificarea acestora .

Funcţiile fwrite şi fread pentru scrierea/citirea unor articole au următoarele prototipuri :

```
size_t fwrite(const void *buffer,size_t size, size_t n_elem,
              FILE  *pf);

size_t fread(void *buffer, size_t size, size_t n_elem, FILE *pf);
```

unde **pf** este pointerul asociat fişierului, **buffer** este un pointer spre zona (tabloul) în care se găsesc elementele care se scriu sau se citesc din fişier, **size** este dimensiunea unui articol, exprimată în octeţi, iar **n_elem** este numărul de articole din cadrul înregistrării .

Funcţia **fwrite(...)** returnează o valoare întreagă, **n/size**, egală cu numărul de articole scrise în fişier, **n** fiind numărul de caractere scrise. În caz de eroare pointerul de fişier nu poate fi determinat.

Funcţia **fread(...)** realizează citirea din fişier şi stochează într-un tablou care are pointerul **buffer** un număr de **n = size μ n_elem** caractere dacă nu se întâlneşte sfârşitul de fişier. Funcţia returnează în acest caz numărul de articole **n_elem** citite complet . Dacă nu s-a putut face citirea a **n_elem** articole ajungându-se la sfârşitul fişierului sau a apărut o eroare atunci pointerul de fişier nu este determinat. Pentru a distinge cele două situaţii (sfârşit de fişier sau eroare) se folosesc funcţiile **feof(...)** şi **ferror(...)** .

Dacă în cazul în care articolele din înregistrare sunt variabile având tipurile uzuale, cu un număr cunoscut de octeţi a înregistrării în memorie, se poate scrie efectiv valoarea numerică (de exemplu **size** este **4** pentru variabile de tip **float**, **size** este **8** pentru cele de tip **double**, etc), în cazul tablourilor sau al structurilor se va folosi operatorul **sizeof(...)**.

Aceste funcţii **fread(...)** şi **fwrite(...)** realizează operaţiile de **I/O, fără formate,** fără a face conversii de date, în cazul fişierelor binare conţinutul fişierului este identic cu imaginea sa din memorie .

Trebuie reţinut de asemenea faptul că în cazul deschiderii unor fişiere binare se va folosi unul din următoarele moduri : **"wb"**, **"rb"**, **"ab"**, **"w+b"** , **"r+b"** sau **"a+b"** .

EXEMPLUL nr. 13.5 . Se va elabora un program în cadrul căruia să se realizeze următoarele: crearea unui fişier cu numele ***FILE_01*** care să conţină elementele (întregi) ale unei matrice de dimensiuni 5 **μ** 5 şi crearea celui de-al doilea fişier ***FILE_02*** care să conţină pe prima înregistrare un singur articol (o valoare de tip **int** - numărul de puncte în care s-au calculat valorile unei funcţii) iar următoarele înregistrări să conţină câte trei articole (de tipul **double**, primul fiind numărul de ordine al punctului considerat, al doilea fiind abscisa iar al treilea valoarea funcţiei).

În cazul fiecăruia din cele două fişiere după crearea acestora (se foloseşte modul de scriere **"wb"**) ele se închid iar apoi se deschid din nou, de data aceasta în modul de citire **"rb"**.

Programul se prezintă în continuare, instrucţiunile care conţin funcţii în legătură cu operaţiile cu fişiere fiind scrise cu caractere bold :

```
#include <conio.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

double func(double x);

int main(void)
{ int i, j, MF[5][5], m, n;
  int MI[5][5]={ {11, 12, 13, 14, 15}, {21, 22, 23, 24, 25},
                 {31, 32, 33, 34, 35}, {41, 42, 43, 44, 45},
                 {51, 52, 53, 54, 55} };
  double a, b, h, x, y, U[3], V[3];
  FILE *pf1, *pf2;
  clrscr();
//
//  Fisierul binar FILE_01 se deschide in mod scriere,
//           se scrie in el si se inchide
//  Fisierul va contine o inregistrare cu 25 articole,
//  valori de tip int  (elementele unei matrice  5 x 5)
```

```
//
  pf1=fopen("FILE_01", "wb");
  if(pf1==NULL) { printf("\n\n The file  FILE_01  can not"
                         " be opened\n"); getch(); exit(1); }
  fwrite(MI, 4, 25, pf1);
  fclose(pf1);
//
// Fisierul binar FILE_01 se deschide în vederea citirii,
//             se citeste si se inchide
//
  pf1=fopen("FILE_01", "rb");
  if(pf1==NULL) { printf("\n\n The file  FILE_01  can not"
                         " be opened\n"); getch(); exit(1); }
  fread(MF, 4, 25, pf1);
  fclose(pf1);
  for(i=0; i<5; i++)
     { printf("\n");
       for(j=0; j<5; j++) printf("%5d", MF[i][j]);
     }
//
//  Fisierul binar FILE_02 se deschide in mod scriere,
//           se scrie in el si se inchide
//
  if((pf2=fopen("FILE_02", "wb"))==NULL)
    { printf("\n\n The file  FILE_02  can not be opened\n");
      getch(); exit(1); }
  printf("\n\n\t a = "); scanf("%le", &a);
  printf("\t b = "); scanf("%le", &b);
  printf("\t h = "); scanf("%le", &h);
  n=(b-a)/h+1;
  fwrite(&n, 4, 1, pf2);
  for(i=0; i<n; i++)
     { U[0]=(double)i; U[1]=x=a+i*h; U[2]=y=func(x);
       printf("\n %4d %10.4lf %14.6le", i, x, y);
       if(i%20==0) getch();
       fwrite(U, 8, 3, pf2);
     }
  fclose(pf2);
//
//  Fisierul binar FILE_02 se deschide in vederea citirii,
//              se citeste si se inchide
//
  if((pf2=fopen("FILE_02", "rb"))==NULL)
    { printf("\n\n The file  FILE_02  can not be opened\n");
      getch(); exit(1); }
  fread(&m, 4, 1, pf2);
  printf("\n");
  for(i=0; i<m; i++)
     { fread(V, 8, 3, pf2);
       printf("\n\t %4d %10.4lf %14.6le",
              (int)V[0], V[1], V[2]);
       if(i%20==0) getch();
```

```
    }
  fclose(pf2);
//
  getch(); return 0;
}

double func(double x)
{return (log(x*x+1)+2.34*sqrt(pow(x, 4)+20))/(2-sin(x/25.5));}
```

Succesiunea fazelor de deschidere, creare, închidere a celor două fişiere este precizată prin comentariile inserate în programul sursă.

Crearea primului fişier este realizată ca urmare a apelului următoarei funcţii :

```
fwrite(MI, 4, 25, pf1);
```

unde **MI** este identificatorul (pointerul) matricei care se introduce în fişier, **4** reprezintă numărul de octeţi ocupaţi de un element al matricei, **25** este numărul de elemente ale matricei , iar **pf1** pointerul asociat fişierului ***FILE_01.***

În cazul celui de-al doilea fişier s-a folosit funcţia :

```
fwrite(&n, 4, 1, pf2);
```

pentru a scrie în fişier valoarea unei variabile întregi **n** , la adresa **&n** , **4** şi **1** reprezentând numărul de octeţi pe care se reprezintă articolul , respectiv numărul de articole, aici fiind vorba de o singură variabilă întreagă . Cel de-al patrulea parametru este pointerul **pf2** , asociat celui de-al doilea fişier.

Prin folosirea repetată (în interiorul unui ciclu programat cu **for**) a funcţiei :

```
fwrite(U, 8, 3, pf2);
```

se realizează scrierea unui număr de **n** înregistrări, fiecare conţinând câte un tablou unidimensional **U** cu elemente de tip **double**, care se înregistrează fiecare pe **8** octeţi, numărul acestor elemente fiind **3** .

În cazul citirii datelor din aceste fişiere s-au folosit funcţiile :

```
fread(MF, 4, 25, pf1);
fread(&m, 4, 1, pf2);
fread(V, 8, 3, pf2);
```

observându-se coincidenţa parametrilor, cu excepţia primului, care diferă numai prin notaţie , cu cei folosiţi la funcţiile **fwrite(...)** .

Ca urmare a execuţiei acestui program, se obţine :

```
11   12   13   14   15
21   22   23   24   25
31   32   33   34   35
41   42   43   44   45
51   52   53   54   55

    a = 1
    b = 2
    h = 0.25
```

```
0     1.0000   5.822321e+00
1     1.2500   6.164078e+00
2     1.5000   6.641872e+00
3     1.7500   7.292610e+00
4     2.0000   8.143755e+00

 0     1.0000   5.822321e+00
 1     1.2500   6.164078e+00
 2     1.5000   6.641872e+00
 3     1.7500   7.292610e+00
 4     2.0000   8.143755e+00
```

În afară de funcţiile anterior prezentate şi utilizate în exemplele de programe o folosire frecventă o mai au şi următoarele:

`int ferror(FILE *pf);` - verifică dacă există vreo eroare la citire sau scriere în fişierul cu pointerul **pf.** Se returnează valoarea **0** dacă nu este nici o eroare, respectiv o valoare diferită de zero dacă există o eroare .

`int fseek(FILE *pf, long offset, int origine);` - realizează mutarea pointerului în fişier la un număr de **offset** octeţi de la **origine** . Argumentul **origine** poate fi una din următoarele constante : **SEEK_SET** (se face poziţionarea după **offset** octeţi de la începutul fişierului) , **SEEK_CUR** (se face poziţionarea după **offset** octeţi începând cu poziţia curentă) , **SEEK_END** (se face poziţionarea după **offset** octeţi după sfârşitul fişierului). Funcţia **fseek(...)** şterge indicatorul de sfârşit de fişier. Valoarea returnată este **0** în cazul în care poziţionarea în fişier s-a putut face sau o valoare diferită de zero în caz contrar .

`long ftell(FILE *pf);` - funcţia returnează o valoare care defineşte poziţia curentă în fişier iar în caz de eroare **-1L** .

În exemplele anterior prezentate s-au considerat fişiere care conţineau caractere sau şiruri de caractere, matrice, tablouri unidimensionale, etc.

Este evident că pentru a face operaţii cu aceste tipuri de date nu era neapărat necesară utilizarea fişierelor.

Sunt însă o serie de probleme, cum ar fi cele privind alcătuirea unei **liste bibliografice** (pentru fiecare carte trebuie precizat autorul, titlul, editura, anul apariţiei, numărul de pagini, preţul, ISBN-ul, o listă de cuvinte cheie, etc), a unei **liste cu studenţii** dintr-un an de studiu (nume, prenume, data naşterii, prenumele părinţilor, adresa permanentă, adresa actuală, număr matricol, an de studiu, numărul grupei, situaţia la examenele din anii universitari anteriori şi din cel curent , etc), a unei **liste cu angajaţii** unei firme (cu multe elemente de identificare), etc. pentru realizarea cărora se impune folosirea fişierelor, cu toate operaţiile care se pot face cu acestea : deschidere, creare/actualizate, citire de date şi prelucrarea acestor date, închiderea fişierului, ştergere, etc.

Reuniunea unor astfel de informaţii de diverse tipuri se face în cadrul unor **structuri**, care se vor putea scrie în fişiere, citi şi prelucra.

EXEMPLUL nr. 13.6 . Se pune problema creerii unui fişier care să conţină o **listă de cărţi**, de exemplu din domeniul programării cu limbajul **C**, pentru fiecare carte fiind precizate următoarele informaţii : ***numele autorului/autorilor*** (şir de caractere), ***titlul*** (şir de caractere), ***editura*** (şir de caractere), ***anul editării*** (un număr întreg cu patru cifre), ***numărul de pagini*** (un

număr intreg), ***preţul*** (număr real – dacă se dă în USD, număr întreg dacă s-ar da în lei). Evident că s-ar mai putea adăuga şi alte elemente – oraşul în care este editura, ISBN-ul, o listă de cuvinte cheie, numărul ediţiei, cuprinsul.

Aceste date se vor organiza sub forma unei structuri cu numele **book ,** definită la începutul programului.

Se va crea un fişier binar **BIBL_02** la prima execuţie a programului, introducând în acesta un număr de **n** articole (structuri) iar la fiecare reutilizare a acestui program se mai poate face adăugarea unor articole, deoarece la deschiderea fişierului s-a folosit modul **"ab"**, ceea ce înseamnă că noile date (articole) se adaugă la sfârşitul fişierului binar.

Pentru a urmări mai uşor programul s-au tipărit cu caractere bold acele instrucţiuni cu care se fac operaţii cu fişierul şi cu elementele structurii.

În cadrul unui ciclu, parcurs de **n** ori, se face introducerea datelor, elementelor fiecărei structuri , scrise sub forma :

`book.nume_elem`

atribuindu-li-se valorile corespunzătoare. Se poate observa că s-a folosit operatorul punct (selectorul direct de componente)

După crearea fişierului (sau adăugarea de articole) acesta se închide.

În cea de-a doua parte a programului se face deschiderea fişierului **BIBL_02**, în mod citire şi se cere afişarea tuturor cărţilor care au fost tipărite în anul YEAR, a cărui valoare se introduce de la tastatură.

Utilizându-se instrucţiunea **while** în care se verifică dacă s-a ajuns la sfârşitul fişierului, cu ajutorul funcţiei **feof(...)** , se selectează cărţile care îndeplinesc această condiţie şi se face tipărirea pentru fiecare din aceste cărţi a tuturor elementelor structurii.

Se face apoi o nouă parcurgere a fişierului şi se afişează numele autorilor tuturor cărţilor din fişier, în ordinea în care se întâlnesc . Reluarea parcurgerii fişierului de la începutul acestuia se face ca urmare a utilizării funcţiei **fseek(...)** cu parametrii ***offset*** = **0** şi ***origine*** = **SEEK_SET .**

Programul este prezentat în continuare :

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

  struct { char  author[40];
           char  title[65];
           char  publ_house[35];
           int   year;
           int   n_pages;
           float price_USD;
         } book ;

int main(void)
{ int i, k=0, n, YEAR;
  FILE *pf; clrscr();
//
  if((pf=fopen("BIBL_02", "ab"))==NULL)
    { printf("\n\n The file  BIBL_02  can not be opened\n");
      getch(); exit(1); }
```

```
  printf("\n The number of titles to be added in file : ");
  scanf("%d", &n);
  for(i=0; i<n; i++)
   {  printf("\n Book number %d\n\n Author [max. 40 char] : ",
              i+1);
      scanf("%s", book.author);
      printf(" Title  [max. 65 char] : ");
      scanf("%s", book.title);
      printf("\t Publishing house [max. 35 char] : ");
      scanf("%s", book.publ_house);
      printf("\t Year [4 digits] : ");
      scanf("%4d", &book.year);
      printf("\t Number of pages : ");
      scanf("%d", &book.n_pages);
      printf("\t Price [USD]     : ");
      scanf("%f", &book.price_USD);
      fwrite(&book, sizeof(book), 1, pf);
    }
//   Fisierul  BIBL_02  cu  n  titluri a fost creat
  fclose(pf);
//      Fisierul  BIBL_02  a fost inchis
  if((pf=fopen("BIBL_02", "rb"))==NULL)
    { printf("\n\n The file BIBL_02 can not be opened\n");
      getch(); exit(1); }
//   Fisierul  BIBL_02  a fost deschis
  printf("\n\t Which is the year do you take interest ?  ");
  scanf("%d", &YEAR);
//
  while( !feof(pf) )
     { fread(&book, sizeof(book), 1, pf);
       if((book.year==YEAR)&&(!feof(pf)))
      { if(k==0)
          printf("\n\n\t The books that have been appeared"
               " in %d are :", YEAR);
        printf("\n %s\n\t %s\n\t\t%s\n\t\t%d\n\t\t%d"
             " pages\n\t\t(price - %6.2f USD)",
               book.author, book.title,   book.publ_house,
               book.year,   book.n_pages, book.price_USD  );
        getch(); k++;
      }
     }
//
  if(k!=0)
       printf("\n\n\t No other books !\n\n");
  else
       printf("\n\t The file does not contain books appeared "
              "in %4d\n", YEAR);
  getch();
//
  fseek(pf, 0,SEEK_SET);
  printf("\n\n\t The list of authors :\n\n");
```

```
  k=0;
  while( !feof(pf))
    { fread(&book, sizeof(book), 1, pf);
      k++;
      if(!feof(pf))
         printf("\n %4d %s", k, book.author);
    }
  fclose(pf);
//remove("BIBL_02");
  getch(); return 0;
}
```

În cazul în care se doreşte ştergerea fişierului înaintea execuţiei programului se va renunţa la slash-urile existente înaintea funcţiei **remove(...)**.

Utilizând programe asemănătoare cu acesta se pot soluționa practic orice probleme legate de fişiere conţinând structuri .

Capitolul XIV

Programarea în mod text

Programele anterior prezentate realizau afişarea datelor în mod secvenţial, utilizatorul reuşind prin folosirea formatelor dispunerea acestora pe linia curentă ce se afişa şi pe liniile următoare, nemaiavând acces la liniile anterioare. De asemenea, culoarea fondului precum şi a caracterelor ce se afişau nu era posibil să fie gestionată prin program.

Pentru a putea afişa informaţii în orice punct al ecranului monitorului, cu folosirea unei culori de fond şi a unei culori pentru caractere, pentru a putea efectua ştergerea unor porţiuni de ecran pe care s-au făcut afişări s-au introdus în limbajul de programare C/C++ o serie de funcţii pentru gestiunea ecranului în ***mod text*** .

În cazul utilizării unui mediu de programare al firmei ***Borland*** toate funcţiile care sunt necesare pentru a lucra în modul text se găsesc în fişierul (***nestandardizat*** conform ANSI C) **conio.h .**

Unele din funcţiile aparţinând acestui fişier au fost deja folosite, după cum s-a putut vedea includerea acestuia prin directiva

```
#include <conio.h>
```

fiind întâlnită aproape la toate programele anterioare, în acestea existând funcţiile :

clrscr(), **getch()**, getche(), putch(...) .

cele mai des utilizate fiind primele două .

Cele **38** de funcţii aparţinând fişierului **conio.h** sunt următoarele (au fost enumerate şi în Capitolul IV) :

cgets	clreol	**clrscr**	**cprintf**	cputs
cscanf	delline	**getch**	getche	getpass
gettext	gettextinfo	**gotoxy**	highvideo	insline
inp	inport	inportb	inpw	kbhit
lowvideo	movetext	normvideo	outp	outport
outportb	outpw	**putch**	puttext	
_setcursortype		**textattr**	**textbackground**	
textcolor	**textmode**	ungetch	wherex	wherey
window				

dintre acestea cele mai frecvent utilizate (în cadrul acestui capitol) au fost scrise cu bold.

14.1 Setarea modului text. Ferestre. Culori.

Ecranul monitorului are 25 de linii (în absenţa unei setări care modifică acest număr) pe care se pot afişa câte 80 de caractere, astfel încât pentru a poziţiona un caracter pe ecran este nevoie de două ***"coordonate"*** , indicele de coloană, cu valori de la 1 la 80 şi indicele de linie, cu valori de la 1 la 25 (sau de la 1 la 50) .

Caracterele afişate în colţurile ecranului au coordonatele : stânga sus – 1 şi 1 , dreapta sus - 80 şi 1 , stânga jos – 1 şi 25 (sau 50) , dreapta jos – 80 şi 25 (sau 50).

Pentru a lucra cu ecranul în mod text se foloseşte funcţia **textmode** a cărei prototip este următorul :

```
void textmode(int mod);
```

unde **mod** poate avea valorile :

- **0** sau **1** dacă se lucrează numai cu primele 40 de coloane ale ecranului, în modul alb/negru respectiv color,
- **2** sau **3** dacă se lucrează cu toate cele 80 de coloane, în modul alb/negru respectiv color,
- **64** dacă dispunem de o placa VGA sau SVGA (se va lucra cu 50 de linii) ,

alte valori posibile fiind 7 şi -1 .

Este evident că în cazul când se foloseşte **mod=64** atunci coordonatele caracterelor de pe ultima linie sunt : stânga jos – 1 şi 50, dreapta jos – 80 şi 50 .

În prezent, dispunând de monitoare color, funcţia **textmode** se va folosi numai în cazul când se doreşte să se seteze un număr de 50 de linii pentru ecranul monitorului,

```
textmode(64);
```

în absenţa acestei specificaţii considerându-se în mod implicit dimensiunile 80×25 .

Pentru a afişa un caracter pe ecran, în afară de coordonatele acestuia trebuie să se mai definească culoarea fondului şi culoarea caracterului afişat. Aceste informaţii sunt precizate ca urmare a folosirii funcţiilor: **textbackground** şi **textcolor** sau a funcţiei **textattr**, care au următoarele prototipuri:

```
void textbackground( int culoare );
void textcolor( int culoare );
void textattr( int atribut );
```

În cazul primelor două funcţii culoarea se alege conform indicaţiilor din tabelul 14.1, fiind disponibile un număr de ***opt culori*** pentru ***fond*** şi un număr de ***16 culori*** pentru ***caractere***, mai existând şi posibilitatea de a realiza clipirea caracterelor afişate .

Cea de-a treia funcţie, **textattr**, permite setarea simultană a acestor caracteristici, valoarea argumentului **atribut** stabilindu-se conform relaţiei :

```
atribut = 16 * culoare_fond + culoare_caracter + clipire ;
```

unde pentru ***clipire*** se ia valoarea **128** (cu clipire) sau **0** (fără clipire) .

Pentru a afişa caractere de culoare ***roşu deschis*** pe un fond ***gri deschis***, fără clipire, se pot folosi următoarele funcţii :

Tabelul 14.1

Culoarea		Valoarea	Mod de utilizare	
Constanta simbolică	Denumirea în l. română	**numerică**	**Fond**	**Text**
BLACK	negru	**0**	**Da**	**Da**
BLUE	albastru	**1**	**Da**	**Da**
GREEN	verde	**2**	**Da**	**Da**
CYAN	turcoaz	**3**	**Da**	**Da**
RED	roşu	**4**	**Da**	**Da**
MAGENTA	purpuriu	**5**	**Da**	**Da**
BROWN	maro	**6**	**Da**	**Da**
LIGHTGRAY	gri deschis	**7**	**Da**	**Da**
DARKGRAY	gri închis	**8**	Nu	**Da**
LIGHTBLUE	albastru deschis	**9**	Nu	**Da**
LIGHTGREEN	verde deschis	**10**	Nu	**Da**
LIGHTCYAN	turcoaz deschis	**11**	Nu	**Da**
LIGHTRED	roşu deschis	**12**	Nu	**Da**
LIGHTMAGENTA	purpuriu deschis	**13**	Nu	**Da**
YELLOW	galben	**14**	Nu	**Da**
WHITE	alb	**15**	Nu	**Da**
BLINK	clipire	**128**	Nu	**Da**

```
textbackground(LIGHTGRAY);      sau      textbackground(7);
textcolor(LIGHTRED);                     textcolor(12);
```

sau mai simplu

```
textattr(124);  // 16*7 + 12 = 124
```

valorile numerice ce corespund culorilor fiind luate din tabelul 14.1 .

În cele prezentate anterior s-a făcut referire la întreg ecranul monitorului, cu 80 de coloane şi 25 (50) de linii .

Există posibilitatea de a ne referi numai la o zonă dreptunghiulară a ecranului, numită ***"fereastră"*** poziţia căreia este definită prin coordonatele (absolute) ale caracterelor aflate în stânga sus respectiv în dreapta jos .

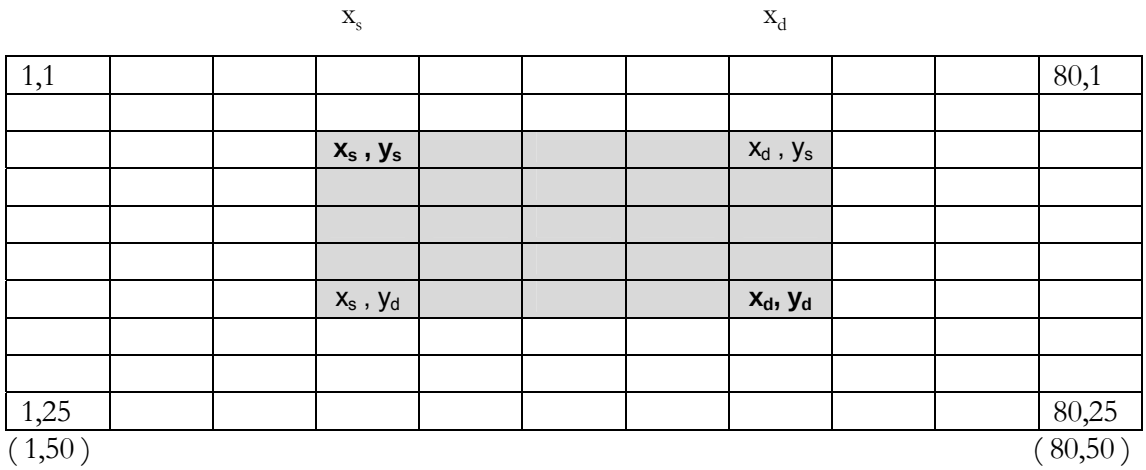
Declararea unei ferestre se face cu funcţia **window**, a cărei prototip este :

```
void window(int x_s_sus, int y_s_sus, int x_d_jos, int y_d_jos);
```

iar după folosirea în program a acestei funcţii

```
window(xs, ys, xj, yj);
```

devine ***"activă"*** numai porţiunea de ecran specificată, toate funcţiile de gestiune a ecranului în mod text folosite referindu-se numai la fereastra activă .



Trebuie precizat că la un moment dat este activă numai o singură fereastră, ultima declarată. În absenţa unei astfel de declaraţii este considerat ca fereastră tot ecranul monitorului .

Ştergerea unei ferestre se face cu funcţia **clrscr** (**cl**ea**r scr**een) având prototipul :

```
void clrscr(void) ;
```

folosită frecvent şi în programele anterioare, a cărei efect este că fereastra anterior declarată (sau întreg ecranul) devine vidă iar cursorul se mută în punctul de coordonate (1,1) .

14.2 Afişarea şi manipularea textelor pe ecran .

Următoarele funcţii, pentru care în continuare se prezintă prototipul şi o scurtă descriere, sunt mai frecvent utilizate:

```
void gotoxy(int col, int lin);
     // Pozitioneaza cursorul in coloana col si linia lin.
     // Coordonatele sunt relative la fereastra activa.

int wherex(void);
     // Returneaza numarul coloanei in care se afla cursorul.

int wherey(void);
     // Returneaza numarul liniei in care se afla cursorul.

int putch(int ch);
     // Afiseaza caracterul ch in pozitia curenta.
     // Returneaza caracterul iar in caz de eroare EOF.

int cputs(const char *str);
     // Afiseaza sirul str, incepand din pozitia curenta.
     // Returneaza ultimul caracter afisat. Nu adauga in mod
     // automat caracterul \n , asa cum se intampla in cazul
     // folosirii functiei  puts.

int cprintf(const char *format [ , argument, ... ]);
     // Afiseaza date sub controlul formatelor.
     // Nu realizeaza transformarea caracterului \n  in \n\r.
     // Returneaza numarul de caractere afisate.
```

```
int movetext(int xs_sursa, int ys_sursa, int xd_sursa,
             int yd_sursa, int xs_dest,  int ys_dest);
     // Copiaza un text existent intr-o zona definita de
     // primele patru coordonate intr-o alta zona de
     // aceeasi marime pentru care se dau coordonatele
     // caracterului din stanga sus.
     // Coordonatele sunt relativ la ecranul intreg.
     // Returneaza 1 in caz de succes, respectiv 0 in caz
     // de eroare.

void delline(void);
     // Sterge linia pe care este pozitionat cursorul.

void insline(void);
     // Inserarea unei linii (goale) la pozitia cursorului.
```

În continuare se vor prezenta o serie de programe care utilizează funcţiile corespunzătoare modului text.

EXEMPLUL nr. 14.1. Unul din cele mai simple programe este cel pentru desenarea drapelelor unor ţări, cu condiţia ca în componenţa drapelului să se găsească numai culorile admise pentru fond, conform celor prezentate în tabelul 14.1.

Cu programul următor se afişează drapelele ***Franţei*** (trei culori dispuse pe orizontală), ***Olandei*** (trei culori dispuse pe verticală), principatului ***Monaco*** (două culori dispuse pe verticală), ***Finlandei*** (o cruce albastră pe fond alb) şi ***Norvegiei*** (pe fond roşu o cruce albă peste care este suprapusă o cruce albastră), în mod similar desenându-se şi drapelele altor ţări.

În cazul fiecărui drapel se fac de mai multe ori următoarele operaţii : declararea unei ferestre, cu funcţia ***window(...)***, corespunzătoare dreptunghiului care se va colora, precizarea culorii de fond cu funcţia ***textbackground(...)***, vidarea ferestrei pentru ca să se obţină colorarea cu culoarea de fond, folosind funcţia ***clrscr()***, urmată de o pauză datorată includerii funcţiei ***delay(...)***. După ce s-au desenat dreptunghiurile colorate se va scrie un text (denumirea ţării căreia îi aparţine drapelul) într-o fereastră cu culoarea de fond **CYAN** (turcoaz), textul fiind scris cu **YELLOW** (galben)

În continuare se prezintă acest program, care conţine comentarii privind culorile utilizate .

```
//  FRANCE   NEDERLAND   MONACO   FINLAND   NORGE

#include <conio.h>
#include <dos.h>

int main(void)
{ int BE=BLACK, B1, B2, B3, d=250, T=YELLOW;
  textcolor(T);

window( 1, 1,80,25);
textbackground(BE); clrscr(); delay(d);
window(11, 5,30,20);
B1=BLUE; textbackground(B1); clrscr(); delay(d);
window(31, 5,50,20);
B2=LIGHTGRAY; textbackground(B2); clrscr(); delay(d);
window(51, 5,70,20);
B3=RED; textbackground(B3); clrscr(); delay(2*d);
window(33,22,50,22);
textbackground(CYAN); cputs("  F R A N C E  "); getch();
```

```
     // S-a desenat steagul Frantei.
     // De la stanga la dreapta : albastru - alb - rosu

clrscr();
window( 1, 1,80,25);
textbackground(BE); clrscr(); delay(d);
window(15, 6,65,10);
B1=RED; textbackground(B1); clrscr(); delay(d);
window(15,11,65,15);
B2=LIGHTGRAY; textbackground(B2); clrscr(); delay(d);
window(15,16,65,20);
B3=BLUE; textbackground(B3); clrscr(); delay(2*d);
window(35,22,50,22);
textbackground(CYAN); cputs("  NEDERLAND  "); getch();
     // S-a desenat steagul regatului Olandei.
     // De sus in jos : rosu - alb - albastru

clrscr();
window( 1, 1,80,25);
textbackground(BE); clrscr(); delay(d);
window(15, 5,65,12);
B1=RED; textbackground(B1); clrscr(); delay(d);
window(15,13,65,20);
B2=LIGHTGRAY; textbackground(B2); clrscr(); delay(2*d);
window(35,22,50,22);
textbackground(CYAN); cputs("  MONACO  "); getch();
     // S-a desenat steagul Principatului Monaco.
     // De sus in jos : rosu - alb

clrscr();
window( 1, 1,80,25);
textbackground(BE); clrscr(); delay(d);
window(15, 6,65,20);
B1=LIGHTGRAY; textbackground(B1); clrscr(); delay(d);
window(15,12,65,14);
B2=BLUE; textbackground(B2); clrscr(); delay(d);
window(30, 6,36,20);
B3=BLUE; textbackground(B3); clrscr(); delay(2*d);
window(30,22,50,22);
textbackground(CYAN); cputs("  SUOMI - FINLAND  "); getch();
     // S-a desenat steagul Finlandei.
     // Pe fond alb o cruce albastra

clrscr();
window( 1, 1,80,25);
textbackground(BE); clrscr(); delay(d);
window(15, 6,65,20);
B1=RED; textbackground(B1); clrscr(); delay(d);
window(15,12,65,14);
B2=LIGHTGRAY; textbackground(B2); clrscr(); delay(d);
window(30, 6,36,20);
B3=LIGHTGRAY; textbackground(B3); clrscr(); delay(d);
window(15,13,65,13);
```

```
B2=BLUE; textbackground(B2); clrscr(); delay(d);
window(32, 6,34,20);
B3=BLUE; textbackground(B3); clrscr(); delay(2*d);
window(35,22,50,22);
textbackground(CYAN); cputs("  N O R G E  "); getch();
     // S-a desenat steagul regatului Norvegiei.
     // Pe fond rosu o cruce alba peste care este
     // suprapusa o cruce albastra

return 0;
}
```

Observaţia nr. 1. După afişarea unui drapel trebuie apăsată o tastă pentru a trece la afişarea următorului.

Observaţia nr. 2. Folosind secvenţa de instrucţiuni pentru desenarea drapelului francez, modificând culorile, se pot desena drapelele Irlandei, Italiei, Guatemalei, Nigeriei, Peru ş. a. iar modificând culorile din secvenţa de instrucţiuni pentru desenarea drapelului olandez se pot obţine drapelele următoarelor state: Rusia, Austria, Bulgaria, Estonia, El Salvador, Sierra Leone, Ungaria, Yemen ş. a.

EXEMPLUL nr. 14.2. În exemplul anterior s-au folosit următoarele funcţii corespunzătoare modului text: ***window(...)*** , ***clrscr( )***, ***textcolor(...)*** , ***textbackground(...)*** şi ***cputs(...)***. În următorul program se vor mai folosi şi funcţiile ***gotoxy(...)*** , ***cprintf(...)*** şi ***movetext(...)***.

Programul realizează afişarea unor texte în ferestre, modificându-se atât culoarea fondului cât şi culoarea caracterelor, conform întregii palete disponibile de culori. La început se afişează textul "Programming in C/C++" pentru toate combinaţiile posibile de culori de fond şi ale caracterelor . În continuare, în trei ferestre care sunt deschise pe rând se afişează în mod repetat câte un şir de caractere iar apoi ferestrele sunt vidate . În final se face afişarea unor texte pe ecran (fereastra este tot ecranul) care apoi se copiază într-o altă parte a ecranului.

Programul este următorul :

```
#include <conio.h>
#include <dos.h>
#include <stdio.h>

int main(void)
{int B, BE, i, T;

textbackground(RED); clrscr();
     // Tot ecranul a devenit o fereastra de culoare RED

window(10, 5,35,20);
for(B=0; B<8; B++)
  {textbackground(B);
   for(T=0; T<16; T++)
     {textcolor(T);cprintf("\n\r Programming in C/C++");
      delay(25);}
  }
     // S-a definit o fereastra a carei culoare este
     // modificata in ciclul exterior iar in interiorul
     // corpului ciclului interior se afiseaza acelasi
     // text cu toate culorile posibile.
```

```
     // Se observa ca s-au folosit caracterele  \n  si \r
     // pentru ca textul ce se afiseaza sa inceapa in
     // aceeasi coloana, pe randuri succesive.

textbackground(RED); clrscr(); getch();
     // Tot ecranul a devenit o fereastra de culoare RED

B=LIGHTGRAY; window( 5, 2,30,10); clrscr();
textbackground(B); textcolor(BLUE); getch();
for(i=0; i<55; i++) {cprintf("Programming "); delay(25);}
     // S-a definit o fereastra activa in coltul din
     // stanga sus, culoarea fondului fiind LIGHTGRAY
     // iar culoarea de afisare a textului BLUE (albastru).
     // S-a afisat de 55 de ori cuvantul "Programming ",
     // unul dupa altul, deoarece lipsesc \n si \r ,
     // dupa completarea unui rand trecandu-se la urmatorul.

window(25, 8,50,16); clrscr(); getch();
textbackground(B); textcolor(YELLOW);
for(i=0; i<80; i++) {cprintf("in "); delay(25);}
     // S-a definit o fereastra activa in mijlocul
     // ecranului, culoarea fondului fiind LIGHTGRAY
     // iar culoarea de afisare a textului YELLOW (galben).
     // S-a afisat de 80 de ori cuvantul "in ", unul dupa
     // altul, deoarece lipsesc \n si \r .

window(45,14,70,22); clrscr(); getch();
textbackground(B); textcolor(RED);
for(i=0; i<70; i++) {cprintf("C/C++ "); delay(25);}
     // S-a definit o fereastra activa in coltul din
     // dreapta jos, culoarea fondului fiind LIGHTGRAY
     // iar culoarea de afisare a textului RED (rosu).
     // S-a afisat de 70 de ori cuvantul "C/C++ ",
     // unul dupa altul, deoarece lipsesc \n si \r .

textbackground(RED); getch(); clrscr();
     // Ultima fereastra (fereastra activa) a devenit de
     // culoare RED. Fereastra din dreapta jos a disparut.
textbackground(RED); getch(); window(25, 8,50,16); clrscr();
     // A disparut fereastra din mijloc.
textbackground(RED); getch(); window( 5, 2,30,10); clrscr();
     // A disparut fereastra din stanga sus.
getch();

window( 1, 1,80,25); textbackground(B); textcolor(RED); clrscr();
     // Tot ecranul a devenit de culoare LIGHTGRAY.

gotoxy( 1, 1);
cputs("   \"Hominis mens discendo alitur et cogitando\"\n\r");
cputs("                - Proverb latin - \n\r");
cputs("(Mintea omului se cultiva prin invatatura si gandire)");
     // Incepand din pozitia 1,1 se face afisarea a trei
     // randuri de text.
```

```
gotoxy(1,5);
cputs("\"Knowledge is power\" - Francis Bacon");
     // Incepand din pozitia 1,5 se afiseaza un text.

getch();

movetext( 1, 1, 60,6, 15, 15);
     // S-a copiat continutul zonei din ecran determinata
     // de 1,1 si 60,6 intr-o alta zona a ecranului activ
     // care are coltul din stanga sus de coordonate  15,15.

getch(); return 0;
}
```

Programul conţine un număr mare de comentarii care explică toate etapele ce se parcurg .

EXEMPLUL nr. 14.3 . Cunoscând valorile returnate ca urmare a apăsării unor taste, de exemplu a tastelor săgeţi : → (77), ↑ (72), ← (75), ↓ (80) precum şi a tastei **ESC** (27) se poate elabora un program pentru deplasarea pe ecran a unui caracter.

Caracterul ce se dorește a fi deplasat se plasează pe mijlocul ecranului iar prin apăsarea tastelor săgeţi, prin intermediul instrucţiunii **switch**, se modifică valoarea variabilelor care definesc poziţia acestuia pe ecran. Înaintea de a face deplasarea caracterului în noua poziţie, pe vechea poziţie a acestuia se afişează un caracter blank, efectuându-se prin aceasta ştergerea caracterului.

Ca şi caracter care se va deplasa s-a ales patratul plin ■ care are codificarea ASCII 254. În cadrul programului care urmează s-au precizat prin comentarii toate etapele ce se parcurg precum şi explicaţiile necesare :

```
#include <conio.h>
#include <stdio.h>

int main()
{ int line=20, column=40, c, c1, c2;
  c1=254;               // Caracter ASCII - patrat plin
  c2=32;                // Caracter ASCII - blank
  textmode(64);         // Ecranul are 50 de linii
  textbackground(7);    // Culoarea de fond - LIGHTGRAY
  textcolor(4);         // Culoarea caracterelor - RED
  clrscr();
  printf("\n Press repeatedly the arrow keys."
         "\n Press ESC and ENTER to finish !\n");
  gotoxy(column, line); putch(c1);
     // Caracterul  c1  este plasat pe coloana
     // column si pe linia line

     //  --> 77 = 4D (H)    <-- 75 = 4B (H)
     //  ^
     //  |   72 = 48 (H)    |   80 = 50 (H)
     //                     V
     // ESC  27 = 1B (H)
```

```
  do
    { c=getch();
      // Se apasa una din tastele sageti sau ESC.
      // Apasarea unei alte taste nu are nici un efect.

      gotoxy(column, line); putch(c2);
      // Se sterge caracterul afisat anterior

      switch(c)
           { case 77:  if(column<80) column++; break;
             case 72:  if(line  > 1) line--;   break;
             case 75:  if(column> 1) column--; break;
             case 80:  if(line  <50) line++;   break;
           }
      // In functie de tasta sageata apasata se modifica
      // cu o unitate valoarea lui column sau line.
      // In cazul apasarii unei alte taste instructiunea
      // switch nu va functiona iar column si line vor
      // ramane cu valori nemodificate.

      gotoxy(column, line); putch(c1);
      // Se afiseaza caracterul intr-o noua pozitie,
      // corespunzator tastei apasate.

      gotoxy(68,2); cprintf("Column = %2d", column);
      gotoxy(68,3); cprintf("Line   = %2d", line);
      // Se afiseaza in coltul din dreapta sus numarul
      // coloanei si al liniei unde se gaseste caracterul.
    }
  while(c != 27) ;
      // Se reia executia instructiunii compuse daca nu
      // s-a apasat tasta ESC.

  getch(); return 0;
}
```

EXEMPLUL nr. 14.4 . O problemă interesantă şi cu multiple utilizări este cea a realizării deplasării unui text pe o linie a ecranului de la stânga la dreapta sau invers. Programul următor realizează acest lucru , permiţând introducerea unui text de la tastatură şi alegerea sensului în care se va face deplasarea acestuia.

```
#include <conio.h>
#include <dos.h>
#include <stdio.h>
#include <string.h>

int main(void)
{ int i, j, n; char c, S[250], SC[81];
  textbackground(7); textcolor(BLUE); clrscr();
  printf("\n\t Please keyboard the string.\n");
  gets(SC);
      // Se tasteaza un sir de caractere notat cu SC, pe o linie.
```

```
  printf("\n The direction of displacement :");
  printf("\n\t\t From left to RIGHT  ---->   r, R");
  printf("\n\t\t From right to LEFT  <----   l, L\n\n");
  c='a';
  while(c!='R'&&c!='r'&&c!='L'&&c!='l')
     { printf("\n\n Your choice is - "); c=getche();}
     // Se apasa R sau r  respectiv L sau l.
     // Daca se apasa o alta tasta se reia operatia.

  getch(); clrscr();
  n=strlen(SC);
     // Se determina numarul de caractere al sirului SC.

  window( 1,10,80,15);
  textbackground(7); textcolor(RED);
     // Se defineste o fereastra, la care culoarea de fond este
     // LIGHTGRAY (gri deschis) iar cea cu care se vor afisa
     // caracterele RED (rosie).

  for(i=0; i<80; i++) {S[i]=S[n+i+80]=' ';}
  for(i=0; i<n; i++) S[i+80]=SC[i];
     // S-a format un sir S la care primele 80 de caractere si
     // ultimele 80 de caractere sunt blank-uri iar intre ele se
     // gasesc caracterele din compunerea sirului SC.

  if(c=='L'||c=='l')
     for(i=0; i<n+81; i++)
     { clrscr();
       for(j=0; j<80; j++) putch(S[i+j]);
       delay(200);
     }
  else
     for(i=n+80; i>=0; i--)
     { clrscr();
       for(j=0; j<80; j++) putch(S[i+j]);
       delay(200);
     }
     // In functie de modul cum s-a precizat sensul deplasarii
     // se face afisarea unui numar de cate 80 de caractere din
     // sirul S, incepand cu afisarea caracterelor 1-80, 2-81,
     // 3-83 ...  daca sirul se va deplasa de la dreapta la
     // stanga sau incepand cu ultimele 80 de caracterele apoi
     // cele anterioare daca deplasarea sirului va fi de la stanga
     // la dreapta.
     // Inainte de fiecare afisare a cate unui numar de 80 de
     // caractere fereastra se sterge, se afiseaza caracterele
     // iar acestea raman pe ecran o durata de 200 milisecunde.

  getch(); return 0;
}
```

După cum s-a văzut, prin comentariile inserate în program s-au dat toate explicaţiile necesare pentru înţelegerea modului de funcţionare a acestuia .

Capitolul XV

Programarea în mod grafic.

Prezentarea unor rezultate sub formă grafică, a unor diagrame care se trasează (în culori) pe ecran în faţa ochilor utilizatorului, a unor imagini în mişcare care pot fi de exemplu nişte corpuri aparţinând unui sistem mecanic vibrant este fără îndoială un lucru foarte spectaculos, care însă nu era realizabil cu versiuni mai vechi ale unor limbaje de programare.

În prezent toate limbajele de programare au o componentă de grafică, inclusiv limbajul C/C++ . Funcţiile ce se utilizează nu sunt standardizate conform ANSI C motiv pentru care cele aparţinând la Borland C/C++ nu corespund cu Microsoft C/C++ .

Funcţiile grafice utilizabile sub Microsoft C/C++ se pot găsi în [Petr93] , [Schi98] pg. 454-483, aceste funcţii fiind cuprinse în fişierul **graph.h** .

Dacă în cazul programării în mod text poziţia unui caracter pe ecran era definită prin coloana (de la 1 la 80) şi linia (de la 1 la 25 sau 50) pe care se află , în cazului modului grafic se lucrează cu elemente de imagine numite **pixeli**. Numărul de pixeli care poate apare pe ecranul unui monitor determină **rezoluţia** acestuia. În prezent rezoluţiile standard sunt următoarele :

Rezoluţie	*Standard*
640**µ**480	**VGA** (**V**ideo **G**raphics **A**rray)
800**µ**600	**SVGA** (**S**uper **VGA**)
1024**µ**768	**XGA** (e**X**tended **G**raphics **A**rray)
1280**µ**1024	**UVGA** (**U**ltra **VGA**)

prima cifră referindu-se la numărul de pixeli pe orizontală iar a doua la cel pe verticală.

În ceea ce priveşte modul de realizare al monitoarelor, acesta sunt cu tuburi catodice (**CRT** – **C**athode **R**ay **T**ube) deci similare televizoarelor sau cu cristale lichide (**LCD** – **L**iquid **C**rystal **D**isplay) în cazul laptop-urilor.

În continuare se va face prezentarea unora din funcţiile grafice corespunzătoare compilatorului **Borland C/C++**, aparţinând fişierului **graphics.h** .

Lista completă a acestor funcţii (83 în total) este următoarea :

arc	bar	bar3d
circle	cleardevice	clearviewpoint
closegraph	detectgraph	drawpoly
ellipse	fillellipse	fillpoly
floodfill	getarccoords	getaspectratio
getbkcolor	getcolor	getdefaultpalette
getdrivername	getfillpattern	getfillsettings
getgraphmode	getimage	getlinesettings

getmaxcolor	getmaxmode	getmaxx
getmaxy	getmodename	getmoderange
getpalette	getpalettesize	getpixel
gettextsettings	getviewsettings	getx
gety	graphdefaults	grapherrormsg
_graphfreemem	_graphgetmem	graphresult
imagesize	initgraph	installuserdriver
installuserfont	line	linerel
lineto	moverel	moveto
outtext	outtextxy	pieslice
putimage	putpixel	rectangle
registerbgidriver	registerfarbgdriver	registerbgifont
registerfargifont	restorecrtmode	sector
setactivepage	setallpalette	setaspectratio
setbkcolor	setcolor	setfillpattern
setfillstyle	setgraphbufsize	setgraphmode
setlinestyle	setpalette	setrgbpalette
settextjustify	settextstyle	setusercharsize
setviewpoint	setvisualpage	setwritemode
textheight	textwidth	

Din două motive, primul fiind că aceste funcţii ***nu sunt standardizate***, conform ANSI C, iar al doilea că descrierea tuturor ar consuma foarte mult spatiu, în continuare s-a făcut o selecţie, prezentându-se numai cele mai importante şi mai frecvent utilizate.

15.1 Setarea modului grafic. Gestiunea culorilor. Ferestre.

După cum s-a văzut, în cazul programării în ***mod text*** poziţia pe ecran a unui caracter era determinată de două coordonate, ***numărul coloanei*** şi ***numărul liniei*** .

În cazul programării în mod grafic poziţia pixelului pe ecran este de asemenea determinată de două coordonate, abscisa acestuia **x**, sensul acestei axe fiind spre dreapta şi ordonata **y**, sensul acestei axe fiind în jos, coordonatele pixelului aflat în colţul din stânga sus (în originea celor două axe de coordonate) fiind **[0 ; 0]**.

Dacă rezoluţia monitorului este **640 µ 480** atunci coordonatele pixelilor din colţurile ecranului sunt :

```
- stânga sus  :  x =   0;  y =   0;
- stânga jos  :  x =   0;  y = 479;
- drepta sus  :  x = 639;  y =   0;
- dreapta jos :  x = 639;  y = 479;
```

Pentru a realiza programarea în mod grafic trebuie ca iniţial să se facă o setare a acestui mod , cu funcţia ***initgraph(...)*** iar la sfârşit se va folosi funcţia **closegraph( )** pentru a "închide" modul grafic.

Folosirea culorilor se va face fără restricţia de la modul text (erau mai puţine culori pentru fond decât pentru scriere), fiind utilizabile toate cele 16 culori prezentate în tabelul 14.1, fie referindu-ne la codificarea lor numerică, de la **0** la **15** fie scriind efectiv denumirea culorii (în limba engleză, cu majuscule), ca o constantă simbolică.

Pentru a stabili culoarea fondului se foloseşte funcţia ***setbkcolor(...)*** iar pentru stabilirea culorii de desenare/scriere funcţia ***setcolor(...)***. Cu funcţia ***getbkcolor( )*** se poate determina culoarea fondului.

Ca şi în cazul programării în mod text, dar folosind alte funcţii, se pot defini şi şterge ferestre, funcţiile fiind ***setviewport(...)*** şi ***clearviewport( )*** , pentru crearea şi ştergerea unei ferestre, respectiv ***cleardevice( )*** pentru ştergerea întregului ecran.

În continuare se dau prototipurile acestor funcţii, urmate de toate explicaţiile necesare privind modul de funcţionare şi semnificaţia parametrilor. În cadrul acestor prototipuri (scrise conform documentaţiei) s-a folosit cuvântul instrucţiune **far** (nestandardizat) care este un modificator de tip aplicat la pointeri şi funcţii . Pointerii de tip **far** se înregistrează pe 32 de biţi .

```
void far initgraph(int far *gdriver, int far *gmode, int far *path);
     // gdriver  este un pointer la o valoare corespunzatoare
     // tipului driver-ului. Se initializeaza cu constanta
     // simbolica  DETECT.
     // gmode  este un pointer la o valoare corespunzatoare
     // setarii realizate cu  gdriver.
     // path  este un pointer spre sirul de caractere
     // corespunzator "drumului" spre subdirectorul  bgi
     // care contine driver-ele.
     // Functia  initgraph  initializeaza modul grafic.

void far closegraph(void);
     // Functia realizeaza iesirea din modul grafic.

void far setbkcolor(int color);
     // Stabileste culoarea fondului, avand ca argument
     // fie un numar de la 0 la 15 fie o constanta simbolica
     //  0 - BLACK      1 - BLUE          2 - GREEN
     //  3 - CYAN       4 - RED           5 - MAGENTA
     //  6 - BROWN      7 - LIGHTGRAY     8 - DARKGRAY
     //  9 - LIGHTBLUE 10 - LIGHTGREEN   11 - LIGHTCYAN
     // 12 - LIGHTRED  13 - LIGHTMAGENTA 14 - YELLOW
     // 15 - WHITE

int far getbkcolor(void);
     // Returneaza o valoare intreaga corespunzatoare
     // culorii de fond

void far setcolor(int color);
     // Stabileste culoarea de desenare si de scriere,
     // avand ca argument fie un numar fie o constanta
     // simbolica (vezi functia setbkcolor).

void far setviewport(int left, int top, int right, int bottom,
                     int clip);
     // Se defineste o fereastra al carei colt din stanga sus
     // are coordonatele absolute [left;top] iar cel din
     // dreapta jos [right;bottom]. Dupa definire aceasta
     // fereastra devine activa.
     // clip defineste trunchierea imaginii.
```

```
     // clip: 0 - imaginea de pe ecran nu se trunchiaza,
     //           ramane atat in fereastra cat si in afara ei,
     //       1 - imaginea va ramane numai in fereastra activa,
     //           disparand de pe restul ecranului.
     // Pozitia curenta devine pixelul de coordonate [0;0]
     // din fereastra activa.

void far clearviewport(void);
     // Se sterge continutul ferestrei active, culoarea
     // respectivei zone va fi culoarea de fond iar
     // pozitia curenta devine cea a pixelului care s-a
     // aflat in coltul din stanga sus.

void far cleardevice(void);
     // Se sterge tot ecranul, toti pixelii au culoarea de
     // fond iar pozitia curenta este cea a pixelului [0;0].
```

EXEMPLUL nr. 15.1 . În continuare se prezintă (parţial) un program în care se utilizează modul grafic :

```
..........
#include <graphics>
..........
int main(void)
{ ........
  int ..., gd=DETECT, gm, .... ;
  ..........
  ..........
  initgraph(&gd, &gm, "c:\\borlandc\\bgi");
  ..........
  setbkcolor(LIGHTGRAY);   // sau  setbkcolor(8);
  setcolor(RED);           // sau  setcolor(4);
  ..........
  setviewport(100, 50, 400, 250, 0);
  ..........
  clearviewport();
  ..........
  getch();
  closegraph();
  ..........
  return 0;
}
```

Se observă că s-au folosit funcţiile ***initgraph(...)*** şi ***closegraph( )*** pentru iniţializarea şi apoi pentru închiderea modului grafic, culoarea de fond folosită a fost LIGHTGRAY care are codificarea 8, utilizându-se funcţia ***setbkcolor(...)*** iar culoarea de desenare/scriere a fost RED care are codificarea 4 , utilizând funcţia ***setcolor(...)***.

S-au mai folosit funcţiile ***setviewport(...)*** ***şi clearviewport()*** pentru crearea şi apoi ştergerea unei ferestre, coordonatele vârfului din stânga sus fiind x=100; y=50 iar ale vârfului din dreapta jos x=400 ; y=250 . Deoarece s-a atribuit valoarea **0** parametrului notat cu **clip** (în cadrul prototipului) imaginea de pe ecran nu se trunchiază .

Funcţia ***initgraph(...)*** conţine ca al treilea parametru şirul

"c:\\borlandc\\bgi"

care determină drumul spre subdirectorul **bgi** care conţine driver-ele.

Funcţia ***initgraph(...)*** se mai poate scrie şi sub forma mai simplă :

```
initgraph(&gd, &gm, " ");
```

cu condiţia ca în directorul curent să se găsească fişierul **egavga.bgi** .

Exemple de utilizare a tuturor acestor funcţii se vor da în cadrul paragrafelor următoare, în asociaţie cu celelalte instrucţiuni .

15.2 Funcţii pentru desenare şi colorare.

Un prim grup de funcţii sunt acelea care se referă la diverse operaţii cu pixeli, cum ar fi : colorarea unui pixel definit prin coordonatele sale, cu o anumită culoare, determinarea culorii unui pixel, determinarea poziţiei pixelului curent (pe orizontală şi pe verticală), stabilirea unei noi poziţii curente. Aceste funcţii sunt următoarele :

```
void far putpixel(int x, int y, int color);
     // Pixelul cu coordonatele relative [x;y] din fereastra
     // activa se coloreaza cu culoarea  color.

unsigned far getpixel(int x, int y);
     // Returneaza un numar din intervalul 0~15, functie de
     // culoarea pixelului de coordonate [x;y].

int far getx(void);
     // Returneaza pozitia pe orizontala a pixelului curent.

int far gety(void);
     // Returneaza pozitia pe verticala a pixelului curent.

void far moveto(int x, int y);
     // Noua pozitie curenta pe ecran devine cea de
     // coordonate [x;y].

void far moverel(int dx, int dy);
     // Noua pozitie curenta pe ecran devine un pixel plasat la
     // dx  pixeli distanta pe orizontala si  dy  pixeli distanta
     // pe verticala de la pozitia curenta anterioara.
```

După cum se vede, pentru fiecare funcţie se prezintă prototipul acesteia şi explicaţiile necesare privind funcţionarea şi semnificaţia parametrilor .

Un alt grup de funcţii se referă la desenarea diferitelor figuri geometrice simple: linii, dreptunghiuri, cercuri, arce de cerc, sectoare circulare, elipse [mai există şi alte funcţii care desenează figuri geometrice, cum ar fi de exemplu : ***bar3(...)***, ***drawpoly(...)***], fiind de asemenea foarte importantă funcţia care defineşte modul cum sunt trasate liniile (continue, punctate, întrerupte, etc) precum şi grosimea lor . Aceste funcţii sunt :

```
void far setlinestyle(int linestyle,unsigned upatter,intthickness);
     // Determina tipul si grosimea liniilor ce se vor trasa.
     // linestyle: linie continua   - 0  sau   SOLID_LINE
     //            linie punctata   - 1  sau   DOTTED_LINE
     //            linie intrerupta - 2  sau   CENTER_LINE
     //                 (linii cu doua lungimi diferite)
     //            linie intrerupta - 3  sau   DASHED_LINE
     //                 (linii de aceeasi lungime)
     //            tip definit de utilizator
     //                              - 4  sau   USERBIT_LINE
     // upattern: 0 daca linestyle != 4, alte valori daca
     //           linestyle = 4.
     // thickness (grosime) = 1 (grosime de un pixel)
     //                       3 (grosime de trei pixeli)

void far line(int xi, int yi, int xf, int yf);
     // Se traseaza o linie care uneste pixelii de
     // coordonate [xi;yi] si [xf;yf].

void far lineto(int xf, int yf);
     // Se traseaza o linie care uneste pixelul curent cu
     // pixelul de coordonate [xf;yf].
     // Pixelul curent devine cel cu coordonatele [xf;yf].

void far linerel(int dx, int dy);
     // Se traseaza o linie care porneste din pixelul curent
     // a carei proiectii pe axa orizontala si verticala sunt
     // dx  si  dy.

void far rectangle(int left, int top, int right, int bottom);
     // Se traseaza un dreptunghi care are varful din stanga
     // sus de coordonate [left;top] iar cel din dreapta
     // jos de coordonate [right;bottom].

void far bar(int left, int top, int right, int bottom);
     // Se deseneaza un domeniu dreptunghiular colorat cu
     // culoarea de fond, fara trasarea marginilor.
     // Semnificatia parametrilor este aceeasi ca si
     // in cazul functiei rectangle.

void far circle(int xC, int yC, int R);
     // Se traseaza un cerc cu centrul in [xC;yC] si raza R.

void far arc(int xC, int yC, int start_angle, int end_angle,
             int R);
     // Se traseaza un arc de cerc cu centrul in [xC;yC] si
     // raza R, unghiul la centru fiind determinat de
     // unghiurile start_angle  si  end_angle, masurate in
     // sens trigonometric.

void far pieslice(int xC, int yC, int start_angle, int end_angle,
                  int R);
     // Se traseaza un sector de cerc. Semnificatia parame-
     // trilor este aceeasi ca si in cazul functiei arc.
```

```
void far ellipse(int xC, int yC, int u_init, int u_fin,
                 int a, int b);
     // Se traseaza un arc de elipsa, cu centrul in [xC;yC],
     // cu semiaxele a si b, unghiul la centru fiind determinat de
     // unghiurile u_init si u_fin, masurate in sens trigonometric.
```

Pentru realizarea haşurării unor porţiuni din ecran, cu diverse tipuri de haşuri precum şi pentru colorarea interiorului unor zone închise se folosesc funcţiile :

```
void far setfillstyle(int pattern, int color);
     // Defineste modul cum se vor colora sau hasura figurile.
     // Pentru color se ia o valoare numerica din intervalul
     // 0 ~ 15 sau o constanta simbolica.
     // pattern poate avea urmatoarele valori:
     // 0  sau  EMPTY_FILL - figura se coloreaza cu culoarea
     //                      de fond,
     // 1  sau  SOLID_FILL - figura se coloreaza uniform,
     // 2 ~ 11             - figura se va hasura cu diferite
     //                      tipuri de hasuri (vezi HELP-ul),
     // 12 sau USER_FILL   - tipul hasurii se defineste de
     //                      utilizator.

void far floodfill(int x, int y, int border_color);
     // Daca pixelul de coordonate [x;y] apartine unui domeniu
     // inchis atunci interiorul acestui domeniu se coloreaza si
     // hasureaza conform setarilor de la functia setfillstyle.
     // Daca punctul este in exteriorul domeniului inchis
     // atunci se hasureaza exteriorul.
```

EXEMPLUL nr. 15.2 . Să considerăm un program de grafică relativ simplu, dar care va utiliza foarte multe din funcţiile anterior prezentate . Pe ecran se va trasa o linie groasă sub care o zonă dreptunghiulară se va haşura iar deasupra acestei linii se va reprezenta o ortocicloidă normală, generată de un punct aparţinând unui cerc de rază **r** care se rostogoleşte pe dreapta fixă, cercul nefiind reprezentat pe ecran. Fondul va fi de culoare gri deschis, linia orizontală şi haşura de culoare albastră iar curba generată de culoare roşie. Între trasările unor porţiuni succesive de curbă se va introduce o funcţie de întârziere.

Programul este următorul :

```
#include <conio.h>
#include <dos.h>
#include <graphics.h>
#include <math.h>
#include <stdio.h>

int main(void)
{ int gd=DETECT, gm,i;
double a, nc, r, ug, ur, x, xO=60, y, yO=298;
r=0; clrscr();
while(r<10.0||r>85.0)
  { printf("\n\t The radius of the circle\n"
           "\n\t\t  r [10 ~ 85] = "); scanf("%le", &r); }
     // Se introduce valoarea razei cercului care va descrie
     // curba cicloida normala.
```

```
initgraph(&gd, &gm, " ");
     // S-a initializat modul grafic.

setbkcolor(LIGHTGRAY); setcolor(BLUE);
     // S-au setat culorile de fond si de desenare.

setlinestyle(0,0,3);
rectangle(10,10,629,469); line(50,300,600,300);
     // S-a stabilit tipul liniei (continua si cu grosimea de trei
     // pixeli).
     // S-a trasat un chenar dreptunghiular, la marginea ecranului.
     // S-a trasat o linie orizontala.

setlinestyle(0,0,1); setfillstyle(3,BLUE);
     // S-a modificat tipul liniei (continua si subtire).
     // S-a stabilit tipul hasurii (linii subtiri, inclinate la
     // 45 grade).

bar(50,303,600,320); floodfill(200, 315, BLUE);
     // S-a definit un domeniu dreptunghiular si s-a hasurat.

setcolor(RED);  // S-a schimbat culoarea de desenare.
nc=540/2/M_PI/r;
     // nc reprezinta numarul de rostogoliri ale cercului.

for(ug=0; ug<360*nc; ug++)
   { ur=ug*M_PI/180;
     x=xO+r*(ur-sin(ur)); y=yO-r*(1-cos(ur));
     if(ug==0) { moveto(x,y); putpixel(x, y, RED);}
     lineto(x,y); delay(5);
   }
/* In ciclu se calculeaza coordonatele punctelor cicloidei
   cu formulele:
            xC = r*[u-sin(u)];
            yC = r*[1-cos(u)];
   si se tine cont ca  xC  trebuie adaugat la xO, care este
   abscisa originii sistemului de coordonate iar  yC  trebuie
   scazut din  yO (ordonata originii) deoarece pe ecran axa
   y  este cu sensul in jos.
   Primul punct al graficului (cand u=0) s-a realizat cu
   functiile  moveto  si  putpixel iar urmatoarele cu linerel.
   In calculul valorii in radiani a unghiului de rostogolire
   ur  s-a folosit constanta M_PI care are valoarea lui
   pi=3.1415926...                                          */
getch();
closegraph();
     // S-a inchis modul grafic.
return 0; }
```

Considerăm comentariile introduse după fiecare etapă a programului suficiente pentru înţelegerea acestuia.

Ca urmare a execuţiei programului se va obţine următoarea figură (aici în alb/negru) :

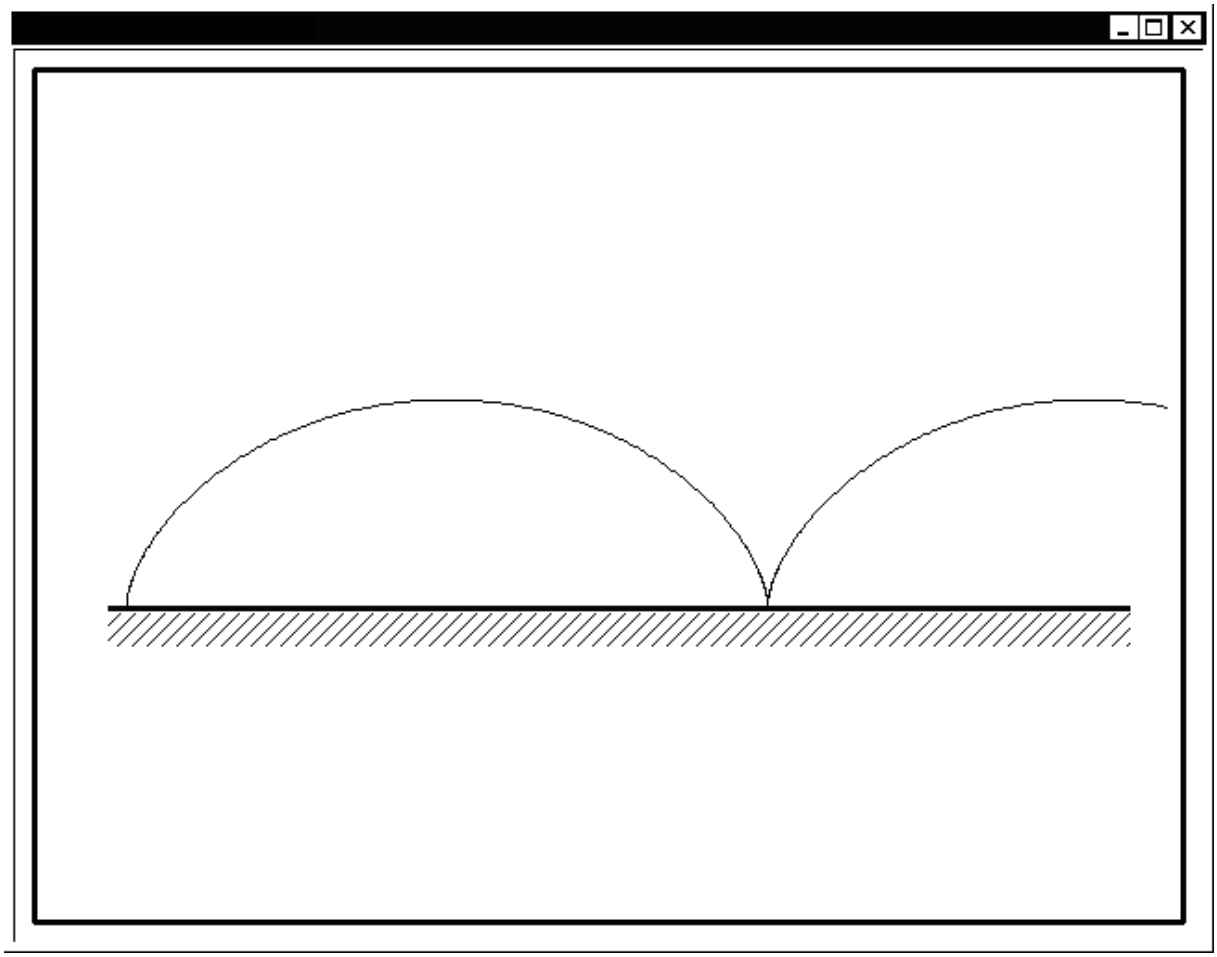


Fig. 15.1

15.3 Funcţii pentru afişarea textelor.

În afară de realizarea unor desene, în cadrul modului grafic este posibilă şi afişarea unor texte, cu diferite tipuri de fonturi, pe orizontală sau pe verticală, cu diferite mărimi ale caracterelor.

Funcţiile care realizează aceste operaţii sunt următoarele :

```
void far settextstyle(int font, int direction, int char_size);
     // Se definesc fonturile ce se vor utiliza, directia de
     // scriere si marimea caracterelor.
     // font:  0  sau  DEFAULT_FONT
     //        1  sau  TRIPLEX_FONT
     //        2  sau  SMALL_FONT
     //        3  sau  SANS_SERIF_FONT
     //        4  sau  GOTHIC_FONT
     // direction:  0  sau  HORIZ_DIR
     //             1  sau  VERT_DIR
     // char_size: 1 ~ 10 (functie de marimea dorita a
     //                    caracterelor)
```

```
void far settextjustify(int horiz, int vert);
     // Se defineste modul de afisare al textelor.
     // horiz - LEFT_TEXT , CENTER_TEXT  sau  RIGHT_TEXT
     // vert  - TOP_TEXT  , CENTER_TEXT  sau  BOTTOM_TEXT

void far outtext(char far *text);
     // Se afiseaza sirul spre care pointeaza text, incepand
     // din pozitia curenta

void far outtextxy(int x, int y, char far *text);
     // Se afiseaza sirul spre care pointeaza text, incepand
     // din pozitia definita de coordonatele  [x;y].
```

După cum se poate observa, sunt disponibile cinci tipuri de fonturi, cu zece mărimi posibile ale caracterelor iar textele se pot afişa fie pe orizontală fie pe verticală .

Dacă respectivele texte sunt şiruri de caractere care au fost introduse de la tastatură sau au fost iniţializate, atunci aceste funcţii sunt suficiente pentru a realiza afişarea lor. Sunt situaţii când trebuie afişate pe ecran diverse informaţii, în special numerice, sub controlul formatelor, caz în care şirul ce se va afişa va fi creat cu funcţia ***sprintf(...)*** a cărei prototip este :

```
int sprintf(char *buffer, const char *format[, argument, ...]);
```

Această funcţie aparţine fişierului **stdio.h** şi realizează reţinerea în şirul al cărui pointer este **buffer** a datelor formatate conform şirului cuprins între ghilimele, urmat de argumente. Funcţia returnează numărul de caractere din şirul care s-a transferat (fără să ia în considerare şi caracterul nul **'\0'**) sau **EOF** în caz de eroare.

EXEMPLUL nr. 15.3 . În programul prezentat în cadrul exemplului 15.2 în afară de partea de desenare nu s-a realizat afişarea unor informaţii sub formă de texte sau a unor informaţii numerice. Este evident că în cazul unor astfel de programe acest lucru este necesar, pe lângă apariţia curbei cicloidale fiind indicat să se cunoască , citindu-se de pe ecran, valori numerice referitoare la raza cercului care se rostogoleşte, precum şi valorile curente ale unghiului de rotaţie şi ale coordonatelor punctului.

S-a reluat programul anterior prezentat, adăugându-se funcţiile ***sprintf(...), settextstyle(...), settextjustify(...)*** şi ***outtextxy(...)*** precum şi o declarare a şirurilor de tip **char** ce se vor afişa, care fie vor conţine texte explicative fie informaţie numerică.

Dacă în varianta iniţială a acestui program, din exemplul nr. 15.2 , s-au inclus comentarii pentru explicarea funcţiilor folosite, în cazul programului următor comentariile ce s-au introdus se referă numai la noile instrucţiuni utilizate .

Programul este următorul :

```
#include <conio.h>
#include <dos.h>
#include <graphics.h>
#include <math.h>
#include <stdio.h>

int main(void)
{ int gd=DETECT, gm,i, np;
char str1[]="The cycloid", str2[]="The radius of the circle = ",
     str3[]="angle = ", str4[]="x     = ",
     str5[]="y     = ", *str_r, *str_ug, *str_x, *str_y;
```

```
      // Se declara iar unele siruri care se vor afisa se
       // initializeaza.

double a,nc,pasg,r,ug,ur,x,x1,x2,xO=60,y,y1,y2,yO=298;
r=0;clrscr();
while(r<10.0||r>85.0)
  {printf("\n\t The radius of the circle\n\n\t\t  r [10 ~ 85] = ");
   scanf("%le", &r); }
sprintf(str_r, "%7.3f", r);
      // Sirul str_r va contine valoarea numerica a razei r
      // a cercului.

initgraph(&gd, &gm, " ");
setbkcolor(LIGHTGRAY); setcolor(BLUE); setlinestyle(0,0,3);
rectangle(10,10,629,469); line(50,300,600,300);
setlinestyle(0,0,1); setfillstyle(3,BLUE);
bar(50,303,600,320); floodfill(200,315,BLUE);
settextstyle(0, 0, 5);
      // Se stabileste ca textele care urmeaza se vor afisa pe
      // orizontala, cu fonturi de marimea 5.

settextjustify(LEFT_TEXT, TOP_TEXT);
      // Se stabileste modul de afisare al textelor care urmeaza.

outtextxy(100, 20, str1);
      // Se afiseaza cu fonturi de marime 5 sirul de caractere
      // continut in str1: "The cycloid".
      // Culoarea de afisare este BLUE.

settextstyle(0,0,1);
      // Se stabileste ca textele care urmeaza se vor afisa pe
      // orizontala cu fonturi de marimea 1.

outtextxy(50, 85,str2); outtextxy(250,85,str_r);
outtextxy(50,100,str3); outtextxy(50,110,str4);
outtextxy(50,120,str5);
      // Se afiseaza continutul sirurilor de caractere str2,
      // str_r, str3, str4, str5. Culoarea de afisare este BLUE.

nc=540/2/M_PI/r; pasg=2.5; np=360*nc/pasg;
for(i=0; i<=np; i++)
   { ug=i*pasg;
     ur=ug*M_PI/180;  sprintf(str_ug,"%8.3f",ug);
     x=r*(ur-sin(ur));sprintf(str_x, "%8.3f", x);
     y=r*( 1-cos(ur));sprintf(str_y, "%8.3f", y);
      // Sirurile str_ug, str_x  si  str_y  vor contine valorile
      // numerice curente ale unghiului de rotire ale cercului si
      // ale coordonatelor punctului cicloidei.

     x=xO+x; y=yO-y;
     if(ug==0) { x1=x; y1=y;}
     setcolor(RED);
     line(x1,y1,x,y); x1=x; y1=y;
     outtextxy(110,100,str_ug); outtextxy(110,110,str_x);
```

```
     outtextxy(110,120,str_y);
      // S-au afisat cu culoare RED valorile curente ale unghiului de
      // rotatie ug al cercului care se rostogoleste si ale abscisei
      // x si ordonatei y ale punctului care genereaza cicloida.

     delay(50);
     if(i==np) break;
     setcolor(LIGHTGRAY);
     outtextxy(110,100,str_ug);
     outtextxy(110,110,str_x);
     outtextxy(110,120,str_y);
      // Se sterge ceea ce s-a afisat cu culoarea  RED deoarece
      // peste afisarea anterioara se face afisarea acelorasi
      // informatii utilizand culoarea fondului.

   }
      // Ciclul este condus de variabila intreaga  i (nr. de
      // ordine al punctului curent) iar  pentru trasarea unei
      // portiuni din cicloida s-a folosit functia  line.

getch(); closegraph(); return 0;
}
```

După cum se poate observa, pe fondul LIGHTGRAY (gri deschis) s-au desenat şi scris cu culoare BLUE (albastră) toate elementele fixe (constante) , cum ar fi chenarul, linia groasă şi partea haşurată, titlul, valoarea razei şi şirurile de caractere : "angle = ", "x = " şi "y = " iar trasarea curbei cicloidă normală şi afişarea valorilor unghiului şi a coordonatelor x şi y (variabile) s-a făcut cu culoarea RED (roşu). În timpul trasării curbei se modifică în permanenţă valorile afişate cu RED, ale unghiului şi coordonatelor x şi y .

Pentru ca pe ecran să rămână informaţia corespunzătoare ultimului punct considerat (la ultima parcurgere a corpului ciclului) s-au introdus instrucţiunile :

```
if(i==np) break;
```

care vor produce ieşirea din ciclu, deci nu se va mai face afişarea peste textul scris cu roşu a aceluiaşi text scris cu culoarea fondului .

Modul cum se face afişarea informaţiei numerice pe ecran este stabilit prin specificatorii de format utilizaţi în cadrul funcţiilor ***sprintf(...)***, raza **r** afişându-se cu descriptorul **%7.3** iar celelalte valori (unghiul **ug** şi coordonatele **x** şi **y**) cu **%8.3f** .

Ca urmare a execuţiei acestui program se obţine imaginea de pe ecran, prezentată în fig. 15.2:

15.4 Realizarea deplasării pe ecran a imaginilor.

Există mai multe metode de a realiza deplasarea unor imagini pe ecran, înaintea apariţiei imaginii în poziţia următoare aceasta fiind ştearsă din poziţia anterioară, astfel încât se crează impresia deplasării (continue) a imaginii pe ecran .

METODA I .Cel mai simplu mod de a realiza acest lucru este de a realiza desenarea şi scrierea cu două culori, una fiind culoarea de desenare/scriere propriu zisă iar cealaltă fiind culoarea de fond. Se procedează în felul următor :

- se realizează imaginea dorită pe ecran,

- imaginea rămâne în poziţia respectivă un anumit interval de timp (se utilizează funcţia ***delay(...)***),

- se realizează a doua oară aceeaşi imagine şi în aceeaşi poziţie pe ecran, dar utilizând ca şi culoare de desenare culoarea fondului, în consecinţă imaginea va dispare din poziţia în care s-a aflat,

- se trece la următoarea etapă , care începe cu realizarea imaginii într-o nouă poziţie .

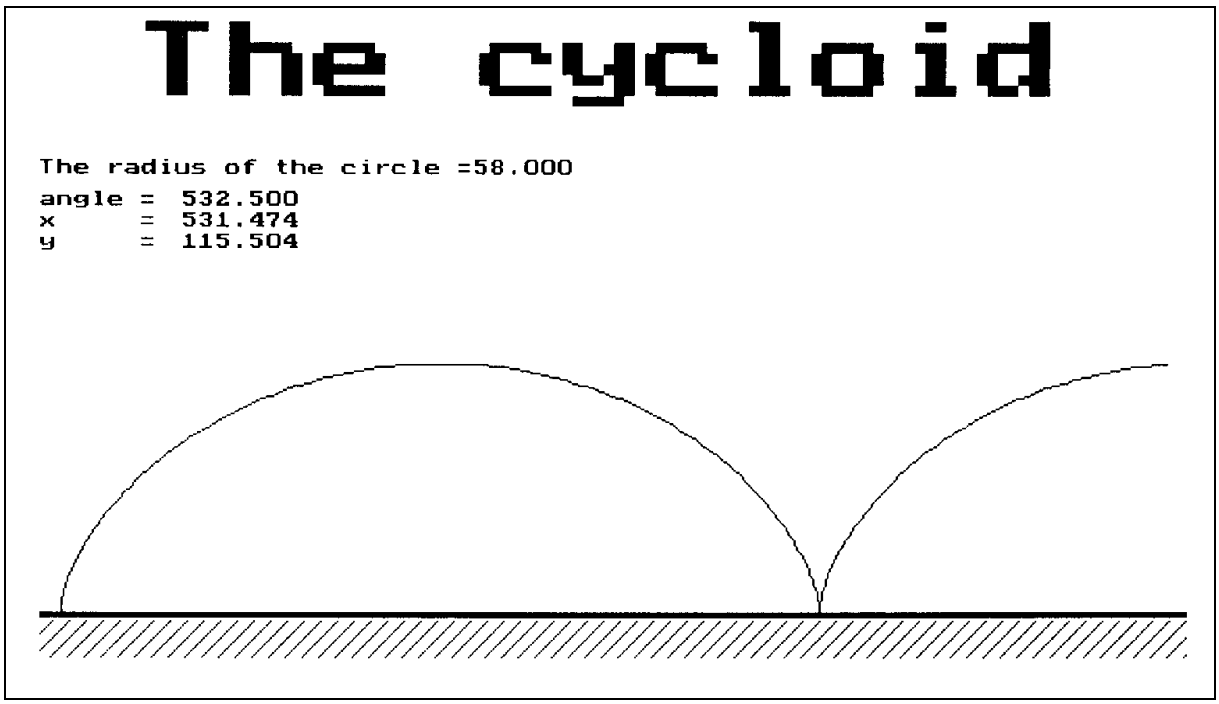


Fig. 15.2

EXEMPLUL nr. 15.4 . Într-un dreptunghi se va scrie textul "This is a rectangle" iar imaginea respectivă va efectua pe ecran o mişcare de translaţie curbilinie, astfel încât vârful din stânga sus al dreptunghiului să descrie o elipsă cu semiaxele **a** şi **b**, elipsa având centrul în punctul de coordonate [215 ; 220] . Liniile se vor trasa continuu iar textul va fi cu fonturi de mărime mică .

Deplasarea imaginii se va face de un număr neprecizat de ori, utilizând instrucţiunea **do – while**, ieşirea din ciclu realizându-se ca urmare a apăsării unei taste, deoarece s-a folosit funcţia **kbhit()** în interiorul parantezei de la **while** .

Această funcţie are prototipul

```
int kbhit(void);
```

aparţine fişierului **conio.h** şi returnează valoarea 0 dacă nu s-a apăsat o tastă respectiv o valoarea întreagă diferită de zero dacă s-a apăsat o tastă .

Programul este următorul :

```
//  Deplasarea unui dreptunghi + text (prima metoda)

#include <conio.h>
#include <dos.h>
#include <graphics.h>
#include <math.h>
#include <stdio.h>
```

```
int main(void)
{ int CF=LIGHTGRAY, CD=BLUE, gd=DETECT, gm, x=400, y=205;
  double a=150, b=100, pas=0.5, r, u, ur;
  char str[]="This is a rectangle";
  initgraph(&gd, &gm, " ");
  setbkcolor(CF); setcolor(CD);
  setlinestyle(0,0,1); settextstyle(0,0,1);
  u=-pas;
  do {u+=pas; ur=-u*M_PI/180;
      x=215+a*cos(ur); y=220+b*sin(ur);
  //        Se scrie/deseneaza cu culoarea CD=BLUE
      setcolor(CD); ellipse(215,220,0,360,a,b);
      rectangle(x,y,x+200,y+50); outtextxy(x+25,y+20,str);
      delay(35);
  //        Se scrie/deseneaza cu culoarea CF (a fondului).
      setcolor(CF); ellipse(215,220,0,360,a,b);
      rectangle(x,y,x+200,y+50); outtextxy(x+25,y+20,str);
     }
  while(!kbhit());
  getch(); closegraph(); return 0;
}
```

Se poate observa că se execută de două ori următoarea succesiune de funcţii :

```
ellipse(215,220,0,360,a,b);
rectangle(x,y,x+200,y+50);
outtextxy(x+25,y+20,str);
```

prima dată folosind ca şi culoare de desenare/scriere culoarea BLUE iar a doua oară culoarea LIGHTGRAY, identică cu cea a fondului, motiv pentru care imaginea va dispare.

Coordonatele vârfului din stânga sus al dreptunghiului (care se găseşte pe elipsă) s-au calculat cu formulele parametrice

$$x = a\cos\varphi\ ;\quad y = b\sin\varphi$$

la care s-au adăugat valorile 215 şi 220 pentru că acestea sunt coordonatele centrului elipsei

În figura 15.3 apare copia ecranului ce se obţine pentru o anumită poziţie a imaginii.

EXEMPLUL 15.5. În exemplele anterior prezentate s-a realizat trasarea unei curbe (o ortocicloidă – exemplul nr. 15.2), trasarea unei curbe şi afişarea repetată a coordonatelor curente (exemplul nr. 15.3) şi deplasarea unei figuri (un dreptunghi – exemplul nr. 15.4).

În acest exemplu se va prezenta un program cu care se face trasarea unei curbe de tip **epicicloidă** (normală, buclată sau scurtată), obţinută ca urmare a rostogolirii unui cerc de rază **r** în exteriorul unui cerc fix de rază **R**, un punct legat de cercul mobil, la distanţa **a** de centrul acestuia, generând epicicloida.

Dacă se notează cu φ unghiul dintre axa Ox şi dreapta care uneşte centrele celor două cercuri iar cu ψ unghiul de rotire al cercului mobil atunci există următoarea relaţie :

$$r\,\psi = R\,\varphi$$

iar coordonatele punctului care generează epicicloida se calculează cu următoarele expresii parametrice :

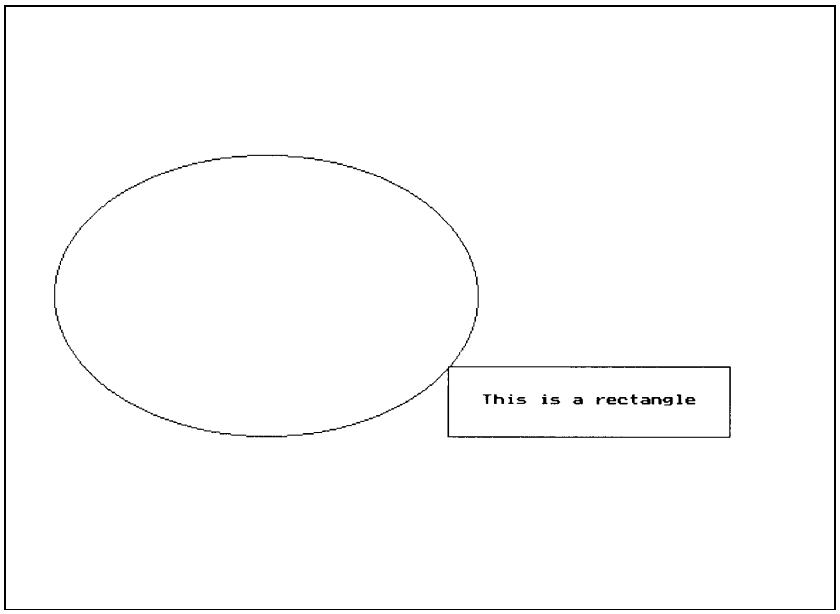


Fig. 15.3

$$x = (R+r)\cos\varphi + a\cos(\pi - \psi - \varphi);\quad y = (R+r)\sin\varphi - a\sin(\pi - \psi - \varphi),$$

aceste relaţii fiind programate în cadrul programului.

Programul care se prezintă în continuare conţine ca şi comentarii explicaţiile necesare pentru înţelegerea modului său de elaborare.

```
#include <conio.h>
#include <dos.h>
#include <graphics.h>
#include <math.h>
#include <stdio.h>

int main(void)
{ int CF=LIGHTGRAY, gd=DETECT, gm, i, j, np;
  double a,pasg=1.5, phig, phir, psir, R, r, xC[400],
         xO=320, xP[400], yC[400], yO=240, yP[400];
  char *str1="x", *str2="y", *str3="R = ", *str4="r = ",
       *str5= "phi_g = ", *str6="x    = ", *str7="y    = ",
       *str8="a = ", str_R[50], str_r[50], str_a[50],
       str_u[50], str_x[50], str_y[50];

clrscr(); R=r=a=-1;
while(R<0||R>150)
  { printf("\n\t  Raza cercului fix  R  [ 0 < R < 150.00 ] = ");
    scanf("%le",&R); }
while(r<0||r>0.5*(210-R))
  { printf("\t Raza cercului mobil r  [ 0 < r < %6.2f ] = ",
           0.5*(210-R)); scanf("%le",&r); }
```

```
while(a<0||a>220-R-r)
  { printf("\t Dist. de la centrul cercului mobil la punct"
           "  a  [0<a<%6.2f] = ",220-R-r); scanf("%le",&a); }
     //  S-au introdus valorile numerice care definesc curba de
     //  tip epicicloida, respectandu-se limitarile impuse.

sprintf(str_R,"%6.2f",R); sprintf(str_r,"%6.2f",r);
sprintf(str_a,"%6.2f",a); np=360/pasg+1;

for(phig=0.0,i=0; phig<=360; phig+=pasg,i++)
   { phir=phig*M_PI/180; psir=R/r*phir;
     xC[i]=xO+(R+r)*cos(phir); yC[i]=yO-(R+r)*sin(phir);
     xP[i]=xO+(R+r)*cos(phir)+ a*cos(M_PI-psir-phir);
     yP[i]=yO-(R+r)*sin(phir)+ a*sin(M_PI-psir-phir);
   }
      // S-au determinat coordonatele centrului C ale cercului
      // mobil si coordonatele punctului P care descrie epicicloida.

initgraph(&gd, &gm, " ");
setbkcolor(CF); setlinestyle(0,0,1); setcolor(BLUE);
outtextxy(10,10,str3); outtextxy(55,10,str_R);
outtextxy(10,20,str4); outtextxy(55,20,str_r);
outtextxy(10,30,str8); outtextxy(55,30,str_a);
outtextxy(20,50,str5); outtextxy(20,60,str6);
outtextxy(20,70,str7);
outtextxy(350, 440, "Generarea unei epicicloide");
     //  S-a facut afisarea unor siruri de caractere care nu se
     //  vor modifica (cu culoarea BLUE).

for(i=1; i<=np; i++)
   { setcolor(BLUE); line(100,yO,540,yO);
     moveto(540,yO+3); linerel(12,-3); linerel(-12,-3);
     linerel(0,6); outtextxy(540,yO+4,str1); line(xO,20,xO,460);
     moveto(xO+3,20); linerel(-6,0); linerel(3,-12);
     linerel(3,12); outtextxy(xO+6,13,str2);
     circle(xO,yO,R-1); circle(xO,yO,2); line(xO,yO,xC[i],yC[i]);
     circle(xC[i],yC[i],2);
     setcolor(RED); circle(xC[i],yC[i],r);
     line(xC[i],yC[i],xP[i],yP[i]); circle(xP[i],yP[i],2);
     phig=i*pasg; sprintf(str_u,"%7.2f",phig);
     outtextxy(80,50,str_u); sprintf(str_x,"%7.2f",xP[i]-xO);
     outtextxy(80,60,str_x); sprintf(str_y,"%7.2f",yO-yP[i]);
     outtextxy(80,70,str_y);

     for(j=1; j<=i; j++)line(xP[j-1],yP[j-1],xP[j],yP[j]);
     delay(100);
     if(i==np-1) break;
     //  Cu culorile BLUE si RED s-a facut afisarea unor texte
     //  si desenarea unor elemente care fie sunt mobile fie se
     //  vor "sterge" ca urmare a deplasarii unor figuri pe ecran.
     //  In cadrul ultimului ciclu s-a facut redesenarea portiunii
     //  de epicicloida, anterior desenata si care a fost partial
     //  stearsa ca urmare a deplasarii cercului mobil si a
```

```
      //  segmentului legat de acesta.

     setcolor(CF); line(100,yO,540,yO);  moveto(540,yO+3);
     linerel(12,-3); linerel(-12,-3); linerel(0,6);
     line(xO,20,xO,460); moveto(xO+3,20); linerel(-6,0);
     linerel(3,-12); linerel(3,12); circle(xO,yO,R-1);
     circle(xO,yO,2); line(xO,yO,xC[i],yC[i]);
     circle(xC[i],yC[i],2); circle(xC[i],yC[i],r);
     line(xC[i],yC[i],xP[i],yP[i]); circle(xP[i],yP[i],2);
     outtextxy(540,yO+4,str1); outtextxy(xO+6,13,str2);
     outtextxy(80,50,str_u); outtextxy(80,60,str_x);
     outtextxy(80,70,str_y);
      //  S-a facut desenarea cu culoarea de fond a tuturor
      //  elementelor mobile de pe ecran precum si a textelor
      //  care se modifica.

   }
getch(); closegraph(); return 0; }
```

Ca urmare a execuţiei acestui program s-a obţinut următoarea imagine :

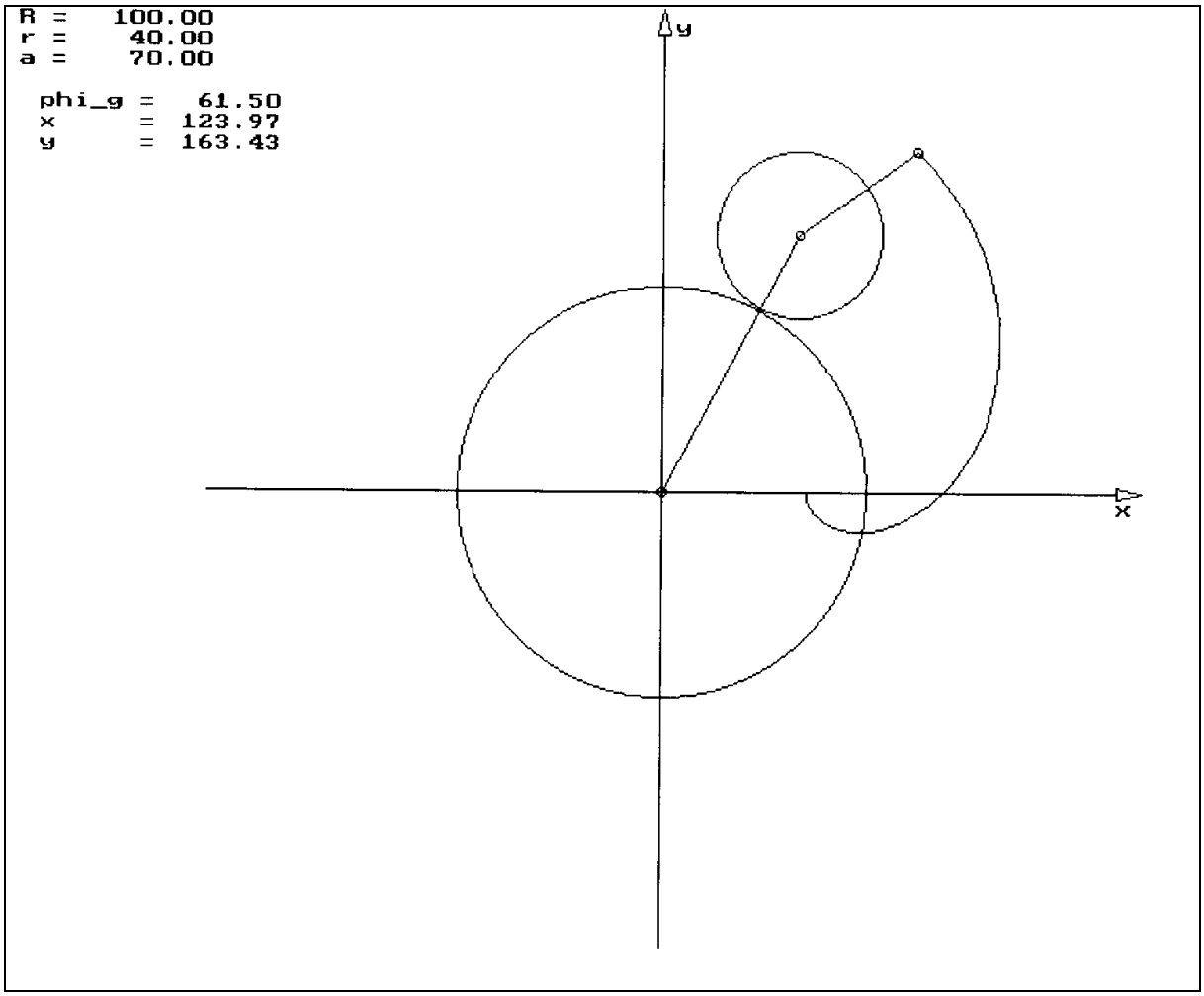


Fig. 15.4

Observaţie . Utilizându-se acest program, cu unele modificări în ceea ce priveşte formulele de calcul, se poate face generarea unor curbe de tip **hipocicloidă** (cercul mobil de rază **r** se rostogoleşte în interiorul cercului fix de rază **R** , **r<R**) sau de tip **pericicloidă** (cercul mobil de rază **r** se rostogoleşte peste cercul fix de rază **R** care se găseşte în interiorul celui mobil, **r>R**).

METODA II . Deplasarea imaginilor pe ecran (fără a fi rotite) se poate realiza dacă se utilizează o serie de funcţii, dintre care ***malloc(...)*** aparţine fişierului standardizat **alloc.h** iar ***imagesize(...), getimage(...), putimage(...)*** fişierului **graphics.h** . Prototipurile acestor funcţii sunt următoarele :

```
unsigned far imagesize(int left,int top,int right,int bottom);
      // Intr-o zona dreptunghiulara a ecranului (coltul din stanga
      // sus are coordonatele [left;top] iar cel din dreapta jos
      // [right;bottom]) se gaseste o imagine.
      // Functia returneaza numarul de octeti (bytes) necesari pentru
      // a inregistra imaginea respectiva.
      // In caz de eroare returneaza –1.

void *malloc(size_t size);
      // Functia face parte din fisierul  stdlib.h
      // Se aloca o zona de memorie de  size  bytes
      // Functia returneaza un pointer de tip void spre zona
      // alocata de memorie.
      // In caz de eroare (nu exista memorie suficienta) se
      // returneaza pointerul NULL.

void far getimage(int left, int top, int right, int bottom,
                  void far *bitmap);
     // Se salveaza imaginile dintr-o parte dreptunghiulara
     // a ecranului determinata de coordonatele [left;top] si
     // [right;bottom].
     // bitmap este pointerul corespunzator zonei din
     // memorie in care este retinuta informatia.

void far putimage(int left, int top, void far *bitmap,
                  int op);
     // Se face afisarea pe ecran, intr-o zona dreptunghiulara
     // al carei colt din stanga sus are coordonatele
     // [left;top] a imaginii pastrate in memorie, incepand de
     // la o locatie determinata de pointerul bitmap.
     // op defineste tipul operatiei care se face intre datele
     // (imaginea) existente pe ecran si cele care se vor
     // afisa in aceeasi zona dreptunghiulara.
     // op:  0  sau  COPY_CUT - apare imaginea noua, a sursei,
     //      1  sau  XOR_PUT  - SAU EXCLUSIV intre imagini,
     //      2  sau  OR_PUT   - SAU intre imagini,
     //      3  sau  AND_PUT  - SI intre imagini,
     //      4  sau  NOT_PUT  - apare imaginea inversa a
     //                         sursei
```

Metoda a II-a constă în următoarele: se crează imaginea într-o zonă dreptunghiulară a ecranului, se determină cu funcţia ***imagesize(...)*** numărul de octeţi necesar pentru înregistrarea în

memorie a informaţiilor conţinute în respectiva zonă, se determină cu ajutorul funcţiei ***malloc(...)*** un pointer asociat zonei de memorie apoi se salvează imaginea în zonă din memorie spre care pointează pointerul, cu funcţia ***getimage(...)*** .

În noua poziţie imaginea se plasează cu ajutorul funcţiei ***putimage(...)***.

Înainte de a plasa imaginea în noua poziţie trebuie ştearsă imaginea din vechea poziţie.

EXEMPLUL nr. 15.6 . Deoarece imaginea a cărei deplasare se realiza cu programul de la exemplul nr. 15.4 executa numai o mişcare de translaţie aceeaşi problemă se va putea rezolva şi cu cea de-a doua metodă .

Imaginea este creată corespunzător poziţiei iniţiale a dreptunghiului şi a textului din interior, după care se determină pointerul **bm** asociat zonei din memorie în care este înregistrată imaginea, imagine care este salvată, ca urmare a parcurgerii următoarelor instrucţiuni şi funcţii :

```
bm=malloc(imagesize(215+a,220,415+a,270));
getimage(215+a,220,415+a,270, bm);
```

Programul este următorul :

```
//  Deplasarea unui dreptunghi + text (a II-a metoda)
#include <alloc.h>
#include <conio.h>
#include <dos.h>
#include <graphics.h>
#include <math.h>
#include <stdio.h>

int main(void)
{ void *bm;
int CD=RED, CF=LIGHTGRAY, gd=DETECT, gm, x, y;
double a=150, b=100, pas=5.0, r, u, ur;
char str[]="This is a rectangle";
initgraph(&gd, &gm, " ");
setbkcolor(CF); setcolor(CD); setlinestyle(0,0,1);
settextstyle(0,0,1);
rectangle(215+a,220,415+a,270); outtextxy(240+a,240,str);
bm=malloc(imagesize(215+a,220,415+a,270));
getimage(215+a,220,415+a,270, bm);
u=-pas;
do {cleardevice();
    u+=pas; ur=-u*M_PI/180; x=215+a*cos(ur); y=220+b*sin(ur);
    putimage(x,y,bm,0); ellipse(215,220,0,360,a,b);
    delay(150);
   }
while(!kbhit());
getch(); closegraph(); return 0;
}
```

Se observă că înainte de afişarea imaginii în noua poziţie se şterge tot ecranul (acest lucru este posibil în cazul acesta) imaginea este plasată în poziţia corespunzătoare şi rămâne pe ecran un anumit interval de timp (150 de milisecunde). Întreruperea deplasării imaginii se face, ca şi în exemplul precedent, ca urmare a apăsării unei taste .

METODA a III-a . În fişierul **graphics.h** există două funcţii care permit ***trecerea de la un ecran grafic la altul***, în general având posibilitatea să se lucreze cu mai multe pagini grafice. Aceste funcţii sunt ***setactivepage(...)*** şi ***setvisualpage(...)*** ale căror prototipuri sunt următoarele :

```
void far setactivepage(int page);
      // Stabileste ca pagina grafica cu numarul specificat este
      // activa.
      // Toate functiile grafice care vor urma se vor referi la pagina
      // declarata activa.

void far setvisualpage(int page);
      // Stabileste si realizeaza afisarea paginii  grafice cu
      // numarul specificat.
```

Dacă în ceea ce priveşte deplasarea pe ecran a unei imagini a cărei formă şi poziţie nu se modifică metoda a II-a este mult mai eficientă decât metoda I, în cazul în care imaginea se modifică sau îşi schimbă poziţia (face de exemplu o rotaţie) numai metoda I mai poate fi aplicată .

Să reamintim încă odată etapele ce trebuie parcurse când se aplica **metoda I** :

- imaginea este creată într-o zonă a ecranului,
- imaginea este "staţionară" un anumit interval de timp,
- imaginea este ştearsă , fiind redesenată cu culoarea de fond,
- imaginea este desenată pe ecran într-o altă poziţie, apropiată de prima, etc,

Lucrând cu **acelaşi ecran** este evident că aceste **etape** sunt **succesive .**

Avantajul **metodei III** ar consta în faptul că lucrând cu **mai multe ecrane** grafice (două sunt suficiente) se poate face o **suprapunere în timp a etapelor**, astfel în timpul cât un ecran este vizibil (câteva milisecunde sau zeci de milisecunde) , pe un alt ecran (ecranul activ), "invizibil", se face ştergerea imaginii şi desenarea acesteia într-o nouă poziţie .

Programele în C/C++ care conţin aceste funcţii, **setactivepage** şi **setvisualpage**, pot fi executate ***numai*** sub sistemul de operare **DOS**, motiv pentru care considerăm că nu e cazul să fie prezentate în exemple.

Pe lângă exemplele prezentate anterior, în care s-au făcut reprezentări grafice ale unor curbe tehnice, s-a prezentat deplasarea pe ecran a unor figuri, considerăm că este necesar să se mai prezinte un ultim program de grafică cu o largă utilizare, într-o multitudine de probleme.

EXEMPLUL nr. 15.7. Una din cele mai frecvent întâlnite probleme de programare este cea privind **calculul valorilor unei funcţii y=f(x) într-un interval** [**a** ; **b**] parcurs cu un pas **h**.

O astfel de problemă este extrem de simplu de rezolvat, corespunzător acestei formulări a problemei, fiind necesară parcurgerea următoarei secvenţe de instrucţiuni :

```
.................
int i,n, .... ;  double a,b,h,x,y, ..... ;
.................
n=(b-a)/h+1;
for(i=0; i<n; i++)
  { x=a+i*h;
    y= ......... ;   // dupa semnul egal se scrie expr. functiei
    printf("\n  %3d  %10.3f  %15.7le", i, x, y);
    ...................
  }
.................
```

Evident că această secvenţă de instrucţiuni "clasică", datorită frecvenţiei ei de utilizare, poate fi modificată prin considerarea unei funcţii separate pentru calculul valorii funcţiei **f(x)** şi care returnează valoarea lui **y**, valoarea funcţiei poate rezulta ca urmare a alegerii unei anumite expresii în funcţie de intervalul în care se găseşte abscisa curentă **x**, etc.

Se pune problema ca în afară de calculul valorilor respectivei funcţii să se facă şi reprezentarea grafică a acesteia, fie pe tot ecranul, fie într-o anumită zonă a acestuia.

Este de la sine înţeles că acest lucru (calculul valorilor şi reprezentarea grafică) nu se poate face simultan .

Etapele ce trebuie parcurse sunt următoarele :

- introducerea datelor şi determinarea numărului de puncte,
- calculul valorilor absciselor **x** şi a ordonatelor **y**, afişarea acestora şi reţinerea lor în două şiruri : X[0], X[1], X[2], ... şi Y[0], Y[1], Y[2], ... ,
- determinarea valorilor minime şi maxime ale elementelor celor două şiruri **X** şi **Y**,
- în funcţie de zona dreptunghiulară a ecranului ce se alocă pentru reprezentarea grafică şi de valorile minime şi maxime anterior determinate se stabilesc scările de reprezentare pe orizontală şi pe verticală ale graficului funcţiei,
- pentru fiecare punct care se va reprezenta pe ecran se determină coordonatele acestuia pe ecran, fie recalculând valorile elementelor şirurilor **X** şi **Y**, fie formând noi şiruri,
- se intră în modul grafic, se trasează axele de coordonate, se trasează curba reprezentativă a funcţiei, se afişează diferite alte informaţii, etc.

Pentru ca programul ce se va prezenta să aibă un caracter cât mai general acesta, în afară de funcţia **main( )** , va mai conţine următoarele funcţii:

- funcţia **sig** în care se determină valoarea funcţiei f(x) corespunzătoare unui **x** dat, această funcţie returnând valoarea calculată **y**, în program s-a luat ca exemplu ***funcţia sigmoidală*** având expresia :

$$y = sig\ (x-\theta) = \frac{1}{1+e^{-(x-\theta)}}$$

(utilizată ca funcţie uzuală de activare în teoria reţelelor neuronale), pentru parametrul **θ** considerând valoarea **θ =** 0.15 .

- funcţia în care se determină valorile minime şi maxime ale elementelor unui şir de valori de tip real, care se va apela odată, pentru şirul **Y** , numele ei fiind **mMrow**,

- funcţia cu care se face trasarea propriu-zisă a graficului, având numele **graf_sig** şi care se găseşte în fişierul ***graf_sig.cpp***,

Programul sursă este prezentat în continuare, în ***primul fişier*** fiind conţinută funcţia **main( )** şi cele două funcţii **sig(...)** şi **mMrow(...)** iar în al ***doilea fişier*** funcţia **graf_sig(...)** .

Conţinutul ***primului fişier*** este următorul :

```
#include <conio.h>
#include <dos.h>
#include <graphics.h>
#include <math.h>
#include <stdio.h>

#include "graf_sig.cpp"

void mMrow(int n, double T[], double *min, double *MAX);
double sig(double x, double theta);
```

```
int main(void)
{ int i, j=0, k=1, n, nn;
  double a, b, h, P[10], theta=0.15, X[800], x, xmax, xmin,
         Y[800], y, ymax, ymin;

clrscr();
printf("\n\t a = "); scanf("%le", &a); printf("\t b = ");
scanf("%le", &b); printf("\t h = ");   scanf("%le", &h);
P[0]=theta; n=(b-a)/h+1;
if(n>800) k=n/800+1;

     // Se face afisarea valorilor functiei. Dupa afisarea
     // a cate 20 de linii trebuie sa se apese o tasta.
     // Se atribuie valori elementelor sirurilor  X  si  Y.
for(i=0; i<n; i++)
   { x=a+i*h; y=sig(x,theta);
     if(i%20==0)
       { getch();
         printf("\n\n\t Functia sigmoidala   [ \351 = %6.3f ]"
             "\n\n\t Nr.crt.     x          sig(x,\351)\n", theta);
       }
     printf("\n\t %4d  %10.3f  %15.6le", i, x, y);
     if(i%k==0) { X[j]=x; Y[j]=y; nn=j; j++; }
   }
if(n>800) n=nn;

     // Se determina si se afiseaza valorile minime si maxime
     // ale absciselor si ordonatelor functiei.
xmin=a; xmax=b; mMrow(n,Y,&ymin,&ymax);
printf("\n\n\n Val. min. = %13.6le  val. max. = %13.6le", ymin,
ymax);

     // Se apeleaza functia pentru trasarea graficului.
graf_sig(n,X,Y,xmin,xmax,ymin,ymax,P);

getch(); return 0;
}

void mMrow(int n, double T[], double *min, double *MAX)
{ int i;  *min=*MAX=T[0];
  for(i=0; i<n; i++)
     { if( T[i] < *min ) *min=T[i];
       if( T[i] > *MAX ) *MAX=T[i]; }
}

double sig(double x, double theta)
{ return 1/(1+exp(-x+theta)); }
```

Conținutul celui de-al ***doilea fişier*** este :

```
void graf_sig(int np,double TX[],double TY[],double TXmin,
           double TXmax,double TYmin,double TYmax,double P[])
{ int gd=DETECT,gm,i;
  double DX,DY,theta,xD,XO,yD,YO;
```

```
  char sirTXmin[50],sirTXmax[50],sirTYmin[50],sirTYmax[50],
       sirtheta[50];

theta = P[0];
sprintf(sirTXmin,"%4.3e",TXmin); sprintf(sirTXmax,"%4.3e",TXmax);
sprintf(sirTYmin,"%4.3e",TYmin); sprintf(sirTYmax,"%4.3e",TYmax);

initgraph(&gd, &gm, " ");
setbkcolor(LIGHTGRAY); settextstyle(0,HORIZ_DIR,1);
settextjustify(LEFT_TEXT,TOP_TEXT); setcolor(BLUE);

     // Urmatoarele sase randuri se vor elimina (sau inlocui) in
     // cazul reprezentarii unei alte functii.
outtextxy(285,10, "Reprezentarea grafica a functiei sigmoidale");
outtextxy(300,25, "                 1       ");
outtextxy(300,35, "sig(x,\351) = ------------- ,   \351 = ");
outtextxy(300,45, "           1 + exp(-x+\351)");
sprintf(sirtheta,"%4.2f",theta); outtextxy(565,35,sirtheta);
delay(250);

     // Se stabileste pozitia pe ecran a axelor de coordonate,
     // in functie de valorile minime si maxime ale absciselor
     // si ordonatelor.
if(TXmin>=0.0) { XO= 80; DX=TXmax; }
if(TXmax< 0.0) { XO=560; DX=-TXmin; }
if(TXmin*TXmax<0.0)
  { XO=80-480*TXmin/(TXmax-TXmin); DX=TXmax-TXmin; }
if(TYmin>=0.0) { YO=400; DY= TYmax; }
if(TYmax< 0.0) { YO= 80; DY=-TYmin; }
if(TYmin*TYmax<0.0)
  { YO=80+320*TYmax/(TYmax-TYmin); DY=TYmax-TYmin; }

     // Se traseaza axa Ox si sageata de la capatul ei.
line(50,YO,590,YO); moveto(590, YO+2); linerel(0,-4);
linerel(10,2); linerel(-10,2); outtextxy(592,YO-12,"x");
delay(250);

     // Se traseaza axa Oy si sageata de la capatul ei.
line(XO,50,XO,430); moveto(XO-2,50); linerel(4,0);
linerel(-2,-10); linerel(-2,10); outtextxy(XO+6,40,"y");
delay(250);

outtextxy(XO+5,YO-10,"O"); delay(250);

setcolor(RED);
     // Se afiseaza valorile minime si maxime ale abscisei.
xD=XO+TXmin*480/DX; line(xD,YO-2,xD,YO+2);
outtextxy(xD-25,YO+5,sirTXmin);
xD=XO+TXmax*480/DX; line(xD,YO-2,xD,YO+2);
outtextxy(xD-25,YO+5,sirTXmax); delay(250);

     // Se afiseaza valorile minime si maxime ale ordonatei.
yD=YO-TYmin*320/DY; line(XO-2,yD,XO+2,yD);
outtextxy(XO-78,yD-8,sirTYmin);
yD=YO-TYmax*320/DY; line(XO-2,yD,XO+2,yD);
```

```
outtextxy(XO+4,yD-4,sirTYmax); delay(250);

     // Se traseaza graficul functiei, recalculandu-se valorile
     // absciselor si ordonatelor, tinand cont de scarile adoptate
     // precum si de pozitia pe ecran a originii sistemului de
     // coordonate.
for(i=0; i<np; i++)
   { xD=XO+TX[i]*480/DX; yD=YO-TY[i]*320/DY;
     if(i==0) moveto(xD,yD);
     if(i>0) lineto(xD,yD); delay(100);
   }
getch(); closegraph();
}
```

Ca urmare a execuției acestui program, considerând următoarele date numerice

$$a = -5.0\ ;\quad b = 10.0\ ;\quad h = 0.25\ ;\quad \theta = 0.15\ ;$$

s-a obținut reprezentarea grafică din fig. 15.5

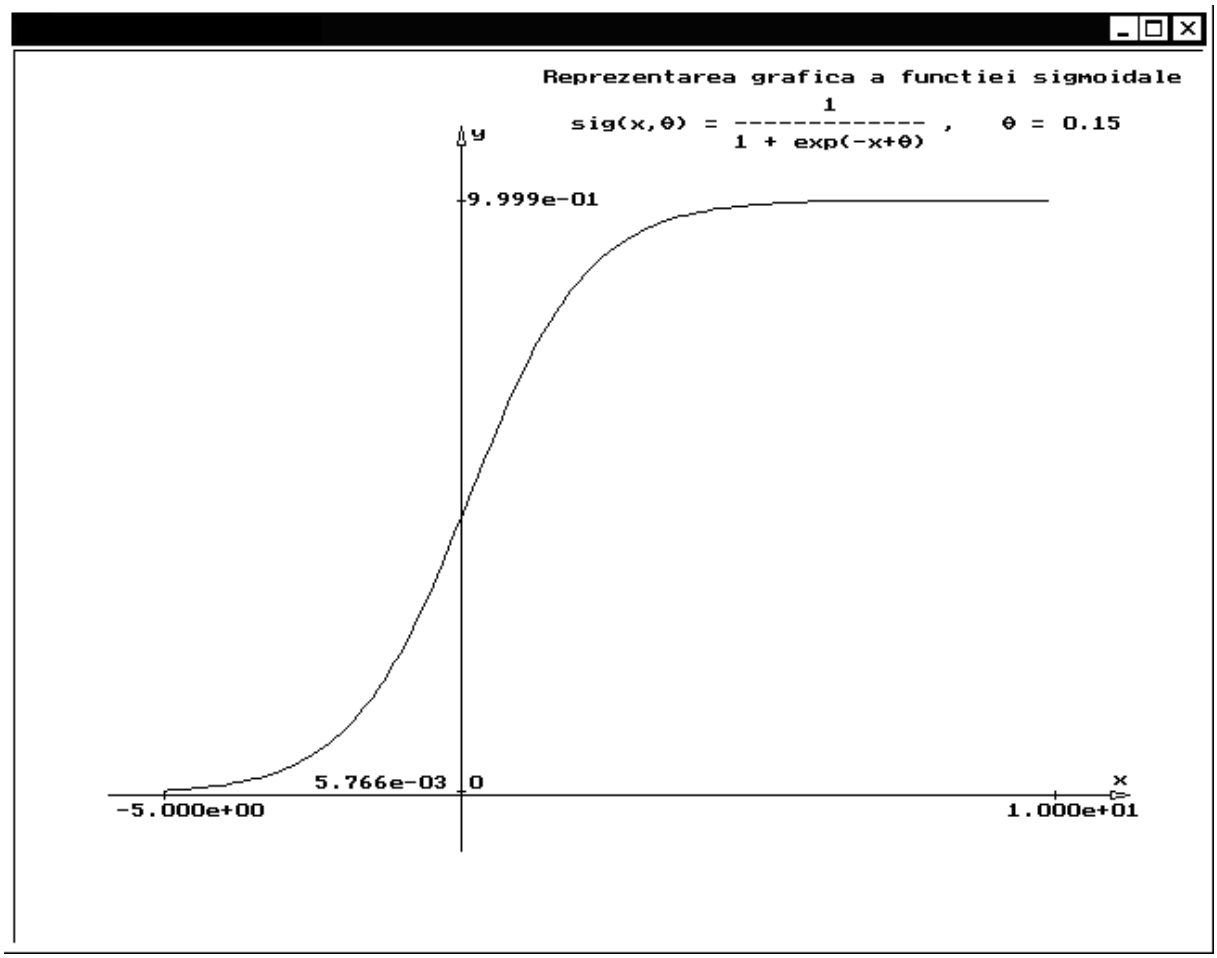


Fig. 15.5

Observaţie. În cazul în care ar trebui făcută reprezentarea grafică a unei funcţii care este dată prin ecuații parametrice :

$$x = f(\varphi) ; \quad y = g(\varphi) ;$$

în al doilea fişier, care conţine funcţia **graf_sig(...)** , se va face o modificare minoră, eliminând câteva linii – cele specificate în programul prezentat .

În primul fişier se vor modifica următoarele :

- funcţia **sig(...)** , eventual cu un alt nume, va trebui să fie rescrisă deoarece în acest caz trebuie să returneze două valori, deci va conţine doi pointeri ca argumente,

- în funcţia **main( )** va trebui apelată de două ori funcţia **mMrow(...)**, odată pentru a determina minimul şi maximul elementelor şirului **X** iar a doua oară pentru şirul **Y**,

- a, **b** şi **h** vor avea alte semnificaţii, fiind valorile iniţială, finală şi pasul de variaţie al parametrului **φ .**

Capitolul XVI

Probleme de programare propuse pentru rezolvare

Pe parcursul capitolelor anterioare fiecare din problemele legate de limbajul de programare au fost exemplificate, considerăm noi, în suficientă măsură pentru ca cititorul să poată înţelege bine atât teoria cât şi problemele complet prezentate.

A învăţa să scrii programe într-un anumit limbaj de programare, **C/C++** sau altul, nu poate fi conceput fără a încerca să rezolvi cât mai multe probleme, aparţinând unor domenii cât mai diferite. Din acest motiv în acest capitol se propun serie de probleme, din diverse domenii, care în mod normal sub aspect matematic nu ar trebui să ridice probleme.

16.1 Operaţii cu şiruri

16.1.1 Se consideră un şir format din **n** termeni, $\mathbf{a_1, a_2, ... , a_n}$. Se va elabora un program în cadrul căruia se vor realiza următoarele: introducerea interactivă a datelor (valoarea lui **n** şi a elementelor şirului), determinarea valorilor minime şi maxime ale elementelor şirului precum şi a poziţiilor acestora, date prin indici (dacă sunt mai multe elemente cu aceeaşi valoare, minimă sau maximă, se va lua în considerare valoarea indicelui primului astfel de element), afişarea valorilor minimă şi maximă a elementelor precum şi a indicilor acestora, se vor schimba între ele valorile elementelor $\mathbf{a_1}$ şi $\mathbf{a_n}$ şi se va afişa şirul astfel modificat.

16.1.2 Se consideră un şir format din **n** termeni, $\mathbf{a_1, ... , a_n}$, cu elemente reale. Să se facă ordonarea crescătoare (sau descrescătoare) a termenilor acestui şir utilizând metoda selecţiei, pe baza schemei logice din Cap. II. Se va face afişarea atât a şirului iniţial cât şi a şirului ordonat.

16.1.3 Se consideră un şir format din **n** termeni, $\mathbf{a_1, ... , a_n}$, cu elemente reale. Să se facă ordonarea crescătoare a termenilor acestui şir utilizând metoda inserării. Procedeul este următorul : se compară pe rând fiecare pereche de elemente dacă este îndeplinită condiţia, începând cu $a_1 \le a_2$, apoi $a_2 \le a_3$, ... , $a_{i-1} \le a_i$ până când la o anumită pereche se obţine $a_i > a_{i+1}$ ceea ce înseamnă că elementul a_{i+1} nu este într-o poziţie corespunzătoare; se ia subşirul a_1 , ... , a_i şi elementul a_{i+1} care se inserează în cadrul acestui subşir, fie la începutul său, dacă $a_{i+1} \le a_1$ sau ca element intermediar dacă $a_1 < a_{i+1} \le a_i$, ca urmare a acestei inserări obţinându-se subşirul a_1 , ... , a_{i+1} ordonat crescător; urmează în continuare să comparăm ultimul element al acestui subşir (a_{i+1}) cu elementul care urmează, a_{i+2} , reluând acelaşi procedeu; când se ajunge ca aceste operaţii să se facă şi pentru elementul a_n înseamnă că şirul este ordonat crescător. Se va face afişarea atât a şirului iniţial cât şi a şirului ordonat.

16.1.4 Să se calculeze în două feluri primele **n** numerele ale lui Fibonacci (folosind recurenţa şi calculul direct – vezi formulele din Cap. II), să se calculeze sumele numerelor obţinute şi să se afişeze rezultatele.

16.1.5 Să se scrie un program care verifică dacă un număr întreg şi pozitiv , introdus de la tastatură, este număr Fibonacci iar dacă nu este atunci să se determine numerele din şirul lui Fibonacci ale căror sumă să fie egală cu respectivul număr.

16.1.6 Se consideră un şir de **N** numere naturale. Să se determine suma numerelor din şir care aparţin şi şirului lui Fibonacci.

16.1.7 Se consideră şirul de **n** termeni $\mathbf{a_1, a_2 , ... , a_n}$. Să se determine valorile termenilor unui nou şir $\mathbf{b_1, b_2 , ... , b_n}$ care se calculează în felul următor:

$$b_i = \frac{a_i}{1+(a_1+a_2+...+a_i)^2}$$

iar cele două şiruri se vor afişa pe ecranul terminalului pe două coloane.

16.1.8 Cunoscându-se următoarele limite:

$$e=\lim_{n\to\infty}\left(1+\frac{1}{n}\right)^n, \qquad e=\lim_{n\to\infty}\frac{n}{\sqrt[n]{n!}},$$

să se calculeze valorile primilor **15** termeni ai şirurilor:

$$a_i=\left(1+\frac{1}{i}\right)^i, \quad b_i=\frac{i}{\sqrt[i]{i!}}$$

afişându-se pentru fiecare **i** valorile lui $\mathbf{a_i}$ şi $\mathbf{b_i}$ precum şi abaterile acestora de la valoarea exactă a numărului **e**, calculată cu funcţia **exp(1.)**. De asemenea, se va tipări şi valoarea numărului **e**.

16.1.9 Se consideră următoarele şiruri :

$$a_n=\frac{1}{n^5}\left(1^4+2^4+3^4+.....+n^4\right),\ b_n=\frac{1}{n}\left[\frac{9n^2}{9n^2+2^2}+\frac{9n^2}{9n^2+5^2}+...+\frac{9n^2}{9n^2+(3n-1)^2}\right],$$

$$c_n=\frac{1}{n}\left[\sqrt{1+\left(\frac{1}{n}\right)^2}+\sqrt{1+\left(\frac{2}{n}\right)^2}+....+\sqrt{1+\left(\frac{n-1}{n}\right)^2}+\sqrt{1+\left(\frac{n}{n}\right)^2}\right]$$

Se cere să se calculeze şi afişeze valorile primilor 20 de termeni ale fiecărui şir .

16.1.10 Se consideră şirurile :

$$a_1=2,\ a_2=2+\frac{1}{2},\ a_3=2+\frac{1}{2+\frac{1}{2}},\ a_4=2+\frac{1}{2+\frac{1}{2+\frac{1}{2}}},...\ b_n=\frac{\sqrt[n]{2}}{n+\frac{1}{1}}+\frac{\sqrt[n]{2^2}}{n+\frac{1}{2}}+...+\frac{\sqrt[n]{2^n}}{n+\frac{1}{n}}-\frac{1}{\ln 2}.$$

Se va elabora programul pentru calculul valorilor primilor **15** termeni ai acestor şiruri.

16.1.11 Se consideră şirurile:

$$a_n = \frac{1}{1} - \frac{1}{2} + \frac{1}{3} ..... - \frac{1}{2n}; \quad b_n = \frac{1}{n+1} + \frac{1}{n+2} + ... + \frac{1}{2n}.$$

Să se elaboreze programul pentru calculul valorilor primilor **25** de termeni ai acestor şiruri.

16.1.12 Se consideră un şir **X** format din **n** termeni (diferiţi de zero). Să se calculeze şi să se afişeze valoarea expresiei :

$$E = \frac{x_1}{x_n} + \frac{x_2}{x_{n-1}} + ... + \frac{x_n}{x_1}.$$

16.1.13 Se consideră un şir de numere reale x_1 , x_2 , x_3 , ... , x_{54} , x_{55}. Să se calculeze :

$$P = x_1 (x_2 + x_3)(x_4 + x_5 + x_6)(x_7 + x_8 + x_9 + x_{10}) .....(x_{46} + x_{47} + ... + x_{55}).$$

16.1.14 Se consideră un şir de numere reale $\mathbf{x_1 , x_2 , x_3 , ... , x_n}$. Să se calculeze :

$$E_1 = x_1 x_n + x_2 x_{n-1} + ... + x_{n-1} x_2 + x_n x_1$$

$$E_2 = x_n (x_n + x_{n-1})(x_n + x_{n-1} + x_{n-2}).....(x_n + x_{n-1} + ... + x_2 + x_1)$$

16.1.15 Se consideră un şir de numere reale $\mathbf{x_1 , x_2 , x_3 , ... , x_n}$ având valori între 0 şi 1. Să se calculeze :

$$E = x_1 + x_2^2 + x_3^3 + x_4^4 + ... + x_{n-3}^4 + x_{n-2}^3 + x_{n-1}^2 + x_n$$

16.1.16 Un şir de numere naturale se formează conform următoarei reguli:

$$x_1 = x_2 = x_3 = 1, \quad x_i = x_{i-3} + x_{i-1} , \quad i = 4,5,... ,n$$

Pentru un **n** dat să se calculeze :

$$E = \frac{x_1}{2} + \frac{x_2}{2^2} + ... + \frac{x_n}{2^n}.$$

16.1.17 Prin **moda** unui şir de **n** termeni $\mathbf{a_1, a_2 , ... , a_n}$ se înţelege valoarea numerică care are cea mai mare frecvenţă de apariţie în cadrul celor **n** termeni. Se va programa introducerea lui **n** şi a elementelor şirului, se va afişa şirul, se va determina şi tipări valoarea elementului ce apare de cele mai multe ori, de asemenea se va tipări şi numărul de apariţii. În cazul în care sunt două sau mai multe elemente diferite care apar în cadrul şirului de acelaşi număr de ori, se vor tipări toate aceste valori şi frecvenţa lor de apariţie. În program se va prevedea posibilitatea de a efectua introducerea repetată a datelor.

16.1.18 Efectuând în mod repetat o serie de măsurători experimentale ale unei aceleiaşi mărimi **x** se obţine şirul de valori $\mathbf{x_1, x_2, .., x_n}$. Datorită unor erori grosolane de măsurare unele mărimi pot să difere destul de mult faţă de majoritatea celorlalte. Vor trebui eliminate acele valori

pentru care raportul dintre abaterea lor faţă de media aritmetică, luată în valoare absolută şi eroarea medie patratică este mai mare decât un număr **r**, ce depinde de **n**, numărul termenilor din şirul de date experimentale. Acest mod de eliminare al valorilor necorespunzătoare se numeşte ***criteriul lui Chouvenet*** (vezi [Cioc79] pg. 105-106). Programul va realiza următoarele:

- introducerea lui **n**, **r** şi a termenilor şirului iniţial $\mathbf{a_1, a_2, ..., a_n}$ precum şi afişarea lui **r** şi a termenilor şirului introdus,
- calculul mediei aritmetice şi a erorii medii patratice

$$m_{aritm} = \frac{\sum_{i=1}^{n} a_i}{n}, \quad \sigma = \frac{\sqrt{\sum_{i=1}^{n} (m_{aritm} - a_i)^2}}{n},$$

- se formează noul şir $\mathbf{b_1, b_2, ..., b_k}$, (k **§** n), al cărui elemente sunt în ordine elementele şirului $\{a_i\}$, i=1, n, din care s-au eliminat acelea care nu îndeplinesc condiţia:

$$\frac{| a_i - m_{aritm} |}{\sigma} < r ,$$

- se calculează noua medie aritmetică,

$$m^*_{aritm} = \frac{\sum_{j=1}^{k} b_j}{k}$$

- se tipăreşte şirul $\{b_j\}$, j=1,k şi media aritmetică a termenilor acestuia,

Observaţie. În tabelul următor se dau valorile coeficientului **r**, care depind de numărul de termeni **n**

N	5	10	20	30	40	50	100	200	500
R	1.68	1.96	2.24	2.39	2.50	2.58	2.80	3.02	3.29

16.1.19 Se consideră şirul cu elemente reale **{ a_i, i = 1, n }.** Programul ce se va elabora va realiza următoarele: introducerea datelor în regim conversaţional, ordonarea crescătoare a termenilor şirului, utilizând unul din algoritmii cunoscuţi, afişarea şirului ordonat, introducerea unei valori reale **v**, inserarea acestei valori în şirul **{ a_i, i = 1, n }** obţinându-se un şir cu **n+1** elemente, afişarea noului şir format din **n+1** termeni.

16.1.20 Se consideră un şir **A** cu elemente întregi, numărul elementelor fiind **n**. Să se formeze un nou şir **B** care să conţină numai elementele distincte ale şirului **A**. Programul ce se va elabora va asigura introducerea repetată a datelor. Şirurile **A** şi **B** se vor tipări pe două coloane.

16.1.21 Se consideră un şir de **n** numere reale $\mathbf{a_1, a_2, .., a_n}$, dintre care unele pot fi nule. Să se determine un nou şir $\mathbf{b_1, b_2, ..., b_k}$, ($k \le n$), care este format din elementele nenule ale şirului **{a_i}**, (i = 1, n), în ordinea în care apar.

16.1.22 Se consideră un şir format din **n** termeni $\mathbf{a_1 , a_2 , ... , a_n}$. Se cere să se formeze un nou şir având acelaşi identificator, astfel încât ultimele **m** elemente (m<n/2) ale acestuia să ajungă pe primele **m** poziţii, valorile elementelor înlocuite atribuindu-se ultimelor **m** elemente. În program se va prevedea afişarea atât a şirului iniţial cât şi a formei finale a acestuia. Se poate

observa că pentru a realiza această operaţie se vor schimba între ele valorile elementelor : a_1 cu a_{n-m+1} , a_2 cu a_{n-m+2}, ... , a_m cu a_n.

16.1.23 Se consideră şirul $a_1 , a_2 , ... , a_{2n}$. Să se formeze două noi şiruri $b_1 , b_2 , ... , b_{2n}$ şi $c_1 , c_2 , ... , c_{2n}$, care conţin elementele şirului iniţial, dar în altă ordine, după cum urmează:

- şirul b_i : $a_1 , a_{n+1} , a_2 , a_{n+2} , ... , a_n , a_{2n}$,
- şirul c_i : $a_1 , a_{2n} , a_2 , a_{2n-1} , ... , a_n , a_{n+1}$.

Se vor afişa cele trei şiruri.

16.1.24 Pentru cazul şirului $\{ a_i, i=1, n \}$ să se elaboreze un program care să rezolve următoarea problemă: introducerea lui **n** şi a elementelor şirurilor, afişarea şirului, introducerea unei valori **k** ($1 \le k \le n$) care reprezintă valoarea indicelui unui element al şirului, eliminarea din şir a elementului a_k , afişarea noului şir obţinut cu **n-1** termeni, introducerea unei valori reale **v**, compararea acestei valori cu fiecare din termenii şirului anterior obţinut şi eliminarea acelor termeni care sunt egali cu **v**, obţinându-se un nou şir cu **n-m-1** termeni, unde **m** este numărul de termeni din şir care erau egali cu **v** şi au fost eliminaţi, afişarea elementelor şirului obţinut la etapa anterioară.

16.1.25 Se consideră şirul de termeni:

$$\frac{1}{1}, \frac{2}{1}, \frac{1}{2}, \frac{3}{1}, \frac{2}{2}, \frac{1}{3}, \frac{4}{1}, \frac{3}{2}, \frac{2}{3}, \frac{1}{4}, \frac{5}{1}, \frac{4}{2}, ...$$

Să se scrie programul pentru calcularea valorilor primilor **n** termeni ai acestui şir.

16.1.26 Se consideră două şiruri de numere reale, **A** cu **m** termeni şi **B** cu **n** termeni, ambele ordonate crescător. Se va elabora programul cu care se va realiza ***interclasarea*** celor două şiruri, formând un nou şir **C**.

16.1.27 Se dă un şir **A** cu **n** elemente numere întregi. Să se scrie un program care determină ce element se află pe poziţia **k** în cadrul şirului ordonat crescător, fără însă a face ordonarea şi fără a utiliza alte şiruri.

16.1.28 Se consideră un şir **A** cu **n** termeni. Se cere să se determine câte elemente se află în intervalul **[b;c]** precum şi media aritmetică a elementelor care nu aparţin acestui interval.

16.1.29 Se consideră un şir de numere reale format din **n** termeni. Se cere să se determine câte numere din şir au proprietatea că sunt mai mari decât numărul anterior şi decât cel următor.

16.1.30 Se consideră un şir de **n** numere (întregi sau reale). Se cere să se determine primele trei numere cu cele mai mari valori (fără a face aranjarea şirului în ordine descrescătoare).

16.1.31 Se consideră un şir format din **n** termeni, numere reale. Se cere să se determine

de câte ori se schimbă semnul numerelor în cadrul şirului.

16.1.32 Se consideră un şir de **n** numere reale. Se cere să se calculeze media aritmetică precum şi numărul numerelor mai mici respectiv mai mari decât media aritmetică.

16.1.33 Se consideră un şir de **n** numere reale. Se cere să se determine indicele **k** al numărului real din şir care este cel mai apropiat de un număr intreg.

16.1.34 Se consideră două şiruri, **A** cu **m** elemente şi **B** cu **n** elemente, conţinând numere întregi. Să se determine şirul **C** care conţine elementele comune ale celor două şiruri .

16.1.35 Se consideră două şiruri, **A** cu **m** elemente şi **B** cu **n** elemente, conţinând numere întregi. Să se determine şirul **C** care conţine toate elementele distincte ale celor două şiruri.

16.1.36 Se consideră două şiruri, **A** cu **m** elemente şi **B** cu **n** elemente, conţinând numere întregi. Să se determine şirul **C** care conţine elemente distincte ale şirului **A** cu excepţia acelora care aparţin şirului **B** precum şi şirul **D** care conţine elemente distincte ale şirului **B** cu excepţia celor care aparţin şirului **A**.

16.1.37 Se consideră un şir **A** conţinând **n** numere întregi. Se cere să se determine două şiruri, **B** şi **C**, primul conţinând numerele din şirul **A** ordonate crescător iar al doilea numerele ordonate descrescător.

16.1.38 Se consideră un şir **A** de **n** numere reale (**"** a_i **ʃ** 0). Succesiunea semnelor a trei numere consecutive din şir

a_1, a_2, a_3; a_2, a_3, a_4 ; ....., a_i, a_{i+1}, a_{i+2}, ..., a_{n-2}, a_{n-1}, a_n

poate fi una din următoarele : **+ + +** , **+ + -** , **+ - +** , **+ - -** , **- + +** , **- + -** , **- - +** şi **- - -** . Să se determine valorile elementelor unui şir cu **8** termeni care conţin numărul de succesiuni de semne de un anumit tip .

16.1.39 În cazul unui şir de **n** numere reale şi strict pozitive sunt valabile următoarele inegalităţi între media armonică, geometrică şi aritmetică :

$$m_{arm} = \frac{n}{\sum_{i=1}^{n} \frac{1}{a_i}} \le m_{geom} = \sqrt[n]{\prod_{i=1}^{n} a_i} \le m_{aritm} = \frac{\sum_{i=1}^{n} a_i}{n}$$

Se va elabora un program în cadrul căruia se va introduce numărul **n** al termenilor şirului, valorile acestor termeni (se vor lua numai valori pozitive), se vor calcula cele trei medii şi se vor afişa valorile termenilor şirului şi valorile celor trei medii calculate. În urma prezentării celor trei rezultate se va putea constata satisfacerea inegalităţii mediilor.

16.1.40 În cazul unor şiruri **A** şi **B** de **n** numere reale (pozitive) sunt valabile următoarele inegalităţi :

- inegalitatea lui **Cauchy** (Augustin Louis Cauchy 1789-1857),

$$\sum_{i=1}^{n} a_i \, b_i \le \sqrt{\sum_{i=1}^{n} a_i^2} \; \sqrt{\sum_{i=1}^{n} b_i^2} \; ,$$

- inegalitatea lui **Cauchy-Hölder** (O. Hölder 1859-1937),

$$\sum_{i=1}^{n} a_i\, b_i \le \left(\sum_{i=1}^{n} a_i^m \right)^{\frac{1}{m}} \left(\sum_{i=1}^{n} b_i^k \right)^{\frac{1}{k}}, \quad \forall a_i, b_i > 0, \quad m, k > 1, \ \frac{1}{m} + \frac{1}{k} = 1,$$

- inegalitatea lui **Minkowski** (Hermann Minkowski 1864-1909),

$$\left[\sum_{i=1}^{n} \left(a_i + b_i \right)^k \right]^{\frac{1}{k}} \le \left[\sum_{i=1}^{n} a_i^k \right]^{\frac{1}{k}} + \left[\sum_{i=1}^{n} b_i^k \right]^{\frac{1}{k}}, \quad k > 1.$$

Se va elabora programul cu care se vor calcula membrii din stânga şi din dreapta ai acestor inegalităţi şi se vor afişa, constatându-se valabilitatea acestor inegalităţi .

16.1.41 Se consideră un şir **A** de **n** de numere întregi, ordonat crescător. Se definesc următorii termeni : ***platou*** – o succesiune de termeni cu valori identice, ***rampă*** – o succesiune de termeni ai căror valori cresc cu câte o unitate. Se va elabora programul cu care se va stabili dacă în cadrul şirului există astfel de situaţii iar dacă da atunci se vor determina numărul de termeni din cel mai "lung" platou respectiv din cea mai lungă rampă.

16.1.42 Se consideră un şir **A** de **n** de numere întregi, ordonat descrescător. Se definesc următorii termeni : ***platou*** – o succesiune de termeni cu valori identice, ***pantă*** – o succesiune de termeni ai căror valori scad cu câte o unitate. Se va elabora programul cu care se va stabili dacă în cadrul şirului există astfel de situaţii iar dacă da atunci se vor determina numărul de termeni din cel mai "lung" platou respectiv din cea mai lungă pantă.

16.1.43 Numerele lui **Bernoulli** sunt definite în modul următor :

$$B_1 = 1;\ \frac{B_1}{2} + \frac{B_2}{1} = 0;\ \frac{B_1}{6} + \frac{B_2}{2} + \frac{B_3}{2} = 0;\ \frac{B_1}{24} + \frac{B_2}{6} + \frac{B_3}{4} + \frac{B_4}{6} = 0;$$

relaţia de recurenţă fiind următoarea:

$$\frac{B_1}{k!} + \frac{B_2}{(k-1)!1!} + \frac{B_3}{(k-2)!2!} + \ldots + \frac{B_k}{1!(k-1)!} = 0, \quad \sum_{i=1}^{k} \frac{B_i}{(k-i+1)!\,(i-1)!} = 0.$$

Considerând cunoscute relaţiile anterioare, se cere să se scrie un program pentru calculul valorilor primelor 25 de numere ale lui Bernoulli.

16.1.44 Să se elaboreze un program (schema logică este prezentată în Cap. II) pentru determinarea elementelor şirurilor $\mathbf{a_0=1,\ a_1\ ,\ \ldots\ ,\ a_{n-1}}$ şi $\mathbf{b_0=1,\ b_1\ ,\ \ldots\ ,\ b_{n-1}}$ utilizându-se următoarele relaţii de recurenţă:

$$\sum_{j=0}^{m} \frac{a_j}{m-j+1} = 1, \quad \sum_{j=0}^{m} \frac{b_j}{m-j+1} = 0, \quad (m = \overline{1, n-1}),$$

16.2 Operaţii cu matrice

16.2.1 Se consideră o matrice patratică $\mathbf{A_{n,n}}$ cu elemente reale. Se se determine un şir $\mathbf{B_n}$ al cărui elemente sunt definite în felul următor :

$$b_i = \begin{cases} 1 & \sum_{j=1}^{i} a_{i,j} > \sum_{j=1}^{i} a_{j,i} \\ -1 & in\ \ caz\ \ contrar \end{cases}$$

16.2.2 Se consideră o matrice **A** cu **m** linii şi **n** coloane, elementele căreia au valorile **0** sau **1**. Să se determine poziţia celei mai mari submatrice care este formată numai din elemente cu valori **1** iar apoi poziţia celei mai mari submatrice care este formată numai din valori de **0**. Observaţie: respectivele poziţii sunt definite prin indicii elementelor din stânga sus şi din dreapta jos a acestor submatrice.

16.2.3 O matrice cu **n** linii şi **n** coloane (**n** – impar) care conţine toate numerele naturale de la **1** la $\mathbf{n^2}$ şi la care suma elementelor de pe fiecare linie, coloană şi diagonală este aceeaşi, se numeşte **patrat magic**. Exemplu :

```
6  1  8
7  5  3
2  9  4
```

Să se scrie un program cu care să se verifice dacă o matrice patratică are proprietatea respectivă. În sens mai larg patratul magic este format din $\mathbf{n^2}$ numere naturale care au proprietatea că sumele lor pe linii, coloane sau diagonale sunt identice, de exemplu :

```
4  11  3
5   6  7
9   1  8
```

aici suma elementelor pe linii, coloane, diagonale fiind 18 .

16.2.4 Se consideră o matrice **A** de dimensiuni **m µ n**, cu elemente reale. Se cere să se formeze doi vectori **B** şi **C**, primul conţinând elementele matricei în ordinea de parcurgere linie după linie iar al doilea în ordinea coloană după coloană.

16.2.5 Se cunosc elementele unei matrice patratice **A** cu **n** linii şi coloane. Se cere să se determine elementele unei matrice patratice **B** (utilizând un număr minim de operaţii), care sunt definite în felul următor :

$$b_{i,j} = \frac{a_{i,j} + a_{j,i}}{2}.$$

16.2.6 Să se afişeze toate matricele patratice **4x4** care au câte un **1** pe fiecare linie şi coloană iar restul elementelor sunt **0** .

16.2.7 Se consideră patru matrice patratice **A**, **B**, **C**, **D** de dimensiuni **n¥n** . Se cere să se formeze matricea **E**, de dimensiuni **2n ¥ 2n**, având forma:

$$E = \begin{bmatrix} A & B \\ C & D \end{bmatrix}$$

În cadrul programului se va introduce **n** şi elementele celor patru matrice, se va determina matricea **E** şi se vor afişa matricele **A**, **B**, **C**, **D** şi **E**.

16.2.8 Să se elaboreze programul pentru determinarea aşa numitului "triunghi" al lui **Pascal** (vezi Cap. II) şi să se afişeze pentru cazul numeric **n** = 12.

16.2.9 Se consideră matricea patratică $\mathbf{A_{n,n}}$, cu elemente reale. Se va elabora programul în cadrul căruia se vor introduce valorile lui **n** şi ale elementelor matricei, se va afişa aceasta, se vor determina valorile elementelor minim şi maxim precum şi poziţiile acestora date de cei doi indici de linie şi de coloană. De asemenea se va calcula suma şi produsul elementelor diferite de zero. Toate valorile calculate se vor afişa.

16.2.10 Se consideră o matrice patratică $\mathbf{A_{n,n}}$ şi se cere să se determine valorile termenilor unui şir **S** cu [(n+1)/2] elemente care reprezintă sumele termenilor matricei **A** aflate pe laturile patratelor ce s-ar obţine luând ca vârfuri pe $\mathbf{a_{11}}$, $\mathbf{a_{1,n}}$, $\mathbf{a_{n,1}}$, $\mathbf{a_{n,n}}$ apoi $\mathbf{a_{2,2}}$, $\mathbf{a_{2,n-1}}$, $\mathbf{a_{n-1,2}}$, $\mathbf{a_{n-1,n-1}}$, urmat de $\mathbf{a_{3,3}}$, $\mathbf{a_{3,n-2}}$, $\mathbf{a_{n-2,3}}$, $\mathbf{a_{n-2,n-2}}$ şi aşa mai departe, luate în ordine de la exterior spre interior. În cadrul programului se va face introducerea elementelor matricei **A**, se vor calcula elementele şirului **S**, se va afişa matricea **A** şi şirul **S**.

16.2.11 Se consideră matricea patratică $\mathbf{A_{n,n}}$. Se va elabora programul în cadrul căruia se va calcula media aritmetică a termenilor strict negativi ce se găsesc sub diagonala principală şi media geometrică a termenilor strict pozitivi de deasupra diagonalei secundare. În program se va prevedea afişarea matricei precum şi a celor două medii calculate. Se va avea în vedere şi posibilitatea ca în zonele respective ale matricei considerate să nu existe nici un element care să îndeplinească condiţiile cerute.

16.2.12 Se consideră două matrice **A** şi **B** de dimensiuni egale, cu **m** linii şi **n** coloane. Se cere să se formeze o nouă matrice $\mathbf{C_{m,n}}$ după următoarea regulă:

- linia **i** a matricei **C** va coincide cu linia **i** a matricei **A** dacă $\mathbf{a_{i,1} = b_{i,1}}$
- linia **i** a matricei **C** va coincide cu linia **i** a matricei **B** dacă $\mathbf{a_{i,1} \neq b_{i,1}}$

Se va prevedea afişarea matricelor **A** şi **B** precum şi a noii matrice determinate.

16.2.13 Să se formeze o matrice de dimensiuni **2nµ2n** prin alipirea a patru matrice pătratice de dimensiuni **nµn**, fiecare dintre acestea având elemente identice, elementele submatricelor din stânga sus având valori 1, cele din dreapta sus 2, stânga jos 3 iar dreapta jos 4. Valoarea lui **n** se va lua din intervalul 4 ~ 10.

16.2.14 Se consideră o matrice patratică **A** cu elemente reale. Se va scrie programul care realizează introducerea valorilor lui **n** (**n** va avea o valoare pară) şi a elementelor matricei, afişarea acesteia iar apoi schimbarea între ele a valorilor elementelor matricei iniţiale, astfel încât în noua formă submatricea din partea din stânga sus să ajungă în partea dreaptă sus, rotită cu 90^0 în sens orar, submatricea din dreapta sus să ajungă în dreapta jos, submatricea din dreapta jos să ajungă în stânga jos iar cea din stânga jos să ajungă în dreapta sus, în toate aceste cazuri asigurându-se şi rotirea cu acelaşi unghi.

16.2.15 Se consideră o matrice patrată $\mathbf{A_{n,n}}$, cu elemente reale. Se va scrie un program în cadrul căruia se vor realiza următoarele: introducerea valorii lui **n** şi a elementelor matricei, afişarea matricei **A**, compararea valorilor absolute ale tuturor elementelor care ocupă poziţii simetrice faţă de diagonala principală şi schimbarea (eventuală) între ele a valorilor astfel ca cele cu valoare absolută mai mare să ajungă deasupra diagonalei principale, afişarea matricei astfel obţinute. Valoarea lui **n** se va lua în intervalul 4~8 iar valorile elementelor matricei se vor alege în mod arbitrar.

16.2.16 Se consideră matricea $\mathbf{A_{m,n}}$, cu multe elemente nule (matrice rară - **sparse matrix**). Se cere să se formeze două şiruri $\{ \mathbf{e_i} \}$ şi $\{ \mathbf{S_i} \}$ primul conţinând ***indicii echivalenţi*** ai elementelor nenule iar al doilea valorile acestor elemente. Programul va permite introducerea repetată a lui **m** şi **n**, a matricei **A** şi afişarea matricei **A** şi a şirurilor **E** şi **S**.

16.2.17 Se consideră două şiruri **E** şi **S**, fiecare cu câte **k** elemente, primul conţinând numere naturale iar al doilea fiind un şir de numere reale. Dacă şirul **E** reprezintă şirul indicilor echivalenţi al termenilor nenuli ai unei matrice **A** cu **m** linii şi **n** coloane iar şirul **S** valorile acestor termeni nenuli se cere să se formeze matricea **A**. În cadrul programului se vor introduce **k, m, n**, valorile termenilor şirurilor **E** şi **S**, se va determina matricea **A** şi se vor afişa şirurile **E, S** şi matricea **A.**

16.2.18 Se consideră şirul $\mathbf{a_1}$ **,** $\mathbf{a_2}$ **, ... ,** $\mathbf{a_n}$. Să se formeze o matrice pătratică **B** de dimensiuni **nµn** ale cărei elemente să fie egale cu cele ale şirului, atribuirile făcându-se în modul cum se arată în tabelul următor:

$$
\begin{array}{cccccc}
a_1 & a_2 & a_3 & .. & a_{n-1} & a_n \\
a_2 & a_3 & a_4 & .. & a_n & a_1 \\
a_3 & a_4 & a_5 & .. & a_1 & a_2 \\
.. & .. & .. & & .. & .. \\
a_{n-1} & a_n & a_1 & .. & a_{n-3} & a_{n-2} \\
a_n & a_1 & a_2 & .. & a_{n-2} & a_{n-1}
\end{array}
$$

Se va afişa şirul iniţial şi matricea obţinută.

16.2.19 Să se determine elementele unei matrice patratice care conţin numerele naturale de la **1** la $\mathbf{n^2}$ şi sunt aranjate sub forma unei spirale. Exemplu :

1	2	3	4	5	6
20	21	22	23	24	7
19	32	33	34	25	8
18	31	36	35	26	9
17	30	29	28	27	10
16	15	14	13	12	11

16.2.20 Se consideră o matrice **A** cu **m** linii şi **n** coloane. Să se insereze o linie cu elemente **0** pe locul liniei **i**, toate liniile, începând cu linia **i** coborând şi o coloană cu elemente **0** pe locul coloanei **j**, coloanele începând de la **j** deplasându-se spre dreapta. Să se afişeze noua matrice **A** cu **m+1** linii şi **n+1** coloane.

16.2.21 Se consideră o matrice cu **n** linii şi coloane. Să se determine poziţia elementului minim al matricei şi produsul scalar al liniei şi coloanei la intersecţia cărora se află elementul minim.

16.2.22 Se consideră o matrice cu **n** linii şi coloane. Să se determine poziţia elementelor matricei cu următoarea proprietate: este cel mai mic pe linia pe care se găseşte şi cel mai mare pe coloană.

16.2.23 Se consideră o matrice cu **n** linii şi coloane. Să se determine poziţia elementelor matricei cu următoarea proprietate : este cel mai mare pe linia pe care se găseşte şi cel mai mic pe coloană .

16.2.24 Un patrat format din numere, cu **n** linii şi coloane, al cărui elemente sunt numere întregi cu valori de la **1** la **n** se numeşte **patrat latin** dacă pe fiecare linie şi coloană sunt cifre diferite, ca în exemplul următor :

1 2 3
2 3 1
3 1 2

Se va elabora programul cu care se va verifica dacă o matrice patrată a cărei elemente sunt numere naturale are această proprietate.

16.2.25 Se consideră o matrice patrată **n ¥ n** având ca elemente numere întregi. Se va elabora programul cu care se vor obţine trei noi matrice, rotind matricea iniţială în sens trigonometric cu 90° , 180° şi 270° , tipărind atât matricea iniţială cât şi cele trei matrice obţinute. Exemplu :

```
 3  2  4 12       12 -1  4  5        5  0  9 -9       -9  1  2  3
 2  4  0 -1  90°   4  0  9  0  180°  4  9  7  1  270°  9  7  4  2
 1  7  9  4    →   2  4  7  9   →   -1  0  4  2   →    0  9  0  4
-9  9  0  5        3  2  1 -9       12  4  2  3        5  4 -1 12
```

16.2.26 Se consideră o matrice patratică **A** cu **n** linii şi coloane. Se consideră două numere naturale **m** şi **k**, ambele mai mici sau egale cu **n**. Se cere să se determine matricea din care se elimină linia **m** şi coloana **k**, afişând apoi matricea obţinută , cu **n-1** linii şi **n-1** coloane.

16.2.27 Se consideră o matrice patratică **A** cu **n** linii şi coloane. Se consideră două numere naturale **m** şi **k**, ambele mai mici sau egale cu **n**. Se cere să se determine noua formă a matricei **A** în care elementele de pe liniile **m** şi **k** şi-au schimbat valorile, afişând apoi matricea obţinută .

16.2.28 Se consideră o matrice patratică **A** cu **n** linii şi coloane. Se consideră două numere naturale **m** şi **k**, ambele mai mici sau egale cu **n**. Se cere să se determine noua formă a matricei **A** în care elementele de pe coloanele **m** şi **k** şi-au schimbat valorile, afişând apoi matricea obţinută .

16.2.29 Se consideră o matrice **A** cu **m** linii şi **n** coloane, cu elemente reale. Se cere să se determine două şiruri, **B** şi **C**, ambele cu câte **n** elemente, primul conţinând valorile maxime care se găsesc pe coloanele matricei iar al doilea conţinând valorile minime , de asemenea de pe coloane.

16.2.30 Se consideră un şir de **n** numere reale a_1 , a_2 , ... , a_i , ... , a_n. Se cere să se formeze următoarea matrice utilizând elementele şirului **A**:

$$\begin{bmatrix} a_1 & a_2 & a_3 & .. & a_{n-1} & a_n \\ a_2 & a_2 & a_3 & .. & a_{n-1} & a_n \\ a_3 & a_3 & a_3 & .. & a_{n-1} & a_n \\ .. & .. & .. & .. & .. & .. \\ a_{n-1} & a_{n-1} & a_{n-1} & .. & a_{n-1} & a_n \\ a_n & a_n & a_n & .. & a_n & a_n \end{bmatrix}$$

16.2.31 Se consideră un şir de **n** numere reale a_1 , a_2 , ... , a_i , ... , a_n. Se cere să se formeze următoarea matrice utilizând elementele şirului **A**:

$$\begin{bmatrix} a_1 & a_1 & a_1 & .. & a_1 & a_1 \\ a_1 & a_2 & a_2 & .. & a_2 & a_2 \\ a_1 & a_2 & a_3 & .. & a_3 & a_3 \\ .. & .. & .. & .. & .. & .. \\ a_1 & a_2 & a_3 & .. & a_{n-1} & a_{n-1} \\ a_1 & a_2 & a_3 & .. & a_{n-1} & a_n \end{bmatrix}$$

16.2.32 Se consideră un şir de **n** numere reale a_1 , a_2 , ... , a_i , ... , a_n. Se cere să se formeze următoarea matrice utilizând elementele şirului **A**:

$$\begin{bmatrix} a_1 & 0 & 0 & .. & 0 & 0 \\ a_2 & a_2 & 0 & .. & 0 & 0 \\ a_3 & a_3 & a_3 & .. & 0 & 0 \\ .. & .. & .. & .. & .. & .. \\ a_{n-1} & a_{n-1} & a_{n-1} & .. & a_{n-1} & 0 \\ a_n & a_n & a_n & .. & a_n & a_n \end{bmatrix}$$

16.2.33 Se consideră un şir de **n** numere reale a_1 , a_2 , ... , a_i , ... , a_n . Se cere să se formeze următoarea matrice utilizând elementele şirului **A** :

$$\begin{bmatrix} 0 & 0 & .. & 0 & 0 & a_1 \\ 0 & 0 & .. & 0 & a_2 & a_2 \\ 0 & 0 & .. & a_3 & a_3 & a_3 \\ .. & .. & .. & .. & .. & .. \\ 0 & a_{n-1} & .. & a_{n-3} & a_{n-2} & a_{n-1} \\ a_n & a_n & .. & a_{n-2} & a_{n-1} & a_n \end{bmatrix}$$

16.2.34 Se consideră un şir de matrice dreptunghiulare $\mathbf{A_1}$, $\mathbf{A_2}$, ... , $\mathbf{A_i}$, ... , $\mathbf{A_n}$, care se înmulţesc, presupunându-se că numărul de linii şi coloane ale acestora satisfac condiţiile pentru ca operaţia de înmulţire să fie posibilă. Dacă se notează cu **l1**, **c1**, **l2**, **c2**, ... , **li**, **ci**, ... , **ln**, **cn**

numărul de linii şi coloane al fiecărei matrice, să se determine care este ordinea optimă pentru a efectua înmulţirile matricelor, astfel încât numărul de operaţii necesar să fie minim.

16.2.35 Se consideră o matrice patrată, **A**, cu **n** linii şi coloane. Utilizând următoarele formule posibile pentru determinarea **normei**

$$\| A \|_{Euclidiana} = \sqrt{\sum_{i=1}^{n} \sum_{j=1}^{n} \left| a_{i,j} \right|^2} \quad , \qquad \| A \|_1 = \sum_{i=1}^{n} \sum_{j=0}^{n} \left| a_{i,j} \right| \quad ,$$

$$\| A \|_r = \max_{i=\overline{1,n}} \sum_{j=1}^{n} \left| a_{i,j} \right| , \qquad \| A \|_c = \max_{j=\overline{1,n}} \sum_{i=1}^{n} \left| a_{i,j} \right|$$

$$\| A \|_\infty = n \ \max \left\{ \left| a_{1,1} \right|, \left| a_{1,2} \right|, ..., \left| a_{i,j} \right|, ..., \left| a_{n,n} \right| \right\},$$

să se calculeze aceasta folosind cele cinci formule .

16.2.36 În cazul unei matrice patratice, cu **n** linii şi coloane, cu elemente reale, să se calculeze sumele $\mathbf{S_1}$ şi $\mathbf{S_2}$ ale elementelor de pe diagonala principală ("***urma***" matricei) şi a celor de pe diagonala secundară .

16.2.37 Se consideră o matrice **A** cu **m** linii şi **n** coloane, cu elemente reale. Să se calculeze :

$$E = \sqrt{\prod_{i=1}^{m} \left(\sum_{j=1}^{n} a_{i,j}^2 \right)}$$

16.2.38 Se consideră o matrice **A** cu **m** linii şi **n** coloane. Se va elabora un program cu care se vor determina elementele unei matrice **B** având aceleaşi dimensiuni, conform următoarei legi: elementul $\mathbf{b_{i,j}}$ este egal cu suma elementelor următoarei submatrice formate din elementele matricei **A**:

$$\begin{matrix} a_{1,j} & .. & a_{1,n} \\ .. & .. & .. \\ a_{i,j} & .. & a_{i,n} \end{matrix}$$

16.2.39 Se consideră o matrice **A** cu **m** linii şi **n** coloane . Se va elabora un program cu care se vor determina elementele unei matrice **B** având aceleaşi dimensiuni, conform următoarei legi: elementul $\mathbf{b_{i,\,j}}$ este egal cu suma valorilor absolute a elementelor submatricei formate din elementele matricei **A** :

$$\begin{matrix} a_{i,1} & .. & a_{i,j} \\ .. & .. & .. \\ a_{m,1} & .. & a_{m,j} \end{matrix}$$

16.2.40 Se consideră o matrice **A** cu **m** linii şi **n** coloane. Se va elabora un program cu care se vor determina elementele unei matrice **B** având aceleaşi dimensiuni, conform

următoarei legi: elementul $\mathbf{b_{i,j}}$ este egal cu suma elementelor submatricei de dimensiuni **m-1** şi **n-1**, care se obţine prin eliminarea liniei **i** şi coloanei **j** din matricea **A** .

16.2.41 Se consideră o matrice patratică **A** cu **n** linii şi coloane . Se va elabora un program cu care se vor determina elementele unei matrice **B** având aceleaşi dimensiuni, conform următoarei legi: elementul $\mathbf{b_{i,j}}$ este egal cu suma valorilor elementelor submatricei formate în felul următor:

$$\begin{matrix} a_{i,i} & .. & a_{j,i} \\ .. & .. & .. \\ a_{i,j} & .. & a_{j,j} \end{matrix}$$

16.2.42 Să se formeze o matrice **A** de dimensiuni **n µ n** , a cărei elemente să aibă valori **unitare** dacă sunt pe diagonalele principală, secundară sau deasupra sau dedesubtul acestor diagonale, respectiv valori de **zero** dacă se află în stânga sau în dreapta diagonalelor principală şi secundară. Exemplu:

```
1 1 1 1 1 1 1`
0 1 1 1 1 1 0
0 0 1 1 1 0 0
0 0 0 1 0 0 0
0 0 1 1 1 0 0
0 1 1 1 1 1 0
1 1 1 1 1 1 1
```

16.2.43 Se consideră două matrice patratice **A** şi **B** de dimensiuni **2n µ 2n.** Programul care se va elabora va rezolva următoarele probleme:

- cele patru submatrice de dimensiuni **n µ n** din care este formată matricea **A** îşi vor schimba poziţiile în felul următor – submatricea din stânga sus va trece în dreapta sus, cea din dreapta sus va trece în dreapta jos, cea din dreapta jos va trece în stânga jos, cea din stânga jos în stânga sus,
- cele patru submatrice de dimensiuni **n µ n** din care este formată matricea **B** îşi vor schimba poziţiile în felul următor – submatricea din stânga sus cu cea din dreapta jos iar submatricea din dreapta sus cu cea din stânga jos .

16.2.44 Se consideră o matrice patratică **A** cu **n** linii şi coloane. Să se determine matricele

$$B = \frac{1}{2}\left(A + A^t\right), \quad C = \frac{1}{2}\left(A - A^t\right)$$

Se va afişa matricea **A** şi cele două matrice obţinute **B** (simetrică) şi **C** (antisimetrică) .

16.2.45 Observaţi modul de obţinere al următoarelor patru matrice :

```
1 10 11 20 21    5  4  3  2  1    1  2  6  7 15    1  3  4 10 11
2  9 12 19 22    6  7  8  9 10    3  5  8 14 16    2  5  9 12 19
3  8 13 18 23   15 14 13 12 11    4  9 13 17 22    6  8 13 18 20
4  7 14 17 24   16 17 18 19 20   10 12 18 21 23    7 14 17 21 24
5  6 15 16 25   25 24 23 22 21   11 19 20 24 25   15 16 22 23 25
```

Scrieţi un program cu care să se formeze patru matrice conform regulilor folosite, matricele având dimensiuni **n μ n** , elementele cărora au valori de la **1** la $\mathbf{n^2}$. Se vor afişa aceste matrice în cazul când **n = 10**.

16.2.46 Să se scrie programul pentru calculul următoarei expresii matriceale :

$$\mathbf{E = A\,B + C\,D + U}$$

toate matricele fiind patratice cu **n** linii şi coloane, elementele matricelor **A**, **B**, **C** şi **D** introducându-se de la tastatură iar **U** este o matrice unitară .

16.2.47 Să se scrie un program cu care să se determine o matrice patratică, având următoarea formă:

$$\begin{bmatrix} 1 & 2 & 3 & .. & n-1 & n \\ 2 & 1 & 2 & .. & n-2 & n-1 \\ 3 & 2 & 1 & .. & n-3 & n-2 \\ .. & .. & .. & .. & .. & .. \\ n-1 & n-2 & n-3 & .. & 1 & 2 \\ n & n-1 & n-2 & .. & 2 & 1 \end{bmatrix}$$

16.2.48 Să se scrie un program cu care să se determine o matrice patratică, având următoarea formă:

$$\begin{bmatrix} \frac{1}{1!} & \frac{1}{2!} & .. & \frac{1}{n!} \\ \frac{1}{(1!)^2} & \frac{1}{(2!)^2} & .. & \frac{1}{(n!)^2} \\ .. & .. & .. & .. \\ \frac{1}{(1!)^n} & \frac{1}{(2!)^n} & .. & \frac{1}{(n!)^2} \end{bmatrix}$$

16.2.49 Se consideră o matrice patratică **A** cu **m** linii şi coloane şi un vector coloană **B** cu **m** elemente. Să se calculeze **A B** şi $\mathbf{B^t\,A}$ **.**

16.2.50 Factorialul generalizat $\mathbf{x^{[n]}}$ se defineşte prin:

$$x^{[n]} = x\,(x-1)\,(x-2)\,....\,(x-n+1)$$

şi se poate exprima printr-un polinom de gradul **n** în **x**, conform formulei:

$$x^{[n]} = \sum_{k=1}^{n+1} S^{*}_{n,k}\; x^{k-1}$$

coeficienţii $\mathbf{S^{*}_{n,k}}$ numindu-se numerele lui **Stirling de speţa I** (James Stirling 1692-1770). Se va elabora programul pentru calculul numerelor lui Stirling de speţa I cu ajutorul cărora se pot exprima toate factorialele generalizate de la $\mathbf{x^{[1]}}$ la $\mathbf{x^{[8]}}$ prin polinoame. Schema logică este prezentată în cadrul Exemplului nr. 13 din Capitolul II. Rezultatele se vor afişa sub forma unei matrice:

n\k	1	2	3	4	5	6	7	8	9
1	**0**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
2	**0**	**-1**	**1**	**0**	**0**	**0**	**0**	**0**	**0**
3	**0**	**2**	**-3**	**1**	**0**	**0**	**0**	**0**	**0**
4	**0**	**-6**	**11**	**-6**	**1**	**0**	**0**	**0**	**0**
5	**0**	**24**	**-50**	**35**	**-10**	**1**	**0**	**0**	**0**
6	**0**	**-120**	**274**	**-225**	**85**	**-15**	**1**	**0**	**0**
7	**0**	**720**	**-1764**	**1624**	**-735**	**175**	**-21**	**1**	**0**
8	**0**	**-5040**	**13068**	**-13132**	**6769**	**-1960**	**322**	**8**	**1**

Se poate demonstra [Hamm62] următoarea formulă de recurenţă între elementele matricei:

$$S^*_{n,k} = S^*_{n-1,k-1} - (n-1)\, S^*_{n-1,k}$$

Se observă de asemenea că:

$S_{n,1} = 0,\quad n=1,2,3,..\quad S_{1,k} = 0,\quad k=3,4,..\qquad S_{1,2} = 1.$

16.2.51 În problema anterioară s-a arătat că factorialele generalizate se pot exprima printr-o combinaţie liniară de puteri ale lui x. În mod analog există şi formula: $x^n = \sum_{k=1}^{n} S^{**}_{n,k}\, x^{[k]}$, care permite exprimarea puterilor lui **x** prin factoriale generalizate, coeficienţii $\mathbf{S^{**}_{n,k}}$ numindu-se numerele lui **Stirling de speţa a II-a**. Se va elabora programul pentru determinarea acestor numere, pentru a putea exprima prin factoriale generalizate primele **8** puteri ale lui **x**. Rezultatele se vor tipări sub forma :

n\k	1	2	3	4	5	6	7	8
1	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
2	**1**	**1**	**0**	**0**	**0**	**0**	**0**	**0**
3	**1**	**3**	**1**	**0**	**0**	**0**	**0**	**0**
4	**1**	**7**	**6**	**1**	**0**	**0**	**0**	**0**
5	**1**	**15**	**25**	**10**	**1**	**0**	**0**	**0**
6	**1**	**31**	**90**	**65**	**15**	**1**	**0**	**0**
7	**1**	**63**	**301**	**350**	**140**	**21**	**1**	**0**
8	**1**	**127**	**966**	**1701**	**1050**	**266**	**28**	**1**

Se observă că: $S_{n,1} = 1,\quad n = 1, 2, 3, ...\quad S_{1,k} = 0,\quad k = 2, 3, 4,...$ iar în [Hamm62] se demonstrează formula de recurenţă:

$$S^{**}_{n,k} = S^{**}_{n-1,k-1} + k\, S^{**}_{n-1,k}\,.$$

16.2.52 Se consideră o matrice **A** patratică , cu **n** linii şi coloane. Trebuie găsită matricea $\mathbf{B} = \mathbf{A^m}$, astfel încât numărul de operaţii de înmulţire matriceală să fie minim.

Indicaţii : dacă **m = 11** atunci se va putea scrie :

$$B = (((((((((A \,.\, A) A) A) A) A) A) A) A) A$$

fiind necesar un număr de **10** înmulţiri matriceale . Deoarece **m** se poate scrie sub următoarea formă binară

$$m = 1011_{(2)} = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

rezultă că :

$$B = A^{1011_{(2)}} = A^{2^3} \cdot A^{0 \cdot 2^2} \cdot A^2 \cdot A = A \cdot A^2 \cdot \left(A^2\right)^3$$

fiind în acest caz necesare numai un număr de **şase** înmulţiri matriceale : A **.** A pentru a obţine A^2 (o înmulţire) apoi A^2 **.** A^2 **.** A^2 **.** A^2 (trei înmulţiri) pentru a obţine A^8 iar în final A **.** A^2 **.** A^8 (două înmulţiri).

16.2.53 Se consideră matricele patratice **A** şi **B** cu **n** linii şi coloane . Se cere elaborarea programului `pentru determinarea matricei **C = A B – B A** .

16.3 Calculul unor expresii

16.3.1 Se introduc valorile a trei variabile reale **a, b** şi **c** . Se cere să se facă schimbarea de valori între aceste variabile astfel încât afişându-se **a, b** şi **c** să se obţină un şir de valori în ordine crescătoare (sau descrescătoare) .

16.3.2 Se dă valoarea unui unghi exprimată în radiani. Se cere să se afişeze valoarea unghiului în grade, minute şi secunde.

16.3.3 Se dă valoarea unui unghi exprimată în grade, minute şi secunde. Se cere să se afişeze în radiani.

16.3.4 Se consideră identităţile:

$$C_n^0 + C_n^3 + C_n^6 + \ldots = \frac{1}{3}\left(2^n + 2\cos\frac{n\pi}{3}\right),$$

$$C_n^1 + C_n^4 + C_n^7 + \ldots = \frac{1}{3}\left[2^n + 2\cos\frac{(n-2)\pi}{3}\right],$$

$$C_n^2 + C_n^5 + C_n^8 + \ldots = \frac{1}{3}\left[2^n + 2\cos\frac{(n-4)\pi}{3}\right],$$

Se va elabora câte un program pentru verificarea fiecărei identităţi, pentru toate valorile lui **n** din intervalul **[15;25]** afişând valorile obţinute ale expresiilor din stânga şi din dreapta semnului egal. Valorile combinărilor se vor calcula utilizând o formulă de recurenţă, stabilită între doi termeni succesivi.

16.3.5 Se va rezolva aceeaşi problemă ca şi în cazul anterior, pentru identităţile:

$$C_n^0 + C_n^4 + C_n^8 + \ldots = \frac{1}{2}\left(2^{n-1} + 2^{\frac{n}{2}}\cos\frac{n\pi}{4}\right), \quad C_n^1 + C_n^5 + C_n^9 + \ldots = \frac{1}{2}\left(2^{n-1} + 2^{\frac{n}{2}}\sin\frac{n\pi}{4}\right),$$

16

$$C_n^2 + C_n^6 + C_n^{10} + ... = \frac{1}{2}\left(2^{n-1} - 2^{\frac{n}{2}} \cos\frac{n\pi}{4}\right),\quad C_n^3 + C_n^7 + C_n^{11} + ... = \frac{1}{2}\left(2^{n-1} - 2^{\frac{n}{2}} \sin\frac{n\pi}{4}\right)$$

16.3.6 Să se scrie programul pentru calculul valorilor funcţiilor ***sin(x)*** şi ***cos(x)*** , forma de prezentare a rezultatelor fiind aceea din memoratoarele matematice.

Grade	Sinus	
	0 10 20 30 40 50 60	
0	0.00000 0.00291 0.00582 0.00873 0.01163 0.01454 0.01745	89
1	0.01745 0.02036 0.02327 0.02618 0.02908 0.03199 0.03490	88
--	- -	--
88	0.99939 0.99949 0.99958 0.99966 0.99972 0.99980 0.99985	1
89	0.99985 0.99990 0.99993 0.99996 0.99998 0.99999 1.00000	0
	60 50 40 30 20 10 0	Grade
	Cosinus	

16.3.7 Să se calculeze valorile funcţiilor **tg(x)** şi **cotg(x)** pentru toate valorile argumentului **x** din intervalul **[0;90º]** cu pasul de **10'**. Rezultatele se vor prezenta sub forma tabelului de la problema anterioară.

16.3.8 Valoarea aproximativă a numărului **π** se poate determina cu formula lui **J. Wallis,** dedusă în **1655** (John Wallis 1616-1703)

$$\pi = \left(\frac{8}{3}\right)\left(\frac{4^2}{3.5}\right)\left(\frac{6^2}{5.7}\right)\left(\frac{8^2}{7.9}\right)\left(\frac{10^2}{9.11}\right)....$$

prin calcularea valorii unui produs conţinând o infinitate de factori, [G.M.] nr. 9, 1978. În cadrul programului realizat se vor calcula toate valorile acestui produs luând pe rând câte 1, 2, ... , 20 factori, afişându-se numai acele valori ale lui **π** care rezultă când se consideră câte 5, 10, 15, 20 factori.

16.3.9 Conform unei formule deduse de **Viète** (François Viète 1540-1603), numărul **π** se poate calcula din următorul produs cu un număr infinit de factori:

$$\frac{2}{\pi} = \sqrt{\frac{1}{2}}\ \sqrt{\frac{1}{2}+\frac{1}{2}\sqrt{\frac{1}{2}}}\ \sqrt{\frac{1}{2}+\frac{1}{2}\sqrt{\frac{1}{2}+\frac{1}{2}\sqrt{\frac{1}{2}}}}\ .....$$

Să se tipărească toate valorile aproximative ale numărului **π** ce rezultă prin luarea în considerare pe rând a unui număr de factori de la **1** la **25**. Valorile lui **π** se vor afişa cu cel puţin **9** cifre semnificative.

16.3.10 Se consideră următoarele **fracţii continue**, cu ajutorul cărora se poate exprima valoarea numărului **π** :

$$\frac{4}{\pi}=1+\cfrac{1}{2+\cfrac{3^2}{2+\cfrac{5^2}{2+\cfrac{7^2}{2+\cfrac{9^2}{2+....}}}}},\quad \frac{\pi}{2}=1+\cfrac{2}{3+\cfrac{1.3}{4+\cfrac{3.5}{4+\cfrac{5.7}{4+\cfrac{7.9}{4+...}}}}},\quad \frac{4}{\pi}=1+\cfrac{2}{7+\cfrac{1.3}{8+\cfrac{3.5}{8+\cfrac{5.7}{8+\cfrac{7.9}{8+...}}}}}$$

prima formulă aparţinând lui **W. Brouncker** (dedusă în 1655), iar celelalte două lui **Euler**. Programul va realiza efectuarea calculelor pentru determinarea valorilor numărului π ce rezultă prin aplicarea celor trei formule, luând pe rând un număr de fracţii cuprins între 1 şi 25, rezultatele prezentându-se sub forma unui tabel cu patru coloane. Valorile reale calculate se vor afişa utilizând cel puţin 10 cifre semnificative.

16.3.11 Se consideră următoarele serii numerice pentru calcularea valorii numărului π,

$$\pi = 4\left(\frac{1}{1}-\frac{1}{3}+\frac{1}{5}-\frac{1}{7}+\frac{1}{9}-...\right),$$

$$\pi = 4\left(\frac{3}{4}+\frac{1}{2.3.4}-\frac{1}{4.5.6}+\frac{1}{6.7.8}-\frac{1}{8.9.10}+...\right),$$

$$\pi = 2\left(1+\frac{1}{3}+\frac{1.2}{3.5}+\frac{1.2.3}{3.5.7}+\frac{1.2.3.4}{3.5.7.9}+...\right),$$

$$\pi = 3\frac{\sqrt{3}}{2}\left(1+\frac{1}{6}+\frac{1.2}{6.10}+\frac{1.2.3}{6.10.14}+\frac{1.2.3.4}{6.10.14.18}+...\right),$$

$$\pi = 4\left[\left(\frac{1}{1.2^1}-\frac{1}{3.2^3}+\frac{1}{5.2^5}-\frac{1}{7.2^7}+...\right)+\left(\frac{1}{1.3^1}-\frac{1}{3.3^3}+\frac{1}{5.3^5}-\frac{1}{7.3^7}+...\right)\right],$$

formule datorate lui **G. W. Leibniz**, **J. Bernoulli** şi **L. Euler**. Se va elabora programul pentru calculul valorilor acestor serii considerând în cadrul fiecărei sume pe rând câte 1, 2, 3, ... , 25 de termeni, afişând rezultatele sub forma unui tabel, valorile reale ale variabilelor prezentându-se cu câte 8 cifre semnificative.

16.3.12 Să se calculeze valoarea numărului π cu formula

$$\pi=\left(\frac{16}{1.5}-\frac{4}{239}\right)-\left(\frac{16}{3.5^3}-\frac{4}{239^3}\right)+\left(\frac{16}{5.5^5}-\frac{4}{239^5}\right)-\left(\frac{16}{7.5^7}-\frac{4}{239^7}\right)+....$$

utilizând un număr de **n** paranteze în calcul .

16.3.13 Sunt cunoscute următoarele formule de calcul ale funcţiilor **sin(x), cos(x), sh(x)** şi **ch(x)** prin produse cu un număr infinit de factori [Fiht64], vol. II, pg. 347 - 350,

$$\sin x = x \prod_{i=1}^{\infty}\left(1-\frac{x^2}{i^2\,\pi^2}\right), \qquad \cos x = \prod_{i=1}^{\infty}\left[1-\frac{x^2}{\left(\frac{2i-1}{2}\pi\right)^2}\right],$$

$$sh\,x = x \prod_{i=1}^{\infty}\left(1+\frac{x^2}{i^2\,\pi^2}\right), \qquad ch\,x = \prod_{i=1}^{\infty}\left[1+\frac{x^2}{\left(\frac{2i-1}{2}\,\pi\right)^2}\right].$$

Se va elabora programul pentru calculul valorilor acestor funcţii pentru un **x** dat, utilizând câte 1, 2, 3, ..., 15 factori din produsele de mai sus.

De asemenea se vor calcula şi afişa valorile acestor funcţii folosind funcţiile **sin(x)**, **cos(x)**, **sinh(x)** şi **cosh(x)** din fişierul ***math.h***

16.3.14 Se consideră cunoscută formula lui **Stirling** pentru aproximarea factorialului:

$$n!\ \approx\ \left(\frac{n}{e}\right)^n\ \sqrt{2\,\pi\,n}\ .$$

Se va elabora programul pentru calculul exact şi aproximativ al factorialului, pentru toate valorile lui **n** din intervalul [5 ; 15] .

16.3.15 Se consideră următoarea expresie a funcţiei **tg(x)** , utilizând fracţii continue :

$$tg(x) = \cfrac{x}{1 - \cfrac{x^2}{3 - \cfrac{x^2}{5 - \cfrac{x^2}{7 - \cfrac{x^2}{2n+1 - \cfrac{x^2}{\ddots}}}}}}$$

linia de fracţie intervenind de **n+1** ori. Pentru un **x** şi **n** dat se cere să se calculeze valoarea acestei funcţii şi să se afişeze **x, n** şi valoarea funcţiei, utilizând fracţiile şi cea dată de funcţia ***tan(x)***.

16.3.16 Polinomul lui **Cebâşev** (Pafnutii Lvovici Cebâşev 1821-1894) de cea mai bună aproximaţie, de gradul **n**, se defineşte în felul următor:

$$\mathbf{T_n(x)=\cos(n\ \arccos(x)),\qquad x \in [\,-1,\,+1\,],}$$

de aici rezultând expresiile primelor două polinoame, de gradul **0** şi **1**, $\mathbf{T_0(x) = 1}$, $\mathbf{T_1(x) = x}$ precum şi relaţia de recurenţă: $\mathbf{T_i(x) = 2\ x\ T_{i-1}(x) - T_{i-2}(x)}$, **(i=2,3,4...)** . Să se calculeze valorile polinoamelor lui Cebâşev, de la gradul 1 până la gradul 8, pentru toate valorile variabilei independente **x** din intervalul **[-1; 1],** cu pasul **0.25**. Rezultatele se vor prezenta sub formă tabelară, pe fiecare linie afişând valoarea lui **x** şi a polinoamelor.

16.3.17 Se cunoaşte următoarea formulă de recurenţă pentru calculul **polinoamelor lui Hermite** (Charles Hermite 1822-1901), de gradul **n** : $\mathbf{H_n(x) = 2\ x\ H_{n-1}\ (x) - 2\ (n-1)\ H_{n-2}\ (x)}$. Polinoamele lui Hermite de gradul zero şi unu au următoarele expresii : $\mathbf{H_0(x) = 1}$, $\mathbf{H_1(x) = 2\ x}$. Se cere să se elaboreze programul pentru calculul valorilor polinoamelor lui Hermite de la gradul **doi** până la gradul **opt**, pentru toate valorile variabilei **x** din intervalul **[-15.,+15.],** cu pasul **0.25**. Rezultatele se vor prezenta sub formă tabelară, pe fiecare rând afişându-se valorile lui **x** şi a polinoamelor.

16.3.18 Se consideră şirul $\{ \mathbf{a_i} \}$, i=1, n , format din **n** termeni, elementele căruia intervin în determinantul **Vandermonde** :

$$V_n = \begin{vmatrix} 1 & 1 & 1 & ... & 1 \\ a_1 & a_2 & a_3 & ... & a_n \\ a_1^2 & a_2^2 & a_3^2 & ... & a_n^2 \\ - & - & - & - & - \\ a_1^{n-1} & a_2^{n-1} & a_3^{n-1} & ... & a_n^{n-1} \end{vmatrix}$$

formula de calcul a valorii determinantului fiind următoarea :

$$V_n = \prod_{1 \le j < i \le n} (a_i - a_j) = (a_2 - a_1)\,(a_3 - a_1)\,(a_4 - a_1)\ .....\ (a_n - a_1)$$
$$(a_3 - a_2)\,(a_4 - a_2)\ .....\ (a_n - a_2)$$
$$.....$$
$$(a_{n-1} - a_{n-2})\ (a_n - a_{n-2})$$
$$(a_n - a_{n-1}).$$

Se cere să se elaboreze programul în cadrul căruia să se realizeze introducerea lui **n** şi a termenilor şirului, calculul elementelor determinantului şi afişarea acestuia, calculul şi afişarea valorii determinantului.

16.3.19 Se consideră funcţia **y** definită în felul următor

$$y = \begin{cases} \sqrt[3]{x^3 - 3x^2}, & x \le 10, \\ \dfrac{\arcsin(x-15)}{x^2+1}, & x > 10, \end{cases}$$

şi se cere să se calculeze valorile acestei funcţii în intervalul **[a, b]**, cu pasul **p**, pentru fiecare punct al acestuia afişându-se numărul de ordine, abscisa şi valoarea funcţiei (în cazul când aceasta este definită).

16.3.20 Pentru o valoare dată a lui **n** (între 1 şi 20), să se calculeze produsele

$$P_1 = \frac{1}{1}\frac{3}{2}\frac{5}{3}\frac{7}{4}\ldots\ldots, \quad P_2 = \frac{1}{2}\frac{3}{4}\frac{5}{6}\frac{7}{8}\ldots\ldots, \quad P_3 = \frac{2}{1}\frac{2}{3}\frac{4}{3}\frac{4}{5}\frac{6}{5}\frac{6}{7}\frac{8}{7}\frac{8}{9}\ldots\ldots,$$

fiecare conţinând **n** factori.

16.3.21 Sunt cunoscute următoarele inegalităţi :

$$E_1 = \frac{1}{n+1} + \frac{1}{n+2} + \frac{1}{n+3} + \ldots + \frac{1}{3n+1} > 1 \qquad , \quad n \geq 1$$

$$E_2 = \frac{8}{5}\left(\frac{1}{4n+1} + \frac{1}{4n+2} + \frac{1}{4n+3} + \ldots + \frac{1}{8n}\right) > 1, \quad n \geq 1$$

$$E_3 = \frac{5}{3}\left(\frac{1}{5n+1} + \frac{1}{5n+2} + \frac{1}{5n+3} + \ldots + \frac{1}{10n}\right) > 1, \quad n \geq 1$$

Se va elabora programul pentru calculul şi tipărirea valorilor expresiilor $\mathbf{E_1}$, $\mathbf{E_2}$ şi $\mathbf{E_3}$ pentru toate valorile lui **n** din intervalul **[1 ; 20]** .

16.3.22 Se consideră sumele:

$$S_1 = \frac{7}{1.8} + \frac{7}{8.15} + \frac{7}{15.22} + \ldots + \frac{7}{(7n-6)(7n+1)} + \frac{1}{7n+1},$$

$$S_2 = 16\left[\frac{1}{4.8} + \frac{1}{8.12} + \frac{1}{12.16} + \ldots + \frac{1}{4n(4n+4)}\right], \quad S_3 = \frac{3}{2} + \cos\frac{2\pi}{2n+1} + \cos\frac{4\pi}{2n+1} + \ldots + \cos\frac{2n\pi}{2n+1}$$

Să se calculeze valorile acestor sume, pentru toate valorile lui **n** din intervalul **[5 ; 15]**. Rezultatele se vor tipări sub forma a patru coloane, prima conţinând valorile lui **n** iar celelalte trei valorile calculate ale sumelor pentru fiecare **n**.

16.3.23 Se consideră suma formată din **10000** termeni:

$$S_1 = \frac{1}{1} - \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{9999} - \frac{1}{10000}$$

precum şi sumele formate din **5000** de termeni :

$$S_2 = \frac{1}{1} + \frac{1}{3} + \frac{1}{5} + ... + \frac{1}{9999}, \qquad S_3 = \frac{1}{2} + \frac{1}{4} + \frac{1}{6} + ... + \frac{1}{10000}$$

Programul va realiza calculul următoarelor sume : $\mathbf{S_{1(s-d)}}$ - suma elementelor din $\mathbf{S_1}$ luate de la stânga la dreapta, $\mathbf{S_{1(d-s)}}$ - suma elementelor din $\mathbf{S_1}$ luate de la dreapta spre stânga, $\mathbf{S_{2(s-d)}}$, $\mathbf{S_{2(d-s)}}$ - sumele elementelor din $\mathbf{S_2}$ luate în cele sensuri de parcurgere, respectiv $\mathbf{S_{3(s-d)}}$, $\mathbf{S_{3(d-s)}}$, $\mathbf{S_4 = S_{2(s-d)} - S_{3(s-d)}}$, $\mathbf{S_5 = S_{2(d-s)} - S_{3(d-s)}}$ şi se vor explica rezultatele obţinute.

16.3.24 Să se elaboreze programul pentru verificarea dublei inegalităţi, cunoscute din manualele de algebră :

$$\sqrt{n} < \frac{1}{\sqrt{1}} + \frac{1}{\sqrt{2}} + \frac{1}{\sqrt{3}} + \ ... \ + \frac{1}{\sqrt{n}} < 2\sqrt{n} \ , \ (n \geq 2) \ ,$$

pentru toate valorile lui **n** cuprinse în intervalul [2; 20] , afişând valorile celor trei termeni ai dublei inegalităţi .

16.3.25 Se consideră sumele :

$$S_1 = \frac{3}{2} + \frac{5}{4} + \frac{7}{8} + \frac{9}{16} + ..., \qquad S_2 = (1) + (1+2) + (1+2+3) + (1+2+3+4) + ...,$$

prima având **n** termeni iar a doua **n** paranteze. Programul ce se va elabora va realiza afişarea valorilor acestor sume pentru numărul de termeni (paranteze) de la **1** la **n**, valorile reale afişându-se cu cel puţin şase cifre semnificative.

16.3.26 Utilizând metoda inducţiei complete se poate demonstra existenţa următoarelor inegalităţi:

$$S_n = \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + ... + \frac{1}{n!} \leq \frac{5n-2}{2n} \ , \ ptr. \ n \geq 1 \ ,$$

$$S_n = \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + ... + \frac{1}{n!} \leq \frac{2n-1}{n} \ , \ ptr. \ n \geq 4 \ .$$

Se va realiza un program pentru verificarea acestor inegalităţi pentru valorile lui **n** din intervalul **[1;15].** Rezultatele se vor afişa pe patru coloane, în ordine : valorile lui n, valorile sumei, valorile termenului (5n-2)/2n urmate pe coloana a patra de valorile termenului (2n-1)/n dacă n ¥ 4 .

16.3.27 Se consideră un număr complex **z** dat sub forma algebrică, **z = a+ib**. Se cere să se realizeze programul pentru calculul valorilor rădăcinilor de ordinul **n** ale acestui număr complex, programându-se calculele:

Toate valorile reale se vor afişa cu cinci cifre semnificative. Argumentele celor **n** rădăcini se vor exprima în **grade**. La determinarea valorii argumentului φ a numărului **z** se va ţine cont de poziţia în planul complex a imaginii geometrice a acestui număr complex.

$$\xi_k = \sqrt[n]{\rho} \left[\cos\frac{\varphi + 2\pi(k-1)}{n} + i \sin\frac{\varphi + 2\pi(k-1)}{n} \right], \quad (k = \overline{1,n})$$

16.3.28 Se consideră sumele de combinări :

$$S_1 = C_{2n}^0 - C_{2n-1}^1 + C_{2n-2}^2 - ..... + (-1)^n \; C_n^n ,$$
$$S_2 = \left(C_n^0\right)^2 - \left(C_n^1\right)^2 + \left(C_n^2\right)^2 - .....+ (-1)^n \left(C_n^n\right)^2 ,$$
$$S_3 = \left(C_n^1\right)^2 + 2\left(C_n^2\right)^2 + 3\left(C_n^3\right)^2 + ... + n\left(C_n^n\right)^2 .$$

Se va elabora un program pentru calculul acestor sume pentru toate valorile lui **n** din intervalul [5;10]. Valorile combinărilor se vor calcula utilizând relaţii de recurenţă, ce se vor stabili între doi termeni succesivi.

16.3.29 Să se calculeze :

$$E = \sqrt{3 + \sqrt{6 + \sqrt{9 + ... + \sqrt{96 + \sqrt{99}}}}}$$

16.3.30 Să se calculeze :

$$E = \sqrt{1 + \sqrt{1 + \sqrt{1 + ... + \sqrt{1 + \sqrt{1}}}}}$$

ştiind că în partea dreaptă a semnului egal este scris radicalul de **n** ori . Valoarea lui **n** se va lua între 10 şi 20 .

16.3.31 Se consideră intervalul (**ln a** , $\mathbf{e^a}$) unde **a>1**. Se cere să se determine câte numere naturale se găsesc în acest interval şi suma acestora.

16.3.32 Să se calculeze valoarea numărului **e** folosind fracţia continuă:

$$e = 1 + \cfrac{1}{1 - \cfrac{1}{2 + \cfrac{1}{3 - \cfrac{1}{2 + \cfrac{1}{5 - \cfrac{1}{2 + \cfrac{1}{7 - \cfrac{1}{2 + \cfrac{1}{9 - .....}}}}}}}}}$$

considerând **n** fracţii .

16.3.33 Se consideră trei variabile : **a**, **b** şi **c**, a căror valori se introduc de la tastatură. Să se ordoneze (crescător sau descrescător) cele trei valori şi să se decidă dacă ele formează o progresie aritmetică sau geometrică sau nu satisfac respectivele condiţii.

16.3.34 Se consideră următoarea expresie :

$$E(k) = \frac{m}{n} + \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + ... + \frac{1}{k} = \frac{M}{N}$$

unde **m** şi **n** sunt numere naturale date. Se cere să se determine pentru toate valorile lui **k** între **1** şi **25** valorile numărătorului şi numitorului fracţiei **M/N** .

16.4 Probleme de geometrie

16.4.1 În cazul unei sfere de rază **R** (valoare reală) să se calculeze suprafaţa **S** şi volumul **V** al acesteia pentru valorile razei din intervalul [**R**$_1$; **R**$_2$] parcurs cu un pas Δ . Se va afişa numărul de ordine al cazului considerat, **R**, **S** şi **V** .

$$S_{sfera} = 4\pi R^2 , \quad V_{sfera} = \frac{4\pi R^3}{3}$$

16.4.2 Cunoscându-se raza să se calculeze suprafaţa şi volumul sferei, volumul cubului în care este înscrisă, volumul neocupat de sferă din cub, volumul cubului înscris în sferă şi volumul rămas neocupat în interiorul sferei

16.4.3 Se consideră un număr de **n** segmente A_1, A_2,..., A_{n-1}, A_n , de lungimi l_1, l_2,..., l_n, care fac cu orizontala unghiurile **α$_1$, α$_2$, …, α$_n$** (exprimate în grade). Să se determine coordonatele capătului A_n al ultimului segment. Se vor afişa lungimile segmentelor, valorile unghiurilor exprimate în grade şi în radiani precum şi coordonatele calculate.

16.4.4 Se consideră trei segmente având lungimile **a**, **b** şi **c**. Se cere să se determine dacă acestea pot fi laturi ale unui triunghi iar în caz afirmativ tipul triunghiului: echilateral, isoscel şi dreptunghic, isoscel, dreptunghic sau oarecare, afişând textul explicativ corespunzător.

16.4.5 Se consideră cunoscute coordonatele carteziene (René Descartes – Cartesius – 1596-1650) a trei puncte din plan. În cazul în care aceste puncte pot fi vârfurile unui triunghi se cere să se determine dacă acesta este echilateral, isoscel şi dreptunghic, dreptunghic sau oarecare.

16.4.6 Un triunghi dreptunghic are catetele de lungimi **b** şi **c** (valori reale). Se vor determina următoarele: lungimea ipotenuzei **a**, valorile unghiurilor ascuţite **B** şi **C** (în grade), raza cercului înscris în triunghi.

16.4.7 Să se verifice dacă cele trei drepte, date prin ecuaţiile: **a x + b y + c = 0** , **d x + e y + f = 0** , **g x + h y + i = 0** sunt concurente. Coeficienţii **a**, **b**, ... , **i** sunt de tip real.

16.4.8 Considerând două puncte în plan **M$_1$ (x$_1$, y$_1$)** şi **M$_2$ (x$_2$, y$_2$)**, de coordonate cunoscute, să se stabilească dacă acestea se găsesc pe aceeaşi orizontală, pe aceeaşi verticală, sunt confundate sau se găsesc pe o linie oblică, afişându-se textul corespunzător.

16.4.9 Să se scrie programul cu care să se determine dacă trei unghiuri **A**, **B** şi **C** (exprimate în grade) pot fi unghiuri ale unui triunghi. Observaţie. Se vor verifica condiţiile : A>0, B>0, C>0, A+B+C = 180 .

16.4.10 Se consideră trei numere reale şi pozitive **a**, **b** şi **c**. Se va elabora programul în cadrul căruia să se realizeze următoarele: introducerea şi afişarea repetată a celor trei numere,

verificarea condiţiei ca cele trei numere să poată fi valorile lungimilor laturilor unui triunghi, în caz contrar trecându-se la introducerea unui alt set de date, calculul şi afişarea valorilor celor trei unghiuri (în radiani şi grade) a triunghiului format, a lungimilor înălţimilor şi medianelor, a razelor cercurilor înscris şi circumscris triunghiului. Se reamintesc următoarele relaţii, cunoscute din geometria plană:

$$S = \sqrt{p(p-a)(p-b)(p-c)} = \frac{a\,b\,c}{4\,R} = p\,r; \quad p = \frac{a+b+c}{2};$$

$$b^2 + c^2 = 2\,m_a^2 + 2\left(\frac{a}{2}\right)^2, \quad (t.\ medianei); \quad \cos A = \frac{b^2 + c^2 - a^2}{2\,b\,c} \quad (t.\ cosinusului).$$

16.4.11 În planul **Oxy** se consideră un triunghi **ABC** ale cărui vârfuri au coordonate cunoscute $\mathbf{x_A}$, $\mathbf{y_A}$, $\mathbf{x_B}$, $\mathbf{y_B}$, $\mathbf{x_C}$, $\mathbf{y_C}$ şi un punct **M** oarecare al cărui coordonate $\mathbf{x_M}$ şi $\mathbf{y_M}$ sunt de asemenea cunoscute. În cadrul programului se va verifica dacă punctul **M** este exterior acestui triunghi sau nu, de asemenea se va face o verificare a faptului că cele trei puncte **A**, **B** şi **C** nu sunt coliniare, tipărindu-se textul "Date initiale incorecte" în cazul coliniarităţii. Programul va realiza introducerea repetată a datelor (coordonatele celor patru puncte), afişarea lor şi a unuia din textele: "Punctul M este exterior triunghiului" , "Punctul M este interior triunghiului" sau "Punctul M este pe o latură". Observaţie: aria unui triunghi având vârfurile în punctele A, B şi C se calculează cu formula:

$$S_{\Delta ABC} = \frac{1}{2}\,abs\left(\begin{vmatrix} x_A & y_A & 1 \\ x_B & y_B & 1 \\ x_C & y_C & 1 \end{vmatrix}\right).$$

16.4.12 Să se elaboreze programul pentru calculul coordonatelor punctelor unei elipse, de semiaxe **a** şi **b**, a valorii razei de curbură $\mathbf{R_C}$ şi a coordonatelor centrului de curbură $\mathbf{x_C}$ şi $\mathbf{y_C}$, în funcţie de parametrul **φ** care intervine în ecuaţiile parametrice **x=a cos φ , y=b sin φ** pentru valorile acestuia în intervalul **[0 ; 90]** parcurs cu un pas **p** (în grade). Se vor folosi următoarele formule de calcul:

$$R_C = \frac{\left(x'^2 + y'^2\right)^{3/2}}{\left|x'y'' - x''y'\right|}, \qquad x' = \frac{dx}{d\varphi}, \; y' = \frac{dy}{d\varphi},$$

$$x_C = x - \frac{y'\left(x'^2 + y'^2\right)}{x'y'' - x''y'}, \quad y_C = y + \frac{x'\left(x'^2 + y'^2\right)}{x'y'' - x''y'}.$$

Valorile obţinute se vor afişa cu câte şase cifre semnificative.

16.4.13 Sunt cunoscute coordonatele a trei puncte din plan $\mathbf{M_1(x_1,y_1)}$, $\mathbf{M_2(x_2,y_2)}$, $\mathbf{M_3(x_3,y_3)}$. Se cere să se determine coordonatele $\mathbf{x_C}$ şi $\mathbf{y_C}$ ale centrului **C** al cercului ce trece prin cele trei puncte. Programul ce se va elabora va asigura introducerea repetată a datelor, pentru mai multe cazuri, calculul coordonatelor $\mathbf{x_C}$, $\mathbf{y_C}$, dacă acest lucru este posibil, afişarea coordonatelor celor trei puncte şi a centrului cercului, dacă calculul s-a putut efectua, respectiv a textului "Puncte coliniare" în caz contrar.

16.4.14 Fie $\mathbf{M_i(x_i, y_i)}$, $\mathbf{1 \le i \le n}$, o mulţime de puncte din plan, de coordonate cunoscute. Să se determine numerele de ordine ale punctelor dreptelor ce aparţin frontierei acoperirii convexe a acestei mulţimi de puncte. O dreaptă ce trece prin punctele $\mathbf{M_i}$ şi $\mathbf{M_j}$ aparţine frontierei dacă toate celelalte puncte ale mulţimii se află de aceeaşi parte a acestei drepte. Se vor afişa coordonatele celor **n** puncte precum şi numerele de ordine şi coordonatele punctelor determinate. Observaţie: dacă se consideră o dreaptă în plan, de ecuaţie $\mathbf{ax + by + c = 0}$ şi două puncte $A_i(x_i, y_i)$ şi $A_k(x_k, y_k)$, poziţia punctelor relativ la dreaptă se determină în felul următor :

$$(a x_i + b y_i + c)(a x_k + b y_k + c) \begin{cases} > 0 & \textit{Punctele sunt de aceeasi parte a dreptei} \\ < 0 & \textit{Punctele sunt de o parte si de alta a dreptei} \\ = 0 & \textit{Unul sau ambele puncte apartin dreptei} \end{cases}$$

16.4.15 Se consideră o dreaptă în plan, dată de ecuaţia generală $\mathbf{a x + b y + c = 0}$ şi două puncte $\mathbf{A_1(x_1,y_1)}$ şi $\mathbf{A_2(x_2,y_2)}$, de coordonate cunoscute. Programul care se va elabora va rezolva următoarele: introducerea valorilor lui **a**, **b**, **c** şi a coordonatelor celor două puncte, urmată de afişarea datelor introduse, calculul expresiilor $\mathbf{E_1 = a x_1 + b y_1 + c}$, $\mathbf{E_2 = a x_2 + b y_2 + c}$, $\mathbf{E_3 = E_1E_2}$, afişarea unor mesaje: "A1 este pe dreapta", dacă $E_1 = 0$, "A2 este pe dreapta", dacă $E_2 = 0$, "Punctele A1 si A2 sunt de aceeasi parte a dreptei", dacă $E_3 > 0$, "Punctele A1 şi A2 sunt in semiplane diferite", dacă $E_3 < 0$.

16.4.16 Se consideră cunoscute coordonatele $(\mathbf{x_i, y_i})$ ale unui număr de **n** puncte din plan. Se va elabora programul în cadrul căruia se realizează introducerea valorilor celor două şiruri, ale absciselor şi ordonatelor celor **n** puncte şi se afişează, se determină coordonatele (x_i, y_i) şi (x_k, y_k) ale punctelor $\mathbf{M_i}$ şi $\mathbf{M_k}$ aflate la distanţă maximă şi se afişează, se determină coordonatele (x_j, y_j) şi (x_l, y_l) ale punctelor $\mathbf{M_j}$ şi $\mathbf{M_l}$ aflate la distanţă minimă şi se afişează. În program se va prevedea posibilitatea de a face introduceri repetate de date. Valoarea numărului de puncte **n** se va lua mai mare decât 10 iar coordonatele celor **n** puncte se vor alege arbitrar.

16.4.17 Fie $\mathbf{A_i\ (x_i, y_i)}$ (i=1, 2, 3) şi $\mathbf{B_j\ (x_j, y_j)}$ (j=1, 2, 3) vârfurile a două triunghiuri $\mathbf{A_1A_2A_3}$ şi $\mathbf{B_1B_2B_3}$, a căror coordonate carteziene sunt cunoscute. În planul **Oxy** cele două triunghiuri pot fi în una din următoarele poziţii: exterioare unul altuia, un vârf al unui triunghi se găseşte în interiorul celuilalt, celelelte două fiind în afară, două vârfuri ale unui triunghi se găsesc în interiorul celuilalt, cel de-al treilea fiind în exterior, un triunghi se găseşte în interiorul celuilalt. Programul ce se va elabora, după ce va realiza introducerea datelor şi afişarea lor într-o formă ordonată, va afişa unul din cele trei mesaje: "Triunghiurile sunt exterioare unul altuia", "Triunghiurile se intersecteaza" sau "Un triunghi este in interiorul altuia".

16.4.18 O funcţie **f(x)** are valorile cunoscute $\mathbf{y_1, y_2, .. , y_n}$ pentru un şir de valori $\mathbf{x_1, x_2, .. , x_n}$ distincte şi crescătoare ale absciselor **x**. Să se elaboreze programul în cadrul căruia să se realizeze următoarele: introducerea şi afişarea celor două şiruri (în program se va prevedea ca să nu fie posibilă introducerea unei valori de abscisă, mai mică decât cea anterioară), verificarea pentru fiecare pereche de puncte $\mathbf{M_iM_{i+1}}$ începând cu $\mathbf{M_2M_3}$ şi terminând cu $\mathbf{M_{n-2}M_{n-1}}$ a faptului dacă punctul din stânga $\mathbf{M_{i-1}}$ şi cel din dreapta $\mathbf{M_{i+2}}$ se găsesc de aceeaşi parte a dreptei ce trece prin $\mathbf{M_iM_{i+1}}$ sau se găsesc de o parte şi de alta a acestei drepte, ceea ce înseamnă că graficul funcţiei are un punct de inflexiune pentru o abscisă din intervalul $\mathbf{[x_i, x_{i+1}]}$. Pentru aceasta se scrie ecuaţia dreptei ce trece prin cele două puncte $\mathbf{(x_i, y_i)}$, $\mathbf{(x_{i+1}, y_{i+1})}$,

$$\frac{y - y_i}{y_{i+1} - y_i} = \frac{x - x_i}{x_{i+1} - x_i}$$

sub forma $F(x,y)=(y-y_i)(x_{i+1}-x_i) - (x-x_i)(y_{i+1}-y_i)$ şi se calculează produsul $P=F(x_{i-1},y_{i-1})\ F(x_{i+2},y_{i+2})$. Dacă acest produs este negativ se afişează valorile absciselor punctelor $\mathbf{M_i}$ şi $\mathbf{M_{i+1}}$, adică $\mathbf{x_i}$ şi $\mathbf{x_{i+1}}$, precum şi mesajul ***"Punct de inflexiune"***.

16.4.19 Ecuaţia generală a unei conice este: $\mathbf{a_{11}\,x^2 + 2\,a_{12}\,x\,y + a_{22}\,y^2 + 2\,a_{13}\,x + 2\,a_{23}\,y + a_{33} = 0.}$ În funcţie de valorile invarianţilor

$$\Delta = \begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{vmatrix}, \quad \delta = \begin{vmatrix} a_{11} & a_{12} \\ a_{12} & a_{22} \end{vmatrix}, \quad I = a_{11} + a_{22},$$

rezultă tipul conicei, după cum se arată în continuare :

$\Delta \neq 0$ (conice propriu zise)	$\delta>0,\ \Delta I > 0$	elipse imaginare,
	$\delta>0,\ \Delta I < 0$	elipse reale,
	$\delta=0,\ \Delta I > 0$	parabole imaginare,
	$\delta=0,\ \Delta I < 0$	parabole reale,
	$\delta=0$	hiperbole,

$\Delta = 0$ (conice degenerate)	$\delta>0$	drepte concurente imaginare,
	$\delta<0$	drepte concurente reale,
	$\delta = 0,\ a_{11}a_{33} - a_{13}^2 > 0$	drepte paralele imaginare,
	$\delta = 0,\ a_{11}a_{33} - a_{13}^2 < 0$	drepte paralele reale, distincte,
	$\delta = 0,\ a_{11}a_{33} - a_{13}^2 = 0$	drepte paralele confundate.

În urma introducerii datelor (a coeficienţilor $\mathbf{a_{ij}}$) şi a calculării valorilor invarianţilor $\mathbf{\Delta}$, $\mathbf{\delta}$ şi $\mathbf{I}$ iar în unele cazuri şi a expresiei $a_{11}\ a_{33} - a_{13}^2$ se va afişa un text privind tipul conicei.

16.4.20. Se consideră o elipsă având semiaxele $\mathbf{a}$ şi $\mathbf{b}$. Se va scrie programul pentru calculul ariei $\mathbf{S}$ a elipsei şi a perimetrului $\mathbf{L}$. Formulele sunt următoarele:

$$S = \pi\, a\, b\,, \quad L = 2\pi\, a \left[1 - \left(\frac{1}{2}\right)^2 \frac{1-E^2}{1} - \left(\frac{1\cdot 3}{2\cdot 4}\right)^2 \frac{\left(1-E^2\right)^2}{3} - \left(\frac{1\cdot 3\cdot 5}{2\cdot 4\cdot 6}\right)^2 \frac{\left(1-E^2\right)^3}{5} - \ldots\ldots \right]$$

unde $\mathbf{E = b/a}$ ($\mathbf{a \geq b}$). Ca date de intrare se vor lua valorile semiaxelor şi numărul $\mathbf{n}$ al termenilor negativi din paranteză.

16.4.21 În planul axelor **Oxy** se consideră un cerc de rază **R** (**R>1**) cu centrul în origine. Se cere să se determine numărul punctelor interioare cercului a căror abscisă şi ordonată au valori întregi .

16.4.22 Pe axa reală **Ox** sunt date coordonatele a **n** perechi de puncte care mărginesc segmente : $\mathbf{a_1}$ şi $\mathbf{b_1}$, $\mathbf{a_2}$ şi $\mathbf{b_2}$, ... , $\mathbf{a_n}$ şi $\mathbf{b_n}$. Să se determine coordonatele capetelor segmentului (dacă acesta există) care rezultă prin "intersecţia" celor **n** segmente .

16.4.23 Se consideră un număr de **n** puncte în plan, având coordonatele : $\mathbf{x_1}$ şi $\mathbf{y_1}$, $\mathbf{x_2}$ şi $\mathbf{y_2}$, ... , $\mathbf{x_n}$ şi $\mathbf{y_n}$. Se cere să se determine raza minimă a cercului cu centrul în origine care conţine toate aceste **n** puncte .

16.4.24 Se consideră trei puncte în plan având coordonatele carteziene cunoscute şi un al patrulea punct având de asemenea coordonate cunoscute. Se cere să se determine valoarea minimă a distanţei dintre al patrulea punct şi laturile triunghiului format de cele trei puncte .

16.4.25 În planul axelor Oxy se găseşte un triunghi având vârfurile în punctele de coordonate : **A(-1;0), B(0,1)** şi **C(1,0)**. Programul care se va elabora va permite introducerea (repetată) a coordonatelor **a** şi **b** ale unui punct şi afişarea unuia din următoarele mesaje : "Punctul este interior triunghiului" , "Punctul este exterior triunghiului" sau "Punctul este pe una din laturile triunghiului" , în funcţie de poziţia respectivului punct. Indicaţie : $y \geq 0$, $|x|+y \leq 1$.

16.4.26 În planul axelor Oxy se găseşte un patrat având vârfurile în punctele de coordonate: A(-1;0), B(0,1) , C(1,0) şi D(0,-1). Programul care se va elabora va permite introducerea (repetată) a coordonatelor **a** şi **b** ale unui punct şi afişarea unuia din următoarele mesaje : "Punctul este interior patratului" , "Punctul este exterior patratului" sau "Punctul este pe una din laturile patratului" , în funcţie de poziţia respectivului punct. Indicaţie : $|x|+|y| \leq 1$.

16.4.27 Sunt cunoscute coordonatele $\mathbf{x_A}$, $\mathbf{y_A}$, $\mathbf{x_B}$, $\mathbf{y_B}$, $\mathbf{x_C}$, $\mathbf{y_C}$, ale vârfurilor unui triunghi **ABC**, care se află în planul axelor **Oxy**. Se cere să se determine suprafaţa triunghiului **ABC** precum şi a triunghiurilor **ABM**, **BCM** şi **CAM**, unde **M** este punctul de întâlnire al medianelor triunghiului **ABC** .

16.4.28 Se consideră un număr de **n** puncte în plan, având coordonatele cunoscute $\mathbf{x_1}$, $\mathbf{y_1}$, $\mathbf{x_2}$, $\mathbf{y_2}$, ... , $\mathbf{x_n}$, $\mathbf{y_n}$. Să se determine elementele unei matrice (simetrice) ale cărei elemente $\mathbf{d_{i,k}}$ au valori egale cu distanţele dintre punctele $\mathbf{P_i}$ şi $\mathbf{P_k}$.

16.4.29 Se consideră două cercuri în planul Oxy, la care se cunosc coordonatele centrelor şi razele. Să se stabilească, afisând mesaje corespunzătoare, dacă cercurile nu se intersectează, sunt cuprinse unul în altul, sunt tangente (exterior sau interior) sau se intersectează .

16.5 Rezolvarea ecuaţiilor neliniare

16.5.1 Se va elabora programul pentru determinarea soluţiei ecuaţiei **f(x)=x^3–[x]–3=0** din intervalul (**1;2**) aplicând metoda înjumătăţirii intervalului de **n = 15** ori. Se va afişa **n**, rădăcina găsită, precum şi eroarea faţă de rezultatul exact **r = 4$^{1/3}$**

16.5.2 Se consideră ecuaţia:

$$x - \frac{1}{1+x^2} = 0$$

care are o rădăcină reală pozitivă. Se cere elaborarea programului pentru determinarea acestei rădăcini cu 10 cifre semnificative exacte, utilizând repetat metoda aproximaţiilor succesive. Se va tipări valoarea rădăcinii precum şi a numărului **n** care reprezintă numărul de aplicări al acestei metode.

16.5.3 Se consideră ecuaţia

$$f(x) = \frac{3x - 2}{x - 1} + \ln x^2 = 0$$

care are o rădăcină reală în intervalul **(-5;-0.25)**. Ştiind că în acest interval

$$f'(x) < 0 \ , \ f''(x) > 0$$

se cere să se determine valoarea rădăcinii cu o precizie **ε = 10^{-5}** utilizând în mod repetat metoda tangentei. Se va afişa rezultatul găsit precum şi numărul necesar de aplicări succesive al metodei tangentei pentru obţinerea preciziei dorite.

16.5.4 Se consideră ecuaţia **f(x) = x^2 – a = 0** unde **a>0**. Să se calculeze valoarea radicalului din **a** determinând rădăcina ecuaţiei cu metoda tangentei, aplicată de **n** ori. Valoarea iniţială a rădăcinii se va lua $x_1 = 1$ dacă a<1, respectiv $x_1 = a$ dacă a>1, iar relaţia de recurenţă este următoarea:

$$x_{i+1} = \frac{x_i^2 + a}{2\,x_i} \ .$$

Se vor considera următoarele valori numerice:

a = 2, n = 15 ; a = 7, n = 8 ; a = 100 , n = 10 ;

programul permiţând introducerea repetată a datelor.

16.5.5 Se consideră ecuaţia **f(x) = ln(x-1) – x + 3 = 0**, care are două rădăcini reale, cuprinse în intervalul **(1.05; 1.95)** respective în **(2.1; 10.0)**. Să se determine valorile acestor două rădăcini aplicând de **n** ori metoda tangentei, afişând valorile găsite ale rădăcinilor după fiecare cinci aplicări consecutive a metodei. Se va alege **n=30**. Pentru alegerea corectă a valorilor iniţiale se ţine cont că derivata a doua a funcţie este negativă în ambele intervale iar derivata întâia este pozitivă în primul interval şi negativă în al doilea.

16.5.6 Se va elabora programul pentru găsirea soluţiilor din mulţimea numerelor întregi **Z**, a ecuaţiei:

$$f(x,y,z) = 3^x + 4^y - 5^z = 0 \quad ,$$

unde x **e** { 2, 3, 4, 5, 6 }, y **e** { 1, 2, 3, 4, 5 }, z **e** { 0, 1, 2, 3, 4, 5 }. Pentru acel caz când valoarea funcţiei f(x,y,z) este nulă se va afişa pe rândul respectiv cuvântul SOLUTIE.

16.5.7 Ecuaţia **f(x) = sin x - lg x = 0** are trei rădăcini, aparţinând intervalelor (**2.,3.**), (**7.,8.**) şi (**8.,9.**). Utilizând de **n** ori metoda înjumătăţirii intervalelor în cazul fiecăruia din cele trei intervale se cere să se determine valorile celor trei rădăcini. Se va lua **n = 25** şi se vor tipări valorile lui **n** şi a celor trei rădăcini găsite.

16.5.8 Se consideră ecuaţia: **f(x) = ln(1 – 2 cos x) = 0**, care are o rădăcină în intervalul (**π/3; π**), iar alta în intervalul (**π; 5π/3**). Se cere să se determine valorile acestor rădăcini aplicând metoda înjumătăţirii intervalelor în mod repetat până când rădăcinile vor fi cuprinse într-un interval mai mic decât **ε = 10^{-5}**.

16.5.9 Se cere să se determine rădăcina reală (pozitivă) a ecuaţiei **f(x) = x^n – a = 0** pentru un **a** şi **n** dat, utilizând simultan metoda coardei şi tangentei, formulele de calcul fiind următoarele:

$$s_{i+1} = \frac{d_i f(s_i) - s_i f(d_i)}{f(s_i) - f(d_i)}, \quad d_{i+1} = d_i - \frac{f(d_i)}{n\, d_i^{n-1}},$$

aceste formule aplicându-se de **m** ori. Ca rădăcină a ecuaţiei se va considera : r = ½ (s_{m+1} + d_{m+1}) ca valori iniţiale pentru şirurile de abscise **{ s$_i$ }** şi **{ d$_i$ }** luându-se

$$s_1 = 0, \quad d_1 = \begin{cases} a & daca\ a > 1 \\ 1 & daca\ a < 1 \end{cases}$$

Valorile numerice vor fi următoarele: a=10, n=5, m=15. În cadrul programului se va prevedea afişarea valorilor introduse **a, n** şi **m** , precum şi a valorilor elementelor şirurilor **{s$_i$}** şi **{d$_i$}**.

16.5.10 Se consideră ecuaţia :

$$f(x) = \sqrt{x + 2\sqrt{x + 2\sqrt{x + \ldots + 2\sqrt{x + 2\sqrt{3x}}}}} - x = 0 \,,$$

funcţia din membrul stâng având **n** radicali. Ştiind că această ecuaţie are o rădăcină reală în intervalul (**2.0**; **4.0**) să se determine valoarea acesteia utilizând metoda calculării valorilor funcţiei în intervalul dat, cu un pas **h**. Programul va asigura introducerea repetată a valorilor lui **n** şi **h** .

16.5.11 Se consideră ecuaţia **f(x) = x^3 – a = 0** şi se cere să se calculeze valoarea **a$^{1/3}$** determinând rădăcina acestei ecuaţii cu metoda tangentei, ştiind că

$$x_1 = \begin{cases} a & daca\ a > 1 \\ 1 & daca\ a < 1 \end{cases} \qquad x_{i+1} = \frac{2\, x_i^3 + a}{3\, x_i^2} \,.$$

Relaţia de recurenţă se va utiliza de cel mult **n** ori, până când este îndeplinită condiţia:

$$|x_{i+1} - x_i| < \varepsilon \ ,$$

ε fiind o constantă pozitivă foarte mică. Programul va asigura introducerea repetată a datelor .

16.5.12 Se consideră ecuaţia **f(x) = x² – a = 0** şi se cere să se calculeze valoarea radicalului din **a** determinând rădăcina ecuaţiei cu metoda tangentei, aplicată până când **|x_{i+1} - x_i | < ε** unde **x_i** şi **x_{i+1}** sunt două valori succesiv calculate ale rădăcinii, conform relaţiei de recurenţă:

$$x_{i+1} = \frac{x_i^2 + a}{2\,x_i} \ ,$$

Valoarea iniţială **x_i** a rădăcinii va fi: **$x_1 = 2^{[m/2]}$** , unde **m** este exponentul ce intervine în următoarea formă de scriere a lui **a** : **$a = 2^m a_1$** , **½ ≤ a_1 < 1** . Valorile numerice vor fi următoarele: **a = 2, ε = 10^{-10}**

16.5.13 Se consideră ecuaţia:

$$f(x) = \arcsin(\ln x) + 2\sqrt{1 - \ln^2 x} = 0 \ ,$$

funcţia **f(x)** fiind definită pe intervalul **(1/e; e]**. Cunoscând reprezentarea grafică a funcţiei **f(x)**, se observă că ecuaţia **f(x)=0** are o rădăcină în intervalul [**0.37**; **1.00**].

Să se determine valoarea acestei rădăcini cu ajutorul unui program în care se utilizează metoda coardei, în mod repetat, până când diferenţa dintre abscisele a două rădăcini succesiv calculate devine mai mică decât un număr dat **ε**. În cadrul programului se va prevedea afişarea tuturor valorilor găsite ale rădăcinii căutate, până la îndeplinirea condiţiei impuse, ca urmare a aplicării repetate a metodei coardei. Valoarea numerică a lui **ε** va fi **10^{-4}**.

16.5.14 Se consideră ecuaţia :

$$f(x) = 1 + \cfrac{1}{1 + \cfrac{1}{1 + \cfrac{1}{1 + \cfrac{1}{1 + \cfrac{\cdot}{1 + \cfrac{1}{x}}}}}} - x = 0 \ ,$$

în membrul stâng al acesteia semnul fracţiei intervenind de **n** ori, ecuaţia având o rădăcină pozitivă în intervalul (**1.50**; **1.75**). Să se determine valoarea acestei rădăcini utilizând metoda calculării valorilor funcţiei în intervalul respectiv cu un pas **h=0.001**, considerând că rădăcina se va găsi la mijlocul intervalului de mărime **h** la capetele căruia funcţia **f(x)** are semne contrare. Se va introduce **n** şi **h** şi se va realiza afişarea lui **n, h** şi a rădăcinii calculate.

16.5.15 Se va scrie programul pentru rezolvarea ecuaţiei de gradul I **ax+b=0,** unde **a** şi **b** sunt variabile cu valori reale. Se vor avea în vedere toate cazurile posibile, afişându-se textele corespunzătoare.

16.5.16 Se va scrie programul pentru rezolvarea ecuaţiei de gradul II , $\mathbf{ax^2+bx+c=0,}$ coeficienţii **a**, **b** şi **c** având valori reale. Se vor considera toate cazurile posibile .

16.5.17 Se consideră ecuatia de gradul trei, $\mathbf{ax^3+bx^2+cx+d=0,}$ având toţi coeficienţii numere întregi. Se cere să se determine rădăcinile întregi ale acestei ecuaţii. Se ştie că rădăcinile întregi (dacă există) sunt printre divizorii (luaţi cu semnul plus sau minus) ai termenului liber **d**.

16.5.18 Utilizând metoda înjumătăţirii intervalelor să se rezolve ecuaţiile, determinând rădăcina care se găseşte în intervalele specificate :

$$\cos\frac{2}{x}-2\sin\frac{1}{x}+\frac{1}{x}=0,\quad r\in[1;2] \qquad e^x+\sqrt{1+e^{2x}}-2=0,\quad r\in[-1;0]$$

$$\sin(\ln x)-\cos(\ln x)+2\ln x=0,\qquad r\in[1;3]$$

asigurând o precizie de $\mathbf{10^{-6}}$ la calculul valorii rădăcinii.

16.5.19 Utilizând metoda **tangentei** (Newton) să se elaboreze programul cu care să se rezolve ecuaţiile următoarele, pornind din punctul de abscisă $\mathbf{x_0}$, precizat pentru fiecare ecuaţie,

$$\lg x-\frac{1}{x^2}-1=0,\quad x_0=4.0, \qquad e^x-2x-21=0,\quad x_0=3.0,$$

$$\cos x-3x=0,\ x_0=0.3, \qquad x^x-10=0,\ x_0=2.5, \qquad \sin x-\frac{x+1}{x-1}=0,\ x_0=-0.4,$$

determinarea soluţiei făcându-se cu o precizie de $\mathbf{10^{-6}}$. În fiecare caz se va afişa şi numărul de utilizări al metodei care a fost necesar pentru asigurarea preciziei impuse a rădăcinii.

16.6 Calculul unor funcţii utilizând dezvoltări în serii

16.6.1 Se consideră dezvoltarea în serie:

$$e^x=1+\frac{x}{1!}+\frac{x^2}{2!}+\frac{x^3}{3!}+\ldots+\frac{x^n}{n!}+\ldots$$

Se cere să se calculeze valoarea numărului **e** folosind **n** termeni din dezvoltarea în serie a funcţiei $\mathbf{e^x}$ pentru **x = 1** şi să se compare rezultatul obţinut cu valoarea funcţiei matematice **exp(1.)**.

16.6.2 Dezvoltarea în serie a patratului funcţiei **sin** este următoarea:

$$\sin^2 x = x^2 - \frac{2^3 x^4}{4!} + \frac{2^5 x^6}{6!} - \ldots = \sum_{n=1}^{\infty} \frac{(-1)^{n+1} 2^{2n-1} x^{2n}}{(2n)!}$$

Pentru un **x** dat să se calculeze valorile acestei funcţii tipărind rezultatele obţinute după considerarea următoarelor numere de termeni : 2, 4, 6, 8, ... , 14

16.6.3 Se cunoaşte următoarea dezvoltare în serie :

$$\ln\left|\frac{1+x}{1-x}\right| = 2\left(\frac{x}{1} + \frac{x^3}{3} + \frac{x^5}{5} + \frac{x^7}{7} + \ldots.\right)$$

Se cere să se calculeze valoarea sumei termenilor seriei, pentru **x = 1/3**, luând pe rând câte 1, 2, 3, ... , **n** termeni şi tipărind de fiecare dată rezultatul obţinut precum şi abaterea faţă de valoarea considerată exactă, calculată cu funcţia **log(2.)**

16.6.4 Se cunoaşte următoarea dezvoltare în serie

$$\ln(1+x) = \frac{x}{1} - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots.$$

Se cere să se calculeze valoarea sumei termenilor seriei, pentru **x = 1**, luând pe rând câte 1, 2, 3, ... , **n** termeni şi afişând de fiecare dată rezultatul obţinut şi abaterea faţă de valoarea considerată exactă, calculată cu ajutorul funcţiei **log(2.)**.

16.6.5 Este cunoscută formula:

$$\sqrt{1+x} = 1 + \frac{1}{2}x - \frac{1}{2.4}x^2 + \frac{1.3}{2.4.6}x^3 - \frac{1.3.5}{2.4.6.8}x^4 + \ldots.$$

Se cere să se calculeze valoarea $\mathbf{a^{1/2}}$, (a>1), utilizând această dezvoltare în serie pentru calculul valorii radicalului din expresia următoare:

$$\sqrt{a} = n\sqrt{1 + \frac{r}{n^2}} \quad ,$$

unde **n** este cel mai mare număr întreg al cărui patrat nu depăşeşte valoarea lui **a**. Din dezvoltarea în serie se vor folosi **25** de termeni. În cadrul programului elaborat se va efectua citirea repetată a valorilor unor numere reale **a**, se va tipări numărul **a** citit, se va determina şi tipări valoarea lui **n** şi se va calcula şi tipări valoarea $\mathbf{a^{1/2}}$ în două feluri, în primul rând utilizând formulele prezentate iar în al doilea rând utilizând funcţia **sqrt**. Valorile radicalului se vor tipări utilizând specificatorul de format **%14.7e** .

16.6.6 Sunt cunoscute următoarele dezvoltări în serie :

$$\sin x = \frac{x}{1!} - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, \quad \cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots \quad (x \ \ in \ \ radiani)$$

$$sh\,x = \frac{x}{1!} + \frac{x^3}{3!} + \frac{x^5}{5!} + \ldots, \qquad ch\,x = 1 + \frac{x^2}{2!} + \frac{x^4}{4!} + \frac{x^6}{6!} + \ldots.$$

$$\arcsin x = x + \frac{1}{2}\,\frac{x^3}{3} + \frac{1.3}{2.4}\,\frac{x^5}{5} + \frac{1.3.5}{2.4.6}\,\frac{x^7}{7} + \ldots \qquad x \in (-1,+1)$$

$$arctg\,x = \frac{x}{1} - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \ldots \qquad x \in (-1,+1)$$

$$\frac{1}{1+x} = 1 - x + x^2 - x^3 + \ldots \qquad x \in (-1,+1)$$

$$\frac{1}{\sqrt{1+x}} = 1 - \frac{1}{2}\,x + \frac{3}{8}\,x^2 - \frac{5}{16}\,x^3 + \frac{35}{128}\,x^4 + \ldots + (-1)^k \frac{(2k-1)!!}{(2k)!!}\,x^k + \ldots \quad x \in (-1,+1)$$

Pentru fiecare caz în parte se va elabora un program în cadrul căruia se vor realiza următoarele: introducerea valorii lui **x**, a numărului **n** de termeni ce se vor lua din dezvoltarea în serie şi a unei valori numerice foarte mici **ε** cu care se vor compara valorile absolute ale termenilor seriei, înainte de a fi însumaţi (dacă un anumit termen este mai mic în valoare absolută decât **ε**, atunci calculul se consideră terminat), calculul valorii funcţiei pornind de la expresia sa, utilizând funcţiile matematice **sin(x)**, **cos(x)**, **exp(x)**, etc., calculul valorii funcţiei utilizând din dezvoltarea în serie cel mult **n** termeni, cu condiţia ca termenii ce se însumează să fie mai mari în valoare absolută decât **ε**, afişarea rezultatelor. Programul va asigura introducerea repetată a unor seturi de date.

16.6.7 Este cunoscut faptul ca o serie de funcţii **f(x)** (funcţia exponenţială, logaritmică, funcţiile trigonometrice) se pot aproxima cu un polinom de gradul **n**,

$$P_n(x) = \sum_{i=1}^{n+1} a_i\, x^{i-1} \ ,$$

dacă variabila **x** aparţine unui interval $[\,\mathbf{x_i}\,,\,\mathbf{x_f}\,]$, diferenţa dintre valoarea funcţiei şi a polinomului fiind mai mică decât un număr **ε**.

$$|\, f(x) - P_n(x)\,| < \varepsilon\,. \qquad \forall\, x \in [\,x_i\,, x_f\,].$$

În tabelul următor sunt date funcţiile respective, gradul polinomului, $\mathbf{x_i}$, $\mathbf{x_f}$, **ε** şi valorile coeficienţilor polinomului de aproximare.

Nr. Crt.	**F(x)**	**x ∈ [x_i , x_f]**	**ε**	**n**	Coeficienţii polinomului
1	e^x	x ∈ [-1,+1]	2.10^{-7}	7	a_1 = 0.9999998, a_2 = 1.0000000, a_3 = 0.5000063, a_4 = 0.1666674, a_5 = 0.0416350, a_6 = 0.0083298, a_7 = 0.0014393, a_8 = 0.0002040
2	ln(1+x)	x ∈ [0,1]	$2.2.10^{-7}$	7	a_1 = 0., a_2 = 0.999981028, a_3 = -0.49947015, a_4 = 0.328233122, a_5 = -0.225873284, a_6 = 0.134639267, a_7 = -0.055119959, a_8 = 0.010757369,
3	sin x	x ∈ [-π/2, π/2]	6.10^{-9}	9	a_1 = 0, a_2 = 1.000000002, a_3 = 0, a_4 =-0.166666589, a_5 = 0, a_6 = 0.008333075, a_7 = 0, a_8 =-0.000198107, a_9 = 0, a_{10} = 0.000002608
4	cos x	x ∈ [-1, +1]	2.10^{-9}	10	a_1 = 1, a_2 =0, a_3 =-0.499999999942, a_4 =0, a_5 = 0.041666665950, a_6 =0, a_7 =-0.001388885683, a_8 =0, a_9 = 0.000024795132, a_{10} =0, a_{11} =-0.000000269591,
5	tg x	x ∈ [-π/4, π/4]	2.10^{-8}	13	a_1 = 0, a_2 = 1.00000002, a_3 = 0, a_4 = 0.33333082, a_5 = 0, a_6 = 0.13339762, a_7 = 0, a_8 = 0.05935836, a_9 = 0, a_{10} = 0.02457096, a_{11} = 0, a_{12} = 0.00294045, a_{13} = 0, a_{14} = 0.00947324

1. Se va elabora un program pentru calcularea valorilor funcţiei **f(x) = e^x**, în intervalul [**a**,**b**], cu un pas **h**, precum şi a diferenţelor dintre valoarea functiei **f(x)** calculată cu formula de aproximare şi a funcţiei **exp(x).** Se vor afişa valorile coeficienţilor polinomului de aproximare, abscisa **x,** valoarea polinomului şi eroarea .

2. Aceeaşi problemă pentru f**(x) = ln(1+x)** ,

3) Aceeaşi problemă pentru **f(x) = sin(x)** ,

4) Aceeaşi problemă pentru **f(x) = cos(x)** ,
5) Aceeaşi problemă pentru **f(x) = tg(x)** .
Observaţie : Valorile polinoamelor de aproximaţie se vor calcula prin programarea formei de scriere a lui **Horner**.

16.7 Calculul integralelor definite

16.7.1 În cazul integralei definite

$$I_n = \int_0^{\pi/2} \cos^n x \ \sin n x \ dx , \qquad I_1 = \int_0^{\pi/2} \cos x \sin x \ dx = \frac{1}{2}, \quad I_k = \frac{1}{2}\left(\frac{1}{k} + I_{k-1}\right)$$

valoarea primei integrale şi relaţia de recurenţă sunt cele anterior prezentate.
Programul ce se va elabora va realiza calculul şi afişarea şirului de valori $\mathbf{I_1, I_2, ..., I_{15}}$.

16.7.2 În cazul integralei definite

$$I_n = \int_0^{\pi/4} tg^{2n} x \ dx , \quad I_1 = \int_0^{\pi/4} tg^2 x \ dx = 1 - \frac{\pi}{4}, \quad I_k = \frac{1}{2k-1} - I_{k-1} \ .$$

valoarea primei integrale şi relaţia de recurenţă sunt cele anterior prezentate. Se cere să se calculeze valorile acestor integrale pentru toate valorile lui **n** de la 1 la 10.

16.7.3 Se consideră integrala definită , valoarea lui $\mathbf{I_0}$ şi formula de recurenţă

$$I_n = \int_0^1 x^n \ e^{x-1} \ dx , \qquad I_0 = 1 - \frac{1}{e} , \quad I_k = 1 - k\, I_{k-1} .$$

Se va elabora programul cu care se vor calcula valorile primelor 20 de integrale. Observând valorile numerice obţinute în cazul integralelor finale să se explice dece aceste rezultate sunt eronate.

16.7.4 Se consideră integrala [Fiht64], vol. II, pg. 586,

$$I = \int_0^{\infty} \frac{\sin \alpha \ x \ \cos \beta \ x}{x} \ dx = \begin{cases} \pi/2, & \alpha > \beta \\ \pi/4, & \alpha = \beta \\ 0, & \alpha < \beta \end{cases}$$

numită ***factorul discontinuu Dirichlet*** (Gustave Lejeune Dirichlet 1805-1859), unde **α>0, β>0** , a cărei valoare se calculează conform formulei prezentate. Se va realiza programul în cadrul căruia se vor introduce două valori pozitive **α** şi **β** şi se va determina care este valoarea integralei, în funcţie de condiţia care este îndeplinită.

16.7.5 Se consideră integrala [Fiht64], vol. II, pg. 586,

$$I=\int_0^{\infty}\frac{\sin\alpha x\ \ \sin\beta x\sin\gamma x}{x}\,dx=\begin{cases}\pi/4, & \alpha<\beta+\gamma\\ \pi/8, & \alpha=\beta+\gamma\\ 0, & \alpha>\beta+\gamma\end{cases}$$

unde **α>β>0** , **α>γ>0** , a cărei valoare se calculează conform formulei prezentate. Se va realiza programul în cadrul căruia se vor introduce trei valori pozitive **α** , **β** şi **γ**, se va determina care din ele este cea mai mare şi se va calcula valoarea integralei, funcţie de condiţia care este îndeplinită.

16.7.6 Ştiind că:

$$\int_0^{\pi/2}\sin^n x\,dx=\begin{cases}\dfrac{(n-1)!!}{n!!}\dfrac{\pi}{2}, & (n\ este\ par)\\ \dfrac{(n-1)!!}{n!!}, & (n\ este\ impar)\end{cases}$$

să se calculeze valorile integralei pentru toate valorile lui **n** din intervalul **[5; 15].**

16.7.7 Cunoscându-se formula:

$$\int_0^{2\pi}\sin^n x\,dx=\begin{cases}\dfrac{(n-1)!!}{n!!}2\pi, & n=2k,\\ 0, & n=2k+1,\end{cases}$$

să se calculeze valorile integralei pentru toate valorile lui **n** din intervalul **[1; 15]**. Valorile integralelor se vor afişa cu cel puţin opt cifre semnificative.

16.7.8 Din calculul integral este cunoscută formula:

$$\int_0^{\pi/2}\sin^m x\cos^n x\ \ dx\ =\begin{cases}\dfrac{(m-1)!!(n-1)!!}{(m+n)!!}\dfrac{\pi}{2} & daca\ \ m\ si\ \ n\ \ sunt\ pare\\ \dfrac{(m-1)!!(n-1)!!}{(m+n)!!} & in\ \ celelalte\ \ cazuri\end{cases}$$

Se cere să se elaboreze programul în cadrul căruia să se calculeze valorile acestei integrale pentru diferite valori ale lui **m** şi **n**, ce se vor introduce în mod repetat, pentru fiecare caz afişându-se valorile introduse ale lui **m** şi **n**, precum şi valoarea integralei.

16.7.9 Integrala de prima speţă a lui Euler (Leonhard Euler 1707-1783) este următoarea:

$$B(a, b) = \int_0^1 x^{a-1} (1 - x)^{b-1} dx ,$$

unde **a, b > 0**. În cazul când **a** şi **b** sunt **numere naturale**, această integrală se calculează cu formula:

$$B(a, b) = \frac{(a - 1)! (b - 1)!}{(a + b - 1)!} .$$

Se va elabora programul pentru calculul funcţiei **B(a,b)** pentru diferite perechi de valori ale lui **a** şi **b** , introduse de la tastatură. De fiecare dată se va afişa **a**, **b** şi valoarea funcţiei **B(a,b)**.

16.7.10 Integrala lui Euler de speţa a II-a este următoarea:

$$\Gamma (a) = \int_0^\infty x^{a-1} e^{-x} dx ,$$

numită funcţia "**gamma"**, calculul acesteia făcându-se cu formula (a>0)

$$\Gamma (a) = \lim_{n\to\infty} n^a \frac{1 \cdot 2 \cdot 3 \cdot \ldots \cdot (n - 1)}{a (a + 1) (a + 2) \ldots (a + n - 1)} .$$

Pentru o valoare dată pentru **a** se vor calcula valorile expresiei ce se trece la limită, pentru următoarele valori ale lui **n** : 5, 10, 15 şi 20 .

16.7.11 Se ştie că integrala definită:

$$I = \int_0^\infty e^{-x^2} d x = \frac{\sqrt{\pi}}{2} = 0.88622692545275....$$

este mărginită de termenii şirurilor **{a_i}** şi **{b_i}**, **$a_i < I < b_i$** , unde

$$a_i = \sqrt{i} \frac{(2i)!!}{(2i+1)!!} , \quad b_i = \sqrt{i} \frac{(2i-3)!!}{(2i-2)!!} \frac{\pi}{2} , \qquad I = \lim_{i \to \infty} a_i = \lim_{i\to\infty} b_i$$

conform [Fiht64] vol. II pag. 569. Se cere să se elaboreze programul pentru calculul valorilor termenilor acestor două şiruri pentru valorile lui **i** de la 5 la 15 iar rezultatele se vor afişa cu opt cifre semnificative.

16.7.12 Se ştie că valoarea integralei:

$$I = \int_0^{\frac{\pi}{2}} \frac{dx}{\sqrt{a^2 \cos^2 x + b^2 \sin^2 x}} \qquad (a > b > 0)$$

satisface condiţiile:

$$\frac{\pi}{2\,p_i} < I < \frac{\pi}{2\,r_i}, \quad (i = 1,2,3,...)$$

şirurile **{ p_i }** şi **{ r_i }** având o limită comună, ele fiind definite prin următoarele relaţii

$$p_1 = \frac{a+b}{2},\ r_1 = \sqrt{a\,b}\ ,\quad p_i = \frac{p_{i-1} + r_{i-1}}{2},\ \ r_i = \sqrt{p_{i-1}\ r_{i-1}}\ ,\quad i = 2, 3, 4, ...$$

conform [Fiht64], vol. II, pag. 134-135. Se cere să se elaboreze programul pentru calculul valorilor expresiilor **$\pi/2p_i$** şi **$\pi/2r_i$**, pentru toate valorile lui **i** de la 1 la 10, pentru o pereche de valori date pentru **a** şi **b**. Rezultatele se vor afişa cu opt cifre semnificative.

Pentru următoarele valori

$$a = \sqrt{2}\ = 1.4142135623...\ ,\ \ b = 1\ ,$$

se va obţine rezultatul : I = 1.3110288...

16.7.13 Este cunoscută formula [Fiht64], vol. II, pg. 135-136

$$I = \int_0^{\frac{\pi}{2}} \frac{dx}{\sqrt{1 - a^2 \sin^2 x}} = \frac{\pi}{2} \lim_{n \to \infty} \prod_{i=1}^{n} (1 + p_i),$$

{ p_i } fiind un şir, al cărui prim termen şi relaţia de recurenţă între doi termeni succesivi sunt :

$$p_1 = \frac{1 - \sqrt{1 - a^2}}{1 + \sqrt{1 - a^2}}\ ,\quad p_i = \frac{1 - \sqrt{1 - p_{i-1}^2}}{1 + \sqrt{1 - p_{i-1}^2}}\ ,\ \ i = 2, 3, 4, ...$$

Se cere să se calculeze valoarea acestei integrale pentru **a= 2.5**, numărul **n** de factori din membrul drept luându-se **25**. Se vor afişa valorile lui **a**, **n** şi a integralei.

16.7.14 Se consideră integrala definită

$$I = \int_0^{\pi/2} \cos\ x\ dx\ = 1$$

Ştiind că $I = \lim_{n \to \infty}\ E(n),$ unde

$$E(n) = \frac{\pi}{2\,n}\left[1 + \cos\frac{\pi}{2n} + \cos\frac{2\pi}{2n} + ... + \cos\frac{(n-1)\pi}{2n}\right]$$

să se calculeze valoarea expresiei **E(n)** pentru o serie de valori ale lui **n** începând cu valoarea **n=5** şi continuând până când diferenţa **|E(n) - 1|** devine mai mică decât $\mathbf{10^{-5}}$, afişându-se ultima valoare folosită a lui **n** precum şi valoarea expresiei **E(n)**.

16.7.15 Se consideră următoarele formule [Kopc75], pag. 169-171,

$$\int_o^1 e^{-x^2}\,dx \approx 1 - \frac{1}{3.1!} + \frac{1}{5.2!} - \frac{1}{7.3!} + \frac{1}{9.4!} \ldots$$

$$\int_0^{\frac{\pi}{4}} \sin x^2\,dx \approx \frac{1}{3.1!}\left(\frac{\pi}{4}\right)^3 - \frac{1}{7.3!}\left(\frac{\pi}{4}\right)^7 + \frac{1}{11.5!}\left(\frac{\pi}{4}\right)^{11} - \frac{1}{15.7!}\left(\frac{\pi}{4}\right)^{15} + \ldots.$$

$$\int_0^{\frac{1}{2}} \frac{1}{1-x^5}\,dx \approx \frac{1}{1.2^1} + \frac{1}{6.2^6} + \frac{1}{11.2^{11}} + \frac{1}{16.2^{16}} + \ldots$$

ce se obțin ca urmare a dezvoltării în serii de puteri a funcţiilor de sub semnul integralei, integrarea termen cu termen şi introducerea limitelor de integrare.

Se va elabora programul pentru calculul valorilor acestor integrale utilizând **n**=20 de termeni din dezvoltările în serie.

16.7.16 Integralele eliptice de **speţa I** şi **II**,

$$F(e,\,k) = \int_0^{\varphi} \frac{d\vartheta}{\sqrt{1 - k^2 \sin^2 \vartheta}}\,,\quad E(e,\,k) = \int_0^{\varphi} \sqrt{1 - k^2 \sin^2 \vartheta}\;d\vartheta$$

în cazul în care $\varphi=\pi/2$, se numesc **complete** şi se pot calcula cu următoarele serii de puteri:

$$F(k) = \frac{\pi}{2}\left\{1 + \left(\frac{1}{2}\right)^2 k^2 + \left(\frac{1.3}{2.4}\right)^2 k^4 + \left(\frac{1.3.5}{2.4.6}\right)^2 k^6 + \ldots\ldots + \left[\frac{(2n-1)!!}{(2n)!!}\right]^2 k^{2n} + \ldots\right\},$$

$$E(k) = \frac{\pi}{2}\left\{1 - \left(\frac{1}{2}\right)^2 \frac{k^2}{1} - \left(\frac{1.3}{2.4}\right)^2 \frac{k^4}{3} - \left(\frac{1.3.5}{2.4.6}\right)^2 \frac{k^6}{5} - \ldots\ldots - \left[\frac{(2n-1)!!}{(2n)!!}\right]^2 \frac{k^{2n}}{2n-1} - \ldots\right\}.$$

Să se calculeze **F(k)** şi **E(k)**, cu ajutorul unui program în cadrul căruia să se utilizeze o ***relaţie de recurenţă*** între termenii succesivi ai seriilor, din fiecare serie luând câte **n** termeni (de exemplu n = 10). Valorile numerice pentru **k** se vor lua din intervalul (-1,+1). Se vor afişa valorile introduse ale lui **k** şi **n** precum şi valorile calculate ale celor două integrale.

16.7.19 În manualele şi culegerile de probleme de calcul integral se găsesc sau se propune demonstrarea următoarelor formule :

$$I_1 = \int_0^1 \frac{1}{x+1}\,dx = \lim_{n\to\infty}\left(\frac{1}{n+1} + \frac{1}{n+2} + \ldots + \frac{1}{2n}\right) = \ln 2 = 0.6931471805\ldots.$$

$$I_2 = \int_0^1 \frac{1}{\sqrt{4-x^2}}dx = \lim_{n\to\infty}\left(\frac{1}{\sqrt{4n^2-1^2}}+\frac{1}{\sqrt{4n^2-2^2}}+...+\frac{1}{\sqrt{4n^2-n^2}}\right)=\frac{\pi}{6}=0.5235987755...$$

$$I_3 = \int_0^1 x^4\ dx = \lim_{n\to\infty}\frac{1}{n^5}\left(1^4+2^4+...+n^4\right)=\frac{1}{5}=0.2$$

$$I_4 = \int_0^1 x^5\ dx = \lim_{n\to\infty}\frac{1}{n^6}\left(1^5+2^5+...+n^5\right)=\frac{1}{6}=0.166666...$$

$$I_5 = \int_0^1 x\ e^{x^2}\ dx = \lim_{n\to\infty}\frac{1}{n^2}\left[\ 1.e^{\left(\frac{1}{n}\right)^2}+2.e^{\left(\frac{2}{n}\right)^2}\ +...+n.e^{\left(\frac{n}{n}\right)^2}\ \right]=\frac{e-1}{\sqrt{2}}=1.2150087325...$$

$$I_6 = \int_0^1 \frac{1}{1+x^2}\ dx = \lim_{n\to\infty}\frac{1}{n}\left[\frac{9n^2}{9n^2+2^2}+\frac{9n^2}{9n^2+5^2}+...+\frac{9n^2}{9n^2+(3n-1)^2}\right]=\frac{\pi}{4}=0.7853981633$$

$$I_7 = \int_0^{\pi}\sin x\ dx = \lim_{n\to\infty}\frac{\pi}{n}\left[\sin\frac{\pi}{n}+\sin\frac{2\pi}{n}+...+\sin\frac{(n-1)\pi}{n}\right]=2.0$$

$$I_8 = \int_0^1 \frac{1}{(1+x)^2}\ dx = \lim_{n\to\infty} n\left[\frac{1}{(n+1)^2}+\frac{1}{(n+2)^2}+...+\frac{1}{(2n)^2}\right]=\frac{1}{2}=0.5$$

$$I_9 = \int_0^1 \frac{1}{\sqrt{2x+5}}\ dx = \lim_{n\to\infty}\frac{1}{\sqrt{n}}\left(\frac{1}{\sqrt{2+5n}}+\frac{1}{\sqrt{4+5n}}+...+\frac{1}{\sqrt{2n+5n}}\right)=\sqrt{7}-\sqrt{5}=0.4096833335...$$

$$I_{10} = \int_0^1 \sqrt{1-x^2}\ dx = \lim_{n\to\infty}\frac{1}{5n^2}\left[\sqrt{(5n)^2-2^2}\ +\sqrt{(5n)^2-7^2}\ +...+\sqrt{(5n)^2-(5n-3)^2}\right]=\frac{\pi}{4}$$

la expresia funcției de sub semnul integralei şi la valorile limitelor ajungându-se prin scrierea expresiilor ce se trec la limită sub forma unor **sume Riemann** (Georg Friedrich Riemann 1826-1866). Pentru fiecare caz în parte (sau pentru mai multe deodată, eventual toate) se va elabora un program în cadrul căruia să se calculeze valorile expresiilor ce se trec la limită pentru toate valorile lui **n** din intervalul **[10; 25]**, pe fiecare linie afişându-se valoarea lui **n**, valoarea calculată a expresiei şi eroarea față de valoarea exactă a integralei.

16.7.20 Se consideră integrala definită:

$$I = \int_0^1 \frac{dx}{x+1}$$

a cărei valoare exactă este **ln 2** = 0.6931471805.... şi se cere să se calculeze valoarea acesteia aplicând în mod repetat formula dreptunghiurilor în cazul împărţirii intervalului în **n** părţi:

$$\int_a^b f(x)dx \approx h \sum_{i=1}^{n} f\left(a - \frac{h}{2} + i\,h \right), \qquad h = \frac{b - a}{n} ,$$

n luând valorile 1, 2, 3, ... , 25. Se vor afişa valorile calculate ale integralei cu formula dreptunghiurilor în fiecare din aceste cazuri, precum şi eroarea faţă de rezultatul exact.

16.7.21 Se consideră expresia:

$$E(a) = \frac{2}{\sqrt{\pi}} \int_o^a e^{-x^2} dx$$

şi se ştie că limita funcţiei **E(a)** este **1** când **a** tinde la infinit. Se cunoaşte de asemenea formula generalizată a trapezelor pentru calculul valorii unei integrale definite:

$$\int_a^b f(x)dx \approx \frac{h}{2}[f(a) + 2\sum_{i=1}^{n} f(a + ih) - f(b)], \qquad h = \frac{b - a}{n}$$

Să se realizeze programul pentru calculul valorilor expresiei **E(a)** pentru diferite valori ale lui **a** şi **n.**

16.7.22 Să se elaboreze un program similar cu cel corespunzator problemei anterioare, în locul formulei generalizate a trapezelor utilizându-se formula generalizată a lui **Simpson**:

$$\int_a^b f(x)\,dx \approx \frac{h}{6}\left[f(a) + 4 \sum_{i=1}^{n} f\left(a - \frac{h}{2} + i\,h \right) + 2 \sum_{i=1}^{n} f(a + i\,h) - f(b) \right], \quad h = \frac{b - a}{n}$$

16.7.23 Calculul valorii unei integrale definite se poate face cu următoarea formulă, numită formula lui **Newton** (Isaac Newton 1642-1727) [Kopc75], pg. 150 ,

$$\int_a^b f(x)dx \approx \frac{3h}{8}\left\{ f(a) - f(b) + 2\sum_{i=1}^{n} f(a + ih) + 3\sum_{i=1}^{n}\left[f\left(a - \frac{2h}{3} + i\,h \right) + f\left(a - \frac{h}{3} + i\,h \right) \right] \right\}$$

unde **h=(b-a)/n**. În cazul **f(x) = 1/(1+x^2)** , **a=0**, **b=1**, să se calculeze valoarea integralei definite pentru toate valorile lui **n** de la 5 la 50, pentru fiecare caz afişându-se valoarea lui **n**, valoarea aproximativă a integralei calculată cu formula lui Newton precum şi eroarea faţă de valoarea exactă **π/4**.

16.7.24 Se consideră formula lui **Newton-Cotes** pentru calculul aproximativ al unei integrale definite,

$$\int_a^b f(x)\ dx \approx (b-a)\sum_{k=1}^{n+1} c_k\ f\left[a+(k-1)h\right], \qquad h=\frac{b-a}{n}$$

unde $\mathbf{c_k}$ sunt coeficienţii lui Cotes, care depind de numărul de părţi **n** în care se împarte intervalul **[a ; b]**. Valorile acestora sunt următoarele :

n	**Coeficienţii lui Cotes**						
	$\mathbf{c_1}$	$\mathbf{c_2}$	$\mathbf{C_3}$	$\mathbf{c_4}$	$\mathbf{c_5}$	$\mathbf{c_6}$	$\mathbf{c_7}$
1	1/2	1/2					
2	1/6	4/6	1/6				
3	1/8	3/8	3/8	1/8			
4	7/90	32/90	12/90	32/90	7/90		
5	19/288	75/288	50/288	50/288	75/288	19/288	
6	41/840	216/840	27/840	272/840	27/840	216/840	41/840

Programul ce se va elabora, va realiza următoarele: introducerea unei matrice **A** şi a unui şir **B** ce conţin numărătorii respectiv numitorii coeficienţilor lui Cotes:

$$A=\begin{bmatrix} 1 & 1 & & & & & \\ 1 & 4 & 1 & & & & \\ 1 & 3 & 3 & 1 & & & \\ 7 & 32 & 12 & 32 & 7 & & \\ 19 & 75 & 50 & 50 & 75 & 19 & \\ 41 & 216 & 27 & 272 & 27 & 216 & 41 \end{bmatrix}, \qquad B=\begin{bmatrix} 2 \\ 6 \\ 8 \\ 90 \\ 288 \\ 840 \end{bmatrix},$$

afişarea datelor introduse, sub forma tabelului de mai sus, coeficienţii fiind afişaţi ca nişte rapoarte de numere naturale, calculul coeficienţilor lui Cotes, calculul repetat al valorii integralei definite

$$I=\int_o^1 \frac{dx}{1+x^2}=\frac{\pi}{4}=0.7853981633.....$$

pentru valorile lui **n** de la **1** la **6** , utilizând formula lui Newton-Cotes şi tipărirea rezultatelor obţinute şi a erorii faţă de rezultatul exact, calculat cu funcţia **atan(1.0)** .

16.7.25 Se consideră următoarea formulă pentru calculul valorii unei integrale definite:

$$\int_{-1}^{+1} f(x)dx = \sum_{i=1}^{n} a_i f(x_i)$$

unde $\mathbf{x_i}$ sunt rădăcinile polinomului lui **Legendre** (Adrien Marie LeGendre 1752-1833) de ordinul **n** ,

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n (x^2 - 1)^n}{dx^n}$$

iar coeficienţii $\mathbf{a_i}$ se calculează cu formula:

$$a_i = \frac{2}{\left(1 - x_i^2\right)\left[P_n'(x_i)\right]^2}$$

această formulă de cuadratură fiind dedusă de către **Gauss** (Karl Friederich Gauss 1777-1855). Valorile absciselor $\mathbf{x_i}$ şi a coeficienţilor $\mathbf{a_i}$, (i=1,n), pentru diferite valori ale lui **n** sunt următoarele:

```
n=1    x1=  0                      a1= 2,
n=2    x1=- 0.57735,  x2 = - x1,   a1= 1, a2=1,
n=3    x1=- 0.77459,  x2=  0,      x3=-x1,
       a1=  0.55555,  a2=  0.88888, a3= a1
n=4    x1=- 0.86133,  x2=- 0.33998, x3=-x2,       x4=-x1
       a1=  0.34785,  a2=  0.65214, a3= a2,       a4=a1
n=5    x1=- 0.90617,  x2=- 0.53846, x3= 0,        x4=x2, x5=x1,
       a1=  0.23692,  a2=  0.47862, a3= 0.56888,  a4=a2 , a5=a1.
```

Se cere să se elaboreze programul pentru calculul valorii integralei definite:

$$I = \int_{-1}^{+1} \frac{1}{x+3} dx = \ln 2 = 0.6931471805....$$

utilizând de cinci ori formula de cuadratură a lui Gauss, pentru fiecare din valorile lui **n**, pentru fiecare caz afişând valoarea lui **n**, valorile şirurilor absciselor şi coeficienţilor, valoarea aproximativă a integralei şi eroarea faţă de rezultatul exact.

16.7.26 Pentru a calcula integrala

$$\int_{a}^{b} f(x)dx$$

utilizând formula de cuadratură a lui **Gauss** se face schimbarea de variabilă

$$x = \frac{b+a}{2} + \frac{b-a}{2} t,$$

pentru ca limitele de integrare să devină **-1** şi **+1**, formula de calcul a integralei fiind

$$\int_a^b f(x)\,dx \approx \frac{b-a}{2}\sum_{i=1}^{n} a_i\, f(x_i)\,, \quad x_i = \frac{b+a}{2} + \frac{b-a}{1}\,t_i\,.$$

Să se elaboreze programul pentru calculul integralei

$$\int_0^1 \frac{1}{1+x^2}\,dx,$$

dacă se consideră **n = 7** iar valorile coeficienţilor $\mathbf{a_i}$ şi a absciselor $\mathbf{t_i}$ sunt următoarele:

$a_1=a_7=0.129484966$; - $t_1=t_7=0.949107912$;
$a_2=a_6=0.279705391$; - $t_2=t_6=0.741531186$;
$a_3=a_5=0.381830051$; - $t_3=t_5=0.405845151$;
$a_4=0.417959184$; $t_4=0$

Se va afişa şirul coeficienţilor, şirul absciselor, valoarea calculată a integralei calculată cu metoda lui Gauss şi valoarea exactă **π/4**.

16.7.27 Se consideră următoarea formulă a lui **Cebâşev** pentru calculul valorii unei integrale definite:

$$\int_{-1}^{+1} f(x)\,dx = \frac{2}{n}\sum_{i=1}^{n} f(x_i)\,,$$

valorile absciselor $\mathbf{x_i}$ în funcţie de valoarea considerată pentru **n** fiind următoarele:

$n=1$ $x_1=0$

$n=2$ $x_1=-0.577350\,,\ x_2=-x_1\,,$

$n=3$ $x_1=-0.707107\,,\ x_2=0\,,\ x_3=-x_1\,,$

$n=4$ $x_1=-0.794654\,,\ x_2=-0.187592\,,\ x_3=-x_2\,,\ x_4=-x_1\,,$

$n=5$ $x_1=-0.832497\,,\ x_2=-0.374541\,,\ x_3=0\,,\ x_4=-x_2\,,\ x_5=-x_1$

$n=6$ $x_1=-0.866247\,,\ x_2=-0.422519\,,\ x_3=-0.266635\,,\ x_4=-x_3\,,$
$x_5=-x_2\,,\ x_6=-x_1\,.$

Se cere să se elaboreze un program pentru calculul valorii integralei definite:

$$I=\int_{-1}^{+1}\frac{1}{x+3}\,dx = \ln 2 = 0.6931471805....$$

utilizând de şase ori formula de cuadratură a lui Cebâşev pentru fiecare din valorile lui **n**, de fiecare dată calculând şi eroarea faţă de valoarea exactă. Pentru fiecare caz considerat se va tipări valoarea lui **n**, valorile celor **n** abscise, valoarea aproximativă a integralei şi eroarea faţă de rezultatul exact.

16.7.28 Se consideră integrala definită (0 < a < 1)

$$I=\int_0^a \frac{arctg\ \ x}{x}\,dx.$$

Cunoscând dezvoltarea în serie a funcţiei **arctg x**,

$$arctg\ x = x - \frac{x^3}{3} + \frac{x^5}{5} - ... + (-1)^{k-1}\frac{x^{2k-1}}{2k-1} + ...$$

valabilă când **x Œ [-1 , 1]**, integrala se poate exprima sub forma

$$I_1 = a - \frac{a^3}{3^2} + \frac{a^5}{5^2} - ... + (-1)^{k+1}\frac{a^{2k-1}}{(2k-1)^2} + ... + (-1)^{n+1}\frac{a^{2n-1}}{(2n-1)^2} + ...$$

Aceeaşi integrală se poate calcula cu ajutorul formulei generalizate a dreptunghiurilor:

$$I_2 = h\sum_{i=1}^{m} f\left(-\frac{h}{2} + ih\right),\quad h = \frac{a}{m},$$

aplicată în intervalul **[0 , a]**, unde

$$f(x) = \frac{arctg\ x}{x}$$

Se va elabora un program în cadrul căruia, pentru o valoare dată lui **a (0<a<1)** se vor calcula valorile lui $\mathbf{I_1}$ utilizând pe rând un număr de termeni din **dezvoltarea în serie**, de la **1** la **20**, afişându-se pe fiecare linie valoarea numărului de termeni şi valoarea lui $\mathbf{I_1}$ iar apoi se va calcula valoarea lui $\mathbf{I_2}$ pentru valorile lui **m** din acelaşi interval, de asemenea afişându-se pe fiecare rând valoarea utilizată a lui **m** şi cea a integralei $\mathbf{I_2}$, calculată cu **metoda dreptunghiurilor**. Programul va permite introducerea repetată a diferitelor valori pentru **a**. În cazul când **a=1** se va obţine

$$I = \int_0^1 \frac{arctg\ x}{x}\ dx = 0.915965.....$$

numărul iraţional ce apare ca rezultat numindu-se ***constanta lui Catalan*** [Fiht64] vol. II, pag. 431.

16.8 Operaţii cu polinoame

16.8.1 Se consideră două polinoame, unul de gradul **m** iar celălalt de gradul **n** cu coeficienţi cunoscuţi :

$$A_m(x) = a_0 + a_1 x + a_2 x^2 + ... + a_m x^m,\quad B_n(x) = b_0 + b_1 x + b_2 x^2 + ... + b_n x^n.$$

Să se determine şirul coeficienţilor polinomului care se obţine ca urmare a adunării (scăderii) celor două polinoame.

16.8.2 Se consideră două polinoame, unul de gradul **m** iar celălalt de gradul **n** cu coeficienţi cunoscuţi :

$$A_m(x) = a_0 + a_1 x + a_2 x^2 + ... + a_m x^m,\quad B_n(x) = b_0 + b_1 x + b_2 x^2 + ... + b_n x^n.$$

Să se determine şirul coeficienţilor polinomului care se obţine ca urmare a înmulţirii celor două polinoame.

16.8.3 Se consideră un polinom de gradul **n** al cărui coeficienţi sunt cunoscuţi, care se împarte cu **x – b** ,

$$a_0 + a_1 x + a_2 x^2 + ... + a_n x^n = \left(c_0 + c_1 x + c_2 x^2 + ... + c_{n-1} x^{n-1} \right)(x-b) + r,$$

obţinându-se un ***polinom cât*** de gradul **n-1** şi un ***rest*** **r** . Să se elaboreze programul cu care să se determine coeficienţii polinomului cât şi restul .

16.8.4 Se consideră polinomul de gradul **n**:

$$A_n(x) = a_1 x^n + a_2 x^{n-1} + ... + a_n \ x + a_{n+1}$$

şi se cere să se elaboreze programul pentru calculul coeficienţilor polinomului obţinut în felul următor:

$$B_n(x) = A_n(x + h) = b_1 x^n + b_2 x^{n-1} + ... + b_n x + b_{n+1}$$

prin înlocuirea nedeterminatei **x** prin **x+h**. În cadrul programului ce se va scrie se va utiliza metoda schemei lui Horner generalizate, cunoscută din algebră.

16.8.5 Se consideră polinomul de gradul **n**

$$P_n(x) = a_1 x^n + a_2 x^{n-1} + ... + a_n x + a_{n+1}$$

şi se cere să se elaboreze programul pentru calculul valorilor acestui polinom şi a celor trei polinoame care rezultă în urma derivării acestuia o dată, de două ori şi de trei ori, în intervalul **[x_i, x_f]** cu pasul **h**.

Programul va realiza următoarele: introducerea datelor **n** , x_i , x_f , **h** şi a şirului coeficienţilor **{ a_i },** calculul coeficienţilor b_i (i = 1, n), c_i (i = 1, n-1) şi d_i (i = 1, n-2) a celor trei polinoame ce rezultă în urma derivării.

$$y' = b_1 x^{n-1} + b_2 x^{n-2} + ... + b_{n-1} x + b_n,$$
$$y'' = c_1 x^{n-2} + c_2 x^{n-3} + ... + c_{n-2} x + c_{n-1},$$
$$y''' = d_1 x^{n-3} + d_2 x^{n-4} + ... + d_{n-3} x + d_{n-2}.$$

programându-se relaţiile de legătură între coeficienţii polinoamelor **{b_i}** cu **{a_i}**, **{c_i}** cu **{b_i}**, **{d_i}** cu **{c_i}**, relaţii ce se vor deduce, afişarea coeficienţilor celor patru polinoame, prezentarea sub formă tabelară a şirului de abscise **x** din intervalul dat şi a valorilor celor patru polinoame corespunzătoare fiecărei abscise.

16.8.6 Polinomul lui **Hermite** de gradul **n** se calculează cu următoarea formulă :

$$H_n(x) = \sum_{k=0}^{\left[\frac{n}{2}\right]} \frac{(-1)^k \ n!}{k! \ (n - 2k)!} (2x)^{n-2k}$$

Se va elabora un program în cadrul căruia pentru diferite valori ale lui **n** şi **x,** introduse repetat, să se calculeze valoarea acestui polinom, prin calcularea sumei termenilor daţi de formulă.

Pentru a calcula valorile termenilor sumei se va utiliza o relaţie de recurenţă între termeni succesivi, care se va deduce.

16.8.7 Se consideră un polinom de gradul **n** cu coeficienţii cunoscuţi,

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + ... + a_n x^n \ .$$

Să se determine elementele unei matrice cu **n+1** coloane şi **n+1** linii care conţine (pe linii) coeficienţii polinomului dat precum şi coeficienţii polinoamelor care rezultă în urma derivărilor succesive ale polinomului iniţial (derivările efectuându-se până când se ajunge la un polinom de gradul zero).

16.8.8 Se consideră un polinom de gradul **n** cu coeficienţii cunoscuţi,

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + ... + a_n x^n \ .$$

Să se determine coeficienţii polinomului $(x - b) P_n(x)$.

16.8.9 Se consideră un polinom de gradul **n** cu coeficienţii cunoscuţi,

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + ... + a_n x^n \ .$$

Să se determine coeficienţii polinomului $R_{n+2}(x) = (x^2 + b^2) P_n(x)$.

16.8.10 Se consideră un polinom de gradul **n** cu coeficienţii cunoscuţi,

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + ... + a_n x^n \ .$$

Să se determine coeficienţii polinomului care rezultă ca urmare a calculului expresiei următoarei integrale:

$$R_{n+1}(x) = \int_0^x P_n(\xi)\, d\xi$$

16.8.11 Se consideră polinoamele definite în felul următor :

$$G_0(x) = 1,\ G_1(x) = x - 1\ ,\ G_i(x) = (x - 2i + 1)\, G_{i-1}(x) - (i-1)^2\, G_{i-2}(x),\ i = 2,3,4,...$$

Se vor deduce relaţiile de recurenţă între coeficienţii polinoamelor de diferite grade şi se vor afişa sub forma unei matrice, pe liniile căreia se vor afla coeficienţii câte unui polinom.

16.9 Sisteme de numeraţie

16.9.1 Se va elabora programul pentru determinarea numerelor prime din intervalul [1;1010]. Problema este cunoscută din antichitate sub denumirea de "ciurul" lui **Eratostene** (284 - 192 î. Ch.). Rezultatele se vor prezenta sub forma:

```
   1    2    3    5    7   11   13   17   19   23
  29   31   37   41   43   47   53   59   61   67
 ...  ...  ...  ...  ...  ...  ...  ...  ... ....
 941  947  953  967  971  977  983  991  997 1009
```

16.9.2 Se consideră două numere întregi **a** şi **b (a>b)**. Se cere să se determine **c.m.m.d.c.** al celor două numere programând procedeul cunoscut sub numele de algoritmul lui

Euclid (450? – 380? î. Ch.). Programul va asigura introducerea repetată a datelor şi afişarea valorii celui mai mare divizor pentru fiecare pereche introdusă de numere întregi .

16.9.3 Se cere să se elaboreze programul pentru tipărirea tabelelor de adunare şi înmulţire a cifrelor octale. Rezultatele se vor prezenta sub forma următoare:

```
-------------------------------  --------------------------------
| + | 0  1  2  3  4  5  6  7 |  | * |  0  1  2  3  4  5  6  7  |
|---|------------------------|  |---|---------------------------|
| 0 | 0  1  2  3  4  5  6  7 |  | 0 |  0  0  0  0  0  0  0  0  |
| 1 | 0  1  3  4  5  6  7 10 |  | 1 |  0  1  2  3  4  5  6  7  |
.............................  .................................
| 7 | 7 10 11 12 13 14 15 16 |  | 7 |  0  7 16 25 34 43 52 61  |
-------------------------------  --------------------------------
```

16.9.4 Să se elaboreze schema logică şi programul pentru realizarea conversiei unor numere cu maximum 10 cifre, scrise în **baza 8**, în numere scrise în **baza 2**. Programul va permite introducerea repetată a datelor. Reamintim faptul că dacă numărul octal se consideră sub forma unui şir de **n** cifre octale corespondentul său în binar va fi un şir format din **3n** cifre binare, fiecărei cifre octale îi va corespunde o grupă de trei cifre binare, numărul format din cele trei cifre binare având valoarea egală cu valoarea cifrei octale căreia îi corespunde.

16.9.5 Să se elaboreze programul pentru realizarea operaţiei de conversie a numerelor întregi scrise în **baza 2** (considerate ca şiruri de caractere) în numere scrise în **baza 8**, asigurându-se introducerea repetată a datelor. Se va ţine cont de observaţia de la problema anterioară, formând grupe de câte trei cifre binare de la dreapta la stinga.

16.9.6 Pentru a realiza scrierea unui număr exprimat în sistemul de numeraţie binar, în sistemul de numeraţie hexazecinal, se procedează în felul următor: începând de la virgulă se formează grupe de câte patru cifre binare care se consideră că formează câte un număr; se evaluează acest număr, a cărui valoare în zecimal poate fi de la 0 (0000_2) la 15 (1111_2) şi se înlocuieşte cu cifra hexazecimală corespunzătoare 0 , 1 , .. , E , F, rezultând un şir de caractere. Programul ce se va elabora va realiza următoarele: introducerea şi afişarea a două şiruri, conţinând cifrele numărului binar, din partea întreagă şi din cea subunitară, calculul valorilor (în zecimal) a numerelor binare formate din câte patru cifre, formarea a două şiruri de caractere conţinând cifrele hexazecimale corespunzătoare valorilor zecimale calculate, afişarea celor 2 şiruri de caractere.

16.9.7 Se cere să se elaboreze programul în cadrul căruia să se realizeze în mod repetat următoarele: citirea şi afişarea unui număr întreg cu maximum 8 cifre, determinarea şirului cifrelor numărului, afişarea cifrelor determinate sub formă de coloană.

16.9.8 Se consideră un şir de **n+m** cifre binare, aparţinând unui număr binar cu **n** cifre în partea întreagă şi **m** cifre în partea subunitară. Se va elabora programul în cadrul căruia se vor realiza următoarele: introducerea lui **n** şi **m** şi a elementelor şirului **A** format din **n+m** termeni (**0** sau **1**), afişarea şirului introdus, calculul valorilor părţii întregi, a celei subunitare şi a numărului în întregime şi afişarea acestora (în sistemul zecimal). Programul va asigura introducerea repetată a datelor, precum şi validarea valorii numerice a fiecărui element $\mathbf{a}_i$ introdus, care poate fi numai **0** sau **1**, în caz contrar revenindu-se la introducerea unor noi valori pentru **n** şi **m**.

16.9.9 Se consideră un număr întreg **a** , în sistemul zecimal. Programul ce se va scrie, va realiza determinarea pe rând a cifrelor aceluiaşi număr exprimat în sistemul de numeraţie binar, pe baza algoritmului cunoscut.

16.9.10 Se consideră un număr subunitar **a** , scris în sistemul zecimal. Programul ce se va scrie va realiza determinarea pe rând a primelor **n** cifre ale numărului binar de aceeaşi valoare cu **a**, pe baza algoritmului cunoscut. Programul va realiza introducerea repetată a datelor, afişarea valorii lui **a** (în zecimal), a numărului **n** de cifre binare ce s-au determinat precum şi a şirului acestor cifre.

16.9.11 Să se determine toate numerele cu câte 3 , 4 şi 5 cifre care să aibă următoarea proprietate:

$$\overline{a_1 a_2 a_3} = a_1! + a_2! + a_3! \qquad \overline{a_1 a_2 a_3 a_4} = a_1! + a_2! + a_3! + a_4!$$

$$\overline{a_1 a_2 a_3 a_4 a_5} = a_1! + a_2! + a_3! + a_4! + a_5!$$

în toate cazurile $\mathbf{a_1 \geq 1.}$ Se vor afişa acele numere pentru care relaţia este verificată.

Exemple: 145 = 1! + 4! + 5!, 40585 = 4! + 0! + 5! + 8! + 5! ş. a.

16.9.12 Să se determine toate numerele întregi din intervalul [1; 99999] care verifică relaţia

$$\sqrt[3]{\overline{a_1 a_2 ... a_n}} = \sum_{i=1}^{n} a_i$$

pe ecran afişându-se doar cazurile care satisfac relaţia. Rezultatele se vor prezenta sub următoarea formă:

```
Nr.gasit   Val. radical       Suma cifrelor
     1          1.000              1
   512          8.000              8
  4913         17.000             17
  5832         18.000             18
 17576         26.000             26
 19683         27.000             27
 .....            ..              ..
```

16.9.13 Să se determine toate numerele întregi din intervalul [**1**; **999999**] care verifică relaţia

$$\sqrt[4]{\overline{a_1 a_2 ... a_6}} = \sum_{i=1}^{6} a_i$$

pe ecran afişându-se doar cazurile care satisfac relaţia de mai sus. Rezultatele se vor prezenta sub forma:

```
Nr.gasit        Val.radical          Suma cifrelor
     1            1.000                 1
  2401            7.000                 7
234256           22.000                22
......           ......               ...
614656           28.000                28
```

16.9.14 Numerele **perfecte** sunt numere întregi a căror valoare este egală cu suma divizorilor acestora, inclusiv 1, de exemplu: 6=1+2+3; 28=1+2+4+7+14; 496=1+2+4+8+16+31+2+124+248; 8128=1+2+4+8+16+32+64+127+254+508+1016+2032+4064; ……

Se va elabora un program pentru determinarea tuturor numerelor perfecte din intervalul [4, 5000], în dreptul fiecărui număr perfect găsit afişându-se şirul divizorilor acestuia.

16.9.15 Să se determine toate numerele întregi din intervalul [1 ; 99999] care verifică relaţia:

$$\sqrt[n]{\overline{a_1 a_2 \cdots a_n}} = \sum_{i=1}^{n} a_i$$

la terminal afişându-se doar cazurile care satisfac relaţia.

```
Nr.gasit          Val.radical          Suma cifrelor
    1               1.000000                1
   81               9.000000                9
  512               8.000000                8
 2401               7.000000                7
 ....               ........               ..
```

16.9.16 Se consideră şirul de numere:

$a_1 = 49$, $a_2 = 4489$, $a_3 = 444889$, $a_4 = 44448889$, …. , $a_k = 444...488…89$, ….

Se va elabora schema logică şi programul pentru calculul primilor opt termeni ai acestui şir precum şi a valorilor radicalilor din aceşti termeni.

Rezultatele se vor prezenta sub forma:

```
k                A(k)              SQRT(A(k))
**               ****              **********
1                  49.0             7.0
2                4489.0             67.0
3              444889.0             667.0
4            44448889.0             6667.0
.
8       ---------------             -------
```

16.9.17 Numerele lui **Armstrong** sunt acelea care îndeplinesc relaţia:

$$\overline{a_1\ a_2 \cdots\ a_n} = a_1^n + a_2^n + \ldots + a_n^n \ ,\ (a_1 \geq 1) \ ,$$

cum ar fi de exemplu: $153=1^3+5^3+3^3$. Să se determine toate numerele cu 2, 3 şi 4 cifre care au această proprietate. Astfel de numere sunt următoarele : 153, 370, 371, 407, 1634, 8208, 9474, ...

16.9.18 Să se determine toate numerele cu 1, 2, 3 şi 4 cifre prin ridicarea cărora la pătrat să se obţină numere a căror cifre finale să fie identice cu cifrele numărului ridicat la patrat, de exemplu : $5^2 = 25$; $6^2 = 36$; $25^2 = 625$; $76^2 = 5776$; $376^2 = 1413376$; $625^2 = 390625$; .....

16.9.19 Dându-se un număr întreg **N** (impar) să se determine cele două numere consecutive a căror sumă să fie egală cu **N**.

16.9.20 Dându-se un număr întreg **N** să se determine dacă există trei numere consecutive a căror sumă să fie egală cu **N**.

16.9.21 Dându-se un număr întreg **N** să se determine dacă există două numere întregi **A** şi **B** (mai mici decât **N**) astfel încât cele trei numere să fie **pitagorice** ($\mathbf{A^2 + B^2 = N^2}$).

16.9.22 Se consideră intervalul de numere întregi [10; 30000]. Se cere să se determine care din numerele din interval sunt **palindrome** (cifrele numărului sunt simetrice în raport cu mijlocul , de exemplu : ..., 33, ..., 131, ..., 4554, ... , 23632 ...).

16.9.23 Laturile unui triunghi au lungimi care se exprimă prin numere întregi. Dacă perimetrul este **P** (număr întreg) să se determine lungimile posibile ale laturilor.

16.9.24 Să se verifice dacă un număr natural **n** este un cub perfect.

16.9.25 Să se determine toate numerele din intervalul [1; 99999] a căror sumă a cifrelor este divizibilă cu 2 şi cu 3 .

16.9.26 Să se determine toate numerele din intervalul [1; 99999] a căror sumă a cifrelor este divizibilă cu **3** şi cu **5** .

16.9.27 Se generează un şir format din 100 de numere aleatoare întregi din intervalul 1 ~ 100. Se cere să se determine care sunt tripletele de numere consecutive care ar putea fi lungimile laturilor unui triunghi .

16.9.28 Se consideră un şir de **n** numere întegi. Se cere să se determine numărul numerelor pare şi al celor impare.

16.9.29 Considerând intervalul de numere întregi [1 ; 9999] să se determine numerele prime care sunt şi palindrome.

16.9.30 Să se determine toate numerele întregi cu **n** cifre care adunate cu oglinditul lor dau un patrat perfect ([Pătr98] pg. 52 pr. 100),

$$\overline{a_1 a_2 \ldots a_{n-1} a_n} + \overline{a_n a_{n-1} \ldots a_2 a_1} = \left(\overline{b_1 b_2 \ldots b_m} \right)^2 .$$

16.9.31 Să se genereze şi să se afişeze toate numerele întregi binare cu **u+z** cifre care au **u** cifre de **1** şi **z** cifre de **0**, de asemenea şi valorile acestor numere în zecimal.

Ca date numerice se vor lua u = 4 ; z = 3 ;

16.9.32 Să se elaboreze un program care să realizeze afişarea "în cuvinte" a valorii unui număr. De exemplu, dacă se va tasta numărul **32456** atunci va trebui să se tipărească textul :

```
trei zeci doua mii patru sute cinci zeci sase
```

16.9.33 Program pentru determinarea şirului de ***cifre romane*** corespunzătoare unui număr întreg **n** din intervalul [**1 ; 2001**] (**M** – 1000, **D** – 500, **C** – 100, **L** – 50, **X** – 10, **V** – 5, **I** – 1).

16.9.34 Se consideră un şir format din **n** litere mari (n **§** 8). Se cere să se determine dacă acest şir de caractere poate reprezenta un număr scris în sistemul de numeraţie roman iar în caz afirmativ să se determine valoarea acestui număr exprimată în sistemul zecimal.

16.9.35 Să se determine toate perechile de numere din intervalul [1; 500] care au proprietatea că fiecare număr este egal cu suma divizorilor celuilalt număr .

De exemplu perechea de numere **220** şi **284** are această proprietate .

Suma divizorilor lui 220 :

1 + 2 + 4 + 5 + 10 + 11 + 20 + 22 + 44 + 55 + 110 = 284

Suma divizorilor lui 284 :

1 + 2 + 4 + 71 + 142 = 220 .

16.9.36 Se consideră trei şiruri (de caractere) care conţin :

- şirul numerelor naturale

0 **1** 2 **3** 4 **5** 6 **7** 8 **9** 1 0 **1 1** 1 2 **1 3** 1 4 **1 5** 1 6 **1 7** 1 8 **1 9** 2 0 **2 1** 2 2 ....

- şirul patratelor numerelor naturale

0 **1** 4 **9** 1 6 **2 5** 3 6 **4 9** 6 4 **8 1** 1 0 0 **1 2 1** ....

- şirul numerelor lui Fibonacci

0 **1** 1 **2** 3 **5** 8 **1 3** 2 1 **3 4** 5 5 **8 9** 1 4 4 **2 3 3** 3 7 7 ....

Se cere să se determine în cazul celor trei şiruri care sunt cifrele care se află pe poziţia a **n**-a . Ca valoare a lui **n** se va lua un număr între 10 şi 50.

16.9.37 Se consideră un număr natural **N**. Se cere să se afişeze valoarea numărului în octal şi hexazecimal şi de asemenea valoarea numărului obţinut din cifrele numărului anterior scrise în ordine inversă, de asemenea în sistemele octal şi hexazecimal.

16.9.38 Se consideră un număr întreg **n** din intervalul [1;27]. Se cere să se determine numărul numerelor cu trei cifre, de la 100 la 999, a căror sumă a cifrelor componente este **n**.

16.9.39 Să se determine toate numerele cu trei cifre care ridicate la patrat sunt egale cu suma cuburilor cifrelor componente .

16.9.40 Se cere să se afişeze toate numerele cu patru cifre, de la 1000 la 9999, la care suma primelor două cifre este egală cu suma ultimelor două cifre (de exemplu 1533).

16.9.41 Se cere să se afişeze toate numerele cu şase cifre, de la 100000 la 999999, la care suma primelor trei cifre este egală cu suma ultimelor trei cifre (de exemplu 457961).

16.9.42 Este cunoscut faptul că orice număr natural $\mathbf{N \geq 8}$ se poate scrie sub forma: $\mathbf{N=3m+5n}$, unde **m** şi **n** au valori întregi şi pozitive . Pentru un **N** dat să se determine toate valorile posibile pentru **m** şi **n** .

16.9.43 Se consideră o fracţie **M/N** unde **M** şi **N** sunt numere naturale. Să se determine fracţia **m/n** care rezultă în urma simplificării fracţiei iniţiale.

16.9.44 Să se afişeze toate numerele cu trei cifre la care suma cifrelor este egală cu un număr **n** ($\mathbf{1 \leq n \leq 27}$) .

16.9.45 În anul 1644 Martin Mersenne**,** în lucrarea "*Cogitata Physico-Mathematica*", a afirmat că numerele $\mathbf{2^n - 1}$ sunt prime pentru următoarele valori ale lui **n** : 2, 3, 5, 7, 13, 17, 19, 31, (67), 127, (257), afirmaţia nefiind însă corectă pentru numerele 67 şi 257 iar numerele 61, 89 şi 107 au fost omise, de asemenea mai există şi alte numere 521, 607, 1279 , etc. [Knut83] pg. 431.

Se va elabora un program cu care se va verifica că primele şapte numere ale lui Mersenne (pentru următoarele valori ale lui **n** : 2, 3, 5, 7, 13, 17, 19) au proprietatea respectivă.

16.9.46 Să se determine dacă există trei numere naturale consecutive a căror produs să fie egal cu **n !**, unde **n** este un număr natural dat .

16.9.47 Se dă un număr natural **N**. Se cere să se determine toate numerele naturale din intervalul [1; N] care sunt divizibile cu 3 şi cu 7.

16.10 Sisteme de ecuaţii

16.10.1 Dacă un sistem de **n** ecuaţii algebrice liniare neomogene se rezolvă cu una din metodele: metoda matricei inverse (1), metoda eliminării totale (2), metoda eliminării parţiale (3) se efectuează următoarele numere de operaţii :

$$N_1 = (n+1)!\ S - 1, \quad N_2 = \frac{n(n+1)(4n-3)}{2}, \quad N_3 = n\left[n(n+1)-1\right],$$

unde

$$S = 1 + \frac{1}{1!} + \frac{1}{2!} + ... + \frac{1}{(n-1)!}$$

Se va elabora un program pentru calculul acestor numere de operaţii pentru toate valorile lui **n** din intervalul [**3** ; **20**]. Corespunzător fiecărui **n** din intervalul specificat se vor afişa valorile lui $\mathbf{N_1}$, $\mathbf{N_2}$ şi $\mathbf{N_3}$.

16.10.2 Se consideră sistemele de **n** ecuaţii algebrice liniare neomogene având matricele coeficienţilor necunoscutelor de forma triunghiulară (cazul **1** - cu elemente nule sub diagonala principală , cazul **2** – cu elemente nule deasupra diagonalei principale, cazul **3** – cu elemente nule sub diagonala secundară, cazul **4** – cu elemente nule deasupra diagonalei secundare), în toate aceste cazuri elementele de pe diagonala principală respectiv secundară fiind nenule .

Pentru fiecare caz în parte se va elabora câte un program, în cadrul căruia se vor programa următoarele: introducerea lui **n**, a elementelor matricei triunghiulare şi a termenilor liberi, afişarea matricei triunghiulare şi a termenilor liberi, calculul necunoscutelor şi afişarea valorilor acestora. Valoarea lui **n** se va lua între 3 şi 6.

În cazul 1 sistemul are forma :

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 + ... + ... + a_{1,j}x_j + ... + a_{1,n}x_n = b_1 \\ a_{2,2}x_2 + a_{2,3}x_3 + ... + ... + a_{2,j}x_j + ... + a_{2,n}x_n = b_2 \\ .... \\ a_{i,i}x_i + ... + a_{i,j}x_j + ... + a_{1,n}x_n = b_i \\ .... \\ a_{n,n}x_n = b_n \end{cases}$$

iar pentru rezolvarea acestuia vor fi folosite formulele :

$$x_n = \frac{b_n}{a_{n,n}}, \qquad x_i = \frac{b_i - \sum_{j=i+1}^{n} a_{i,j}x_j}{a_{i,i}}, \quad i = n-1, n-2, ..., 3, 2, 1$$

În mod similar se pot deduce relaţiile necesare pentru rezolvarea sistemelor corespunzătoare cazurilor 2, 3 şi 4 .

16.10.3 Să se rezolve următoarele două sisteme de ecuaţii, cu două respectiv cu trei necunoscute,

$$\begin{cases} a_{11}\,x_1 + a_{12}\,x_2 = b_1 \\ a_{21}\,x_1 + a_{22}\,x_2 = b_2 \end{cases}, \qquad \begin{cases} c_{11}\,x_1 + c_{12}\,x_2 + c_{13}\,x_3 = d_1 \\ c_{21}\,x_1 + c_{22}\,x_2 + c_{23}\,x_3 = d_2 \\ c_{31}\,x_1 + c_{32}\,x_2 + c_{33}\,x_3 = d_3 \end{cases},$$

utilizând regula lui **Cramer** (Gabriel Cramer 1704-1752) în cazul când acestea sunt compatibile şi determinate.

16.10.4 Determinarea valorilor soluţiilor unui sistem de **n** ecuaţii algebrice liniare neomogene

$$\mathbf{A\,X = B},$$

matricea **A** fiind simetrică ($\mathbf{A^t = A}$), se face cu următoarea metodă:

- Matricea **A** se exprimă prin produsul a două matrice triunghiulare $\mathbf{A = T^t\ T}$

unde:

$$T = \begin{bmatrix} t_{1,1} & t_{1,2} & t_{1,3} & \dots & t_{1,n} \\ 0 & t_{2,2} & t_{2,3} & \dots & t_{2,n} \\ 0 & 0 & t_{3,3} & \dots & t_{3,n} \\ \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & t_{n,n} \end{bmatrix}$$

valorile elementelor nenule determinându-se cu formulele prezentate în [Bucu73] pg. 52-54, [DeMa73] § 8.8, [Iorg96] pg. 36-37, [Simi95] pg. 47-48, [Ursu00] pg. 84-87, 189-191 .

Sistemul se va putea scrie sub forma **:** $\mathbf{T^t\,TX=B}$ sau $\mathbf{T^t\,Y=B}$ unde s-a notat **Y=TX**, care este un sistem cu matricea coeficienţilor necunoscutelor triunghiulară şi se poate rezolva, determinând pe **Y** iar apoi se rezolvă sistemul **TX=Y** , de asemenea cu matricea coeficienţilor necunoscutelor triunghiulară, determinând pe **X** .

Se va elabora un program pentru calculul soluţiilor unui sistem de forma **AX=B**, cu matricea **A** simetrică în cadrul căruia se vor realiza următoarele:

- introducerea lui **n**, a elementelor matricei **A** şi a vectorului **B** al termenilor liberi,
- calculul elementelor matricei **T** ,
- calculul elementelor vectorului **Y**, în urma rezolvării sistemului $\mathbf{T^t\,Y=B,}$
- calculul elementelor vectorului soluţie **X**, rezolvând sistemul **T X=Y**, .
- afişarea matricelor **A**, **T** şi a soluţiilor **X** .

Observaţie . Pentru ca factorizarea $\mathbf{A = T^t\,T}$ să fie posibilă în corpul numerelor reale este necesar ca matricea **A** să fie pozitiv definită .

16.10.5 Determinarea valorilor soluţiilor unui sistem de **n** ecuaţii algebrice liniare neomogene cu **n** necunoscute ,

$$\mathbf{A\,X = B},$$

matricea **A** fiind patratică oarecare, se face cu următoarea metodă:

- Matricea **A** se exprimă prin produsul a două matrice triunghiulare $\mathbf{A = L\,U}$

unde

$$A = L\,U = \begin{bmatrix} l_{1,1} & 0 & 0 & \dots & 0 \\ l_{2,1} & l_{2,2} & 0 & \dots & 0 \\ l_{3,1} & l_{3,2} & l_{3,3} & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots \\ l_{n,1} & l_{n,2} & l_{n,3} & \dots & l_{n,n} \end{bmatrix} \begin{bmatrix} 1 & u_{1,2} & u_{1,3} & \dots & u_{1,n} \\ 0 & 1 & u_{2,3} & \dots & u_{2,n} \\ 0 & 0 & 1 & \dots & u_{3,n} \\ \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix}$$

valorile elementelor nenule determinându-se cu formulele prezentate în [DeMa73] § 8.9, [Iorg96] pg. 34-36, [Simi95] pg. 44-46, [Pres94] pg. 43-48, [Ursu00] pg. 73-80, 186-189 (vezi Ex. 10.9).

Sistemul se va putea scrie sub forma: **LUX=B** sau **LY=B** unde s-a notat **Y=UX**, care este un sistem cu matricea coeficienţilor necunoscutelor triunghiulară şi se poate rezolva, determinând pe **Y** iar apoi se rezolvă sistemul **UX=Y** , de asemenea cu matricea coeficienţilor necunoscutelor triunghiulară, determinând pe **X** .

Se va elabora un program pentru calculul soluţiilor unui sistem de forma **AX=B**, cu matricea **A** patratică în cadrul căruia se vor realiza următoarele:

- introducerea lui **n**, a elementelor matricei **A** şi a vectorului **B** al termenilor liberi,
- calculul elementelor matricelor **L** şi **U** ,
- calculul elementelor vectorului **Y**, în urma rezolvării sistemului **LY=B,**
- calculul elementelor vectorului soluţie **X**, rezolvând sistemul **UX=Y**, .
- afişarea matricelor **A**, **L , U** şi a soluţiilor **X** .

Observaţie: pentru ca factorizarea **A=LU** să fie posibilă este necesar ca pe parcursul calculelor să nu se facă împărţiri cu valori nule, în consecinţă se vor verifica valorile elementelor care apar ca numitori în expresiile ce trebuie calculate .

16.11 Calculul derivatelor

16.11.1 Se consideră funcţia **f(x)=sin(x)** . Se va elabora programul în cadrul căruia să se realizeze următoarele:

- determinarea unui şir cu **n** termeni $\mathbf{y_1}, \mathbf{y_2}, \dots, \mathbf{y_n}$ având ca valori valorile functiei **f(x)**, calculate în **n** puncte ale unui interval parcurs cu un pas **h**, pornind de la abscisa iniţială **x = a** ;
- calculul valorilor şirului derivatelor de ordinul întâi, utilizând formulele de derivare numerică,

$$d_1 = \frac{y_2 - y_1}{h}, \qquad d_i = \frac{y_{i+1} - y_{i-1}}{2h},\ i = \overline{2,\, n-1}\,, \qquad d_n = \frac{y_n - y_{n-1}}{h}$$

- calculul valorilor şirului $\mathbf{e_i}$ al derivatelor de ordinul întîi, utilizând formula exactă,
- calculul valorilor şirului $\mathbf{s_i}$ al derivatelor de ordinul doi, utilizând formulele de derivare numerică (se utilizează aceleaşi formule, înlocuind $\mathbf{y_i}$ cu $\mathbf{d_i}$ respectiv $\mathbf{d_i}$ cu $\mathbf{s_i}$),
- calculul valorilor şirului $\mathbf{g_i}$ al derivatelor de ordinul doi, utilizând formula exactă,
- afişarea, pentru fiecare punct al intervalului, a valorilor lui **i** , a abscisei, exprimată în radiani şi în grade, a ordonatei $\mathbf{y_i}$ precum şi a derivatelor $\mathbf{d_i}$, $\mathbf{e_i}$, $\mathbf{s_i}$ şi $\mathbf{g_i}$

Ca urmare a prezentării datelor se va putea observa apariţia unor erori faţă de valorile exacte în cazul aplicării formulelor de derivare numerică.

16.11.2 În cazul unei funcţii **f(x)** date sub forma unui şir de valori $\mathbf{y_1}, \mathbf{y_2}, \dots, \mathbf{y_i}, \dots, \mathbf{y_n}$, corespunzătoare unor abscise echidistante, pasul de variaţie fiind **h,** se pot utiliza următoarele formule de derivare numerică pentru calculul primei derivate

$$d_1 = \frac{-25y_1 + 48y_2 - 36y_3 + 16y_{41} - 3y_n}{12h},\ d_2 = \frac{-3y_1 - 10y_2 + 18y_3 - 6y_4 + y_5}{12h}.$$

$$d_i = \frac{y_{i-2} - 8y_{i-1} + 8y_{i+1} - y_{i+2}}{12h},\ i = \overline{3, n-2},$$

$$d_{n-1} = \frac{-y_{n-4} + 6y_{n-3} - 18y_{n-2} + 10y_{n-1} + 3y_n}{12h},\ d_n = \frac{3y_{n-4} - 16y_{n-3} + 36y_{n\,2} - 48y_{n-1} + 25y_n}{12h}.$$

dacă funcţia se înlocuieşte pe porţiuni cu polinoame **Lagrange** de interpolare de ordinul patru (ce trec prin cinci puncte succesive) iar aceste derivate se calculează considerând că tangentele la reprezentarea grafică a funcţiei în cele cinci puncte succesive se pot înlocui cu tangentele la polinoamele Lagrange de ordinul patru ce trec prin aceleaşi puncte [DeMa73] cap. XV.

Se va elabora programul în cadrul căruia se vor realiza următoarele:

- calculul valorilor elementelor şirului $y_1, y_2, \dots, y_n$ pentru funcţia $f(x) = e^{-0.5x} \sin 3x$, considerând $n = 100$ iar intervalul fiind $[-3.5 ; 10.0]$,
- calculul derivatelor funcţiei utilizând formulele prezentate precum şi formula exactă,
- afişarea pentru fiecare din cele n puncte a numărului de ordine, abscisei, ordonatei, precum şi a celor două valori calculate ale derivatei.

16.11.3 Se consideră funcţia: $f(x) = e^{ax} x^k$ unde $a, x \in R$, $k \in N$. Se cere să se determine valoarea derivatei de ordinul n a acestei funcţii pentru valori date ale lui a, k şi x.

În cadrul programului ce se va elabora se va utiliza formula lui **Leibniz**:

$$[g(x)\,h(x)]^{(n)} = C_n^0\, g^{(n)}(x)\, h(x) + C_n^1\, g^{(n-1)}(x)\, h'(x) +$$
$$+ C_n^2\, g^{(n-2)}(x)\, h''(x) + \dots + C_n^{n-1}\, g'(x)\, h^{(n-1)}(x) + C_n^n\, g(x)\, h^{(n)}(x)$$

unde : $g(x) = e^{ax}$, $g^{(i)}(x) = a^i e^{ax}$, $h(x) = x^k$.

Pentru fiecare set de date citite se vor afişa valorile lui a, k, x precum şi valoarea calculată a derivatei.

16.11.4 Se consideră funcţia $f(x) = e^{ax} \cos bx$, a şi b fiind două constante cunoscute. Se cere să se calculeze pentru un x şi n daţi valoarea derivatei de ordinul n a acestei funcţii.

Pentru a nu se utiliza în acest caz formula de derivare a lui Leibniz, se va proceda în felul următor: se formează funcţia complexă $g(x) = e^{ax} \cos bx + i\, e^{ax} \sin bx = e^{(a+ib)x}$ a cărei derivată de ordinul n este $g^{(n)}(x) = (a + ib)^n e^{(a+ib)x}$, derivata de ordinul n a functiei $f(x)$ fiind partea reală a funcţiei $g^{(n)}(x)$, deci $f^{(n)}(x) = \mathrm{Re}\,[(a + ib)^n e^{(a+ib)x}]$.

Se vor afişa valorile introduse ale lui a, b, x şi n, precum şi valoarea derivatei.

16.12 Programarea în mod text şi mod grafic

16.12.1 Se consideră intervalul $(\ln a, e^a)$ unde $a>1$. Se cere să se determine câte numere naturale se găsesc în acest interval. Se vor trasa curbele $\ln x$ şi e^x în sistemul de axe

Oxy, o dreaptă verticală prin punctul de abscisă **x=a** şi pe această dreaptă , între reprezentările grafice ale celor două funcţii vor apărea notate cu câte un cerc de rază mică punctele având ca ordonate numere întregi.

16.12.2 Utilizând modul text, să se deseneze o spirală pornind din mijlocul ecranului, în sens orar, folosind caracterul ***** .

16.12.3 Să se deseneze un număr de **10** patrate, primul având lungimea laturilor egală cu 300 pixeli iar următoarele având vârfurile plasate pe laturile patratului anterior astfel încât să fie împărţite respectivele laturi în raportul **2 : 3 .** Culorile cu care se vor desena cele 10 patrate vor rezulta aleator.

16.12.4 Să se deseneze o placă dreptunghiulară care are o gaură circulară plasată în interiorul acesteia. Se va face cotarea respectivului desen. Liniile de contur se vor desena cu grosimea de trei pixeli iar cele de cotă cu grosimea de un pixel.

16.12.5 Să se deseneze şi coteze un şurub cu cap hexagonal, reprezentat într-o singură vedere.

16.12.6 Să se deseneze în proiecţie axonometrică sistemul de axe carteziene Oxyz.

16.12.7 Să se deseneze şi apoi să se şteargă alternativ, patrate şi cercuri, poziţia şi culoarea de desenare rezultând aleator, de asemenea lungimea laturii patratului şi a razei cercului vor rezulta aleator, cu valori în intervalul [25; 75] pixeli. În cazul în care un patrat sau un cerc nu ar încăpea pe ecran nu se va trasa.

16.12.8 Să se scrie un program care să genereze aleator litere mari pe prima linie a ecranului iar apoi să "cadă" până pe ultima linie cu viteze variabile, proporţionale cu numărul de ordine al literei în alfabet. Litera **A** va cădea în 3 secunde iar litera **Z** în 0.5 secunde.

16.13 Probleme diverse

16.13.1 Ştiind că un **inch** este egal cu **25.4 [mm]** să se elaboreze programul care să afişeze valorile lungimilor de la **0** la **250** [mm], cu pasul de **0.25** [mm], exprimate în inch.

16.13.2 Relaţiile dintre temperatura exprimată în grade **Celsius** (Anders Celsius 1701-1744) şi grade **Fahrenheit** (Daniel Gabriel Fahrenheit 1686-1736) respectiv dintre temperatura exprimată în grade **Celsius** şi **Reamur** (René Antoine Ferchault de Reamur 1683-1757) sunt următoarele

$$t_{Fahrenheit} = 9\, t_{Celsius}/5 + 32\,, \quad t_{Reamur} = 4\, t_{Celsius}/5\,.$$

Se va elabora un program cu care se vor determina temperaturile exprimate în grade **Fahrenheit** şi grade **Reamur** pentru temperaturile în grade **Celsius** din intervalul [**- 80** ; **60**] parcurs cu pasul **1**.

16.13.3 Se consideră mecanismul "***cruce de Malta***" cu **6** canale, utilizat pentru transformarea mişcării de rotaţie continuă a arborelui **I** într-o mişcare intermitentă a arborelui **II** .

Cunoscând distanţa **d** dintre arbori se cere să se calculeze valorile unghiului de rotire ψ al piesei conduse, în funcţie de valorile unghiului φ al piesei conducătoare, pentru o rotaţie completă a acesteia. Formulele de calcul sunt prezentate într-un exemplu din Cap. II, inclusiv schema logică.

Se va lua **d = 0.25 [m]** iar pasul de variaţie al unghiului **φ** se va considera **Δφ = 1 [grad]**.

16.13.4 În cazul unei roţi dinţate cu **z** dinţi drepţi, necorijaţi, ***cota peste*** **n** ***dinţi*** se calculează cu formula :

$$c = m\,[\,(n - 0.5)\,\pi + z\,(\mathrm{tg}\,\alpha_0 - \alpha_0)\,]\,\cos\alpha_0\,, \qquad \alpha_0 = 20^o,$$

unde **m** este modulul. În cadrul programului se va realiza introducerea repetată a mai multor seturi de date, valorile lui **m**, **n** şi numerele de dinţi z_1 şi z_2, calculându-se cota peste **n** dinţi pentru toate roţile dinţate având numărul de dinţi între z_1 şi z_2, cu pasul **1**.

Se vor considera următoarele seturi de date numerice: m=1.0, n=4, z_1=20, z_2=45; m=2.0, n=7, z_1=50, z_2=70; m=5.0, n=6, z_1=40, z_2=60.

16.13.5 Se va elabora programul în cadrul căruia să se realizeze următoarele:

- introducerea unui număr real, strict pozitiv, aparţinând intervalului [0.20; 1.00],
- determinarea numerelor întregi **m** şi **n** din intervalul [20; 100], al căror raport **m/n** aproximează cel mai exact numărul real introdus, afişarea acestor numere, precum şi a diferenţei dintre numărul real introdus şi numărul real găsit **m/n**.

Programul va asigura posibilitatea de introducere repetată a datelor.

În cazul în care două sau mai multe rapoarte de numere întregi aproximează cel mai bine numărul real introdus (de exemplu: 0.4=20/50.=40/100.), programul va reţine valorile mai mici ale numerelor **m** şi **n**. Se va avea în vedere exemplul B din Capitolul II.

16.13.6 Să se elaboreze un program în cadrul căruia să se realizeze următoarele: introducerea numelui şi prenumelui fiecărui student din semigrupă şi a notelor obţinute la examenele din sesiunea anterioară, calcularea mediei aritmetice a notelor, afişarea numelui, prenumelui, notelor şi mediei aritmetice, ordonarea numelor studenţilor în ordinea descrescătoare a mediilor şi afişarea acestora precum şi a mediilor obţinute. În cazul în care un student nu s-a prezentat la un anumit examen se va considera nota 0 .

16.13.7 Se consideră ecuaţia prin rezolvarea căreia se determină poziţia de echilibru, determinată de unghiul **φ**, a unui sistem mecanic ([Pope78]):

$$f(\varphi,\ m,\ L,\ k) = \frac{2\ k\ L\ \cos\varphi}{\sin\varphi}(\ 1 - \sin\varphi\ ) - 9.81\ m\ =\ 0\ .$$

unde **L** este o lungime (**L=0.5 [m]**), **k** este o constantă elastică (**k = 10^5 [N/m]**) iar **m** este o masă [kg] soluţia aparţinând intervalului [**0 ; π/2**] .

Se cere să se elaboreze un program pentru determinarea valorilor unghiului **φ** care rezultă prin rezolvarea ecuaţiei de echilibru, aplicând de **25 ori** metoda înjumătăţirii intervalelor, pentru toate valorile masei **m** din intervalul **10.0 ~ 200.0 [kg]** cu un pas de **10**.

16.13.8 Se consideră funcţia **f : Ñ → Â** dată de următoarea expresie

$$f(r) = \frac{1}{1 - r^2 + 2\ j\ \zeta\ r}\ ,\ (j = \sqrt{-1}\ )$$

utilizată în teoria vibraţiilor mecanice [Ursu98], **r** fiind variabila independentă iar **ζ** un parametru. Se vor calcula valorile acestei funcţii (partea reală, partea imaginară, modulul şi argumentul) pentru

toate valorile variabilei **r** din intervalul [0.0 ; 3.5] cu pasul 0.1, pentru următoarele valori ale parametrului ζ = 0.1, 0.25, 0.5, 0.707, 1. şi 2.

16.13.9 Arborele 1 se roteşte cu turaţia $\mathbf{n_1 = 1000}$ [rot/min] şi este legat de arborele 2 cu care face un unghi α printr-un cuplaj cardanic. Dacă φ este unghiul de rotire al arborelui 1 se ştie că turaţia arborelui 2 se calculează cu formula:

$$n_2 = n_1 \frac{\cos\alpha}{1 - \sin^2\alpha \ \cos^2\varphi} .$$

Considerând că unghiul α dintre arbori poate lua valorile: 2.5º, 5.0º, 7.5º, 10.0º, 12.5º, se cere să se elaboreze programul pentru calculul turaţiilor $\mathbf{n_2}$ pentru toate valorile unghiului φ din intervalul [**0º;360º**], cu pasul unu, pentru cele cinci valori posibile ale unghiului α.

Valorile turaţiilor $\mathbf{n_2}$ se vor afişa cu opt cifre semnificative, sub următoarea formă:

```
                        Turatia  n2
   phi       α =  2.5   α =  5.0   α =  7.5   α = 10.0   α = 12.5
   0.0       ........   ........   ........   ........   ........
   1.0       ........   ........   ........   ........   ........
   ...       ........   ........   ........   ........   ........
 360.0       ........   ........   ........   ........   ........
```

16.13.10 Un punct $\mathbf{M_i}$ legat de un cerc de rază **r** , care se rostogoleşte pe o dreaptă fixă **Ox**, descrie o **ortocicloidă** în raport cu sistemul de axe **Oxy**, ecuaţiile parametrice ale acestei curbe fiind următoarele : $\mathbf{x = r\,\varphi - l_i\ sin\,\varphi}$, $\mathbf{y = r - l_i\ cos\,\varphi}$, $\mathbf{l_i}$ fiind distanţa de la punctul $\mathbf{M_i}$ la centrul cercului. Dacă $\mathbf{l_i = r}$ atunci ortocicloida se numeşte ***normală*** iar în celelalte două cazuri, $\mathbf{l_i < r}$ respectiv $\mathbf{l_i > r}$, se numeşte ***scurtată*** respectiv ***alungită***.

Se cere să se elaboreze programul pentru calculul coordonatelor celor trei tipuri de curbe, pentru toate valorile parametrului φ din intervalul [0 ; 360] cu un pas de 5º. Valorile numerice vor fi următoarele: r=10 ; l_1=7.5 ; l_2=10 ; l_3=12.5 iar rezultatele se vor afişa pe şapte coloane, pe prima valorile în grade ale unghiului φ iar pe următoarele abscisele şi ordonatele punctelor care descriu cele trei curbe .

16.13.11 Să se elaboreze programul pentru tipărirea calendarului anual, cunoscându-se următoarele date iniţiale:

- ziua din săptămână când este 1 ianuarie (1 ianuarie 2002 – marţi),
- numărul de zile al lunii februarie (februarie are 29 de zile în anii care sunt divizibili cu 4).

Afişarea calendarului se va face în forma de prezentare din agende.

16.13.12 Ultima treaptă a rachetei purtătoare a unui satelit artificial al Pământului, ajunge la distanţa $\mathbf{r_0}$, măsurată de la centrul Pământului, după parcurgerea traiectoriei active, având viteza $\mathbf{v_0}$, a cărei direcţie face cu verticala unghiul α_0.

Să se calculeze elementele traiectoriei satelitului, în coordonate polare, pentru cazul unei rotaţii complete în jurul Pământului. Dacă satelitul ajunge la o distanţă mai mică de 200 km de suprafaţa Pământului calculul traiectoriei se va întrerupe.

Ecuaţia polară a conicei de tip elipsă este

$$\rho = \frac{p}{1 - e\cos(\theta - \theta_0)}$$

unde

$$p = \frac{r_0^2\, v_0^2 \sin^2\alpha_0}{f\,M},\quad e = \sqrt{1+\frac{2p}{r_0}\left(\frac{p}{2r_0\sin^2\alpha_0}-1\right)},\quad \theta_0 = \arctan\frac{p}{(r_0-p)tg\,\alpha_0},$$

f = 6.664 10^{-8} [uCGS] (constanta atracţiei universale), **M = 2 .10^{33}** [g] (masa Pământului).

Programul elaborat va realiza introducerea şi efectuarea calculelor pentru următoarele seturi de date:

```
r0 = 6700 [km],    v0 = 8.5  [km/s],    α0 = 80°,
r0 = 6700 [km],    v0 = 9.2  [km/s],    α0 = 75°,
r0 = 6700 [km],    v0 = 6.3  [km/s],    α0 = 90°,
r0 = 6700 [km],    v0 =10.25 [km/s],    α0 = 85°.
```

Pentru fiecare set de date se vor afişa valorile introduse **r_0, v_0** şi **α_0** şi coordonatele polare **θ** şi **r** pe două coloane, pasul de variaţie pentru **θ** alegându-se de **2.5°.**

16.13.13 Pentru fiecare valoare a parametrului **r** din intervalul [0.5; 2.0] parcurs cu pasul 0.05 ecuaţia :

$$\mathbf{f(\varphi) = \cos\varphi - r\,\sin(r\,\varphi) = 0}$$

să se rezolve prin aplicarea de **25** de ori a metodei înjumătăţirii intervalului. Intervalul iniţial, ce conţine singura rădăcină reală ce convine problemei, este **[0; π/2]**.

Observaţie. Această problemă intervine în cazul studiului poziţiei de echilibru a unui sistem mecanic, fiind prezentată în [Pope78] pg. 189-192.

16.13.14 Se consideră o tablă de şah pe care se găseste o singură piesă numită regina. Cunoscându-se regulile de deplasare ale acestei piese se cere să se elaboreze programul pentru determinarea elementelor unei matrice 8**μ**8 valorile cărora sunt numărul de poziţii pe care le poate ocupa regina pornind din unul din cele 64 patrate ale tablei de şah.

16.13.15 În cadrul calculului geometric al angrenajelor se utilizează funcţia:

$$\mathbf{inv\,\alpha = tg\,\alpha - \alpha}$$

Se cere să se elaboreze programul pentru calculul valorilor acestei funcţii în intervalul [**18° 00'** ; **22°**], pasul de variaţie al argumentului **α** fiind **10'**.

Rezultatele se vor afişa sub forma următoare:

```
Minute\Grade    18     19     20     21
     0         ....   ....   ....   ....
    10         ....   ....   ....   ....
    ..         ....   ....   ....   ....
    60         ....   ....   ....   ....
```

16.13.16 Ştiind perioadele ciclurilor **emoţional** (23 zile), **fizic** (28 zile) şi **intelectual** (33 zile) să se elaboraze un program pentru determinarea bioritmurilor unei persoane la care se cunoaşte data naşterii (an, lună, zi), pentru săptămâna curentă.

16.13.17 O mutare a piesei de şah numită nebun este dată printr-o literă (una din primele opt) şi o cifră (de la 1 la 8), de exemplu d7, aceasta fiind poziţia finală pe care ajunge pe tabla de şah.

Se va determina o matrice cu opt linii şi coloane, care va conţine cifra **1** pe toate poziţiile la care ar putea ajunge nebunul respectiv cifra **0** pe poziţiile unde acestă piesă nu poate ajunge. Valoarea **1** va fi şi în poziţia unde se află piesa.

Datele se vor afişa sub următoarea formă :

```
      1 2 3 4 5 6 7 8

a     0 0 0 1 0 0 0 0
b     0 0 0 0 1 0 0 0
c     0 0 0 0 0 1 0 1
d     0 0 0 0 0 0 1 0
e     0 0 0 0 0 1 0 1
f     0 0 0 0 1 0 0 0
g     0 0 0 1 0 0 0 0
h     0 0 1 0 0 0 0 0
```

16.13.18 O mutare a piesei de şah numită cal este dată printr-o literă (una din primele opt) şi o cifră (de la 1 la 8), de exemplu b4, aceasta fiind poziţia finală pe care ajunge pe tabla de şah.

Se va determina o matrice cu opt linii şi coloane, care va conţine cifra **1** pe toate poziţiile la care ar putea ajunge calul respectiv cifra **0** pe poziţiile unde acestă piesă nu poate ajunge. Valoarea **1** va fi şi în poziţia unde se află piesa.

Datele se vor afişa sub următoarea formă :

```
      1 2 3 4 5 6 7 8

a     0 1 0 0 0 1 0 0
b     0 0 0 1 0 0 0 0
c     0 1 0 0 0 1 0 0
d     0 0 1 0 1 0 0 0
e     0 0 0 0 0 0 0 0
f     0 0 0 0 0 0 0 0
g     0 0 0 0 0 0 0 0
h     0 0 0 0 0 0 0 0
```

16.13.19 O mutare a piesei de şah numită regină este dată printr-o **literă** (una din primele opt) şi o **cifră** (de la 1 la 8), de exemplu d7, aceasta fiind poziţia finală pe care ajunge pe tabla de şah.

Se va determina o matrice cu opt linii şi coloane, care va conţine cifra **1** pe toate poziţiile la care ar putea ajunge regina respectiv cifra **0** pe poziţiile unde acestă piesă nu poate ajunge. Valoarea **1** va fi şi în poziţia unde se află piesa.

Datele se vor afişa sub următoarea formă :

```
      1 2 3 4 5 6 7 8

a     0 0 0 1 0 0 1 0
b     0 0 0 0 1 0 1 0
c     0 0 0 0 0 1 1 1
d     1 1 1 1 1 1 1 1
e     0 0 0 0 0 1 1 1
f     0 0 0 0 1 0 1 0
g     0 0 0 1 0 0 1 0
h     0 0 1 0 0 0 1 0
```

16.13.20 Se va elabora un program care să ceară introducerea numelui, prenumelui şi a unei parole, fiind permise trei încercări pentru fiecare, tipărind după caz un text ("**Wrong surname**", "**Wrong name**", "**Wrong password**", "**Access denied**" sau "**You are welcome**".

Se vor folosi funcţii din **string.h** .

16.13.21 Să se elaboreze un program pentru afişarea tablei înmulţirii, rezultatele prezentându-se sub următoarea formă :

```
¥    0  1  2  3  4  5  6  7  8  9

0    0  0  0  0  0  0  0  0  0  0
1    0  1  2  3  4  5  6  7  8  9
.    .  .  .  .  .  .  .  .  .  .
.    .  .  .  .  .  .  .  .  .  .
8    0  8 16 24 32 40 48 56 64 72
9    0  9 18 27 36 45 54 63 72 81
```

16.13.22 În prezent (2001) în România sunt în circulaţie bancnotele de 500000, 100000, 50000, 10000 şi 5000 lei. Având o sumă de bani exprimată printr-un număr întreg **N** la care ultimele trei cifre sunt zero să se determine numărul minim de bancnote corespunzător acestei sume.

În cazul unei anumite sume **N** = 562 465 000 rezultatele se vor tipări sub forma :

```
562467000 = 500000 µ 1124
            100000 µ    4
             50000 µ    1
             10000 µ    1
              5000 µ    1
```

16.13.23 Se va elabora programul cu care se vor determina acei timpi (exprimaţi în ore, minute şi secunde) la care acele unui ceas care indică ora şi minutul sunt suprapuse.

16.13.24 Se va elabora programul cu care se vor determina acei timpi (exprimaţi în ore, minute şi secunde) la care acele unui ceas care indică ora şi minutul fac un unghi de 180 grade.

16.13.25 Să se determine în cazul unui ceas cu trei indicatoare (pentru oră, minut şi secundă) dacă există şi alţi timpi decât 0.00 (12.00) şi 6.00 (18.00) la care cele trei indicatoare sunt suprapuse sau în prelungire.

16.13.26 Cunoscând data naşterii unei persoane să se afişeze numele zodiei acesteia. Intervalele sunt următoarele:

```
20.1 – 18.2 Vărsător        23. 7 – 22. 8 Leu
19.2 – 20.3 Peşti           23. 8 – 22. 9 Fecioara
21.3 – 19.4 Berbec          23. 9 – 22.10 Balanţa
20.4 – 20.5 Taur            23.10 – 22.11 Scorpion
21.5 – 21.6 Gemeni          23.11 – 21.12 Săgetător
22.6 – 22.7 Rac             22.12 – 19. 1 Capricorn
```

16.13.27 Se citesc de la tastatură două numere **n** ($0<n<15$) şi **s** ($0<s<10^6$) şi apoi **n** valori distincte, fiecare valoare aparţinând intervalului [-1000; 1000]. Să se determine toate

mulţimile de numere dintre cele date, fiecare mulţime având proprietatea că suma elementelor ei este egală cu **s**. Fiecare mulţime se va afişa pe o linie, elementele ei fiind scrise în ordine crescătoare, despărţite prin câte un spaţiu sau câte o virgulă. (Bacalaureat iulie 2001, subiectul IV, varianta 5)

16.13.28 Se introduce de la tastatură un şir de caractere. Să se determine câte consoane şi câte vocale conţine şirul.

16.13.29 Să se scrie un program care preia o dată calendaristică de la tastatură şi determină cărei zile a săptămânii îi corespunde. Se ştie că ziua de 1 ianuarie 2100 este vineri.

16.13.30 Să se scrie un program care determină de câte ori data de 13 a lunii coincide cu ziua de vineri a săptămânii, pentru intervalul 1 ianuarie 1900 – 31 decembrie 2002. Se ştie că ziua de 1 ianuarie 2100 este vineri.

16.13.31 Să se calculeze următoarele expresii

$$E_1 = 0.1\,n - \sum_{i=1}^{n} 0.1,\ E_2 = 0.2\,n - \sum_{i=1}^{n} 0.2,\ E_3 = 0.3n - \sum_{i=1}^{n} 0.3,\ E_4 = 0.4n - \sum_{i=1}^{n} 0.4,\ E_5 = 0.5n - \sum_{i=1}^{n} 0.5$$

pentru toate valorile lui **n** din intervalul [50; 500] parcurs cu pasul 25. Să se explice motivul pentru care unele rezultate ce se vor afişa sunt eronate.

Capitolul XVII

Anexa nr. 1

Codul **ASCII** (**A**merican **S**tandard **C**ode **I**nformation **I**nterchange) extins

CARACTER	CODIFICARE în binar fl	în zecimal în octal	fl	în hexazecimal
Coduri de control ASCII				
NUL (**NUL**l - '\0')	00000000	000	**0**	00
☺ **SOH** (**S**tart **O**f **H**eading)	00000001	001	1	01
☻ **STX** (**S**tart of **TeX**t)	00000010	002	2	02
♥ **ETX** (**E**nd of **TeX**t)	00000011	003	3	03
♦ **EOT** (**E**nd **O**f **T**ransit)	00000100	004	4	04
♣ **ENQ** (**ENQ**uiry)	00000101	005	5	05
♠ **ACK** (**ACK**nowledge)	00000110	006	6	06
• **BEL** (**BEL**l - '\a')	00000111	007	**7**	07
◘ **BS** (**B**ack **S**pace -'\b')	00001000	010	**8**	08
○ **HT** (**H**orizontal **T**ab -'\t')	00001001	011	**9**	09
◙ **LF** (**L**ine **F**eed - '\n')	00001010	012	**10**	0A
♂ **VT** (**V**ertical **T**ab -'\v')	00001011	013	**11**	0B
♀ **FF** (**F**orm **F**eed - '\f')	00001100	014	**12**	0C
♪ **CR** (**C**arriage **R**eturn - '\r')	00001101	015	**13**	0D
♫ **SO** (**S**hift **O**ut)	00001110	016	14	0E
✹ **SI** (**S**hift **I**n)	00001111	017	15	0F
► **DLE** (**D**ata **L**ine **E**scape)	00010000	020	16	10
◄ **DC1** (**D**evice **C**ontrol **1**)	00010001	021	17	11
ó **DC2** (**D**evice **C**ontrol **2**)	00010010	022	18	12
‼ **DC3** (**D**evice **C**ontrol **3**)	00010011	023	19	13
¶ **DC4** (**D**evice **C**ontrol **4**)	00010100	024	20	14
§ **NAK** (**N**egative **A**c**K**nowledge)	00011001	025	21	15
■ **SYN** (**SYN**cronous idle)	00011000	026	22	16
↨ **ETB** (**E**nd of **T**ransmit **B**lock)	00010111	027	23	17
↑ **CAN** (**CAN**cel)	00011000	030	24	18
↓ **EM** (**E**nd of **M**edium)	00011001	031	25	19

←**SUB** (**SUB**stitute)	00011010	032	26	1A
→**ESC** (**ESC**ape)	00011011	033	**27**	1B
└ **FS** (**F**ile **S**eparator)	00011100	034	28	1C
↔ **GS** (**G**roup **S**eparator)	00011101	035	29	1D
▲ **RS** (**R**ecord **S**eparator)	00011110	036	30	1E
▼ **US** (**U**nite **S**eparator)	00011111	037	31	1F
BLANK	00100000	040	32	20
!	00100001	041	33	21
" ghilimele	00100010	042	34	22
# diez	00100011	043	35	23
$ dollar	00100100	044	36	24
%	00100101	045	37	25
& ampersand	00100110	046	38	26
' apostrof	00100111	047	39	27
(	00101000	047	40	28
)	00101001	050	41	29
***** asterisc	00101010	051	42	2A
+	00101011	052	43	2B
,	00101100	053	44	2C
-	00101101	054	45	2D
.	00101110	056	46	2E
/ slash	00101111	057	47	2F
0 **Cifre**	00110000	**060**	**48**	**30**
1	00110001	061	49	31
2	00110010	062	50	32
3	00110011	063	51	33
4	00110100	064	52	34
5	00110101	065	53	35
6	00110110	066	54	36
7	00110111	067	55	37
8	00111000	070	56	38
9	00111001	071	57	39
:	00111010	072	58	3A
;	00111011	073	59	3B
<	00111100	074	60	3C
=	00111101	075	61	3D
>	00111110	076	62	3E
?	00111111	077	63	3F
@ a mic rond	01000000	100	64	40
A **Litere mari**	01000001	**101**	**65**	**41**
B	01000010	102	66	42
C	01000011	103	67	43

D	01000100	104	68	44
E	01000101	105	69	45
F	01000110	106	70	46
G	01000111	107	71	47
H	01001000	110	72	48
I	01001001	111	73	49
J	01001010	112	74	4A
K	01001011	113	75	4B
L	01001100	114	76	4C
M	01001101	115	77	4D
N	01001110	116	78	4E
O	01001111	117	79	4F
P	01010000	120	80	50
Q	01010001	121	81	51
R	01010010	122	82	52
S	01010011	123	83	53
T	01010100	124	84	54
U	01010101	125	85	55
V	01010110	126	86	56
W	01010111	127	87	57
X	01011000	130	88	58
Y	01011001	131	89	59
Z	01011010	132	90	5A
[	01011011	133	91	5B
\ backslash	01011100	134	92	5C
]	01011101	135	93	5D
^ caret	01011110	136	94	5E
_ underscore (subliniere)	01011111	137	95	5F
` accent grav	01100000	140	96	60
a **Litere mici**	01100001	**141**	**97**	**61**
b	01100010	142	98	62
c	01100011	143	99	63
d	01100100	144	100	64
e	01100101	145	101	65
f	01100110	146	102	66
g	01100111	147	103	67
h	01101000	150	104	68
i	01101001	151	105	69
j	01101010	152	106	6A
k	01101011	153	107	6B
l	01101100	154	108	6C
m	01101101	155	109	6D
n	01101110	156	110	6E
o	01101111	157	111	6F

p		01110000	160	112	70
q		01110001	161	113	71
r		01110010	162	114	72
s		01110011	163	115	73
t		01110100	164	116	74
u		01110101	165	117	75
v		01110110	166	118	76
w		01110111	167	119	77
x		01111000	170	120	78
y		01111001	171	121	79
z		01111010	172	122	7A
{		01111011	173	123	7B
|		01111100	174	124	7C
}		01111101	175	125	7D
~	tilda	01111110	176	126	7E
⌂	DEL	01111111	177	127	7F
Ç	**Caracterele ASCII din**	10000000	200	128	80
ü	**setul extins**	10000001	201	129	81
é		10000010	202	130	82
â		10000011	203	131	83
ä		10000100	204	132	84
à		10000101	205	133	85
å		10000110	206	134	86
ç		10000111	207	135	87
ê		10001000	210	136	88
ë		10001001	211	137	89
è		10001010	212	138	8A
Ï		10001011	213	139	8B
Î		10001100	214	140	8C
Ì		10001101	215	141	8D
Ä		10001110	216	142	8E
Å		10001111	217	143	8F
É		10010000	220	144	90
æ		10010001	221	145	91
Æ		10010010	222	146	92
ô		10010011	223	147	93
ö		10010100	224	148	94
ò		10010101	225	149	95
û		10010110	226	150	96
ù		10010111	227	151	97
ÿ		10011000	230	152	98
Ö		10011001	231	153	99
Ü		10011010	232	154	9A
¢	cent	10011011	233	155	9B

£	liră sterlină	10011100	234	156	9C
¥	yen	10011101	235	157	9D
₧	peseta	10011110	236	158	9E
ƒ	funcţie	10011111	237	159	9F
á		10100000	240	160	A0
Í		10100001	241	161	A1
ó		10100010	242	162	A2
ú		10100011	243	163	A3
ñ		10100100	244	164	A4
Ñ		10100101	245	165	A5
ª		10100110	246	166	A6
º		10100111	247	167	A7
¿		10101000	250	168	A8
⌈	colţ stg. sus	10101001	251	169	A9
⌉	colţ dr. sus	10101010	252	170	AA
½		10101011	253	171	AB
¼		10101100	254	172	AC
¡		10101101	255	173	AD
«	deschidere citat	10101110	256	174	AE
»	închidere citat	10101111	257	175	AF
░		10110000	260	176	B0
▒		10110001	261	177	B1
▓		10110010	262	178	B2
│	**Caractere pentru**	10110011	263	179	B3
┤	**desenarea tabelelor**	10110100	264	180	B4
╡		10110101	265	181	B5
╢		10110110	266	182	B6
╖		10110111	267	183	B7
╕		10111000	270	184	B8
╣		10111001	271	185	B9
║		10111010	272	186	BA
╗		10111011	273	187	BB
╝		10111100	274	188	BC
╜		10111101	275	189	BD
╛		10111110	276	190	BE
┐		10111111	277	191	BF
└		11000000	300	192	C0
┴		11000001	301	193	C1
┬		11000010	302	194	C2
├		11000011	303	195	C3
─		11000100	304	196	C4
┼		11000101	305	197	C5
╞		11000110	306	198	C6
╟		11000111	307	199	C7
╚		11001000	310	200	C8

╔		11001001	311	201	C9
╩		11001010	312	202	CA
╦		11001011	313	203	CB
╠		11001100	314	204	CC
═		11001101	315	205	CD
╬		11001110	316	206	CE
╧		11001111	317	207	CF
╨		11010000	320	208	D0
╤		11010001	321	209	D1
╥		11010010	322	210	D2
╙		11010011	323	211	D3
╘		11010100	324	212	D4
╒		11010101	325	213	D5
╓		11010110	326	214	D6
╫		11010111	327	215	D7
╪		11011000	330	216	D8
┘		11011001	331	217	D9
┌		11011010	332	218	DA
▉	sp. în video invers	11011011	333	219	DB
▄	sp. jos în video inv.	11011100	334	220	DC
▌	sp. stg. în video inv.	11011101	335	221	DD
▐	sp. dr. în video invers	11011110	336	222	DE
▀	sp. sus în video invers	11011111	337	223	DF
α		11100000	340	224	E0
β		11100001	341	225	E1
Γ		11100010	342	226	E2
p		11100011	343	227	E3
Σ		11100100	344	228	E4
σ		11100101	345	229	E5
µ		11100110	346	230	E6
τ		11100111	347	231	E7
Φ		11101000	350	232	E8
Θ		11101001	351	233	E9
Ω		11101010	352	234	EA
δ		11101011	353	235	EB
∞		11101100	354	236	EC
φ		11101101	355	237	ED
ε		11101110	356	238	EE
»	intersecţie	11101111	357	239	EF
≡		11110000	360	240	F0
±		11110001	361	241	F1
≥		11110010	362	242	F2
≤		11110011	363	243	F3

⌠		11110100	364	244	F4
⌡		11110101	365	245	F5
∏		11110110	366	246	F6
≈		11110111	367	247	F7
°	grad	11111000	370	248	F8
•	cerculeţ	11111001	371	249	F9
·	bulină mică	11111010	372	250	FA
√	rădăcină patrată	11111011	373	251	FB
ⁿ	la puterea n	11111100	374	252	FC
²	la puterea 2	11111101	375	253	FD
■	pătrat	11111110	376	254	FE
□	spaţiu fantomă	11111111	377	255	FF

Acest tabel, conţinând caracterele **ASCII** din setul extins, cu reprezentările lor în binar, octal, zecimal şi hexacimal se poate obţine (cu excepţia caracterelor cu denumirile BEL, BS, HT, LF, CR şi SUB) utilizând următorul program :

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>

int main()
{ int i; char str[10]; clrscr();
  for(i=0; i<256; i++)
     { itoa(i, str, 2);
       switch(i)
        {case  0: printf("\n\t  NUL %8s  %3o  %3d  %2X\n",
                    str, i, i, i); break;
         case  7: printf("\n\t  BEL %8s  %3o  %3d  %2X\n",
                    str, i, i, i); break;
         case  8: printf("\n\t  BS  %8s  %3o  %3d  %2X\n",
                    str, i, i, i); break;
         case  9: printf("\n\t  HT  %8s  %3o  %3d  %2X\n",
                    str, i, i, i); break;
         case 10: printf("\n\t  LF  %8s  %3o  %3d  %2X\n",
                    str, i, i, i); break;
         case 13: printf("\n\t  CR  %8s  %3o  %3d  %2X\n",
                    str, i, i, i); break;
         case 26: printf("\n\t  SUB %8s  %3o  %3d  %2X\n",
                    str, i, i, i); break;
         default: printf("\n\t  %c   %8s  %3o  %3d  %2X\n",
                     i, str, i, i, i);
        }
       if(i%11==0) getch();
     }
  getch(); return 0;
}
```

În locul caracterelor care nu se pot afişa se tipăreşte denumirea acestora.

Pentru a putea afişa **toate** caracterele **ASCII** (inclusiv BEL, BS, HT, LF, CR şi SUB) se poate folosi programul următor, la care funcţia s-a scris în ASM.

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>

//  Functia  display_char(..)  este scrisa in ASM.
//  Tipareste un caracter ASCII si muta pozitia
//  cursorului cu o unitate la dreapta.

void display_char(unsigned char i)
{asm{
     mov al,i   // codul caracterului
     mov cx,1   // numarul de caractere de scris (unul dupa
                // altul)
     mov bh,0   // fundal (0 pentru negru);pentru pagina video
               // curenta
     mov bl,10  // culoare caracter (7=gri deschis)
     mov ah,9   // Functia 9 a intreruperii 10h (servicii
                // video)
     int 0x10   // scrierea unui caracter pe ecran
     mov ah,3   // Functia 3, citeste pozitia cursorului
     int 0x10   // acum in DH este linia, in DL este coloana
     inc dl     // coloana urmatoare
     mov ah,2   // Functia 2, muta cursorul la noua pozitie
     int 0x10
    }
}

int main(void)
{ char str[10]; clrscr();
  printf("\n\t Char  Binary   Octal   Dec.  Hexa.\n");
  for(unsigned char c=0; c<128; c++)
     { itoa(c, str, 2); printf("\n\t ");
       display_char(c);
       printf("   %8s    %3o    %3d   %3X", str, c, c, c);
       if(c%20==0) getch();
     }
  getch(); return 0;
}
```

Anexa nr. 2

CODURILE ASCII EXTINSE ale TASTELOR pentru ANSI.SYS

F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11
0;59	0;60	0;61	0;62	0;63	0;64	0;65	0;66	0;67	0;68	0;133

F12	Insert	Delete	Home	End	Page Up	Page Down	↑	←	↓	→
0;134	0;82	0;83	0;71	0;79	0;73	0;81	0;72	0;75	0;80	0;77

Cu ajutorul programului următor se pot determina aceste coduri ASCII ce corespund tastelor calculatorului.

```
#include <conio.h>
#include <stdio.h>

int main()
{ int n;
  for(;;)
   {  clrscr(); printf("\n\t Press a  key !\n\n");
      printf("\n %d", getch()); printf(" %d", getch());
      printf("\n\n Press  0  <Enter> if you want to stop.\n"
             "\n Press 1  <Enter> to continue.  ");
      scanf("%d", &n);
      if(n==0) break;
   }
  return 0;
}
```

De asemenea trebuie specificat că există coduri care corespund la apăsarea simultană a două taste, una dintre acestea fiind

Alt sau Ctrl sau Shift

iar cealaltă poate fi o literă, cifră, alte caractere ASCII sau o tastă funcţională.

Exemple :

Alt şi F3	Ctrl şi Home	Shift şi F2
0 ; 106	0 ; 119	0 ; 85

În [Muel97] pg. 1049-1050 se poate găsi tabelul complet, conţinând toate aceste coduri .

Anexa nr. 3

PREFIXELE SISTEMULUI METRIC

Prefix	Simbol S I	Factor de multiplicare
peta	**P**	1 000 000 000 000 000 = 10^{15}
tera	**T**	1 000 000 000 000 = 10^{12}
giga	**G**	1 000 000 000 = 10^{9}
mega	**M**	1 000 000 = 10^{6}
kilo	**k**	1 000 = 10^{3}
hecto	**h**	100 = 10^{2}
deca	**da**	10 = 10^{1}
deci	**d**	0.1 = 10^{-1}
centi	**c**	0.01 = 10^{-2}
mili	**m**	0.001 = 10^{-3}
micro	**µ**	0.000 001 = 10^{-6}
nano	**n**	0.000 000 001 = 10^{-9}
pico	**p**	0.000 000 000 001 = 10^{-12}
femto	**f**	0.000 000 000 000 001 = 10^{-15}

Anexa nr. 4

PUTERILE lui 2

n	2^n	n	2^n
0	1	17	131072
1	2	18	262144
2	4	19	524288
3	8	20	1048576
4	16	21	2097152
5	32	22	4194304
6	64	23	8388608
7	**128**	24	16777216
8	**256**	25	33554432
9	512	26	67108864
10	1024	27	134217728
11	2048	28	268435456
12	4096	29	536870912
13	8192	30	1073741824
14	16384	**31**	**2147483648**
15	**32768**	**32**	**4294967296**
16	**65536**	..	....

XVIII

Bibliografie

A

[Абра88] **Абрамов, С. А. и др.,** Задачи по программированию, Москва, Наука, 1988, 224 стр.

[Abra70] **Abramowitz, M., Stegun, Irene A.**, Handbook of Mathematical Functions with Formulas, Graphs and Mathematical Tables, New York, Dover Publishers, 1970, 1046 pp.

[Adam98] **Adamson, T., Antonakos, J. L., Mansfield, K. C.**, Structured C for Engineering and Technology, Prentice Hall, 1998, 861 pg.

[Amme86a] **Ammeraal, L.,** C for Programmers, New York, John Wiley, 1986, 118 pg.

[Amme86b] **Ammeraal, L.,** Programming Principles in Computer Graphics, Chichester, John Wiley, 1986, 168 pg.

[Amme87] **Ammeraal, L.**, Computer Graphics for the IBM PC, New York, John Wiley, 1987, 150 pg.

[Anta01] **Antal, T. A.**, Limbajul C ANSI, Cluj-Napoca, Risoprint, 2001, 253 pg.

[Anto01] **Antonakos, J. L., Mansfield, K. C.**, Structured C for Engineering and Technology, Prentice Hall, 2001, 885 pp.

[Argh98] **Arghir, Mariana**, Limbajul C standard, Bucureşti, Ed. did. şi ped., 1998, 224 pg.

[Atan95] **Atanasiu, A.,** Concursuri de informatică. Probleme propuse, Bucureşti, Ed. Petrion, 1995, 192 pg.

[Atki90] **Atkinson, L., Atkinson, M**., Using C, QUE Corporation, 1990, 945 pg.

B

[Bark96] **Barkakati, N**., Borland C++ 4.0 , Bucureşti, Ed. Teora, 1996, 896 pg.

[Bark91] **Barclay, K.**, ANSI C, Problem Solving and Programming, Prentice Hall, 1991, 536 pg.

[Boia92] **Boian, F. M., Vancea, A., Iurian, S**., Limbajul C. Culegere de probleme, Cluj-Napoca, Atel. de mult. al Univ. "Babeş-Bolyai", 1992

[BORL88] **BORLAND** International, Turbo C. User's Guide. Version 2.0, 1988, 412 pp., Borland International, Scott Valley, CA.

[Brow99] **Brown, Brian,** C Programming, 1999, http://gd.tuwien.ac.at /languages/c/programming-bbrown/

[Bucu73] **Bucur, C. M.,** Metode numerice, Timişoara, Facla, 1973, 238 pg.

C

[Căpr91] **Căprariu, V**., Ghid de utilizare Turbo C2.0, Cluj-Napoca, Microinformatica, 1991, 252 pg.

[Carn69] **Carnahan, B., Luther, H. A., Wilkes, J. O.,** Applied Numerical Methods, New York, John Wiley, 1969, 604 pg.

[Chu_68] **Chu, Y.,** Bazele proiectării calculatoarelor numerice, Bucureşti, Ed. tehnică, 1968, 658 pg.

[Cioc79] **Ciocârdia, C., Ungureanu, I.,** Bazele cercetării experimentale în tehnologia construcţiilor de maşini, Bucureşti, Ed. didactică şi pedagogică, 1979, 243 pg.

[Cris92] **Cristea, V. ş. a.,** Limbajul C standard, Bucureşti, Teora, 1992, 117 pg.

[Cull97] **Cullens, C. ş. a.,** Utilizare Visual C++4, Bucureşti, Teora, 1997, 712 pg.

D

[Davi95] **Davies, P.**, The Indispensable Guide to C with Engineering Applications, Addison Wesley, 1995, 792 pp.

[Deit01] **Deitel, H. M., Deitel, P. J.**, C How to Program, Prentice Hall, 2001, 1253 pp.

[DeMa73] **Demidovich, B. P., Maron, I. A.**, Computational Mathematics, Moscow, Mir Publishers, 1973, 691 pg.

[Dorb73] **Dorband, W. ş. a.,** Praxis der FORTRAN Programmierung, Berlin, VEB Verlag Technik, 1973

E

[Enge96] **Engeln-Müllges, G., Uhlig, F.,** Numerical Algorithms with C, New York, Springer, 1996, 596 pp.

[Ette00] **Etter, Delores M., Ingber, Jeanine A.**, Engineering Problem Solving with C, Prentice Hall, 2000, 600 pp.

[Etti94] **Ettinger, J.,** Programming in C++, Mac Millan Press, 1994

F

[Falk51] **Falk, S.,** Ein übersichtliches Schema für die Matrizenmultiplication, ZAMM, 1951, vol. 31, pg. 152-153.

[Fiht64] **Fihtenholţ, G. M.,** Curs de calcul diferenţial şi integral, vol. I, 552 pg., vol. II, 738 pg., Bucureşti, Ed. tehnică, 1964

[Fiht65] **Fihtenholţ, G. M.,** Curs de calcul diferenţial şi integral, vol. III, Bucureşti, Ed. tehnică, 1965, 576 pg.

[Flow95] **Flowers, B. H.,** An Introduction to Numerical Methods in C++, Clarendon Press, 1995, 491 pg.

[Fole96] **Foley, J. D. a. o.**, Computer Graphics. Principles and Practice in C, Addison Wesley, 1996, 1200 pp.

G

[Geha85] **Gehani, N.**, C : An Advanced Introduction, Computer Science Press, 1985 (l. rusă, Moscova, Radio i sviazi, 1988, 271 pg.)

[Geor75] **Georgescu, H., Bâscă, V.**, Programe în limbajul FORTRAN, Bucureşti, Ed. Albatros, 1975, 330 pg.

[Ghir76] **Ghircoiaşiu, N.,** Curs de matematici speciale, vol. I, Cluj-Napoca, Lito Inst. Politehnic, 1976, 303 pg.

[Giur98] **Giurgiu, M.,** Programarea calculatoarelor, Cluj-Napoca, Risoprint, 1998, 251 pg.

[Glae94] **Glaeser, G.**, Fast Algorithms for 3D Graphics, New York, Springer, 1994, 306 pg.

H

[Hamm62] **Hamming, R. W.,** Numerical Methods for Scientists and Engineers, New York, McGraw Hill, 1962

[Hanl99] **Hanly, J. R.,** Problem Solving and Program Design in C, Addison Wesley, 1999, 276 pg.

[Henr62] **Henrici, P.,** Discrete Variable Methods for Ordinary Differential Equations, New York, John Wiley, 1962, 407 pg.

[Hoga84] **Hogan, T.**, The C Programmer's Handbook, Bowie, Prentice Hall, 1984, 264 pg.

[Holm95] **Holmes, Steve**, C Programming, University of Strathclyde, Glasgow, 1995, http://www.strath.ac.uk//CC/Courses/NewCcourse/ccourse.html

[Hyma96] **Hyman, M., Arnson, B.,** Visual C++ 2 pentru toţi, Bucureşti, Ed. Teora, 1996, 454 pg.

I

[Igna01] **Ignat, I.**, Programarea calculatoarelor. Îndrumător de lucrări, Cluj-Napoca, U.T. Pres, 2001, 115 pg.

[Iorg96] **Iorga, V. ş. a.,** Programare numerică, Bucureşti, Teora, 1996, 256 pg.
[Iorg97] **Iorga, V., Kalisz, Eugenia, Ţăpuş, C.,** Concursuri de programare. Probleme şi soluţii, Bucureşti, Ed. Teora, 1997
[ITCI88] **ITCI** Cluj-Napoca, Limbajul C. Programare, Cluj-Napoca, 1988, 114 pg.

J

[Jaes91] **Jaeschke, R.,** The Dictionary of Standard C, London, McGraw Hill, 1991, 164 pp.
[Jams00] **Jamsa, K., Klander, L.**, Totul despre C şi C++. Manualul fundamental de programare în C şi C++, Bucureşti, Teora, 2000, 1328 pg.
[Jank77] **Janke, E., Emde, F., Lösch, F**., Tafeln höherer Functionen, Stuttgart, B. G. Teubner Verlagsgesellschaft, 1960 (l. rusă, Moskva, Nauka, 1977, 342 pg.)
[Jams90] **Jamsa, K.,** Graphics Programming with Microsoft C and Microsoft Quick C, London, Microsoft Press, 1990, 600 pg.
[Jens98] **Jensen, Ted**, A Tutorial on Pointers and Arrays in C, version 1.1, July 1998, http://pw2.netcom.com/~tjensen/ptr/pointers.htm
[Jewe91] **Jewell, T. K.,** Computer Applications for Engineers, New York, John Wiley, 1991, 778 pg.
[John96] **Johnsonbaugh, R., Kalin, M.**, Applications Programming in ANSI C, Prentice Hall, 1996, 875 pp.

K

[Kacs94] **Kacso, A., Kacso, Daniela**, Turbo C. Tehnici de programare, Cluj-Napoca, Ed. Microinformatica, 1994
[Кара88] **Каранчук, В. П. и др**., Основы применения ЭВМ, Москва, Радио и связь, 1988, 287 стр
[Kása77] **Kása, Z., Blaga, P., Ţâmbulea, L.,** Culegere de algoritmi, Cluj-Napoca, Lito Univ. "Babeş-Bolyai", 1977, 130 pg.
[Кась86] **Касьянов, В. Н., Сабельфельд, В. К.,** Сборник заданий по практикуму на ЭВМ, Москва, Наука, 1986, 272 стр
[Kell98] **Kelley, A., Pohl, I.**, A Book on C. Programming in C, Addison Wesley, 1998, 752 pp.
[Kern78] **Kernighan, Brian W., Ritchie, Dennis M.,** ***The C Programming Language***, Englewood Cliffs, Prentice Hall, **1978** (ed. II – 1989, 274 pg.)
[Kopc75] **Kopchenova, N. V., Maron, I. A.,** Computational Mathematics. Worked Examples and Problems with Elements of Theory, Moscow, Mir Publishers, 1975, 395 pg.
[Knut74] **Knuth, D. E.,** Tratat de programarea calculatoarelor. Algoritmi fundamentali, Bucureşti, Ed. Tehnică, 1974, 676 pg.
[Knut76] **Knuth, D. E.,** Tratat de programarea calculatoarelor. Sortare şi căutare, Bucureşti, Ed. Tehnică, 1976, 736 pg.
[Knut83] **Knuth, D. E.,** Tratat de programarea calculatoarelor. Algoritmi seminumerici, Bucureşti, Ed. Tehnică, 1983, 722 pg..
[Koll98] **Koller, E., Roşculet, Monica**, Visual C++. Programarea aplicaţiilor Windows cu MFC, Bucureşti, Ed. Teora, 1998, 456 pg.

L

[Livo80] **Livovschi, L.,** Scheme logice, Bucureşti, Ed. Tehnică, 1980, 277 pg .
[Lupe98] **Lupea, I., Lupea, Mihaela**, Limbajul C. Teorie şi aplicaţii, Cluj-Napoca, Ed. Casa Cărţii de Ştiinţă, 1998, 376 pg.

M

[Marc95] **Marcu, V., Naghiu, D.**, C++. Teorie. Aplicaţii, Cluj-Napoca, Transilvania Press, 1995, 231 pg.
[Mars99] **Marshall, A. Dave**, Programming in C, UNIX System Calls and Subroutines using C, Univ. of Cardiff, 1999, http://www.cs.cf.ac.uk/Dave/C/CE.html
[McGr98] **McGregor, J., McGregor, R., Watt, A.**, Simple C, Addison Wesley, 1998, 240 pp.

[Meye78] **Meyer, B., Baudoin, C**., Méthodes de Programmation, Paris, Eyrolles, 1978.

[Mitr94] **Mitrana, V.,** Provocarea algoritmilor. Probleme pentru concursurile de informatică, Bucureşti, Ed. Agni, 1994

[Moca93] **Mocanu, M**. ş. a., 333 probleme de programare, Bucureşti, Teora, 1993.

[Mold78] **Moldovan, G**., Scheme logice şi programe FORTRAN. Culegere de probleme, Bucureşti, Ed. didactică şi pedagogică, 1978, 188 pg.

[Mold95] **Moldovan, G., Cioban, V., Lupea, Mihaela**, Probleme pentru programare în limbajul C, Cluj-Napoca, Universitatea "Babeş-Bolyai", 1995, 151 pg.

[Muel97] **Mueller, S.,** PC. Depanare şi modernizare, Bucureşti, Teora, 1997, 1295 pg.

N

[Naka93] **Nakamura, S**., Applied Numerical Methods in C, Prentice Hall, 1993, 604 pg.

[Negr_1] **Negrescu, L**., Limbajele C şi C++ pentru începători. Volumul I. Limbajul C, Cluj-Napoca, Ed. Microinformatica, 1994, 784 pg.

[Negr_2] **Negrescu, L**., Limbajele C şi C++ pentru începători. Volumul II. Limbajul C++, Cluj-Napoca, Microinformatica, 1994, 671 pg.

[Negr_3] **Negrescu, L.,** Limbajele C şi C++ pentru începători, vol. III. Limbajele C şi C++ în aplicaţii, Cluj-Napoca, Ed. Albastră, 1996, 1029 pg.

[Nico95] **Nicolae, M., Niculescu, S**., Programare în mod text. Aplicaţii Turbo C 2.0, Bucureşti, Ed. Tehnică, 1995, 228 pg.

[Nort98] **Norton, P**., Secrete PC, Bucureşti, Ed. Teora, 1998, 640 pg.

O

[Orte99] **Ortega, J. M, Grimshaw, A. S**., An Introduction to C++ and Numerical Methods, Oxford, Univ. Press, 1999, 288 pg.

P

[Pătr98] **Pătrut, B.,** Aplicaţii în C şi C++, Bucureşti, Ed. Teora, 1998, 167 pg.

[Petro93] **Petrovici, V., Goicea, F**., Programarea în limbajul C, Bucureşti, Ed. Tehnică, 1993, 327 pg.

[Пиль89] **Пильщиков, В. Н**., Сборник упражнений по языку Паскаль, Москва, Наука, 1989, 156 стр.

[Pîsl01] **Pîslă, Doina Liana**, Programarea calculatoarelor. Limbajul C, Cluj-Napoca, Ed. Todesco, 2001, 293 pg.

[Plau96] **Plauger, P. J., Brodie, J.,** Dinkum C Library Reference, Dinkumware Genuine Software, 1996, http://www.dinkumware.com/htm_cl/index.html

[Pope99] **Popescu, Diana Ioana**, Programare în limbajul C, Dej, DSG Press, 1999, 288 pg.

[Pope78] **Popescu, P. ş. a.**, Culegere de probleme de mecanică tehnică. Statica, Cluj-Napoca, Institutul Politehnic, 1978, 263 pg

[Post94] **Postolache, M.,** Metode numerice, Bucureşti, Ed. Sirius, 1994, 251 pg.

[Pres94] **Press, W. H. a. o.,** Numerical Recipes in C. The Art of Scientific Computing, Cambridge, University Press, 1994, 735 pg.

R

[Ramt98] **Ramteke, T.**, Introduction to C and C++ for Technical Students, Prentice Hall, 1998, 488 pp.

[Rine97] **Rinehard, M. L.,** Să învăţăm C++ pas cu pas, Bucureşti, Ed. Tehnică, 1997, 478 pg.

[Ritc93] **Ritchie, Dennis M**., The Development of the C Language, http://cm.bell-labs.com/cm/cs/who/dmr/chist.html

[Robe95] **Roberts, E. S.**, The Art and Science of C. A Library Based Introduction to Computer Science, Addison Wesley, 1995, 704 pp.

S

[Schi97] **Schildt, H.,** C++. Manual complet, Bucureşti, Teora, 1997, 644 pg.

[Schi98] **Schildt, H**., C. Manual complet, Bucureşti, Teora, 1998, 835 pg.

[Simi95] **Simionescu, I., Dranga, M., Moise, V**., Metode numerice în tehnică. Aplicaţii în FORTRAN, Bucureşti, Ed. tehnică, 1995, 192 pg.

[Smeu95] **Smeureanu, I., Ivan, I., Dârdală, M.,** Limbajul C/C++ prin exemple, Bucureşti, Ed. Cison, 1995, 256 pg.

[Stoi98] **Stoilescu, D.,** Aplicaţii C/C++, vol. I, Galaţi, Ed. Radial, 1998, 288 pg.

[Stro86] **Stroustrup, Bjarne,** ***The C++ Programming Language***, Addison Wesley, **1986**, (ediţia a II-a – 1991, ediţia a III–a – 1997 – 928 pg.).

[Шура78] **Шураков, В. В., Морозов, В. П.,** Задачник по основам алгоритмизации, алгязыкам и машинной обработке, Москва, Статистика, 1978, 198 стр.

[Свет80] **Светозарова, Г. И. и др.,** Практикум по программированию на алгоритмических языках, Москва, Наука, 1980, 320 стр .

[Swan96] **Swan, T.,** Învăţăm C … pas cu pas, Bucureşti, Ed. Tehnică, 1996, 416 pg.

T

[Thom97] **Thomson, W. J.,** Atlas for Computing Mathematical Functions. An Illustrated Guide for Practitioners. With Programs in C and Mathematica, New York, John Wiley, 1997, 903 pg.

[Tizz86] **Tizzard, K.,** C for Professional Programmers, Chichester, Ellis Horwood, 1986, 211 pp.

U

[Ursu76] **Ursu, N.**, Contribuţii la studiul vibraţiilor de încovoiere şi torsiune ale sistemelor cu mai multe grade de libertate, cu aplicaţii în cazul arborilor, Teză de doctorat, Inst. Politehnic Bucureşti, 1976

[Ursu79] **Ursu, N.**, Programare, Cluj-Napoca, Institutul Politehnic, 1979, 284 pg.

[Ursu84] **Ursu, N.,** Vibraţii mecanice, Cluj-Napoca, Institutul Politehnic, 1984, 252 pg.

[Ursu98a] **Ursu-Fischer, N**., Vibraţiile sistemelor mecanice. Teorie şi aplicaţii, Cluj-Napoca, Ed. Casa Cărţii de Ştiinţă, 1998, 452 pg.

[Ursu98b] **Ursu-Fischer, N. Popescu, Diana, Ursu, M.,** Programarea şi utilizarea calculatoarelor, Cluj-Napoca, Ed. Napoca Star, 1998, 162 pg.

[Ursu00] **Ursu-Fischer, N., Ursu, M.,** Metode numerice în tehnică şi programe în C/C++, vol. I, Cluj-Napoca, Ed. Casa Cărţii de Ştiinţă, 2000, 282 pg.

[Ursu01] **Ursu, M.**, Modeling of Quantum Cascade Lasers, Aachen, Shaker Verlag, 2001, 105 pp.

V

[Vasi95] **Vasiu, L., Grama, R., Aldica, Alexandra**, Turbo Pascal 7.0. Programe, ed. III, Cluj-Napoca, Editura Albastră, 1995, 423 pg .

[Vett90] **Vetterling, W. T. a. o.,** Numerical Recipes. Example Book (C), Cambridge, University Press, 1990, 239 pg.

[Vlad92] **Vlada, M. ş. a.,** Grafică pe calculator în limbajele Pascal şi C. Implementare şi aplicaţii, vol. I - 209 pg., vol. II - 249 pg., Bucureşti, Ed. Tehnică, 1992.

[Vrac82] **Vraciu, G., Popa, A.,** Metode numerice cu aplicaţii în tehnica de calcul, vol. I, Craiova, Scrisul Românesc, 1982, 243 pg.

W

[Wats93] **Watson, A., Feather, C., Ritchie, D. M.**, BCPL to B to C, 1993, http://www.lysator.liu.se/c/msb-on-b.html

[Weav88] **Weaver, Carol**, The Complete C Video Course, Zortech Limited, London, 1988, 356 pp., 1 floppy 5.25", 10 video tapes

[Will97] **Williams, M.,** Bazele Visual C++ 4.0, Bucureşti, Teora, 1997, 528 pg.